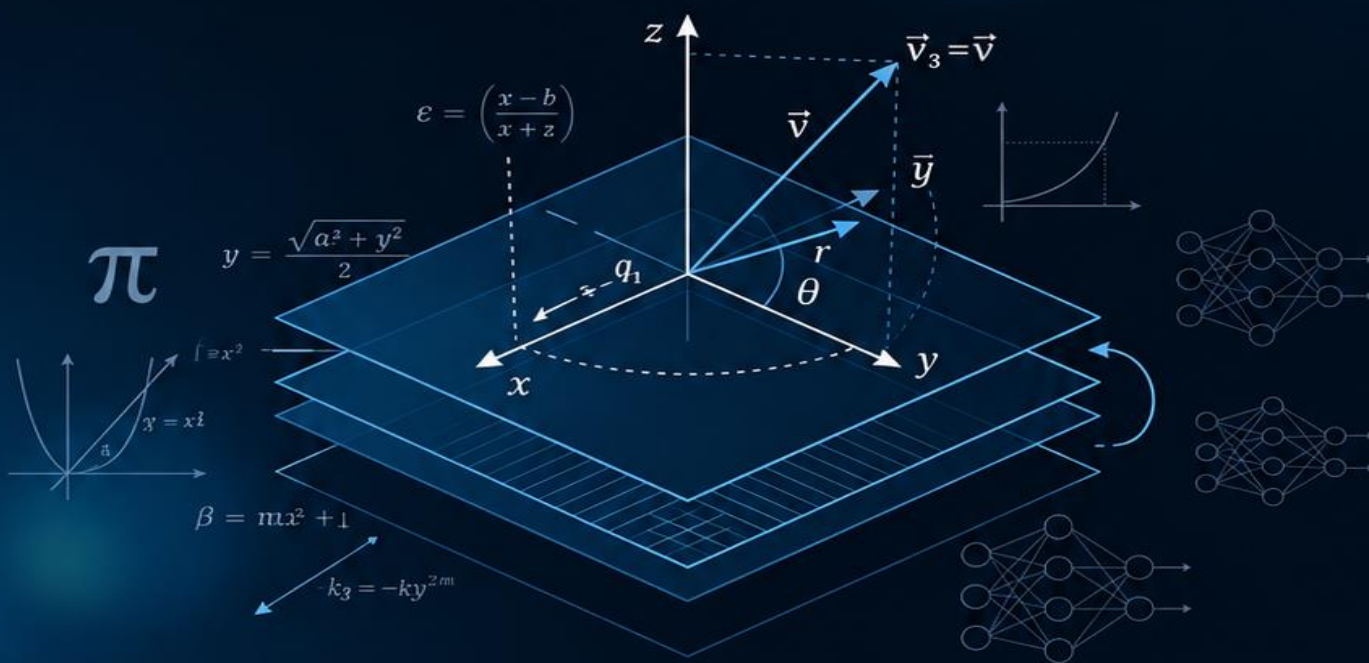


A PREMIUM MINIMALIST TECHNICAL BOOK FOR MODERN AI SERIES

AI ENGINEERING FROM SCRATCH

VOLUME 01

FOUNDATIONS



PHASES 1-3

1 MATH FOUNDATIONS — 22 Lessons

The intuition behind every AI algorithm, through code.

2 ML FUNDAMENTALS — 18 Lessons

Classical ML — still the backbone of most production AI.

3 DEEP LEARNING CORE — 13 Lessons

Neural networks from first principles.

AI Engineering From Scratch

Volume I — Foundations

Phases 1, 2, 3: Math Foundations | Machine Learning Fundamentals | Deep Learning Core

Book Edition by [Kourosh Rahimi](#)

Series Overview

AI Engineering From Scratch is a complete multi-volume book series designed to guide readers from mathematical foundations to advanced artificial intelligence systems.

Each phase is designed as an independent book. New phases may be added in the future as the field of AI continues to evolve.

Open Source Foundation

This book incorporates material from the open-source project AI Engineering From Scratch.

Original work: Copyright (c) 2026 Rohit Ghumare

Licensed under the MIT License.

Book formatting, organization, diagrams, flowcharts, and additional editorial material by Kourosh Rahimi.

The original copyright notice and the full text of the MIT License are included in this book.

Overall Summary

This is a comprehensive and highly practical volume (Volume I) of a series called "**AI Engineering From Scratch**." It covers the foundational phases (1, 2, and 3) of AI education, totaling 53 lessons. The book is structured with a clear philosophy: build everything from scratch before using high-level libraries.

1. **Phase 1: Math Foundations:** This section establishes the mathematical bedrock. It covers linear algebra (vectors, matrices, transformations, SVD), calculus (gradients, chain rule, optimization), probability (distributions, Bayes' theorem), and other essential topics like numerical stability, information theory, and graph theory. The emphasis is on geometric intuition and implementing core operations from scratch.
2. **Phase 2: Machine Learning Fundamentals:** This part moves from math to the core algorithms of classical machine learning. It covers everything from the basic definition of ML to linear and logistic regression, decision trees/random forests, SVMs, KNN, unsupervised learning, and critical ML pipelines like feature engineering, model evaluation, and hyperparameter tuning. Each algorithm is implemented from scratch.
3. **Phase 3: Deep Learning Core:** This final section of Volume I builds a deep learning framework from the ground up. Starting with the perceptron, it progresses to multi-layer networks, backpropagation, activation functions, loss functions, and optimizers. It then covers regularization, initialization, and learning rate schedules. The phase culminates in building a mini deep learning framework, then introduces how to use production-grade tools like PyTorch and JAX, and ends with crucial debugging strategies.

Key Takeaways from the Document

The "From Scratch" Philosophy: The book consistently emphasizes building implementations without external libraries to understand the fundamental concepts. This is a powerful and effective way to learn.

Practical, Not Just Theoretical: Every lesson includes "Build It" and "Use It" sections. The "Build It" sections are where the from-scratch code goes, and the "Use It" sections show how to do the same thing with libraries like NumPy, PyTorch, or scikit-learn.

Problem-Driven Learning: Each lesson starts with a "Problem" section that explains the why behind the concept, making it immediately relevant.

Clear Explanations: The writing is direct and avoids unnecessary jargon, making complex topics accessible.

Broad and Deep: The scope is enormous. It covers the mathematical, classical, and deep learning foundation required to move into advanced fields like NLP, computer vision, and generative AI.

Open Source Foundation: The book acknowledges its roots in an open-source project, making it a community resource.

Table of Contents

Series Overview.....	2
Open Source Foundation	2
Overall Summary.....	3
Key Takeaways from the Document	4
Phase 1: Math Foundations	9
Linear Algebra Intuition.....	9
Vectors, Matrices & Operations	22
Matrix Transformations	33
Calculus for Machine Learning.....	46
Chain Rule & Automatic Differentiation	65
Probability and Distributions	80
Bayes' Theorem.....	94
Optimization.....	109
Information Theory	122
Dimensionality Reduction	137
Singular Value Decomposition	150
Tensor Operations.....	169
Numerical Stability	198
Norms and Distances	217
Statistics for Machine Learning.....	247
Sampling Methods	276
Linear Systems	297
Convex Optimization.....	315
Complex Numbers for AI	346
The Fourier Transform	370
Graph Theory for Machine Learning	397
Stochastic Processes	412
Phase 2: Machine Learning Fundamentals	430

What Is Machine Learning	430
Linear Regression	444
Logistic Regression	460
Decision Trees and Random Forests	476
Support Vector Machines.....	489
K-Nearest Neighbors and Distances.....	514
Unsupervised Learning.....	527
Feature Engineering & Selection.....	543
Model Evaluation	559
Bias-Variance Tradeoff	578
Ensemble Methods	593
Hyperparameter Tuning.....	617
ML Pipelines	647
Naive Bayes	671
Time Series Fundamentals	693
Anomaly Detection.....	716
Handling Imbalanced Data	732
Feature Selection	749
Phase 3: Deep Learning Core	768
The Perceptron.....	768
Multi-Layer Networks and Forward Pass	780
Backpropagation from Scratch.....	793
Activation Functions.....	807
Loss Functions	822
Optimizers.....	836
Regularization	850
Weight Initialization and Training Stability	866
Learning Rate Schedules and Warmup	878
Build Your Own Mini Framework.....	893
Introduction to PyTorch	912
Introduction to JAX.....	932

Debugging Neural Networks	951
End of Volume I	975

PHASE 01 MATH FOUNDATIONS

AI Engineering From Scratch

A Complete Reference for AI Engineers

22 Lessons

AI ENGINEERING FROM SCRATCH

Phase 1: Math Foundations

Linear Algebra Intuition

Every AI model is just matrix math wearing a fancy hat.

Type: Learn **Languages:** Python, Julia **Prerequisites:** Phase 0 **Time:** ~60 minutes

Learning Objectives

- Implement vector and matrix operations (addition, dot product, matrix multiply) from scratch in Python
- Explain geometrically what the dot product, projection, and Gram-Schmidt process do
- Determine linear independence, rank, and basis of a set of vectors using row reduction
- Connect linear algebra concepts to their AI applications: embeddings, attention scores, and LoRA

The Problem

Open any ML paper. Within the first page, you'll see vectors, matrices, dot products, and transformations. Without linear algebra intuition, these are just symbols. With it, you can see what a neural network is actually doing – moving points around in space.

You don't need to be a mathematician. You need to see what these operations mean geometrically, then code them yourself.

The Concept

Vectors Are Points (and Directions)

A vector is just a list of numbers. But those numbers mean something – they're coordinates in space.

2D vector [3, 2]:

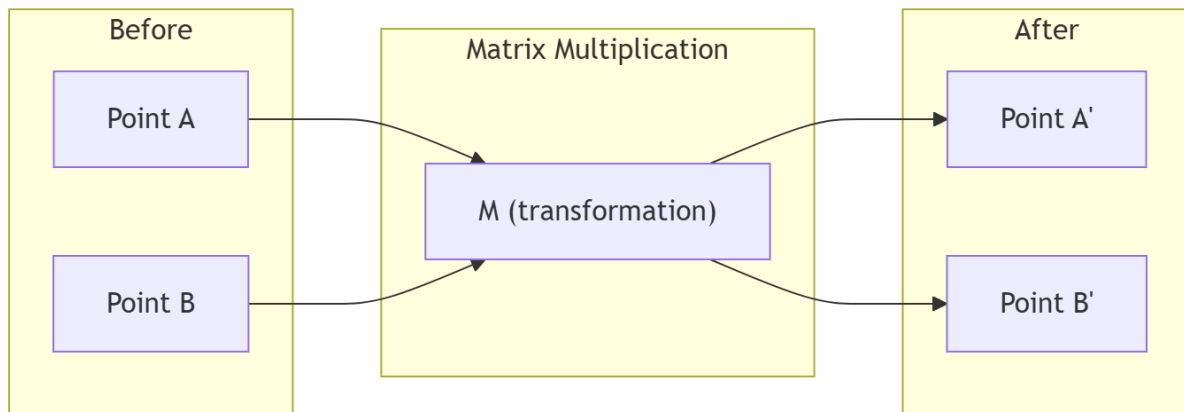
x	y	Point
3	2	The vector points from origin (0,0) to (3, 2) on the plane

The vector has magnitude $\sqrt{3^2 + 2^2} = \sqrt{13}$ and points up and to the right.

In AI, vectors represent everything: - A word → a vector of 768 numbers (its “meaning” in embedding space) - An image → a vector of millions of pixel values - A user → a vector of preferences

Matrices Are Transformations

A matrix transforms one vector into another. It can rotate, scale, stretch, or project.



In AI, matrices ARE the model: - Neural network weights → matrices that transform input into output - Attention scores → matrices that decide what to focus on - Embeddings → matrices that map words to vectors

The Dot Product Measures Similarity

The dot product of two vectors tells you how similar they are.

$$a \cdot b = a_1 \times b_1 + a_2 \times b_2 + \dots + a_n \times b_n$$

Same direction: $a \cdot b > 0$ (similar)

Perpendicular: $a \cdot b = 0$ (unrelated)

Opposite direction: $a \cdot b < 0$ (dissimilar)

This is literally how search engines, recommendation systems, and RAG work – find vectors with high dot products.

Linear Independence

Vectors are linearly independent if no vector in the set can be written as a combination of the others. If v_1, v_2, v_3 are independent, they span a 3D space. If one is a combination of the others, they only span a plane.

Why it matters for AI: your feature matrix should have linearly independent columns. If two features are perfectly correlated (linearly dependent), the model cannot distinguish their effects. This causes multicollinearity in regression – the weight matrix becomes unstable, and small input changes produce wild output swings.

Concrete example:


```

v1 = [1, 0, 0]
v2 = [0, 1, 0]
v3 = [2, 1, 0]    # v3 = 2*v1 + v2

```

v1 and v2 are independent – neither is a scalar multiple or combination of the other. But $v3 = 2*v1 + v2$, so $\{v1, v2, v3\}$ is a dependent set. These three vectors all lie in the xy-plane. No matter how you combine them, you cannot reach $[0, 0, 1]$. You have three vectors but only two dimensions of freedom.

In a dataset: if $feature_3 = 2*feature_1 + feature_2$, adding feature_3 gives the model zero new information. Worse, it makes the normal equations singular – there is no unique solution for the weights.

Basis and Rank

A basis is a minimal set of linearly independent vectors that span the entire space. The number of basis vectors is the dimension of the space.

The standard basis for 3D space is $\{[1,0,0], [0,1,0], [0,0,1]\}$. But any three independent vectors in 3D form a valid basis. The choice of basis is a choice of coordinate system.

Rank of a matrix = number of linearly independent columns = number of linearly independent rows. If $rank < \min(rows, cols)$, the matrix is rank-deficient. This means: - The system has infinitely many solutions (or none) - Information is lost in the transformation - The matrix cannot be inverted

Situation	Rank	What it means for ML
Full rank ($rank = \min(m, n)$)	Maximum possible	Unique least-squares solution exists. Model is well-conditioned.
Rank deficient ($rank < \min(m, n)$)	Below maximum	Features are redundant. Infinitely many weight solutions. Regularization needed.
Rank 1	1	Every column is a scaled copy of one vector. All data lies on a line.
Near rank-deficient (small singular values)	Numerically low	Matrix is ill-conditioned. Tiny input noise causes large output changes. Use SVD truncation or ridge regression.

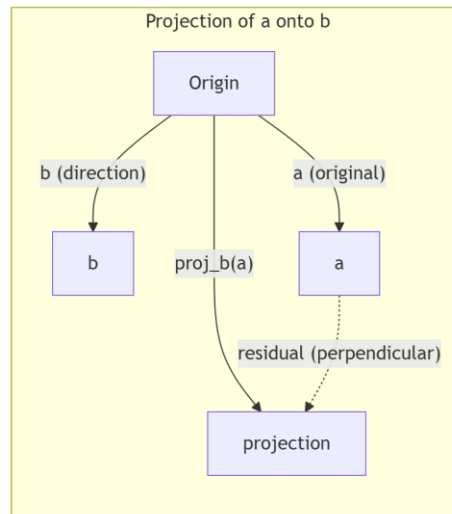
Projection

Projecting vector **a** onto vector **b** gives the component of **a** in the direction of **b**:

$$\text{proj}_b(a) = (a \cdot b / b \cdot b) * b$$

The residual $(a - \text{proj}_b(a))$ is perpendicular to b. This orthogonal decomposition is the foundation of least-squares fitting.

Projection is everywhere in ML: - Linear regression minimizes the distance from observations to the column space – the solution IS a projection - PCA projects data onto the directions of maximum variance - Attention in transformers computes projections of queries onto keys



Example: $a = [3, 4]$, $b = [1, 0]$

$$\text{proj}_b(a) = (3 \cdot 1 + 4 \cdot 0) / (1^2 + 0^2) \cdot [1, 0] = 3 \cdot [1, 0] = [3, 0]$$

The projection drops the y-component. This is dimensionality reduction in its simplest form – throw away the directions you don't care about.

Gram-Schmidt Process

Converting any set of independent vectors into an orthonormal basis. Orthonormal means every vector has length 1 and every pair is perpendicular.

The algorithm: 1. Take the first vector, normalize it 2. Take the second vector, subtract its projection onto the first, normalize 3. Take the third vector, subtract its projections onto all previous vectors, normalize 4. Repeat for remaining vectors

Input: $v_1, v_2, v_3, \dots$ (linearly independent)

$$u_1 = v_1 / |v_1|$$

$$w_2 = v_2 - (v_2 \text{ dot } u_1) * u_1$$

$$u_2 = w_2 / |w_2|$$

$$w_3 = v_3 - (v_3 \text{ dot } u_1) * u_1 - (v_3 \text{ dot } u_2) * u_2$$

$$u_3 = w_3 / |w_3|$$

Output: $u_1, u_2, u_3, \dots$ (orthonormal basis)

This is how QR decomposition works internally. Q is the orthonormal basis, R captures the projection coefficients. QR decomposition is used in: - Solving linear systems (more stable than Gaussian elimination) - Computing eigenvalues (QR algorithm) - Least-squares regression (the standard numerical method)

Build It

Step 1: Vectors from scratch (Python)

```
class Vector:
    def __init__(self, components):
        self.components = list(components)
        self.dim = len(self.components)

    def __add__(self, other):
        return Vector([a + b for a, b in zip(self.components, other.components)])

    def __sub__(self, other):
        return Vector([a - b for a, b in zip(self.components, other.components)])

    def dot(self, other):
        return sum(a * b for a, b in zip(self.components, other.components))

    def magnitude(self):
        return sum(x**2 for x in self.components) ** 0.5

    def normalize(self):
        mag = self.magnitude()
        return Vector([x / mag for x in self.components])

    def cosine_similarity(self, other):
        return self.dot(other) / (self.magnitude() * other.magnitude())

    def __repr__(self):
        return f"Vector({self.components})"

a = Vector([1, 2, 3])
b = Vector([4, 5, 6])

print(f"a + b = {a + b}")
print(f"a · b = {a.dot(b)}")
print(f"|a| = {a.magnitude():.4f}")
print(f"cosine similarity = {a.cosine_similarity(b):.4f}")
```

Step 2: Matrices from scratch (Python)

```
class Matrix:
    def __init__(self, rows):
        self.rows = [list(row) for row in rows]
        self.shape = (len(self.rows), len(self.rows[0]))
```

```

def __matmul__(self, other):
    if isinstance(other, Vector):
        return Vector([
            sum(self.rows[i][j] * other.components[j] for j in range(self.shape[1]))
        ])
    rows = []
    for i in range(self.shape[0]):
        row = []
        for j in range(other.shape[1]):
            row.append(sum(
                self.rows[i][k] * other.rows[k][j]
                for k in range(self.shape[1])
            ))
        rows.append(row)
    return Matrix(rows)

def transpose(self):
    return Matrix([
        [self.rows[j][i] for j in range(self.shape[0])]
        for i in range(self.shape[1])
    ])

def __repr__(self):
    return f"Matrix({self.rows})"

rotation_90 = Matrix([[0, -1], [1, 0]])
point = Vector([3, 1])

rotated = rotation_90 @ point
print(f"Original: {point}")
print(f"Rotated 90°: {rotated}")

```

Step 3: Why this matters for AI

```

import random

random.seed(42)
weights = Matrix([[random.gauss(0, 0.1) for _ in range(3)] for _ in range(2)])
input_vector = Vector([1.0, 0.5, -0.3])

output = weights @ input_vector
print(f"Input (3D): {input_vector}")
print(f"Output (2D): {output}")
print("This is what a neural network layer does -- matrix multiplication.")

```

Step 4: Julia version

```
a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]

println("a + b = ", a + b)
println("a · b = ", a · b)           # Julia supports unicode operators
println("|a| = ", √(a · a))
println("cosine = ", (a · b) / (√(a · a) * √(b · b)))

# Matrix-vector multiplication
W = [0.1 -0.2 0.3; 0.4 0.5 -0.1]
x = [1.0, 0.5, -0.3]
println("Wx = ", W * x)
println("This is a neural network layer.")
```

Step 5: Linear independence and projection from scratch (Python)

```
def is_linearly_independent(vectors):
    n = len(vectors)
    dim = len(vectors[0].components)
    mat = Matrix([v.components[:] for v in vectors])
    rows = [row[:] for row in mat.rows]
    rank = 0
    for col in range(dim):
        pivot = None
        for row in range(rank, len(rows)):
            if abs(rows[row][col]) > 1e-10:
                pivot = row
                break
        if pivot is None:
            continue
        rows[rank], rows[pivot] = rows[pivot], rows[rank]
        scale = rows[rank][col]
        rows[rank] = [x / scale for x in rows[rank]]
        for row in range(len(rows)):
            if row != rank and abs(rows[row][col]) > 1e-10:
                factor = rows[row][col]
                rows[row] = [rows[row][j] - factor * rows[rank][j] for j in range(dim)]
        rank += 1
    return rank == n

def project(a, b):
    scalar = a.dot(b) / b.dot(b)
    return Vector([scalar * x for x in b.components])

def gram_schmidt(vectors):
    orthonormal = []
    for v in vectors:
        w = v
```

```

    for u in orthonormal:
        proj = project(w, u)
        w = w - proj
    if w.magnitude() < 1e-10:
        continue
    orthonormal.append(w.normalize())
return orthonormal

v1 = Vector([1, 0, 0])
v2 = Vector([1, 1, 0])
v3 = Vector([1, 1, 1])
basis = gram_schmidt([v1, v2, v3])
for i, u in enumerate(basis):
    print(f"u{i+1} = {u}")
    print(f" |u{i+1}| = {u.magnitude():.6f}")

print(f"u1 · u2 = {basis[0].dot(basis[1]):.6f}")
print(f"u1 · u3 = {basis[0].dot(basis[2]):.6f}")
print(f"u2 · u3 = {basis[1].dot(basis[2]):.6f}")

```

Use It

Now the same thing with NumPy – what you’ll actually use in practice:

```

import numpy as np

a = np.array([1, 2, 3], dtype=float)
b = np.array([4, 5, 6], dtype=float)

print(f"a + b = {a + b}")
print(f"a · b = {np.dot(a, b)}")
print(f"|a| = {np.linalg.norm(a):.4f}")
print(f"cosine = {np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)):.4f}")

W = np.random.randn(2, 3) * 0.1
x = np.array([1.0, 0.5, -0.3])
print(f"Wx = {W @ x}")

```

Rank, Projection, and QR with NumPy

```

import numpy as np

A = np.array([[1, 2], [2, 4]])
print(f"Rank: {np.linalg.matrix_rank(A)}")

a = np.array([3, 4])
b = np.array([1, 0])
proj = (np.dot(a, b) / np.dot(b, b)) * b

```

```

print(f"Projection of {a} onto {b}: {proj}")

Q, R = np.linalg.qr(np.random.randn(3, 3))
print(f"Q is orthogonal: {np.allclose(Q @ Q.T, np.eye(3))}")
print(f"R is upper triangular: {np.allclose(R, np.triu(R))}")

```

PyTorch – Tensors Are Vectors with Autodiff

```

import torch

x = torch.randn(3, requires_grad=True)
y = torch.tensor([1.0, 0.0, 0.0])

similarity = torch.dot(x, y)
similarity.backward()

print(f"x = {x.data}")
print(f"y = {y.data}")
print(f"dot product = {similarity.item():.4f}")
print(f"d(dot)/dx = {x.grad}")

```

The gradient of the dot product with respect to x is just y . PyTorch computed this automatically. Every operation in a neural network is built from operations like this – matrix multiplies, dot products, projections – and autodiff tracks gradients through all of them.

You just built from scratch what NumPy does in one line. Now you know what’s happening under the hood.

Connections

Everything in this lesson connects to specific parts of modern AI:

Concept	Where it shows up
Dot product	Attention scores in transformers, cosine similarity in RAG
Matrix multiply	Every neural network layer, every linear transformation
Linear independence	Feature selection, avoiding multicollinearity
Rank	Determining if a system is solvable, LoRA (low-rank adaptation)
Projection	Linear regression (projecting onto column space), PCA
Gram-Schmidt / QR	Numerical solvers, eigenvalue computation

Concept	Where it shows up
Orthonormal basis	Stable numerical computation, whitening transforms

LoRA deserves special mention. It fine-tunes large language models by decomposing weight updates into low-rank matrices. Instead of updating a 4096x4096 weight matrix (16M parameters), LoRA updates two matrices of size 4096x16 and 16x4096 (131K parameters). The rank-16 constraint means LoRA assumes the weight update lives in a 16-dimensional subspace of the full 4096-dimensional space. That is linear algebra doing real work.

Exercises

1. Implement `Vector.angle_between(other)` that returns the angle in degrees between two vectors
2. Create a 2D scaling matrix that doubles the x-coordinate and triples the y-coordinate, then apply it to the vector `[1, 1]`
3. Given 5 random word-like vectors (dimension 50), find the two most similar using cosine similarity
4. Verify that the Gram-Schmidt output is truly orthonormal: check that every pair has dot product 0 and every vector has magnitude 1
5. Create a 3x3 matrix with rank 2. Verify using the `rank()` method. Then explain what geometric object the columns span.
6. Project the vector `[1, 2, 3]` onto `[1, 1, 1]`. What does the result represent geometrically?

Key Terms

Term	What people say	What it actually means
Vector	"An arrow"	A list of numbers representing a point or direction in n-dimensional space
Matrix	"A table of numbers"	A transformation that maps vectors from one space to another
Dot product	"Multiply and sum"	A measure of how aligned two vectors are – the core of similarity search
Embedding	"Some AI magic"	A vector that represents the meaning of something (word, image, user)
Linear independence	"They don't overlap"	No vector in the set can be written as a combination of the others

Term	What people say	What it actually means
Rank	"How many dimensions"	The number of linearly independent columns (or rows) in a matrix
Projection	"The shadow"	The component of one vector in the direction of another
Basis	"The coordinate axes"	A minimal set of independent vectors that span the space
Orthonormal	"Perpendicular unit vectors"	Vectors that are mutually perpendicular and each have length 1

You are a linear algebra tutor for AI engineers. Your approach:

1. Always explain concepts geometrically first — what does this operation DO in space?
2. Connect every concept to its AI application (embeddings, attention, transformers)
3. Show the math, but never without the intuition
4. Use ASCII diagrams to visualize transformations

When the student asks about a concept:

- Start with a one-sentence intuition
- Draw an ASCII diagram showing the geometric meaning
- Show the math notation
- Show a Python implementation from scratch (no NumPy)
- Show the NumPy equivalent
- Explain where this appears in real AI systems

Key connections to always make: - Dot product → similarity/attention scores - Matrix multiplication → neural network layers - Eigenvalues → PCA / dimensionality reduction - Transpose → attention (Q, K, V) - Normalization → unit vectors / cosine similarity

Quiz

Pre-lesson check

1. What does the dot product of two vectors measure?
 - a) The distance between the two vectors

- b) How similar or aligned the two vectors are **(correct)**
 - c) The angle between the vectors in degrees
 - d) The number of dimensions the vectors share
Explanation: The dot product measures alignment: positive means same direction (similar), zero means perpendicular (unrelated), negative means opposite direction (dissimilar). This is the basis of similarity search in AI.
2. In AI, what does 'embedding' refer to?
- a) Inserting one model inside another
 - b) A vector representation that captures the meaning of something (word, image, user) **(correct)**
 - c) A technique for compressing model weights
 - d) The process of converting code to machine instructions
Explanation: An embedding maps discrete objects (words, images, users) to continuous vectors in a high-dimensional space where similar items are close together. It is the bridge between real-world concepts and mathematical operations.

Post-lesson check

3. Three vectors $v_1=[1,0,0]$, $v_2=[0,1,0]$, $v_3=[2,1,0]$ are given. Are they linearly independent?
- a) Yes, because there are three vectors in 3D space
 - b) No, because $v_3 = 2*v_1 + v_2$, so v_3 is a linear combination of the others **(correct)**
 - c) Yes, because none of the vectors are identical
 - d) No, because they all have a zero component
*Explanation: $v_3 = 2v_1 + v_2$, making the set linearly dependent. All three vectors lie in the xy-plane and cannot reach $[0,0,1]$. Despite having 3 vectors in 3D, they only span a 2D subspace.**
4. What does the rank of a matrix tell you in the context of machine learning?

- a) How fast the matrix can be multiplied
 - b) The number of linearly independent columns, indicating how many dimensions of useful information it contains **(correct)**
 - c) The maximum value in the matrix
 - d) The number of non-zero entries in the matrix
Explanation: Rank equals the number of linearly independent columns. In ML, a rank-deficient feature matrix means redundant features, infinitely many weight solutions, and the need for regularization.
5. How does LoRA (Low-Rank Adaptation) use linear algebra to efficiently fine-tune large language models?
- a) It removes unused rows from weight matrices to shrink the model
 - b) It decomposes weight updates into two small low-rank matrices instead of updating the full weight matrix **(correct)**
 - c) It converts all weights from float32 to int8
 - d) It freezes the embedding layer and only trains the output head
Explanation: LoRA decomposes a 4096×4096 weight update into two matrices of size 4096×16 and 16×4096 (rank-16), reducing trainable parameters from 16M to 131K by assuming updates live in a low-dimensional subspace.
-

Vectors, Matrices & Operations

Every neural network is just matrix multiplication with extra steps.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lesson 01 (Linear Algebra Intuition) **Time:** ~60 minutes

Learning Objectives

- Build a Matrix class with element-wise operations, matrix multiplication, transpose, determinant, and inverse
- Distinguish element-wise multiplication from matrix multiplication and explain when each applies
- Implement a single dense neural network layer (`relu(W @ x + b)`) using only the from-scratch Matrix class
- Explain broadcasting rules and how bias addition works in neural network frameworks

The Problem

You want to build a neural network. You read the code and see this:

```
output = activation(weights @ input + bias)
```

That `@` is matrix multiplication. The `weights` are a matrix. The `input` is a vector. If you do not know what those operations do, this line is magic. If you do know, it is the entire forward pass of a layer in three operations.

Every image your model processes is a matrix of pixel values. Every word embedding is a vector. Every layer of every neural network is a matrix transformation. You cannot build AI systems without being fluent in matrix operations the same way you cannot write code without understanding variables.

This lesson builds that fluency from scratch.

The Concept

Vectors: ordered lists of numbers

A vector is a list of numbers with a direction and magnitude. In AI, vectors represent data points, features, or parameters.

```
v = [3, 4]      -- a 2D vector
w = [1, 0, -2]  -- a 3D vector
```

A 2D vector `[3, 4]` points to coordinates (3, 4) on a plane. Its length (magnitude) is 5 (the 3-4-5 triangle).

Matrices: grids of numbers

A matrix is a 2D grid. Rows and columns. An $m \times n$ matrix has m rows and n columns.

```
A = | 1  2  3 |    -- 2x3 matrix (2 rows, 3 columns)
    | 4  5  6 |
```

In neural networks, weight matrices transform input vectors into output vectors. A layer with 784 inputs and 128 outputs uses a 128×784 weight matrix.

Why shapes matter

Matrix multiplication has a strict rule: $(m \times n) @ (n \times p) = (m \times p)$. The inner dimensions must match.

```
(128 x 784) @ (784 x 1) = (128 x 1)
  weights      input      output
Inner dimensions: 784 = 784 -- valid
```

If you get a shape mismatch error in PyTorch, this is why.

The operations map

Operation	What it does	Neural network use
Addition	Element-wise combine	Adding bias to output
Scalar multiply	Scale every element	Learning rate * gradients
Matrix multiply	Transform vectors	Layer forward pass
Transpose	Flip rows and columns	Backpropagation
Determinant	Single number summary	Checking invertibility
Inverse	Undo a transformation	Solving linear systems
Identity	Do-nothing matrix	Initialization, residual connections

Element-wise vs matrix multiplication

This distinction trips up beginners constantly.

Element-wise: multiply matching positions. Both matrices must be the same shape.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

Matrix multiplication: dot products of rows and columns. Inner dimensions must match.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1*5+2*7 & 1*6+2*8 \\ 3*5+4*7 & 3*6+4*8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Different operations, different results, different rules.

Broadcasting

When you add a bias vector to a matrix of outputs, the shapes do not match. Broadcasting stretches the smaller array to fit.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + [10, 20, 30]$$

Broadcasting stretches the vector across rows:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Every modern framework does this automatically. Understanding it prevents confusion when shapes seem wrong but the code runs.

Build It

Step 1: Vector class

```
class Vector:
    def __init__(self, data):
        self.data = list(data)
        self.size = len(self.data)

    def __repr__(self):
        return f"Vector({self.data})"

    def __add__(self, other):
        return Vector([a + b for a, b in zip(self.data, other.data)])

    def __sub__(self, other):
        return Vector([a - b for a, b in zip(self.data, other.data)])

    def __mul__(self, scalar):
        return Vector([x * scalar for x in self.data])
```

```

def dot(self, other):
    return sum(a * b for a, b in zip(self.data, other.data))

def magnitude(self):
    return sum(x ** 2 for x in self.data) ** 0.5

```

Step 2: Matrix class with core operations

```

class Matrix:
    def __init__(self, data):
        self.data = [list(row) for row in data]
        self.rows = len(self.data)
        self.cols = len(self.data[0])
        self.shape = (self.rows, self.cols)

    def __repr__(self):
        rows_str = "\n ".join(str(row) for row in self.data)
        return f"Matrix({self.shape}): \n {rows_str}"

    def __add__(self, other):
        return Matrix([
            [self.data[i][j] + other.data[i][j] for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def __sub__(self, other):
        return Matrix([
            [self.data[i][j] - other.data[i][j] for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def scalar_multiply(self, scalar):
        return Matrix([
            [self.data[i][j] * scalar for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def element_wise_multiply(self, other):
        return Matrix([
            [self.data[i][j] * other.data[i][j] for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def matmul(self, other):
        return Matrix([
            [
                sum(self.data[i][k] * other.data[k][j] for k in range(self.cols))
                for j in range(other.cols)
            ]
            for i in range(self.rows)
        ])

```

```

def transpose(self):
    return Matrix([
        [self.data[j][i] for j in range(self.rows)]
        for i in range(self.cols)
    ])

def determinant(self):
    if self.shape == (1, 1):
        return self.data[0][0]
    if self.shape == (2, 2):
        return self.data[0][0] * self.data[1][1] - self.data[0][1] * self.data[1][0]
    ]

    det = 0
    for j in range(self.cols):
        minor = Matrix([
            [self.data[i][k] for k in range(self.cols) if k != j]
            for i in range(1, self.rows)
        ])
        det += ((-1) ** j) * self.data[0][j] * minor.determinant()
    return det

def inverse_2x2(self):
    det = self.determinant()
    if det == 0:
        raise ValueError("Matrix is singular, no inverse exists")
    return Matrix([
        [self.data[1][1] / det, -self.data[0][1] / det],
        [-self.data[1][0] / det, self.data[0][0] / det]
    ])

@staticmethod
def identity(n):
    return Matrix([
        [1 if i == j else 0 for j in range(n)]
        for i in range(n)
    ])

```

Step 3: See it work

```

A = Matrix([[1, 2], [3, 4]])
B = Matrix([[5, 6], [7, 8]])

print("A + B =", (A + B).data)
print("A @ B =", A.matmul(B).data)
print("A^T =", A.transpose().data)
print("det(A) =", A.determinant())
print("A^-1 =", A.inverse_2x2().data)

I = Matrix.identity(2)
print("A @ A^-1 =", A.matmul(A.inverse_2x2()).data)

```


Step 4: Connect to neural networks

```
import random

inputs = Matrix([[0.5], [0.8], [0.2]])
weights = Matrix([
    [random.uniform(-1, 1) for _ in range(3)]
    for _ in range(2)
])
bias = Matrix([[0.1], [0.1]])

def relu_matrix(m):
    return Matrix([[max(0, val) for val in row] for row in m.data])

pre_activation = weights.matmul(inputs) + bias
output = relu_matrix(pre_activation)

print(f"Input shape: {inputs.shape}")
print(f"Weight shape: {weights.shape}")
print(f"Output shape: {output.shape}")
print(f"Output: {output.data}")
```

This is a single dense layer: $\text{output} = \text{relu}(W @ x + b)$. Every dense layer in every neural network does exactly this.

Use It

NumPy does everything above in fewer lines and orders of magnitude faster.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("A + B =\n", A + B)
print("A * B (element-wise) =\n", A * B)
print("A @ B (matrix multiply) =\n", A @ B)
print("A^T =\n", A.T)
print("det(A) =", np.linalg.det(A))
print("A^-1 =\n", np.linalg.inv(A))
print("I =\n", np.eye(2))

inputs = np.random.randn(3, 1)
weights = np.random.randn(2, 3)
bias = np.array([[0.1], [0.1]])
output = np.maximum(0, weights @ inputs + bias)
```

```
print(f"\nNeural network layer: {weights.shape} @ {inputs.shape} = {output.shape}")
print(f"Output:\n{output}")
```

The @ operator in Python calls `__matmul__`. NumPy implements it with optimized BLAS routines written in C and Fortran. Same math, 100x faster.

Broadcasting in NumPy:

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
bias = np.array([10, 20, 30])
print(matrix + bias)
```

NumPy automatically broadcasts the 1D bias across both rows. This is how bias addition works in every neural network framework.

Exercises

1. **Verify the inverse.** Multiply `A @ A.inverse_2x2()` and confirm you get the identity matrix. Try it with three different 2x2 matrices. What happens when the determinant is zero?
2. **Implement 3x3 inverse.** Extend the Matrix class to compute inverses for 3x3 matrices using the adjugate method. Test it against NumPy's `np.linalg.inv`.
3. **Build a two-layer network.** Using only your Matrix class (no NumPy), create a two-layer neural network: input (3) -> hidden (4) -> output (2). Initialize random weights, run a forward pass, and verify all shapes are correct.

Key Terms

Term	What people say	What it actually means
Vector	"An arrow"	An ordered list of numbers. In AI: a point in high-dimensional space.
Matrix	"A table of numbers"	A linear transformation. It maps vectors from one space to another.
Matrix multiply	"Just multiply the numbers"	Dot products between every row of the first matrix and every column of the second. Order matters.
Transpose	"Flip it"	Swap rows and columns. Turns an $m \times n$ matrix into $n \times m$. Critical in backpropagation.
Determinant	"Some number from the matrix"	Measures how much the matrix scales area (2D) or volume (3D). Zero means the transformation crushes a dimension.
Inverse	"Undo the matrix"	The matrix that reverses the transformation. Only exists when the determinant is not zero.

Term	What people say	What it actually means
Identity matrix	"The boring matrix"	The matrix equivalent of multiplying by 1. Used in residual connections (ResNets).
Broadcasting	"Magic shape fixing"	Stretching a smaller array to match a larger one by repeating along missing dimensions.
Element-wise	"Regular multiplication"	Multiply matching positions. Both arrays must have the same shape (or be broadcastable).

You are a math tutor who teaches linear algebra through geometric intuition. Your goal is to make matrix operations feel physical and visual, not abstract.

When explaining matrix concepts, follow these principles:

1. Start with geometry, not formulas. A matrix is a transformation that stretches, rotates, or squishes space. Show what happens to a unit square or unit vectors before writing any equations.
2. Connect every operation to neural networks. Do not teach math in isolation. After explaining what an operation does geometrically, immediately show where it appears in a real network.
3. Use concrete small examples. Work with 2x2 and 2x3 matrices so the student can verify by hand. Never jump to high dimensions before the low-dimensional case is solid.
4. Distinguish element-wise from matrix multiplication early and often. This is the most common source of bugs for beginners. Show both side by side with the same inputs so the difference is obvious.
5. Teach shapes as the primary debugging tool. Before computing anything, have the student predict the output shape. If they can predict shapes, they understand the operation.

When a student asks about a matrix operation, structure your response as:

- What it does geometrically (one sentence, with a visual if possible)
- The formula (compact, no unnecessary notation)
- A 2x2 or 2x3 worked example with actual numbers
- Where this shows up in neural networks (specific layer, specific step)
- A common mistake to watch for

Operations you should be prepared to explain:

- Addition: combining transformations, bias addition in networks
- Scalar multiplication: scaling gradients by learning rate

- Matrix multiplication: the core of every layer's forward pass
- Transpose: swapping input/output perspectives, used in backpropagation
- Determinant: measuring how much a transformation scales space, checking if inverse exists
- Inverse: undoing a transformation, solving linear systems
- Identity: the do-nothing transformation, residual connections
- Broadcasting: how bias vectors add to output matrices without explicit expansion

Avoid: - Abstract proofs without geometric grounding - Jumping to high dimensions before 2D/3D is clear
 - Using "obvious" or "trivially" or "it can be shown that" - Presenting formulas without worked numeric examples

Quiz

Pre-lesson check

1. For matrix multiplication $(m \times n) @ (n \times p)$, what must be true about the dimensions?
 - a) m must equal p
 - b) The inner dimensions n must match **(correct)**
 - c) All dimensions must be equal
 - d) m must be greater than p
Explanation: Matrix multiplication requires the number of columns in the first matrix (n) to equal the number of rows in the second matrix (n). The result has shape $(m \times p)$.
2. What is the identity matrix?
 - a) A matrix of all ones
 - b) A square matrix with ones on the diagonal and zeros elsewhere that acts as the multiplicative identity **(correct)**
 - c) A matrix where every element is unique
 - d) The transpose of any given matrix
Explanation: The identity matrix I has ones on the diagonal and zeros everywhere else.

Multiplying any matrix by I returns the original matrix unchanged, like multiplying a number by 1.

Post-lesson check

3. What is the key difference between element-wise multiplication and matrix multiplication?
 - a) Element-wise is faster while matrix multiplication is more accurate
 - b) Element-wise multiplies matching positions (same shape required), matrix multiplication takes dot products of rows and columns (inner dimensions must match) **(correct)**
 - c) They produce the same result but use different notation
 - d) Element-wise only works on vectors while matrix multiplication works on matrices
Explanation: Element-wise (Hadamard) product multiplies corresponding elements and requires identical shapes. Matrix multiplication computes dot products between rows and columns with the rule $(m,n) @ (n,p) = (m,p)$.
4. In the expression 'output = relu($W @ x + b$)', what role does broadcasting play?
 - a) It broadcasts the computation across multiple GPUs
 - b) It automatically stretches the bias vector b to match the shape of $W @ x$ so they can be added **(correct)**
 - c) It converts the data types of W and x to match
 - d) It repeats the relu activation across all elements
Explanation: $W @ x$ produces a column vector, and b is also a vector. Broadcasting stretches b across the batch dimension if needed, allowing element-wise addition without explicit shape matching.
5. What does a determinant of zero for a matrix indicate?
 - a) The matrix has all zero entries
 - b) The matrix is singular: it crushes at least one dimension, cannot be inverted, and has no unique solution **(correct)**

c) The matrix is the identity matrix

d) The matrix performs a rotation

Explanation: A zero determinant means the transformation collapses space by at least one dimension (e.g., mapping 2D to a line). The matrix has no inverse, and linear systems using it have either no solution or infinitely many.

Matrix Transformations

A matrix is a machine that reshapes space. Learn what it does to every point, and you understand the whole transformation.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lessons 01-02 (Linear Algebra Intuition, Vectors & Matrices Operations) **Time:** ~75 minutes

Learning Objectives

- Construct rotation, scaling, shearing, and reflection matrices and apply them to 2D and 3D points
- Compose multiple transformations by matrix multiplication and verify that order matters
- Compute eigenvalues and eigenvectors of 2x2 matrices from the characteristic equation
- Explain why eigenvalues determine PCA directions, RNN stability, and spectral clustering behavior

The Problem

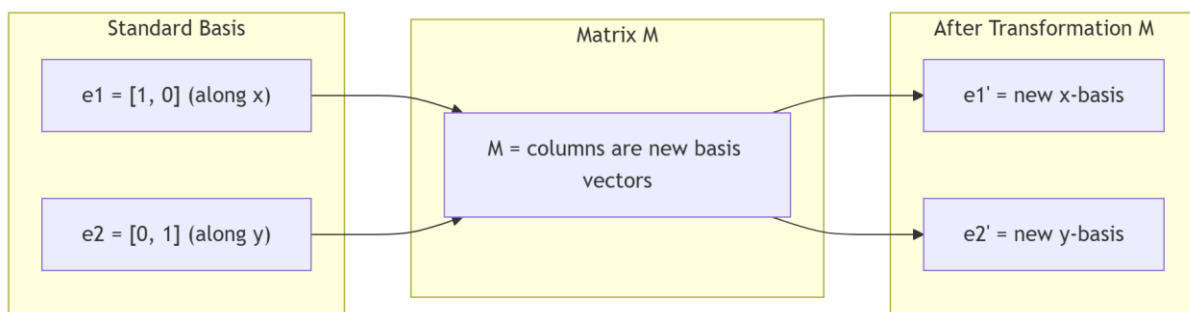
You read about PCA and see “find the eigenvectors of the covariance matrix.” You read about model stability and see “check if all eigenvalues have magnitude less than 1.” You read about data augmentation and see “apply a random rotation.” None of this makes sense until you understand what matrices do to space geometrically.

Matrices are not just grids of numbers. They are spatial machines. A rotation matrix spins points. A scaling matrix stretches them. A shearing matrix tilts them. Every transformation a neural network applies to data is one of these operations or a composition of them. This lesson makes those operations concrete.

The Concept

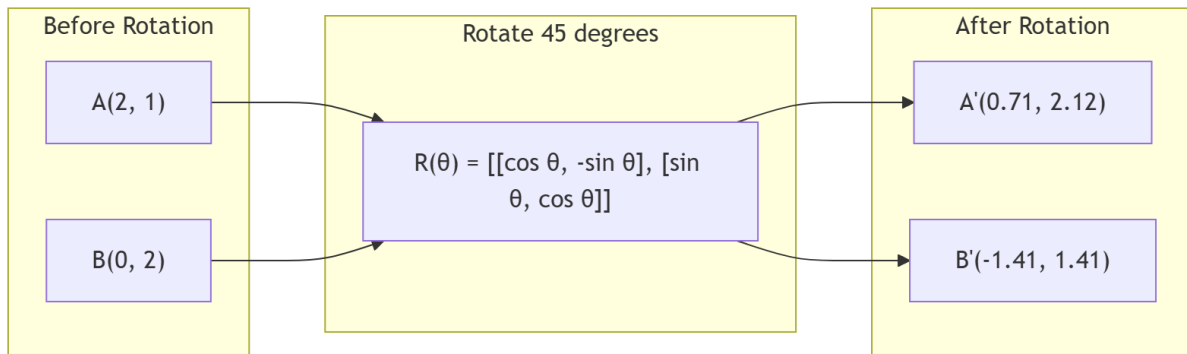
Transformations as matrices

Every linear transformation in 2D can be written as a 2x2 matrix. The matrix tells you exactly where the basis vectors $[1, 0]$ and $[0, 1]$ end up. Everything else follows.



Rotation

A 2D rotation by angle θ keeps distances and angles intact. It moves every point along a circular arc.



In 3D, you rotate around an axis. Each axis has its own rotation matrix:

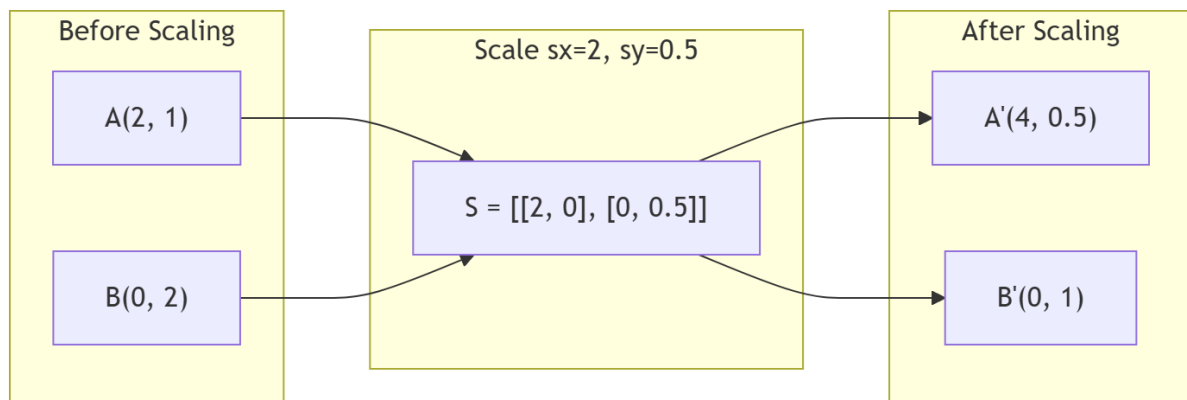
$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{array}{l} \text{Rotate around z-axis} \\ \text{(x-y plane spins, z stays)} \end{array}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \quad \begin{array}{l} \text{Rotate around x-axis} \\ \text{(y-z plane spins, x stays)} \end{array}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad \begin{array}{l} \text{Rotate around y-axis} \\ \text{(x-z plane spins, y stays)} \end{array}$$

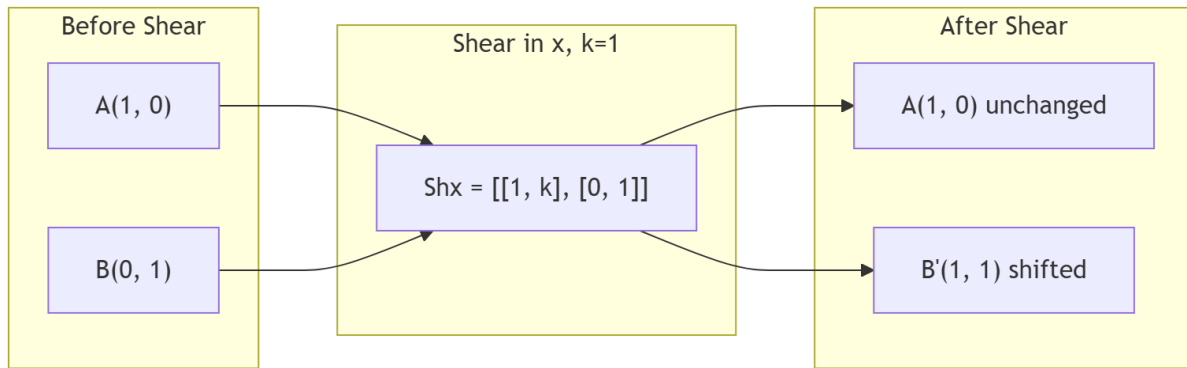
Scaling

Scaling stretches or compresses along each axis independently.



Shearing

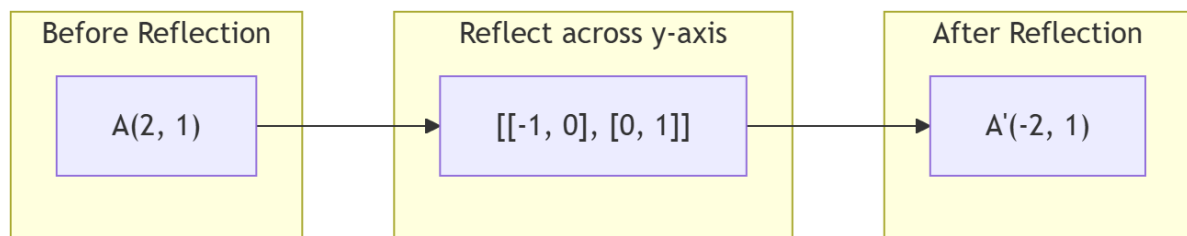
Shearing tilts one axis while keeping the other fixed. It turns rectangles into parallelograms.



Shear matrices: - $Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$ shifts x by $k * y$ - $Sh_y = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$ shifts y by $k * x$

Reflection

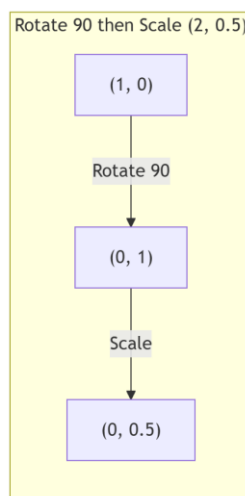
Reflection mirrors points across an axis or line.



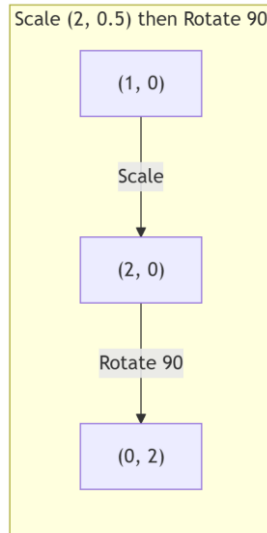
Reflection matrices: - Reflect across y-axis: $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ - Reflect across x-axis: $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Composition: chaining transformations

Applying transformation A then B is the same as multiplying their matrices: $result = B @ A @ point$. Order matters. Rotate then scale gives different results than scale then rotate.



Composed: $S @ R = \begin{bmatrix} 0 & -2 \\ 0.5 & 0 \end{bmatrix}$



Composed: $R @ S = \begin{bmatrix} 0 & -0.5 \\ 2 & 0 \end{bmatrix}$

Different results. Matrix multiplication is not commutative.

Eigenvalues and eigenvectors

Most vectors change direction when a matrix hits them. Eigenvectors are special: the matrix only scales them, never rotates them. The scaling factor is the eigenvalue.

$$A @ v = \text{lambda} * v$$

v is the eigenvector (direction that survives)
 lambda is the eigenvalue (how much it stretches)

Example: $A = \begin{vmatrix} 2 & 1 \\ 1 & 2 \end{vmatrix}$

Eigenvector $[1, 1]$ with eigenvalue 3:

$$A @ [1, 1] = [3, 3] = 3 * [1, 1] \quad (\text{same direction, scaled by 3})$$

Eigenvector $[1, -1]$ with eigenvalue 1:

$$A @ [1, -1] = [1, -1] = 1 * [1, -1] \quad (\text{same direction, unchanged})$$

The matrix stretches space by 3x along $[1, 1]$ and keeps $[1, -1]$ unchanged. Every other direction is a mix of these two.

Eigendecomposition

If a matrix has n linearly independent eigenvectors, it can be decomposed:

```
A = V @ D @ V(-1)
```

V = matrix whose columns are eigenvectors

D = diagonal matrix of eigenvalues

V⁽⁻¹⁾ = inverse of V

This says: rotate into eigenvector coordinates, scale along each axis, rotate back.

Why eigenvalues matter

PCA. The eigenvectors of the covariance matrix are the principal components. The eigenvalues tell you how much variance each component captures. Sort by eigenvalue, keep the top k, and you have dimensionality reduction.

Stability. In recurrent networks and dynamical systems, eigenvalues with magnitude > 1 cause outputs to explode. Magnitude < 1 causes them to vanish. This is the vanishing/exploding gradient problem stated in one sentence.

Spectral methods. Graph neural networks use eigenvalues of the adjacency matrix. Spectral clustering uses eigenvalues of the Laplacian. The eigenvectors reveal the structure of the graph.

Determinant as volume scaling factor

The determinant of a transformation matrix tells you how much it scales area (2D) or volume (3D).

```
det = 1:   area preserved (rotation)
det = 2:   area doubled
det = 0:   space crushed to lower dimension (singular)
det = -1:  area preserved but orientation flipped (reflection)
```

```
| det(Rotation) | = 1      (always)
| det(Scale sx, sy) | = sx * sy
| det(Shear) | = 1        (area preserved)
| det(Reflection) | = -1  (orientation flipped)
```

Build It

Step 1: Transformation matrices from scratch (Python)

```
import math

def rotation_2d(theta):
    c, s = math.cos(theta), math.sin(theta)
    return [[c, -s], [s, c]]
```

```

def scaling_2d(sx, sy):
    return [[sx, 0], [0, sy]]

def shearing_2d(kx, ky):
    return [[1, kx], [ky, 1]]

def reflection_x():
    return [[1, 0], [0, -1]]

def reflection_y():
    return [[-1, 0], [0, 1]]

def mat_vec_mul(matrix, vector):
    return [
        sum(matrix[i][j] * vector[j] for j in range(len(vector)))
        for i in range(len(matrix))
    ]

def mat_mul(a, b):
    rows_a, cols_b = len(a), len(b[0])
    cols_a = len(a[0])
    return [
        [sum(a[i][k] * b[k][j] for k in range(cols_a)) for j in range(cols_b)]
        for i in range(rows_a)
    ]

point = [1.0, 0.0]
angle = math.pi / 4

rotated = mat_vec_mul(rotation_2d(angle), point)
print(f"Rotate (1,0) by 45 deg: ({rotated[0]:.4f}, {rotated[1]:.4f})")

scaled = mat_vec_mul(scaling_2d(2, 3), [1.0, 1.0])
print(f"Scale (1,1) by (2,3): ({scaled[0]:.1f}, {scaled[1]:.1f})")

sheared = mat_vec_mul(shearing_2d(1, 0), [1.0, 1.0])
print(f"Shear (1,1) kx=1: ({sheared[0]:.1f}, {sheared[1]:.1f})")

reflected = mat_vec_mul(reflection_y(), [2.0, 1.0])
print(f"Reflect (2,1) across y: ({reflected[0]:.1f}, {reflected[1]:.1f})")

```

Step 2: Composition of transformations

```

R = rotation_2d(math.pi / 2)
S = scaling_2d(2, 0.5)

rotate_then_scale = mat_mul(S, R)
scale_then_rotate = mat_mul(R, S)

point = [1.0, 0.0]
result1 = mat_vec_mul(rotate_then_scale, point)

```

```

result2 = mat_vec_mul(scale_then_rotate, point)

print(f"Rotate 90 then scale: ({result1[0]:.2f}, {result1[1]:.2f})")
print(f"Scale then rotate 90: ({result2[0]:.2f}, {result2[1]:.2f})")
print(f"Same? {result1 == result2}")

```

Step 3: Eigenvalues from scratch (2x2)

For a 2x2 matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, eigenvalues solve the characteristic equation: $\lambda^2 - (a+d)\lambda + (ad - bc) = 0$.

```

def eigenvalues_2x2(matrix):
    a, b = matrix[0]
    c, d = matrix[1]
    trace = a + d
    det = a * d - b * c
    discriminant = trace ** 2 - 4 * det
    if discriminant < 0:
        real = trace / 2
        imag = (-discriminant) ** 0.5 / 2
        return (complex(real, imag), complex(real, -imag))
    sqrt_disc = discriminant ** 0.5
    return ((trace + sqrt_disc) / 2, (trace - sqrt_disc) / 2)

def eigenvector_2x2(matrix, eigenvalue):
    a, b = matrix[0]
    c, d = matrix[1]
    if abs(b) > 1e-10:
        v = [b, eigenvalue - a]
    elif abs(c) > 1e-10:
        v = [eigenvalue - d, c]
    else:
        if abs(a - eigenvalue) < 1e-10:
            v = [1, 0]
        else:
            v = [0, 1]
    mag = (v[0] ** 2 + v[1] ** 2) ** 0.5
    return [v[0] / mag, v[1] / mag]

A = [[2, 1], [1, 2]]
vals = eigenvalues_2x2(A)
print(f"Matrix: {A}")
print(f"Eigenvalues: {vals[0]:.4f}, {vals[1]:.4f}")

for val in vals:
    vec = eigenvector_2x2(A, val)
    result = mat_vec_mul(A, vec)
    scaled = [val * vec[0], val * vec[1]]
    print(f" lambda={val:.1f}, v=[round(x,4) for x in vec]")

```

```
print(f"    A@v = {[round(x,4) for x in result]}")
print(f"    1*v = {[round(x,4) for x in scaled]}")
```

Step 4: Determinant as volume scaling factor

```
def det_2x2(matrix):
    return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]

print(f"det(rotation 45) = {det_2x2(rotation_2d(math.pi/4)):.4f}")
print(f"det(scale 2,3)    = {det_2x2(scaling_2d(2, 3)):.1f}")
print(f"det(shear kx=1)   = {det_2x2(shearing_2d(1, 0)):.1f}")
print(f"det(reflect y)    = {det_2x2(reflection_y()):.1f}")

singular = [[1, 2], [2, 4]]
print(f"det(singular)     = {det_2x2(singular):.1f}")
print("Singular: columns are proportional, space collapses to a line.")
```

Use It

NumPy handles all of this with optimized routines.

```
import numpy as np

theta = np.pi / 4
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

point = np.array([1.0, 0.0])
print(f"Rotate (1,0) by 45 deg: {R @ point}")

S = np.diag([2.0, 3.0])
composed = S @ R
print(f"Scale(2,3) after Rotate(45): {composed @ point}")

A = np.array([[2, 1], [1, 2]], dtype=float)
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"\nEigenvalues: {eigenvalues}")
print(f"Eigenvectors (columns):\n{eigenvectors}")

for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    lam = eigenvalues[i]
    print(f"    A @ v{i} = {A @ v}, lambda * v{i} = {lam * v}")

print(f"\ndet(R) = {np.linalg.det(R):.4f}")
print(f"det(S) = {np.linalg.det(S):.1f}")

B = np.array([[3, 1], [0, 2]], dtype=float)
```

```

vals, vecs = np.linalg.eig(B)
D = np.diag(vals)
V = vecs
reconstructed = V @ D @ np.linalg.inv(V)
print(f"\nEigendecomposition A = V @ D @ V^-1:")
print(f"Original:\n{B}")
print(f"Reconstructed:\n{reconstructed}")

```

3D rotations with NumPy

```

def rotation_3d_z(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

def rotation_3d_x(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

point_3d = np.array([1.0, 0.0, 0.0])
rotated_z = rotation_3d_z(np.pi / 2) @ point_3d
rotated_x = rotation_3d_x(np.pi / 2) @ point_3d

print(f"\n3D point: {point_3d}")
print(f"Rotate 90 around z: {np.round(rotated_z, 4)}")
print(f"Rotate 90 around x: {np.round(rotated_x, 4)}")

```

Exercises

1. Apply rotation, scaling, and shearing to a unit square (corners at [0,0], [1,0], [1,1], [0,1]). Print the transformed corners for each. Verify that rotation preserves distances between corners.
2. Find the eigenvalues of the matrix $\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$ by hand using the characteristic equation. Then verify with your from-scratch function and with NumPy.
3. Create a composition of three transformations (rotate 30 degrees, scale by [1.5, 0.8], shear with $k_x=0.3$) and apply it to 8 points arranged in a circle. Print before and after coordinates. Compute the determinant of the composed matrix and verify it equals the product of the individual determinants.

Key Terms

Term	What people say	What it actually means
Rotation matrix	"Spins things"	An orthogonal matrix that moves points along circular arcs while preserving distances and angles. Determinant is always 1.
Scaling matrix	"Makes things bigger"	A diagonal matrix that stretches or compresses independently along each axis. Determinant is the product of scale factors.

Term	What people say	What it actually means
Shearing matrix	"Slants things"	A matrix that shifts one coordinate proportionally to another, turning rectangles into parallelograms. Determinant is 1.
Reflection	"Mirrors things"	A matrix that flips space across an axis or plane. Determinant is -1.
Composition	"Do two things"	Multiplying transformation matrices to chain operations. Order matters: $B @ A$ means apply A first, then B.
Eigenvector	"Special direction"	A direction that the matrix only scales, never rotates. The transformation's fingerprint.
Eigenvalue	"How much it stretches"	The scalar factor by which the matrix scales its eigenvector. Can be negative (flip) or complex (rotation).
Eigendecomposition	"Break the matrix apart"	Writing a matrix as $V @ D @ V^{-1}$, separating it into its fundamental scaling directions and magnitudes.
Determinant	"A single number from a matrix"	The factor by which the transformation scales area (2D) or volume (3D). Zero means the transformation is irreversible.
Characteristic equation	"Where eigenvalues come from"	$\det(A - \lambda * I) = 0$. The polynomial whose roots are the eigenvalues.

You are a geometric transformation analyzer. Your job is to take a matrix and explain exactly what it does to space.

When a user provides a 2x2 or 3x3 matrix, decompose it into its geometric components and explain each one.

Structure your response as:

1. **Determinant analysis.** Compute the determinant. State whether the transformation preserves area ($\det = 1$ or -1), scales area ($|\det| \neq 1$), or collapses a dimension ($\det = 0$). If the determinant is negative, note that orientation is flipped.
2. **Eigenvalue/eigenvector analysis.** Compute the eigenvalues and eigenvectors. Identify directions that survive the transformation unchanged (scaled only). If eigenvalues are complex, the transformation involves rotation.
3. **Decomposition into primitives.** Break the matrix into a composition of:
 - Rotation: angle θ from the eigenvalue argument or from SVD
 - Scaling: factors along each axis from singular values or eigenvalue magnitudes

- Shearing: off-diagonal contribution after removing rotation and scaling
 - Reflection: present if determinant is negative
4. **What happens to the unit square.** Describe where the four corners $[0,0]$, $[1,0]$, $[1,1]$, $[0,1]$ end up. State the new shape (parallelogram, rectangle, line, etc.).
 5. **Visualization suggestion.** Recommend a specific way to plot the transformation: the unit square before and after, the unit circle mapped to an ellipse, or basis vectors showing the column picture.

Use this decision framework for identifying the transformation type:

Matrix pattern	Transformation
$[[\cos, -\sin], [\sin, \cos]]$	Pure rotation by θ
$[[a, 0], [0, d]]$ with $a, d > 0$	Axis-aligned scaling
$[[1, k], [0, 1]]$ or $[[1, 0], [k, 1]]$	Pure shear
Determinant = -1, orthogonal	Pure reflection
Symmetric with positive eigenvalues	Scaling along eigenvector directions
General	Compose rotation, scaling, shear from SVD: $A = U S V^T$

For 3x3 matrices, also identify: - The axis of rotation (the eigenvector with eigenvalue 1) - Whether the transformation is proper ($\det > 0$) or improper ($\det < 0$)

Avoid: - Listing matrix entries without geometric interpretation - Skipping the determinant (it is the single most informative number) - Giving only abstract math without connecting to what happens visually - Ignoring the case where eigenvalues are complex (this means rotation is involved)

When eigenvalues are complex conjugates $a \pm bi$: - The rotation angle is $\arctan(b/a)$ - The scaling factor per rotation is $\sqrt{a^2 + b^2}$ - The transformation spirals: it rotates and scales simultaneously

Always end with a one-sentence summary: “This matrix [rotates/scales/shears/reflects] space by [specific amounts].”

Quiz

Pre-lesson check

1. What is an eigenvector of a matrix?

- a) The largest row in the matrix
 - b) A vector that the matrix only scales (never rotates) when multiplied **(correct)**
 - c) A vector perpendicular to all columns of the matrix
 - d) The diagonal entries of the matrix expressed as a vector
*Explanation: An eigenvector v satisfies $Av = \lambda v$, meaning the matrix A only stretches v by the scalar factor λ (the eigenvalue) without changing its direction.**
2. What does the determinant of a 2D transformation matrix represent geometrically?
- a) The angle of rotation applied by the matrix
 - b) The factor by which the matrix scales area **(correct)**
 - c) The number of eigenvectors the matrix has
 - d) The trace of the matrix
Explanation: The determinant measures how much the transformation scales area. $\det=1$ preserves area (rotation), $\det=2$ doubles area, $\det=0$ crushes to a lower dimension, and $\det=-1$ preserves area but flips orientation.

Post-lesson check

3. Why does the order of matrix transformations matter? (i.e., why is $R @ S$ different from $S @ R$?)
- a) Matrix addition is not commutative
 - b) Matrix multiplication is not commutative: rotating then scaling gives a different result than scaling then rotating **(correct)**
 - c) The determinants are different for each order
 - d) One order produces a larger matrix than the other
Explanation: Matrix multiplication is not commutative. Rotating $(1,0)$ by 90 degrees then scaling by $(2,0.5)$ gives $(0,0.5)$, but scaling first then rotating gives $(0,2)$. The geometric operations compose differently.

4. In a recurrent neural network, what happens when the weight matrix has eigenvalues with magnitude greater than 1?
- a) The network learns faster
 - b) Outputs explode exponentially over time steps (exploding gradient problem) **(correct)**
 - c) The network becomes more stable
 - d) The eigenvalues converge to 1 over training
Explanation: Repeated multiplication by a matrix amplifies the eigenvalue directions. Eigenvalues > 1 cause exponential growth (exploding gradients), while eigenvalues < 1 cause exponential decay (vanishing gradients).
5. The matrix $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ has eigenvalues 3 and 1. What does eigendecomposition $A = V @ D @ V^{-1}$ reveal?
- a) A is equivalent to two rotations
 - b) A stretches space by 3x along the $[1,1]$ direction and leaves the $[1,-1]$ direction unchanged **(correct)**
 - c) A compresses all vectors by a factor of 2
 - d) A has rank 1 and maps all vectors to a line
Explanation: The eigenvalue 3 with eigenvector $[1,1]$ means A stretches 3x along the diagonal. The eigenvalue 1 with eigenvector $[1,-1]$ means A leaves the anti-diagonal unchanged. D holds $\{3,1\}$, V holds the eigenvectors.
-

Calculus for Machine Learning

Derivatives tell you which way is downhill. That is all a neural network needs to learn.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-03 **Time:** ~60 minutes

Learning Objectives

- Compute numerical and analytical derivatives for common ML functions (x^2 , sigmoid, cross-entropy)
- Implement gradient descent from scratch to minimize a loss function in 1D and 2D
- Derive the gradient of a linear regression model and train it via manual weight updates
- Explain the Hessian matrix, Taylor series approximations, and their connection to optimization methods

The Problem

You have a neural network with millions of weights. Each weight is a knob. You need to figure out which direction to turn every single knob to make the model slightly less wrong. Calculus gives you that direction.

Without calculus, training a neural network would mean trying random changes and hoping for the best. With derivatives, you know exactly how each weight affects the error. You turn every knob the right way, every time.

The Concept

What is a derivative?

A derivative measures the rate of change. For a function $y = f(x)$, the derivative $f'(x)$ tells you: if you nudge x by a tiny amount, how much does y change?

Geometrically, the derivative is the slope of the tangent line at a point.

$f(x) = x^2$:

x	f(x)	f'(x) (slope)
0	0	0 (flat, at the bottom)
1	1	2
2	4	4 (tangent line slope at this point)

x	f(x)	f'(x) (slope)
3	9	6

At x=2, the slope is 4. If you move x a tiny bit to the right, y increases by about 4 times that amount. At x=0, the slope is 0. You are at the bottom of the bowl.

The formal definition:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

In code, you skip the limit and just use a very small h. That is the numerical derivative.

Partial derivatives: one variable at a time

Real functions have many inputs. A neural network loss depends on thousands of weights. A partial derivative holds all variables constant except one, then takes the derivative with respect to that one.

$$f(x, y) = x^2 + 3xy + y^2$$

$$\frac{df}{dx} = 2x + 3y \quad (\text{treat } y \text{ as a constant})$$

$$\frac{df}{dy} = 3x + 2y \quad (\text{treat } x \text{ as a constant})$$

Each partial derivative answers: if I nudge just this one weight, how does the loss change?

The gradient: vector of all partial derivatives

The gradient collects every partial derivative into one vector. For a function f(x, y, z), the gradient is:

$$\text{grad } f = [\frac{df}{dx}, \frac{df}{dy}, \frac{df}{dz}]$$

The gradient points in the direction of steepest ascent. To minimize a function, go in the opposite direction.

Contour plot of f(x,y) = x² + y²:

The function forms a bowl shape with concentric circles as contour lines. The minimum is at (0, 0).

Point	grad f	-grad f (descent direction)
(1, 1)	[2, 2] (points uphill, away from minimum)	[-2, -2] (points downhill, toward minimum)
(0, 0)	[0, 0] (flat, at the minimum)	[0, 0]

This is gradient descent in a picture. Compute the gradient, negate it, take a step.

The connection to optimization

Training a neural network is optimization. You have a loss function $L(w_1, w_2, \dots, w_n)$ that measures how wrong the model is. You want to minimize it.

Gradient descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} * dL/dw$$

For every weight:

1. Compute the partial derivative of loss with respect to that weight
2. Subtract a small multiple of it from the weight
3. Repeat

The learning rate controls step size. Too big and you overshoot. Too small and you crawl.

Loss landscape (1D slice):

The loss function $L(w)$ forms a curve with peaks and valleys as the weight w varies.

Feature	Description
Global minimum	The lowest point on the entire curve – the best solution
Local minimum	A valley that is lower than its neighbors but not the lowest overall
Slope	Gradient descent follows the slope downhill from any starting point

Gradient descent follows the slope downhill. It can get stuck in local minima, but in high-dimensional spaces (millions of weights) this is rarely a practical problem.

Numerical vs analytical derivatives

There are two ways to compute a derivative.

Analytical: apply calculus rules by hand. For $f(x) = x^2$, the derivative is $f'(x) = 2x$. Exact. Fast.

Numerical: approximate using the definition. Compute $f(x+h)$ and $f(x-h)$ for a tiny h , then use the difference.

Numerical (central difference):

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$

$h = 0.0001$ works well in practice

Numerical derivatives are slower but work for any function. Analytical derivatives are fast but require you to derive the formula. Neural network frameworks use a third approach: automatic differentiation, which computes exact derivatives mechanically. You will see that in Phase 3.

Derivatives by hand for simple functions

These are the derivatives you will see over and over in ML.

Function	Derivative	Used in
-----	-----	-----
$f(x) = x^2$	$f'(x) = 2x$	Loss functions (MSE)
$f(x) = wx + b$	$f'(w) = x$	Linear layer (gradient w.r.t. weight)
	$f'(b) = 1$	Linear layer (gradient w.r.t. bias)
	$f'(x) = w$	Linear layer (gradient w.r.t. input)
$f(x) = e^x$	$f'(x) = e^x$	Softmax, attention
$f(x) = \ln(x)$	$f'(x) = 1/x$	Cross-entropy loss
$f(x) = 1/(1+e^{-x})$	$f'(x) = f(x)(1-f(x))$	Sigmoid activation

For $f(x) = x^2$:

$$f(x) = x^2 \quad f'(x) = 2x$$

x	f(x)	f'(x)	meaning
-2	4	-4	slope tilts left (decreasing)
-1	1	-2	slope tilts left (decreasing)
0	0	0	flat (minimum!)
1	1	2	slope tilts right (increasing)
2	4	4	slope tilts right (increasing)

For $f(w) = wx + b$ with $x=3$, $b=1$:

$$f(w) = 3w + 1 \quad f'(w) = 3$$

The derivative with respect to w is just x .

If x is big, a small change in w causes a big change in output.

The chain rule

When functions are composed, the chain rule tells you how to differentiate.

If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$

Example: $y = (3x + 1)^2$

outer: $f(u) = u^2$ $f'(u) = 2u$

inner: $g(x) = 3x + 1$ $g'(x) = 3$

$dy/dx = 2(3x + 1) * 3 = 6(3x + 1)$

Neural networks are chains of functions: input -> linear -> activation -> linear -> activation -> loss.

Backpropagation is the chain rule applied repeatedly from output to input. That is the entire algorithm.

The Hessian Matrix

The gradient tells you the slope. The Hessian tells you the curvature.

The Hessian is the matrix of second-order partial derivatives. For a function $f(x_1, x_2, \dots, x_n)$, entry (i, j) of the Hessian is:

$$H[i][j] = d^2f / (dx_i * dx_j)$$

For a 2-variable function $f(x, y)$:

$$H = \begin{vmatrix} d^2f/dx^2 & d^2f/dxdy \\ d^2f/dydx & d^2f/dy^2 \end{vmatrix}$$

What the Hessian tells you at a critical point (where gradient = 0):

Hessian property	Meaning	Example surface
Positive definite (all eigenvalues > 0)	Local minimum	Bowl pointing up
Negative definite (all eigenvalues < 0)	Local maximum	Bowl pointing down
Indefinite (mixed eigenvalues)	Saddle point	Horse saddle shape

Example: $f(x, y) = x^2 - y^2$ (a saddle function)

$$\begin{aligned} df/dx &= 2x & df/dy &= -2y \\ d^2f/dx^2 &= 2 & d^2f/dy^2 &= -2 & d^2f/dxdy &= 0 \end{aligned}$$

$$H = \begin{vmatrix} 2 & 0 \\ 0 & -2 \end{vmatrix}$$

Eigenvalues: 2 and -2 (one positive, one negative)
--> Saddle point at (0, 0)

Compare with $f(x, y) = x^2 + y^2$ (a bowl):

$$H = \begin{vmatrix} 2 & 0 \\ 0 & 2 \end{vmatrix}$$

Eigenvalues: 2 and 2 (both positive)
--> Local minimum at (0, 0)

Why the Hessian matters in ML:

Newton's method uses the Hessian to take better optimization steps than gradient descent. Instead of just following the slope, it accounts for curvature:

Newton's update: $w_{\text{new}} = w_{\text{old}} - H^{-1} * \text{gradient}$
Gradient descent: $w_{\text{new}} = w_{\text{old}} - \text{lr} * \text{gradient}$

Newton's method converges faster because the Hessian "rescales" the gradient – steep directions get smaller steps, flat directions get larger steps.

The catch: for a neural network with N parameters, the Hessian is $N \times N$. A model with 1 million parameters would need a 1 trillion-entry matrix. That is why we use approximations.

Method	What it uses	Cost	Convergence
Gradient descent	First derivatives only	$O(N)$ per step	Slow (linear)
Newton's method	Full Hessian	$O(N^3)$ per step	Fast (quadratic)
L-BFGS	Approximate Hessian from gradient history	$O(N)$ per step	Medium (superlinear)
Adam	Per-parameter adaptive rates (diagonal Hessian approx)	$O(N)$ per step	Medium
Natural gradient	Fisher information matrix (statistical Hessian)	$O(N^2)$ per step	Fast

In practice, Adam is the default optimizer for deep learning. It approximates second-order information cheaply by tracking the running mean and variance of gradients per parameter.

Taylor Series Approximation

Any smooth function can be approximated locally by a polynomial:

$$f(x + h) = f(x) + f'(x)h + (1/2)f''(x)h^2 + (1/6)f'''(x)h^3 + \dots$$

The more terms you include, the better the approximation – but only near the point x .

Why Taylor series matter for ML:

- **First-order Taylor = gradient descent.** When you use $f(x + h) \sim f(x) + f'(x)h$, you are making a linear approximation. Gradient descent minimizes this linear model to choose $h = -1/f'(x)$.
- **Second-order Taylor = Newton's method.** Using $f(x + h) \sim f(x) + f'(x)h + (1/2)f''(x)h^2$, you get a quadratic model. Minimizing it gives $h = -f'(x)/f''(x)$ – Newton's step.
- **Loss function design.** MSE and cross-entropy are smooth, which means their Taylor expansions are well-behaved. This is not an accident. Smooth losses make optimization predictable.

Approximation order	What it captures	Optimization method
-----	-----	-----
0th order (constant)	Just the value	Random search
1st order (linear)	Slope	Gradient descent
2nd order (quadratic)	Curvature	Newton's method
Higher orders	Finer structure	Rarely used in ML

The key insight: all gradient-based optimization is really about approximating the loss function locally and stepping to the minimum of that approximation.

Integrals in ML

Derivatives tell you rates of change. Integrals compute accumulations – area under a curve.

In ML, you rarely compute integrals by hand, but the concept is everywhere:

Probability. For a continuous random variable with density $p(x)$:

$$P(a < X < b) = \text{integral from } a \text{ to } b \text{ of } p(x) \, dx$$

The area under the probability density curve between a and b is the probability of landing in that range.

Expected value. The average outcome weighted by probability:

$$E[f(X)] = \text{integral of } f(x) * p(x) \, dx$$

The expected loss over a data distribution is an integral. Training minimizes an empirical approximation of this.

KL divergence. Measures how different two distributions are:

$$KL(p \parallel q) = \text{integral of } p(x) * \log(p(x) / q(x)) \, dx$$

Used in VAEs, knowledge distillation, and Bayesian inference.

Normalization constants. In Bayesian inference:

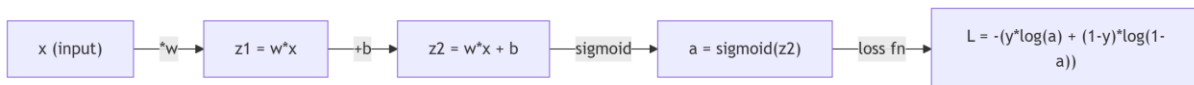
$$p(w \mid \text{data}) = p(\text{data} \mid w) * p(w) / \text{integral of } p(\text{data} \mid w) * p(w) \, dw$$

The denominator is an integral over all possible parameter values. It is often intractable, which is why we use approximations like MCMC and variational inference.

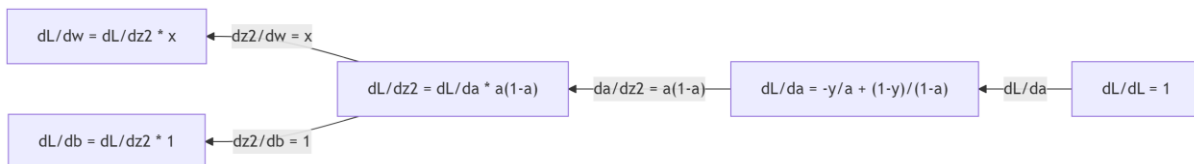
Integral concept	Where it appears in ML
Area under curve	Probability from density functions
Expected value	Loss functions, risk minimization
KL divergence	VAEs, policy optimization, distillation
Normalization	Bayesian posteriors, softmax denominator
Marginal likelihood	Model comparison, evidence lower bound (ELBO)

Multivariable Chain Rule in a Computation Graph

The chain rule does not just apply to scalar functions in a line. In a neural network, variables fan out and merge. Here is how derivatives flow through a simple forward pass:



The backward pass computes gradients right to left:



Each arrow multiplies by the local derivative. The gradient for any parameter is the product of all local derivatives along the path from loss to that parameter. When paths branch and merge, you sum the contributions (multivariate chain rule).

This is all backpropagation is: the chain rule applied systematically through a computation graph, from output to inputs.

The Jacobian matrix

When a function maps a vector to a vector (like a neural network layer), its derivative is a matrix. The Jacobian contains every partial derivative of every output with respect to every input.

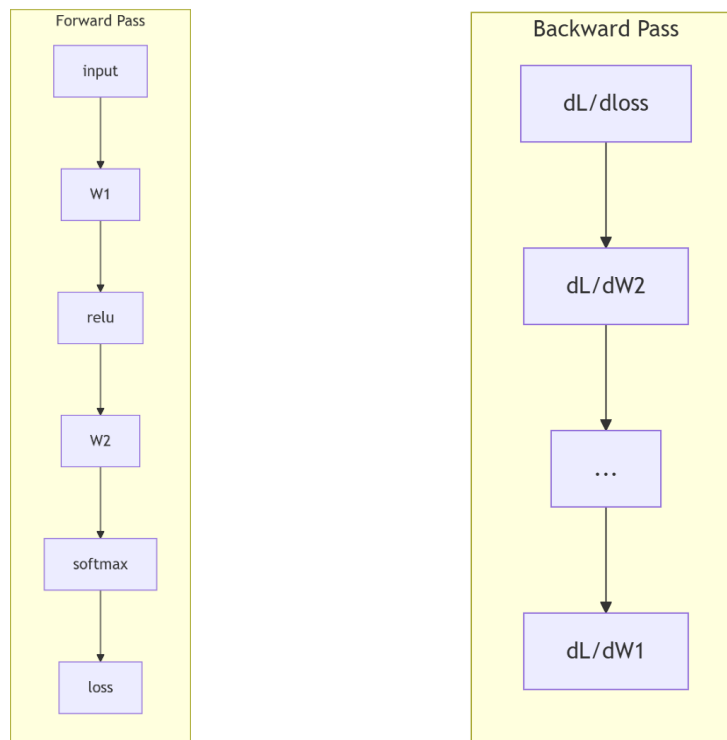
For $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian J is an $m \times n$ matrix:

	x1	x2	...	xn
f1	$df1/dx1$	$df1/dx2$	...	$df1/dxn$
f2	$df2/dx1$	$df2/dx2$	...	$df2/dxn$
...	...	...	...	...
fm	$dfm/dx1$	$dfm/dx2$	...	dfm/dxn

You will not compute Jacobians by hand for neural networks. PyTorch handles it. But knowing it exists helps you understand shapes in backpropagation: if a layer maps $\mathbb{R}^n$ to $\mathbb{R}^m$, its Jacobian is $m \times n$. The gradient flows backward through the transpose of this matrix.

Why this matters for neural networks

Every weight in a neural network gets a gradient. The gradient tells you how to adjust that weight to reduce the loss.



Each weight update: $-W1 = W1 - lr * dL/dW1$ $-W2 = W2 - lr * dL/dW2$

The forward pass computes the prediction and loss. The backward pass computes the gradient of the loss with respect to every weight. Then every weight takes a small step downhill. Repeat for millions of steps. That is deep learning.

Build It

Step 1: Numerical derivative from scratch

```
def numerical_derivative(f, x, h=1e-7):
    return (f(x + h) - f(x - h)) / (2 * h)

def f(x):
    return x ** 2

for x in [-2, -1, 0, 1, 2]:
    numerical = numerical_derivative(f, x)
    analytical = 2 * x
    print(f"x={x:2d} f'(x) numerical={numerical:.6f} analytical={analytical:.1f}")
)
```

The numerical derivative matches the analytical one to many decimal places.

Step 2: Partial derivatives and gradients

```
def numerical_gradient(f, point, h=1e-7):
    gradient = []
    for i in range(len(point)):
        point_plus = list(point)
        point_minus = list(point)
        point_plus[i] += h
        point_minus[i] -= h
        partial = (f(point_plus) - f(point_minus)) / (2 * h)
        gradient.append(partial)
    return gradient

def f_multi(point):
    x, y = point
    return x**2 + 3*x*y + y**2

grad = numerical_gradient(f_multi, [1.0, 2.0])
print(f"Numerical gradient at (1,2): {[f'{g:.4f}' for g in grad]}")
print(f"Analytical gradient at (1,2): [2*1+3*2, 3*1+2*2] = [{2*1+3*2}, {3*1+2*2}]")
```

Step 3: Gradient descent to find the minimum of $f(x) = x^2$

```
x = 5.0
lr = 0.1
```

```

for step in range(20):
    grad = 2 * x
    x = x - lr * grad
    print(f"step {step:2d}  x={x:8.4f}  f(x)={x**2:10.6f}")

```

Starting at $x=5$, each step moves closer to $x=0$ (the minimum).

Step 4: Gradient descent on a 2D function

```

def f_2d(point):
    x, y = point
    return x**2 + y**2

point = [4.0, 3.0]
lr = 0.1
for step in range(30):
    grad = numerical_gradient(f_2d, point)
    point = [p - lr * g for p, g in zip(point, grad)]
    loss = f_2d(point)
    if step % 5 == 0 or step == 29:
        print(f"step {step:2d}  point=({point[0]:7.4f}, {point[1]:7.4f})  f={loss:
.6f}")

```

Step 5: Comparing numerical and analytical derivatives

```

import math

test_functions = [
    ("x^2",      lambda x: x**2,      lambda x: 2*x),
    ("x^3",      lambda x: x**3,      lambda x: 3*x**2),
    ("sin(x)",   lambda x: math.sin(x), lambda x: math.cos(x)),
    ("e^x",      lambda x: math.exp(x), lambda x: math.exp(x)),
    ("1/x",      lambda x: 1/x,        lambda x: -1/x**2),
]

x = 2.0
print(f"{'Function':<12} {'Numerical':>12} {'Analytical':>12} {'Error':>12}")
print("-" * 50)
for name, f, df in test_functions:
    num = numerical_derivative(f, x)
    ana = df(x)
    err = abs(num - ana)
    print(f"{name:<12} {num:12.6f} {ana:12.6f} {err:12.2e}")

```

Step 6: Computing the Hessian numerically

```

def hessian_2d(f, x, y, h=1e-5):
    fxx = (f(x + h, y) - 2 * f(x, y) + f(x - h, y)) / (h ** 2)
    fyy = (f(x, y + h) - 2 * f(x, y) + f(x, y - h)) / (h ** 2)
    fxy = (f(x + h, y + h) - f(x + h, y - h) - f(x - h, y + h) + f(x - h, y - h)) / (4

```

```

* h ** 2)
    return [[fxx, fxy], [fxy, fyy]]

def saddle(x, y):
    return x ** 2 - y ** 2

def bowl(x, y):
    return x ** 2 + y ** 2

H_saddle = hessian_2d(saddle, 0.0, 0.0)
H_bowl = hessian_2d(bowl, 0.0, 0.0)
print(f"Saddle Hessian: {H_saddle}") # [[2, 0], [0, -2]] -- mixed signs
print(f"Bowl Hessian: {H_bowl}") # [[2, 0], [0, 2]] -- both positive

```

The Hessian of the saddle function has eigenvalues 2 and -2 (mixed signs, confirming a saddle point). The bowl has eigenvalues 2 and 2 (both positive, confirming a minimum).

Step 7: Taylor approximation in action

```

import math

def taylor_approx(f, f_prime, f_double_prime, x0, h, order=2):
    result = f(x0)
    if order >= 1:
        result += f_prime(x0) * h
    if order >= 2:
        result += 0.5 * f_double_prime(x0) * h ** 2
    return result

x0 = 0.0
for h in [0.1, 0.5, 1.0, 2.0]:
    true_val = math.sin(h)
    t1 = taylor_approx(math.sin, math.cos, lambda x: -math.sin(x), x0, h, order=1)
    t2 = taylor_approx(math.sin, math.cos, lambda x: -math.sin(x), x0, h, order=2)
    print(f"h={h:.1f} sin(h)={true_val:.4f} order1={t1:.4f} order2={t2:.4f}")

```

Near $x_0=0$, $\sin(x) \sim x$ (first-order Taylor). The approximation is excellent for small h but breaks down for large h . This is why gradient descent works best with small learning rates – each step assumes the linear approximation is accurate.

Step 8: Why this matters for a neural network

```

import random

random.seed(42)

w = random.gauss(0, 1)
b = random.gauss(0, 1)
lr = 0.01

```

```

xs = [1.0, 2.0, 3.0, 4.0, 5.0]
ys = [3.0, 5.0, 7.0, 9.0, 11.0]

for epoch in range(200):
    total_loss = 0
    dw = 0
    db = 0
    for x, y in zip(xs, ys):
        pred = w * x + b
        error = pred - y
        total_loss += error ** 2
        dw += 2 * error * x
        db += 2 * error
    dw /= len(xs)
    db /= len(xs)
    total_loss /= len(xs)
    w -= lr * dw
    b -= lr * db
    if epoch % 40 == 0 or epoch == 199:
        print(f"epoch {epoch:3d}  w={w:.4f}  b={b:.4f}  loss={total_loss:.6f}")

print(f"\nLearned: y = {w:.2f}x + {b:.2f}")
print(f"Actual: y = 2x + 1")

```

Every gradient-based training loop follows this pattern: predict, compute loss, compute gradients, update weights.

Use It

With NumPy, the same operations are faster and more concise:

```

import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([3, 5, 7, 9, 11], dtype=float)

w, b = np.random.randn(), np.random.randn()
lr = 0.01

for epoch in range(200):
    pred = w * x + b
    error = pred - y
    loss = np.mean(error ** 2)
    dw = np.mean(2 * error * x)
    db = np.mean(2 * error)
    w -= lr * dw
    b -= lr * db

print(f"Learned: y = {w:.2f}x + {b:.2f}")

```


You just built gradient descent from scratch. PyTorch automates the gradient computation, but the update loop is identical.

Exercises

1. Implement `numerical_second_derivative(f, x)` using `numerical_derivative` called twice. Verify that the second derivative of x^3 at $x=2$ is 12.
2. Use gradient descent to find the minimum of $f(x, y) = (x - 3)^2 + (y + 1)^2$. Start from $(0, 0)$. The answer should converge to $(3, -1)$.
3. Add momentum to the gradient descent loop: maintain a velocity vector that accumulates past gradients. Compare convergence speed with and without momentum on $f(x) = x^4 - 3x^2$.

Key Terms

Term	What people say	What it actually means
Derivative	"The slope"	The rate of change of a function at a point. Tells you how much the output changes per unit change in input.
Partial derivative	"Derivative of one variable"	The derivative with respect to one variable while all others are held constant.
Gradient	"Direction of steepest ascent"	A vector of all partial derivatives. Points in the direction that increases the function fastest.
Gradient descent	"Go downhill"	Subtract the gradient (times a learning rate) from the parameters to reduce the loss. The core of neural network training.
Learning rate	"Step size"	A scalar that controls how big each gradient descent step is. Too large: diverge. Too small: converge slowly.
Chain rule	"Multiply the derivatives"	The rule for differentiating composed functions: $df/dx = df/dg * dg/dx$. The mathematical basis of backpropagation.
Jacobian	"Matrix of derivatives"	When a function maps vectors to vectors, the Jacobian is the matrix of all partial derivatives of outputs with respect to inputs.
Numerical derivative	"Finite differences"	Approximating a derivative by evaluating the function at two nearby points and computing the slope between them.
Backpropagation	"Reverse-mode autodiff"	Computing gradients layer by layer from output to input using the chain rule. How neural networks learn.
Hessian	"Matrix of second derivatives"	The matrix of all second-order partial derivatives. Describes the curvature of a function. Positive definite Hessian at a critical point means local minimum.

Term	What people say	What it actually means
Taylor series	“Polynomial approximation”	Approximating a function near a point using its derivatives: $f(x+h) \sim f(x) + f'(x)h + (1/2)f''(x)h^2 + \dots$. The basis for understanding why gradient descent and Newton’s method work.
Integral	“Area under the curve”	The accumulation of a quantity over a range. In ML, integrals define probabilities, expected values, and KL divergence.

Gradient Computation for ML

Practical reference for computing gradients of loss functions, activation functions, and layer operations used in neural networks.

Decision Checklist

1. Is the function composed of simple primitives (power, exp, log, trig)? Use analytical derivatives and the chain rule.
2. Is the function a custom or black-box operation? Use numerical differentiation: $(f(x+h) - f(x-h)) / (2h)$ with $h = 1e-7$.
3. Is the function built from tensor operations in PyTorch/JAX? Let autograd handle it. Verify with numerical check.
4. Do you need the gradient of a scalar loss w.r.t. a matrix of weights? Apply the chain rule through the computation graph, one node at a time.
5. Is there a non-differentiable operation (argmax, rounding, sampling)? Use a straight-through estimator or reparameterization trick.

When to use each approach

Approach	When to use	Cost
Analytical (hand-derived)	Simple functions, verifying autograd output	Free at runtime
Numerical (finite differences)	Debugging, gradient checking, black-box functions	$2n$ forward passes for n parameters
Automatic differentiation	Any differentiable computation graph (the default)	One backward pass
Symbolic (SymPy, Mathematica)	Deriving closed-form gradients for papers	Compile time only

Quick reference: common derivatives

Function	$f(x)$	$f'(x)$	ML context
MSE loss	$(1/n) \sum (y_{\text{hat}} - y)^2$	$(2/n)(y_{\text{hat}} - y)$	Regression
Cross-entropy (binary)	$-(y \log(p) + (1-y) \log(1-p))$	$p - y$ (after sigmoid)	Binary classification
Cross-entropy (multi)	$-\log(p_{\text{true_class}})$	$p - \text{one_hot}(y)$ (after softmax)	Multi-class classification
Sigmoid	$1 / (1 + e^{-x})$	$\sigma(x) * (1 - \sigma(x))$	Output gates, binary output
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$1 - \tanh(x)^2$	Hidden activations (legacy)
ReLU	$\max(0, x)$	1 if $x > 0$, 0 if $x < 0$	Default hidden activation
Leaky ReLU	$\max(0.01x, x)$	1 if $x > 0$, 0.01 if $x < 0$	Avoiding dead neurons
GELU	$x * \Phi(x)$	$\Phi(x) + x * \phi(x)$	Transformers
Softmax_i	$e^{x_i} / \sum(e^{x_j})$	$s_i(1 - s_i)$ for $i=j$, $-s_i*s_j$ for $i \neq j$	Output layer (Jacobian)
Log-softmax	$x_i - \log(\sum(e^{x_j}))$	1 - softmax(x_i) for the i-th entry	Numerically stable CE
Linear layer	$y = Wx + b$	$dL/dW = dL/dy * x^T$, $dL/db = dL/dy$	Every layer
L2 regularization	$\lambda * \sum(w^2)$	$2 * \lambda * w$	Weight decay
L1 regularization	$\lambda * \sum(w)$	$\lambda * \text{sign}(w)$	Sparsity

Common mistakes

- Forgetting the $1/n$ factor in batch-averaged losses (MSE, cross-entropy). The gradient is scaled by batch size.
- Computing softmax gradient as a vector when it is actually a Jacobian matrix. For cross-entropy + softmax combined, the gradient simplifies to $(p - y)$, which avoids the full Jacobian.
- Applying the chain rule in the wrong order. Work backward from the loss: $dL/dW = dL/dy * dy/dW$.

- Using h that is too large ($h = 0.1$) or too small ($h = 1e-15$) for numerical derivatives. Stick to $h = 1e-7$ for float64.
- Forgetting that ReLU has undefined gradient at exactly $x = 0$. In practice, set it to 0 or 0.5.

Gradient checking recipe

```
For each parameter w:
    numeric_grad = (loss(w + h) - loss(w - h)) / (2h)
    auto_grad = backward pass value
    relative_error = |numeric - auto| / max(|numeric|, |auto|, 1e-8)
    assert relative_error < 1e-5
```

Relative error above $1e-3$ means something is wrong. Between $1e-5$ and $1e-3$, investigate.

Quiz

Pre-lesson check

1. What does the derivative of a function at a point tell you?
 - a) The value of the function at that point
 - b) The rate of change (slope) of the function at that point **(correct)**
 - c) The area under the function up to that point
 - d) The maximum value the function can reach

Explanation: The derivative $f'(x)$ measures how much the output changes per unit change in input. Geometrically, it is the slope of the tangent line at that point.
2. What is a gradient in the context of machine learning?
 - a) A measure of model accuracy
 - b) A vector of all partial derivatives that points in the direction of steepest ascent **(correct)**
 - c) The learning rate used during training
 - d) The difference between predicted and actual values

Explanation: The gradient collects every partial derivative into one vector. It points in the

direction that increases the function fastest. To minimize loss, you move opposite the gradient.

Post-lesson check

3. In gradient descent, what does the update rule ' $w = w - lr * dL/dw$ ' accomplish?
- a) It increases the loss to test model robustness
 - b) It adjusts each weight in the direction that reduces the loss, scaled by the learning rate **(correct)**
 - c) It resets the weight to its initial value minus the gradient
 - d) It normalizes the weight to have magnitude 1
Explanation: dL/dw tells you which direction increases the loss. Subtracting it (times the learning rate) moves the weight in the direction that decreases the loss. This is repeated for every weight in the model.
4. Why can't Newton's method (which uses the Hessian matrix) be directly applied to neural networks with millions of parameters?
- a) Newton's method only works for convex functions
 - b) The Hessian is an $N \times N$ matrix, requiring $O(N^2)$ storage and $O(N^3)$ computation per step, which is intractable for millions of parameters **(correct)**
 - c) Newton's method requires analytical derivatives which cannot be computed for neural networks
 - d) Newton's method converges too slowly for deep networks
Explanation: For $N=1$ million parameters, the Hessian has 1 trillion entries. Computing and inverting it is impossible. This is why we use first-order methods (SGD, Adam) that approximate second-order information cheaply.
5. What is the numerical (central difference) approximation for $f'(x)$?
- a) $f(x+h) / h$
 - b) $(f(x+h) - f(x)) / h$

c) $(f(x+h) - f(x-h)) / (2 \cdot h)$ **(correct)**

d) $f(x) \cdot h$

Explanation: The central difference $(f(x+h) - f(x-h)) / (2h)$ is more accurate than the forward difference because it averages the slope on both sides of x , canceling out the leading error term.

Chain Rule & Automatic Differentiation

The chain rule is the engine behind every neural network that learns.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lesson 04 (Derivatives & Gradients) **Time:** ~90 minutes

Learning Objectives

- Build a minimal autograd engine (Value class) that records operations and computes gradients via reverse-mode autodiff
- Implement forward and backward passes through a computation graph using topological sort
- Construct and train a multi-layer perceptron on XOR using only the from-scratch autograd engine
- Verify autodiff correctness using gradient checking against numerical finite differences

The Problem

You can compute derivatives of simple functions. But a neural network is not a simple function. It is hundreds of functions composed together: matrix multiply, add bias, apply activation, matrix multiply again, softmax, cross-entropy loss. The output is a function of a function of a function.

To train the network, you need the gradient of the loss with respect to every single weight. Doing this by hand is impossible for millions of parameters. Doing it numerically (finite differences) is too slow.

The chain rule gives you the math. Automatic differentiation gives you the algorithm. Together they let you compute exact gradients through arbitrary compositions of functions in time proportional to a single forward pass.

This is how PyTorch, TensorFlow, and JAX work. You will build a miniature version from scratch.

The Concept

The Chain Rule

If $y = f(g(x))$, the derivative of y with respect to x is:

$$dy/dx = dy/dg * dg/dx = f'(g(x)) * g'(x)$$

Multiply the derivatives along the chain. Each link contributes its local derivative.

Example: $y = \sin(x^2)$

$$\begin{array}{ll} g(x) = x^2 & g'(x) = 2x \\ f(g) = \sin(g) & f'(g) = \cos(g) \end{array}$$

$$dy/dx = \cos(x^2) * 2x$$

For deeper compositions, the chain extends:

$$y = f(g(h(x)))$$

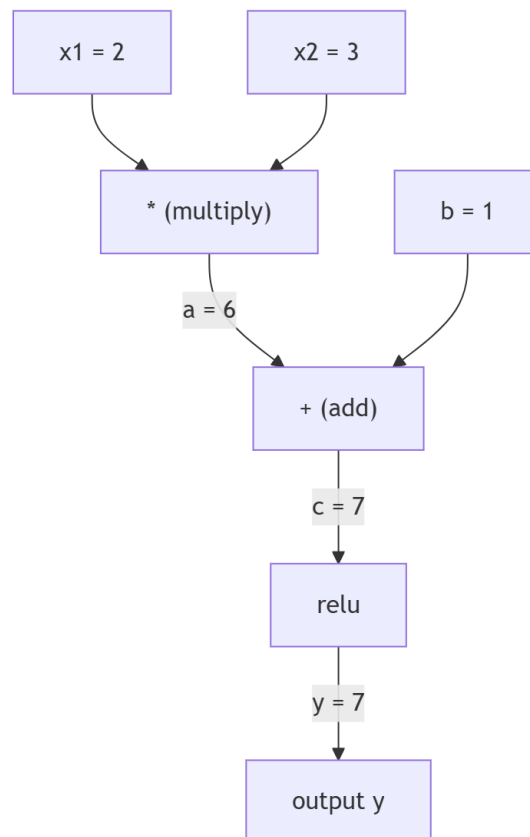
$$dy/dx = f'(g(h(x))) * g'(h(x)) * h'(x)$$

Every layer in a neural network is one link in this chain.

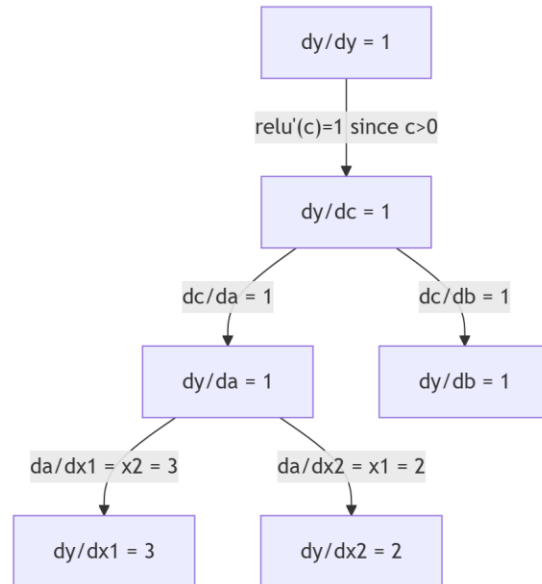
Computational Graphs

A computational graph makes the chain rule visual. Every operation becomes a node. Data flows forward through the graph. Gradients flow backward.

Forward pass (compute values):



Backward pass (compute gradients):



The backward pass applies the chain rule at every node, propagating gradients from output to inputs.

Forward Mode vs Reverse Mode

There are two ways to apply the chain rule through a graph.

Forward mode starts at the inputs and pushes derivatives forward. It computes $dx/dx = 1$ and propagates through each operation. Good when you have few inputs and many outputs.

Forward mode: seed $dx/dx = 1$, propagate forward

```
x = 2      (dx/dx = 1)
a = x^2    (da/dx = 2x = 4)
y = sin(a) (dy/dx = cos(a) * da/dx = cos(4) * 4 = -2.615)
```

Reverse mode starts at the output and pulls gradients backward. It computes $dy/dy = 1$ and propagates through each operation in reverse. Good when you have many inputs and few outputs.

Reverse mode: seed $dy/dy = 1$, propagate backward

```
y = sin(a) (dy/dy = 1)
a = x^2    (dy/da = cos(a) = cos(4) = -0.654)
x = 2      (dy/dx = dy/da * da/dx = -0.654 * 4 = -2.615)
```

Neural networks have millions of inputs (weights) and one output (loss). Reverse mode computes all gradients in one backward pass. This is why backpropagation uses reverse mode.

Mode	Seed	Direction	Best when
Forward	$dx_i/dx_i = 1$	Input to output	Few inputs, many outputs
Reverse	$dy/dy = 1$	Output to input	Many inputs, few outputs (neural nets)

Dual Numbers for Forward Mode

Forward mode can be implemented elegantly with dual numbers. A dual number has the form $a + b \cdot \epsilon$ where $\epsilon^2 = 0$.

```
Dual number: (value, derivative)

(2, 1) means: value is 2, derivative w.r.t. x is 1

Arithmetic rules:
(a, a') + (b, b') = (a+b, a'+b')
(a, a') * (b, b') = (a*b, a'*b + a*b')
sin(a, a')          = (sin(a), cos(a)*a')
```

Seed the input variable with derivative 1. The derivative propagates automatically through every operation.

Building an Autograd Engine

An autograd engine needs three things:

1. **Value wrapping.** Wrap every number in an object that stores its value and gradient.
2. **Graph recording.** Every operation records its inputs and the local gradient function.
3. **Backward pass.** Topological sort the graph, then walk it in reverse, applying the chain rule at each node.

This is exactly what PyTorch's autograd does. The `torch.Tensor` class wraps values, records operations when `requires_grad=True`, and computes gradients when you call `.backward()`.

How PyTorch Autograd Works Under the Hood

When you write PyTorch code:

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x + 1
y.backward()
print(x.grad)  # 7.0 = 2*x + 3 = 2*2 + 3
```

PyTorch internally:

1. Creates a Tensor node for x with `requires_grad=True`
2. Every operation (`**`, `*`, `+`) creates a new node and records the backward function
3. `y.backward()` triggers reverse-mode autodiff through the recorded graph
4. Each node's `grad_fn` computes local gradients and passes them to parent nodes
5. Gradients accumulate in `.grad` attributes via addition (not replacement)

The graph is dynamic (define-by-run). A new graph is built on every forward pass. This is why PyTorch supports control flow (if/else, loops) inside models.

Build It

Step 1: The Value class

```
class Value:
    def __init__(self, data, children=(), op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(children)
        self._op = op

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"
```

Every `Value` stores its numeric data, its gradient (initially zero), a backward function, and pointers to child nodes that produced it.

Step 2: Arithmetic operations with gradient tracking

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), '+')
    def _backward():
        self.grad += out.grad
        other.grad += out.grad
    out._backward = _backward
    return out

def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out

def relu(self):
```

```

out = Value(max(0, self.data), (self,), 'relu')
def _backward():
    self.grad += (1.0 if out.data > 0 else 0.0) * out.grad
out._backward = _backward
return out

```

Each operation creates a closure that knows how to compute local gradients and multiply by the upstream gradient (`out.grad`). The `+=` handles the case where a value is used in multiple operations.

Step 3: The backward pass

```

def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)

    self.grad = 1.0
    for v in reversed(topo):
        v._backward()

```

Topological sort ensures every node's gradient is fully computed before it propagates to its children. The seed gradient is 1.0 ($dy/dy = 1$).

Step 4: More operations for a complete engine

The basic Value class handles addition, multiplication, and relu. A real autograd engine needs more. Here are the operations you need to build neural networks:

```

def __neg__(self):
    return self * -1

def __sub__(self, other):
    return self + (-other)

def __radd__(self, other):
    return self + other

def __rmul__(self, other):
    return self * other

def __rsub__(self, other):
    return other + (-self)

def __pow__(self, n):

```

```

out = Value(self.data ** n, (self,), f'**{n}')
def _backward():
    self.grad += n * (self.data ** (n - 1)) * out.grad
out._backward = _backward
return out

def __truediv__(self, other):
    return self * (other ** -1) if isinstance(other, Value) else self * (Value(other) ** -1)

def exp(self):
    import math
    e = math.exp(self.data)
    out = Value(e, (self,), 'exp')
    def _backward():
        self.grad += e * out.grad
    out._backward = _backward
    return out

def log(self):
    import math
    out = Value(math.log(self.data), (self,), 'log')
    def _backward():
        self.grad += (1.0 / self.data) * out.grad
    out._backward = _backward
    return out

def tanh(self):
    import math
    t = math.tanh(self.data)
    out = Value(t, (self,), 'tanh')
    def _backward():
        self.grad += (1 - t ** 2) * out.grad
    out._backward = _backward
    return out

```

Why each operation matters:

Operation	Backward rule	Used in
<code>__sub__</code>	Reuses add + neg	Loss computation (pred - target)
<code>__pow__</code>	$n * x^{(n-1)}$	Polynomial activations, MSE (error ²)
<code>__truediv__</code>	Reuses mul + pow(-1)	Normalization, learning rate scaling
<code>exp</code>	$\exp(x) * \text{upstream}$	Softmax, log-likelihood
<code>log</code>	$(1/x) * \text{upstream}$	Cross-entropy loss, log probabilities

Operation	Backward rule	Used in
tanh	$(1 - \tanh^2) * \text{upstream}$	Classic activation function

The clever part: `__sub__` and `__truediv__` are defined in terms of existing operations. They get correct gradients for free because the chain rule composes through the underlying add/mul/pow operations.

Step 5: Mini MLP from scratch

With a complete Value class, you can build a neural network. No PyTorch. No NumPy. Just Values and the chain rule.

```
import random

class Neuron:
    def __init__(self, n_inputs):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(n_inputs)]
        self.b = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]

class Layer:
    def __init__(self, n_inputs, n_outputs):
        self.neurons = [Neuron(n_inputs) for _ in range(n_outputs)]

    def __call__(self, x):
        return [n(x) for n in self.neurons]

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, sizes):
        self.layers = [Layer(sizes[i], sizes[i+1]) for i in range(len(sizes)-1)]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x[0] if len(x) == 1 else x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]
```

A Neuron computes $\tanh(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b)$. A Layer is a list of neurons. An MLP stacks layers. Every weight is a Value, so calling `loss.backward()` propagates gradients to every parameter.

Training on XOR:

```
random.seed(42)
model = MLP([2, 4, 1]) # 2 inputs, 4 hidden neurons, 1 output

xs = [[0, 0], [0, 1], [1, 0], [1, 1]]
ys = [-1, 1, 1, -1] # XOR pattern (using -1/1 for tanh)

for step in range(100):
    preds = [model(x) for x in xs]
    loss = sum((p - y) ** 2 for p, y in zip(preds, ys))

    for p in model.parameters():
        p.grad = 0.0
    loss.backward()

    lr = 0.05
    for p in model.parameters():
        p.data -= lr * p.grad

    if step % 20 == 0:
        print(f"step {step:3d} loss = {loss.data:.4f}")

print("\nPredictions after training:")
for x, y in zip(xs, ys):
    print(f"input={x} target={y:2d} pred={model(x).data:6.3f}")
```

This is micrograd. A complete neural network training loop in pure Python with automatic differentiation. Every commercial deep learning framework does the same thing at massive scale.

Step 6: Gradient checking

How do you know your autodiff is correct? Compare it against numerical derivatives. This is gradient checking.

```
def gradient_check(build_expr, x_val, h=1e-7):
    x = Value(x_val)
    y = build_expr(x)
    y.backward()
    autodiff_grad = x.grad

    y_plus = build_expr(Value(x_val + h)).data
    y_minus = build_expr(Value(x_val - h)).data
    numerical_grad = (y_plus - y_minus) / (2 * h)

    diff = abs(autodiff_grad - numerical_grad)
    return autodiff_grad, numerical_grad, diff
```

Test it on a complex expression:

```
def expr(x):
    return (x ** 3 + x * 2 + 1).tanh()

ad, num, diff = gradient_check(expr, 0.5)
print(f"Autodiff: {ad:.8f}")
print(f"Numerical: {num:.8f}")
print(f"Difference: {diff:.2e}")
# Difference should be < 1e-5
```

Gradient checking is essential when implementing new operations. If your backward pass has a bug, the numerical check catches it. Every serious deep learning implementation runs gradient checks during development.

When to use gradient checking:

Situation	Do gradient check?
Adding a new operation to your autograd	Yes, always
Debugging a training loop that won't converge	Yes, check gradients first
Production training	No, too slow (2x forward passes per parameter)
Unit tests for autograd code	Yes, automate it

Step 7: Verify against manual calculation

```
x1 = Value(2.0)
x2 = Value(3.0)
a = x1 * x2          # a = 6.0
b = a + Value(1.0)   # b = 7.0
y = b.relu()         # y = 7.0

y.backward()

print(f"y = {y.data}")          # 7.0
print(f"dy/dx1 = {x1.grad}")    # 3.0 (= x2)
print(f"dy/dx2 = {x2.grad}")    # 2.0 (= x1)
```

Manual check: $y = \text{relu}(x_1 \cdot x_2 + 1)$. Since $x_1 \cdot x_2 + 1 = 7 > 0$, relu is identity. $dy/dx_1 = x_2 = 3$. $dy/dx_2 = x_1 = 2$. The engine matches.

Use It

Verify against PyTorch

```
import torch

x1 = torch.tensor(2.0, requires_grad=True)
x2 = torch.tensor(3.0, requires_grad=True)
a = x1 * x2
b = a + 1.0
y = torch.relu(b)
y.backward()

print(f"PyTorch dy/dx1 = {x1.grad.item()}") # 3.0
print(f"PyTorch dy/dx2 = {x2.grad.item()}") # 2.0
```

Same gradients. Your engine computes the same result as PyTorch because the math is the same: reverse-mode autodiff via the chain rule.

A more complex expression

```
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)
f = (a * b + c).relu() # relu(2*(-3) + 10) = relu(4) = 4

f.backward()
print(f"df/da = {a.grad}") # -3.0 (= b)
print(f"df/db = {b.grad}") # 2.0 (= a)
print(f"df/dc = {c.grad}") # 1.0
```

Exercises

1. Add `__pow__` to the `Value` class so you can compute x^n . Verify that $d/dx(x^3)$ at $x=2$ equals 12.0 .
2. Add `tanh` as an activation function. Verify that $\tanh'(0) = 1$ and $\tanh'(2) = 0.0707$ (approx).
3. Build a computation graph for a single neuron: $y = \text{relu}(w_1 \cdot x_1 + w_2 \cdot x_2 + b)$. Compute all five gradients and verify against PyTorch.
4. Implement forward-mode autodiff using dual numbers. Create a `Dual` class and verify it gives the same derivatives as your reverse-mode engine.

Key Terms

Term	What people say	What it actually means
Chain rule	“Multiply the derivatives”	The derivative of composed functions equals the product of each function’s local derivative, evaluated at the right point
Computational graph	“The network diagram”	A directed acyclic graph where nodes are operations and edges carry values (forward) or gradients (backward)
Forward mode	“Push derivatives forward”	Autodiff that propagates derivatives from inputs to outputs. One pass per input variable.
Reverse mode	“Backpropagation”	Autodiff that propagates gradients from outputs to inputs. One pass per output variable.
Autograd	“Automatic gradients”	A system that records operations on values, builds a graph, and computes exact gradients via the chain rule
Dual numbers	“Value plus derivative”	Numbers of the form $a + b \cdot \epsilon$ ($\epsilon^2 = 0$) that carry derivative information through arithmetic
Topological sort	“Dependency order”	Ordering graph nodes so every node comes after all its dependencies. Required for correct gradient propagation.
Gradient accumulation	“Add, don’t replace”	When a value feeds into multiple operations, its gradient is the sum of all incoming gradient contributions
Dynamic graph	“Define by run”	A computation graph rebuilt on every forward pass, allowing Python control flow inside models (PyTorch style)
Gradient checking	“Numerical verification”	Comparing autodiff gradients against numerical finite-difference gradients to verify correctness. Essential for debugging.
MLP	“Multi-layer perceptron”	A neural network with one or more hidden layers of neurons. Each neuron computes a weighted sum plus bias, then applies an activation function.
Neuron	“Weighted sum + activation”	The basic unit: $\text{output} = \text{activation}(w_1x_1 + w_2x_2 + \dots + b)$. The weights and bias are learnable parameters.

You are an expert in automatic differentiation and computational graph mechanics. You help engineers build, debug, and extend autograd systems.

When someone asks about gradients, backpropagation, or autodiff:

1. Draw the computational graph as ASCII. Label each node with its operation, forward value, and local gradient.
2. Walk the backward pass step by step. Show the chain rule multiplication at each node.
3. Identify common bugs:
 - Forgetting to zero gradients between backward passes (gradients accumulate by default)
 - Using in-place operations that break the graph
 - Detaching tensors from the graph unintentionally
 - Non-differentiable operations (argmax, integer indexing) silently returning zero gradients
4. When verifying gradients, compare against finite differences: $(f(x+h) - f(x-h)) / (2h)$ with $h = 1e-5$.

Debugging checklist for wrong gradients:

- Is `requires_grad=True` set on the right tensors?
- Are gradients being zeroed before each backward pass?
- Is any operation breaking the graph (`.item()`, `.numpy()`, `.detach()`)?
- Are there any in-place operations (`+=`, `.zero_()`) on tensors that need gradients?
- Is the loss scalar? `.backward()` only works on scalar outputs without a `gradient` argument.
- For custom autograd functions, does the backward return the right number of gradients (one per input)?

Key relationships to always check:

- $d/dx(x^n) = n * x^{(n-1)}$
- $d/dx(\text{relu}(x)) = 1$ if $x > 0$, 0 otherwise
- $d/dx(\text{sigmoid}(x)) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$
- $d/dx(\text{tanh}(x)) = 1 - \text{tanh}(x)^2$
- $d/dx(\text{softmax})$ produces a Jacobian matrix, not a simple vector
- For matrix multiply $Y = X @ W$, $dL/dX = dL/dY @ W^T$ and $dL/dW = X^T @ dL/dY$

Quiz

Pre-lesson check

1. What does the chain rule in calculus state?

- a) The derivative of a sum is the sum of derivatives
 - b) The derivative of composed functions is the product of their individual derivatives
(correct)
 - c) The integral of a product is the product of integrals
 - d) Functions can only be differentiated if they are linear
*Explanation: For $y = f(g(x))$, the chain rule says $dy/dx = f'(g(x)) \cdot g'(x)$. You multiply the derivatives at each link in the chain. This is the mathematical basis of backpropagation.**
2. What is a computational graph?
- a) A plot of loss vs training steps
 - b) A directed graph where nodes are operations and edges carry values forward or gradients backward **(correct)**
 - c) A diagram showing the architecture of a neural network
 - d) A graph database used for storing training data
Explanation: A computational graph represents the sequence of operations in a computation. Data flows forward through the graph, and gradients flow backward during backpropagation.

Post-lesson check

3. Why does PyTorch use reverse-mode autodiff (backpropagation) instead of forward-mode?
- a) Reverse mode is more numerically accurate
 - b) Reverse mode computes all gradients in one backward pass, while forward mode needs one pass per input variable – neural networks have millions of inputs but one loss output
(correct)
 - c) Reverse mode uses less memory
 - d) Forward mode cannot handle non-linear functions
Explanation: Neural networks have millions of weight inputs but produce a single scalar

loss. Reverse mode needs one backward pass total. Forward mode would need one pass per weight – millions of passes – making it impractical.

4. In the Value class autograd engine, why does the backward function use '+' instead of '=' for accumulating gradients?
 - a) To average the gradients across multiple samples
 - b) Because a value used in multiple operations receives gradient contributions from each one, which must be summed (**correct**)
 - c) To prevent numerical overflow in the gradients
 - d) Because Python does not support the '=' operator in closures
*Explanation: When a value feeds into multiple operations (e.g., x used in both xy and $x+z$), its total gradient is the sum of all incoming gradient contributions. Using '=' would overwrite earlier contributions.**
 5. What is gradient checking and when should you use it?
 - a) Checking that gradients are below a threshold to prevent exploding gradients
 - b) Comparing autodiff gradients against numerical finite-difference gradients to verify correctness of the backward pass (**correct**)
 - c) Checking that all parameters received non-zero gradients during training
 - d) Monitoring gradient magnitudes during training to detect vanishing gradients
Explanation: Gradient checking computes numerical derivatives via $(f(x+h) - f(x-h)) / 2h$ and compares them to autodiff gradients. Use it when adding new operations to your autograd engine or debugging training failures.
-

Probability and Distributions

Probability is the language AI uses to express uncertainty.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 **Time:** ~75 minutes

Learning Objectives

- Implement PMFs and PDFs from scratch for Bernoulli, categorical, Poisson, uniform, and normal distributions
- Compute expected value, variance, and use the Central Limit Theorem to explain why Gaussians dominate
- Build softmax and log-softmax functions with the numerical stability trick (subtract max logit)
- Calculate cross-entropy loss from logits and connect it to negative log-likelihood

The Problem

A classifier outputs $[0.03, 0.91, 0.06]$. A language model picks the next word from 50,000 candidates. A diffusion model generates images by sampling from learned distributions. All of these are probability in action.

Every prediction a model makes is a probability distribution. Every loss function measures how far the predicted distribution is from the true one. Every training step adjusts parameters to make one distribution look more like another. Without probability, you cannot read a single ML paper, debug a single model, or understand why your training loss is NaN.

The Concept

Events, Sample Spaces, and Probability

The sample space S is the set of all possible outcomes. An event is a subset of the sample space. Probability maps events to numbers between 0 and 1.

```
Coin flip:  
S = {H, T}  
P(H) = 0.5, P(T) = 0.5  
  
Single die roll:  
S = {1, 2, 3, 4, 5, 6}  
P(even) = P({2, 4, 6}) = 3/6 = 0.5
```

Three axioms define all of probability: 1. $P(A) \geq 0$ for any event A 2. $P(S) = 1$ (something always happens) 3. $P(A \text{ or } B) = P(A) + P(B)$ when A and B cannot both occur

Everything else (Bayes' theorem, expectations, distributions) follows from these three rules.

Conditional Probability and Independence

$P(A|B)$ is the probability of A given that B happened.

$$P(A|B) = P(A \text{ and } B) / P(B)$$

Example: deck of cards

$$\begin{aligned} P(\text{King} | \text{Face card}) &= P(\text{King and Face card}) / P(\text{Face card}) \\ &= (4/52) / (12/52) \\ &= 4/12 = 1/3 \end{aligned}$$

Two events are independent when knowing one tells you nothing about the other:

$$\begin{aligned} \text{Independent: } P(A|B) &= P(A) \\ \text{Equivalent to: } P(A \text{ and } B) &= P(A) * P(B) \end{aligned}$$

Coin flips are independent. Drawing cards without replacement is not.

Probability Mass Functions vs Probability Density Functions

Discrete random variables have a probability mass function (PMF). Each outcome has a specific probability that you can read off directly.

PMF: $P(X = k)$

Fair die:

$$P(X = 1) = 1/6$$

$$P(X = 2) = 1/6$$

...

$$P(X = 6) = 1/6$$

Sum of all probabilities = 1

Continuous random variables have a probability density function (PDF). The density at a single point is not a probability. Probability comes from integrating the density over an interval.

PDF: $f(x)$

$$P(a \leq X \leq b) = \text{integral of } f(x) \text{ from } a \text{ to } b$$

$f(x)$ can be greater than 1 (density, not probability)

$$\text{integral from } -\infty \text{ to } +\infty \text{ of } f(x) dx = 1$$

This distinction matters in ML. Classification outputs are PMFs (discrete choices). VAE latent spaces use PDFs (continuous).

Common Distributions

Bernoulli: one trial, two outcomes. Models binary classification.

$$\begin{aligned}P(X = 1) &= p \\P(X = 0) &= 1 - p \\ \text{Mean} &= p, \quad \text{Variance} = p(1-p)\end{aligned}$$

Categorical: one trial, k outcomes. Models multi-class classification (softmax output).

$$\begin{aligned}P(X = i) &= p_i, \quad \text{where sum of } p_i = 1 \\ \text{Example: } P(\text{cat}) &= 0.7, \quad P(\text{dog}) = 0.2, \quad P(\text{bird}) = 0.1\end{aligned}$$

Uniform: all outcomes equally likely. Used for random initialization.

$$\begin{aligned}\text{Discrete: } P(X = k) &= 1/n \text{ for } k \text{ in } \{1, \dots, n\} \\ \text{Continuous: } f(x) &= 1/(b-a) \text{ for } x \text{ in } [a, b]\end{aligned}$$

Normal (Gaussian): the bell curve. Parameterized by mean (μ) and variance (σ^2).

$$\begin{aligned}f(x) &= (1 / \sqrt{2\pi\sigma^2}) * \exp(-(x - \mu)^2 / (2\sigma^2)) \\ \text{Standard normal: } \mu &= 0, \quad \sigma = 1 \\ &68\% \text{ of data within } 1 \text{ sigma} \\ &95\% \text{ within } 2 \text{ sigma} \\ &99.7\% \text{ within } 3 \text{ sigma}\end{aligned}$$

Poisson: counts of rare events in a fixed interval. Models event rates.

$$\begin{aligned}P(X = k) &= (\lambda^k * e^{-\lambda}) / k! \\ \text{Mean} &= \lambda, \quad \text{Variance} = \lambda\end{aligned}$$

Expected Value and Variance

Expected value is the weighted average outcome.

$$\begin{aligned}\text{Discrete: } E[X] &= \text{sum of } x_i * P(X = x_i) \\ \text{Continuous: } E[X] &= \text{integral of } x * f(x) \, dx\end{aligned}$$

Variance measures spread around the mean.

$$\begin{aligned}\text{Var}(X) &= E[(X - E[X])^2] = E[X^2] - (E[X])^2 \\ \text{Standard deviation} &= \sqrt{\text{Var}(X)}\end{aligned}$$

In ML, expected value appears as the loss function (average loss over the data distribution). Variance tells you about model stability. High variance in gradients means noisy training.

Joint and Marginal Distributions

A joint distribution $P(X, Y)$ describes two random variables together.

Joint PMF example (X = weather, Y = umbrella):

	Y=0 (no umbrella)	Y=1 (umbrella)	Marginal P(X)
X=0 (sun)	0.40	0.10	$P(X=0) = 0.50$
X=1 (rain)	0.05	0.45	$P(X=1) = 0.50$
Marginal P(Y)	$P(Y=0) = 0.45$	$P(Y=1) = 0.55$	1.00

The marginal distribution sums out the other variable:

$$P(X = x) = \text{sum over all } y \text{ of } P(X = x, Y = y)$$

The row and column totals in the table above are the marginals.

Why the Normal Distribution Shows Up Everywhere

The Central Limit Theorem: the sum (or average) of many independent random variables converges to a normal distribution, regardless of the original distribution.

Roll 1 die: uniform distribution (flat)
Average of 2 dice: triangular (peaked)
Average of 30 dice: nearly perfect bell curve

This works for ANY starting distribution.

This is why: - Measurement errors are approximately normal (many small independent sources) - Weight initializations in neural networks use normal distributions - Gradient noise in SGD is approximately normal (sum of many sample gradients) - The normal distribution is the maximum entropy distribution for a given mean and variance

Log Probabilities

Raw probabilities cause numerical problems. Multiplying many small probabilities together quickly underflows to zero.

```
P(sentence) = P(word1) * P(word2) * ... * P(word_n)
             = 0.01 * 0.003 * 0.02 * ...
             -> 0.0 (underflow after ~30 terms)
```

Log probabilities fix this. Multiplications become additions.

```
log P(sentence) = log P(word1) + log P(word2) + ... + log P(word_n)
                = -4.6 + -5.8 + -3.9 + ...
                -> finite number (no underflow)
```

Rules: - $\log(a * b) = \log(a) + \log(b)$ - log probabilities are always ≤ 0 (since $0 < P \leq 1$) - More negative = less likely - Cross-entropy loss is the negative log probability of the correct class

Softmax as a Probability Distribution

Neural networks output raw scores (logits). Softmax converts them into a valid probability distribution.

```
softmax(z_i) = exp(z_i) / sum(exp(z_j) for all j)
```

Properties:

- All outputs are in $(0, 1)$
- All outputs sum to 1
- Preserves relative ordering of inputs
- $\exp()$ amplifies differences between logits

The softmax trick: subtract the max logit before exponentiating to prevent overflow.

```
z = [100, 101, 102]
exp(102) = overflow

z_shifted = z - max(z) = [-2, -1, 0]
exp(0) = 1 (safe)
```

Same result, no overflow.

Log-softmax combines softmax and log for numerical stability. PyTorch uses this internally for cross-entropy loss.

Sampling

Sampling means drawing random values from a distribution. In ML: - Dropout randomly samples which neurons to zero out - Data augmentation samples random transformations - Language models sample the next token from the predicted distribution - Diffusion models sample noise and progressively denoise

Sampling from arbitrary distributions requires techniques like inverse transform sampling, rejection sampling, or the reparameterization trick (used in VAEs).

Build It

Step 1: Probability basics

```
import math
import random

def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def combinations(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def conditional_probability(p_a_and_b, p_b):
    return p_a_and_b / p_b

p_king_given_face = conditional_probability(4/52, 12/52)
print(f"P(King | Face card) = {p_king_given_face:.4f}")
```

Step 2: PMF and PDF from scratch

```
def bernoulli_pmf(k, p):
    return p if k == 1 else (1 - p)

def categorical_pmf(k, probs):
    return probs[k]

def poisson_pmf(k, lam):
    return (lam ** k) * math.exp(-lam) / factorial(k)

def uniform_pdf(x, a, b):
    if a <= x <= b:
        return 1.0 / (b - a)
    return 0.0

def normal_pdf(x, mu, sigma):
    coeff = 1.0 / (sigma * math.sqrt(2 * math.pi))
    exponent = -0.5 * ((x - mu) / sigma) ** 2
    return coeff * math.exp(exponent)
```

Step 3: Expected value and variance

```
def expected_value(values, probabilities):
    return sum(v * p for v, p in zip(values, probabilities))
```

```
def variance(values, probabilities):
    mu = expected_value(values, probabilities)
    return sum(p * (v - mu) ** 2 for v, p in zip(values, probabilities))

die_values = [1, 2, 3, 4, 5, 6]
die_probs = [1/6] * 6
mu = expected_value(die_values, die_probs)
var = variance(die_values, die_probs)
print(f"Die: E[X] = {mu:.4f}, Var(X) = {var:.4f}, SD = {var**0.5:.4f}")
```

Step 4: Sampling from distributions

```
def sample_bernoulli(p, n=1):
    return [1 if random.random() < p else 0 for _ in range(n)]

def sample_categorical(probs, n=1):
    cumulative = []
    total = 0
    for p in probs:
        total += p
        cumulative.append(total)
    samples = []
    for _ in range(n):
        r = random.random()
        for i, c in enumerate(cumulative):
            if r <= c:
                samples.append(i)
                break
    return samples

def sample_normal_box_muller(mu, sigma, n=1):
    samples = []
    for _ in range(n):
        u1 = random.random()
        u2 = random.random()
        z = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
        samples.append(mu + sigma * z)
    return samples
```

Step 5: Softmax and log probabilities

```
def softmax(logits):
    max_logit = max(logits)
    shifted = [z - max_logit for z in logits]
    exps = [math.exp(z) for z in shifted]
    total = sum(exps)
    return [e / total for e in exps]

def log_softmax(logits):
    max_logit = max(logits)
```

```

shifted = [z - max_logit for z in logits]
log_sum_exp = max_logit + math.log(sum(math.exp(z) for z in shifted))
return [z - log_sum_exp for z in logits]

def cross_entropy_loss(logits, target_index):
    log_probs = log_softmax(logits)
    return -log_probs[target_index]

```

Step 6: Central Limit Theorem demonstration

```

def demonstrate_clt(dist_fn, n_samples, n_averages):
    averages = []
    for _ in range(n_averages):
        samples = [dist_fn() for _ in range(n_samples)]
        averages.append(sum(samples) / len(samples))
    return averages

```

Step 7: Visualization

```

import matplotlib.pyplot as plt

xs = [mu + sigma * (i - 500) / 100 for i in range(1001)]
ys = [normal_pdf(x, mu, sigma) for x, mu, sigma in ...]
plt.plot(xs, ys)

```

Full implementations with all visualizations are in `code/probability.py`.

Use It

With NumPy and SciPy, everything above is one-liners:

```

import numpy as np
from scipy import stats

normal = stats.norm(loc=0, scale=1)
samples = normal.rvs(size=10000)
print(f"Mean: {np.mean(samples):.4f}, Std: {np.std(samples):.4f}")
print(f"P(X < 1.96) = {normal.cdf(1.96):.4f}")

logits = np.array([2.0, 1.0, 0.1])
from scipy.special import softmax, log_softmax
probs = softmax(logits)
log_probs = log_softmax(logits)
print(f"Softmax: {probs}")
print(f"Log-softmax: {log_probs}")

```

You built these from scratch. Now you know what the library calls are doing.

Exercises

1. Implement inverse transform sampling for the exponential distribution. Verify by sampling 10,000 values and comparing the histogram to the true PDF.
2. Build a joint distribution table for two loaded dice. Compute the marginal distributions and check whether the dice are independent.
3. Compute the cross-entropy loss for a 5-class classifier that outputs logits $[2.0, 0.5, -1.0, 3.0, 0.1]$ when the correct class is index 3. Then verify your answer with PyTorch's `nn.CrossEntropyLoss`.
4. Write a function that takes a list of log probabilities and returns the most likely sequence, the total log probability, and the equivalent raw probability. Test it with a sentence of 50 words where each word has probability 0.01.

Key Terms

Term	What people say	What it actually means
Sample space	"All the possibilities"	The set S of every possible outcome of an experiment
PMF	"The probability function"	A function that gives the exact probability of each discrete outcome, summing to 1
PDF	"The probability curve"	A density function for continuous variables. Integrate it over an interval to get probability
Conditional probability	"Probability given something"	$P(A B) = P(A \text{ and } B) / P(B)$. The foundation of Bayesian thinking and Bayes' theorem
Independence	"They don't affect each other"	$P(A \text{ and } B) = P(A) * P(B)$. Knowing one event tells you nothing about the other
Expected value	"The average"	The probability-weighted sum of all outcomes. The loss function is an expected value
Variance	"How spread out"	The expected squared deviation from the mean. High variance = noisy, unstable estimates
Normal distribution	"The bell curve"	$f(x) = (1/\sqrt{2\pi\sigma^2}) * \exp(-(x-\mu)^2/(2*\sigma^2))$. Appears everywhere due to the CLT
Central Limit Theorem	"Averages become normal"	The mean of many independent samples converges to a normal distribution regardless of the source

Term	What people say	What it actually means
Joint distribution	“Two variables together”	$P(X, Y)$ describes the probability of every combination of X and Y outcomes
Marginal distribution	“Sum out the other variable”	$P(X) = \sum_y P(X, Y)$. Recovers one variable’s distribution from the joint
Log probability	“Log of the probability”	$\log P(x)$. Turns products into sums, preventing numerical underflow in long sequences
Softmax	“Turn scores into probabilities”	$\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$. Maps real-valued logits to a valid probability distribution
Cross-entropy	“The loss function”	$-\sum(p_{\text{true}} * \log(p_{\text{predicted}}))$. Measures how different two distributions are. Lower is better
Logits	“Raw model outputs”	Unnormalized scores before softmax. Named after the logistic function
Sampling	“Drawing random values”	Generating values according to a probability distribution. How models generate output

Probability Distribution Selection

How to pick the right distribution when modeling data, designing loss functions, or setting priors.

Decision Checklist

1. Is the outcome discrete (categories, counts) or continuous (measurements, scores)?
2. Is the outcome bounded (e.g., $[0, 1]$) or unbounded?
3. How many possible outcomes are there? Two? k ? Infinite?
4. Is the data symmetric or skewed?
5. Are events independent or correlated?
6. Are you modeling a rate, a count, a proportion, or a measurement?

Distribution decision tree

```

Is the variable discrete?
  Yes --> Only 2 outcomes? --> Bernoulli (p)
      |   k outcomes, one trial? --> Categorical (p1...pk)
      |   k outcomes, n trials? --> Multinomial (n, p1...pk)
      |   Count of successes in n trials? --> Binomial (n, p)
      |   Count of events per interval? --> Poisson (lambda)

```

		Count of trials until first success? --> Geometric (p)
		Count of trials until r successes? --> Negative Binomial (r, p)
No	-->	Symmetric, bell-shaped? --> Normal (μ, σ)
		Positive values, right-skewed? --> Log-normal or Exponential
		Bounded in $[0, 1]$? --> Beta (α, β)
		Positive values, flexible shape? --> Gamma (α, β)
		Time between events? --> Exponential (λ)
		Heavy tails needed? --> Student's t (ν) or Cauchy
		Multivariate, bell-shaped? --> Multivariate Normal
		On a simplex (sums to 1)? --> Dirichlet (α)

Mapping real-world ML scenarios to distributions

Scenario	Distribution	Parameters
Binary classification output	Bernoulli	$p = \text{sigmoid}(\text{logit})$
Multi-class classification output	Categorical	$p = \text{softmax}(\text{logits})$
Token prediction in language models	Categorical over vocab	p from softmax
Pixel intensity (normalized)	Beta or Uniform $[0, 1]$	Depends on image stats
Word count in a document	Poisson	$\lambda = \text{avg word count}$
Time between user requests	Exponential	$\lambda = \text{request rate}$
Measurement error	Normal	$\mu = 0, \sigma$ from data
Weight initialization	Normal or Uniform	Kaiming/Xavier rules
VAE latent space prior	Standard Normal	$\mu = 0, \sigma = 1$
Bayesian prior on proportions	Beta	α, β from belief
Bayesian prior on category weights	Dirichlet	α vector
Noise in regression targets	Normal	$\mu = 0, \sigma$ estimated
Outlier-robust regression	Student's t	low degrees of freedom
Duration/lifetime modeling	Weibull or Gamma	shape and scale
Topic distribution per document (LDA)	Dirichlet	$\alpha < 1$ for sparse

When distributions go wrong

- Using Normal when data has a hard lower bound (e.g., prices, distances). The normal assigns nonzero probability to negative values. Use log-normal or γ instead.
- Using Poisson when the variance differs from the mean. Poisson assumes mean = variance. If variance > mean, use negative binomial.
- Using Bernoulli for multi-class problems. Bernoulli is strictly binary. Use categorical for $k > 2$.
- Assuming independence when observations are correlated. Time series, spatial data, and grouped data violate independence. Use autoregressive or hierarchical models.

Common mistakes

- Confusing PDF values with probabilities. A PDF can exceed 1. Probability comes from integrating the PDF over an interval.
- Forgetting that softmax outputs are categorical probabilities, not independent Bernoulli probabilities. They sum to 1 by construction.
- Using a uniform prior when you have domain knowledge. Informative priors reduce variance without biasing the result if chosen well.
- Treating log-probabilities as probabilities. Log-probs are always negative (or zero). They do not sum to 1.

Quick reference: distribution properties

Distribution	Support	Mean	Variance	Key property
Bernoulli(p)	$\{0, 1\}$	p	$p(1-p)$	Simplest discrete
Binomial(n, p)	$\{0..n\}$	np	$np(1-p)$	Sum of n Bernoulli
Poisson(λ)	$\{0, 1, 2, \dots\}$	λ	λ	Mean = variance
Normal(μ, s^2)	$(-\infty, \infty)$	μ	s^2	Max entropy for given mean/var
Exponential(λ)	$[0, \infty)$	$1/\lambda$	$1/\lambda^2$	Memoryless
Beta(a, b)	$[0, 1]$	$a/(a+b)$	$ab/((a+b)^2(a+b+1))$	Conjugate to Binomial
Gamma(a, b)	$(0, \infty)$	a/b	a/b^2	Conjugate to Poisson
Dirichlet(α)	Simplex	α_i/sum	(see formula)	Conjugate to Categorical

Quiz

Pre-lesson check

1. What is the difference between a probability mass function (PMF) and a probability density function (PDF)?
 - a) PMFs are used for continuous variables, PDFs for discrete variables
 - b) PMFs give exact probabilities for discrete outcomes, while PDFs give densities for continuous variables that must be integrated over an interval to get probability **(correct)**
 - c) There is no difference; they are the same concept with different names
 - d) PMFs always sum to 0.5, PDFs integrate to 1
Explanation: For discrete variables, the PMF gives $P(X=k)$ directly. For continuous variables, the PDF $f(x)$ is a density – $P(a \leq X \leq b)$ requires integrating $f(x)$ from a to b . The density at a single point is not a probability.
2. What does the Central Limit Theorem state?
 - a) All data follows a normal distribution
 - b) The mean of many independent random samples converges to a normal distribution regardless of the source distribution **(correct)**
 - c) Large datasets always have low variance
 - d) The probability of rare events decreases as sample size increases
Explanation: The CLT says that the average of many independent random variables approaches a Gaussian, no matter what the original distribution looks like. This explains why the normal distribution appears everywhere.

Post-lesson check

3. Why does softmax subtract the maximum logit before exponentiating (the 'softmax trick')?
 - a) To make all probabilities equal

- b) To prevent numerical overflow from exponentiating large numbers while producing mathematically identical results **(correct)**
 - c) To normalize the logits to have mean zero
 - d) To speed up the computation by reducing the number of exponentiations
Explanation: $\exp(100)$ overflows to ∞ . Subtracting $\max(\text{logits})$ shifts all values so the largest is 0. $\exp(0)=1$ is safe. The subtraction cancels out in the normalization, giving identical probabilities.
4. Cross-entropy loss for classification simplifies to $-\log(q(\text{true_class}))$. What does this mean intuitively?
- a) Multiply the predicted probabilities by the true labels
 - b) Penalize the model based on how low its predicted probability is for the correct class – lower prediction means higher loss **(correct)**
 - c) Average the log probabilities across all classes
 - d) Compute the entropy of the true distribution
Explanation: If the model predicts 0.9 for the correct class, $\text{loss} = -\log(0.9) = 0.105$ (low). If it predicts 0.01, $\text{loss} = -\log(0.01) = 4.6$ (high). The loss punishes low confidence in the correct answer.
5. Why do language models work with log probabilities instead of raw probabilities?
- a) Log probabilities are easier to interpret visually
 - b) Multiplying many small probabilities causes numerical underflow to zero; log probabilities convert products to sums, avoiding this **(correct)**
 - c) Log probabilities are required by the transformer architecture
 - d) Raw probabilities cannot represent values less than 0.01
*Explanation: $P(\text{sentence}) = P(\text{word1}) P(\text{word2}) * \dots$ quickly underflows to 0.0 with float64. $\log P(\text{sentence}) = \log P(\text{word1}) + \log P(\text{word2}) + \dots$ stays in a finite range. Products become sums.**
-

Bayes' Theorem

Probability is about what you expect. Bayes' theorem is about what you learn.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lesson 06 (Probability Fundamentals) **Time:** ~75 minutes

Learning Objectives

- Apply Bayes' theorem to compute posterior probabilities from priors, likelihoods, and evidence
- Build a Naive Bayes text classifier from scratch with Laplace smoothing and log-space computation
- Compare MLE and MAP estimation and explain how MAP corresponds to L2 regularization
- Implement sequential Bayesian updating using Beta-Binomial conjugate priors for A/B testing

The Problem

A medical test is 99% accurate. You test positive. What are the chances you actually have the disease?

Most people say 99%. The real answer depends on how rare the disease is. If 1 in 10,000 people have it, a positive result only gives you about a 1% chance of being sick. The other 99% of positive results are false alarms from healthy people.

This is not a trick question. It is Bayes' theorem. Every spam filter, every medical diagnostic, every machine learning model that quantifies uncertainty uses this exact reasoning. You start with a belief. You see evidence. You update.

If you build ML systems without understanding this, you will misinterpret model outputs, set bad thresholds, and ship overconfident predictions.

The Concept

From joint probability to Bayes

You already know from Lesson 06 that conditional probability is:

$$P(A|B) = P(A \text{ and } B) / P(B)$$

And symmetrically:

$$P(B|A) = P(A \text{ and } B) / P(A)$$

Both expressions share the same numerator: $P(A \text{ and } B)$. Set them equal and rearrange:

$$P(A \text{ and } B) = P(A|B) * P(B) = P(B|A) * P(A)$$

Therefore:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

That is Bayes' theorem. Four quantities, one equation.

The four parts

Part	Name	What it means
$P(A B)$	Posterior	Your updated belief about A after seeing evidence B
$P(B A)$	Likelihood	How probable the evidence B is if A is true
$P(A)$	Prior	Your belief about A before seeing any evidence
$P(B)$	Evidence	Total probability of seeing B under all possibilities

The evidence term $P(B)$ acts as a normalizer. You can expand it using the law of total probability:

$$P(B) = P(B|A) * P(A) + P(B|\text{not } A) * P(\text{not } A)$$

Medical test example

A disease affects 1 in 10,000 people. The test is 99% accurate (catches 99% of sick people, gives false positives 1% of the time).

$$\begin{aligned}
 P(\text{sick}) &= 0.0001 && (\text{prior: disease is rare}) \\
 P(\text{positive}|\text{sick}) &= 0.99 && (\text{likelihood: test catches it}) \\
 P(\text{positive}|\text{healthy}) &= 0.01 && (\text{false positive rate}) \\
 \\
 P(\text{positive}) &= P(\text{positive}|\text{sick}) * P(\text{sick}) + P(\text{positive}|\text{healthy}) * P(\text{healthy}) \\
 &= 0.99 * 0.0001 + 0.01 * 0.9999 \\
 &= 0.000099 + 0.009999 \\
 &= 0.010098 \\
 \\
 P(\text{sick}|\text{positive}) &= P(\text{positive}|\text{sick}) * P(\text{sick}) / P(\text{positive}) \\
 &= 0.99 * 0.0001 / 0.010098 \\
 &= 0.0098 \\
 &= 0.98\%
 \end{aligned}$$

Less than 1%. The prior dominates. When a condition is rare, even accurate tests produce mostly false positives. This is why doctors order confirmation tests.

Spam filter example

You receive an email containing the word “lottery”. Is it spam?

```
P(spam) = 0.3 (30% of email is spam)
P("lottery"|spam) = 0.05 (5% of spam emails contain "lottery")
P("lottery"|not spam) = 0.001 (0.1% of legitimate emails contain "lottery")

P("lottery") = 0.05 * 0.3 + 0.001 * 0.7
              = 0.015 + 0.0007
              = 0.0157

P(spam|"lottery") = 0.05 * 0.3 / 0.0157
                  = 0.955
                  = 95.5%
```

One word shifts the probability from 30% to 95.5%. A real spam filter applies Bayes across hundreds of words simultaneously.

Naive Bayes: independence assumption

Naive Bayes extends this to multiple features by assuming all features are conditionally independent given the class:

```
P(class | feature_1, feature_2, ..., feature_n)
  = P(class) * P(feature_1|class) * P(feature_2|class) * ... * P(feature_n|class)
  / P(feature_1, feature_2, ..., feature_n)
```

The “naive” part is the independence assumption. In text, word occurrences are not independent (“New” and “York” are correlated). But the assumption works surprisingly well in practice because the classifier only needs to rank classes, not produce calibrated probabilities.

Since the denominator is the same for all classes, you can skip it and just compare numerators:

```
score(class) = P(class) * product of P(feature_i | class)
```

Pick the class with the highest score.

Maximum likelihood estimation (MLE)

How do you get $P(\text{feature}|\text{class})$ from training data? Count.

$$P(\text{"free"}|\text{spam}) = (\text{number of spam emails containing "free"}) / (\text{total spam emails})$$

This is MLE: choose the parameter values that make the observed data most likely. You are maximizing the likelihood function, which for discrete counts reduces to relative frequency.

Problem: if a word never appears in spam during training, MLE gives it probability zero. One unseen word kills the entire product. Fix this with Laplace smoothing:

$$P(\text{word}|\text{class}) = (\text{count}(\text{word}, \text{class}) + 1) / (\text{total_words_in_class} + \text{vocabulary_size})$$

Adding 1 to every count ensures no probability is ever zero.

Maximum a posteriori (MAP)

MLE asks: what parameters maximize $P(\text{data}|\text{parameters})$?

MAP asks: what parameters maximize $P(\text{parameters}|\text{data})$?

By Bayes' theorem:

$$P(\text{parameters}|\text{data}) \text{ proportional to } P(\text{data}|\text{parameters}) * P(\text{parameters})$$

MAP adds a prior over the parameters themselves. If you believe parameters should be small, you encode that as a prior that penalizes large values. This is identical to L2 regularization in ML. The “ridge” penalty in ridge regression is literally a Gaussian prior on the weights.

Estimation	Optimizes	ML equivalent
MLE	$P(\text{data} \text{params})$	Unregularized training
MAP	$P(\text{data} \text{params}) * P(\text{params})$	L2 / L1 regularization

Bayesian vs frequentist: the practical difference

Frequentists treat parameters as fixed unknowns. They ask: “If I repeated this experiment many times, what would happen?”

Bayesians treat parameters as distributions. They ask: “Given what I have observed, what do I believe about the parameters?”

For building ML systems, the practical difference:

Aspect	Frequentist	Bayesian
Output	Point estimate	Distribution over values
Uncertainty	Confidence intervals (about procedure)	Credible intervals (about parameter)
Small data	Can overfit	Prior acts as regularization
Computation	Usually faster	Often requires sampling (MCMC)

Most production ML is frequentist (SGD, point estimates). Bayesian methods shine when you need calibrated uncertainty (medical decisions, safety-critical systems) or when data is scarce (few-shot learning, cold start).

Why Bayesian thinking matters for ML

The connection is deeper than analogy:

Priors are regularization. A Gaussian prior on weights is L2 regularization. A Laplace prior is L1. Every time you add a regularization term, you are making a Bayesian statement about what parameter values you expect.

Posterior is uncertainty. A single predicted probability tells you nothing about how confident the model is in that estimate. Bayesian methods give you a distribution: "I think $P(\text{spam})$ is between 0.8 and 0.95."

Bayes updates are online learning. Today's posterior becomes tomorrow's prior. When your model sees new data, it updates its beliefs incrementally instead of retraining from scratch.

Model comparison is Bayesian. Bayesian information criterion (BIC), marginal likelihood, and Bayes factors all use Bayesian reasoning to choose between models without overfitting.

Build It

Step 1: Bayes theorem function

```
def bayes(prior, likelihood, false_positive_rate):
    evidence = likelihood * prior + false_positive_rate * (1 - prior)
    posterior = likelihood * prior / evidence
    return posterior

result = bayes(prior=0.0001, likelihood=0.99, false_positive_rate=0.01)
print(f"P(sick|positive) = {result:.4f}")
```


Step 2: Naive Bayes classifier

```
import math
from collections import defaultdict

class NaiveBayes:
    def __init__(self, smoothing=1.0):
        self.smoothing = smoothing
        self.class_counts = defaultdict(int)
        self.word_counts = defaultdict(lambda: defaultdict(int))
        self.class_word_totals = defaultdict(int)
        self.vocab = set()

    def train(self, documents, labels):
        for doc, label in zip(documents, labels):
            self.class_counts[label] += 1
            words = doc.lower().split()
            for word in words:
                self.word_counts[label][word] += 1
                self.class_word_totals[label] += 1
                self.vocab.add(word)

    def predict(self, document):
        words = document.lower().split()
        total_docs = sum(self.class_counts.values())
        vocab_size = len(self.vocab)
        best_class = None
        best_score = float("-inf")
        for cls in self.class_counts:
            score = math.log(self.class_counts[cls] / total_docs)
            for word in words:
                count = self.word_counts[cls].get(word, 0)
                total = self.class_word_totals[cls]
                score += math.log((count + self.smoothing) / (total + self.smoothing *
vocab_size))
            if score > best_score:
                best_score = score
                best_class = cls
        return best_class
```

Log probabilities prevent underflow. Multiplying many small probabilities produces numbers too tiny for floating point. Summing log-probabilities is numerically stable and mathematically equivalent.

Step 3: Train on spam data

```
train_docs = [
    "win free money now",
    "free lottery ticket winner",
    "claim your prize today free",
    "urgent offer free cash",
    "congratulations you won free",
```

```

    "meeting tomorrow at noon",
    "project update attached",
    "can we schedule a call",
    "quarterly report review",
    "lunch on thursday sounds good",
    "team standup notes attached",
    "please review the pull request",
]

train_labels = [
    "spam", "spam", "spam", "spam", "spam",
    "ham", "ham", "ham", "ham", "ham", "ham", "ham",
]

classifier = NaiveBayes()
classifier.train(train_docs, train_labels)

test_messages = [
    "free money waiting for you",
    "meeting rescheduled to friday",
    "you won a free prize",
    "please review the attached report",
]

for msg in test_messages:
    print(f" '{msg}' -> {classifier.predict(msg)}")

```

Step 4: Inspect the learned probabilities

```

def show_top_words(classifier, cls, n=5):
    vocab_size = len(classifier.vocab)
    total = classifier.class_word_totals[cls]
    probs = {}
    for word in classifier.vocab:
        count = classifier.word_counts[cls].get(word, 0)
        probs[word] = (count + classifier.smoothing) / (total + classifier.smoothing *
vocab_size)
    sorted_words = sorted(probs.items(), key=lambda x: x[1], reverse=True)
    for word, prob in sorted_words[:n]:
        print(f"    {word}: {prob:.4f}")

print("\nTop spam words:")
show_top_words(classifier, "spam")
print("\nTop ham words:")
show_top_words(classifier, "ham")

```

Use It

Scikit-learn ships production-ready naive Bayes implementations:

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(train_docs)
clf = MultinomialNB()
clf.fit(X_train, train_labels)

X_test = vectorizer.transform(test_messages)
predictions = clf.predict(X_test)
for msg, pred in zip(test_messages, predictions):
    print(f" {msg} -> {pred}")

```

Same algorithm. CountVectorizer handles tokenization and vocabulary building. MultinomialNB handles smoothing and log-probabilities internally. Your from-scratch version does the same thing in 40 lines.

Conjugate Priors

When the prior and posterior belong to the same family of distributions, the prior is called “conjugate.” This makes Bayesian updating algebraically clean – you get a closed-form posterior without numerical integration.

Likelihood	Conjugate Prior	Posterior	Example
Bernoulli	Beta(a, b)	Beta(a + successes, b + failures)	Coin flip bias estimation
Normal (known variance)	Normal(mu_0, sigma_0)	Normal(weighted mean, smaller variance)	Sensor calibration
Poisson	Gamma(a, b)	Gamma(a + sum of counts, b + n)	Modeling arrival rates
Multinomial	Dirichlet(α)	Dirichlet(α + counts)	Topic modeling, language models

Why this matters: without conjugate priors, you need Monte Carlo sampling or variational inference to approximate the posterior. With conjugate priors, you just update two numbers.

The Beta distribution is the most common conjugate prior in practice. Beta(a, b) represents your belief about a probability parameter. The mean is $a/(a+b)$. The larger $a+b$, the more concentrated (confident) the distribution.

Special cases of the Beta prior: - Beta(1, 1) = uniform. You have no opinion about the parameter. - Beta(10, 10) = peaked at 0.5. You strongly believe the parameter is near 0.5. - Beta(1, 10) = skewed toward 0. You believe the parameter is small.

The update rule is dead simple:

Prior: $\text{Beta}(a, b)$
Data: s successes, f failures
Posterior: $\text{Beta}(a + s, b + f)$

No integrals. No sampling. Just addition.

Sequential Bayesian Updating

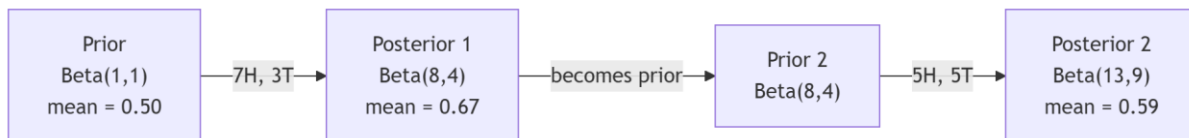
Bayesian inference is naturally sequential. Today's posterior becomes tomorrow's prior. This is how real systems learn incrementally without reprocessing all historical data.

Concrete example: estimating whether a coin is fair.

Day 1: No data yet. Start with $\text{Beta}(1, 1)$ – a uniform prior. You have no opinion. - Prior mean: 0.5 - Prior is flat across $[0, 1]$

Day 2: Observe 7 heads, 3 tails. Posterior = $\text{Beta}(1 + 7, 1 + 3) = \text{Beta}(8, 4)$ - Posterior mean: $8/12 = 0.667$ - Evidence suggests the coin is biased toward heads

Day 3: Observe 5 more heads, 5 more tails. Use yesterday's posterior as today's prior. Posterior = $\text{Beta}(8 + 5, 4 + 5) = \text{Beta}(13, 9)$ - Posterior mean: $13/22 = 0.591$ - The balanced new data pulled the estimate back toward 0.5



The order of observations does not matter. $\text{Beta}(1,1)$ updated with all 12 heads and 8 tails at once gives $\text{Beta}(13, 9)$ – the same result. Sequential updating and batch updating are mathematically equivalent. But sequential updating lets you make decisions at each step without storing raw data.

This is the foundation of online learning in production ML systems. Thompson sampling for bandits, incremental recommendation systems, and streaming anomaly detectors all use this pattern.

Connection to A/B Testing

A/B testing is Bayesian inference in disguise.

Setup: you are testing two button colors. Variant A (blue) and variant B (green). You want to know which one gets more clicks.

The Bayesian A/B test:

1. **Prior.** Start with $\text{Beta}(1, 1)$ for both variants. No prior preference.
2. **Data.** Variant A: 50 clicks out of 1000 views. Variant B: 65 clicks out of 1000 views.
3. **Posteriors.**

- A: $\text{Beta}(1 + 50, 1 + 950) = \text{Beta}(51, 951)$. Mean = 0.051
- B: $\text{Beta}(1 + 65, 1 + 935) = \text{Beta}(66, 936)$. Mean = 0.066

4. **Decision.** Compute $P(B > A)$ – the probability that B’s true conversion rate is higher than A’s.

Computing $P(B > A)$ analytically is hard. But Monte Carlo makes it trivial:

1. Draw 100,000 samples from $\text{Beta}(51, 951)$ -> samples_A
2. Draw 100,000 samples from $\text{Beta}(66, 936)$ -> samples_B
3. $P(B > A) = \text{fraction of samples where } B > A$

If $P(B > A) > 0.95$, you ship variant B. If it is between 0.05 and 0.95, you keep collecting data. If $P(B > A) < 0.05$, you ship variant A.

Advantages over frequentist A/B testing: - You get a direct probability statement: “there is a 97% chance B is better” - No p-value confusion. No “fail to reject the null hypothesis” hedging. - You can check results at any time without inflating false positive rates (no “peeking problem”) - You can incorporate prior knowledge (e.g., previous tests suggest conversion rates are usually 3-8%)

Aspect	Frequentist A/B	Bayesian A/B
Output	p-value	$P(B > A)$
Interpretation	“How surprising is this data if $A=B$?”	“How likely is B better than A?”
Early stopping	Inflates false positives	Safe at any point (given a well-chosen prior and correctly specified model)
Prior knowledge	Not used	Encoded as Beta prior
Decision rule	$p < 0.05$	$P(B > A) > \text{threshold}$

Exercises

1. **Multiple tests.** A patient tests positive twice on independent tests (both 99% accurate, disease prevalence 1 in 10,000). What is $P(\text{sick})$ after both tests? Use the posterior from the first test as the prior for the second.
2. **Smoothing impact.** Run the spam classifier with smoothing values of 0.01, 0.1, 1.0, and 10.0. How do the top word probabilities change? What happens with smoothing=0 and a word that appears only in ham?

3. **Add features.** Extend the NaiveBayes class to also use message length (short/long) as a feature alongside word counts. Estimate $P(\text{short}|\text{spam})$ and $P(\text{short}|\text{ham})$ from the training data and fold it into the prediction score.
4. **MAP by hand.** Given observed data (7 heads in 10 coin flips), compute the MAP estimate of the bias using a $\text{Beta}(2,2)$ prior. Compare it to the MLE estimate (7/10).

Key Terms

Term	What people say	What it actually means
Prior	"My initial guess"	$P(\text{hypothesis})$ before observing evidence. In ML: the regularization term.
Likelihood	"How well the data fits"	$P(\text{evidence} \text{hypothesis})$. How probable the observed data is under a specific hypothesis.
Posterior	"My updated belief"	$P(\text{hypothesis} \text{evidence})$. The prior multiplied by the likelihood, then normalized.
Evidence	"The normalizing constant"	$P(\text{data})$ across all hypotheses. Ensures the posterior sums to 1.
Naive Bayes	"That simple text classifier"	A classifier that assumes features are independent given the class. Works well despite the false assumption.
Laplace smoothing	"Add-one smoothing"	Adding a small count to every feature to prevent zero probabilities from unseen data.
MLE	"Just use the frequencies"	Choose parameters that maximize $P(\text{data} \text{parameters})$. No prior. Can overfit with small data.
MAP	"MLE with a prior"	Choose parameters that maximize $P(\text{data} \text{parameters}) * P(\text{parameters})$. Equivalent to regularized MLE.
Log-probability	"Work in log space"	Using $\log(P)$ instead of P to avoid floating-point underflow when multiplying many small numbers.
False positive	"A wrong alarm"	The test says positive, but the true state is negative. Drives the base rate fallacy.

You are a Bayesian reasoning tutor. Your job is to help users apply Bayes' theorem correctly to real-world problems.

When a user describes a scenario involving uncertain evidence, guide them through the full Bayesian calculation.

Structure your response as:

1. **Identify the hypothesis (H) and the evidence (E).** State exactly what H and E are in plain language. If the problem involves multiple hypotheses (H1, H2, ...), list them all. They must be mutually exclusive and exhaustive.
2. **State the prior P(H).** This is the probability of the hypothesis before seeing any evidence. Ask: "How common is this in the general population or dataset?" If no prior is given, prompt the user for one. The prior is where most mistakes happen.
3. **State the likelihood P(E|H).** This is how probable the evidence is if the hypothesis is true. Ask: "If H were true, how often would we observe E?"
4. **State P(E|not H).** This is the false positive rate or the probability of seeing the evidence when the hypothesis is false. Ask: "If H were false, how often would we still observe E?"
5. **Compute the evidence P(E).** Use the law of total probability: $P(E) = P(E|H) * P(H) + P(E|not H) * P(not H)$
6. **Apply Bayes' theorem.** $P(H|E) = P(E|H) * P(H) / P(E)$ Show the full calculation with numbers substituted.
7. **Interpret the result.** Explain what the posterior means in the context of the original problem. Compare the prior to the posterior to show how much the evidence shifted the belief.

Use this decision framework for common pitfalls:

Mistake	How to catch it
Base rate neglect	Is P(H) very small (< 0.01)? If so, even strong evidence may not overcome a rare prior.
Confusing P(E given H) with P(H given E)	These are different quantities. A test being 99% accurate does NOT mean a positive result means 99% chance of disease.
Forgetting to expand P(E)	P(E) must account for ALL ways E can occur, including false positives from not-H.
Not updating sequentially	When there are multiple pieces of evidence, use the posterior from the first update as the prior for the next update.

For multi-step updates (e.g., two positive tests): - First update: $P(H|E1) = P(E1|H) * P(H) / P(E1)$ - Second update: use $P(H|E1)$ as the new prior, then apply Bayes again with E2

For Naive Bayes classification: - Score each class: $\log P(\text{class}) + \sum(\log P(\text{feature}_i | \text{class}))$ - The class with the highest score wins - You can skip computing P(E) since it is the same for all classes

Avoid: - Giving the answer without showing the full calculation - Skipping the prior (it is the most important and most overlooked term) - Using percentages and fractions interchangeably without converting (pick one and stick with it) - Assuming independence of evidence without stating the assumption

Quiz

Pre-lesson check

1. In Bayes' theorem, what is the 'prior'?
 - a) The probability of the evidence
 - b) Your belief about a hypothesis before observing any evidence **(correct)**
 - c) The probability of the evidence given the hypothesis
 - d) The final updated probability after seeing evidence
Explanation: The prior $P(A)$ represents your initial belief about hypothesis A before seeing any data. It gets updated to the posterior $P(A|B)$ after observing evidence B .
2. A rare disease affects 1 in 10,000 people. A 99% accurate test returns positive. What is the approximate probability you have the disease?
 - a) 99%
 - b) About 1% **(correct)**
 - c) 50%
 - d) 10%
Explanation: Despite 99% test accuracy, the disease is so rare (0.01%) that most positive results come from healthy people (false positives). Bayes' theorem gives $P(\text{sick}|\text{positive}) \approx 0.98\%$, not 99%.

Post-lesson check

3. What is Laplace smoothing in a Naive Bayes classifier and why is it necessary?

- a) It normalizes feature values to have zero mean and unit variance
 - b) It adds a small count to every feature to prevent zero probabilities from unseen words, which would zero out the entire product **(correct)**
 - c) It smooths the decision boundary between classes
 - d) It reduces the dimensionality of the feature space
Explanation: Without smoothing, a word never seen in spam training data gets $P(\text{word}|\text{spam})=0$, making the entire product zero regardless of other strong spam indicators. Adding 1 to each count prevents this.
4. How does MAP (Maximum A Posteriori) estimation differ from MLE (Maximum Likelihood Estimation)?
- a) MAP uses a larger dataset than MLE
 - b) MAP incorporates a prior distribution over parameters, equivalent to adding regularization, while MLE only uses the likelihood **(correct)**
 - c) MAP is faster to compute than MLE
 - d) MLE always produces better results than MAP
*Explanation: MAP maximizes $P(\text{data}|\text{params}) P(\text{params})$ while MLE maximizes $P(\text{data}|\text{params})$ alone. The prior $P(\text{params})$ acts as regularization – a Gaussian prior equals L2 regularization, a Laplace prior equals L1.**
5. In sequential Bayesian updating, you start with Beta(1,1) and observe 7 heads and 3 tails. What is the posterior?
- a) Beta(7, 3)
 - b) Beta(8, 4) **(correct)**
 - c) Beta(1.7, 1.3)
 - d) Normal(0.7, 0.1)
Explanation: The Beta-Binomial conjugate update adds successes to the first parameter and failures to the second: $\text{Beta}(1+7, 1+3) = \text{Beta}(8, 4)$. The posterior mean is $8/12 = 0.667$.

Optimization

Training a neural network is nothing more than finding the bottom of a valley.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 04-05 (Derivatives, Gradients) **Time:** ~75 minutes

Learning Objectives

- Implement vanilla gradient descent, SGD with momentum, and Adam from scratch
- Compare optimizer convergence on the Rosenbrock function and explain why Adam adapts per-weight learning rates
- Distinguish convex from non-convex loss landscapes and explain the role of saddle points in high dimensions
- Configure learning rate schedules (step decay, cosine annealing, warmup) for training stability

The Problem

You have a loss function. It tells you how wrong your model is. You have gradients. They tell you which direction makes the loss worse. Now you need a strategy for walking downhill.

The naive approach is simple: move opposite the gradient. Scale the step by some number called the learning rate. Repeat. This is gradient descent, and it works. But “works” has caveats. Too large a learning rate and you overshoot the valley entirely, bouncing between walls. Too small and you crawl toward the answer over thousands of unnecessary steps. Hit a saddle point and you stop moving even though you have not found a minimum.

Every optimizer in deep learning is an answer to the same question: how do you get to the bottom of the valley faster and more reliably?

The Concept

What optimization means

Optimization is finding the input values that minimize (or maximize) a function. In machine learning, the function is the loss. The inputs are the model’s weights. Training is optimization.

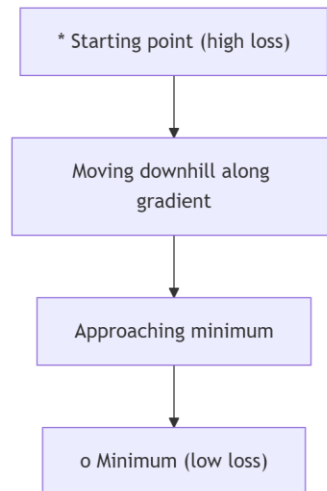
```
minimize  $L(w)$  where:  
   $L$  = loss function  
   $w$  = model weights (could be millions of parameters)
```

Gradient descent (vanilla)

The simplest optimizer. Compute the gradient of the loss with respect to every weight. Move each weight in the opposite direction of its gradient. Scale the step by the learning rate.

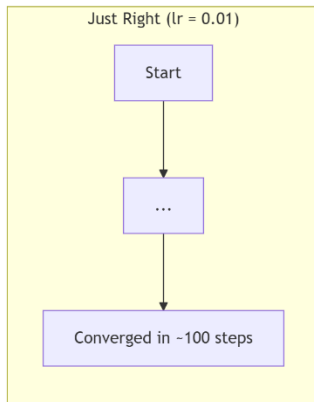
```
w = w - lr * gradient
```

That is the entire algorithm. One line.



Learning rate: the most important hyperparameter

The learning rate controls step size. It determines everything about convergence.



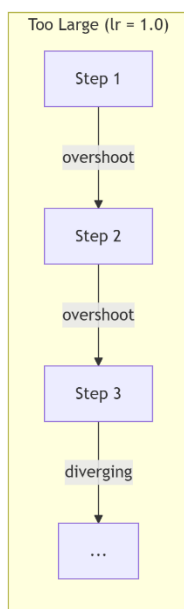
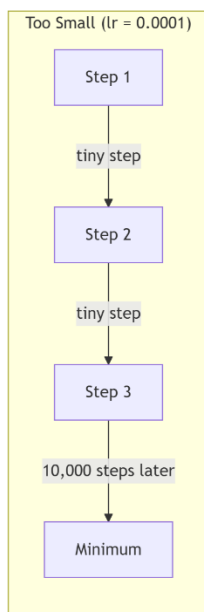
There is no formula for the right learning rate. You find it by experiment. Common starting points: 0.001 for Adam, 0.01 for SGD with momentum.

SGD vs batch vs mini-batch

Vanilla gradient descent computes the gradient over the entire dataset before taking one step. This is called batch gradient descent. It is stable but slow.

Stochastic gradient descent (SGD) computes the gradient on a single random sample and steps immediately. It is noisy but fast.

Mini-batch gradient descent splits the difference. Compute the gradient over a small batch (32, 64, 128, 256 samples), then step. This is what everyone actually uses.



Variant	Batch size	Gradient quality	Speed per step	Noise
Batch GD	Entire dataset	Exact	Slow	None
SGD	1 sample	Very noisy	Fast	High
Mini-batch	32-256	Good estimate	Balanced	Moderate

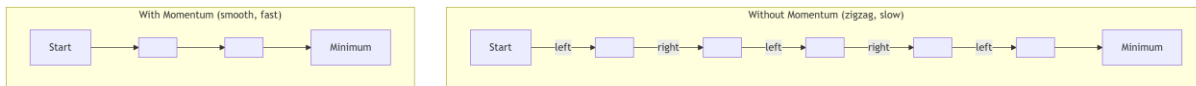
The noise in SGD and mini-batch is not a bug. It helps escape shallow local minima and saddle points.

Momentum: the ball rolling downhill

Vanilla gradient descent only looks at the current gradient. If the gradient zigzags (common in narrow valleys), progress is slow. Momentum fixes this by accumulating past gradients into a velocity term.

```
v = beta * v + gradient
w = w - lr * v
```

The analogy: a ball rolling downhill. It does not stop and restart at every bump. It builds speed in consistent directions and dampens oscillations.



β (typically 0.9) controls how much history to keep. Higher β means more momentum, smoother paths, but slower response to direction changes.

Adam: adaptive learning rates

Different weights need different learning rates. A weight that rarely gets large gradients should take bigger steps when it finally does. A weight that gets huge gradients constantly should take smaller steps.

Adam (Adaptive Moment Estimation) tracks two things per weight:

1. First moment (m): running average of gradients (like momentum)
2. Second moment (v): running average of squared gradients (gradient magnitude)

```
m = beta1 * m + (1 - beta1) * gradient
v = beta2 * v + (1 - beta2) * gradient^2

m_hat = m / (1 - beta1^t)    bias correction
v_hat = v / (1 - beta2^t)    bias correction

w = w - lr * m_hat / (sqrt(v_hat) + epsilon)
```

The division by $\sqrt{v_hat}$ is the key insight. Weights with large gradients get divided by a large number (small effective step). Weights with small gradients get divided by a small number (large effective step). Each weight gets its own adaptive learning rate.

Default hyperparameters: $lr=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$. These defaults work well for most problems.

Learning rate schedules

A fixed learning rate is a compromise. Early in training, you want large steps to make fast progress. Late in training, you want small steps to fine-tune near the minimum.

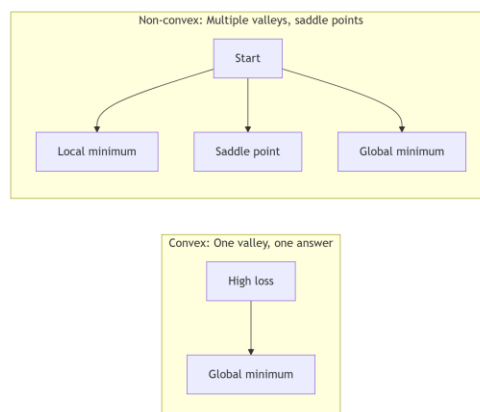
Common schedules:

Schedule	Formula	Use case
Step decay	$lr = lr * \text{factor every } N \text{ epochs}$	Simple, manual control
Exponential decay	$lr = lr_0 * \text{decay}^t$	Smooth reduction
Cosine annealing	$lr = lr_min + 0.5 * (lr_max - lr_min) * (1 + \cos(\pi * t / T))$	Transformers, modern training
Warmup + decay	Linear ramp up, then decay	Large models, prevents early instability

Convex vs non-convex

A convex function has one minimum. Gradient descent always finds it. A quadratic like $f(x) = x^2$ is convex.

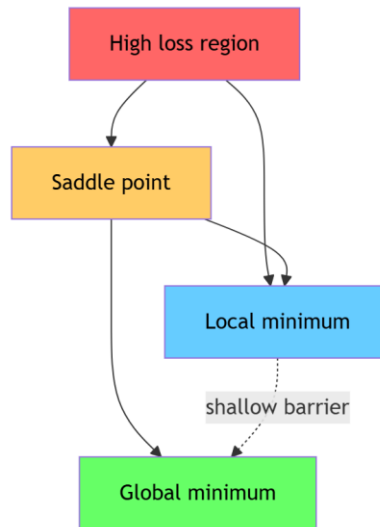
Neural network loss functions are non-convex. They have many local minima, saddle points, and flat regions.



In practice, local minima in high-dimensional neural networks are rarely a problem. Most local minima have loss values close to the global minimum. Saddle points (flat in some directions, curved in others) are the real obstacle. Momentum and noise from mini-batches help escape them.

Loss landscape visualization

The loss is a function of all weights. For a model with 1 million weights, the loss landscape lives in 1,000,001-dimensional space. We visualize it by picking two random directions in weight space and plotting the loss along those directions, producing a 2D surface.



Sharp minima generalize poorly. Flat minima generalize well. This is one reason SGD with momentum often outperforms Adam on final test accuracy: its noise prevents settling into sharp minima.

Build It

Step 1: Define a test function

The Rosenbrock function is a classic optimization benchmark. Its minimum is at (1, 1) inside a narrow curved valley that is easy to find but hard to follow.

```
f(x, y) = (1 - x)^2 + 100 * (y - x^2)^2

def rosenbrock(params):
    x, y = params
    return (1 - x) ** 2 + 100 * (y - x ** 2) ** 2

def rosenbrock_gradient(params):
    x, y = params
    df_dx = -2 * (1 - x) + 200 * (y - x ** 2) * (-2 * x)
    df_dy = 200 * (y - x ** 2)
    return [df_dx, df_dy]
```


Step 2: Vanilla gradient descent

```
class GradientDescent:
    def __init__(self, lr=0.001):
        self.lr = lr

    def step(self, params, grads):
        return [p - self.lr * g for p, g in zip(params, grads)]
```

Step 3: SGD with momentum

```
class SGDMomentum:
    def __init__(self, lr=0.001, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.velocity = None

    def step(self, params, grads):
        if self.velocity is None:
            self.velocity = [0.0] * len(params)
        self.velocity = [
            self.momentum * v + g
            for v, g in zip(self.velocity, grads)
        ]
        return [p - self.lr * v for p, v in zip(params, self.velocity)]
```

Step 4: Adam

```
class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.m = None
        self.v = None
        self.t = 0

    def step(self, params, grads):
        if self.m is None:
            self.m = [0.0] * len(params)
            self.v = [0.0] * len(params)

        self.t += 1

        self.m = [
            self.beta1 * m + (1 - self.beta1) * g
            for m, g in zip(self.m, grads)
        ]
        self.v = [
            self.beta2 * v + (1 - self.beta2) * g ** 2
```

```

        for v, g in zip(self.v, grads)
    ]

    m_hat = [m / (1 - self.beta1 ** self.t) for m in self.m]
    v_hat = [v / (1 - self.beta2 ** self.t) for v in self.v]

    return [
        p - self.lr * mh / (vh ** 0.5 + self.epsilon)
        for p, mh, vh in zip(params, m_hat, v_hat)
    ]

```

Step 5: Run and compare

```

def optimize(optimizer, func, grad_func, start, steps=5000):
    params = list(start)
    history = [params[:]]
    for _ in range(steps):
        grads = grad_func(params)
        params = optimizer.step(params, grads)
        history.append(params[:])
    return history

start = [-1.0, 1.0]

gd_history = optimize(GradientDescent(lr=0.0005), rosenbrock, rosenbrock_gradient, start)
sgd_history = optimize(SGDMomentum(lr=0.0001, momentum=0.9), rosenbrock, rosenbrock_gradient, start)
adam_history = optimize(Adam(lr=0.01), rosenbrock, rosenbrock_gradient, start)

for name, history in [("GD", gd_history), ("SGD+M", sgd_history), ("Adam", adam_history)]:
    final = history[-1]
    loss = rosenbrock(final)
    print(f"{name:6s} -> x={final[0]:.6f}, y={final[1]:.6f}, loss={loss:.8f}")

```

Expected output: Adam converges fastest. SGD with momentum follows a smoother path. Vanilla GD makes slow progress along the narrow valley.

Use It

In practice, use PyTorch or JAX optimizers. They handle parameter groups, weight decay, gradient clipping, and GPU acceleration.

```

import torch

model = torch.nn.Linear(784, 10)

sgd = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
adam = torch.optim.Adam(model.parameters(), lr=0.001)
adamw = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)

```

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(adam, T_max=100)
```

Rules of thumb:

- Start with Adam (lr=0.001). It works for most problems without tuning.
- Switch to SGD with momentum (lr=0.01, momentum=0.9) when you need the best final accuracy and can afford more tuning.
- Use AdamW (Adam with decoupled weight decay) for transformers.
- Always use a learning rate schedule for training runs longer than a few epochs.
- If training is unstable, reduce the learning rate. If training is too slow, increase it.

Exercises

1. **Learning rate sweep.** Run vanilla gradient descent on the Rosenbrock function with learning rates [0.0001, 0.0005, 0.001, 0.005, 0.01]. Plot or print the final loss after 5000 steps for each. Find the largest learning rate that still converges.
2. **Momentum comparison.** Run SGD with momentum values [0.0, 0.5, 0.9, 0.99] on the Rosenbrock function. Track the loss at every step. Which momentum value converges fastest? Which overshoots?
3. **Saddle point escape.** Define the function $f(x, y) = x^2 - y^2$ (a saddle point at the origin). Start at (0.01, 0.01). Compare how vanilla GD, SGD with momentum, and Adam behave. Which escapes the saddle point?
4. **Implement learning rate decay.** Add an exponential decay schedule to the GradientDescent class: $lr = lr_0 * 0.999^{step}$. Compare convergence with and without decay on the Rosenbrock function.

Key Terms

Term	What people say	What it actually means
Gradient descent	“Go downhill”	Update weights by subtracting the gradient scaled by the learning rate. The most basic optimizer.
Learning rate	“Step size”	A scalar that controls how far each update moves the weights. Too large causes divergence. Too small wastes compute.
Momentum	“Keep rolling”	Accumulate past gradients into a velocity vector. Dampens oscillations and accelerates movement through consistent directions.

Term	What people say	What it actually means
SGD	“Random sampling”	Stochastic gradient descent. Compute gradient on a random subset instead of the full dataset. Almost always means mini-batch SGD in practice.
Mini-batch	“A chunk of data”	A small subset of training data (32-256 samples) used to estimate the gradient. Balances speed and gradient accuracy.
Adam	“The default optimizer”	Adaptive Moment Estimation. Tracks per-weight running averages of gradients and squared gradients to give each weight its own learning rate.
Bias correction	“Fix the cold start”	Adam’s first and second moments are initialized to zero. Bias correction divides by $(1 - \beta^t)$ to compensate during early steps.
Learning rate schedule	“Change lr over time”	A function that adjusts the learning rate during training. Large steps early, small steps late.
Convex function	“One valley”	A function where any local minimum is the global minimum. Gradient descent always finds it. Neural network losses are not convex.
Saddle point	“Flat but not a minimum”	A point where the gradient is zero but it is a minimum in some directions and a maximum in others. Common in high dimensions.
Loss landscape	“The terrain”	The loss function plotted over weight space. Visualized by slicing along two random directions.
Convergence	“Getting there”	The optimizer has reached a point where further steps do not meaningfully reduce the loss.

You are an optimization advisor for machine learning practitioners. Your job is to recommend the right optimizer, learning rate, and schedule for a given training scenario.

When a user describes their problem, ask clarifying questions if needed, then recommend a specific optimizer configuration. Structure your response as:

1. Recommended optimizer and why
2. Starting hyperparameters (learning rate, momentum, betas, weight decay)
3. Learning rate schedule
4. Warning signs to watch for during training
5. When to switch to a different optimizer

Use this decision framework:

First project or prototype: - Use Adam with $\text{lr}=0.001$. Do not tune anything else until the model trains.

Training a transformer (GPT, BERT, ViT, any attention-based model): - Use AdamW with $\text{lr}=1\text{e-}4$ to $3\text{e-}4$, $\text{weight_decay}=0.01$ to 0.1 . - Use linear warmup for 5-10% of total steps, then cosine decay to 0. - Gradient clipping at $\text{max_norm}=1.0$.

Training a CNN for image classification: - Start with SGD, $\text{lr}=0.1$, $\text{momentum}=0.9$, $\text{weight_decay}=1\text{e-}4$. - Use step decay (divide lr by 10 at epochs 30, 60, 90 for a 100-epoch run). - SGD with momentum often beats Adam on final test accuracy for CNNs.

Fine-tuning a pretrained model: - Use AdamW with $\text{lr}=1\text{e-}5$ to $5\text{e-}5$ (10x to 100x smaller than pretraining lr). - Short warmup (100-500 steps), then linear or cosine decay. - Freeze early layers if the dataset is small.

Training a GAN: - Use Adam with $\text{lr}=1\text{e-}4$ to $2\text{e-}4$, $\text{beta1}=0.0$ (not the default 0.9), $\text{beta2}=0.9$. - Lower beta1 reduces momentum, which helps with GAN instability. - Use separate optimizers for generator and discriminator.

Reinforcement learning: - Use Adam with $\text{lr}=3\text{e-}4$. - Gradient clipping is critical. Use $\text{max_norm}=0.5$. - Learning rate schedules are less common; fixed lr often works.

Diagnosing training problems:

Loss is NaN or exploding: - Reduce learning rate by 10x. - Add gradient clipping ($\text{max_norm}=1.0$). - Check for numerical issues in the data (∞ , nan values).

Loss plateaus early: - Increase learning rate. - Check if the model has enough capacity. - Verify the data pipeline is not feeding the same batch repeatedly.

Loss is noisy but trending down: - This is normal for SGD and mini-batch training. - Increase batch size to reduce noise if needed. - Do not reduce learning rate too early.

Training loss drops but validation loss rises (overfitting): - Add weight decay (L2 regularization). - Use dropout, data augmentation, or reduce model size. - This is not an optimizer problem.

Adam converges fast but final accuracy is lower than expected: - Switch to SGD with momentum for the final training run. - Adam finds sharp minima; SGD with momentum finds flatter minima that generalize better. - Use a cosine annealing schedule with SGD.

Avoid: - Recommending grid search over optimizers. Pick one based on the architecture and problem type. - Suggesting learning rates without specifying the optimizer. $\text{lr}=0.1$ for SGD is normal; $\text{lr}=0.1$ for Adam will diverge immediately. - Ignoring weight decay. It is not optional for transformers and large models. - Treating optimizer choice as permanent. Start with Adam to validate the pipeline, then switch to SGD+momentum if final accuracy matters.

Quiz

Pre-lesson check

1. What does 'optimization' mean in the context of training a neural network?

- a) Making the code run faster
- b) Finding the model weights that minimize the loss function **(correct)**
- c) Reducing the number of parameters in the model
- d) Choosing the best hardware for training

Explanation: Training a neural network IS optimization. The loss function measures how wrong the model is, and optimization finds the weight values that make it as small as possible.

2. What happens if the learning rate is too large during gradient descent?

- a) Training converges faster to the global minimum
- b) The optimizer overshoots the minimum and the loss diverges (bounces or increases) **(correct)**
- c) The gradients become zero
- d) The model automatically reduces the learning rate

Explanation: A learning rate that is too large causes each step to overshoot the valley, potentially bouncing between walls or diverging entirely. The loss increases instead of decreasing.

Post-lesson check

3. How does Adam differ from vanilla gradient descent?

- a) Adam uses a fixed learning rate while GD uses adaptive rates
- b) Adam tracks running averages of gradients and squared gradients to give each weight its own adaptive learning rate **(correct)**

- c) Adam computes exact second derivatives while GD uses first derivatives
 - d) Adam processes the full dataset per step while GD uses mini-batches
Explanation: Adam maintains per-weight first moment (gradient direction) and second moment (gradient magnitude) estimates. Dividing by $\sqrt{\text{second moment}}$ gives small steps for weights with large gradients and large steps for weights with small gradients.
4. Why is the noise in mini-batch SGD considered beneficial rather than just a nuisance?
- a) Noise reduces memory usage during training
 - b) Noise helps the optimizer escape shallow local minima and saddle points that a noiseless optimizer would get stuck in (**correct**)
 - c) Noise is not beneficial; it always slows convergence
 - d) Noise makes the loss function convex
Explanation: The stochastic noise from random mini-batches provides random perturbations that can push the optimizer out of shallow local minima and saddle points, leading to better generalization.
5. What does a cosine annealing learning rate schedule do?
- a) Increases the learning rate throughout training following a cosine curve
 - b) Starts with a high learning rate and smoothly decreases it following a cosine curve, with optional warmup (**correct**)
 - c) Alternates the learning rate between two fixed values
 - d) Sets the learning rate to the cosine of the current epoch number
Explanation: Cosine annealing smoothly reduces the learning rate from lr_max to lr_min following a half-cosine curve. This provides large steps early for fast progress and small steps late for fine convergence.
-

Information Theory

Information theory measures surprise. Loss functions are built on it.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lesson 06 (Probability) **Time:** ~60 minutes

Learning Objectives

- Compute entropy, cross-entropy, and KL divergence from scratch and explain their relationship
- Derive why minimizing cross-entropy loss is equivalent to maximizing log-likelihood
- Calculate mutual information between features and a target to rank feature importance
- Explain perplexity as the effective vocabulary size a language model chooses from

The Problem

You call `CrossEntropyLoss()` in every classification model you train. You see “perplexity” in every language model paper. You read about KL divergence in VAEs, distillation, and RLHF. These are not disconnected concepts. They are all the same idea wearing different hats.

Information theory gives you the language to reason about uncertainty, compression, and prediction. Claude Shannon invented it in 1948 to solve communication problems. Turns out, training a neural network is a communication problem: the model is trying to transmit the correct label through a noisy channel of learned weights.

This lesson builds every formula from scratch so you see where they come from and why they work.

The Concept

Information Content (Surprise)

When something unlikely happens, it carries more information. A coin landing heads? Not surprising. A lottery win? Very surprising.

The information content of an event with probability p is:

$$I(x) = -\log(p(x))$$

Using log base 2 gives you bits. Using natural log gives you nats. Same idea, different units.

Event	Probability	Surprise (bits)
Fair coin heads	0.5	1.0
Rolling a 6	0.167	2.58
1-in-1000 event	0.001	9.97
Certain event	1.0	0.0

Certain events carry zero information. You already knew they would happen.

Entropy (Average Surprise)

Entropy is the expected surprise across all possible outcomes of a distribution.

$$H(P) = -\sum (p(x) * \log_2(p(x))) \text{ for all } x$$

A fair coin has maximum entropy for a binary variable: 1 bit. A biased coin (99% heads) has low entropy: 0.08 bits. You already know what will happen, so each flip tells you almost nothing.

$$\begin{aligned} \text{Fair coin: } H &= -(0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)) = 1.0 \text{ bit} \\ \text{Biased coin: } H &= -(0.99 * \log_2(0.99) + 0.01 * \log_2(0.01)) = 0.08 \text{ bits} \end{aligned}$$

Entropy measures the irreducible uncertainty in a distribution. You cannot compress below it.

Cross-Entropy (The Loss Function You Use Every Day)

Cross-entropy measures the average surprise when you use distribution Q to encode events that actually come from distribution P.

$$H(P, Q) = -\sum (p(x) * \log_2(q(x))) \text{ for all } x$$

P is the true distribution (the labels). Q is your model's predictions. If Q matches P perfectly, cross-entropy equals entropy. Any mismatch makes it larger.

In classification, P is a one-hot vector (the true class has probability 1, everything else 0). This simplifies cross-entropy to:

$$H(P, Q) = -\log(q(\text{true_class}))$$

That is the entire cross-entropy loss formula for classification. Maximize the predicted probability of the correct class.

KL Divergence (Distance Between Distributions)

KL divergence measures how much extra surprise you get from using Q instead of P.

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum (p(x) * \log_2(p(x) / q(x))) \text{ for all } x \\ &= H(P, Q) - H(P) \end{aligned}$$

Cross-entropy is entropy plus KL divergence. Since entropy of the true distribution is constant during training, minimizing cross-entropy is the same as minimizing KL divergence. You are pushing your model's distribution toward the true distribution.

KL divergence is not symmetric: $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$. It is not a true distance metric.

Mutual Information

Mutual information measures how much knowing one variable tells you about another.

$$\begin{aligned} I(X; Y) &= H(X) - H(X|Y) \\ &= H(X) + H(Y) - H(X, Y) \end{aligned}$$

If X and Y are independent, mutual information is zero. Knowing one tells you nothing about the other. If they are perfectly correlated, mutual information equals the entropy of either variable.

In feature selection, high mutual information between a feature and the target means the feature is useful. Low mutual information means it is noise.

Conditional Entropy

$H(Y|X)$ measures how much uncertainty remains about Y after you observe X .

$$H(Y|X) = H(X, Y) - H(X)$$

Two extremes: - If X completely determines Y , then $H(Y|X) = 0$. Knowing X eliminates all uncertainty about Y . Example: X = temperature in Celsius, Y = temperature in Fahrenheit. - If X tells you nothing about Y , then $H(Y|X) = H(Y)$. Knowing X does not reduce your uncertainty at all. Example: X = coin flip, Y = tomorrow's weather.

Conditional entropy is always non-negative and never exceeds $H(Y)$:

$$0 \leq H(Y|X) \leq H(Y)$$

In machine learning, conditional entropy appears in decision trees. At each split, the algorithm picks the feature X that minimizes $H(Y|X)$ – the feature that removes the most uncertainty about the label Y .

Joint Entropy

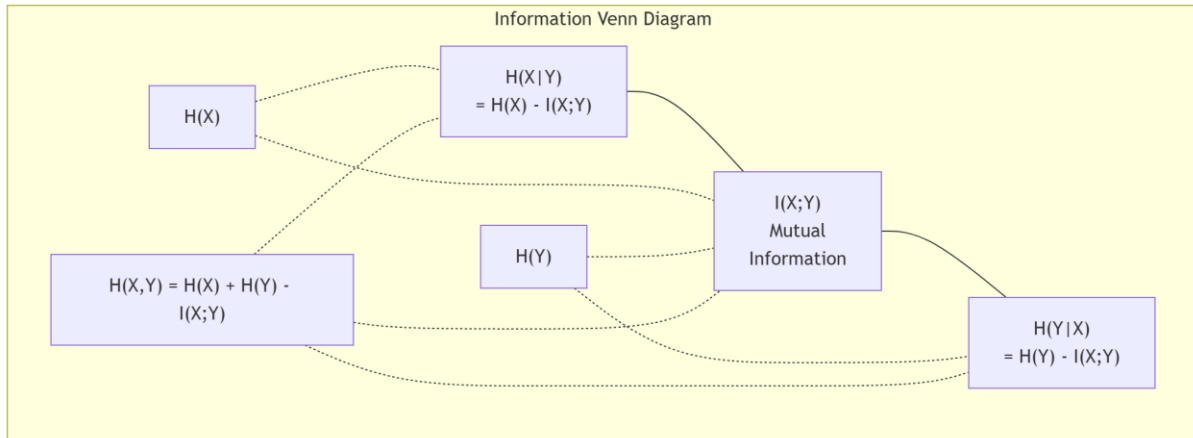
$H(X, Y)$ is the entropy of the joint distribution of X and Y together.

$$H(X, Y) = -\sum_x \sum_y p(x, y) \cdot \log(p(x, y)) \quad \text{for all } x, y$$

Key property:

$$H(X, Y) \leq H(X) + H(Y)$$

Equality holds when X and Y are independent. If they share information, the joint entropy is less than the sum of individual entropies. The “missing” entropy is exactly the mutual information.



The relationships: - $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y) - I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) - H(X,Y) = H(X) + H(Y) - I(X;Y)$

Mutual Information (Deep Dive)

Mutual information $I(X;Y)$ quantifies how much knowing one variable reduces uncertainty about the other.

$$\begin{aligned}
 I(X;Y) &= H(X) - H(X|Y) \\
 &= H(Y) - H(Y|X) \\
 &= H(X) + H(Y) - H(X,Y) \\
 &= \sum \sum p(x,y) * \log(p(x,y) / (p(x) * p(y)))
 \end{aligned}$$

Properties: - $I(X;Y) \geq 0$ always. You never lose information by observing something. - $I(X;Y) = 0$ if and only if X and Y are independent. - $I(X;Y) = I(Y;X)$. It is symmetric, unlike KL divergence. - $I(X;X) = H(X)$. A variable shares all its information with itself.

Mutual information for feature selection. In ML, you want features that are informative about the target. Mutual information gives you a principled way to rank features:

1. For each feature X_i , compute $I(X_i; Y)$ where Y is the target variable.
2. Rank features by MI score.
3. Keep the top k features.

This works for any relationship between feature and target – linear, nonlinear, monotonic, or not. Correlation only catches linear relationships. MI catches everything.

Method	Detects	Computational cost	Handles categorical?
Pearson correlation	Linear relationships	$O(n)$	No
Spearman correlation	Monotonic relationships	$O(n \log n)$	No

Method	Detects	Computational cost	Handles categorical?
Mutual information	Any statistical dependency	$O(n \log n)$ with binning	Yes

Label Smoothing and Cross-Entropy

Standard classification uses hard targets: $[0, 0, 1, 0]$. The true class gets probability 1, everything else gets 0. Label smoothing replaces these with soft targets:

$$\text{soft_target} = (1 - \epsilon) * \text{hard_target} + \epsilon / \text{num_classes}$$

With $\epsilon = 0.1$ and 4 classes: - Hard target: $[0, 0, 1, 0]$ - Soft target: $[0.025, 0.025, 0.925, 0.025]$

From an information theory perspective, label smoothing increases the entropy of the target distribution. Hard one-hot targets have entropy 0 – there is no uncertainty. Soft targets have positive entropy.

Why this helps: - Prevents the model from driving logits to extreme values (infinite logits would be needed to perfectly match a one-hot target under cross-entropy) - Acts as regularization: the model cannot be 100% confident - Improves calibration: predicted probabilities better reflect true uncertainty - Reduces the gap between training and inference behavior

The cross-entropy loss with label smoothing becomes:

$$L = (1 - \epsilon) * \text{CE}(\text{hard_target}, \text{prediction}) + \epsilon * H_{\text{uniform}}(\text{prediction})$$

The second term penalizes predictions that are far from uniform – a direct regularization on confidence.

Why Cross-Entropy Is THE Classification Loss

Three perspectives, same conclusion.

Information theory view. Cross-entropy measures how many bits you waste by using your model's distribution instead of the true distribution. Minimizing it makes your model the most efficient encoder of reality.

Maximum likelihood view. For N training samples with true classes y_i :

$$\begin{aligned} \text{Likelihood} &= \text{product}(q(y_i)) \\ \text{Log-likelihood} &= \text{sum}(\log(q(y_i))) \\ \text{Negative log-likelihood} &= -\text{sum}(\log(q(y_i))) \end{aligned}$$

That last line is cross-entropy loss. Minimizing cross-entropy = maximizing the likelihood of the training data under your model.

Gradient view. The gradient of cross-entropy with respect to the logits is simply (predicted - true). Clean, stable, and fast to compute. This is why it pairs perfectly with softmax.

Bits vs Nats

The only difference is the log base.

```
log base 2    -> bits      (information theory tradition)
log base e    -> nats      (machine learning convention)
log base 10   -> hartleys  (rarely used)
```

1 nat = $1/\ln(2)$ bits = 1.4427 bits. PyTorch and TensorFlow use natural log (nats) by default.

Perplexity

Perplexity is the exponential of cross-entropy. It tells you the effective number of equally likely choices the model is uncertain between.

```
Perplexity = 2^H(P,Q)  (if using bits)
Perplexity = e^H(P,Q)  (if using nats)
```

A language model with perplexity 50 is, on average, as confused as if it had to pick uniformly from 50 possible next tokens. Lower is better.

GPT-2 achieved perplexity ~30 on common benchmarks. Modern models are in the single digits for well-represented domains.

Build It

Step 1: Information content and entropy

```
import math

def information_content(p, base=2):
    if p <= 0 or p > 1:
        return float('inf') if p <= 0 else 0.0
    return -math.log(p) / math.log(base)

def entropy(probs, base=2):
    return sum(
        p * information_content(p, base)
        for p in probs if p > 0
    )

fair_coin = [0.5, 0.5]
biased_coin = [0.99, 0.01]
fair_die = [1/6] * 6
```

```

print(f"Fair coin entropy: {entropy(fair_coin):.4f} bits")
print(f"Biased coin entropy: {entropy(biased_coin):.4f} bits")
print(f"Fair die entropy: {entropy(fair_die):.4f} bits")

```

Step 2: Cross-entropy and KL divergence

```

def cross_entropy(p, q, base=2):
    total = 0.0
    for pi, qi in zip(p, q):
        if pi > 0:
            if qi <= 0:
                return float('inf')
            total += pi * (-math.log(qi) / math.log(base))
    return total

def kl_divergence(p, q, base=2):
    return cross_entropy(p, q, base) - entropy(p, base)

true_dist = [0.7, 0.2, 0.1]
good_model = [0.6, 0.25, 0.15]
bad_model = [0.1, 0.1, 0.8]

print(f"Entropy of true dist: {entropy(true_dist):.4f} bits")
print(f"CE (good model): {cross_entropy(true_dist, good_model):.4f} bits")
print(f"CE (bad model): {cross_entropy(true_dist, bad_model):.4f} bits")
print(f"KL divergence (good): {kl_divergence(true_dist, good_model):.4f} bits")
print(f"KL divergence (bad): {kl_divergence(true_dist, bad_model):.4f} bits")

```

Step 3: Cross-entropy as classification loss

```

def softmax(logits):
    max_logit = max(logits)
    exps = [math.exp(z - max_logit) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

def cross_entropy_loss(true_class, logits):
    probs = softmax(logits)
    return -math.log(probs[true_class])

logits = [2.0, 1.0, 0.1]
true_class = 0

probs = softmax(logits)
loss = cross_entropy_loss(true_class, logits)

print(f"Logits: {logits}")
print(f"Softmax: {[f'{p:.4f}' for p in probs]}")
print(f"True class: {true_class}")

```

```
print(f"Loss:          {loss:.4f} nats")
print(f"Perplexity:   {math.exp(loss):.2f}")
```

Step 4: Cross-entropy equals negative log-likelihood

```
import random

random.seed(42)

n_samples = 1000
n_classes = 3
true_labels = [random.randint(0, n_classes - 1) for _ in range(n_samples)]
model_logits = [[random.gauss(0, 1) for _ in range(n_classes)] for _ in range(n_samples)]

ce_loss = sum(
    cross_entropy_loss(label, logits)
    for label, logits in zip(true_labels, model_logits)
) / n_samples

nll = -sum(
    math.log(softmax(logits)[label])
    for label, logits in zip(true_labels, model_logits)
) / n_samples

print(f"Cross-entropy loss:      {ce_loss:.6f}")
print(f"Negative log-likelihood: {nll:.6f}")
print(f"Difference:              {abs(ce_loss - nll):.2e}")
```

Step 5: Mutual information

```
def mutual_information(joint_probs, base=2):
    rows = len(joint_probs)
    cols = len(joint_probs[0])

    margin_x = [sum(joint_probs[i][j] for j in range(cols)) for i in range(rows)]
    margin_y = [sum(joint_probs[i][j] for i in range(rows)) for j in range(cols)]

    mi = 0.0
    for i in range(rows):
        for j in range(cols):
            pxy = joint_probs[i][j]
            if pxy > 0:
                mi += pxy * math.log(pxy / (margin_x[i] * margin_y[j])) / math.log(base)
    )

    return mi

independent = [[0.25, 0.25], [0.25, 0.25]]
dependent = [[0.45, 0.05], [0.05, 0.45]]
```

```
print(f"MI (independent): {mutual_information(independent):.4f} bits")
print(f"MI (dependent): {mutual_information(dependent):.4f} bits")
```

Use It

The same concepts using NumPy, the way you will use them in practice:

```
import numpy as np

def np_entropy(p):
    p = np.asarray(p, dtype=float)
    mask = p > 0
    result = np.zeros_like(p)
    result[mask] = p[mask] * np.log(p[mask])
    return -result.sum()

def np_cross_entropy(p, q):
    p, q = np.asarray(p, dtype=float), np.asarray(q, dtype=float)
    mask = p > 0
    return -(p[mask] * np.log(q[mask])).sum()

def np_kl_divergence(p, q):
    return np_cross_entropy(p, q) - np_entropy(p)

true = np.array([0.7, 0.2, 0.1])
pred = np.array([0.6, 0.25, 0.15])
print(f"Entropy: {np_entropy(true):.4f} nats")
print(f"Cross-ent: {np_cross_entropy(true, pred):.4f} nats")
print(f"KL div: {np_kl_divergence(true, pred):.4f} nats")
```

You built from scratch what `torch.nn.CrossEntropyLoss()` does internally. Now you know why the loss goes down during training: your model's predicted distribution is getting closer to the true distribution, measured in nats of wasted information.

Exercises

1. Compute the entropy of the English alphabet assuming uniform distribution (26 letters). Then estimate it using actual letter frequencies. Which is higher and why?
2. A model outputs logits [5.0, 2.0, 0.5] for a sample with true class 1. Compute the cross-entropy loss by hand, then verify with your `cross_entropy_loss` function. What logits would give zero loss?
3. Show that KL divergence is not symmetric. Pick two distributions P and Q and compute $D_{KL}(P || Q)$ and $D_{KL}(Q || P)$. Explain why they differ.
4. Build a function that computes perplexity for a sequence of token predictions. Given a list of (true_token_index, predicted_logits) pairs, return the perplexity of the sequence.

Key Terms

Term	What people say	What it actually means
Information content	“Surprise”	The number of bits (or nats) needed to encode an event: $-\log(p)$
Entropy	“Randomness”	The average surprise across all outcomes of a distribution. Measures irreducible uncertainty.
Cross-entropy	“The loss function”	Average surprise when using model distribution Q to encode events from true distribution P.
KL divergence	“Distance between distributions”	Extra bits wasted by using Q instead of P. Equals cross-entropy minus entropy. Not symmetric.
Mutual information	“How related are X and Y”	Reduction in uncertainty about X from knowing Y. Zero means independent.
Softmax	“Turn logits into probabilities”	Exponentiate and normalize. Maps any real-valued vector to a valid probability distribution.
Perplexity	“How confused the model is”	Exponential of cross-entropy. The effective vocabulary size the model is choosing from at each step.
Bits	“Shannon’s unit”	Information measured with log base 2. One bit resolves one fair coin flip.
Nats	“ML’s unit”	Information measured with natural log. Used by PyTorch and TensorFlow by default.
Negative log-likelihood	“NLL loss”	Identical to cross-entropy loss for one-hot labels. Minimizing it maximizes the probability of correct predictions.

Information Theory for ML

When to use entropy, cross-entropy, KL divergence, and mutual information in machine learning systems.

Decision Checklist

1. Measuring uncertainty in a single distribution? Use **entropy**.
2. Measuring how well a model approximates true labels? Use **cross-entropy** (this is your classification loss).
3. Measuring distance between two distributions? Use **KL divergence**.
4. Checking if two variables are related? Use **mutual information**.

- Reporting language model quality? Use **perplexity** (exponential of cross-entropy).
- Distilling one model into another? Minimize **KL divergence** from teacher to student.

When to use each measure

Measure	Formula	Use case	ML application
Entropy $H(P)$	$-\sum(p \log p)$	How uncertain is this distribution?	Data complexity, maximum entropy models
Cross-entropy $H(P, Q)$	$-\sum(p \log q)$	How good is model Q at predicting true P?	Classification loss, language model loss
KL divergence $D(P Q)$	$\sum(p \log(p/q))$	How different are P and Q?	VAE loss (ELBO), knowledge distillation, RLHF
Mutual information $I(X; Y)$	$H(X) - H(X Y)$	How much does Y tell us about X?	Feature selection, representation learning
Perplexity	$\exp(H(P, Q))$ or 2^H	How confused is the model?	Language model evaluation
Conditional entropy $H(X Y)$	$-\sum(p(x,y) \log p(x y))$	Remaining uncertainty in X after knowing Y	Feature informativeness

Key relationships

$$\begin{aligned} \text{Cross-entropy} &= \text{Entropy} + \text{KL divergence} \\ H(P, Q) &= H(P) + D_{\text{KL}}(P || Q) \end{aligned}$$

Since $H(P)$ is constant during training:
Minimizing cross-entropy = Minimizing KL divergence

$$\begin{aligned} \text{Mutual information} &= \text{Entropy} - \text{Conditional entropy} \\ I(X; Y) &= H(X) - H(X|Y) = H(Y) - H(Y|X) \end{aligned}$$

$$\begin{aligned} \text{Perplexity} &= \exp(\text{cross-entropy in nats}) \\ &= 2^{(\text{cross-entropy in bits})} \end{aligned}$$

Quick reference: formulas and units

Formula	Bits (log base 2)	Nats (log base e)
Information: $-\log(p)$	$-\log_2(p)$	$-\ln(p)$

Formula	Bits (log base 2)	Nats (log base e)
Entropy: $-\sum(p \log p)$	bits	nats
1 nat =	1.4427 bits	1 nat
PyTorch default	–	nats
Information theory papers	bits	–

Interpreting values

Entropy value	What it means
0	Deterministic. One outcome has probability 1.
$\log(n)$	Maximum uncertainty. Uniform distribution over n outcomes.
Low	Distribution is peaked. Model is confident.
High	Distribution is flat. Model is uncertain.

Perplexity value	Language model quality
1	Perfect prediction (never happens in practice)
10	Choosing among ~10 equally likely tokens on average
50	GPT-2 level on standard benchmarks
< 10	State-of-the-art for well-represented domains

Common mistakes

- Computing KL divergence and treating it as symmetric. $D_{KL}(P || Q) \neq D_{KL}(Q || P)$. For a symmetric measure, use Jensen-Shannon divergence: $JS = 0.5 * KL(P || M) + 0.5 * KL(Q || M)$ where $M = 0.5 * (P+Q)$.
- Forgetting that cross-entropy with one-hot labels simplifies to $-\log(p_{true_class})$. You do not need to sum over all classes when the true distribution is one-hot.
- Using log base 2 in code but reporting nats (or vice versa). PyTorch uses natural log by default. Multiply by $\log_2(e) = 1.4427$ to convert nats to bits.
- Computing entropy of an empty or zero-probability event. Convention: $0 * \log(0) = 0$, because $\lim_{p \rightarrow 0} p * \log(p) = 0$.

- Comparing perplexity across different vocabularies. A model with vocab size 50k and perplexity 30 is not directly comparable to one with vocab size 10k and perplexity 30.

Where each concept appears in production ML

Concept	Where you see it
Cross-entropy loss	Every classification model (nn.CrossEntropyLoss)
KL divergence	VAE ELBO, PPO clipping, knowledge distillation
Entropy regularization	Exploration bonus in RL (higher entropy = more exploration)
Mutual information	Feature selection, InfoNCE loss (contrastive learning)
Perplexity	Language model benchmarks (lower = better)
Label smoothing	Replaces one-hot with soft targets, reduces cross-entropy overconfidence
Temperature scaling	Divides logits by T before softmax, controls entropy of output

Quiz

Pre-lesson check

1. What does 'entropy' measure in information theory?
 - a) The energy of a physical system
 - b) The average surprise or uncertainty in a probability distribution **(correct)**
 - c) The number of bits in a binary message
 - d) The error rate of a communication channel

*Explanation: Entropy $H(P) = -\sum(p(x) \log(p(x)))$ measures the average amount of surprise across all outcomes. High entropy means high uncertainty (uniform distribution). Low entropy means the outcome is predictable.**
2. What is the cross-entropy loss commonly used for in neural networks?
 - a) Regression tasks with continuous outputs

- b) Classification tasks, measuring how far predicted probabilities are from true labels **(correct)**
- c) Generating new data samples
- d) Regularizing model weights to prevent overfitting
*Explanation: Cross-entropy loss $H(P, Q) = -\sum(p(x)\log(q(x)))$ measures the difference between the true distribution (labels) and the model's predicted distribution. It is THE standard loss for classification.**

Post-lesson check

3. Why is minimizing cross-entropy equivalent to minimizing KL divergence during training?
 - a) Because KL divergence is always zero during training
 - b) Because cross-entropy = entropy + KL divergence, and the entropy of the true labels is constant, so minimizing cross-entropy minimizes KL divergence **(correct)**
 - c) Because cross-entropy and KL divergence are the same formula
 - d) Because the model's entropy equals zero at convergence
Explanation: $H(P, Q) = H(P) + D_{KL}(P || Q)$. Since the true distribution P doesn't change during training, $H(P)$ is constant. Minimizing $H(P, Q)$ is the same as minimizing $D_{KL}(P || Q)$ – pushing the model toward the true distribution.
4. A language model has perplexity 50 on a test set. What does this mean?
 - a) The model makes 50 errors per sentence
 - b) On average, the model is as uncertain as if it were choosing uniformly from 50 possible next tokens at each step **(correct)**
 - c) The model has 50 layers
 - d) The model was trained for 50 epochs
Explanation: Perplexity = $e^{(\text{cross-entropy})}$. A perplexity of 50 means the model's uncertainty at each token is equivalent to picking randomly from 50 equally likely options. Lower perplexity means better predictions.

5. How does mutual information differ from Pearson correlation for feature selection?
- a) Mutual information is faster to compute
 - b) Mutual information detects any statistical dependency (linear or nonlinear) while correlation only detects linear relationships **(correct)**
 - c) Pearson correlation works for any data type while mutual information only works for continuous data
 - d) They always give the same feature rankings
- Explanation: Mutual information $I(X;Y)$ captures all statistical dependencies between variables, including nonlinear and non-monotonic ones. Pearson correlation only measures linear association, missing many important relationships.*
-

Dimensionality Reduction

High-dimensional data has structure. You find it by looking from the right angle.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations), 03 (Eigenvalues & Eigenvectors), 06 (Probability & Distributions) **Time:** ~90 minutes

Learning Objectives

- Implement PCA from scratch: center data, compute the covariance matrix, eigendecompose, and project
- Use explained variance ratio and the elbow method to choose the number of principal components
- Compare PCA, t-SNE, and UMAP for visualizing MNIST digits in 2D and explain their tradeoffs
- Apply kernel PCA with an RBF kernel to separate nonlinear data structures that standard PCA cannot handle

The Problem

You have a dataset with 784 features per sample. Maybe it is pixel values of handwritten digits. Maybe it is gene expression levels. Maybe it is user behavior signals. You cannot visualize 784 dimensions. You cannot plot them. You cannot even think about them.

But most of those 784 features are redundant. The actual information lives on a much smaller surface. A handwritten “7” does not need 784 independent numbers to describe it. It needs a few: the angle of the stroke, the length of the crossbar, how much it leans. The rest is noise.

Dimensionality reduction finds that smaller surface. It takes your 784-dimensional data and compresses it to 2, 10, or 50 dimensions while keeping the structure that matters.

The Concept

The curse of dimensionality

High-dimensional spaces are unintuitive. Three things break as dimensions grow.

Distance becomes meaningless. In high dimensions, the distance between any two random points converges to the same value. If every point is roughly the same distance from every other point, nearest-neighbor search stops working.

Dimension	Avg distance ratio (max/min between random points)
2	~5.0
10	~1.8

100	~1.2
1000	~1.02

Volume concentrates in corners. A unit hypercube in d dimensions has 2^d corners. In 100 dimensions, nearly all the volume is in the corners, far from the center. Data points spread to the edges and your models starve for data in the interior.

You need exponentially more data. To maintain the same density of samples in a space, going from 2D to 20D means you need 10^{18} times more data. You never have enough. Reducing dimensions brings the data density back to something workable.

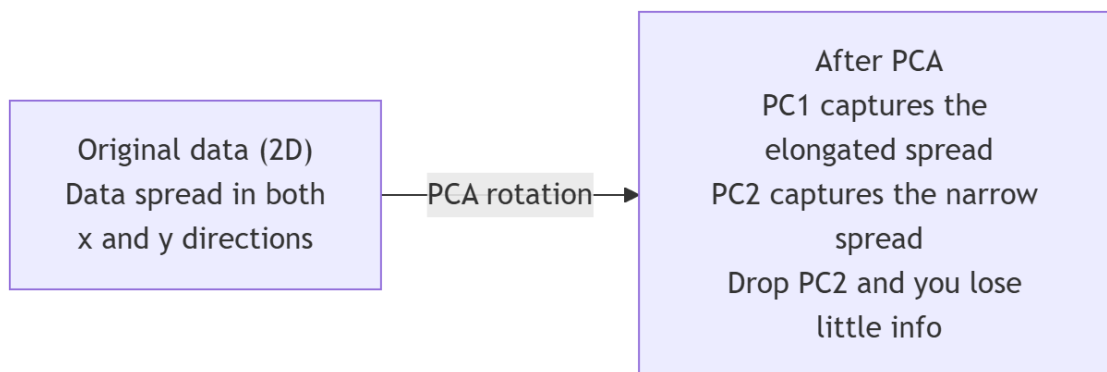
PCA: find the directions that matter

Principal Component Analysis (PCA) finds the axes along which your data varies the most. It rotates your coordinate system so the first axis captures the most variance, the second captures the next most, and so on.

The algorithm:

1. Center the data (subtract the mean from each feature)
2. Compute covariance (how features move together)
3. Eigendecomposition (find the principal directions)
4. Sort by eigenvalue (biggest variance first)
5. Project (keep top k eigenvectors, drop the rest)

Why eigendecomposition? The covariance matrix is symmetric and positive semi-definite. Its eigenvectors are orthogonal directions in feature space. The eigenvalues tell you how much variance each direction captures. The eigenvector with the largest eigenvalue points along the direction of maximum variance.



- **Before PCA:** Data cloud is spread diagonally across both x and y axes
- **After PCA:** Coordinate system is rotated so PC1 aligns with the direction of maximum variance (elongated spread) and PC2 aligns with the direction of minimum variance (narrow spread)
- **Dimensionality reduction:** Dropping PC2 projects the data onto PC1, losing very little information

Explained variance ratio

Each principal component captures a fraction of the total variance. The explained variance ratio tells you how much.

Component	Eigenvalue	Explained ratio	Cumulative
PC1	4.73	0.473	0.473
PC2	2.51	0.251	0.724
PC3	1.12	0.112	0.836
PC4	0.89	0.089	0.925
...			

When the cumulative explained variance reaches 0.95, you know that many components capture 95% of the information. Everything after that is mostly noise.

Choosing the number of components

Three strategies:

1. **Threshold.** Keep enough components to explain 90-95% of the variance.
2. **Elbow method.** Plot explained variance per component. Look for a sharp drop-off.
3. **Downstream performance.** Use PCA as preprocessing. Sweep k and measure your model's accuracy. The best k is wherever accuracy plateaus.

t-SNE: preserve neighborhoods

t-Distributed Stochastic Neighbor Embedding (t-SNE) is designed for visualization. It maps high-dimensional data to 2D (or 3D) while preserving which points are near each other.

The intuition: in the original space, compute a probability distribution over pairs of points based on their distances. Near points get high probability. Far points get low probability. Then find a 2D arrangement where the same probability distribution holds. Points that were neighbors in 784 dimensions stay neighbors in 2D.

Key properties of t-SNE: - Non-linear. It can unfold complex manifolds that PCA cannot. - Stochastic. Different runs produce different layouts. - Perplexity parameter controls how many neighbors to consider (typical range: 5-50). - Distances between clusters in the output are not meaningful. Only the clusters themselves are. - Slow on large datasets. $O(n^2)$ by default.

UMAP: faster, better global structure

Uniform Manifold Approximation and Projection (UMAP) works similarly to t-SNE but with two advantages: - Faster. It uses approximate nearest-neighbor graphs instead of computing all pairwise distances. - Better global structure. The relative positions of clusters in the output tend to be more meaningful than in t-SNE.

UMAP builds a weighted graph in high-dimensional space (the “fuzzy topological representation”) and then finds a low-dimensional layout that preserves this graph as well as possible.

Key parameters: - `n_neighbors`: how many neighbors define local structure (similar to perplexity). Higher values preserve more global structure. - `min_dist`: how tightly points pack together in the output. Lower values create denser clusters.

When to use which

Method	Use case	Preserves	Speed
PCA	Preprocessing before training	Global variance	Fast (exact), works on millions of samples
PCA	Quick exploratory visualization	Linear structure	Fast
t-SNE	Publication-quality 2D plots	Local neighborhoods	Slow (< 10k samples ideal)
UMAP	2D visualization at scale	Local + some global structure	Medium (handles millions)
PCA	Feature reduction for models	Variance-ranked features	Fast
t-SNE / UMAP	Understanding cluster structure	Cluster separation	Medium to slow

Rule of thumb: use PCA for preprocessing and data compression. Use t-SNE or UMAP when you need to visualize structure in 2D.

Kernel PCA

Standard PCA finds linear subspaces. It rotates your coordinate system and drops axes. But what if the data lies on a nonlinear manifold? A circle in 2D cannot be separated by any line. Standard PCA will not help.

Kernel PCA applies PCA in a high-dimensional feature space induced by a kernel function, without explicitly computing the coordinates in that space. This is the kernel trick – the same idea behind SVMs.

The algorithm: 1. Compute the kernel matrix K where $K_{ij} = k(x_i, x_j)$ 2. Center the kernel matrix in feature space 3. Eigendecompose the centered kernel matrix 4. The top eigenvectors (scaled by $1/\sqrt{\text{eigenvalue}}$) are the projections

Common kernel functions:

Kernel	Formula	Good for
RBF (Gaussian)	$\exp(-\gamma * x - y ^2)$	Most nonlinear data, smooth manifolds
Polynomial	$(x \cdot y + c)^d$	Polynomial relationships
Sigmoid	$\tanh(\alpha * x \cdot y + c)$	Neural network-like mappings

When to use kernel PCA vs standard PCA:

Criterion	Standard PCA	Kernel PCA
Data structure	Linear subspace	Nonlinear manifold
Speed	$O(\min(n^2 d, d^2 n))$	$O(n^2 d + n^3)$
Interpretability	Components are linear combinations of features	Components lack direct feature interpretation
Scalability	Works on millions of samples	Kernel matrix is $n \times n$, memory-limited
Reconstruction	Direct inverse transform	Requires pre-image approximation

The classic example: concentric circles in 2D. Two rings of points, one inside the other. Standard PCA projects both onto the same line – useless for classification. Kernel PCA with an RBF kernel maps the inner circle and outer circle to different regions, making them linearly separable.

Reconstruction Error

How good is your dimensionality reduction? You compressed 784 dimensions to 50. What did you lose?

Measure reconstruction error: 1. Project data to k dimensions: $X_{\text{reduced}} = X @ W_k$ 2. Reconstruct: $X_{\text{hat}} = X_{\text{reduced}} @ W_k^T$ 3. Compute MSE: $\text{mean}((X - X_{\text{hat}})^2)$

For PCA, reconstruction error has a clean relationship to explained variance:

```
Reconstruction error = sum of eigenvalues NOT included
Total variance = sum of ALL eigenvalues
Fraction lost = (sum of dropped eigenvalues) / (sum of all eigenvalues)
```

The explained variance ratio for each component is:

```
explained_ratio_k = eigenvalue_k / sum(all eigenvalues)
```

Plotting cumulative explained variance against number of components gives you the “elbow” curve. The right number of components is where: - The curve flattens out (diminishing returns) - Cumulative variance crosses your threshold (usually 0.90 or 0.95) - Downstream task performance plateaus

Reconstruction error is useful beyond choosing k. You can use it for anomaly detection: samples with high reconstruction error are outliers that do not fit the learned subspace. This is the basis of PCA-based anomaly detection in production systems.

Build It

Step 1: PCA from scratch

```
import numpy as np

class PCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.components = None
        self.mean = None
        self.eigenvalues = None
        self.explained_variance_ratio_ = None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean

        cov_matrix = np.cov(X_centered, rowvar=False)

        eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

        sorted_idx = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[sorted_idx]
        eigenvectors = eigenvectors[:, sorted_idx]

        self.components = eigenvectors[:, :self.n_components].T
        self.eigenvalues = eigenvalues[:self.n_components]
        total_var = np.sum(eigenvalues)
        self.explained_variance_ratio_ = self.eigenvalues / total_var

        return self

    def transform(self, X):
        X_centered = X - self.mean
        return X_centered @ self.components.T

    def fit_transform(self, X):
        self.fit(X)
        return self.transform(X)
```

Step 2: Test on synthetic data

```
np.random.seed(42)
n_samples = 500

t = np.random.uniform(0, 2 * np.pi, n_samples)
x1 = 3 * np.cos(t) + np.random.normal(0, 0.2, n_samples)
x2 = 3 * np.sin(t) + np.random.normal(0, 0.2, n_samples)
x3 = 0.5 * x1 + 0.3 * x2 + np.random.normal(0, 0.1, n_samples)

X_synthetic = np.column_stack([x1, x2, x3])

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_synthetic)

print(f"Original shape: {X_synthetic.shape}")
print(f"Reduced shape: {X_reduced.shape}")
print(f"Explained variance ratios: {pca.explained_variance_ratio_}")
print(f"Total variance captured: {sum(pca.explained_variance_ratio_):.4f}")
```

Step 3: MNIST digits in 2D

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml("mnist_784", version=1, as_frame=False, parser="auto")
X_mnist = mnist.data[:5000].astype(float)
y_mnist = mnist.target[:5000].astype(int)

pca_mnist = PCA(n_components=50)
X_pca50 = pca_mnist.fit_transform(X_mnist)
print(f"50 components capture {sum(pca_mnist.explained_variance_ratio_):.2%} of variance")

pca_2d = PCA(n_components=2)
X_pca2d = pca_2d.fit_transform(X_mnist)
print(f"2 components capture {sum(pca_2d.explained_variance_ratio_):.2%} of variance")
```

Step 4: Compare with sklearn

```
from sklearn.decomposition import PCA as SklearnPCA
from sklearn.manifold import TSNE

sklearn_pca = SklearnPCA(n_components=2)
X_sklearn_pca = sklearn_pca.fit_transform(X_mnist)

print(f"\nOur PCA explained variance: {pca_2d.explained_variance_ratio_}")
print(f"Sklearn PCA explained variance: {sklearn_pca.explained_variance_ratio_}")
```

```

diff = np.abs(np.abs(X_pca2d) - np.abs(X_sklearn_pca))
print(f"Max absolute difference: {diff.max():.10f}")

tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_mnist)
print(f"\nt-SNE output shape: {X_tsne.shape}")

```

Step 5: UMAP comparison

```

try:
    from umap import UMAP

    reducer = UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X_mnist)
    print(f"UMAP output shape: {X_umap.shape}")
except ImportError:
    print("Install umap-learn: pip install umap-learn")

```

Use It

PCA as preprocessing before a classifier:

```

from sklearn.decomposition import PCA as SklearnPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X_mnist, y_mnist, test_size=0.2, random_state=42
)

results = {}
for k in [10, 30, 50, 100, 200]:
    pca_k = SklearnPCA(n_components=k)
    X_tr = pca_k.fit_transform(X_train)
    X_te = pca_k.transform(X_test)

    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(X_tr, y_train)
    acc = accuracy_score(y_test, clf.predict(X_te))
    var_captured = sum(pca_k.explained_variance_ratio_)
    results[k] = (acc, var_captured)
    print(f"k={k:>3d} accuracy={acc:.4f} variance={var_captured:.4f}")

```

Performance plateaus well before 784 dimensions. That plateau is your operating point.

Exercises

1. Modify the PCA class to support `inverse_transform`. Reconstruct MNIST digits from 10, 50, and 200 components. Print the reconstruction error (mean squared difference from the original) for each.
2. Run t-SNE on the same MNIST subset with perplexity values of 5, 30, and 100. Describe how the output changes. Why does perplexity affect cluster tightness?
3. Take a dataset with 50 features where only 5 are informative (generate one with `sklearn.datasets.make_classification`). Apply PCA and check whether the explained variance curve correctly identifies that the data is effectively 5-dimensional.

Key Terms

Term	What people say	What it actually means
Curse of dimensionality	"Too many features"	Distances, volumes, and data density all behave counterintuitively as dimensions grow. Models need exponentially more data to compensate.
PCA	"Reduce dimensions"	Rotate your coordinate system so the axes align with the directions of maximum variance, then drop the low-variance axes.
Principal component	"An important direction"	An eigenvector of the covariance matrix. The direction in feature space along which the data varies most.
Explained variance ratio	"How much info this component has"	The fraction of total variance captured by one principal component. Sum the top k ratios to see how much k components preserve.
Covariance matrix	"How features correlate"	A symmetric matrix where entry (i,j) measures how feature i and feature j move together. Diagonal entries are individual variances.
t-SNE	"That cluster plot"	A nonlinear method that maps high-dimensional data to 2D by preserving pairwise neighborhood probabilities. Good for visualization, not for preprocessing.
UMAP	"Faster t-SNE"	A nonlinear method based on topological data analysis. Preserves both local and some global structure. Scales better than t-SNE.
Perplexity	"A t-SNE knob"	Controls the effective number of neighbors each point considers. Low perplexity focuses on very local structure. High perplexity captures broader patterns.
Manifold	"The surface the data lives on"	A lower-dimensional surface embedded in a higher-dimensional space. A sheet of paper crumpled in 3D is a 2D manifold.

You are an expert at selecting and applying dimensionality reduction methods. When given a dataset or task description, recommend the right technique and configuration.

Decision Framework

Step 1: Identify the goal

- **Preprocessing for a model** (classification, regression, clustering): Use PCA. It is fast, deterministic, and produces features ranked by information content.
- **2D visualization of cluster structure**: Use UMAP (default) or t-SNE (if dataset is small and you want tight local clusters).
- **Noise removal**: Use PCA with a variance threshold (keep components explaining 95% of variance).
- **Feature compression for storage or speed**: Use PCA. Choose k by downstream task performance, not just variance.

Step 2: Check constraints

Constraint	Recommendation
Dataset > 100k samples	PCA or UMAP. Avoid t-SNE ($O(n^2)$ without approximation).
Need deterministic results	PCA. t-SNE and UMAP are stochastic.
Nonlinear manifold structure	UMAP or t-SNE. PCA only captures linear relationships.
Need to transform new data	PCA (has an exact transform). UMAP supports approximate transform. t-SNE does not transform new points.
Interpretable components	PCA. Each component is a weighted combination of original features.
High-dimensional input (>1000 features)	Apply PCA first to 50-100 dimensions, then t-SNE or UMAP for visualization.

Step 3: Configure parameters

PCA: - `n_components`: Start with cumulative explained variance ≥ 0.95 . For visualization, use 2. For preprocessing, sweep k and measure downstream accuracy.

t-SNE: - `perplexity`: 5-50. Low values (5-10) for small, tight clusters. High values (30-50) for broader structure. Try multiple values. - `n_iter`: At least 1000. Watch for convergence. - Always apply PCA first to reduce to 50 dimensions before t-SNE.

UMAP: - `n_neighbors`: 5-50. Low for local detail, high for global layout. Default 15 is reasonable. - `min_dist`: 0.0-1.0. Low values pack clusters tightly. Default 0.1 works for most cases. - `metric`: “euclidean” for dense data, “cosine” for text embeddings.

Step 4: Validate

- For PCA: check explained variance curve. A sharp elbow confirms low intrinsic dimensionality.
- For t-SNE/UMAP: run multiple times with different seeds. Clusters that appear consistently are real. Clusters that move around are artifacts.
- For preprocessing: measure downstream task performance. If accuracy does not drop after reduction, you kept the signal.

Common Mistakes

- Using t-SNE output as input features for a model. t-SNE is for visualization only.
- Interpreting distances between t-SNE clusters as meaningful. Only cluster membership matters.
- Applying PCA without centering. Always subtract the mean first.
- Choosing PCA components by count instead of by explained variance. 50 components in one dataset is very different from 50 in another.
- Running t-SNE on raw high-dimensional data. Always reduce with PCA first.

Quiz

Pre-lesson check

1. What is the ‘curse of dimensionality’?
 - a) High-dimensional data takes too long to download
 - b) As dimensions grow, distances become meaningless, volume concentrates in corners, and you need exponentially more data (**correct**)
 - c) Neural networks cannot process data with more than 100 features
 - d) High-dimensional data always contains noise
Explanation: In high dimensions, all pairwise distances converge to similar values, data points spread to corners, and maintaining sample density requires exponentially more data. Dimensionality reduction counteracts these effects.
2. What does PCA find?

- a) The most important features by name
 - b) The orthogonal directions of maximum variance in the data **(correct)**
 - c) Clusters of similar data points
 - d) The optimal number of features to keep
- Explanation: PCA computes the eigenvectors of the covariance matrix, which are orthogonal directions ranked by how much data variance they capture. The first principal component points along the direction of maximum spread.*

Post-lesson check

3. After running PCA on 784-dimensional MNIST data with $k=50$ components, you find 95% of variance is captured. What does this tell you?
 - a) Only 50 pixels matter in each image
 - b) The data effectively lives in a ~50-dimensional subspace; the remaining 734 dimensions are mostly noise or redundancy **(correct)**
 - c) 95% of images belong to the same class
 - d) The model will achieve 95% accuracy

Explanation: 95% explained variance with 50 components means the essential structure of 784-dimensional data is captured by just 50 directions. The rest carries only 5% of the variation – mostly noise.
4. Why should you NOT use t-SNE as preprocessing before training a classifier?
 - a) t-SNE is too slow for large datasets
 - b) t-SNE is designed for visualization only: it distorts global distances, is stochastic, and the output coordinates have no consistent meaning across runs **(correct)**
 - c) t-SNE reduces data to exactly 2 dimensions which is too few
 - d) t-SNE requires the labels to be known in advance

Explanation: t-SNE preserves local neighborhoods but distorts global structure. Distances

between clusters are meaningless, and different runs produce different layouts. Use PCA for preprocessing and t-SNE/UMAP only for visualization.

5. When would you choose kernel PCA over standard PCA?

- a) When you have more samples than features
- b) When the data lies on a nonlinear manifold that standard PCA cannot separate, like concentric circles **(correct)**

c) When you need the fastest possible computation

d) When you want interpretable principal components

Explanation: Standard PCA finds linear subspaces. If data has nonlinear structure (e.g., two concentric rings), PCA projects both onto the same line. Kernel PCA maps data to a higher-dimensional space where the structure becomes linear.

Singular Value Decomposition

SVD is the Swiss Army knife of linear algebra. Every matrix has one. Every data scientist needs one.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors & Matrices Operations), 03 (Matrix Transformations) **Time:** ~120 minutes

Learning Objectives

- Implement SVD via power iteration and explain the geometric meaning of U , Σ , and V^T
- Apply truncated SVD for image compression and measure the compression ratio vs reconstruction error
- Compute the Moore-Penrose pseudoinverse via SVD to solve overdetermined least-squares systems
- Connect SVD to PCA, recommendation systems (latent factors), and Latent Semantic Analysis in NLP

The Problem

You have a 1000x2000 matrix. Maybe it is user-movie ratings. Maybe it is a document-term frequency table. Maybe it is the pixel values of an image. You need to compress it, denoise it, find hidden structure in it, or solve a least-squares system with it. Eigendecomposition only works on square matrices. Even then, it requires the matrix to have a full set of linearly independent eigenvectors.

SVD works on any matrix. Any shape. Any rank. No conditions. It decomposes the matrix into three factors that reveal the geometry of what the matrix does to space. It is the most general and most useful factorization in all of linear algebra.

The Concept

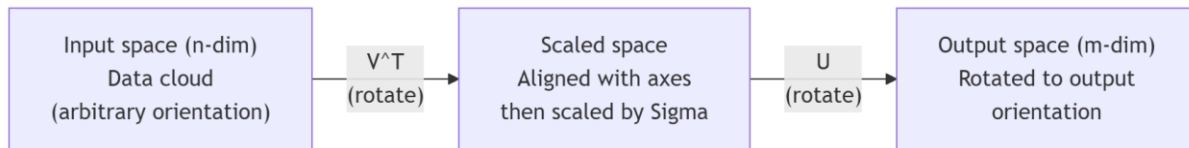
What SVD does geometrically

Every matrix, regardless of shape, performs three operations in sequence: rotate, scale, rotate. SVD makes this decomposition explicit.

$$A = U * \text{Sigma} * V^T$$

$m \times n$	$m \times m$	$m \times n$	$n \times n$
(any)	(rotate)	(scale)	(rotate)

Given any matrix A , SVD factors it into: - V^T rotates vectors in the input space (n -dimensional) - Σ scales along each axis (stretches or compresses) - U rotates the result into the output space (m -dimensional)



Think of it this way. You hand SVD a matrix. It tells you: “This matrix takes a sphere of inputs, first rotates it by V^T , then stretches it into an ellipsoid by Σ , then rotates the ellipsoid by U .” The singular values are the lengths of the ellipsoid’s axes.

The full decomposition

For a matrix A with shape $m \times n$:

$$A = U * \text{Sigma} * V^T$$

where:

U is $m \times m$, orthogonal ($U^T U = I$)

Sigma is $m \times n$, diagonal (singular values on the diagonal)

V is $n \times n$, orthogonal ($V^T V = I$)

The singular values $\text{sigma}_1 \geq \text{sigma}_2 \geq \dots \geq \text{sigma}_r > 0$
where $r = \text{rank}(A)$

The columns of U are called left singular vectors. The columns of V are called right singular vectors. The diagonal entries of Σ are called singular values. They are always non-negative and conventionally sorted in decreasing order.

Left singular vectors, singular values, right singular vectors

Each component of the SVD has a distinct geometric meaning.

Right singular vectors (columns of V): These form an orthonormal basis for the input space ($\mathbb{R}^n$). They are the directions in input space that the matrix maps to orthogonal directions in output space. Think of them as the natural coordinate system for the domain.

Singular values (diagonal of Σ): These are the scaling factors. The i -th singular value tells you how much the matrix stretches vectors along the i -th right singular vector. A singular value of zero means the matrix crushes that direction entirely.

Left singular vectors (columns of U): These form an orthonormal basis for the output space ($\mathbb{R}^m$). The i -th left singular vector is the direction in output space where the i -th right singular vector lands (after scaling).

The relationship between them:

$$A * v_i = \sigma_i * u_i$$

The matrix A takes the i-th right singular vector v_i , scales it by σ_i , and maps it to the i-th left singular vector u_i .

This gives you a coordinate-by-coordinate picture of what any matrix does.

Outer product form

The SVD can be written as a sum of rank-1 matrices:

$$A = \sigma_1 * u_1 * v_1^T + \sigma_2 * u_2 * v_2^T + \dots + \sigma_r * u_r * v_r^T$$

Each term $\sigma_i * u_i * v_i^T$ is a rank-1 matrix (an outer product). The full matrix is the sum of r such matrices, where r is the rank.

This form is the foundation of low-rank approximation. Each term adds one layer of structure. The first term captures the single most important pattern. The second captures the next most important. And so on. Truncating this sum gives you the best possible approximation at any given rank.

Rank-1 approx: $A_1 = \sigma_1 * u_1 * v_1^T$
(captures the dominant pattern)

Rank-2 approx: $A_2 = \sigma_1 * u_1 * v_1^T + \sigma_2 * u_2 * v_2^T$
(captures the two most important patterns)

Rank-k approx: $A_k = \text{sum of top } k \text{ terms}$
(optimal by the Eckart-Young theorem)

Relationship to eigendecomposition

SVD and eigendecomposition are deeply connected. The singular values and vectors of A come directly from the eigenvalues and eigenvectors of $A^T A$ and $A A^T$.

$$\begin{aligned} A^T A &= V * \Sigma^T * U^T * U * \Sigma * V^T \\ &= V * \Sigma^T * \Sigma * V^T \\ &= V * D * V^T \end{aligned}$$

where $D = \Sigma^T * \Sigma$ is a diagonal matrix with σ_i^2 on the diagonal.

So:

- The right singular vectors (V) are eigenvectors of $A^T A$
- The singular values squared (σ_i^2) are eigenvalues of $A^T A$

Similarly:

$$\begin{aligned} A A^T &= U * \text{Sigma} * V^T * V * \text{Sigma}^T * U^T \\ &= U * \text{Sigma} * \text{Sigma}^T * U^T \end{aligned}$$

So:

- The left singular vectors (U) are eigenvectors of $A A^T$
- The eigenvalues of $A A^T$ are also σ_i^2

This connection tells you three things: 1. Singular values are always real and non-negative (they are square roots of eigenvalues of a positive semi-definite matrix). 2. You could compute SVD via eigendecomposition of $A^T A$, but this squares the condition number and loses numerical precision. Dedicated SVD algorithms avoid this. 3. When A is square and symmetric positive semi-definite, SVD and eigendecomposition are the same thing.

Truncated SVD: low-rank approximation

The Eckart-Young-Mirsky theorem states that the best rank-k approximation to A (in both Frobenius and spectral norm) is obtained by keeping only the top k singular values and their corresponding vectors:

$$A_k = U_k * \text{Sigma}_k * V_k^T$$

where:

U_k is $m \times k$ (first k columns of U)
 Sigma_k is $k \times k$ (top-left $k \times k$ block of Sigma)
 V_k is $n \times k$ (first k columns of V)

Approximation error = σ_{k+1} (in spectral norm)
= $\sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$ (in Frobenius norm)

This is not just “a good” approximation. It is provably the best possible approximation of rank k. No other rank-k matrix is closer to A.

Component	Relative magnitude	Kept in rank-3 approx?
σ_1	Largest	Yes
σ_2	Large	Yes
σ_3	Medium-large	Yes
σ_4	Medium	No (error)

Component	Relative magnitude	Kept in rank-3 approx?
sigma_5	Medium-small	No (error)
sigma_6	Small	No (error)
sigma_7	Very small	No (error)
sigma_8	Tiny	No (error)

Keep top 3: A_3 captures the three largest singular values. Error = remaining values (sigma_4 through sigma_8).

If singular values decay fast, a small k captures most of the matrix. If they decay slowly, the matrix has no low-rank structure.

Image compression with SVD

A grayscale image is a matrix of pixel intensities. An 800x600 image has 480,000 values. SVD lets you approximate it with far fewer.

Original image: $800 \times 600 = 480,000$ values

SVD with rank k:

U_k: $800 \times k$ values

Sigma_k: k values

V_k: $600 \times k$ values

Total: $k * (800 + 600 + 1) = k * 1401$ values

k=10: 14,010 values (2.9% of original)

k=50: 70,050 values (14.6% of original)

k=100: 140,100 values (29.2% of original)

The compression ratio improves as k gets smaller,
but visual quality degrades.

The key insight: natural images have rapidly decaying singular values. The first few singular values capture the broad structure (shapes, gradients). The later ones capture fine detail and noise. Truncating at rank 50 often produces an image that looks nearly identical to the original while using 85% less storage.

SVD for recommendation systems

The Netflix Prize made this famous. You have a user-movie ratings matrix where most entries are missing.

	Movie1	Movie2	Movie3	Movie4	Movie5
User1	[5	?	3	?	1]
User2	[?	4	?	2	?]
User3	[3	?	5	?	?]
User4	[?	?	?	4	3]

? = unknown rating

The idea: this ratings matrix has low rank. Users do not have completely independent tastes. There are a handful of latent factors (action vs. drama, old vs. new, cerebral vs. visceral) that explain most preferences.

SVD on the (filled-in) ratings matrix decomposes it into: - U : user profiles in latent factor space - Σ : importance of each latent factor - V^T : movie profiles in latent factor space

A user's predicted rating for a movie is the dot product of their user profile with the movie's profile (weighted by singular values). The low-rank approximation fills in the missing entries.

In practice, you use variants like Simon Funk's incremental SVD or ALS (alternating least squares) that handle missing data directly. But the core idea is the same: latent factor decomposition via SVD.

SVD in NLP: Latent Semantic Analysis

Latent Semantic Analysis (LSA), also called Latent Semantic Indexing (LSI), applies SVD to a term-document matrix.

	Doc1	Doc2	Doc3	Doc4
"cat"	[3	0	1	0]
"dog"	[2	0	0	1]
"fish"	[0	4	1	0]
"pet"	[1	1	1	1]
"ocean"	[0	3	0	0]

After SVD with rank $k=2$:

Each document becomes a point in 2D "concept space."

Each term becomes a point in the same 2D space.

Documents about similar topics cluster together.

Terms with similar meanings cluster together.

"cat" and "dog" end up near each other (land pets).

"fish" and "ocean" end up near each other (water concepts).

Doc1 and Doc3 cluster if they share similar topics.

LSA was one of the first successful methods for capturing semantic similarity from raw text. It works because synonymous terms tend to appear in similar documents, so SVD groups them into the same latent dimensions. Modern word embeddings (Word2Vec, GloVe) can be seen as descendants of this idea.

SVD for noise reduction

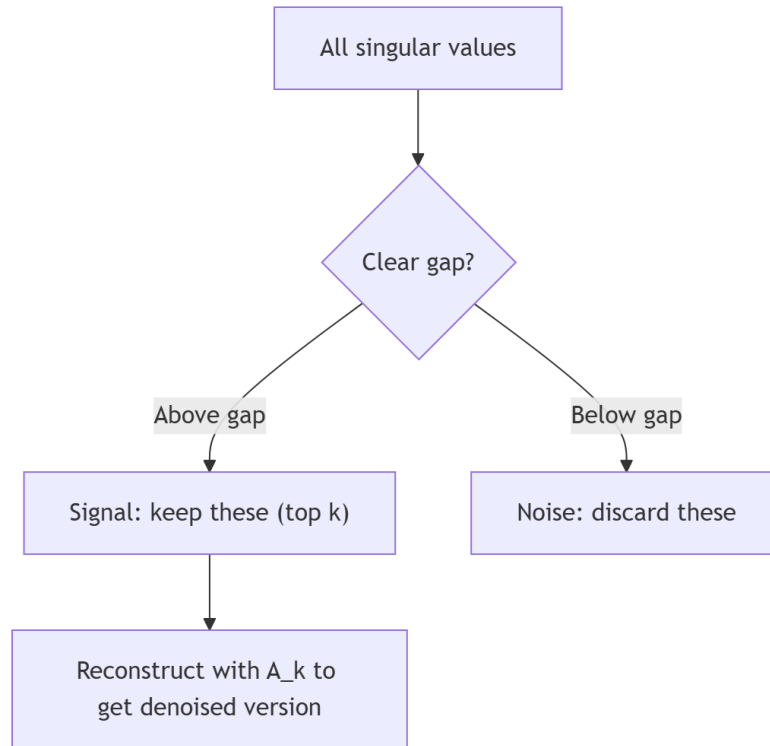
Noisy data has signal concentrated in the top singular values and noise spread across all singular values. Truncating removes the noise floor.

Clean signal singular values:

Component	Magnitude	Type
sigma_1	Very large	Signal
sigma_2	Large	Signal
sigma_3	Medium	Signal
sigma_4	Near zero	Negligible
sigma_5	Near zero	Negligible

Noisy signal singular values (noise adds to all):

Component	Magnitude	Type
sigma_1	Very large	Signal
sigma_2	Large	Signal
sigma_3	Medium	Signal
sigma_4	Small	Noise
sigma_5	Small	Noise
sigma_6	Small	Noise
sigma_7	Small	Noise



This is used in signal processing, scientific measurement, and data cleaning. Any time you have a matrix corrupted by additive noise, truncated SVD is a principled way to separate signal from noise.

Pseudoinverse via SVD

The Moore-Penrose pseudoinverse A^+ generalizes matrix inversion to non-square and singular matrices. SVD makes computing it trivial.

If $A = U * \text{Sigma} * V^T$, then:

$$A^+ = V * \text{Sigma}^+ * U^T$$

where Sigma^+ is formed by:

1. Transpose Sigma (swap rows and columns)
2. Replace each non-zero diagonal entry sigma_i with $1/\text{sigma}_i$
3. Leave zeros as zeros

For A ($m \times n$): A^+ is ($n \times m$)

For Sigma ($m \times n$): Sigma^+ is ($n \times m$)

The pseudoinverse solves least-squares problems. If $Ax = b$ has no exact solution (overdetermined system), then $x = A^+ b$ is the least-squares solution (minimizes $\|Ax - b\|$).

Overdetermined system (more equations than unknowns):

$$\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} x = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix} \quad \text{No exact solution exists.}$$

$$x_{ls} = A^+ b = V * \Sigma^+ * U^T * b$$

This gives the x that minimizes the sum of squared residuals.
Same result as the normal equations $(A^T A)^{-1} A^T b$,
but numerically more stable.

Numerical stability advantages

Computing eigendecomposition of $A^T A$ squares the singular values (eigenvalues of $A^T A$ are σ_i^2). This squares the condition number, amplifying numerical errors.

Example:

A has singular values $[1000, 1, 0.001]$

Condition number of A : $1000 / 0.001 = 10^6$

$A^T A$ has eigenvalues $[10^6, 1, 10^{-6}]$

Condition number of $A^T A$: $10^6 / 10^{-6} = 10^{12}$

Computing SVD directly: works with condition number 10^6

Computing via $A^T A$: works with condition number 10^{12}
(6 extra digits of precision lost)

Modern SVD algorithms (Golub-Kahan bidiagonalization) work directly on A , never forming $A^T A$. This is why you should always prefer `np.linalg.svd(A)` over `np.linalg.eig(A.T @ A)`.

Connection to PCA

PCA IS SVD on centered data. This is not an analogy. It is literally the same computation.

Given data matrix X ($n_{\text{samples}} \times n_{\text{features}}$), centered (mean subtracted):

Covariance matrix: $C = (1/(n-1)) * X^T X$

PCA finds eigenvectors of C . But:

$$X = U * \Sigma * V^T \quad (\text{SVD of } X)$$

$$X^T X = V * \Sigma^2 * V^T$$

$$C = (1/(n-1)) * V * \text{Sigma}^2 * V^T$$

So the principal components are exactly the right singular vectors V . The explained variance for each component is $\text{sigma}_i^2 / (n-1)$.

In sklearn, PCA is implemented using SVD, not eigendecomposition. It is faster and more numerically stable.

This means everything you learned about dimensionality reduction in Lesson 10 is SVD under the hood. PCA is the most common application of SVD in machine learning.

Build It

Step 1: SVD from scratch using power iteration

The idea: to find the largest singular value and its vectors, use power iteration on $A^T A$ (or $A A^T$). Then deflate the matrix and repeat for the next singular value.

```
import numpy as np

def power_iteration(M, num_iters=100):
    n = M.shape[1]
    v = np.random.randn(n)
    v = v / np.linalg.norm(v)

    for _ in range(num_iters):
        Mv = M @ v
        v = Mv / np.linalg.norm(Mv)

    eigenvalue = v @ M @ v
    return eigenvalue, v

def svd_from_scratch(A, k=None):
    m, n = A.shape
    if k is None:
        k = min(m, n)

    sigmas = []
    us = []
    vs = []

    A_residual = A.copy().astype(float)

    for _ in range(k):
        AtA = A_residual.T @ A_residual
        eigenvalue, v = power_iteration(AtA, num_iters=200)

        if eigenvalue < 1e-10:
            break
```

```

    sigma = np.sqrt(eigenvalue)
    u = A_residual @ v / sigma

    sigmas.append(sigma)
    us.append(u)
    vs.append(v)

    A_residual = A_residual - sigma * np.outer(u, v)

    U = np.column_stack(us) if us else np.empty((m, 0))
    S = np.array(sigmas)
    V = np.column_stack(vs) if vs else np.empty((n, 0))

    return U, S, V

```

Step 2: Test and compare with NumPy

```

np.random.seed(42)
A = np.random.randn(5, 4)

U_ours, S_ours, V_ours = svd_from_scratch(A)
U_np, S_np, Vt_np = np.linalg.svd(A, full_matrices=False)

print("Our singular values:", np.round(S_ours, 4))
print("NumPy singular values:", np.round(S_np, 4))

A_reconstructed = U_ours @ np.diag(S_ours) @ V_ours.T
print(f"Reconstruction error: {np.linalg.norm(A - A_reconstructed):.8f}")

```

Step 3: Image compression demo

```

def compress_image_svd(image_matrix, k):
    U, S, Vt = np.linalg.svd(image_matrix, full_matrices=False)
    compressed = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    return compressed

image = np.random.randn(200, 300)
rows, cols = 200, 300
image = np.random.randn(rows, cols)

for k in [1, 5, 10, 20, 50]:
    compressed = compress_image_svd(image, k)
    error = np.linalg.norm(image - compressed) / np.linalg.norm(image)
    original_size = rows * cols
    compressed_size = k * (rows + cols + 1)
    ratio = compressed_size / original_size
    print(f"k={k:>3d}  error={error:.4f}  storage={ratio:.1%}")

```

Step 4: Noise reduction

```
np.random.seed(42)
clean = np.outer(np.sin(np.linspace(0, 4*np.pi, 100)),
                 np.cos(np.linspace(0, 2*np.pi, 80)))
noise = 0.3 * np.random.randn(100, 80)
noisy = clean + noise

U, S, Vt = np.linalg.svd(noisy, full_matrices=False)
denoised = U[:, :5] @ np.diag(S[:5]) @ Vt[:, 5, :]

print(f"Noisy error:      {np.linalg.norm(noisy - clean):.4f}")
print(f"Denoised error: {np.linalg.norm(denoised - clean):.4f}")
print(f"Improvement:     {(1 - np.linalg.norm(denoised - clean) / np.linalg.norm(noisy -
clean)):.1%}")
```

Step 5: Pseudoinverse

```
A = np.array([[1, 1], [2, 1], [3, 1]], dtype=float)
b = np.array([3, 5, 6], dtype=float)

U, S, Vt = np.linalg.svd(A, full_matrices=False)
S_inv = np.diag(1.0 / S)
A_pinv = Vt.T @ S_inv @ U.T

x_svd = A_pinv @ b
x_lstsq = np.linalg.lstsq(A, b, rcond=None)[0]
x_pinv = np.linalg.pinv(A) @ b

print(f"SVD pseudoinverse solution: {x_svd}")
print(f"np.linalg.lstsq solution:    {x_lstsq}")
print(f"np.linalg.pinv solution:      {x_pinv}")
```

Use It

Full working demos are in `code/svd.py`. Run it to see SVD applied to image compression, recommendation systems, latent semantic analysis, and noise reduction.

```
python svd.py
```

The Julia version in `code/svd.jl` demonstrates the same concepts using Julia's native `svd()` function and `LinearAlgebra` package.

```
julia svd.jl
```

Exercises

1. Implement the full SVD from scratch without using power iteration. Instead, compute the eigendecomposition of $A^T A$ to get V and the singular values, then compute $U = A V \Sigma^{-1}$. Compare numerical accuracy with your power iteration version and with NumPy.
2. Load a real grayscale image (or convert one to grayscale). Compress it at ranks 1, 5, 10, 25, 50, 100. For each rank, compute the compression ratio and the relative error. Find the rank where the image becomes visually acceptable.
3. Build a tiny recommendation system. Create a 10x8 user-movie ratings matrix with some known entries. Fill missing entries with row means. Compute SVD and reconstruct a rank-3 approximation. Use the reconstructed matrix to predict the missing ratings. Verify that the predictions are reasonable.
4. Create a 100x50 document-term matrix with 3 synthetic topics. Each topic has 5 associated terms. Add noise. Apply SVD and verify that the top 3 singular values are much larger than the rest. Project documents into the 3D latent space and check that documents from the same topic cluster together.
5. Generate a clean low-rank matrix (rank 3, size 50x40) and add Gaussian noise at different levels ($\sigma = 0.1, 0.5, 1.0, 2.0$). For each noise level, find the optimal truncation rank by sweeping k from 1 to 40 and measuring reconstruction error against the clean matrix. Plot how the optimal k changes with noise level.

Key Terms

Term	What people say	What it actually means
SVD	"Factor any matrix"	Decompose A into $U \Sigma V^T$ where U and V are orthogonal and Σ is diagonal with non-negative entries. Works for any matrix of any shape.
Singular value	"How important this component is"	The i -th diagonal entry of Σ . Measures how much the matrix stretches along the i -th principal direction. Always non-negative, sorted in decreasing order.
Left singular vector	"Output direction"	A column of U . The direction in output space that the i -th right singular vector maps to (after scaling by σ_i).
Right singular vector	"Input direction"	A column of V . The direction in input space that the matrix maps to the i -th left singular vector (after scaling by σ_i).
Truncated SVD	"Low-rank approximation"	Keep only the top k singular values and their vectors. Produces the provably best rank- k approximation to the original matrix (Eckart-Young theorem).

Term	What people say	What it actually means
Rank	“True dimensionality”	The number of non-zero singular values. Tells you how many independent directions the matrix actually uses.
Pseudoinverse	“Generalized inverse”	$V \Sigma^+ U^T$. Inverts non-zero singular values, leaves zeros as zeros. Solves least-squares problems for non-square or singular matrices.
Condition number	“How sensitive to errors”	$\sigma_{\max} / \sigma_{\min}$. A large condition number means small input changes cause large output changes. SVD reveals this directly.
Latent factor	“Hidden variable”	A dimension in the low-rank space discovered by SVD. In recommendations, a latent factor might correspond to genre preference. In NLP, it might correspond to a topic.
Frobenius norm	“Total matrix size”	Square root of the sum of squared entries. Equals the square root of the sum of squared singular values. Used to measure approximation error.
Eckart-Young theorem	“SVD gives the best compression”	For any target rank k , the truncated SVD minimizes the approximation error over all possible rank- k matrices.
Power iteration	“Find the biggest eigenvector”	Repeatedly multiply a random vector by the matrix and normalize. Converges to the eigenvector with the largest eigenvalue. The building block of many SVD algorithms.

You are an expert at applying Singular Value Decomposition to practical engineering problems. When given a task involving matrices, data compression, noise, missing data, or linear systems, determine whether SVD is the right tool and how to apply it.

Decision Framework

Step 1: Identify the problem type

- **Data compression / dimensionality reduction:** Use truncated SVD. Keep top k singular values. Choose k by energy threshold (95% is a common target) or by downstream task performance.
- **Noise reduction:** Compute full SVD. Look for a gap in the singular value spectrum. Truncate below the gap. The gap separates signal from noise.
- **Missing data / recommendations:** Fill missing entries (row means or zeros), compute SVD, reconstruct with low rank. In production, use ALS or incremental SVD that handle missing data natively.
- **Least-squares / pseudoinverse:** Compute SVD. Invert non-zero singular values. Multiply $V \Sigma^+ U^T$ by the target vector. More stable than normal equations.

- **Text similarity / topic modeling:** Build term-document matrix. Apply SVD (this is LSA/LSI). Project documents and terms into the low-rank space. Use cosine similarity for comparisons.
- **Numerical rank determination:** Compute SVD. Count singular values above a threshold (relative to the largest). This is more reliable than row reduction.
- **Matrix norm computation:** Spectral norm = largest singular value. Frobenius norm = $\sqrt{\text{sum of squared singular values}}$. Nuclear norm = sum of singular values.
- **Condition number:** $\sigma_{\max} / \sigma_{\min}$. Tells you how sensitive the system is to perturbations.

Step 2: Choose the right variant

Situation	Method	Why
Dense matrix, full decomposition needed	<code>np.linalg.svd(A)</code> / <code>svd(A)</code> in Julia	Standard algorithm, numerically stable
Only top k components needed	<code>scipy.sparse.linalg.svds(A, k)</code>	Faster than full SVD when k is small
Sparse matrix	<code>scipy.sparse.linalg.svds</code>	Handles sparse storage efficiently
Streaming data	Incremental SVD / online SVD	Updates decomposition without recomputing from scratch
Missing data (recommendations)	ALS, Funk SVD, or NMF	Standard SVD requires a complete matrix
Very large matrix (millions of rows)	Randomized SVD (<code>sklearn.utils.extmath.randomized_svd</code>)	$O(mn \log k)$ instead of $O(mn \min(m,n))$
PCA on centered data	SVD of centered data matrix	Equivalent to eigendecomposition of covariance, but more stable

Step 3: Choose the rank k

- **Energy threshold:** Compute cumulative energy = $\text{sum}(\sigma_1^2 \dots \sigma_k^2) / \text{sum}(\text{all } \sigma^2)$. Stop when energy exceeds 0.95 (or 0.99 for high-fidelity tasks).
- **Gap detection:** Plot singular values. Look for a sharp drop. The gap indicates the boundary between signal and noise.
- **Cross-validation:** For downstream tasks, sweep k and measure performance on held-out data.
- **Elbow method:** Plot reconstruction error vs k. The elbow is where adding more components stops helping.
- **Domain knowledge:** If you know the data has d underlying factors, use $k = d$.

Step 4: Validate results

- **Reconstruction error:** Compute $\|A - A_k\| / \|A\|$. Should be small if the truncation is meaningful.
- **Explained variance:** For PCA/compression, report the fraction of total variance (energy) captured.
- **Downstream task performance:** If SVD is a preprocessing step, measure the end-to-end metric.
- **Visual inspection:** For images, compare original and reconstructed visually. For recommendations, check predictions against known ratings.

Common Mistakes

- Computing SVD via eigendecomposition of $A^T A$. This squares the condition number and loses numerical precision. Use a dedicated SVD routine.
- Using full SVD when only the top k components are needed. For large matrices, use truncated or randomized SVD.
- Applying SVD directly to a matrix with missing entries. Standard SVD requires a complete matrix. Use matrix completion methods (ALS, Funk SVD) instead.
- Ignoring centering. For PCA, the data must be centered (mean subtracted) before SVD. Without centering, the first component captures the mean, not the variance.
- Over-truncating. If you keep too few singular values, you lose signal. If you keep too many, you keep noise. Use energy thresholds or cross-validation.
- Confusing SVD with eigendecomposition. SVD works on any matrix (any shape, any rank). Eigendecomposition requires a square matrix with a full set of eigenvectors. For symmetric positive semi-definite matrices they are the same.

Code Patterns

Quick compression

```
U, S, Vt = np.linalg.svd(A, full_matrices=False)
k = np.searchsorted(np.cumsum(S**2) / np.sum(S**2), 0.95) + 1
A_compressed = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
```

Pseudoinverse for least squares

```
U, S, Vt = np.linalg.svd(A, full_matrices=False)
S_inv = np.array([1/s if s > 1e-10 else 0 for s in S])
x = Vt.T @ np.diag(S_inv) @ U.T @ b
```

Denoising

```
U, S, Vt = np.linalg.svd(noisy_data, full_matrices=False)
k = find_gap(S)
clean_data = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
```

Large-scale PCA

```
from sklearn.utils.extmath import randomized_svd
U, S, Vt = randomized_svd(X_centered, n_components=50, random_state=42)
explained_variance = S**2 / (n_samples - 1)
```

When NOT to use SVD

- The matrix is very sparse and you only need a few components. Use sparse eigensolvers directly.
- You need non-negative factors (topic modeling, spectral unmixing). Use NMF instead.
- The data has strong non-linear structure that linear methods cannot capture. Use autoencoders or manifold learning.
- You need real-time updates on streaming data and the matrix changes constantly. Use incremental/online SVD or approximate methods.
- The matrix fits in memory but is so large that even randomized SVD is too slow. Consider sketching methods or sampling-based approaches.

Computational Cost

Method	Time	Space
Full SVD of $m \times n$ matrix	$O(mn \min(m,n))$	$O(mn)$
Truncated SVD (top k)	$O(mnk)$	$O((m+n)k)$
Randomized SVD (top k)	$O(mn \log k)$	$O((m+n)k)$
Power iteration (1 vector)	$O(mn * \text{iters})$	$O(m+n)$

For a 10000 x 5000 matrix: - Full SVD: ~250 billion operations - Truncated SVD ($k=50$): ~2.5 billion operations - Randomized SVD ($k=50$): ~500 million operations

Choose the method that matches your scale and accuracy requirements.

Quiz

Pre-lesson check

1. What advantage does SVD have over eigendecomposition?
 - a) SVD is always faster to compute
 - b) SVD works on any matrix of any shape, while eigendecomposition requires square matrices with full eigenvector sets **(correct)**
 - c) SVD produces smaller output matrices
 - d) SVD does not require numerical computation
Explanation: Eigendecomposition only works on square matrices that have n linearly independent eigenvectors. SVD decomposes ANY $m \times n$ matrix into $U \Sigma^ V^T$ with no restrictions on shape or rank.**
2. What does 'low-rank approximation' mean?
 - a) Removing rows with low values from a matrix
 - b) Approximating a matrix by keeping only its most important components, producing a simpler matrix with fewer independent directions **(correct)**
 - c) Converting a matrix to a lower precision data type
 - d) Sorting the rows of a matrix by their magnitude
Explanation: A rank- k approximation keeps only the top k singular values and their vectors, discarding the rest. The Eckart-Young theorem proves this is the BEST possible approximation of that rank.

Post-lesson check

3. In SVD $A = U * \Sigma * V^T$, what geometric operation does each factor represent?
 - a) U scales, Σ rotates, V^T translates
 - b) V^T rotates in input space, Σ scales along principal axes, U rotates into output space **(correct)**
 - c) U compresses, Σ expands, V^T normalizes

- d) All three factors perform the same operation: rotation
Explanation: SVD reveals that every matrix performs: (1) V^T rotates inputs to align with principal directions, (2) Σ stretches/compresses along each axis, (3) U rotates the result into the output space. Rotate, scale, rotate.
4. Why does sklearn implement PCA using SVD instead of eigendecomposition of the covariance matrix?
- a) SVD produces different results that are more accurate for ML
- b) SVD works directly on the data matrix without forming the covariance matrix, avoiding squaring the condition number and improving numerical stability (**correct**)
- c) SVD is easier to parallelize on GPUs
- d) SVD does not require centering the data
Explanation: Forming $A^T A$ squares the singular values (and condition number). If A has singular values $[1000, 0.001]$, $A^T A$ has eigenvalues $[10^6, 10^{-6}]$ – 6 digits of precision lost. SVD avoids this by working directly on A .
5. How does truncated SVD enable recommendation systems to predict missing ratings?
- a) It fills missing entries with the average rating
- b) It decomposes the ratings matrix into latent user and movie profiles; the dot product of a user profile with a movie profile predicts the missing rating (**correct**)
- c) It clusters similar users together and copies their ratings
- d) It trains a neural network on the observed ratings
Explanation: SVD decomposes the ratings matrix into user profiles (U), latent factor importance (Σ), and movie profiles (V^T). The low-rank reconstruction fills in missing entries based on the latent factors (genre, era, style) that explain user preferences.
-

Tensor Operations

Tensors are the common language between data and deep learning. Every image, every sentence, every gradient flows through them.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations) **Time:** ~90 minutes

Learning Objectives

- Implement a tensor class with shape, strides, reshape, transpose, and element-wise operations from scratch
- Apply broadcasting rules to operate on tensors of different shapes without copying data
- Write einsum expressions for dot products, matrix multiplications, outer products, and batched operations
- Trace the exact tensor shapes through every step of multi-head attention

The Problem

You build a transformer. The forward pass looks clean. You run it and get: `RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x768 and 512x768)`. You stare at the shapes. You try a transpose. Now it says `Expected 4D input (got 3D input)`. You add an `unsqueeze`. Something else breaks.

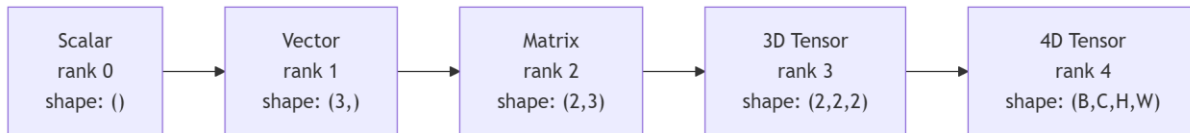
Shape errors are the most common bug in deep learning code. They are not hard conceptually – each operation has a shape contract – but they multiply fast. A transformer has dozens of reshapes, transposes, and broadcasts chained together. One wrong axis and the error cascades. Worse, some shape mistakes do not throw errors at all. They silently produce garbage by broadcasting along the wrong dimension or summing over the wrong axis.

Matrices handle pairwise relationships between two sets of things. Real data does not fit into two dimensions. A batch of 32 RGB images at 224x224 is a 4D tensor: `(32, 3, 224, 224)`. Self-attention with 12 heads is also 4D: `(batch, heads, seq_len, head_dim)`. You need a data structure that generalizes to any number of dimensions, with operations that compose cleanly across all of them. That structure is the tensor. Master its operations and shape errors become trivially debuggable.

The Concept

What a tensor is

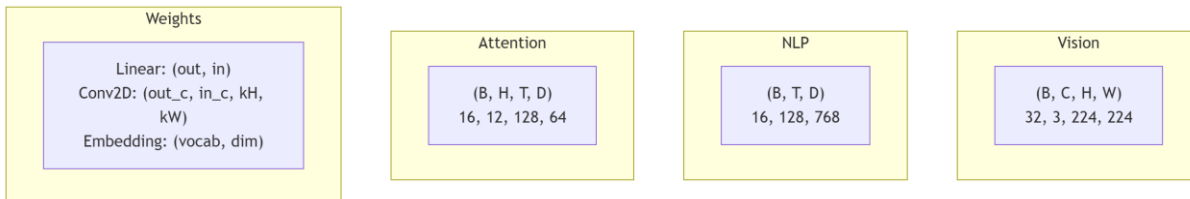
A tensor is a multi-dimensional array of numbers with a uniform data type. The number of dimensions is the **rank** (or **order**). Each dimension is an **axis**. The **shape** is a tuple listing the size along each axis.



Total elements = product of all sizes. A shape (2, 3, 4) holds $2 * 3 * 4 = 24$ elements.

Tensor shapes in deep learning

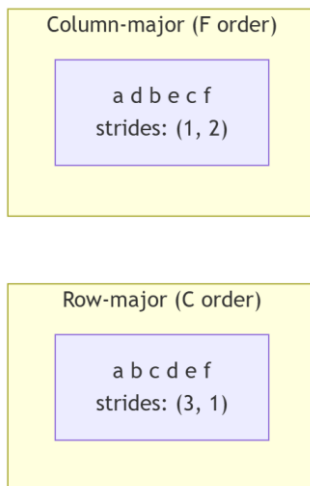
Different data types map to specific tensor shapes by convention.



PyTorch uses NCHW (channels-first). TensorFlow defaults to NHWC (channels-last). Mismatched layouts cause silent slowdowns or errors.

How memory layout works

A 2D array in memory is a 1D sequence of bytes. **Strides** tell you how many elements to skip to move one step along each axis.



Transpose does not move data. It swaps the strides, making the tensor **non-contiguous** – the elements for a row are no longer adjacent in memory.

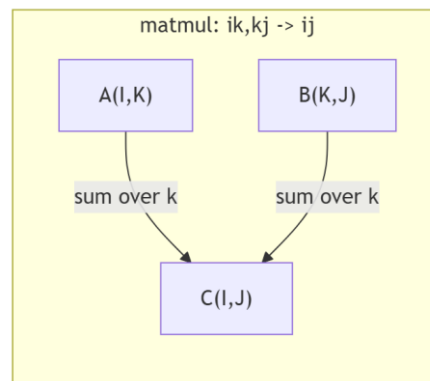
Broadcasting rules

Broadcasting lets you operate on tensors of different shapes without copying data. Align shapes from the right. Two dimensions are compatible when they are equal or one is 1. Fewer dimensions get padded with 1s on the left.

Tensor A:	(8, 1, 6, 1)
Tensor B:	(7, 1, 5)
Padded B:	(1, 7, 1, 5)
Result:	(8, 7, 6, 5)

Einsum: the universal tensor operation

Einstein summation labels each axis with a letter. Axes in the input but not the output get summed. Axes in both are kept.



Key patterns: $i, i \rightarrow$ (dot product), $i, j \rightarrow ij$ (outer product), $ii \rightarrow$ (trace), $ij \rightarrow ji$ (transpose), $bij, bjk \rightarrow bik$ (batch matmul), $bhtd, bhjd \rightarrow bhts$ (attention scores).

Build It

The code lives in `code/tensors.py`. Each step references the implementation there.

Step 1: Tensor storage and strides

A tensor stores a flat list of numbers plus shape metadata. Strides tell the indexing logic how to map multi-dimensional indices to flat positions.

```
class Tensor:
    def __init__(self, data, shape=None):
        if isinstance(data, (list, tuple)):
            self._data, self._shape = self._flatten_nested(data)
        elif isinstance(data, np.ndarray):
            self._data = data.flatten().tolist()
            self._shape = tuple(data.shape)
        else:
            self._data = [data]
            self._shape = ()

        if shape is not None:
            total = reduce(lambda a, b: a * b, shape, 1)
            if total != len(self._data):
```

```

        raise ValueError(
            f"Cannot reshape {len(self._data)} elements into shape {shape}"
        )
        self._shape = tuple(shape)

        self._strides = self._compute_strides(self._shape)

    @staticmethod
    def _compute_strides(shape):
        if len(shape) == 0:
            return ()
        strides = [1] * len(shape)
        for i in range(len(shape) - 2, -1, -1):
            strides[i] = strides[i + 1] * shape[i + 1]
        return tuple(strides)

```

For shape (3, 4), strides are (4, 1) – skip 4 elements to advance one row, skip 1 element to advance one column.

Step 2: Reshape, squeeze, unsqueeze

Reshape changes the shape without changing element order. The total number of elements must stay the same. Use -1 for one dimension to infer its size.

```

t = Tensor(list(range(12)), shape=(2, 6))
r = t.reshape((3, 4))
r = t.reshape((-1, 3))

```

Squeeze removes axes of size 1. Unsqueeze inserts one. Unsquizing is critical for broadcasting – a bias vector (D,) added to a batch (B, T, D) needs unsquizing to (1, 1, D).

```

t = Tensor(list(range(6)), shape=(1, 3, 1, 2))
s = t.squeeze()
v = Tensor([1, 2, 3])
u = v.unsqueeze(0)

```

Step 3: Transpose and permute

Transpose swaps two axes. Permute reorders all axes. This is how you convert between NCHW and NHWC.

```

mat = Tensor(list(range(6)), shape=(2, 3))
tr = mat.transpose(0, 1)

t4d = Tensor(list(range(24)), shape=(1, 2, 3, 4))
perm = t4d.permute((0, 2, 3, 1))

```

After transpose or permute, the tensor is non-contiguous in memory. In PyTorch, view fails on non-contiguous tensors – use reshape or call .contiguous() first.

Step 4: Element-wise operations and reductions

Element-wise ops (add, multiply, subtract) apply independently to each element and preserve shape. Reductions (sum, mean, max) collapse one or more axes.

```
a = Tensor([[1, 2], [3, 4]])
b = Tensor([[10, 20], [30, 40]])
c = a + b
d = a * 2
s = a.sum(axis=0)
```

Global average pooling in a CNN: $(B, C, H, W).mean(axis=[2, 3])$ produces (B, C) . Sequence mean pooling in NLP: $(B, T, D).mean(axis=1)$ produces (B, D) .

Step 5: Broadcasting with NumPy

The `demo_broadcasting_numpy()` function in `tensors.py` shows the core patterns.

```
activations = np.random.randn(4, 3)
bias = np.array([0.1, 0.2, 0.3])
result = activations + bias

images = np.random.randn(2, 3, 4, 4)
scale = np.array([0.5, 1.0, 1.5]).reshape(1, 3, 1, 1)
result = images * scale

a = np.array([1, 2, 3]).reshape(-1, 1)
b = np.array([10, 20, 30, 40]).reshape(1, -1)
outer = a * b
```

Pairwise distance via broadcasting: reshape $(M, 2)$ to $(M, 1, 2)$ and $(N, 2)$ to $(1, N, 2)$, subtract, square, sum along last axis, take square root. Result: (M, N) .

Step 6: Einsum operations

The `demo_einsum()` and `demo_einsum_gallery()` functions walk through every common pattern.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])
dot = np.einsum("i,i->", a, b)

A = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
B = np.array([[7, 8, 9], [10, 11, 12]], dtype=float)
matmul = np.einsum("ik,kj->ij", A, B)

batch_A = np.random.randn(4, 3, 5)
batch_B = np.random.randn(4, 5, 2)
batch_mm = np.einsum("bij,bjk->bik", batch_A, batch_B)
```

The computational cost of a contraction is the product of all index sizes (kept and summed). For $b_{ij}, b_{jk} \rightarrow b_{ik}$ with $B=32, I=128, J=64, K=128$: $32 * 128 * 64 * 128 = 33,554,432$ multiply-adds.

Step 7: Attention mechanism via einsum

The `demo_attention_einsum()` function implements multi-head attention end to end.

```
B, H, T, D = 2, 4, 8, 16
E = H * D

X = np.random.randn(B, T, E)
W_q = np.random.randn(E, E) * 0.02

Q = np.einsum("bte,ek->bt", X, W_q)
Q = Q.reshape(B, T, H, D).transpose(0, 2, 1, 3)

scores = np.einsum("bhtd,bhkd->bht", Q, K) / np.sqrt(D)
weights = softmax(scores, axis=-1)
attn_output = np.einsum("bht,bhkd->bht", weights, V)

concat = attn_output.transpose(0, 2, 1, 3).reshape(B, T, E)
output = np.einsum("bte,ek->bt", concat, W_o)
```

Every step is a tensor operation: projection (matmul via einsum), head splitting (reshape + transpose), attention scores (batch matmul via einsum), weighted sum (batch matmul via einsum), head merging (transpose + reshape), output projection (matmul via einsum).

Use It

Scratch vs NumPy

Operation	Scratch (Tensor class)	NumPy
Create	<code>Tensor([[1,2],[3,4]])</code>	<code>np.array([[1,2],[3,4]])</code>
Reshape	<code>t.reshape((3,4))</code>	<code>a.reshape(3,4)</code>
Transpose	<code>t.transpose(0,1)</code>	<code>a.T</code> or <code>a.transpose(0,1)</code>
Squeeze	<code>t.squeeze(0)</code>	<code>np.squeeze(a, 0)</code>
Sum	<code>t.sum(axis=0)</code>	<code>a.sum(axis=0)</code>
Einsum	N/A	<code>np.einsum("ij,jk->ik", a, b)</code>

Scratch vs PyTorch

```
import torch

t = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
t.shape
t.stride()
t.is_contiguous()

t.reshape(3, 2)
t.unsqueeze(0)
t.transpose(0, 1)
t.transpose(0, 1).contiguous()

torch.einsum("ik,kj->ij", A, B)
```

PyTorch adds autograd, GPU support, and optimized BLAS kernels. The shape semantics are identical. If you understand the scratch version, PyTorch shape errors become readable.

Every neural network layer as a tensor operation

Operation	Tensor Form	Einsum
Linear layer	$Y = X @ W.T + b$	"bd,od->bo" + bias
Attention QKV	$Q = X @ W_q$	"btd,dh->bth"
Attention scores	$Q @ K.T / \sqrt{d}$	"bhtd,bhsd->bhts"
Attention output	$\text{softmax}(\text{scores}) @ V$	"bhts,bhsd->bhtd"
Batch norm	$(X - \mu) / \sigma * \gamma$	element-wise + broadcast
Softmax	$\exp(x) / \text{sum}(\exp(x))$	element-wise + reduction

Exercises

1. **Easy – Reshape round-trip.** Take a tensor of shape (2, 3, 4). Reshape it to (6, 4), then to (24,), then back to (2, 3, 4). Verify element order is preserved at each step by printing the flat data.
2. **Medium – Implement broadcasting.** Extend the Tensor class with a `broadcast_to(shape)` method that expands dimensions of size 1 to match a target shape. Then modify `_elementwise_op` to automatically broadcast before operating. Test with shapes (3, 1) and (1, 4) producing (3, 4).
3. **Hard – Build einsum from scratch.** Implement a basic `einsum(subscripts, *tensors)` function that handles at least: dot product (`i, i->`), matrix multiply (`ij, jk->ik`), outer product

($i, j \rightarrow ij$), and transpose ($ij \rightarrow ji$). Parse the subscript string, identify contracted indices, and loop over all index combinations. Compare your results against `np.einsum`.

4. **Hard – Attention shape tracker.** Write a function that takes `batch_size`, `seq_len`, `embed_dim`, and `num_heads` as inputs and prints the exact shape at every step of multi-head attention: input, Q/K/V projection, head split, attention scores, softmax weights, weighted sum, head merge, output projection. Verify against the `demo_attention_einsum()` output.

Key Terms

Term	What people say	What it actually means
Tensor	“A matrix but more dimensions”	A multi-dimensional array with uniform type and defined shape, strides, and operations
Rank	“The number of dimensions”	The number of axes. A matrix has rank 2, not rank equal to its matrix rank
Shape	“The size of the tensor”	A tuple listing the size along each axis. (2, 3) means 2 rows, 3 columns
Stride	“How memory is laid out”	The number of elements to skip to advance one position along each axis
Broadcasting	“It just works when shapes differ”	A strict set of rules: align from right, dimensions must be equal or one must be 1
Contiguous	“The tensor is normal”	Elements stored sequentially in memory with no gaps or reordering from the logical layout
Einsum	“A fancy way to write matmul”	A general notation that expresses any tensor contraction, outer product, trace, or transpose in one line
View	“Same as reshape”	A tensor sharing the same memory buffer but with different shape/stride metadata. Fails on non-contiguous data
Contraction	“Summing over an index”	The general operation where a shared index between tensors is multiplied and summed, producing a lower-rank result
NCHW / NHWC	“PyTorch vs TensorFlow format”	Memory layout conventions for image tensors. NCHW puts channels before spatial dims, NHWC puts them after

Code Appendix: `tensors.py`

```
import numpy as np
from functools import reduce
```

```

from itertools import product as iterproduct

class Tensor:
    def __init__(self, data, shape=None):
        if isinstance(data, (list, tuple)):
            self._data, self._shape = self._flatten_nested(data)
        elif isinstance(data, np.ndarray):
            self._data = data.flatten().tolist()
            self._shape = tuple(data.shape)
        else:
            self._data = [data]
            self._shape = ()

        if shape is not None:
            total = reduce(lambda a, b: a * b, shape, 1)
            if total != len(self._data):
                raise ValueError(
                    f"Cannot reshape {len(self._data)} elements into shape {shape}"
                )
            self._shape = tuple(shape)

        self._strides = self._compute_strides(self._shape)

    def _flatten_nested(self, data):
        if not isinstance(data, (list, tuple)):
            return [data], ()
        if len(data) == 0:
            return [], (0,)

        sub_results = [self._flatten_nested(item) for item in data]
        sub_shape = sub_results[0][1]
        for i, (_, s) in enumerate(sub_results):
            if s != sub_shape:
                raise ValueError(
                    f"Inconsistent shapes at index {i}: {s} vs {sub_shape}"
                )

        flat = []
        for sub_data, _ in sub_results:
            flat.extend(sub_data)

        return flat, (len(data),) + sub_shape

    @staticmethod
    def _compute_strides(shape):
        if len(shape) == 0:
            return ()
        strides = [1] * len(shape)
        for i in range(len(shape) - 2, -1, -1):
            strides[i] = strides[i + 1] * shape[i + 1]
        return tuple(strides)

```

```

@property
def shape(self):
    return self._shape

@property
def rank(self):
    return len(self._shape)

@property
def size(self):
    return len(self._data)

@property
def strides(self):
    return self._strides

def _flat_index(self, indices):
    if len(indices) != len(self._shape):
        raise IndexError(
            f"Expected {len(self._shape)} indices, got {len(indices)}"
        )
    idx = 0
    for i, (ind, stride) in enumerate(zip(indices, self._strides)):
        if ind < 0 or ind >= self._shape[i]:
            raise IndexError(
                f"Index {ind} out of range for axis {i} with size {self._shape[i]}
    }"
        )
    idx += ind * stride
    return idx

def __getitem__(self, indices):
    if not isinstance(indices, tuple):
        indices = (indices,)
    if len(indices) == len(self._shape):
        return self._data[self._flat_index(indices)]
    raise IndexError("Partial indexing not supported in this basic implementation")

def __setitem__(self, indices, value):
    if not isinstance(indices, tuple):
        indices = (indices,)
    self._data[self._flat_index(indices)] = value

def reshape(self, new_shape):
    new_shape = list(new_shape)
    neg_idx = -1
    known_product = 1
    for i, s in enumerate(new_shape):
        if s == -1:
            if neg_idx != -1:
                raise ValueError("Only one dimension can be -1")
            neg_idx = i

```



```

        else:
            known_product *= s

    if neg_idx != -1:
        new_shape[neg_idx] = self.size // known_product

    total = reduce(lambda a, b: a * b, new_shape, 1)
    if total != self.size:
        raise ValueError(
            f"Cannot reshape {self.size} elements into shape {tuple(new_shape)}"
        )

    result = Tensor.__new__(Tensor)
    result._data = self._data[:]
    result._shape = tuple(new_shape)
    result._strides = self._compute_strides(result._shape)
    return result

def squeeze(self, dim=None):
    if dim is not None:
        if self._shape[dim] != 1:
            return self.reshape(self._shape)
        new_shape = list(self._shape)
        new_shape.pop(dim)
        return self.reshape(tuple(new_shape) if new_shape else ())
    new_shape = tuple(s for s in self._shape if s != 1)
    if not new_shape:
        new_shape = ()
    return self.reshape(new_shape)

def unsqueeze(self, dim):
    if dim < 0:
        dim = len(self._shape) + 1 + dim
    new_shape = list(self._shape)
    new_shape.insert(dim, 1)
    return self.reshape(tuple(new_shape))

def transpose(self, dim0, dim1):
    perm = list(range(self.rank))
    perm[dim0], perm[dim1] = perm[dim1], perm[dim0]
    return self.permute(perm)

def permute(self, dims):
    if sorted(dims) != list(range(self.rank)):
        raise ValueError(f"Invalid permutation {dims} for rank {self.rank}")

    new_shape = tuple(self._shape[d] for d in dims)
    result = Tensor.__new__(Tensor)
    result._shape = new_shape
    result._strides = self._compute_strides(new_shape)
    result._data = [0] * self.size

    old_strides = self._strides

```

```

    for old_indices in iterproduct(*(range(s) for s in self._shape)):
        new_indices = tuple(old_indices[d] for d in dims)
        old_flat = sum(i * s for i, s in zip(old_indices, old_strides))
        new_flat = sum(
            i * s for i, s in zip(new_indices, result._strides)
        )
        result._data[new_flat] = self._data[old_flat]

    return result

def flatten(self, start_dim=0, end_dim=-1):
    if end_dim < 0:
        end_dim = self.rank + end_dim
    new_shape = (
        list(self._shape[:start_dim])
        + [reduce(lambda a, b: a * b, self._shape[start_dim:end_dim + 1], 1)]
        + list(self._shape[end_dim + 1:])
    )
    return self.reshape(tuple(new_shape))

def _elementwise_op(self, other, op):
    if isinstance(other, (int, float)):
        result_data = [op(x, other) for x in self._data]
        return Tensor(result_data, shape=self._shape)
    if not isinstance(other, Tensor):
        raise TypeError(f"Unsupported type {type(other)}")
    if self._shape != other._shape:
        raise ValueError(
            f"Shape mismatch: {self._shape} vs {other._shape}. "
            "Use broadcast() first."
        )
    result_data = [op(a, b) for a, b in zip(self._data, other._data)]
    return Tensor(result_data, shape=self._shape)

def __add__(self, other):
    return self._elementwise_op(other, lambda a, b: a + b)

def __mul__(self, other):
    return self._elementwise_op(other, lambda a, b: a * b)

def __sub__(self, other):
    return self._elementwise_op(other, lambda a, b: a - b)

def sum(self, axis=None):
    if axis is None:
        return sum(self._data)
    if axis < 0:
        axis = self.rank + axis
    new_shape = list(self._shape)
    axis_size = new_shape.pop(axis)

    result_size = reduce(lambda a, b: a * b, new_shape, 1)
    result_data = [0.0] * result_size

```

```

result_strides = self._compute_strides(tuple(new_shape))

for indices in iterproduct(*(range(s) for s in self._shape)):
    old_flat = sum(i * s for i, s in zip(indices, self._strides))
    new_indices = indices[:axis] + indices[axis + 1:]
    if new_indices:
        new_flat = sum(
            i * s for i, s in zip(new_indices, result_strides)
        )
    else:
        new_flat = 0
    result_data[new_flat] += self._data[old_flat]

if not new_shape:
    return result_data[0]
return Tensor(result_data, shape=tuple(new_shape))

def to_list(self):
    if self.rank == 0:
        return self._data[0]
    return self._build_nested(self._data, self._shape, 0)

def _build_nested(self, data, shape, offset):
    if len(shape) == 1:
        return data[offset:offset + shape[0]]
    result = []
    stride = reduce(lambda a, b: a * b, shape[1:], 1)
    for i in range(shape[0]):
        result.append(self._build_nested(data, shape[1:], offset + i * stride))
    return result

def __repr__(self):
    return f"Tensor(shape={self._shape}, data={self.to_list()})"

def to_numpy(self):
    return np.array(self._data).reshape(self._shape)

def demo_basic_tensor():
    print("=" * 60)
    print("BASIC TENSOR OPERATIONS")
    print("=" * 60)

    scalar = Tensor(3.14)
    print(f"Scalar: shape={scalar.shape}, rank={scalar.rank}, value={scalar.to_list()}")

    vector = Tensor([1, 2, 3, 4, 5])
    print(f"Vector: shape={vector.shape}, rank={vector.rank}")

    matrix = Tensor([[1, 2, 3], [4, 5, 6]])
    print(f"Matrix: shape={matrix.shape}, rank={matrix.rank}")

```

```

print(f" matrix[0, 1] = {matrix[0, 1]}")
print(f" matrix[1, 2] = {matrix[1, 2]}")

tensor_3d = Tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(f"3D Tensor: shape={tensor_3d.shape}, rank={tensor_3d.rank}")
print(f" tensor[1, 0, 1] = {tensor_3d[1, 0, 1]}")

print(f"\nStrides for shape {matrix.shape}: {matrix.strides}")
print(f"Strides for shape {tensor_3d.shape}: {tensor_3d.strides}")
print()

def demo_reshape_operations():
    print("=" * 60)
    print("RESHAPE OPERATIONS")
    print("=" * 60)

    data = Tensor(list(range(12)), shape=(2, 6))
    print(f"Original: shape={data.shape}")
    print(f" {data.to_list()}")

    r1 = data.reshape((3, 4))
    print(f"\nReshaped to (3, 4): {r1.to_list()}")

    r2 = data.reshape((2, 2, 3))
    print(f"Reshaped to (2, 2, 3): {r2.to_list()}")

    r3 = data.reshape((-1, 3))
    print(f"Reshaped to (-1, 3): shape={r3.shape}, {r3.to_list()}")

    t = Tensor(list(range(6)), shape=(1, 3, 1, 2))
    print(f"\nBefore squeeze: shape={t.shape}")
    s = t.squeeze()
    print(f"After squeeze(): shape={s.shape}")
    s0 = t.squeeze(dim=0)
    print(f"After squeeze(0): shape={s0.shape}")

    v = Tensor([1, 2, 3])
    print(f"\nVector shape: {v.shape}")
    print(f"unsqueeze(0): {v.unsqueeze(0).shape}")
    print(f"unsqueeze(1): {v.unsqueeze(1).shape}")
    print(f"unsqueeze(-1): {v.unsqueeze(-1).shape}")

    mat = Tensor(list(range(6)), shape=(2, 3))
    print(f"\nOriginal: shape={mat.shape}, {mat.to_list()}")
    tr = mat.transpose(0, 1)
    print(f"Transpose(0,1): shape={tr.shape}, {tr.to_list()}")

    t4d = Tensor(list(range(24)), shape=(1, 2, 3, 4))

```

```

perm = t4d.permute((0, 2, 3, 1))
print(f"\nPermute (1,2,3,4) -> (0,2,3,1): {t4d.shape} -> {perm.shape}")

batch_conv = Tensor(list(range(2 * 4 * 4 * 2)), shape=(2, 4, 4, 2))
flat = batch_conv.flatten(start_dim=1)
print(f"\nFlatten (2,4,4,2) from dim 1: shape={flat.shape}")
print()

def demo_broadcasting_numpy():
    print("=" * 60)
    print("BROADCASTING (NumPy)")
    print("=" * 60)

    print("\n--- Adding bias to batch ---")
    activations = np.random.randn(4, 3)
    bias = np.array([0.1, 0.2, 0.3])
    result = activations + bias
    print(f"activations: {activations.shape}")
    print(f"bias: {bias.shape}")
    print(f"result: {result.shape}")

    print("\n--- Channel-wise scaling ---")
    images = np.random.randn(2, 3, 4, 4)
    scale = np.array([0.5, 1.0, 1.5]).reshape(1, 3, 1, 1)
    result = images * scale
    print(f"images: {images.shape}")
    print(f"scale: {scale.shape}")
    print(f"result: {result.shape}")

    print("\n--- Outer product via broadcasting ---")
    a = np.array([1, 2, 3]).reshape(-1, 1)
    b = np.array([10, 20, 30, 40]).reshape(1, -1)
    outer = a * b
    print(f"a: {a.shape}, b: {b.shape}")
    print(f"outer product: {outer.shape}")
    print(outer)

    print("\n--- Pairwise distances via broadcasting ---")
    points_a = np.random.randn(5, 2)
    points_b = np.random.randn(3, 2)
    diff = points_a[:, np.newaxis, :] - points_b[np.newaxis, :, :]
    distances = np.sqrt(np.sum(diff ** 2, axis=-1))
    print(f"points_a: {points_a.shape}")
    print(f"points_b: {points_b.shape}")
    print(f"diff: {diff.shape}")
    print(f"distances: {distances.shape}")

    print("\n--- Broadcasting rules check ---")

```

```

shapes_to_test = [
    ((8, 1, 6, 1), (7, 1, 5)),
    ((3, 4), (4,)),
    ((2, 1, 3), (1, 4, 3)),
    ((3, 1), (1, 4)),
]
for sa, sb in shapes_to_test:
    a = np.zeros(sa)
    b = np.zeros(sb)
    try:
        result = a + b
        print(f" {sa} + {sb} -> {result.shape}")
    except ValueError as e:
        print(f" {sa} + {sb} -> ERROR: {e}")

print()

def demo_einsum():
    print("=" * 60)
    print("EINSUM NOTATION")
    print("=" * 60)

    print("\n--- Dot product: i,i-> ---")
    a = np.array([1.0, 2.0, 3.0])
    b = np.array([4.0, 5.0, 6.0])
    result = np.einsum("i,i->", a, b)
    verify = np.dot(a, b)
    print(f" einsum: {result}, np.dot: {verify}")

    print("\n--- Outer product: i,j->ij ---")
    a = np.array([1.0, 2.0, 3.0])
    b = np.array([10.0, 20.0])
    result = np.einsum("i,j->ij", a, b)
    verify = np.outer(a, b)
    print(f" einsum:\n{result}")
    print(f" np.outer:\n{verify}")

    print("\n--- Matrix multiply: ik,kj->ij ---")
    A = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
    B = np.array([[7, 8, 9], [10, 11, 12]], dtype=float)
    result = np.einsum("ik,kj->ij", A, B)
    verify = A @ B
    print(f" A: {A.shape}, B: {B.shape}")
    print(f" einsum result:\n{result}")
    print(f" matmul result:\n{verify}")

    print("\n--- Trace: ii-> ---")
    M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]], dtype=float)
    result = np.einsum("ii->", M)

```

```

verify = np.trace(M)
print(f" einsum: {result}, np.trace: {verify}")

print("\n--- Transpose: ij->ji ---")
result = np.einsum("ij->ji", A)
verify = A.T
print(f" einsum:\n{result}")

print("\n--- Diagonal: ii->i ---")
result = np.einsum("ii->i", M)
verify = np.diag(M)
print(f" einsum: {result}, np.diag: {verify}")

print("\n--- Sum over axis: ij->i (row sums) ---")
result = np.einsum("ij->i", A)
verify = A.sum(axis=1)
print(f" einsum: {result}, sum(axis=1): {verify}")

print("\n--- Batch matrix multiply: bij,bjk->bik ---")
batch_A = np.random.randn(4, 3, 5)
batch_B = np.random.randn(4, 5, 2)
result = np.einsum("bij,bjk->bik", batch_A, batch_B)
verify = np.matmul(batch_A, batch_B)
print(f" batch_A: {batch_A.shape}, batch_B: {batch_B.shape}")
print(f" einsum result: {result.shape}")
print(f" matmul result: {verify.shape}")
print(f" match: {np.allclose(result, verify)}")

print("\n--- Hadamard (element-wise) product: ij,ij->ij ---")
C = np.array([[1, 2], [3, 4]], dtype=float)
D = np.array([[5, 6], [7, 8]], dtype=float)
result = np.einsum("ij,ij->ij", C, D)
verify = C * D
print(f" einsum:\n{result}")
print(f" element-wise:\n{verify}")

print()

def demo_attention_einsum():
    print("=" * 60)
    print("ATTENTION MECHANISM via EINSUM")
    print("=" * 60)

    B, H, T, D = 2, 4, 8, 16
    E = H * D

    np.random.seed(42)

    X = np.random.randn(B, T, E)

```

```

print(f"Input X: {X.shape} (batch, seq_len, embed_dim)")

W_q = np.random.randn(E, E) * 0.02
W_k = np.random.randn(E, E) * 0.02
W_v = np.random.randn(E, E) * 0.02

Q = np.einsum("bte,ek->bt", X, W_q)
K = np.einsum("bte,ek->bt", X, W_k)
V = np.einsum("bte,ek->bt", X, W_v)
print(f"Q, K, V: {Q.shape}")

Q = Q.reshape(B, T, H, D).transpose(0, 2, 1, 3)
K = K.reshape(B, T, H, D).transpose(0, 2, 1, 3)
V = V.reshape(B, T, H, D).transpose(0, 2, 1, 3)
print(f"After split heads: Q={Q.shape}, K={K.shape}, V={V.shape}")

scores = np.einsum("bhtd,bhsd->bhts", Q, K) / np.sqrt(D)
print(f"Attention scores: {scores.shape}")

def softmax(x, axis=-1):
    e = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e / np.sum(e, axis=axis, keepdims=True)

weights = softmax(scores, axis=-1)
print(f"Attention weights: {weights.shape}")
print(f" weights sum per query (should be 1.0): {weights[0, 0, 0].sum():.6f}")

attn_output = np.einsum("bhts,bhsd->bhtd", weights, V)
print(f"Attention output: {attn_output.shape}")

concat = attn_output.transpose(0, 2, 1, 3).reshape(B, T, E)
print(f"Concatenated heads: {concat.shape}")

W_o = np.random.randn(E, E) * 0.02
output = np.einsum("bte,ek->bt", concat, W_o)
print(f"Final output: {output.shape}")
print()

def demo_memory_layout():
    print("=" * 60)
    print("MEMORY LAYOUT")
    print("=" * 60)

    a = np.array([[1, 2, 3], [4, 5, 6]])
    print(f"Array shape: {a.shape}")
    print(f"Strides (bytes): {a.strides}")
    print(f"Strides (elements): {tuple(s // a.itemsize for s in a.strides)}")
    print(f"C-contiguous: {a.flags['C_CONTIGUOUS']}")
    print(f"F-contiguous: {a.flags['F_CONTIGUOUS']}")

```



```

print(f"Memory layout: {a.ravel()}")

print("\n--- After transpose ---")
b = a.T
print(f"Transposed shape: {b.shape}")
print(f"Strides (bytes): {b.strides}")
print(f"C-contiguous: {b.flags['C_CONTIGUOUS']}")
print(f"F-contiguous: {b.flags['F_CONTIGUOUS']}")
print(f"Note: transpose swapped strides without moving data")

print("\n--- Contiguous copy ---")
c = np.ascontiguousarray(b)
print(f"After ascontiguousarray:")
print(f"  C-contiguous: {c.flags['C_CONTIGUOUS']}")
print(f"  Strides: {c.strides}")

print("\n--- Row-major vs Column-major ---")
row_major = np.array([[1, 2, 3], [4, 5, 6]], order='C')
col_major = np.array([[1, 2, 3], [4, 5, 6]], order='F')
print(f"Row-major (C) flat: {row_major.ravel(order='K')}")
print(f"Col-major (F) flat: {col_major.ravel(order='K')}")
print(f"Row-major strides: {row_major.strides}")
print(f"Col-major strides: {col_major.strides}")

print("\n--- Stride tricks: creating a view ---")
x = np.arange(12).reshape(3, 4)
print(f"Original:\n{x}")
print(f"Strides: {x.strides}")
sliced = x[:, ::2]
print(f"Every other column (x[:, ::2]):\n{sliced}")
print(f"Sliced strides: {sliced.strides}")
print(f"Sliced is contiguous: {sliced.flags['C_CONTIGUOUS']}")
print()

def demo_ai_tensor_shapes():
    print("=" * 60)
    print("COMMON AI TENSOR SHAPES")
    print("=" * 60)

    print("\n--- Vision: (B, C, H, W) ---")
    B, C, H, W = 32, 3, 224, 224
    images = np.random.randn(B, C, H, W).astype(np.float32)
    print(f"Image batch: {images.shape}")
    print(f"  Total elements: {images.size:,}")
    print(f"  Memory (float32): {images.nbytes / 1024 / 1024:.1f} MB")

    kernel = np.random.randn(64, 3, 3, 3).astype(np.float32)

```

```

print(f"Conv2D kernel (64 filters, 3x3): {kernel.shape}")

print("\n--- NLP: (B, T, D) ---")
B, T, D = 16, 512, 768
embeddings = np.random.randn(B, T, D).astype(np.float32)
print(f"Token embeddings: {embeddings.shape}")
print(f" Total elements: {embeddings.size:,}")
print(f" Memory (float32): {embeddings.nbytes / 1024 / 1024:.1f} MB")

vocab_size = 50257
embed_table = np.random.randn(vocab_size, D).astype(np.float32)
print(f"Embedding table (GPT-2): {embed_table.shape}")
print(f" Memory: {embed_table.nbytes / 1024 / 1024:.1f} MB")

print("\n--- Attention: (B, H, T, D_head) ---")
H = 12
D_head = D // H
Q = np.random.randn(B, H, T, D_head).astype(np.float32)
print(f"Query tensor: {Q.shape}")
print(f" Head dim: {D_head}")
attn_scores = np.random.randn(B, H, T, T).astype(np.float32)
print(f"Attention scores: {attn_scores.shape}")
print(f" Memory: {attn_scores.nbytes / 1024 / 1024:.1f} MB")

print("\n--- Weight shapes ---")
shapes = {
    "Linear (768 -> 3072)": (3072, 768),
    "Linear (3072 -> 768)": (768, 3072),
    "Conv2D (3->64, 7x7)": (64, 3, 7, 7),
    "Conv2D (64->128, 3x3)": (128, 64, 3, 3),
    "LayerNorm (768)": (768,),
    "Embedding (50257, 768)": (50257, 768),
    "Positional (1024, 768)": (1024, 768),
}
for name, shape in shapes.items():
    params = reduce(lambda a, b: a * b, shape, 1)
    print(f" {name}: {shape} -> {params:,} params")

print("\n--- Layout conversion: NCHW <-> NHWC ---")
nchw = np.random.randn(2, 3, 4, 4)
nhwc = np.transpose(nchw, (0, 2, 3, 1))
back = np.transpose(nhwc, (0, 3, 1, 2))
print(f"NCHW: {nchw.shape}")
print(f"NHWC: {nhwc.shape}")
print(f"Back to NCHW: {back.shape}")
print(f"Round-trip match: {np.allclose(nchw, back)}")

print("\n--- Reshaping for multi-head attention ---")
B, T, D = 4, 128, 768

```

```

H = 12
D_head = D // H
x = np.random.randn(B, T, D)
print(f"Input: {x.shape}")

step1 = x.reshape(B, T, H, D_head)
print(f"After reshape to (B,T,H,D_head): {step1.shape}")

step2 = step1.transpose(0, 2, 1, 3)
print(f"After transpose to (B,H,T,D_head): {step2.shape}")

step3 = step2.transpose(0, 2, 1, 3).reshape(B, T, D)
print(f"Merge heads back: {step3.shape}")
print(f"Round-trip match: {np.allclose(x, step3)}")

print()

def demo_reduction_operations():
    print("=" * 60)
    print("REDUCTION OPERATIONS")
    print("=" * 60)

    x = np.random.randn(2, 3, 4)
    print(f"Input shape: {x.shape}")

    print(f"\n sum():          {x.sum().shape if hasattr(x.sum(), 'shape') else 'scalar'}")
    print(f" sum(axis=0):      {x.sum(axis=0).shape}")
    print(f" sum(axis=1):      {x.sum(axis=1).shape}")
    print(f" sum(axis=2):      {x.sum(axis=2).shape}")
    print(f" sum(axis=(1,2)):  {x.sum(axis=(1,2)).shape}")

    print(f"\n mean(axis=0):     {x.mean(axis=0).shape}")
    print(f" max(axis=-1):     {x.max(axis=-1).shape}")
    print(f" argmax(axis=-1):  {x.argmax(axis=-1).shape}")

    print("\n--- Global Average Pooling (vision) ---")
    feature_map = np.random.randn(2, 64, 7, 7)
    pooled = feature_map.mean(axis=(2, 3))
    print(f" Feature map: {feature_map.shape}")
    print(f" After GAP: {pooled.shape}")

    print("\n--- Sequence mean pooling (NLP) ---")
    hidden_states = np.random.randn(4, 128, 768)
    mask = np.ones((4, 128, 1))
    mask[:, 100:, :] = 0
    pooled = (hidden_states * mask).sum(axis=1) / mask.sum(axis=1)
    print(f" Hidden states: {hidden_states.shape}")

```

```

print(f" Mask:           {mask.shape}")
print(f" Pooled:         {pooled.shape}")

print()

def demo_custom_tensor_class():
    print("=" * 60)
    print("CUSTOM TENSOR CLASS DEMO")
    print("=" * 60)

    t = Tensor([[1, 2, 3], [4, 5, 6]])
    print(f"Created: {t}")
    print(f"Shape: {t.shape}, Rank: {t.rank}, Size: {t.size}")
    print(f"Strides: {t.strides}")
    print(f"Element [1,2]: {t[1, 2]}")

    r = t.reshape((3, 2))
    print(f"\nReshaped to (3,2): {r}")

    r2 = t.reshape((-1,))
    print(f"Flattened: {r2}")

    u = t.unsqueeze(0)
    print(f"\nUnsqueeze(0): shape={u.shape}")
    s = u.squeeze(0)
    print(f"Squeeze(0): shape={s.shape}")

    tr = t.transpose(0, 1)
    print(f"\nTranspose: {tr}")

    a = Tensor([[1, 2], [3, 4]])
    b = Tensor([[10, 20], [30, 40]])
    print(f"\na + b: {(a + b).to_list()}")
    print(f"a * b: {(a * b).to_list()}")
    print(f"a * 2: {(a * 2).to_list()}")

    print(f"\nSum all: {a.sum()}")
    print(f"Sum axis 0: {a.sum(axis=0).to_list()}")
    print(f"Sum axis 1: {a.sum(axis=1).to_list()}")

    np_arr = t.to_numpy()
    print(f"\nConverted to numpy: {np_arr.shape}")

    t3d = Tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
    perm = t3d.permute((2, 0, 1))
    print(f"\n3D tensor {t3d.shape} permuted (2,0,1): {perm.shape}")
    print(f" {perm.to_list()}")

```

```

flat = t3d.flatten(start_dim=1)
print(f"Flatten from dim 1: {flat.shape} -> {flat.to_list()}")

print()

def demo_einsum_gallery():
    print("=" * 60)
    print("EINSUM GALLERY: ALL COMMON PATTERNS")
    print("=" * 60)

    two_operand_ops = [
        ("Vector dot product",      "i,i->",      (4,),      (4,)),
        ("Outer product",           "i,j->ij",     (3,),      (4,)),
        ("Matrix-vector product",   "ij,j->i",   (3, 4),     (4,)),
        ("Matrix multiply",         "ij,jk->ik", (3, 4),     (4, 5)),
        ("Batch matmul",            "bij,bjk->bik", (2, 3, 4), (2, 4, 5)),
        ("Batch outer product",     "bi,bj->bij", (2, 3),     (2, 4)),
        ("Frobenius norm squared",  "ij,ij->",   (3, 4),     (3, 4)),
        ("Tensor contraction",      "ijk,jkl->il", (2, 3, 4), (3, 4, 5)),
    ]

    single_operand_ops = [
        ("Trace",                   "ii->",      (4, 4)),
        ("Diagonal",                "ii->i",      (4, 4)),
        ("Row sum",                  "ij->i",      (3, 4)),
        ("Column sum",               "ij->j",      (3, 4)),
        ("Transpose",                "ij->ji",     (3, 4)),
    ]

    np.random.seed(0)
    for name, subscripts, shape_a, shape_b in two_operand_ops:
        a = np.random.randn(*shape_a)
        b = np.random.randn(*shape_b)
        result = np.einsum(subscripts, a, b)
        result_shape = result.shape if hasattr(result, 'shape') and result.shape else '
scalar'
        print(f" {name:30s} {subscripts:15s} "
              f"{shape_a} x {shape_b} -> {result_shape}")

    for name, subscripts, shape_a in single_operand_ops:
        a = np.random.randn(*shape_a)
        result = np.einsum(subscripts, a)
        result_shape = result.shape if hasattr(result, 'shape') and result.shape else '
scalar'
        print(f" {name:30s} {subscripts:15s} "
              f"{shape_a} -> {result_shape}")

    print()
    print("--- Bilinear form (3-operand einsum): i,ij,j-> ---")
    x = np.array([1.0, 2.0, 3.0])
    W = np.array([[1, 0, 0], [0, 2, 0], [0, 0, 3]], dtype=float)

```

```

y = np.array([1.0, 1.0, 1.0])
result = np.einsum("i,ij,j->", x, W, y)
manual = x @ W @ y
print(f"  x: {x.shape}, W: {W.shape}, y: {y.shape}")
print(f"  x^T W y = einsum: {result}, manual: {manual}")

print()

if __name__ == "__main__":
    demo_custom_tensor_class()
    demo_basic_tensor()
    demo_reshape_operations()
    demo_broadcasting_numpy()
    demo_memory_layout()
    demo_einsum()
    demo_einsum_gallery()
    demo_attention_einsum()
    demo_ai_tensor_shapes()
    demo_reduction_operations()

```

I have a tensor shape error in my deep learning code. Help me fix it.

Error message: [paste the error here]

My tensor shapes: - [name]: [shape] - [name]: [shape]

The operation I'm trying to do: [describe it]

When debugging, follow this exact process:

Step 1: Identify the operation type. What operation produced the error? Map it to one of these: - Matrix multiply / Linear layer (inner dimensions must match) - Broadcasting (align from right, each dim must be equal or 1) - Concatenation (all dims match except the cat dimension) - Convolution (expects specific rank and channel position) - Reshape (total elements must be preserved)

Step 2: Write out the shape contract. For the identified operation, write the expected shapes explicitly:

```

matmul(A, B): A is (... , m, k), B is (... , k, n) -> (... , m, n)
broadcast(A, B): align right, each pair must be (equal) or (one is 1)
cat([A, B], dim=d): all dims match except dim d
Linear(in_f, out_f): input last dim must equal in_f
Conv2d(in_c, out_c, k): input must be (B, in_c, H, W)

```

Step 3: Find the mismatch. Compare actual shapes against the contract. Identify the exact dimension that violates the rule.

Step 4: Choose the minimal fix. Pick from this table:

Symptom	Fix
Missing batch dimension	<code>.unsqueeze(0)</code>
Missing channel dimension	<code>.unsqueeze(1)</code>
Extra size-1 dimension	<code>.squeeze(dim)</code>
Inner dims wrong for matmul	<code>.transpose(-1, -2)</code> or check weight shape
Need NCHW from NHWC	<code>.permute(0, 3, 1, 2)</code>
Need NHWC from NCHW	<code>.permute(0, 2, 3, 1)</code>
Flatten spatial dims for linear	<code>.flatten(1)</code> or <code>.reshape(B, -1)</code>
Split heads: (B,T,D) to (B,H,T,D/H)	<code>.reshape(B, T, H, D//H).transpose(1, 2)</code>
Merge heads: (B,H,T,D/H) to (B,T,D)	<code>.transpose(1, 2).reshape(B, T, H*(D//H))</code>
Non-contiguous tensor with <code>.view()</code>	<code>.contiguous().view(...)</code> or use <code>.reshape(...)</code>

Step 5: Verify the fix. Show the resulting shapes at each step. Confirm total elements are preserved across any reshape. Confirm the operation's shape contract is now satisfied.

Step 6: Check for silent bugs. Even if shapes match, verify: - Broadcasting is happening along the intended axis (not accidentally) - Reduction is summing over the right dimension - The batch dimension (dim 0) survives through the entire forward pass - Transpose + reshape is used (not just reshape) when dimension ordering matters

Format your response as:

```
OPERATION: [what operation failed]
EXPECTED: [shape contract]
ACTUAL: [what shapes were provided]
MISMATCH: [which dimension, why]
FIX: [exact code]
RESULT: [shapes after fix]
```

You are a tensor shape debugger. Your job is to identify shape mismatches in deep learning code and recommend exact fixes.

When a user describes a shape error or provides tensor shapes and an operation, do the following:

Structure your response as:

1. **State the operation and its shape requirements.** For every operation, write out the expected shapes explicitly.
2. **Identify the mismatch.** Point to the exact dimension that violates the rule.
3. **Recommend a fix.** Provide the specific reshape, transpose, unsqueeze, or permute call needed.
4. **Verify the fix.** Show the resulting shapes step by step.

Use this decision framework for common operations:

Operation	Shape rule	Error pattern
matmul(A, B)	A is (... , m, k), B is (... , k, n), result is (... , m, n)	Inner dimensions (k) must match
A + B (broadcast)	Align from the right. Each dim must be equal or one must be 1	Dimensions differ and neither is 1
cat([A, B], dim=d)	All dims match EXCEPT dim d	Non-cat dimensions differ
Linear(in, out)	Input last dim must equal in	Last dim != in_features
Conv2d(in_c, out_c, k)	Input must be (B, in_c, H, W)	Wrong number of dims or channel mismatch
Embedding(vocab, dim)	Input must be integer tensor	Float input or index out of range
BatchNorm(C)	Input (B, C, ...) must have C channels at dim 1	C mismatch
softmax(dim=d)	No shape requirement, but wrong dim gives wrong probabilities	Summing over batch instead of class dim

Broadcasting rules (check from right to left):

```
Rule 1: Dimensions are equal -> compatible
Rule 2: One dimension is 1 -> broadcast (expand) to match the other
Rule 3: One tensor has fewer dims -> pad with 1s on the left
Otherwise: error
```

Common fixes for shape problems:

Problem	Fix
Need to add batch dim	<code>x.unsqueeze(0)</code>
Need to add channel dim	<code>x.unsqueeze(1)</code>
Need to remove size-1 dim	<code>x.squeeze(dim)</code>
matmul inner dims wrong	<code>x.transpose(-1, -2)</code> or check weight shape
NCHW when NHWC needed	<code>x.permute(0, 2, 3, 1)</code>
NHWC when NCHW needed	<code>x.permute(0, 3, 1, 2)</code>
Flatten spatial dims for linear	<code>x.flatten(1)</code> or <code>x.reshape(B, -1)</code>
Attention shape (B,T,D) to (B,H,T,D/H)	<code>x.reshape(B, T, H, D//H).transpose(1, 2)</code>
Merge heads back (B,H,T,D/H) to (B,T,D)	<code>x.transpose(1, 2).reshape(B, T, H * (D//H))</code>

When diagnosing shape errors:

- Print the shape of every tensor involved: `print(x.shape, w.shape)`
- Count the total elements: product of all dimensions must be preserved across reshape
- After transpose or permute, the tensor is non-contiguous. Use `.contiguous()` before `.view()`, or just use `.reshape()`
- The batch dimension (dim 0) should survive every operation in the forward pass

Avoid: - Guessing the fix without checking the operation's shape contract - Using reshape when the dimension ordering matters (transpose + reshape, not just reshape) - Recommending `.view()` on non-contiguous tensors without `.contiguous()` - Ignoring that einsum can often replace a chain of transpose + matmul + reshape

Quiz

Pre-lesson check

1. What does the 'shape' of a tensor describe?
 - a) A tuple listing the size along each axis (**correct**)

- b) The total number of elements in the tensor
 - c) The data type of the tensor elements
 - d) The memory address of the tensor
Explanation: The shape is a tuple listing the size along each axis. For example, a tensor with shape (2, 3, 4) has 2 elements along axis 0, 3 along axis 1, and 4 along axis 2.
2. In PyTorch, what layout does an image batch tensor use by default?
- a) NHWC (batch, height, width, channels)
 - b) NCHW (batch, channels, height, width) **(correct)**
 - c) CHWN (channels, height, width, batch)
 - d) WHCN (width, height, channels, batch)
Explanation: PyTorch defaults to NCHW (channels-first) layout. TensorFlow defaults to NHWC (channels-last). Mismatched layouts cause silent errors or performance issues.

Post-lesson check

3. What is the result shape when broadcasting tensors of shape (8, 1, 6, 1) and (7, 1, 5)?
- a) (8, 7, 6, 1)
 - b) (8, 1, 6, 5)
 - c) (8, 7, 6, 5) **(correct)**
 - d) Broadcasting fails — shapes are incompatible
Explanation: Align shapes from the right: (8,1,6,1) and (1,7,1,5). Dimensions are compatible when equal or one is 1. The result takes the maximum along each axis: (8, 7, 6, 5).
4. In the einsum expression 'bhtd,bhsd->bhts', what happens to the index 'd'?
- a) It is kept in the output as a new axis

- b) It is summed over (contracted) because it appears in both inputs but not the output **(correct)**
- c) It is broadcast across both tensors
- d) It is transposed between the two input tensors
Explanation: In einsum, any index that appears in the inputs but not the output is summed over. Index 'd' appears in both 'bhtd' and 'bhsd' but not in the output 'bhts', so it is contracted (multiplied and summed).

5. Why does calling `.view()` fail on a transposed tensor in PyTorch?

- a) Transposed tensors have a different data type
 - b) The tensor is non-contiguous in memory after transpose, and view requires contiguous data **(correct)**
 - c) View only works on 2D tensors
 - d) Transpose changes the total number of elements
Explanation: Transpose swaps strides without moving data, making the tensor non-contiguous. The `.view()` operation requires contiguous memory layout. Use `.reshape()` or call `.contiguous()` first.
-

Numerical Stability

Floating point is a leaky abstraction. It will bite you during training, and you will not see it coming.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 **Time:** ~120 minutes

Learning Objectives

- Implement numerically stable softmax and log-sum-exp using the max-subtraction trick
- Identify overflow, underflow, and catastrophic cancellation in floating-point computations
- Verify analytical gradients against numerical gradients using centered finite differences
- Explain why bfloat16 is preferred over float16 for training and how loss scaling prevents gradient underflow

The Problem

Your model trains for three hours, then the loss becomes NaN. You add a print statement. The logits are fine at step 9,000. At step 9,001 they are ∞ . By step 9,002 every gradient is nan and training is dead.

Or: your model trains to completion but accuracy is 2% worse than the paper claims. You check everything. Architecture matches. Hyperparameters match. Data matches. The problem is that the paper used float32 and you used float16 without the right scaling. Thirty-two bits of accumulated rounding error quietly ate your accuracy.

Or: you implement cross-entropy loss from scratch. It works on small logits. When logits exceed 100, it returns ∞ . The softmax overflowed because $\exp(100)$ is larger than float32 can represent. Every ML framework handles this with a two-line trick. You did not know the trick existed.

Numerical stability is not a theoretical concern. It is the difference between a training run that succeeds and one that silently fails. Every serious ML bug you will debug eventually comes down to floating point.

The Concept

IEEE 754: How Computers Store Real Numbers

Computers store real numbers as floating point values following the IEEE 754 standard. A float has three parts: a sign bit, an exponent, and a mantissa (significand).

Float32 layout (32 bits total):

[1 sign] [8 exponent] [23 mantissa]

$$\text{Value} = (-1)^{\text{sign}} * 2^{(\text{exponent} - 127)} * 1.\text{mantissa}$$

The mantissa determines precision (how many significant digits). The exponent determines range (how large or small a number can be).

Format	Bits	Exponent	Mantissa	Decimal digits	Range (approx)
float64	64	11	52	~15-16	+/- 1.8e308
float32	32	8	23	~7-8	+/- 3.4e38
float16	16	5	10	~3-4	+/- 65,504
bfloat16	16	8	7	~2-3	+/- 3.4e38

float32 gives you about 7 decimal digits of precision. That means it can tell apart 1.0000001 and 1.0000002, but not 1.00000001 and 1.00000002. After 7 digits, everything is rounding noise.

float16 gives you about 3 digits. The largest number it can represent is 65,504. That is disturbingly small for ML where logits, gradients, and activations routinely exceed this.

bfloat16 is Google's answer to float16's range problem. It has the same 8-bit exponent as float32 (same range, up to 3.4e38) but only 7 mantissa bits (less precision than float16). For training neural networks, range matters more than precision, so bfloat16 usually wins.

Why $0.1 + 0.2 \neq 0.3$

The number 0.1 cannot be represented exactly in binary floating point. In base 2, it is a repeating fraction:

0.1 in binary = 0.0001100110011001100110011... (repeating forever)

Float32 truncates this to 23 bits of mantissa. The stored value is approximately 0.100000001490116. Similarly, 0.2 is stored as approximately 0.200000002980232. Their sum is 0.300000004470348, not 0.3.

```
In Python:
>>> 0.1 + 0.2
0.30000000000000004

>>> 0.1 + 0.2 == 0.3
False
```

This matters for ML because:

1. Loss comparisons like `if loss < threshold` can give wrong answers
2. Accumulating many small values (gradient updates over thousands of steps) drifts from the true sum
3. Checksums and reproducibility tests fail if you compare floats with `==`

The fix: never compare floats with `==`. Use `abs(a - b) < ε` or `math.isclose()`.

Catastrophic Cancellation

When you subtract two nearly equal floating point numbers, the significant digits cancel and you are left with rounding noise promoted to leading digits.

```
a = 1.0000001    (stored as 1.00000011920929 in float32)
b = 1.0000000    (stored as 1.00000000000000 in float32)

True difference:  0.0000001
Computed:         0.00000011920929

Relative error: 19.2%
```

That is a 19% relative error from a single subtraction. In ML, this happens whenever you:

- Compute variance of data with a large mean: $E[x^2] - E[x]^2$ when $E[x]$ is large
- Subtract nearly equal log-probabilities
- Compute finite-difference gradients with too-small ϵ

The fix: rearrange formulas to avoid subtracting large, nearly equal numbers. For variance, use the Welford algorithm or center the data first. For log-probabilities, work in log-space throughout.

Overflow and Underflow

Overflow happens when a result is too large to represent. Underflow happens when it is too small (closer to zero than the smallest representable positive number).

```
Float32 boundaries:
Maximum:  3.4028235e+38
Minimum positive (normal): 1.175e-38
Minimum positive (denorm): 1.401e-45
Overflow:  anything > 3.4e38 becomes inf
Underflow: anything < 1.4e-45 becomes 0.0
```

The `exp()` function is the primary source of overflow in ML:

```
exp(88.7) = 3.40e+38    (barely fits in float32)
exp(89.0) = inf         (overflow)
exp(-87.3) = 1.18e-38   (barely above underflow)
exp(-104) = 0.0         (underflow to zero)
```

The `log()` function hits the other direction:

```

log(0.0)    = -inf
log(-1.0)   = nan
log(1e-45)  = -103.3      (fine)
log(1e-46)  = -inf        (input underflowed to 0, then log(0) = -inf)

```

In ML, $\exp()$ appears in softmax, sigmoid, and probability computations. $\log()$ appears in cross-entropy, log-likelihoods, and KL divergence. The combination $\log(\exp(x))$ is a minefield without the right tricks.

The Log-Sum-Exp Trick

Computing $\log(\sum(\exp(x_i)))$ directly is numerically dangerous. If any x_i is large, $\exp(x_i)$ overflows. If all x_i are very negative, every $\exp(x_i)$ underflows to zero and $\log(0)$ is $-\infty$.

The trick: subtract the maximum value before exponentiating.

```

log(sum(exp(x_i))) = max(x) + log(sum(exp(x_i - max(x))))

```

Why this works: after subtracting $\max(x)$, the largest exponent is $\exp(0) = 1$. No overflow is possible. At least one term in the sum is 1, so the sum is at least 1, and $\log(1) = 0$. No underflow to $-\infty$ is possible.

Proof:

```

log(sum(exp(x_i)))
= log(sum(exp(x_i - c + c)))           (add and subtract c)
= log(sum(exp(x_i - c) * exp(c)))      (exp(a+b) = exp(a)*exp(b))
= log(exp(c) * sum(exp(x_i - c)))      (factor out exp(c))
= c + log(sum(exp(x_i - c)))            (log(a*b) = log(a) + log(b))

```

Set $c = \max(x)$ and overflow is eliminated.

This trick appears everywhere in ML: - Softmax normalization - Cross-entropy loss computation - Log-probability summation in sequence models - Mixture of Gaussians - Variational inference

Why Softmax Needs the Max-Subtraction Trick

Softmax converts logits to probabilities:

```

softmax(x_i) = exp(x_i) / sum(exp(x_j))

```

Without the trick, logits of [100, 101, 102] cause overflow:

```
exp(100) = 2.69e43
exp(101) = 7.31e43
exp(102) = 1.99e44
sum      = 2.99e44
```

These overflow float32 (max $\sim 3.4e38$)? No, $2.69e43 < 3.4e38$? Actually: $\exp(88.7)$ is already at the float32 limit. $\exp(100) = \text{inf}$ in float32.

With the trick, subtract $\max(x) = 102$:

```
exp(100 - 102) = exp(-2) = 0.135
exp(101 - 102) = exp(-1) = 0.368
exp(102 - 102) = exp(0)  = 1.000
sum = 1.503

softmax = [0.090, 0.245, 0.665]
```

The probabilities are identical. The computation is safe. This is not an optimization. It is a requirement for correctness.

NaN and Inf: Detection and Prevention

nan (Not a Number) and ∞ (∞) propagate virally through computation. One nan in a gradient update makes the weight nan, which makes every subsequent output nan. Training is dead within one step.

How ∞ appears: - $\exp()$ of a large positive number - Division by zero: $1.0 / 0.0$ - float32 overflow in accumulations

How nan appears: - $0.0 / 0.0$ - $\infty - \infty$ - $\infty * 0$ - $\sqrt{()}$ of a negative number - $\log()$ of a negative number - Any arithmetic involving an existing nan

Detection:

```
import math

math.isnan(x)      # True if x is nan
math.isinf(x)      # True if x is +inf or -inf
math.isfinite(x)   # True if x is neither nan nor inf
```

Prevention strategies:

1. Clamp inputs to $\exp()$: $\exp(\text{clamp}(x, -80, 80))$
2. Add ϵ to denominators: $x / (y + 1e-8)$
3. Add ϵ inside $\log()$: $\log(x + 1e-8)$

4. Use stable implementations (log-sum-exp, stable softmax)
5. Gradient clipping to prevent weight explosion
6. Check for nan/∞ after every forward pass during debugging

Numerical Gradient Checking

Analytical gradients (from backpropagation) can have bugs. Numerical gradient checking verifies them by computing gradients with finite differences.

The centered difference formula:

$$df/dx \approx (f(x + h) - f(x - h)) / (2h)$$

This is $O(h^2)$ accurate, much better than the forward difference $(f(x+h) - f(x)) / h$ which is only $O(h)$.

Choosing h : too large and the approximation is wrong. Too small and catastrophic cancellation destroys the answer. $h = 1e-5$ to $1e-7$ is typical.

The check: compute the relative difference between analytical and numerical gradients.

$$\text{relative_error} = |grad_analytical - grad_numerical| / \max(|grad_analytical|, |grad_numerical|, 1e-8)$$

Rules of thumb: - $\text{relative_error} < 1e-7$: perfect, gradient is correct - $\text{relative_error} < 1e-5$: acceptable, probably correct - $\text{relative_error} > 1e-3$: something is wrong - $\text{relative_error} > 1$: gradient is completely wrong

Always check gradients when implementing a new layer or loss function. PyTorch provides `torch.autograd.gradcheck()` for this.

Mixed Precision Training

Modern GPUs have specialized hardware (Tensor Cores) that compute float16 matrix multiplications 2-8x faster than float32. Mixed precision training exploits this:

1. Maintain float32 master copy of weights
2. Forward pass in float16 (fast)
3. Compute loss in float32 (prevents overflow)
4. Backward pass in float16 (fast)
5. Scale gradients to float32
6. Update float32 master weights

The problem with pure float16 training: gradients are often very small ($1e-8$ or smaller). Float16 underflows anything below $\sim 6e-8$ to zero. Your model stops learning because all gradient updates are zero.

The fix is loss scaling:

1. Multiply loss by a large scale factor (e.g., 1024)
2. Backward pass computes gradients of ($\text{loss} * 1024$)
3. All gradients are 1024x larger (pushed above float16 underflow)
4. Divide gradients by 1024 before updating weights
5. Net effect: same update, but no underflow

Dynamic loss scaling adjusts the scale factor automatically. Start with a large value (65536). If gradients overflow to ∞ , halve it. If N steps pass without overflow, double it.

bfloat16 vs float16: Why bfloat16 Wins for Training

```
float16:  [1 sign] [5 exponent] [10 mantissa]
bfloat16: [1 sign] [8 exponent] [7 mantissa]
```

float16 has more precision (10 mantissa bits vs 7) but limited range (max $\sim 65,504$). bfloat16 has less precision but the same range as float32 (max $\sim 3.4e38$).

For training neural networks:

- Activations and logits regularly exceed 65,504 during training spikes. float16 overflows; bfloat16 handles it.
- Loss scaling is required with float16 but usually unnecessary with bfloat16 because its range covers the gradient magnitude spectrum.
- bfloat16 is a simple truncation of float32: drop the bottom 16 bits of the mantissa. Conversion is trivial and lossless in the exponent.

float16 is preferred for inference where values are bounded and precision matters more. bfloat16 is preferred for training where range matters more. This is why TPUs and modern NVIDIA GPUs (A100, H100) have native bfloat16 support.

Gradient Clipping

Exploding gradients happen when gradients grow exponentially through many layers (common in RNNs, deep networks, and transformers). A single large gradient can corrupt all weights in one step.

Two types of clipping:

Clip by value: clamp each gradient element independently.

```
grad = clamp(grad, -max_val, max_val)
```

Simple but can change the direction of the gradient vector.

Clip by norm: scale the entire gradient vector so its norm does not exceed a threshold.

```
if ||grad|| > max_norm:  
    grad = grad * (max_norm / ||grad||)
```

Preserves the direction of the gradient. This is what `torch.nn.utils.clip_grad_norm_()` does. It is the standard choice.

Typical values: `max_norm=1.0` for transformers, `max_norm=0.5` for RL, `max_norm=5.0` for simpler networks.

Gradient clipping is not a hack. It is a safety mechanism. Without it, a single outlier batch can produce a gradient large enough to ruin weeks of training.

Normalization Layers as Numerical Stabilizers

Batch normalization, layer normalization, and RMS normalization are usually presented as regularizers that help training converge. They are also numerical stabilizers.

Without normalization, activations can grow or shrink exponentially through layers:

```
Layer 1: values in [0, 1]  
Layer 5: values in [0, 100]  
Layer 10: values in [0, 10,000]  
Layer 50: values in [0, inf]
```

Normalization recenters and rescales activations at every layer:

```
LayerNorm(x) = (x - mean(x)) / (std(x) + epsilon) * gamma + beta
```

The ϵ (typically $1e-5$) prevents division by zero when all activations are identical. The learned parameters γ and β let the network restore any scale it needs.

This keeps values in a numerically safe range throughout the network, preventing both overflow in the forward pass and gradient explosion in the backward pass.

Common ML Numerical Bugs

Bug: Loss is NaN after a few epochs. Cause: logits grew too large, softmax overflowed. Or learning rate is too high and weights diverged. Fix: use stable softmax (max subtraction), reduce learning rate, add gradient clipping.

Bug: Loss is stuck at $\log(\text{num_classes})$. Cause: model outputs are near-uniform probabilities. Often means gradients are vanishing or the model is not learning at all. Fix: check that data labels are correct, verify the loss function, check for dead ReLUs.

Bug: Validation accuracy is lower than expected by 1-3%. Cause: mixed precision without proper loss scaling. Gradient underflow silently zeroes out small updates. Fix: enable dynamic loss scaling, or switch to bfloat16.

Bug: Gradient norms are 0.0 for some layers. Cause: dead ReLU neurons (all inputs negative), or float16 underflow. Fix: use LeakyReLU or GELU, use gradient scaling, check weight initialization.

Bug: Model works on one GPU but gives different results on another. Cause: non-deterministic floating point accumulation order. GPU parallel reductions sum in different orders on different hardware, and floating point addition is not associative. Fix: accept small differences ($1e-6$), or set `torch.use_deterministic_algorithms(True)` and accept the speed penalty.

Bug: `exp()` returns ∞ in loss computation. Cause: raw logits passed to `exp()` without the max-subtraction trick. Fix: use `torch.nn.functional.log_softmax()` which implements log-sum-exp internally.

Bug: Training diverges after switching from float32 to float16. Cause: float16 cannot represent gradient magnitudes below $6e-8$ or activations above 65,504. Fix: use mixed precision with loss scaling (AMP), or use bfloat16 instead.

Build It

Step 1: Demonstrate floating point precision limits

```
print("=== Floating Point Precision ===")
print(f"0.1 + 0.2 = {0.1 + 0.2}")
print(f"0.1 + 0.2 == 0.3? {0.1 + 0.2 == 0.3}")
print(f"Difference: {(0.1 + 0.2) - 0.3:.2e}")
```

Step 2: Implement naive vs stable softmax

```
import math

def softmax_naive(logits):
    exps = [math.exp(z) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

def softmax_stable(logits):
    max_logit = max(logits)
    exps = [math.exp(z - max_logit) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]
```

```

safe_logits = [2.0, 1.0, 0.1]
print(f"Naive: {softmax_naive(safe_logits)}")
print(f"Stable: {softmax_stable(safe_logits)}")

dangerous_logits = [100.0, 101.0, 102.0]
print(f"Stable: {softmax_stable(dangerous_logits)}")
# softmax_naive(dangerous_logits) would return [nan, nan, nan]

```

Step 3: Implement stable log-sum-exp

```

def logsumexp_naive(values):
    return math.log(sum(math.exp(v) for v in values))

def logsumexp_stable(values):
    c = max(values)
    return c + math.log(sum(math.exp(v - c) for v in values))

safe = [1.0, 2.0, 3.0]
print(f"Naive: {logsumexp_naive(safe):.6f}")
print(f"Stable: {logsumexp_stable(safe):.6f}")

large = [500.0, 501.0, 502.0]
print(f"Stable: {logsumexp_stable(large):.6f}")
# logsumexp_naive(large) returns inf

```

Step 4: Implement stable cross-entropy

```

def cross_entropy_naive(true_class, logits):
    probs = softmax_naive(logits)
    return -math.log(probs[true_class])

def cross_entropy_stable(true_class, logits):
    max_logit = max(logits)
    shifted = [z - max_logit for z in logits]
    log_sum_exp = math.log(sum(math.exp(s) for s in shifted))
    log_prob = shifted[true_class] - log_sum_exp
    return -log_prob

logits = [2.0, 5.0, 1.0]
true_class = 1
print(f"Naive: {cross_entropy_naive(true_class, logits):.6f}")
print(f"Stable: {cross_entropy_stable(true_class, logits):.6f}")

```

Step 5: Gradient checking

```

def numerical_gradient(f, x, h=1e-5):
    grad = []
    for i in range(len(x)):

```

```

        x_plus = x[:]
        x_minus = x[:]
        x_plus[i] += h
        x_minus[i] -= h
        grad.append((f(x_plus) - f(x_minus)) / (2 * h))
    return grad

def check_gradient(analytical, numerical, tolerance=1e-5):
    for i, (a, n) in enumerate(zip(analytical, numerical)):
        denom = max(abs(a), abs(n), 1e-8)
        rel_error = abs(a - n) / denom
        status = "OK" if rel_error < tolerance else "FAIL"
        print(f" param {i}: analytical={a:.8f} numerical={n:.8f} "
              f"rel_error={rel_error:.2e} [{status}]")

def f(params):
    x, y = params
    return x**2 + 3*x*y + y**3

def f_grad(params):
    x, y = params
    return [2*x + 3*y, 3*x + 3*y**2]

point = [2.0, 1.0]
analytical = f_grad(point)
numerical = numerical_gradient(f, point)
check_gradient(analytical, numerical)

```

Use It

Mixed precision simulation

```

import struct

def float32_to_float16_round(x):
    packed = struct.pack('f', x)
    f32 = struct.unpack('f', packed)[0]
    packed16 = struct.pack('e', f32)
    return struct.unpack('e', packed16)[0]

def simulate_bfloat16(x):
    packed = struct.pack('f', x)
    as_int = int.from_bytes(packed, 'little')
    truncated = as_int & 0xFFFF0000
    repacked = truncated.to_bytes(4, 'little')
    return struct.unpack('f', repacked)[0]

```

Gradient clipping

```

def clip_by_norm(gradients, max_norm):
    total_norm = math.sqrt(sum(g**2 for g in gradients))

```

```

if total_norm > max_norm:
    scale = max_norm / total_norm
    return [g * scale for g in gradients]
return gradients

grads = [10.0, 20.0, 30.0]
clipped = clip_by_norm(grads, max_norm=5.0)
print(f"Original norm: {math.sqrt(sum(g**2 for g in grads)):.2f}")
print(f"Clipped norm: {math.sqrt(sum(g**2 for g in clipped)):.2f}")
print(f"Direction preserved: {[c/clipped[0] for c in clipped]} == {[g/grads[0] for g i
n grads]}")

```

NaN/Inf detection

```

def check_tensor(name, values):
    has_nan = any(math.isnan(v) for v in values)
    has_inf = any(math.isinf(v) for v in values)
    if has_nan or has_inf:
        print(f"WARNING {name}: nan={has_nan} inf={has_inf}")
        return False
    return True

check_tensor("good", [1.0, 2.0, 3.0])
check_tensor("bad", [1.0, float('nan'), 3.0])
check_tensor("ugly", [1.0, float('inf'), 3.0])

```

See `code/numerical.py` for complete implementations with all edge cases demonstrated.

Exercises

1. **Catastrophic cancellation.** Compute the variance of `[1000000.0, 1000001.0, 1000002.0]` using the naive formula $E[x^2] - E[x]^2$ in float32. Then compute it using Welford's online algorithm. Compare the errors against the true variance (0.6667).
2. **Precision hunt.** Find the smallest positive float32 value `x` such that `1.0 + x == 1.0` in Python. This is the machine ϵ . Verify it matches `numpy.finfo(numpy.float32).eps`.
3. **Log-sum-exp edge cases.** Test your `logsumexp_stable` function with: (a) all values equal, (b) one value much larger than the rest, (c) all values very negative (-1000). Verify it gives correct results where the naive version fails.
4. **Gradient checking a neural network layer.** Implement a single linear layer $y = Wx + b$ and its analytical backward pass. Use `numerical_gradient` to verify correctness for a 3x2 weight matrix.
5. **Loss scaling experiment.** Simulate training with float16: create random gradients in the range `[1e-9, 1e-3]`, convert to float16, and measure what fraction become zero. Then apply loss scaling (multiply by 1024), convert to float16, scale back, and measure the zero fraction again.

Key Terms

Term	What people say	What it actually means
IEEE 754	“The float standard”	International standard defining binary floating point formats, rounding rules, and special values (∞ , nan). Every modern CPU and GPU implements it.
Machine ϵ	“The precision limit”	The smallest value e such that $1.0 + e \neq 1.0$ in a given float format. For float32, it is about $1.19e-7$.
Catastrophic cancellation	“Precision loss from subtraction”	When subtracting nearly equal floating point numbers, significant digits cancel and rounding noise dominates the result.
Overflow	“Number too big”	A result exceeds the maximum representable value and becomes ∞ . $\exp(89)$ overflows float32.
Underflow	“Number too small”	A result is closer to zero than the smallest representable positive number and becomes 0.0. $\exp(-104)$ underflows float32.
Log-sum-exp trick	“Subtract the max first”	Computing $\log(\sum(\exp(x)))$ by factoring out $\exp(\max(x))$ to prevent overflow and underflow. Used in softmax, cross-entropy, and log-probability math.
Stable softmax	“Softmax that does not explode”	Subtracting $\max(\text{logits})$ before exponentiating. Numerically identical result, no overflow possible.
Gradient checking	“Verify your backprop”	Comparing analytical gradients from backpropagation against numerical gradients from finite differences to catch implementation bugs.
Mixed precision	“Float16 forward, float32 backward”	Using lower-precision floats for speed-critical operations and higher-precision floats for numerically sensitive operations. Typical speedup is 2-3x.
Loss scaling	“Prevent gradient underflow”	Multiplying the loss by a large constant before backprop so gradients stay in float16’s representable range, then dividing by the same constant before weight updates.
bfloat16	“Brain floating point”	Google’s 16-bit format with 8 exponent bits (same range as float32) and 7 mantissa bits (less precision than float16). Preferred for training.
Gradient clipping	“Cap the gradient norm”	Scaling the gradient vector so its norm does not exceed a threshold. Prevents exploding gradients from ruining weights.

Term	What people say	What it actually means
NaN	“Not a Number”	Special float value from undefined operations ($0/0$, $\infty-\infty$, $\sqrt{-1}$). Propagates through all subsequent arithmetic.
Inf	“Infinity”	Special float value from overflow or division by zero. Can combine to produce NaN ($\infty - \infty$, $\infty * 0$).
Numerical gradient	“Brute force derivative”	Approximating a derivative by evaluating $f(x+h)$ and $f(x-h)$ and dividing by $2h$. Slow but reliable for verification.

You are a numerical stability debugger for machine learning training runs. Your job is to diagnose why a model produces NaN, Inf, or silently wrong results, and provide the exact fix.

When a user reports a numerical issue, follow this diagnostic protocol:

Step 1: Classify the symptom

Ask which symptom they see, if not already stated:

- Loss is NaN
- Loss is Inf or -Inf
- Loss suddenly spikes then becomes NaN
- Gradients are NaN or Inf
- Gradients are all zeros
- Model outputs are all the same value
- Accuracy is lower than expected (silent numerical error)
- Training works in float32 but fails in float16

Step 2: Check the five most common causes in order

Cause 1: Unstable softmax or cross-entropy

Symptoms: NaN loss, Inf loss, loss spikes when logits become large.

Check: Are logits being passed directly to `exp()` without the max-subtraction trick?

Fix: Replace manual softmax with stable implementation. In PyTorch, use `F.log_softmax()` or `nn.CrossEntropyLoss()` which accepts raw logits and handles stability internally. Never compute `softmax()` then `log()` separately.

```
# Wrong
probs = torch.softmax(logits, dim=-1)
loss = -torch.log(probs[target])
```

```
# Right
loss = F.cross_entropy(logits, target)
```

Cause 2: Learning rate too high

Symptoms: Loss spikes, gradients explode, weights become Inf then NaN within a few steps.

Check: Print the gradient norm at each step. If it exceeds 100 or grows exponentially, the learning rate is too high.

Fix: Reduce learning rate by 10x. Add gradient clipping with `max_norm=1.0`.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

Cause 3: Division by zero or $\log(0)$

Symptoms: NaN or Inf in specific layers, often in normalization or loss computation.

Check: Look for division operations, `log()` calls, and `1/V()` calls. Check if any denominator can be zero.

Fix: Add ϵ to every denominator and inside every `log()`:

```
# Wrong
normalized = x / x.std()
log_prob = torch.log(prob)

# Right
normalized = x / (x.std() + 1e-8)
log_prob = torch.log(prob + 1e-8)
```

Cause 4: Float16 overflow or underflow

Symptoms: Works in float32, fails in float16. Gradients become zero (underflow) or Inf (overflow).

Check: Are activations or logits exceeding 65,504 (float16 max)? Are gradients smaller than $6e-8$ (float16 min positive)?

Fix: Enable automatic mixed precision with dynamic loss scaling:

```
scaler = torch.cuda.amp.GradScaler()
with torch.cuda.amp.autocast():
    output = model(input)
    loss = criterion(output, target)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Or switch to bfloat16 which has the same range as float32:

```
with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
    output = model(input)
    loss = criterion(output, target)
```

Cause 5: Weight initialization issues

Symptoms: Gradients are zero from the start, or they explode immediately at step 1.

Check: Print the mean and std of each layer's weights after initialization. They should be roughly mean=0, std proportional to $1/\sqrt{\text{fan_in}}$.

Fix: Use proper initialization. Xavier/Glorot for tanh/sigmoid, Kaiming/He for ReLU:

```
# For ReLU networks
nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu')

# For transformers
nn.init.xavier_uniform_(layer.weight)
```

Step 3: Insert diagnostic hooks

If the cause is not immediately clear, recommend inserting these checks:

```
# After forward pass
for name, param in model.named_parameters():
    if param.grad is not None:
        if torch.isnan(param.grad).any():
            print(f"NaN gradient in {name} at step {step}")
        if torch.isinf(param.grad).any():
            print(f"Inf gradient in {name} at step {step}")
        grad_norm = param.grad.norm().item()
        if grad_norm > 100:
            print(f"Large gradient in {name}: norm={grad_norm:.2f}")

# After each layer (register hooks)
def check_activations(name):
    def hook(module, input, output):
        if isinstance(output, torch.Tensor):
            if torch.isnan(output).any():
                print(f"NaN output in {name}")
            if torch.isinf(output).any():
                print(f"Inf output in {name}")
            print(f"{name}: min={output.min():.4f} max={output.max():.4f} mean={output.mean():.4f}")
        return hook
    return hook

for name, module in model.named_modules():
    module.register_forward_hook(check_activations(name))
```

Step 4: Provide the fix

Structure every fix as: 1. The exact code change (before and after) 2. Why it works (one sentence) 3. How to verify it worked (what to check after applying the fix)

Decision tree summary

Loss is NaN?

- |-> Check softmax/cross-entropy implementation
- |-> Check for $\log(0)$ or $0/0$
- |-> Check learning rate (try 10x smaller)
- |-> Check for $\text{Inf} * 0$ in gradient computation

Loss is Inf?

- |-> Check $\exp()$ calls (logits too large?)
- |-> Check division by near-zero values
- |-> Check float16 range overflow

Gradients all zero?

- |-> Check for dead ReLU (all negative inputs)
- |-> Check float16 gradient underflow
- |-> Check weight initialization
- |-> Check if loss is computed correctly (detached tensor?)

Silent accuracy loss?

- |-> Check float precision (float16 vs float32)
- |-> Check accumulation order (non-deterministic reductions)
- |-> Check loss scaling in mixed precision
- |-> Check batch normalization running stats (eval vs train mode)

Different results on different hardware?

- |-> Floating point is not associative: $(a+b)+c \neq a+(b+c)$
- |-> GPU parallel reductions sum in hardware-dependent order
- |-> Accept $1e-6$ differences or use deterministic mode

Avoid: - Suggesting “just use float64” as a solution. It is 2x slower and masks the real bug. - Ignoring the distinction between float16 and bfloat16. They have different failure modes. - Recommending ϵ values larger than $1e-6$. Large epsilons hide bugs and bias results. - Saying “add gradient clipping” without also investigating the root cause. Clipping is a safety net, not a fix for broken math.

Quiz

Pre-lesson check

1. What is the approximate range of numbers that float32 can represent?

- a) $\pm 65,504$
- b) $\pm 3.4e38$ **(correct)**
- c) $\pm 1.8e308$
- d) $\pm 1.0e10$

Explanation: Float32 has an 8-bit exponent giving a range of approximately $\pm 3.4e38$. Float16 is limited to $\pm 65,504$, and float64 reaches $\pm 1.8e308$.

2. Why does $0.1 + 0.2$ not equal 0.3 in floating-point arithmetic?

- a) Python rounds all decimals to integers
- b) 0.1 and 0.2 cannot be represented exactly in binary floating point **(correct)**
- c) The CPU has a bug in its addition circuit
- d) 0.3 is not a valid floating-point number

Explanation: The number 0.1 is a repeating fraction in binary (like $1/3$ in decimal). Float32 truncates it, so the stored value is approximately 0.100000001490116 . The accumulated error makes the sum differ from 0.3 .

Post-lesson check

3. In the stable softmax implementation, why do you subtract $\max(\text{logits})$ before exponentiating?

- a) It makes the output probabilities more uniform
- b) It prevents $\exp()$ from overflowing by ensuring the largest exponent is 0 **(correct)**
- c) It converts logits from float16 to float32
- d) It normalizes the logits to have zero mean

Explanation: After subtracting $\max(\text{logits})$, the largest value is 0 and $\exp(0) = 1$, which cannot overflow. All other values are negative, so their exponentials are less than 1 . The probabilities are mathematically identical to the naive version.

4. When using centered finite differences for gradient checking, what happens if the step size h is too small (e.g., $1e-15$)?

- a) The approximation becomes more accurate
- b) Catastrophic cancellation destroys the result because $f(x+h)$ and $f(x-h)$ are nearly identical **(correct)**

c) The function evaluation becomes faster

d) The gradient automatically becomes zero

Explanation: When h is extremely small, $f(x+h)$ and $f(x-h)$ differ only in their last few significant digits. Subtracting them cancels the leading digits, leaving mostly rounding noise. Typical good values are $h = 1e-5$ to $1e-7$.

5. Why is bfloat16 generally preferred over float16 for neural network training?

- a) bfloat16 has more mantissa bits, giving better precision
- b) bfloat16 has the same exponent range as float32, avoiding overflow on large activations without loss scaling **(correct)**

c) bfloat16 uses less memory than float16

d) bfloat16 is supported by more GPU architectures than float16

Explanation: bfloat16 has 8 exponent bits (same as float32, range up to $3.4e38$) while float16 has only 5 exponent bits (max $\sim 65,504$). During training, activations and gradients can exceed 65,504, causing float16 overflow. bfloat16 handles this without loss scaling.

Norms and Distances

Your distance function defines what “similar” means. Choose wrong and everything downstream breaks.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations) **Time:** ~90 minutes

Learning Objectives

- Implement L1, L2, cosine, Mahalanobis, Jaccard, and edit distance functions from scratch
- Select the appropriate distance metric for a given ML task and explain why alternatives fail
- Connect L1 and L2 norms to LASSO and Ridge regularization and their geometric constraint regions
- Demonstrate how the same dataset produces different nearest neighbors under different metrics

The Problem

You have two vectors. Maybe they are word embeddings. Maybe they are user profiles. Maybe they are pixel arrays. You need to know: how close are they?

The answer depends entirely on which distance function you pick. Two data points can be nearest neighbors under one metric and far apart under another. Your KNN classifier, your recommendation engine, your vector database, your clustering algorithm, your loss function – they all depend on this choice. Get it wrong and your model optimizes for the wrong thing.

There is no universal best distance. L2 works for spatial data. Cosine similarity dominates NLP. Jaccard handles sets. Edit distance handles strings. Mahalanobis accounts for correlations. Wasserstein moves probability mass. Each one encodes a different assumption about what “similar” means.

This lesson builds every major distance function from scratch, shows you when each one is the right tool, and demonstrates how the same data produces completely different nearest neighbors depending on which metric you use.

The Concept

Norms: measuring vector magnitude

A norm measures the “size” of a vector. Every distance function between two vectors can be written as the norm of their difference: $d(a, b) = \|a - b\|$. So understanding norms is understanding distances.

L1 Norm (Manhattan distance)

The L1 norm sums the absolute values of all components.

$$||x||_1 = |x_1| + |x_2| + \dots + |x_n|$$

It is called Manhattan distance because it measures how far you walk on a city grid where you can only move along axes. No diagonals.

Point A = (1, 1)

Point B = (4, 5)

$$\text{L1 distance} = |4-1| + |5-1| = 3 + 4 = 7$$

On a grid, you walk 3 blocks east and 4 blocks north.

When to use L1: - High-dimensional sparse data (text features, one-hot encodings) - When you want robustness to outliers (a single huge difference does not dominate) - Feature selection problems (L1 regularization promotes sparsity)

Connection to L1 regularization (Lasso): adding $||w||_1$ to your loss function penalizes the sum of absolute weight values. This pushes small weights to exactly zero, performing automatic feature selection. The L1 penalty creates diamond-shaped constraint regions in weight space, and the corners of diamonds lie on the axes where some weights are zero.

Connection to loss functions: Mean Absolute Error (MAE) is the average L1 distance between predictions and targets. It penalizes all errors linearly, making it robust to outliers compared to MSE.

L2 Norm (Euclidean distance)

The L2 norm is the straight-line distance. Square root of the sum of squared components.

$$||x||_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

This is the distance you learned in geometry class. Pythagoras in n dimensions.

Point A = (1, 1)

Point B = (4, 5)

$$\text{L2 distance} = \sqrt{(4-1)^2 + (5-1)^2} = \sqrt{9 + 16} = \sqrt{25} = 5.0$$

The straight line, cutting diagonally through the grid.

When to use L2: - Low-to-medium dimensional continuous data - When the feature scales are comparable - Physical distances (spatial data, sensor readings) - Image similarity at the pixel level

Connection to L2 regularization (Ridge): adding $||w||_2^2$ to your loss function penalizes large weights. Unlike L1, it does not push weights to zero. It shrinks all weights toward zero proportionally. The L2

penalty creates circular constraint regions, so there are no corners on axes. Weights get small but rarely exactly zero.

Connection to loss functions: Mean Squared Error (MSE) is the average of L2 distances squared. Squaring penalizes large errors more heavily than small ones.

MAE (L1 loss):	$ y - y_{\text{hat}} $	Linear penalty. Robust to outliers.
MSE (L2 loss):	$(y - y_{\text{hat}})^2$	Quadratic penalty. Sensitive to outliers.

Lp Norms: the general family

L1 and L2 are special cases of the Lp norm:

$$\|x\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{1/p}$$

Different values of p produce different shaped “unit balls” (the set of all points at distance 1 from the origin):

p=1:	Diamond shape	(corners on axes)
p=2:	Circle/sphere	(the usual round ball)
p=3:	Superellipse	(rounded square)
p=inf:	Square/hypercube	(flat sides along axes)

L-∞ Norm (Chebyshev distance)

As p approaches ∞, the Lp norm converges to the maximum absolute component.

$$\|x\|_{\infty} = \max(|x_1|, |x_2|, \dots, |x_n|)$$

The distance between two points is determined by the single dimension where they differ the most. All other dimensions are ignored.

Point A = (1, 1)
Point B = (4, 5)
L-inf distance = $\max(4-1 , 5-1) = \max(3, 4) = 4$

When to use L-∞: - When the worst-case deviation in any single dimension matters - Game boards (a king in chess moves in L-∞: one step in any direction costs 1) - Manufacturing tolerances (every dimension must be within spec)

Cosine Similarity and Cosine Distance

Cosine similarity measures the angle between two vectors, ignoring their magnitudes.

$$\text{cos_sim}(a, b) = (a \cdot b) / (||a||_2 * ||b||_2)$$

It ranges from -1 (opposite directions) to +1 (same direction). Perpendicular vectors have cosine similarity 0.

Cosine distance converts it to a distance: $\text{cosine_distance} = 1 - \text{cosine_similarity}$. This ranges from 0 (identical direction) to 2 (opposite direction).

$$\begin{aligned} a &= (1, 0) & b &= (1, 1) \\ \text{cos_sim} &= (1*1 + 0*1) / (1 * \sqrt{2}) = 1/\sqrt{2} = 0.707 \\ \text{cos_dist} &= 1 - 0.707 = 0.293 \end{aligned}$$

Why cosine dominates NLP and embeddings: in text, document length should not affect similarity. A document about cats that is twice as long as another document about cats should still be “similar.” Cosine similarity ignores magnitude (length) and only cares about direction. Two documents with the same word distribution but different lengths point in the same direction and get cosine similarity 1.0.

When to use cosine similarity: - Text similarity (TF-IDF vectors, word embeddings, sentence embeddings) - Any domain where magnitude is noise and direction is signal - Recommendation systems (user preference vectors) - Embedding search (vector databases almost always use cosine or dot product)

Dot Product Similarity vs Cosine Similarity

The dot product of two vectors is:

$$\begin{aligned} a \cdot b &= a_1*b_1 + a_2*b_2 + \dots + a_n*b_n \\ &= ||a|| * ||b|| * \cos(\text{angle}) \end{aligned}$$

Cosine similarity is the dot product normalized by both magnitudes. When both vectors are already unit-normalized (magnitude = 1), dot product and cosine similarity are identical.

$$\begin{aligned} \text{If } ||a|| &= 1 \text{ and } ||b|| = 1: \\ a \cdot b &= \cos(\text{angle between } a \text{ and } b) \end{aligned}$$

When they differ: dot product includes magnitude information. A vector with larger magnitude gets a higher dot product score. This matters in some retrieval systems where you want “popular” items to rank higher. The magnitude acts as an implicit quality or importance signal.

$$\begin{aligned} a &= (3, 0) & b &= (1, 0) & c &= (0, 1) \\ \text{dot}(a, b) &= 3 & \text{dot}(a, c) &= 0 \\ \text{cos}(a, b) &= 1.0 & \text{cos}(a, c) &= 0.0 \end{aligned}$$

Both agree on direction, but dot product also reflects magnitude.

In practice: - Use cosine similarity when you want pure directional similarity - Use dot product when magnitudes carry meaningful information - Many vector databases (Pinecone, Weaviate, Qdrant) let you choose between them - If your embeddings are L2-normalized, the choice does not matter

Mahalanobis Distance

Euclidean distance treats all dimensions equally. But if your features are correlated or have different scales, L2 gives misleading results.

Mahalanobis distance accounts for the covariance structure of the data.

$$d_M(x, y) = \sqrt{(x - y)^T * S^{-1} * (x - y)}$$

where S is the covariance matrix of the data.

Intuitively: Mahalanobis distance first decorrelates and normalizes the data (whitening), then computes L2 distance in that transformed space. If S is the identity matrix (uncorrelated, unit variance features), Mahalanobis distance reduces to Euclidean distance.

Example: height and weight are correlated.

Someone 6'2" and 180 lbs is not unusual.

Someone 5'0" and 180 lbs is unusual.

Euclidean distance might say they are equally far from the mean.

Mahalanobis distance correctly identifies the second as an outlier because it accounts for the height-weight correlation.

When to use Mahalanobis distance: - Outlier detection (points with large Mahalanobis distance from the mean are outliers) - Classification when features have different scales and correlations - When you have enough data to estimate a reliable covariance matrix - Quality control in manufacturing (multivariate process monitoring)

Jaccard Similarity (for sets)

Jaccard similarity measures overlap between two sets.

$$J(A, B) = |A \cap B| / |A \cup B|$$

It ranges from 0 (no overlap) to 1 (identical sets). Jaccard distance = 1 - Jaccard similarity.

A = {cat, dog, fish}

B = {cat, bird, fish, snake}

```
Intersection = {cat, fish}          size = 2
Union = {cat, dog, fish, bird, snake} size = 5

Jaccard similarity = 2/5 = 0.4
Jaccard distance = 0.6
```

When to use Jaccard: - Comparing sets of tags, categories, or features - Document similarity based on word presence (not frequency) - Near-duplicate detection (MinHash approximation of Jaccard) - Comparing binary feature vectors (presence/absence data) - Evaluating segmentation models (Intersection over Union = Jaccard)

Edit Distance (Levenshtein Distance)

Edit distance counts the minimum number of single-character operations needed to transform one string into another. The operations are: insert, delete, or substitute.

```
"kitten" -> "sitting"

kitten -> sitten (substitute k -> s)
sitten -> sittin (substitute e -> i)
sittin -> sitting (insert g)

Edit distance = 3
```

Computed using dynamic programming. Fill a matrix where entry (i, j) is the edit distance between the first i characters of string A and the first j characters of string B.

	"	s	i	t	t	i	n	g
"	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

When to use edit distance: - Spell checking and correction - DNA sequence alignment (with weighted operations) - Fuzzy string matching - Deduplication of messy text data

KL Divergence (not a distance, but used like one)

KL divergence measures how one probability distribution differs from another. Covered in Lesson 09, but it belongs in this discussion because people use it as a “distance” despite it not being one.

$$D_{KL}(P \parallel Q) = \sum(p(x) * \log(p(x) / q(x)))$$

Critical property: KL divergence is NOT symmetric.

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$$

This means it fails the basic requirement of a distance metric. It also does not satisfy the triangle inequality. It is a divergence, not a distance.

Forward KL ($D_{KL}(P \parallel Q)$) is “mean-seeking”: Q tries to cover all modes of P. Reverse KL ($D_{KL}(Q \parallel P)$) is “mode-seeking”: Q focuses on a single mode of P.

When you see KL divergence: - VAEs (the KL term in the ELBO pushes the latent distribution toward a prior) - Knowledge distillation (student tries to match teacher’s distribution) - RLHF (the KL penalty keeps the fine-tuned model close to the base model) - Policy gradient methods (constraining policy updates)

Wasserstein Distance (Earth Mover’s Distance)

Wasserstein distance measures the minimum “work” needed to transform one probability distribution into another. Think of it as: if one distribution is a pile of dirt and the other is a hole, how much dirt do you have to move and how far?

$$W(P, Q) = \inf \text{ over all transport plans } \gamma \text{ of } E[d(x, y)]$$

For 1D distributions, it simplifies to the integral of the absolute difference of the cumulative distribution functions:

$$W_1(P, Q) = \int |CDF_P(x) - CDF_Q(x)| dx$$

Why Wasserstein matters: - It is a true metric (symmetric, satisfies triangle inequality) - It provides gradients even when distributions do not overlap (KL divergence goes to ∞) - This property made it central to Wasserstein GANs (WGANs), which solved the training instability of original GANs

Distributions with no overlap:

P: [1, 0, 0, 0, 0] Q: [0, 0, 0, 0, 1]

KL divergence: infinity (log of zero)

Wasserstein: 4 (move all mass 4 bins)

Wasserstein gives a meaningful gradient. KL does not.

When to use Wasserstein: - GAN training (WGAN, WGAN-GP) - Comparing distributions that may not overlap - Optimal transport problems - Image retrieval (comparing color histograms)

Why Different Tasks Need Different Distances

Task	Best distance	Why
Text similarity	Cosine	Magnitude is noise, direction is meaning
Image pixel comparison	L2	Spatial relationships matter, features are comparable scale
Sparse high-dim features	L1	Robust, does not amplify rare large differences
Set overlap (tags, categories)	Jaccard	Data is naturally set-valued, not vectorial
String matching	Edit distance	Operations map to human editing intuition
Outlier detection	Mahalanobis	Accounts for feature correlations and scales
Comparing distributions	KL divergence	Measures information lost by using Q instead of P
GAN training	Wasserstein	Provides gradients even when distributions do not overlap
Embeddings (vector DB)	Cosine or dot product	Embeddings are trained to encode meaning in direction
Recommendation	Dot product	Magnitude can encode popularity or confidence
DNA sequences	Weighted edit distance	Substitution costs vary by nucleotide pair
Manufacturing QC	$L-\infty$	Worst-case deviation in any dimension matters

Connection to Loss Functions

Loss functions are distance functions applied to predictions vs targets.

Loss function	Distance it uses	Behavior
MSE	L2 squared	Penalizes large errors heavily
MAE	L1	Penalizes all errors equally
Huber loss	L1 for large errors, L2 for small errors	Best of both: robust to outliers, smooth gradient near zero
Cross-entropy	KL divergence	Measures distribution mismatch
Hinge loss	$\max(0, \text{margin} - d)$	Only penalizes below margin

Triplet loss	L2 (typically)	Pulls positives close, pushes negatives away
Contrastive loss	L2	Similar pairs close, dissimilar pairs beyond margin

Connection to Regularization

Regularization adds a norm penalty on the weights to the loss function.

L1 regularization (Lasso): $\text{loss} + \lambda * ||w||_1$

- > Sparse weights. Some weights become exactly zero.
- > Automatic feature selection.
- > Solution has corners (non-differentiable at zero).

L2 regularization (Ridge): $\text{loss} + \lambda * ||w||_2^2$

- > Small weights. All weights shrink toward zero.
- > No feature selection (nothing goes to exactly zero).
- > Smooth solution everywhere.

Elastic Net: $\text{loss} + \lambda_1 * ||w||_1 + \lambda_2 * ||w||_2^2$

- > Combines sparsity of L1 with stability of L2.
- > Groups of correlated features are kept or dropped together.

Why L1 produces sparsity but L2 does not: picture the constraint region in 2D weight space. L1 is a diamond, L2 is a circle. The loss function's contours (ellipses) are most likely to touch the diamond at a corner, where one weight is zero. They touch the circle at a smooth point, where both weights are nonzero.

Nearest Neighbor Search

Every distance function implies a nearest neighbor search problem: given a query point, find the closest points in a dataset.

Exact nearest neighbor search is $O(n * d)$ per query in a dataset of n points with d dimensions. For large datasets, this is too slow.

Approximate Nearest Neighbor (ANN) algorithms trade a small amount of accuracy for massive speed gains:

Algorithm	Approach	Used by
KD-trees	Axis-aligned space partition	scikit-learn (low-dim)
Ball trees	Nested hyperspheres	scikit-learn (medium-d)

im)		
LSH	Random hash projections	Near-duplicate detection
HNSW	Hierarchical navigable small-world graph	FAISS, Qdrant, Weaviate
IVF	Inverted file index with cluster-based search	FAISS (billion-scale)
Product quant.	Compress vectors, search in compressed space	FAISS (memory-constrained)

HNSW (Hierarchical Navigable Small World) is the dominant algorithm in modern vector databases. It builds a multi-layer graph where each node connects to its approximate nearest neighbors. Search starts at the top layer (sparse, long jumps) and descends to the bottom layer (dense, short jumps).

Build It

Step 1: All norm and distance functions

See `code/distances.py` for the complete implementation. Every function is built from scratch using only basic Python math.

Step 2: Same data, different distances, different neighbors

The demo in `distances.py` creates a dataset, picks a query point, and shows how the nearest neighbor changes depending on the distance metric. The point that is “closest” under L1 may not be closest under L2 or cosine.

Step 3: Embedding similarity search

The code includes a mock embedding similarity search that finds the most similar “documents” to a query using cosine similarity vs L2 distance, showing that the rankings can differ.

Use It

The most common practical use: finding similar items in a vector database.

```
import numpy as np

def cosine_similarity_matrix(X):
    norms = np.linalg.norm(X, axis=1, keepdims=True)
    norms = np.where(norms == 0, 1, norms)
    X_normalized = X / norms
    return X_normalized @ X_normalized.T

embeddings = np.random.randn(1000, 768)
```



```

sim_matrix = cosine_similarity_matrix(embeddings)

query_idx = 0
similarities = sim_matrix[query_idx]
top_k = np.argsort(similarities)[::-1][1:6]
print(f"Top 5 most similar to item 0: {top_k}")
print(f"Similarities: {similarities[top_k]}")

```

When you call `model.encode(text)` and then search a vector database, this is what happens under the hood. The embedding model maps text to vectors. The vector database computes cosine similarity (or dot product) between your query vector and every stored vector, using ANN algorithms to avoid checking all of them.

Exercises

1. Compute L1, L2, and L^∞ distances between (1, 2, 3) and (4, 0, 6). Verify that $L^\infty \leq L2 \leq L1$ always holds for any pair of points. Prove why this ordering is guaranteed.
2. Create two vectors where cosine similarity is high (> 0.9) but L2 distance is large (> 10). Explain geometrically what is happening. Then create two vectors where cosine similarity is low (< 0.3) but L2 distance is small (< 0.5).
3. Implement a function that takes a dataset and a query point and returns the nearest neighbor under L1, L2, cosine, and Mahalanobis distance. Find a dataset where all four disagree on which point is nearest.
4. Compute the Wasserstein distance between [0.5, 0.5, 0, 0] and [0, 0, 0.5, 0.5] by hand using the CDF method. Then compute it between [0.25, 0.25, 0.25, 0.25] and [0, 0, 0.5, 0.5]. Which is larger and why?
5. Implement MinHash for approximate Jaccard similarity. Generate 100 random sets, compute exact Jaccard for all pairs, and compare with MinHash approximation using 50, 100, and 200 hash functions. Plot the approximation error.

Key Terms

Term	What people say	What it actually means
Norm	"Size of a vector"	A function that maps a vector to a non-negative scalar, satisfying triangle inequality, absolute homogeneity, and zero only for the zero vector
L1 norm	"Manhattan distance"	Sum of absolute component values. Produces sparsity in optimization. Robust to outliers

Term	What people say	What it actually means
L2 norm	"Euclidean distance"	Square root of sum of squared components. The straight-line distance in Euclidean space
Lp norm	"Generalized norm"	The p-th root of the sum of p-th powers of absolute components. L1 and L2 are special cases
L-∞ norm	"Max norm" or "Chebyshev distance"	The maximum absolute component value. The limit of Lp as p approaches ∞
Cosine similarity	"Angle between vectors"	Dot product normalized by both magnitudes. Ranges from -1 to +1. Ignores vector length
Cosine distance	"1 minus cosine similarity"	Converts cosine similarity to a distance. Ranges from 0 to 2
Dot product	"Unnormalized cosine"	Sum of component-wise products. Equals cosine similarity times both magnitudes
Mahalanobis distance	"Correlation-aware distance"	L2 distance in a space that has been whitened (decorrelated and normalized) using the data covariance matrix
Jaccard similarity	"Set overlap"	Size of intersection divided by size of union. For sets, not vectors
Edit distance	"Levenshtein distance"	Minimum insertions, deletions, and substitutions to transform one string into another
KL divergence	"Distance between distributions"	Not a true distance (not symmetric). Measures extra bits from using Q to encode P
Wasserstein distance	"Earth mover's distance"	Minimum work to transport mass from one distribution to another. A true metric
Approximate nearest neighbor	"ANN search"	Algorithms (HNSW, LSH, IVF) that find approximately closest points much faster than exact search
HNSW	"The vector DB algorithm"	Hierarchical Navigable Small World graph. Multi-layer graph for fast approximate nearest neighbor search
L1 regularization	"Lasso"	Adding the L1 norm of weights to the loss. Drives weights to zero (sparsity)
L2 regularization	"Ridge" or "weight decay"	Adding the squared L2 norm of weights to the loss. Shrinks weights toward zero without sparsity

Term	What people say	What it actually means
Elastic Net	"L1 + L2"	Combines L1 and L2 regularization. Handles correlated feature groups better than either alone

Code Appendix: *distances.py*

```

import math
import random

def l1_norm(x):
    return sum(abs(xi) for xi in x)

def l2_norm(x):
    return math.sqrt(sum(xi ** 2 for xi in x))

def lp_norm(x, p):
    if p == float('inf'):
        return max(abs(xi) for xi in x)
    return sum(abs(xi) ** p for xi in x) ** (1 / p)

def l1f_norm(x):
    return max(abs(xi) for xi in x)

def l1_distance(a, b):
    return sum(abs(ai - bi) for ai, bi in zip(a, b))

def l2_distance(a, b):
    return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))

def lp_distance(a, b, p):
    diff = [ai - bi for ai, bi in zip(a, b)]
    return lp_norm(diff, p)

def l1f_distance(a, b):
    return max(abs(ai - bi) for ai, bi in zip(a, b))

def dot_product(a, b):
    return sum(ai * bi for ai, bi in zip(a, b))

def cosine_similarity(a, b):

```

```

dot = dot_product(a, b)
norm_a = l2_norm(a)
norm_b = l2_norm(b)
if norm_a == 0 or norm_b == 0:
    return 0.0
return dot / (norm_a * norm_b)

def cosine_distance(a, b):
    return 1.0 - cosine_similarity(a, b)

def mahalanobis_distance(x, y, cov_matrix):
    n = len(x)
    diff = [xi - yi for xi, yi in zip(x, y)]

    inv_cov = invert_matrix(cov_matrix)

    temp = [0.0] * n
    for i in range(n):
        for j in range(n):
            temp[i] += diff[j] * inv_cov[j][i]

    result = sum(temp[i] * diff[i] for i in range(n))
    return math.sqrt(max(0, result))

def invert_matrix(matrix):
    n = len(matrix)
    augmented = [row[:] + [1.0 if i == j else 0.0 for j in range(n)] for i, row in enumerate(matrix)]

    for col in range(n):
        max_row = col
        for row in range(col + 1, n):
            if abs(augmented[row][col]) > abs(augmented[max_row][col]):
                max_row = row
        augmented[col], augmented[max_row] = augmented[max_row], augmented[col]

        pivot = augmented[col][col]
        if abs(pivot) < 1e-12:
            raise ValueError("Matrix is singular or near-singular")
        for j in range(2 * n):
            augmented[col][j] /= pivot

        for row in range(n):
            if row != col:
                factor = augmented[row][col]
                for j in range(2 * n):
                    augmented[row][j] -= factor * augmented[col][j]

    return [row[n:] for row in augmented]

```

```

def jaccard_similarity(set_a, set_b):
    if not set_a and not set_b:
        return 1.0
    intersection = len(set_a & set_b)
    union = len(set_a | set_b)
    return intersection / union

def jaccard_distance(set_a, set_b):
    return 1.0 - jaccard_similarity(set_a, set_b)

def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],
                    dp[i][j - 1],
                    dp[i - 1][j - 1]
                )

    return dp[m][n]

def kl_divergence(p, q):
    total = 0.0
    for pi, qi in zip(p, q):
        if pi > 0:
            if qi <= 0:
                return float('inf')
            total += pi * math.log(pi / qi)
    return total

def wasserstein_1d(p, q):
    assert len(p) == len(q), "Distributions must have the same number of bins"
    n = len(p)
    cdf_p = [0.0] * n
    cdf_q = [0.0] * n

    cdf_p[0] = p[0]

```

```

cdf_q[0] = q[0]
for i in range(1, n):
    cdf_p[i] = cdf_p[i - 1] + p[i]
    cdf_q[i] = cdf_q[i - 1] + q[i]

return sum(abs(cdf_p[i] - cdf_q[i]) for i in range(n))

def compute_covariance(data):
    n = len(data)
    d = len(data[0])
    means = [sum(data[i][j] for i in range(n)) / n for j in range(d)]
    centered = [[data[i][j] - means[j] for j in range(d)] for i in range(n)]
    cov = [[0.0] * d for _ in range(d)]
    for i in range(d):
        for j in range(d):
            cov[i][j] = sum(centered[k][i] * centered[k][j] for k in range(n)) / (n - 1
    )
    return cov

def normalize_vector(v):
    norm = l2_norm(v)
    if norm == 0:
        return v[:]
    return [vi / norm for vi in v]

def find_nearest_neighbor(query, dataset, distance_fn, **kwargs):
    best_idx = 0
    best_dist = float('inf')
    for i, point in enumerate(dataset):
        d = distance_fn(query, point, **kwargs)
        if d < best_dist:
            best_dist = d
            best_idx = i
    return best_idx, best_dist

def find_k_nearest(query, dataset, distance_fn, k=5, **kwargs):
    distances = []
    for i, point in enumerate(dataset):
        d = distance_fn(query, point, **kwargs)
        distances.append((i, d))
    distances.sort(key=lambda x: x[1])
    return distances[:k]

def demo_norms():
    print("=" * 65)
    print("NORMS: MEASURING VECTOR SIZE")
    print("=" * 65)

```

```

vectors = [
    ("(3, 4)", [3, 4]),
    ("(1, 1, 1, 1)", [1, 1, 1, 1]),
    ("(5, 0, 0)", [5, 0, 0]),
    ("(1, 2, 3, 4, 5)", [1, 2, 3, 4, 5]),
]

print(f" {'Vector':<20s} {'L1':>8s} {'L2':>8s} {'L3':>8s} {'L-inf':>8s}")
print(f" {'-' * 20} {'-' * 8} {'-' * 8} {'-' * 8} {'-' * 8}")
for name, v in vectors:
    print(f" {name:<20s} {l1_norm(v):>8.3f} {l2_norm(v):>8.3f} "
          f"{lp_norm(v, 3):>8.3f} {linf_norm(v):>8.3f}")

print()
print(" Note: L-inf <= L2 <= L1 always holds.")
print()

def demo_distances():
    print("=" * 65)
    print("DISTANCES BETWEEN TWO POINTS")
    print("=" * 65)

    a = [1, 2, 3]
    b = [4, 0, 6]

    print(f" A = {a}")
    print(f" B = {b}")
    print()
    print(f" L1 (Manhattan): {l1_distance(a, b):.4f}")
    print(f" L2 (Euclidean): {l2_distance(a, b):.4f}")
    print(f" L3: {lp_distance(a, b, 3):.4f}")
    print(f" L-inf (Chebyshev): {linf_distance(a, b):.4f}")
    print(f" Cosine distance: {cosine_distance(a, b):.4f}")
    print(f" Cosine similarity: {cosine_similarity(a, b):.4f}")
    print(f" Dot product: {dot_product(a, b):.4f}")
    print()

def demo_cosine_vs_dot():
    print("=" * 65)
    print("COSINE SIMILARITY vs DOT PRODUCT")
    print("=" * 65)

    a = [1, 2, 3]
    b = [2, 4, 6]
    c = [3, 1, 0]

    print(f" A = {a}")
    print(f" B = {b} (A scaled by 2)")

```

```

print(f" C = {c} (different direction)")
print()
print(f" {'Pair':<10s} {'Cosine':>10s} {'Dot':>10s}")
print(f" {'-' * 10} {'-' * 10} {'-' * 10}")
print(f" {'A vs B':<10s} {cosine_similarity(a, b):>10.4f} {dot_product(a, b):>
10.4f}")
print(f" {'A vs C':<10s} {cosine_similarity(a, c):>10.4f} {dot_product(a, c):>
10.4f}")
print(f" {'B vs C':<10s} {cosine_similarity(b, c):>10.4f} {dot_product(b, c):>
10.4f}")
print()
print(" Cosine says A and B are identical (same direction).")
print(" Dot product says B is more similar because of larger magnitude.")
print()

a_norm = normalize_vector(a)
b_norm = normalize_vector(b)
c_norm = normalize_vector(c)

print(" After L2 normalization:")
print(f" {'Pair':<10s} {'Cosine':>10s} {'Dot':>10s}")
print(f" {'-' * 10} {'-' * 10} {'-' * 10}")
print(f" {'A vs B':<10s} {cosine_similarity(a_norm, b_norm):>10.4f} {dot_produ
ct(a_norm, b_norm):>10.4f}")
print(f" {'A vs C':<10s} {cosine_similarity(a_norm, c_norm):>10.4f} {dot_produ
ct(a_norm, c_norm):>10.4f}")
print()
print(" After normalization, cosine and dot product are identical.")
print()

def demo_mahalanobis():
    print("=" * 65)
    print("MAHALANOBIS DISTANCE")
    print("=" * 65)

    random.seed(42)
    n = 200
    data = []
    for _ in range(n):
        x = random.gauss(0, 3)
        y = 0.8 * x + random.gauss(0, 1)
        data.append([x, y])

    cov = compute_covariance(data)
    mean = [sum(d[0] for d in data) / n, sum(d[1] for d in data) / n]

    point_along = [mean[0] + 3, mean[1] + 0.8 * 3]
    point_perp = [mean[0] + 1, mean[1] - 3]

```



```

l2_along = l2_distance(mean, point_along)
l2_perp = l2_distance(mean, point_perp)
mah_along = mahalanobis_distance(mean, point_along, cov)
mah_perp = mahalanobis_distance(mean, point_perp, cov)

print(f" Data: {n} points with correlated features (r ~ 0.8)")
print(f" Mean: ({mean[0]:.2f}, {mean[1]:.2f})")
print(f" Covariance: [[{cov[0][0]:.2f}, {cov[0][1]:.2f}], [{cov[1][0]:.2f}, {c
ov[1][1]:.2f}]]")
print()
print(f" Point along correlation axis: {[round(x, 2) for x in point_along]}")
print(f" L2 distance from mean: {l2_along:.4f}")
print(f" Mahalanobis distance: {mah_along:.4f}")
print()
print(f" Point perpendicular to axis: {[round(x, 2) for x in point_perp]}")
print(f" L2 distance from mean: {l2_perp:.4f}")
print(f" Mahalanobis distance: {mah_perp:.4f}")
print()
print(" L2 says both points are similar distances from the mean.")
print(" Mahalanobis correctly identifies the perpendicular point as")
print(" more unusual given the correlation structure of the data.")
print()

def demo_jaccard():
    print("=" * 65)
    print("JACCARD SIMILARITY (SETS)")
    print("=" * 65)

    pairs = [
        ({"cat", "dog", "fish"}, {"cat", "bird", "fish", "snake"}),
        ({"python", "java", "rust"}, {"python", "java", "rust"}),
        ({"a", "b", "c"}, {"d", "e", "f"}),
        ({"ml", "ai", "data"}, {"ml", "ai", "data", "ops", "cloud"}),
    ]

    for a, b in pairs:
        j = jaccard_similarity(a, b)
        print(f" A = {sorted(a)}")
        print(f" B = {sorted(b)}")
        print(f" Jaccard similarity: {j:.4f}")
        print(f" Jaccard distance: {1 - j:.4f}")
        print()

def demo_edit_distance():
    print("=" * 65)
    print("EDIT DISTANCE (LEVENSHTEIN)")
    print("=" * 65)

```

```

pairs = [
    ("kitten", "sitting"),
    ("sunday", "saturday"),
    ("hello", "hello"),
    ("", "abc"),
    ("algorithm", "altruistic"),
    ("python", "pytorch"),
]

for s1, s2 in pairs:
    d = edit_distance(s1, s2)
    print(f" '{s1}' -> '{s2}': distance = {d}")

print()

def demo_kl_divergence():
    print("=" * 65)
    print("KL DIVERGENCE (NOT SYMMETRIC)")
    print("=" * 65)

    p = [0.9, 0.1]
    q = [0.5, 0.5]

    kl_pq = kl_divergence(p, q)
    kl_qp = kl_divergence(q, p)

    print(f" P = {p}")
    print(f" Q = {q}")
    print(f" KL(P || Q) = {kl_pq:.4f} nats")
    print(f" KL(Q || P) = {kl_qp:.4f} nats")
    print(f" Difference: {abs(kl_pq - kl_qp):.4f}")
    print(f" KL divergence is NOT a distance metric.")
    print()

    p2 = [0.25, 0.25, 0.25, 0.25]
    q2 = [0.1, 0.1, 0.1, 0.7]

    print(f" P = {p2}")
    print(f" Q = {q2}")
    print(f" KL(P || Q) = {kl_divergence(p2, q2):.4f} nats")
    print(f" KL(Q || P) = {kl_divergence(q2, p2):.4f} nats")
    print()

def demo_wasserstein():
    print("=" * 65)
    print("WASSERSTEIN DISTANCE (EARTH MOVER'S DISTANCE)")
    print("=" * 65)

    cases = [

```

```

    ("Identical",
     [0.25, 0.25, 0.25, 0.25],
     [0.25, 0.25, 0.25, 0.25]),
    ("Shifted right by 1",
     [0.5, 0.5, 0.0, 0.0],
     [0.0, 0.5, 0.5, 0.0]),
    ("Shifted right by 2",
     [0.5, 0.5, 0.0, 0.0],
     [0.0, 0.0, 0.5, 0.5]),
    ("Opposite ends",
     [1.0, 0.0, 0.0, 0.0],
     [0.0, 0.0, 0.0, 1.0]),
    ("Spread vs concentrated",
     [0.25, 0.25, 0.25, 0.25],
     [0.0, 0.0, 0.0, 1.0]),
]

for name, p, q in cases:
    w = wasserstein_1d(p, q)
    kl = kl_divergence(p, q)
    kl_str = f"{kl:.4f}" if kl != float('inf') else "inf"
    print(f"    {name}")
    print(f"        P = {p}")
    print(f"        Q = {q}")
    print(f"        Wasserstein: {w:.4f}    KL: {kl_str}")
    print()

print("    Wasserstein provides finite, meaningful distances even when")
print("    distributions do not overlap (where KL goes to infinity).")
print()

def demo_different_neighbors():
    print("=" * 65)
    print("SAME DATA, DIFFERENT METRICS, DIFFERENT NEAREST NEIGHBORS")
    print("=" * 65)

    random.seed(123)
    n_points = 8
    dim = 5

    dataset = []
    for i in range(n_points):
        if i < 3:
            point = [random.gauss(0, 1) for _ in range(dim)]
        elif i < 6:
            base = [random.gauss(0, 0.5) for _ in range(dim)]
            base[0] *= 5
            point = base
        else:
            point = [random.gauss(3, 0.3) for _ in range(dim)]
    dataset.append(point)

```

```

query = [1.0, 0.5, -0.5, 1.0, 0.2]

print(f" Query: {[round(x, 2) for x in query]}")
print()
print(f" {'Point':<8s} {'L1':>8s} {'L2':>8s} {'Cosine':>8s} {'L-inf':>8s}")
print(f" {'-' * 8} {'-' * 8} {'-' * 8} {'-' * 8} {'-' * 8}")

results = {"L1": [], "L2": [], "Cosine": [], "L-inf": []}

for i, point in enumerate(dataset):
    d_l1 = l1_distance(query, point)
    d_l2 = l2_distance(query, point)
    d_cos = cosine_distance(query, point)
    d_linf = linf_distance(query, point)

    results["L1"].append((i, d_l1))
    results["L2"].append((i, d_l2))
    results["Cosine"].append((i, d_cos))
    results["L-inf"].append((i, d_linf))

    print(f" P{i:<6d} {d_l1:>8.3f} {d_l2:>8.3f} {d_cos:>8.4f} {d_linf:>8.3
f}")

print()
print(" Nearest neighbor by metric:")
for metric_name, dists in results.items():
    best = min(dists, key=lambda x: x[1])
    print(f" {metric_name:<8s}: Point {best[0]} (distance = {best[1]:.4f})")

l1_best = min(results["L1"], key=lambda x: x[1])[0]
l2_best = min(results["L2"], key=lambda x: x[1])[0]
cos_best = min(results["Cosine"], key=lambda x: x[1])[0]
linf_best = min(results["L-inf"], key=lambda x: x[1])[0]

all_same = (l1_best == l2_best == cos_best == linf_best)
if not all_same:
    print()
    print(" The metrics DISAGREE on which point is nearest.")
    print(" Your distance function defines your notion of similarity.")
print()

def demo_embedding_search():
    print("=" * 65)
    print("EMBEDDING SIMILARITY SEARCH")
    print("=" * 65)

    random.seed(77)
    dim = 64

    documents = [

```

```

"machine learning algorithms",
"deep neural networks",
"natural language processing",
"computer vision models",
"reinforcement learning agents",
"database query optimization",
"web server configuration",
"network security protocols",
]

embeddings = []
for i, doc in enumerate(documents):
    base = [random.gauss(0, 1) for _ in range(dim)]
    if i < 5:
        for j in range(10):
            base[j] += 2.0
    else:
        for j in range(10, 20):
            base[j] += 2.0
    if i in [0, 1]:
        for j in range(20, 25):
            base[j] += 1.5
    embeddings.append(base)

query_embedding = embeddings[0][:]
noise = [random.gauss(0, 0.3) for _ in range(dim)]
query_embedding = [q + n for q, n in zip(query_embedding, noise)]

print(f" Query: '{documents[0]}' (with noise)")
print(f" Embedding dimension: {dim}")
print()

cosine_scores = []
l2_scores = []
dot_scores = []

for i in range(len(documents)):
    cos = cosine_similarity(query_embedding, embeddings[i])
    l2 = l2_distance(query_embedding, embeddings[i])
    dp = dot_product(query_embedding, embeddings[i])
    cosine_scores.append((i, cos))
    l2_scores.append((i, l2))
    dot_scores.append((i, dp))

cosine_ranked = sorted(cosine_scores, key=lambda x: -x[1])
l2_ranked = sorted(l2_scores, key=lambda x: x[1])
dot_ranked = sorted(dot_scores, key=lambda x: -x[1])

print(f" {'Rank':<6s} {'Cosine':<35s} {'L2':<35s} {'Dot Product':<35s}")
print(f" {'-' * 6} {'-' * 35} {'-' * 35} {'-' * 35}")
for rank in range(len(documents)):
    ci, cs = cosine_ranked[rank]

```

```

    li, ls = l2_ranked[rank]
    di, ds = dot_ranked[rank]
    cos_str = f"{documents[ci][:25]:<25s} ({cs:.3f})"
    l2_str = f"{documents[li][:25]:<25s} ({ls:.1f})"
    dot_str = f"{documents[di][:25]:<25s} ({ds:.1f})"
    print(f" {rank + 1:<6d} {cos_str:<35s} {l2_str:<35s} {dot_str:<35s}")

print()
print(" Cosine similarity focuses on direction (topic similarity).")
print(" L2 distance is sensitive to magnitude differences.")
print(" Dot product blends direction and magnitude.")
print()

def demo_knn_classification():
    print("=" * 65)
    print("KNN CLASSIFICATION: DISTANCE METRIC CHANGES THE PREDICTION")
    print("=" * 65)

    random.seed(99)

    training_data = [
        ([1.0, 5.0], "A"),
        ([1.5, 4.5], "A"),
        ([2.0, 4.0], "A"),
        ([5.0, 1.0], "B"),
        ([4.5, 1.5], "B"),
        ([4.0, 2.0], "B"),
        ([3.0, 3.0], "C"),
        ([3.5, 2.5], "C"),
        ([2.5, 3.5], "C"),
    ]

    query = [2.8, 2.8]

    print(f" Query: {query}")
    print(f" Training set: {len(training_data)} points, 3 classes")
    print()

    k = 3
    for metric_name, dist_fn in [("L1", l1_distance), ("L2", l2_distance),
                                ("Cosine", cosine_distance), ("L-inf", linf_distance)
    ]:
        distances = []
        for point, label in training_data:
            d = dist_fn(query, point)
            distances.append((d, label, point))
        distances.sort(key=lambda x: x[0])

        neighbors = distances[:k]
        votes = {}
        for d, label, point in neighbors:

```

```

        votes[label] = votes.get(label, 0) + 1
    prediction = max(votes, key=votes.get)

    print(f" Metric: {metric_name}")
    for d, label, point in neighbors:
        print(f" Neighbor: {point} class={label} dist={d:.4f}")
    print(f" Prediction (k={k}): {prediction}")
    print()

def demo_regularization():
    print("=" * 65)
    print("L1 vs L2 REGULARIZATION EFFECT ON WEIGHTS")
    print("=" * 65)

    random.seed(42)
    n_features = 10
    weights = [random.gauss(0, 2) for _ in range(n_features)]

    print(f" Original weights: {[round(w, 3) for w in weights]}")
    print(f" L1 norm: {l1_norm(weights):.4f}")
    print(f" L2 norm: {l2_norm(weights):.4f}")
    print()

    lr = 0.1

    w_l1 = weights[:]
    for step in range(50):
        for i in range(n_features):
            grad = lr * (1 if w_l1[i] > 0 else (-1 if w_l1[i] < 0 else 0))
            w_l1[i] -= grad
            if abs(w_l1[i]) < lr:
                w_l1[i] = 0.0

    w_l2 = weights[:]
    for step in range(50):
        for i in range(n_features):
            grad = lr * 2 * w_l2[i]
            w_l2[i] -= grad

    print(f" After L1 regularization (50 steps):")
    print(f" Weights: {[round(w, 3) for w in w_l1]}")
    print(f" Zeros: {sum(1 for w in w_l1 if w == 0.0)}/{n_features}")
    print(f" L1 norm: {l1_norm(w_l1):.4f}")
    print()
    print(f" After L2 regularization (50 steps):")
    print(f" Weights: {[round(w, 3) for w in w_l2]}")
    print(f" Zeros: {sum(1 for w in w_l2 if abs(w) < 1e-10)}/{n_features}")
    print(f" L2 norm: {l2_norm(w_l2):.4f}")
    print()

```

```

print(" L1 drives 'small' weights to exactly zero (sparsity).")
print(" L2 shrinks all weights but none reach exactly zero.")
print()

def demo_norm_ordering():
    print("=" * 65)
    print("NORM ORDERING: L-inf <= L2 <= L1 (always)")
    print("=" * 65)

    random.seed(55)
    for trial in range(5):
        dim = random.randint(2, 10)
        a = [random.gauss(0, 5) for _ in range(dim)]
        b = [random.gauss(0, 5) for _ in range(dim)]

        d1 = l1_distance(a, b)
        d2 = l2_distance(a, b)
        dinf = linf_distance(a, b)

        holds = dinf <= d2 <= d1
        print(f" dim={dim:>2d} L1={d1:>8.3f} L2={d2:>8.3f} L-inf={dinf:>8.3f}
} ordering holds: {holds}")

    print()
    print(" For any p1 < p2: ||x||_p2 <= ||x||_p1")
    print(" Higher p values focus on fewer (larger) components.")
    print()

if __name__ == "__main__":
    demo_norms()
    demo_distances()
    demo_cosine_vs_dot()
    demo_mahalanobis()
    demo_jaccard()
    demo_edit_distance()
    demo_kl_divergence()
    demo_wasserstein()
    demo_norm_ordering()
    demo_different_neighbors()
    demo_embedding_search()
    demo_knn_classification()
    demo_regularization()

```

You are a distance metric advisor for machine learning and data science practitioners. Your job is to recommend the right distance or similarity function for a given task.

When a user describes their problem, ask clarifying questions if needed, then recommend a specific distance metric. Structure your response as:

1. Recommended distance metric and why

2. How to implement it (formula and code snippet)
3. Common pitfalls with this metric
4. When to switch to a different metric
5. If using a vector database, which index type pairs best

Use this decision framework:

Text similarity (embeddings, documents, queries): - Use cosine similarity. Text embeddings encode meaning in direction, not magnitude. Longer documents should not be penalized. - If embeddings are already L2-normalized, dot product is equivalent and faster. - Avoid L2 distance for text. A short document and a long document about the same topic will have large L2 distance despite similar meaning.

Image similarity (pixel-level): - Use L2 distance for raw pixel comparisons. - Use cosine similarity for learned image embeddings (CLIP, ResNet features). - Avoid L1 for pixel data. It does not match human perception of image similarity.

Recommendation systems: - Use dot product when magnitude encodes confidence or popularity. - Use cosine similarity when you want pure preference direction regardless of engagement volume. - Consider matrix factorization methods that learn the right similarity implicitly.

Set-valued data (tags, categories, binary features): - Use Jaccard similarity. It handles variable-size sets correctly. - For approximate Jaccard on large sets, use MinHash with locality-sensitive hashing. - Do not convert sets to vectors just to use cosine. Jaccard is the natural metric.

String matching (names, addresses, typo correction): - Use edit distance (Levenshtein) for general string similarity. - Use Jaro-Winkler for short strings like names (gives more weight to matching prefixes). - For phonetic matching, combine with Soundex or Metaphone.

Outlier detection: - Use Mahalanobis distance. It accounts for correlations between features. - Requires a reliable covariance matrix estimate. Need at least 10x more samples than features. - Falls back to L2 when features are uncorrelated and same-scale.

Comparing probability distributions: - Use KL divergence when one distribution is a reference (true distribution) and you want to measure how far the other is. - Remember KL is not symmetric. $D_{KL}(P || Q) \neq D_{KL}(Q || P)$. - Use Wasserstein distance when distributions may not overlap or when you need a true metric. - Use Jensen-Shannon divergence (symmetrized KL) when you need symmetry but both distributions are continuous.

GAN training: - Use Wasserstein distance. It provides meaningful gradients when generator and discriminator distributions do not overlap. - Original GAN loss (based on JSD/KL) has vanishing gradient problems that Wasserstein avoids.

High-dimensional sparse data (bag-of-words, one-hot encodings): - Use cosine similarity for TF-IDF vectors. - Use L1 distance when robustness to outliers matters. - Avoid L2 in very high dimensions. All pairwise L2 distances converge to similar values (curse of dimensionality).

Time series: - Use Dynamic Time Warping (DTW) for sequences of different lengths or with temporal shifts. - Use L2 on aligned, same-length sequences. - Avoid cosine similarity for raw time series. Temporal ordering matters and cosine ignores it.

Graph or network data: - Use graph edit distance for small graphs. - Use graph kernels (Weisfeiler-Lehman, random walk) for comparing graph structures. - For node similarity within a graph, use shortest path distance or commute time distance.

Manufacturing and quality control: - Use L^∞ distance when every dimension must be within tolerance. - Use Mahalanobis distance for multivariate process monitoring.

Choosing between approximate nearest neighbor algorithms: - HNSW: best recall/speed tradeoff for most use cases. Default choice for vector databases. - IVF: good for very large datasets (billions). Needs training on representative data. - LSH: fast and simple for approximate nearest neighbors. Works well with cosine and Jaccard. - Product quantization: when memory is the bottleneck. Compresses vectors at cost of some accuracy.

Common mistakes to warn about: - Using L2 distance on unnormalized features. Always standardize first unless features are naturally comparable. - Using cosine similarity on sparse binary vectors with few nonzero entries. Jaccard is usually better. - Assuming KL divergence is symmetric. It is not. Always specify direction. - Using L2 in very high dimensions without checking whether pairwise distances have collapsed. - Forgetting to handle zero vectors when computing cosine similarity (division by zero). - Using edit distance on long strings without considering the $O(n*m)$ time and space cost.

Quiz

Pre-lesson check

1. What does cosine similarity measure between two vectors?
 - a) The Euclidean distance between them
 - b) The angle between them, ignoring their magnitudes **(correct)**
 - c) The sum of their element-wise products
 - d) The maximum difference in any single dimension

Explanation: Cosine similarity measures the cosine of the angle between two vectors,

normalized by their magnitudes. It ranges from -1 (opposite) to +1 (same direction) and ignores vector length, which is why it dominates in NLP where document length is noise.

2. Why is L1 distance called 'Manhattan distance'?

- a) It was invented in Manhattan
- b) It measures distance along a grid, as if walking on city blocks **(correct)**
- c) It only works in exactly two dimensions
- d) It computes the maximum difference across all dimensions

Explanation: L1 distance sums the absolute differences along each axis, which corresponds to the shortest path on a grid where you can only move along axes (no diagonals) — like walking on a city street grid.

Post-lesson check

3. Why does L1 regularization (Lasso) produce sparse weights while L2 regularization (Ridge) does not?

- a) L1 uses a larger penalty constant than L2
- b) The L1 diamond constraint has corners on the axes where weights are zero; loss contours are likely to touch at corners **(correct)**
- c) L2 regularization only applies to the bias terms
- d) L1 regularization clips weights below a threshold to zero

Explanation: The L1 constraint region is a diamond shape with corners aligned with the axes. The loss function's contour ellipses are most likely to first touch the diamond at a corner, where one or more weights are exactly zero. The L2 constraint is a circle with no corners, so the touch point has all weights nonzero.

4. What advantage does Wasserstein distance have over KL divergence when comparing probability distributions?

- a) Wasserstein is always smaller than KL divergence

b) Wasserstein provides meaningful gradients even when distributions do not overlap, while KL goes to ∞

c) Wasserstein is symmetric but KL is not

d) Both B and C are correct **(correct)**

Explanation: Wasserstein distance is a true metric (symmetric, satisfies triangle inequality) and provides gradients even for non-overlapping distributions. KL divergence is asymmetric and goes to ∞ when distributions don't overlap, which is why WGANs replaced KL-based GANs.

5. When would you use Mahalanobis distance instead of Euclidean distance?

a) When features are all binary (0 or 1)

b) When features have different scales and are correlated with each other **(correct)**

c) When you are comparing strings instead of vectors

d) When you need a faster computation than L2

Explanation: Mahalanobis distance accounts for the covariance structure of the data, decorrelating and normalizing features before computing L2 distance. It correctly handles features at different scales and with correlations, whereas Euclidean distance treats all dimensions equally.

Statistics for Machine Learning

Statistics is how you know if your model actually works or just got lucky.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 06 (Probability and Distributions), 07 (Bayes' Theorem) **Time:** ~120 minutes

Learning Objectives

- Compute descriptive statistics, Pearson/Spearman correlation, and covariance matrices from scratch
- Perform hypothesis tests (t-test, chi-squared) and interpret p-values and confidence intervals correctly
- Use bootstrap resampling to construct confidence intervals for any metric without distributional assumptions
- Distinguish statistical significance from practical significance using effect size measures

The Problem

You trained two models. Model A scores 0.87 on your test set. Model B scores 0.89. You deploy Model B. Three weeks later, production metrics are worse than before. What happened?

Model B did not actually outperform Model A. The 0.02 difference was noise. Your test set was too small, or the variance too high, or both. You shipped randomness dressed up as improvement.

This happens constantly. Kaggle leaderboard shakeups. Papers that fail to reproduce. A/B tests that declare winners based on a few hundred samples. The root cause is always the same: someone skipped the statistics.

Statistics gives you the tools to distinguish signal from noise. It tells you when a difference is real, how confident you should be, and how much data you need before you can trust a result. Every ML pipeline, every model comparison, every experiment needs statistics. Without it, you are guessing.

The Concept

Descriptive Statistics: Summarizing Your Data

Before you model anything, you need to know what your data looks like. Descriptive statistics compress a dataset into a few numbers that capture its shape.

Measures of central tendency answer “where is the middle?”

```
Mean:    sum of all values / count
         mu = (1/n) * sum(x_i)
```

Median: middle value when sorted
Robust to outliers. If you have [1, 2, 3, 4, 1000], the mean is 202
but the median is 3.

Mode: most frequent value
Useful for categorical data. For continuous data, rarely informative.

The mean is the balance point. The median is the halfway mark. When they diverge, your distribution is skewed. Income distributions have mean >> median (right skew from billionaires). Loss distributions during training often have mean << median (left skew from easy samples).

Measures of spread answer “how dispersed is the data?”

Variance: average squared deviation from the mean
 $\sigma^2 = (1/n) * \sum((x_i - \mu)^2)$

Standard deviation: square root of variance
 $\sigma = \sqrt{\sigma^2}$
Same units as the data, so more interpretable.

Range: max - min
Sensitive to outliers. Almost never useful alone.

IQR: Q3 - Q1 (interquartile range)
The range of the middle 50% of the data.
Robust to outliers. Used for box plots and outlier detection.

Percentiles divide sorted data into 100 equal parts. The 25th percentile (Q1) means 25% of values fall below this point. The 50th percentile is the median. The 75th percentile is Q3.

For latency monitoring:

P50 = median latency	(typical user experience)
P95 = 95th percentile	(bad but not worst case)
P99 = 99th percentile	(tail latency, often 10x the median)

In ML, you care about percentiles for inference latency, prediction confidence distributions, and understanding error distributions. A model with low average error but terrible P99 error might be useless for safety-critical applications.

Sample vs population statistics. When computing variance from a sample, divide by (n-1) instead of n. This is Bessel’s correction. It compensates for the fact that your sample mean is not the true population

mean. With n in the denominator, you systematically underestimate the true variance. With $(n-1)$, the estimate is unbiased.

```
Population variance:  $\sigma^2 = (1/N) * \sum((x_i - \mu)^2)$   
Sample variance:  $s^2 = (1/(n-1)) * \sum((x_i - \bar{x})^2)$ 
```

In practice: if n is large (thousands of samples), the difference is negligible. If n is small (dozens of samples), it matters.

Correlation: How Variables Move Together

Correlation measures the strength and direction of a linear relationship between two variables.

Pearson correlation coefficient measures linear association:

```
 $r = \frac{\sum((x_i - \bar{x})(y_i - \bar{y}))}{(n * s_x * s_y)}$   
  
 $r = +1$ : perfect positive linear relationship  
 $r = -1$ : perfect negative linear relationship  
 $r = 0$ : no linear relationship (but there might be a nonlinear one!)  
  
Range:  $[-1, 1]$ 
```

Pearson assumes the relationship is linear and both variables are roughly normally distributed. It is sensitive to outliers. A single extreme point can drag r from 0.1 to 0.9.

Spearman rank correlation measures monotonic association:

```
1. Replace each value with its rank (1, 2, 3, ...)  
2. Compute Pearson correlation on the ranks  
  
Spearman catches any monotonic relationship, not just linear.  
If  $y = x^3$ , Pearson gives  $r < 1$  but Spearman gives  $\rho = 1$ .
```

When to use each:

```
Pearson: Both variables are continuous and roughly normal.  
          You care about the linear relationship specifically.  
          No extreme outliers.  
  
Spearman: Ordinal data (rankings, ratings).  
           Data is not normally distributed.  
           You suspect a monotonic but not linear relationship.  
           Outliers are present.
```

The golden rule: correlation does not imply causation. Ice cream sales and drowning deaths are correlated because both increase in summer. Your model's accuracy and the number of parameters are correlated, but adding parameters does not automatically improve accuracy (see: overfitting).

Covariance Matrix

The covariance between two variables measures how they vary together:

$$\text{Cov}(X, Y) = (1/n) * \sum((x_i - \bar{x})(y_i - \bar{y}))$$

$\text{Cov}(X, Y) > 0$: X and Y tend to increase together

$\text{Cov}(X, Y) < 0$: when X increases, Y tends to decrease

$\text{Cov}(X, Y) = 0$: no linear co-movement

For d features, the covariance matrix C is a d x d matrix where $C[i][j] = \text{Cov}(\text{feature}_i, \text{feature}_j)$. The diagonal entries $C[i][i]$ are the variances of each feature.

$$C = \begin{vmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \text{Cov}(x_1, x_3) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \text{Cov}(x_2, x_3) \\ \text{Cov}(x_3, x_1) & \text{Cov}(x_3, x_2) & \text{Var}(x_3) \end{vmatrix}$$

Properties:

- Symmetric: $C[i][j] = C[j][i]$
- Positive semi-definite: all eigenvalues ≥ 0
- Diagonal = variances
- Off-diagonal = covariances

Connection to PCA. PCA eigendecomposes the covariance matrix. The eigenvectors are the principal components (directions of maximum variance). The eigenvalues tell you how much variance each component captures. This is exactly what Lesson 10 covered, but now you see why the covariance matrix is the right thing to decompose: it encodes all pairwise linear relationships in your data.

Connection to correlation. The correlation matrix is the covariance matrix of standardized variables (each divided by its standard deviation). Correlation normalizes covariance so all values fall in $[-1, 1]$.

Hypothesis Testing

Hypothesis testing is a framework for making decisions under uncertainty. You start with a claim, collect data, and determine if the data is consistent with the claim.

The setup:

Null hypothesis (H_0): the default assumption, usually "no effect"
Alternative hypothesis (H_1): what you are trying to show

Example:

H_0 : Model A and Model B have the same accuracy

H_1 : Model B has higher accuracy than Model A

The p-value is the probability of seeing data as extreme as what you observed, assuming H_0 is true. It is NOT the probability that H_0 is true. This is the single most common misunderstanding in statistics.

$p\text{-value} = P(\text{data this extreme} \mid H_0 \text{ is true})$

If $p\text{-value} < \alpha$ (typically 0.05):

Reject H_0 . The result is "statistically significant."

If $p\text{-value} \geq \alpha$:

Fail to reject H_0 . You do not have enough evidence.

This does NOT mean H_0 is true.

Confidence intervals give a range of plausible values for a parameter:

95% confidence interval for the mean:

$\bar{x} \pm z * (s / \sqrt{n})$

where $z = 1.96$ for 95% confidence

Interpretation: if you repeated this experiment many times, 95% of the computed intervals would contain the true mean. It does NOT mean there is a 95% probability the true mean is in this specific interval.

The width of the confidence interval tells you about precision. Wide intervals mean high uncertainty. Narrow intervals mean your estimate is precise (but not necessarily accurate, if your data is biased).

The t-test

The t-test compares means. There are several flavors.

One-sample t-test: is the population mean different from a hypothesized value?

$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$

degrees of freedom = $n - 1$

Two-sample t-test (independent): are two group means different?

$t = (\bar{x}_1 - \bar{x}_2) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$

This is Welch's t-test, which does not assume equal variances.

Always use Welch's unless you have a specific reason for equal variance s.

Paired t-test: when measurements come in pairs (same model evaluated on same data splits):

Compute $d_i = x_i - y_i$ for each pair
Then run a one-sample t-test on the d_i values against $\mu_0 = 0$

In ML, the paired t-test is common: you run both models on the same 10 cross-validation folds and compare their scores pairwise.

Chi-squared Test

The chi-squared test checks if observed frequencies match expected frequencies. Useful for categorical data.

```
chi^2 = sum((observed - expected)^2 / expected)
```

Example: does a language model's output distribution match the training distribution across categories?

Category	Observed	Expected
Positive	120	100
Negative	80	100

$chi^2 = (120-100)^2/100 + (80-100)^2/100 = 4 + 4 = 8$

With 1 degree of freedom, $chi^2 = 8$ gives $p < 0.005$.
The difference is significant.

A/B Testing for ML Models

A/B testing in ML is not the same as web A/B testing. Model comparison has specific challenges:

1. Same test set: Both models must be evaluated on identical data. Different test sets make comparison meaningless.
2. Multiple metrics: Accuracy alone is not enough. You need precision, recall, F1, latency, and fairness metrics.
3. Variance: Use cross-validation or bootstrap to estimate the variance of each metric, not just point estimates.
4. Data leakage: If the test set was used during model selection,

your comparison is biased. Hold out a final test set.

The procedure:

1. Define your metric and significance level ($\alpha = 0.05$)
2. Run both models on the same k-fold cross-validation splits
3. Collect paired scores: $[(a_1, b_1), (a_2, b_2), \dots, (a_k, b_k)]$
4. Compute differences: $d_i = b_i - a_i$
5. Run a paired t-test on the differences
6. Check: is the mean difference significantly different from 0?
7. Compute a confidence interval for the mean difference
8. Compute effect size (Cohen's d) to judge practical significance

Statistical Significance vs Practical Significance

A result can be statistically significant but practically meaningless. With enough data, even a trivial difference becomes statistically significant.

Example:

Model A accuracy: 0.9234
Model B accuracy: 0.9237
 $n = 1,000,000$ test samples
p-value = 0.001

Statistically significant? Yes.

Practically significant? A 0.03% improvement is not worth the engineering cost of deploying a new model.

Effect size quantifies how big the difference is, independent of sample size:

$$\text{Cohen's } d = (\text{mean}_1 - \text{mean}_2) / \text{pooled_std}$$

$d = 0.2$: small effect
 $d = 0.5$: medium effect
 $d = 0.8$: large effect

Always report both the p-value and the effect size. The p-value tells you if the difference is real. The effect size tells you if it matters.

Multiple Comparison Problem

When you test many hypotheses, some will be “significant” by chance. If you test 20 things at $\alpha = 0.05$, you expect 1 false positive even when nothing is real.

$P(\text{at least one false positive}) = 1 - (1 - \alpha)^m$

$m = 20$ tests, $\alpha = 0.05$:

$P(\text{false positive}) = 1 - 0.95^{20} = 0.64$

You have a 64% chance of at least one false positive.

Bonferroni correction: divide α by the number of tests.

Adjusted $\alpha = \alpha / m = 0.05 / 20 = 0.0025$

Only reject H_0 if $p\text{-value} < 0.0025$.

Conservative but simple. Works when tests are independent.

In ML, this matters when you compare a model across multiple metrics, test many hyperparameter configurations, or evaluate on multiple datasets.

Bootstrap Methods

Bootstrapping estimates the sampling distribution of a statistic by resampling your data with replacement. No assumptions about the underlying distribution required.

The algorithm:

1. You have n data points
2. Draw n samples WITH replacement (some points appear multiple times, some not at all)
3. Compute your statistic on this bootstrap sample
4. Repeat B times (typically $B = 1000$ to 10000)
5. The distribution of bootstrap statistics approximates the sampling distribution

Bootstrap confidence interval (percentile method):

Sort the B bootstrap statistics

95% CI = [2.5th percentile, 97.5th percentile]

Why bootstrap matters for ML:

- Test set accuracy is a point estimate. Bootstrap gives you confidence intervals.
- You cannot assume metric distributions are normal (especially for AUC, F1, precision at k).
- Bootstrap works for ANY statistic: median, ratio of two means,

- difference in AUC between two models.
- No closed-form formula needed.

Bootstrap for model comparison:

1. You have predictions from Model A and Model B on the same test set
2. For each bootstrap iteration:
 - a. Resample test indices with replacement
 - b. Compute `metric_A` and `metric_B` on the resampled set
 - c. Store `diff = metric_B - metric_A`
3. 95% CI for the difference:
[2.5th percentile of `diffs`, 97.5th percentile of `diffs`]
4. If the CI does not contain 0, the difference is significant

This is more robust than the paired t-test because it makes no distributional assumptions.

Parametric vs Non-parametric Tests

Parametric tests assume a specific distribution (usually normal):

t-test:	assumes normally distributed data (or large n by CLT)
ANOVA:	assumes normality and equal variances
Pearson r:	assumes bivariate normality

Non-parametric tests make no distributional assumptions:

Mann-Whitney U:	compares two groups (replaces independent t-test)
Wilcoxon signed-rank:	compares paired data (replaces paired t-test)
Spearman rho:	correlation on ranks (replaces Pearson)
Kruskal-Wallis:	compares multiple groups (replaces ANOVA)

When to use non-parametric:

- Small sample size ($n < 30$) and data is clearly non-normal
- Ordinal data (ratings, rankings)
- Heavy outliers you cannot remove
- Skewed distributions

When to use parametric:

- Large sample size (CLT makes the test statistic approximately normal)
- Data is roughly symmetric without extreme outliers
- More statistical power (better at detecting real differences)

In ML experiments, you typically have small n (5 or 10 cross-validation folds), so non-parametric tests like Wilcoxon signed-rank are often more appropriate than t-tests.

Central Limit Theorem: Practical Implications

The CLT says the distribution of sample means approaches a normal distribution as n grows, regardless of the underlying population distribution.

If $X_1, X_2, \dots, X_n$ are iid with mean μ and variance σ^2 :

$$\bar{X} \sim \text{Normal}(\mu, \sigma^2 / n) \quad \text{as } n \rightarrow \infty$$

Works for $n \geq 30$ in most cases.

For highly skewed distributions, you might need $n \geq 100$.

Why this matters for ML:

1. Justifies confidence intervals and t-tests on aggregated metrics
2. Explains why averaging over cross-validation folds gives stable estimates even when individual folds vary wildly
3. Mini-batch gradient descent works because the average gradient over a batch approximates the true gradient (CLT in action)
4. Ensemble methods: averaging predictions from many models gives more stable output than any single model

What CLT does NOT do:

- Does NOT make your data normal. It makes the MEAN of samples normal.
- Does NOT work for heavy-tailed distributions with infinite variance (Cauchy distribution).
- Does NOT apply to dependent data (time series without correction).

Common Statistical Mistakes in ML Papers

1. **Testing on the training set.** Guarantees overfitting. Always hold out data the model never sees during training.
2. **No confidence intervals.** Reporting a single accuracy number without uncertainty makes results unreproducible and unverifiable.
3. **Ignoring multiple comparisons.** Testing 50 configurations and reporting the best one without correction inflates false positive rates.
4. **Confusing statistical and practical significance.** A p-value of 0.001 on a 0.01% accuracy improvement is not meaningful.

5. **Using accuracy on imbalanced data.** 99% accuracy on a dataset with 99% negative class means the model learned nothing. Use precision, recall, F1, or AUC.
6. **Cherry-picking metrics.** Reporting only the metric where your model wins. Honest evaluation reports all relevant metrics.
7. **Leaking information across train/test splits.** Normalizing before splitting, or using future data to predict the past.
8. **Small test sets with no variance estimates.** Evaluating on 100 samples and claiming 2% improvement is noise, not signal.
9. **Assuming independence when data is not independent.** Medical images from the same patient, multiple sentences from the same document. Observations within a group are correlated.
10. **P-hacking.** Trying different tests, subsets, or exclusion criteria until you get $p < 0.05$. The result is an artifact of the search.

Building It

You will implement:

1. **Descriptive statistics from scratch** (mean, median, mode, standard deviation, percentiles, IQR)
2. **Correlation functions** (Pearson and Spearman, with the covariance matrix)
3. **Hypothesis tests** (one-sample t-test, two-sample t-test, chi-squared test)
4. **Bootstrap confidence intervals** (for any statistic, no assumptions needed)
5. **A/B test simulator** (generate data, test, check for Type I and Type II errors)
6. **Statistical vs practical significance demo** (showing that large n makes everything “significant”)

All from scratch, using only math and random. No numpy, no scipy.

Key Terms

Term	Definition
Mean	Sum of values divided by count. Sensitive to outliers.
Median	Middle value of sorted data. Robust to outliers.
Standard deviation	Square root of variance. Measures spread in original units.
Percentile	Value below which a given percentage of data falls.
IQR	Interquartile range. Q3 minus Q1. The spread of the middle 50%.

Term	Definition
Pearson correlation	Measures linear association between two variables. Range [-1, 1].
Spearman correlation	Measures monotonic association using ranks.
Covariance matrix	Matrix of pairwise covariances between all features.
Null hypothesis	Default assumption of no effect or no difference.
p-value	Probability of data this extreme given the null hypothesis is true.
Confidence interval	Range of plausible values for a parameter at a given confidence level.
t-test	Tests whether means differ significantly. Uses the t-distribution.
Chi-squared test	Tests whether observed frequencies differ from expected frequencies.
Effect size	Magnitude of a difference, independent of sample size. Cohen's d is common.
Bonferroni correction	Divides significance threshold by number of tests to control false positives.
Bootstrap	Resampling with replacement to estimate sampling distributions.
Type I error	False positive. Rejecting H0 when it is true.
Type II error	False negative. Failing to reject H0 when it is false.
Statistical power	Probability of correctly rejecting a false H0. Power = 1 minus Type II error rate.
Central limit theorem	Sample means converge to a normal distribution as sample size grows.
Parametric test	Assumes a specific distribution for the data (usually normal).
Non-parametric test	Makes no distributional assumptions. Works on ranks or signs.

Code Appendix: statistics.py

```

import math
import random

random.seed(42)

def mean(data):
    return sum(data) / len(data)

```



```

def median(data):
    s = sorted(data)
    n = len(s)
    mid = n // 2
    if n % 2 == 0:
        return (s[mid - 1] + s[mid]) / 2
    return s[mid]

def mode(data):
    counts = {}
    for x in data:
        counts[x] = counts.get(x, 0) + 1
    max_count = max(counts.values())
    modes = [k for k, v in counts.items() if v == max_count]
    modes.sort()
    return modes[0]

def variance(data, sample=True):
    n = len(data)
    m = mean(data)
    total = sum((x - m) ** 2 for x in data)
    if sample and n > 1:
        return total / (n - 1)
    return total / n

def std_dev(data, sample=True):
    return math.sqrt(variance(data, sample))

def percentile(data, p):
    s = sorted(data)
    n = len(s)
    k = (p / 100) * (n - 1)
    f = math.floor(k)
    c = math.ceil(k)
    if f == c:
        return s[int(k)]
    return s[f] * (c - k) + s[c] * (k - f)

def iqr(data):
    return percentile(data, 75) - percentile(data, 25)

def covariance(x, y, sample=True):
    n = len(x)
    mx = mean(x)
    my = mean(y)
    total = sum((xi - mx) * (yi - my) for xi, yi in zip(x, y))

```

```

    if sample and n > 1:
        return total / (n - 1)
    return total / n

def pearson_correlation(x, y):
    n = len(x)
    mx = mean(x)
    my = mean(y)
    sx = std_dev(x, sample=False)
    sy = std_dev(y, sample=False)
    if sx == 0 or sy == 0:
        return 0.0
    cov = sum((xi - mx) * (yi - my) for xi, yi in zip(x, y)) / n
    return cov / (sx * sy)

def rank_data(data):
    indexed = sorted(enumerate(data), key=lambda pair: pair[1])
    ranks = [0.0] * len(data)
    i = 0
    while i < len(indexed):
        j = i
        while j < len(indexed) - 1 and indexed[j + 1][1] == indexed[i][1]:
            j += 1
        avg_rank = (i + j) / 2.0 + 1.0
        for k in range(i, j + 1):
            ranks[indexed[k][0]] = avg_rank
        i = j + 1
    return ranks

def spearman_correlation(x, y):
    rx = rank_data(x)
    ry = rank_data(y)
    return pearson_correlation(rx, ry)

def covariance_matrix(data):
    d = len(data)
    n = len(data[0])
    means = [mean(data[i]) for i in range(d)]
    matrix = [[0.0] * d for _ in range(d)]
    for i in range(d):
        for j in range(i, d):
            cov = sum(
                (data[i][k] - means[i]) * (data[j][k] - means[j])
                for k in range(n)
            ) / (n - 1)
            matrix[i][j] = cov
            matrix[j][i] = cov
    return matrix

```

```

def t_statistic_one_sample(data, mu_0):
    n = len(data)
    m = mean(data)
    s = std_dev(data, sample=True)
    return (m - mu_0) / (s / math.sqrt(n))

def t_statistic_two_sample(data1, data2):
    n1 = len(data1)
    n2 = len(data2)
    m1 = mean(data1)
    m2 = mean(data2)
    v1 = variance(data1, sample=True)
    v2 = variance(data2, sample=True)
    se = math.sqrt(v1 / n1 + v2 / n2)
    if se == 0:
        return 0.0
    return (m1 - m2) / se

def welch_df(data1, data2):
    n1 = len(data1)
    n2 = len(data2)
    v1 = variance(data1, sample=True)
    v2 = variance(data2, sample=True)
    num = (v1 / n1 + v2 / n2) ** 2
    denom = (v1 / n1) ** 2 / (n1 - 1) + (v2 / n2) ** 2 / (n2 - 1)
    if denom == 0:
        return n1 + n2 - 2
    return num / denom

def t_cdf_approx(t_val, df):
    x = df / (df + t_val * t_val)
    if t_val < 0:
        return 0.5 * _regularized_beta(x, df / 2, 0.5)
    return 1.0 - 0.5 * _regularized_beta(x, df / 2, 0.5)

def _regularized_beta(x, a, b):
    if x <= 0:
        return 0.0
    if x >= 1:
        return 1.0
    n_steps = 200
    total = 0.0
    dt = x / n_steps
    for i in range(n_steps):
        t = (i + 0.5) * dt
        total += t ** (a - 1) * (1 - t) ** (b - 1) * dt
    beta_val = _beta_function(a, b)
    if beta_val == 0:

```

```

        return 0.0
    return total / beta_val

def _beta_function(a, b):
    return math.exp(math.lgamma(a) + math.lgamma(b) - math.lgamma(a + b))

def p_value_two_sided(t_val, df):
    p_left = t_cdf_approx(abs(t_val), df)
    return 2.0 * (1.0 - p_left)

def one_sample_ttest(data, mu_0=0):
    n = len(data)
    t = t_statistic_one_sample(data, mu_0)
    df = n - 1
    p = p_value_two_sided(t, df)
    return {"t_statistic": t, "df": df, "p_value": p}

def two_sample_ttest(data1, data2):
    t = t_statistic_two_sample(data1, data2)
    df = welch_df(data1, data2)
    p = p_value_two_sided(t, df)
    return {"t_statistic": t, "df": df, "p_value": p}

def paired_ttest(data1, data2):
    diffs = [a - b for a, b in zip(data1, data2)]
    return one_sample_ttest(diffs, mu_0=0)

def chi_squared_test(observed, expected):
    chi2 = sum(
        (o - e) ** 2 / e for o, e in zip(observed, expected) if e > 0
    )
    df = len(observed) - 1
    p = chi_squared_p_value(chi2, df)
    return {"chi2": chi2, "df": df, "p_value": p}

def chi_squared_p_value(chi2, df):
    if chi2 <= 0:
        return 1.0
    return 1.0 - _lower_incomplete_gamma_ratio(df / 2.0, chi2 / 2.0)

def _lower_incomplete_gamma_ratio(a, x):
    if x <= 0:
        return 0.0
    n_steps = 500
    dt = x / n_steps

```

```

total = 0.0
for i in range(n_steps):
    t = (i + 0.5) * dt
    if t > 0:
        total += math.exp((a - 1) * math.log(t) - t) * dt
gamma_a = math.exp(math.lgamma(a))
if gamma_a == 0:
    return 0.0
return total / gamma_a

def bootstrap_statistic(data, stat_func, n_bootstrap=5000, ci=95):
    n = len(data)
    bootstrap_stats = []
    for _ in range(n_bootstrap):
        sample = [data[random.randint(0, n - 1)] for _ in range(n)]
        bootstrap_stats.append(stat_func(sample))
    bootstrap_stats.sort()
    lower_pct = (100 - ci) / 2
    upper_pct = 100 - lower_pct
    ci_lower = percentile(bootstrap_stats, lower_pct)
    ci_upper = percentile(bootstrap_stats, upper_pct)
    return {
        "estimate": stat_func(data),
        "ci_lower": ci_lower,
        "ci_upper": ci_upper,
        "ci_level": ci,
        "n_bootstrap": n_bootstrap,
        "std_error": std_dev(bootstrap_stats, sample=True),
    }

def bootstrap_compare(data1, data2, stat_func, n_bootstrap=5000, ci=95):
    n1 = len(data1)
    n2 = len(data2)
    diffs = []
    for _ in range(n_bootstrap):
        s1 = [data1[random.randint(0, n1 - 1)] for _ in range(n1)]
        s2 = [data2[random.randint(0, n2 - 1)] for _ in range(n2)]
        diffs.append(stat_func(s2) - stat_func(s1))
    diffs.sort()
    lower_pct = (100 - ci) / 2
    upper_pct = 100 - lower_pct
    ci_lower = percentile(diffs, lower_pct)
    ci_upper = percentile(diffs, upper_pct)
    observed_diff = stat_func(data2) - stat_func(data1)
    significant = ci_lower > 0 or ci_upper < 0
    return {
        "observed_diff": observed_diff,
        "ci_lower": ci_lower,
        "ci_upper": ci_upper,
        "significant": significant,
        "ci_level": ci,
    }

```

```

    }

def cohens_d(data1, data2):
    m1 = mean(data1)
    m2 = mean(data2)
    n1 = len(data1)
    n2 = len(data2)
    v1 = variance(data1, sample=True)
    v2 = variance(data2, sample=True)
    pooled = math.sqrt(((n1 - 1) * v1 + (n2 - 1) * v2) / (n1 + n2 - 2))
    if pooled == 0:
        return 0.0
    return (m2 - m1) / pooled

def interpret_cohens_d(d):
    d = abs(d)
    if d < 0.2:
        return "negligible"
    if d < 0.5:
        return "small"
    if d < 0.8:
        return "medium"
    return "large"

def bonferroni_correction(p_values, alpha=0.05):
    m = len(p_values)
    adjusted_alpha = alpha / m
    results = []
    for p in p_values:
        results.append({
            "original_p": p,
            "adjusted_alpha": adjusted_alpha,
            "significant": p < adjusted_alpha,
        })
    return results

def generate_normal(n, mu=0, sigma=1):
    samples = []
    for _ in range(n // 2 + 1):
        u1 = random.random()
        u2 = random.random()
        while u1 == 0:
            u1 = random.random()
        z0 = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
        z1 = math.sqrt(-2 * math.log(u1)) * math.sin(2 * math.pi * u2)
        samples.append(mu + sigma * z0)
        samples.append(mu + sigma * z1)
    return samples[:n]

```

```

def ab_test_simulator(
    n_per_group=100,
    true_effect=0.0,
    base_mean=50,
    base_std=10,
    alpha=0.05,
):
    group_a = generate_normal(n_per_group, base_mean, base_std)
    group_b = generate_normal(n_per_group, base_mean + true_effect, base_std)

    result = two_sample_ttest(group_a, group_b)
    d = cohens_d(group_a, group_b)
    boot = bootstrap_compare(group_a, group_b, mean, n_bootstrap=2000)

    return {
        "group_a_mean": mean(group_a),
        "group_b_mean": mean(group_b),
        "observed_diff": mean(group_b) - mean(group_a),
        "true_effect": true_effect,
        "t_test": result,
        "cohens_d": d,
        "effect_interpretation": interpret_cohens_d(d),
        "bootstrap": boot,
        "significant_ttest": result["p_value"] < alpha,
        "significant_bootstrap": boot["significant"],
    }

def run_multiple_ab_tests(
    n_tests=20,
    n_per_group=100,
    true_effect=0.0,
    alpha=0.05,
):
    p_values = []
    significant_count = 0
    for _ in range(n_tests):
        group_a = generate_normal(n_per_group, 50, 10)
        group_b = generate_normal(n_per_group, 50 + true_effect, 10)
        result = two_sample_ttest(group_a, group_b)
        p_values.append(result["p_value"])
        if result["p_value"] < alpha:
            significant_count += 1

    corrected = bonferroni_correction(p_values, alpha)
    corrected_significant = sum(1 for r in corrected if r["significant"])

    return {
        "n_tests": n_tests,
        "true_effect": true_effect,
        "false_positive_rate": significant_count / n_tests if true_effect == 0 else None,
    }

```

```

        "uncorrected_significant": significant_count,
        "corrected_significant": corrected_significant,
        "p_values": p_values,
    }

def statistical_vs_practical_significance(small_n=30, large_n=10000, effect=0.1):
    small_a = generate_normal(small_n, 50, 10)
    small_b = generate_normal(small_n, 50 + effect, 10)
    small_result = two_sample_ttest(small_a, small_b)
    small_d = cohens_d(small_a, small_b)

    large_a = generate_normal(large_n, 50, 10)
    large_b = generate_normal(large_n, 50 + effect, 10)
    large_result = two_sample_ttest(large_a, large_b)
    large_d = cohens_d(large_a, large_b)

    return {
        "small_sample": {
            "n": small_n,
            "p_value": small_result["p_value"],
            "cohens_d": small_d,
            "significant": small_result["p_value"] < 0.05,
            "interpretation": interpret_cohens_d(small_d),
        },
        "large_sample": {
            "n": large_n,
            "p_value": large_result["p_value"],
            "cohens_d": large_d,
            "significant": large_result["p_value"] < 0.05,
            "interpretation": interpret_cohens_d(large_d),
        },
        "true_effect": effect,
    }

if __name__ == "__main__":
    print("=" * 60)
    print("DESCRIPTIVE STATISTICS")
    print("=" * 60)
    data = [23, 45, 12, 67, 34, 89, 21, 56, 43, 78, 31, 64, 19, 52, 41]
    print(f>Data: {data}")
    print(f"Mean:      {mean(data):.2f}")
    print(f"Median:    {median(data):.2f}")
    print(f"Mode:      {mode(data)}")
    print(f"Std Dev:   {std_dev(data):.2f}")
    print(f"Variance: {variance(data):.2f}")
    print(f"P25:      {percentile(data, 25):.2f}")
    print(f"P50:      {percentile(data, 50):.2f}")
    print(f"P75:      {percentile(data, 75):.2f}")
    print(f"IQR:      {iqr(data):.2f}")

```



```

skewed = [1, 2, 3, 4, 5, 6, 7, 8, 9, 1000]
print(f"\nSkewed data: {skewed}")
print(f"Mean: {mean(skewed):.2f} (pulled by outlier)")
print(f"Median: {median(skewed):.2f} (robust to outlier)")

print("\n" + "=" * 60)
print("CORRELATION")
print("=" * 60)
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y_linear = [2.1, 3.9, 6.2, 7.8, 10.1, 12.3, 13.8, 16.1, 18.0, 20.2]
print(f"Linear relationship:")
print(f" Pearson: {pearson_correlation(x, y_linear):.4f}")
print(f" Spearman: {spearman_correlation(x, y_linear):.4f}")

y_quadratic = [xi ** 2 for xi in x]
print(f"Quadratic relationship (y = x^2):")
print(f" Pearson: {pearson_correlation(x, y_quadratic):.4f} (not perfect, relationship is nonlinear)")
print(f" Spearman: {spearman_correlation(x, y_quadratic):.4f} (perfect, relationship is monotonic)")

y_none = [random.gauss(0, 1) for _ in x]
print(f"No relationship (random):")
print(f" Pearson: {pearson_correlation(x, y_none):.4f}")
print(f" Spearman: {spearman_correlation(x, y_none):.4f}")

print("\n" + "=" * 60)
print("COVARIANCE MATRIX")
print("=" * 60)
feature1 = [random.gauss(0, 1) for _ in range(100)]
feature2 = [f + random.gauss(0, 0.5) for f in feature1]
feature3 = [random.gauss(0, 1) for _ in range(100)]
cov_mat = covariance_matrix([feature1, feature2, feature3])
print("3-feature covariance matrix:")
for row in cov_mat:
    print(f" [{row[0]:7.3f} {row[1]:7.3f} {row[2]:7.3f}]")
print("Feature 1 and 2 are correlated (constructed that way).")
print("Feature 3 is independent.")

print("\n" + "=" * 60)
print("HYPOTHESIS TESTING: ONE-SAMPLE T-TEST")
print("=" * 60)
sample = generate_normal(50, mu=52, sigma=10)
result = one_sample_ttest(sample, mu_0=50)
print(f"Testing if population mean = 50 (true mean = 52)")
print(f" Sample mean: {mean(sample):.2f}")
print(f" t-statistic: {result['t_statistic']:.4f}")
print(f" df: {result['df']}")

```

```

print(f" p-value:      {result['p_value']:.4f}")
print(f" Significant at alpha=0.05: {result['p_value'] < 0.05}")

print("\n" + "=" * 60)
print("HYPOTHESIS TESTING: TWO-SAMPLE T-TEST")
print("=" * 60)
model_a_scores = generate_normal(30, mu=0.85, sigma=0.05)
model_b_scores = generate_normal(30, mu=0.88, sigma=0.05)
result = two_sample_ttest(model_a_scores, model_b_scores)
d = cohens_d(model_a_scores, model_b_scores)
print(f"Model A mean: {mean(model_a_scores):.4f}")
print(f"Model B mean: {mean(model_b_scores):.4f}")
print(f" t-statistic: {result['t_statistic']:.4f}")
print(f" p-value:      {result['p_value']:.4f}")
print(f" Cohen's d:    {d:.4f} ({interpret_cohens_d(d)})")
print(f" Significant: {result['p_value'] < 0.05}")

print("\n" + "=" * 60)
print("PAIRED T-TEST (CROSS-VALIDATION)")
print("=" * 60)
cv_a = [0.82, 0.85, 0.81, 0.84, 0.83, 0.86, 0.80, 0.84, 0.82, 0.85]
cv_b = [0.84, 0.87, 0.83, 0.86, 0.85, 0.88, 0.83, 0.86, 0.85, 0.87]
result = paired_ttest(cv_a, cv_b)
print(f"Model A folds: {cv_a}")
print(f"Model B folds: {cv_b}")
print(f" Mean diff:    {mean([b - a for a, b in zip(cv_a, cv_b)]):.4f}")
print(f" t-statistic: {result['t_statistic']:.4f}")
print(f" p-value:      {result['p_value']:.4f}")
print(f" Significant: {result['p_value'] < 0.05}")

print("\n" + "=" * 60)
print("CHI-SQUARED TEST")
print("=" * 60)
observed = [120, 80, 95, 105]
expected = [100, 100, 100, 100]
result = chi_squared_test(observed, expected)
print(f"Observed: {observed}")
print(f"Expected: {expected}")
print(f" chi-squared: {result['chi2']:.4f}")
print(f" df:          {result['df']}")
print(f" p-value:      {result['p_value']:.4f}")
print(f" Significant: {result['p_value'] < 0.05}")

print("\n" + "=" * 60)
print("BOOTSTRAP CONFIDENCE INTERVALS")
print("=" * 60)
data = generate_normal(50, mu=100, sigma=15)
boot_mean = bootstrap_statistic(data, mean, n_bootstrap=5000)

```

```

boot_median = bootstrap_statistic(data, median, n_bootstrap=5000)
print(f"Sample size: 50, true mean: 100")
print(f"Bootstrap mean:    {boot_mean['estimate']:.2f}  "
      f"95% CI: [{boot_mean['ci_lower']:.2f}, {boot_mean['ci_upper']:.2f}]  "
      f"SE: {boot_mean['std_error']:.2f}")
print(f"Bootstrap median: {boot_median['estimate']:.2f}  "
      f"95% CI: [{boot_median['ci_lower']:.2f}, {boot_median['ci_upper']:.2f}]  "
      f"SE: {boot_median['std_error']:.2f}")

print("\nBootstrap model comparison:")
scores_a = generate_normal(40, mu=0.85, sigma=0.04)
scores_b = generate_normal(40, mu=0.88, sigma=0.04)
comp = bootstrap_compare(scores_a, scores_b, mean, n_bootstrap=5000)
print(f"  Model A mean: {mean(scores_a):.4f}")
print(f"  Model B mean: {mean(scores_b):.4f}")
print(f"  Diff:          {comp['observed_diff']:.4f}")
print(f"  95% CI:         [{comp['ci_lower']:.4f}, {comp['ci_upper']:.4f}])")
print(f"  Significant:    {comp['significant']} (CI excludes 0)")

print("\n" + "=" * 60)
print("A/B TEST SIMULATOR")
print("=" * 60)
print("\nTest 1: No real effect (true_effect = 0)")
ab1 = ab_test_simulator(n_per_group=200, true_effect=0.0)
print(f"  Group A mean: {ab1['group_a_mean']:.2f}")
print(f"  Group B mean: {ab1['group_b_mean']:.2f}")
print(f"  Observed diff: {ab1['observed_diff']:.2f}")
print(f"  p-value: {ab1['t_test']['p_value']:.4f}")
print(f"  Significant (t-test): {ab1['significant_ttest']}")
print(f"  Cohen's d: {ab1['cohens_d']:.4f} ({ab1['effect_interpretation']})")

print("\nTest 2: Real effect (true_effect = 5)")
ab2 = ab_test_simulator(n_per_group=200, true_effect=5.0)
print(f"  Group A mean: {ab2['group_a_mean']:.2f}")
print(f"  Group B mean: {ab2['group_b_mean']:.2f}")
print(f"  Observed diff: {ab2['observed_diff']:.2f}")
print(f"  p-value: {ab2['t_test']['p_value']:.4f}")
print(f"  Significant (t-test): {ab2['significant_ttest']}")
print(f"  Cohen's d: {ab2['cohens_d']:.4f} ({ab2['effect_interpretation']})")

print("\n" + "=" * 60)
print("MULTIPLE COMPARISON PROBLEM")
print("=" * 60)
print("\n20 tests with NO real effect (all null hypotheses true):")
multi = run_multiple_ab_tests(n_tests=20, true_effect=0.0)

```

```

print(f" Tests significant (uncorrected): {multi['uncorrected_significant']/20}")
print(f" Tests significant (Bonferroni): {multi['corrected_significant']/20}")
print(f" Expected false positives at alpha=0.05: ~1")
print(f" Bonferroni adjusted alpha: {0.05/20:.4f}")

print("\n" + "=" * 60)
print("STATISTICAL VS PRACTICAL SIGNIFICANCE")
print("=" * 60)
result = statistical_vs_practical_significance(
    small_n=30, large_n=10000, effect=0.1
)
print(f"\nTrue effect: {result['true_effect']} (tiny)")
print(f"\nSmall sample (n={result['small_sample']['n']}):")
print(f" p-value: {result['small_sample']['p_value']:.4f}")
print(f" Cohen's d: {result['small_sample']['cohens_d']:.4f} ({result['small_sample']['interpretation']})")
print(f" Significant: {result['small_sample']['significant']}")
print(f"\nLarge sample (n={result['large_sample']['n']}):")
print(f" p-value: {result['large_sample']['p_value']:.4f}")
print(f" Cohen's d: {result['large_sample']['cohens_d']:.4f} ({result['large_sample']['interpretation']})")
print(f" Significant: {result['large_sample']['significant']}")
print(f"\nLesson: large n can make a negligible effect 'significant'.")
print("Always check effect size, not just p-values.")

print("\n" + "=" * 60)
print("POWER ANALYSIS SIMULATION")
print("=" * 60)
print("\nHow often do we detect a real effect (true_effect=3)?")
n_sims = 200
detected = 0
for _ in range(n_sims):
    a = generate_normal(50, 50, 10)
    b = generate_normal(50, 53, 10)
    res = two_sample_ttest(a, b)
    if res["p_value"] < 0.05:
        detected += 1
print(f" Power (n=50, effect=3, std=10): {detected/n_sims:.2f}")
print(f" ({detected}/{n_sims} simulations detected the effect)")

detected_large = 0
for _ in range(n_sims):
    a = generate_normal(200, 50, 10)
    b = generate_normal(200, 53, 10)
    res = two_sample_ttest(a, b)
    if res["p_value"] < 0.05:
        detected_large += 1
print(f" Power (n=200, effect=3, std=10): {detected_large/n_sims:.2f}")

```

```
print(f" ({detected_large}/{n_sims} simulations detected the effect)")
print(" Larger samples give more power to detect real effects.")
```

Statistical Testing for ML

How to pick the right test when comparing models, running A/B experiments, or validating results.

Decision Checklist

1. What are you comparing? Means, proportions, distributions, or correlations?
2. How many groups? One sample vs reference, two groups, or multiple groups?
3. Are observations paired (same test set, same folds) or independent?
4. Is the data normally distributed? If $n < 30$ and not clearly normal, use non-parametric.
5. Is the data continuous, ordinal, or categorical?
6. How many tests are you running? Apply correction if more than one.

Decision tree

```
Comparing means?
  Two groups?
    Paired (same data splits)? --> Paired t-test (or Wilcoxon signed-rank if non-normal)
    Independent? --> Welch's t-test (or Mann-Whitney U if non-normal)
  Multiple groups?
    Paired? --> Repeated measures ANOVA (or Friedman test)
    Independent? --> One-way ANOVA (or Kruskal-Wallis)

Comparing proportions?
  Two groups? --> Chi-squared test or Fisher's exact test (small n)
  Multiple groups? --> Chi-squared test

Comparing distributions?
  Is one distribution a reference? --> Kolmogorov-Smirnov test
  Are both empirical? --> Two-sample KS test

Measuring association?
  Both continuous, roughly normal? --> Pearson correlation
  Ordinal or non-normal? --> Spearman rank correlation
  Categorical x Categorical? --> Chi-squared test of independence

Running many tests?
  Apply Bonferroni correction: alpha_adjusted = alpha / number_of_tests
```

Or use Holm-Bonferroni (less conservative, still controls family-wise error)

When to use each test

Test	Data type	Assumptions	ML use case
Paired t-test	Continuous, paired	Normal differences	Compare 2 models on same k-fold splits
Wilcoxon signed-rank	Continuous/ordinal, paired	None (non-parametric)	Compare 2 models, small k (5-10 folds)
Welch's t-test	Continuous, independent	Roughly normal	Compare model on two separate datasets
Mann-Whitney U	Continuous/ordinal, independent	None	Compare latency distributions
ANOVA	Continuous, 3+ groups	Normal, equal variance	Compare multiple model architectures
Kruskal-Wallis	Continuous/ordinal, 3+ groups	None	Compare multiple models, non-normal metrics
Chi-squared	Categorical counts	Expected count ≥ 5	Compare class distributions, confusion matrices
Fisher's exact	Categorical counts	Small samples	Rare event comparison
KS test	Continuous	None	Check if predictions follow expected distribution
Bootstrap CI	Any statistic	None	Confidence interval for AUC, F1, any metric
McNemar's test	Paired binary	None	Compare two classifiers on same test set

Model comparison recipe

1. Define metric and significance level ($\alpha = 0.05$) before running experiments.
2. Run both models on the same k-fold cross-validation splits ($k = 5$ or 10).
3. Collect paired scores: $(a_1, b_1), (a_2, b_2), \dots, (a_k, b_k)$.
4. Compute differences: $d_i = b_i - a_i$.

5. Run paired test (Wilcoxon for $k \leq 10$, paired t-test for $k > 10$ or normal diffs).
6. Report: p-value, mean difference, 95% confidence interval, effect size (Cohen's d).
7. If $p < \alpha$ AND effect size is meaningful, the difference is real and worth acting on.

Common mistakes

- Using an independent test when data is paired. If both models were evaluated on the same test folds, you must use a paired test. Independent tests throw away the pairing and lose statistical power.
- Reporting $p < 0.05$ without effect size. A statistically significant 0.1% accuracy improvement is not worth deploying. Always compute Cohen's d or the raw mean difference.
- Comparing models across different test sets. The test set MUST be identical for both models. Different test sets make comparison meaningless.
- Running 20 comparisons and reporting the best one without Bonferroni correction. With 20 tests at $\alpha = 0.05$, you expect 1 false positive by chance.
- Using accuracy on imbalanced data. On a 99% majority class, a trivial classifier achieves 99%. Use F1, precision-recall AUC, or Matthews correlation coefficient.
- Treating cross-validation folds as independent samples. They share training data, which violates the independence assumption. The corrected resampled t-test accounts for this.

Quick reference: effect size interpretation

Cohen's d	Interpretation
0.2	Small effect
0.5	Medium effect
0.8	Large effect
> 1.0	Very large effect

What to report	Why
p-value	Is the difference real?
Confidence interval	How big could the difference be?
Effect size (Cohen's d)	Is the difference meaningful?
Sample size (n or k folds)	Can we trust the result?

Quiz

Pre-lesson check

1. What does a p-value of 0.03 mean in a hypothesis test?
 - a) There is a 3% probability the null hypothesis is true
 - b) There is a 3% probability of seeing data this extreme if the null hypothesis were true **(correct)**
 - c) The model improved by 3%
 - d) 97% of the data supports the alternative hypothesis
Explanation: The p-value is the probability of observing data as extreme as what you got, assuming the null hypothesis is true. It is NOT the probability that H_0 is true — this is the single most common misunderstanding in statistics.
2. Why do you divide by (n-1) instead of n when computing sample variance?
 - a) It makes the computation faster
 - b) It accounts for the fact that the sample mean is not the true population mean (Bessel's correction) **(correct)**
 - c) It converts the variance to standard deviation
 - d) It only applies when the sample size is odd
Explanation: Bessel's correction (dividing by n-1) compensates for the bias introduced by using the sample mean instead of the true population mean. Without it, sample variance systematically underestimates the true population variance.

Post-lesson check

3. You test 20 different model configurations at $\alpha = 0.05$. What is the approximate probability of at least one false positive?
 - a) 5%

b) 25%

c) 64% **(correct)**

d) 95%

Explanation: $P(\text{at least one false positive}) = 1 - (1 - 0.05)^{20} = 1 - 0.95^{20} \approx 0.64$ (64%). This is the multiple comparison problem. Bonferroni correction addresses it by testing each at $\alpha/20 = 0.0025$.

4. Model A scores 0.9234 and Model B scores 0.9237 on 1 million test samples with p-value = 0.001. What should you conclude?

a) Model B is significantly better and should be deployed immediately

b) The difference is statistically significant but a 0.03% improvement may not be practically significant **(correct)**

c) The test is invalid because the sample size is too large

d) Model A is better because it was tested first

Explanation: With 1 million samples, even trivially small differences become statistically significant. The p-value confirms the difference is real, but the effect size (0.03% accuracy gain) may not justify the engineering cost of deployment. Always report both p-value and effect size.

5. What advantage does bootstrap have over the paired t-test for comparing two ML models?

a) Bootstrap always produces smaller p-values

b) Bootstrap requires no distributional assumptions and works for any metric (AUC, F1, median) **(correct)**

c) Bootstrap needs fewer samples to reach significance

d) Bootstrap can only be used with neural networks

Explanation: Bootstrap resampling estimates the sampling distribution of any statistic by resampling with replacement. Unlike the t-test, it does not assume normality. It works for any metric — AUC, F1, precision@k, median — without needing a closed-form formula.

Sampling Methods

Sampling is how AI explores the space of possibilities.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 06-07 (Probability, Bayes' Theorem) **Time:** ~120 minutes

Learning Objectives

- Implement inverse CDF, rejection, and importance sampling from scratch using only uniform random numbers
- Build temperature, top-k, and top-p (nucleus) sampling for language model token generation
- Explain the reparameterization trick and why it enables backpropagation through sampling in VAEs
- Run Metropolis-Hastings MCMC to sample from an unnormalized target distribution

The Problem

A language model finishes processing your prompt and produces a vector of 50,000 logits. One for every token in its vocabulary. Now it has to pick one. How?

If it always picks the highest-probability token, every response is identical. Deterministic. Boring. If it picks uniformly at random, the output is gibberish. The answer lives somewhere between these extremes, and that somewhere is controlled by sampling.

Sampling is not limited to text generation. Reinforcement learning estimates policy gradients by sampling trajectories. VAEs learn latent representations by sampling from learned distributions and backpropagating through the randomness. Diffusion models generate images by sampling noise and iteratively denoising. Monte Carlo methods estimate integrals that have no closed-form solution. MCMC algorithms explore high-dimensional posterior distributions that are impossible to enumerate.

Every generative AI system is a sampling system. The sampling strategy determines the quality, diversity, and controllability of the output. This lesson builds every major sampling method from scratch, starting from uniform random numbers and ending with the techniques that power modern LLMs and generative models.

The Concept

Why Sampling Matters

Sampling appears in four fundamental roles across AI and machine learning:

Generation. Language models, diffusion models, and GANs all produce output by sampling. The sampling algorithm directly controls creativity, coherence, and diversity. Temperature, top-k, and nucleus sampling are the knobs that engineers turn daily.

Training. Stochastic gradient descent samples mini-batches. Dropout samples neurons to deactivate. Data augmentation samples random transformations. Importance sampling reweights samples to reduce gradient variance in reinforcement learning (PPO, TRPO).

Estimation. Many quantities in ML have no closed-form solution. The expected loss over a data distribution, the partition function of an energy-based model, the evidence in Bayesian inference. Monte Carlo estimation approximates all of these by averaging over samples.

Exploration. MCMC algorithms explore posterior distributions in Bayesian inference. Evolutionary strategies sample parameter perturbations. Thompson sampling balances exploration and exploitation in bandits.

The core challenge: you can only sample directly from simple distributions (uniform, normal). For everything else, you need a method to convert simple samples into samples from your target distribution.

Uniform Random Sampling

Every sampling method starts here. A uniform random number generator produces values in $[0, 1]$ where every sub-interval of equal length has equal probability.

$$U \sim \text{Uniform}(0, 1)$$

$$P(a \leq U \leq b) = b - a \quad \text{for } 0 \leq a \leq b \leq 1$$

Properties:

$$E[U] = 0.5$$

$$\text{Var}(U) = 1/12$$

To sample uniformly from a discrete set of n items, generate U and return $\text{floor}(n * U)$. To sample from a continuous range $[a, b]$, compute $a + (b - a) * U$.

The key insight: a single uniform random number contains exactly the right amount of randomness to produce one sample from any distribution. The trick is finding the right transformation.

Inverse CDF Method (Inverse Transform Sampling)

The cumulative distribution function (CDF) maps values to probabilities:

$$F(x) = P(X \leq x)$$

Properties:

F is non-decreasing

$$F(-\infty) = 0$$

$$F(+\infty) = 1$$

F maps the real line to $[0, 1]$

The inverse CDF maps probabilities back to values. If $U \sim \text{Uniform}(0, 1)$, then $X = F_{\text{inverse}}(U)$ follows the target distribution.

Algorithm:

1. Generate $u \sim \text{Uniform}(0, 1)$
2. Return $F_{\text{inverse}}(u)$

Why it works:

$$P(X \leq x) = P(F_{\text{inverse}}(U) \leq x) = P(U \leq F(x)) = F(x)$$

Exponential distribution example:

PDF: $f(x) = \lambda \exp(-\lambda x)$, $x \geq 0$

CDF: $F(x) = 1 - \exp(-\lambda x)$

Solve $F(x) = u$ for x :

$$\begin{aligned} u &= 1 - \exp(-\lambda x) \\ \exp(-\lambda x) &= 1 - u \\ x &= -\ln(1 - u) / \lambda \end{aligned}$$

Since $(1 - U)$ and U have the same distribution:

$$x = -\ln(u) / \lambda$$

This works perfectly when you can write down F_{inverse} in closed form. For the normal distribution, there is no closed-form inverse CDF, so we use other methods (Box-Muller, or numerical approximation).

Discrete version: For discrete distributions, build the CDF as a cumulative sum, generate U , and find the first index where the cumulative sum exceeds U . This is how `sample_categorical` works in Lesson 06.

Rejection Sampling

When you cannot invert the CDF but can evaluate the target PDF up to a constant, rejection sampling works.

Target distribution: $p(x)$ (can evaluate, possibly unnormalized)

Proposal distribution: $q(x)$ (can sample from)

Bound: M such that $p(x) \leq M * q(x)$ for all x

Algorithm:

1. Sample $x \sim q(x)$
2. Sample $u \sim \text{Uniform}(0, 1)$
3. If $u < p(x) / (M * q(x))$, accept x
4. Otherwise, reject and go to step 1

Acceptance rate = $1/M$

The tighter the bound M , the higher the acceptance rate. In low dimensions (1-3), rejection sampling works well. In high dimensions, the acceptance rate drops exponentially because most of the proposal volume gets rejected. This is the curse of dimensionality for rejection sampling.

Example: sampling from a truncated normal. Use a uniform proposal over the truncated range. The envelope M is the maximum of the normal PDF in that range.

Example: sampling from a semicircle. Propose uniformly in the bounding rectangle. Accept if the point falls inside the semicircle. This is how Monte Carlo computes π : the acceptance rate equals the area ratio $\pi/4$.

Importance Sampling

Sometimes you do not need samples from the target distribution $p(x)$. You need to estimate an expectation under $p(x)$, and you have samples from a different distribution $q(x)$.

Goal: estimate $E_p[f(x)] = \text{integral of } f(x) * p(x) \, dx$

Rewrite:

$$\begin{aligned} E_p[f(x)] &= \text{integral of } f(x) * (p(x)/q(x)) * q(x) \, dx \\ &= E_q[f(x) * w(x)] \end{aligned}$$

where $w(x) = p(x) / q(x)$ are the importance weights.

Estimator:

$$E_p[f(x)] \sim (1/N) * \sum(f(x_i) * w(x_i)) \quad \text{where } x_i \sim q(x)$$

This is critical in reinforcement learning. In PPO (Proximal Policy Optimization), you collect trajectories under an old policy π_{old} but want to optimize a new policy π_{new} . The importance weight is $\pi_{\text{new}}(a|s) / \pi_{\text{old}}(a|s)$. PPO clips these weights to prevent the new policy from diverging too far from the old one.

The variance of the importance sampling estimator depends on how similar q is to p . If q is very different from p , a few samples get enormous weights and dominate the estimate. Self-normalized importance sampling divides by the sum of weights to reduce this problem:

$$E_p[f(x)] \sim \sum(w_i * f(x_i)) / \sum(w_i)$$

Monte Carlo Estimation

Monte Carlo estimation approximates integrals by averaging random samples. The law of large numbers guarantees convergence.

Goal: estimate $I = \text{integral of } g(x) \, dx \text{ over domain } D$

Method:

1. Sample $x_1, \dots, x_N$ uniformly from D
2. $I \sim (\text{Volume of } D / N) * \sum(g(x_i))$

Error: $O(1 / \sqrt{N})$ regardless of dimension

The error rate is dimension-independent. This is why Monte Carlo methods dominate in high dimensions where grid-based integration is impossible.

Estimating π :

Sample (x, y) uniformly from $[-1, 1] \times [-1, 1]$
Count how many fall inside the unit circle: $x^2 + y^2 \leq 1$
 $\pi \sim 4 * (\text{count inside}) / (\text{total count})$

Estimating expectations:

$E[f(X)] \sim (1/N) * \sum(f(x_i))$ where $x_i \sim p(x)$

The sample mean converges to the true expectation.
Variance of the estimator = $\text{Var}(f(X)) / N$

Markov Chain Monte Carlo (MCMC): Metropolis-Hastings

MCMC constructs a Markov chain whose stationary distribution is the target distribution $p(x)$. After enough steps, samples from the chain are (approximately) samples from $p(x)$.

Target: $p(x)$ (known up to a normalizing constant)
Proposal: $q(x'|x)$ (how to propose the next state given the current state)

Metropolis-Hastings algorithm:

1. Start at some x_0
2. For $t = 1, 2, \dots, T$:
 - a. Propose $x' \sim q(x'|x_t)$
 - b. Compute acceptance ratio:
 $\alpha = [p(x') * q(x_t|x')] / [p(x_t) * q(x'|x_t)]$
 - c. Accept with probability $\min(1, \alpha)$:
 - If $u < \alpha$ ($u \sim \text{Uniform}(0,1)$): $x_{t+1} = x'$
 - Otherwise: $x_{t+1} = x_t$
3. Discard first B samples (burn-in)
4. Return remaining samples

For symmetric proposals ($q(x'|x) = q(x|x')$), the ratio simplifies to $p(x')/p(x)$. This is the original Metropolis algorithm.

Why it works. The acceptance rule ensures detailed balance: the probability of being at x and moving to x' equals the probability of being at x' and moving to x . Detailed balance implies that $p(x)$ is the stationary distribution of the chain.

Practical considerations: - Burn-in: discard early samples before the chain reaches equilibrium - Thinning: keep every k -th sample to reduce autocorrelation - Proposal scale: too small and the chain moves slowly (high acceptance, slow exploration); too large and most proposals are rejected (low acceptance, stuck in place) - The optimal acceptance rate for a Gaussian proposal in high dimensions is approximately 0.234

Gibbs Sampling

Gibbs sampling is a special case of MCMC for multivariate distributions. Instead of proposing a move in all dimensions at once, it updates one variable at a time from its conditional distribution.

```
Target:  $p(x_1, x_2, \dots, x_d)$ 

Algorithm:
  For each iteration  $t$ :
    Sample  $x_1^{t+1} \sim p(x_1 \mid x_2^t, x_3^t, \dots, x_d^t)$ 
    Sample  $x_2^{t+1} \sim p(x_2 \mid x_1^{t+1}, x_3^t, \dots, x_d^t)$ 
    ...
    Sample  $x_d^{t+1} \sim p(x_d \mid x_1^{t+1}, x_2^{t+1}, \dots, x_{d-1}^{t+1})$ 
  )
```

Gibbs sampling requires that you can sample from each conditional distribution $p(x_i \mid x_{-i})$. This is straightforward for many models: - Bayesian networks: conditionals follow from the graph structure - Gaussian mixtures: conditionals are Gaussian - Ising models: each spin's conditional depends only on its neighbors

The acceptance rate is always 1 (every proposal is accepted) because sampling from the exact conditional automatically satisfies detailed balance.

Limitation. When variables are highly correlated, Gibbs sampling mixes slowly because updating one variable at a time cannot make large diagonal moves through the distribution.

Temperature Sampling (Used in LLMs)

Language models output logits $z_1, \dots, z_V$ for each token in the vocabulary. Softmax converts these to probabilities. Temperature rescales the logits before softmax:

```
 $p_i = \exp(z_i / T) / \sum(\exp(z_j / T))$ 

 $T = 1.0$ : standard softmax (original distribution)
 $T \rightarrow 0$ : argmax (deterministic, always picks highest logit)
 $T \rightarrow \infty$ : uniform (all tokens equally likely)
```

```
T < 1.0: sharpens the distribution (more confident, less diverse)
T > 1.0: flattens the distribution (less confident, more diverse)
```

Why it works. Dividing logits by $T < 1$ amplifies differences between logits. If $z_1 = 2$ and $z_2 = 1$, dividing by $T = 0.5$ gives $z_1/T = 4$ and $z_2/T = 2$, making the gap larger. After softmax, the highest-logit token gets a much larger share.

In practice: - $T = 0.0$: greedy decoding, best for factual Q&A - $T = 0.3-0.7$: slightly creative, good for code generation - $T = 0.7-1.0$: balanced, good for general conversation - $T = 1.0-1.5$: creative writing, brainstorming - $T > 1.5$: increasingly random, rarely useful

Temperature does not change which tokens are possible. It changes the probability mass allocated to each token.

Top-k Sampling

Top-k sampling restricts the candidate set to the k tokens with the highest probabilities, then renormalizes and samples from that restricted set.

Algorithm:

1. Compute softmax probabilities for all V tokens
2. Sort tokens by probability (descending)
3. Keep only the top k tokens
4. Renormalize: $p_i' = p_i / \sum(p_j \text{ for } j \text{ in top-}k)$
5. Sample from the renormalized distribution

$k = 1$: greedy decoding

$k = V$: no filtering (standard sampling)

$k = 40$: typical setting, removes long tail of unlikely tokens

Top-k prevents the model from selecting extremely unlikely tokens (typos, nonsense) that exist in the long tail of the vocabulary distribution. The problem: k is fixed regardless of context. When the model is confident (one token has 95% probability), $k = 40$ still allows 39 alternatives. When the model is uncertain (probability is spread across 1000 tokens), $k = 40$ cuts off plausible options.

Top-p (Nucleus) Sampling

Top-p sampling dynamically adjusts the candidate set size. Instead of keeping a fixed number of tokens, it keeps the smallest set of tokens whose cumulative probability exceeds p .

Algorithm:

1. Compute softmax probabilities for all V tokens
2. Sort tokens by probability (descending)
3. Find smallest k such that sum of top- k probabilities $\geq p$
4. Keep only those k tokens

5. Renormalize and sample

`p = 0.9`: keeps tokens covering 90% of probability mass
`p = 1.0`: no filtering
`p = 0.1`: very restrictive, nearly greedy

When the model is confident, nucleus sampling keeps few tokens (maybe 2-3). When the model is uncertain, it keeps many (maybe 200). This adaptive behavior is why nucleus sampling generally produces better text than top-k.

Common combinations: - Temperature 0.7 + top-p 0.9: good general-purpose setting - Temperature 0.0 (greedy): best for deterministic tasks - Temperature 1.0 + top-k 50: Fan et al. (2018) original paper setting

Top-k and top-p can be combined. Apply top-k first, then top-p on the remaining set.

Reparameterization Trick (Used in VAEs)

Variational autoencoders (VAEs) learn by encoding inputs into a distribution in latent space, sampling from that distribution, and decoding the sample back. The problem: you cannot backpropagate through a sampling operation.

Standard sampling (not differentiable):

$z \sim N(\mu, \sigma^2)$

The randomness blocks gradient flow.

$d/d_{\mu} [\text{sample from } N(\mu, \sigma^2)] = ???$

The reparameterization trick separates the randomness from the parameters:

Reparameterized sampling:

$\epsilon \sim N(0, 1)$ (fixed random noise, no parameters)

$z = \mu + \sigma * \epsilon$ (deterministic function of parameters)

Now z is a deterministic, differentiable function of μ and σ .

$d(z)/d(\mu) = 1$

$d(z)/d(\sigma) = \epsilon$

Gradients flow through μ and σ .

This works because $N(\mu, \sigma^2)$ has the same distribution as $\mu + \sigma * N(0, 1)$. The key insight: move the randomness to a parameter-free source (ϵ), then express the sample as a differentiable transformation of the parameters.

In the VAE training loop: 1. Encoder outputs μ and $\log(\sigma^2)$ for each input 2. Sample $\epsilon \sim N(0, 1)$ 3. Compute $z = \mu + \sigma * \epsilon$ 4. Decode z to reconstruct the input 5. Backpropagate through steps 4, 3, 2, 1 (possible because step 3 is differentiable)

Without the reparameterization trick, VAEs cannot be trained with standard backpropagation. This single insight made VAEs practical.

Gumbel-Softmax (Differentiable Categorical Sampling)

The reparameterization trick works for continuous distributions (Gaussian). For discrete categorical distributions, we need a different approach. Gumbel-Softmax provides a differentiable approximation to categorical sampling.

The Gumbel-Max trick (non-differentiable):

```
To sample from a categorical distribution with log-probabilities log(p_1), ..., log(p_k):
1. Sample g_i ~ Gumbel(0, 1) for each category
   (g = -log(-log(u)), where u ~ Uniform(0, 1))
2. Return argmax(log(p_i) + g_i)
```

This produces exact categorical samples.

Gumbel-Softmax (differentiable approximation):

```
Replace the hard argmax with a soft softmax:
y_i = exp((log(p_i) + g_i) / tau) / sum(exp((log(p_j) + g_j) / tau))

tau (temperature) controls the approximation:
tau -> 0: approaches a one-hot vector (hard categorical)
tau -> inf: approaches uniform (1/k, 1/k, ..., 1/k)
tau = 1.0: soft approximation
```

Gumbel-Softmax produces a continuous relaxation of a discrete sample. The output is a probability vector (soft one-hot) instead of a hard one-hot. Gradients flow through the softmax. During the forward pass in training, you can use the “straight-through” estimator: use the hard argmax for the forward pass but the soft Gumbel-Softmax gradients for the backward pass.

Applications: - Discrete latent variables in VAEs - Neural architecture search (choosing discrete operations) - Hard attention mechanisms - Reinforcement learning with discrete actions

Stratified Sampling

Standard Monte Carlo sampling can leave gaps in the sample space by chance. Stratified sampling forces even coverage by dividing the space into strata and sampling from each.

Standard Monte Carlo:

Sample N points uniformly from $[0, 1]$
Some regions may have clusters, others gaps

Stratified sampling:

Divide $[0, 1]$ into N equal strata: $[0, 1/N), [1/N, 2/N), \dots, [(N-1)/N, 1)$

Sample one point uniformly within each stratum

$x_i = (i + u_i) / N$ where $u_i \sim \text{Uniform}(0, 1)$, $i = 0, \dots, N-1$

Stratified sampling always has lower or equal variance compared to standard Monte Carlo:

$\text{Var}(\text{stratified}) \leq \text{Var}(\text{standard Monte Carlo})$

The improvement is largest when $f(x)$ varies smoothly.

For piecewise-constant functions, stratified sampling is exact.

Applications: - Numerical integration (quasi-Monte Carlo) - Training data splits (ensuring class balance in each fold) - Importance sampling with stratification (combining both techniques) - NeRF (Neural Radiance Fields) uses stratified sampling along camera rays

Connection to Diffusion Models

Diffusion models generate images through a sampling process. The forward process adds Gaussian noise to an image over T steps until it becomes pure noise. The reverse process learns to denoise, recovering the original image step by step.

Forward process (known):

$x_t = \sqrt{\alpha_t} * x_{t-1} + \sqrt{1 - \alpha_t} * \epsilon$
where $\epsilon \sim N(0, I)$

After T steps: $x_T \sim N(0, I)$ (pure noise)

Reverse process (learned):

$x_{t-1} = (1/\sqrt{\alpha_t}) * (x_t - (1 - \alpha_t)/\sqrt{1 - \alpha_{\bar{t}}}) * \epsilon_{\theta}(x_t, t) + \sigma_t * z$
where $z \sim N(0, I)$

Each denoising step is a sampling step.

The connection to the methods in this lesson: - Each denoising step uses the reparameterization trick (sample noise, apply deterministic transform) - The noise schedule $\{\alpha_t\}$ controls a form of temperature annealing - Training uses Monte Carlo estimation to approximate the ELBO (evidence lower

bound) - Ancestral sampling in diffusion models is a Markov chain (each step depends only on the current state)

The entire image generation process is iterative sampling: start from noise, and at each step, sample a slightly less noisy version conditioned on the learned denoising model.

Build It

Step 1: Uniform and inverse CDF sampling

```
import math
import random

def sample_uniform(a, b):
    return a + (b - a) * random.random()

def sample_exponential_inverse_cdf(lam):
    u = random.random()
    return -math.log(u) / lam
```

Generate 10,000 exponential samples and verify the mean is $1/\lambda$.

Step 2: Rejection sampling

```
def rejection_sample(target_pdf, proposal_sample, proposal_pdf, M):
    while True:
        x = proposal_sample()
        u = random.random()
        if u < target_pdf(x) / (M * proposal_pdf(x)):
            return x
```

Use rejection sampling to draw from a truncated normal distribution. Verify the shape by histogramming the samples.

Step 3: Importance sampling

```
def importance_sampling_estimate(f, target_pdf, proposal_pdf, proposal_sample, n):
    total = 0
    for _ in range(n):
        x = proposal_sample()
        w = target_pdf(x) / proposal_pdf(x)
        total += f(x) * w
    return total / n
```

Estimate $E[X^2]$ under a normal distribution using a uniform proposal. Compare to the known answer ($\mu^2 + \sigma^2$).

Step 4: Monte Carlo estimation of π

```
def monte_carlo_pi(n):
    inside = 0
    for _ in range(n):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)
        if x*x + y*y <= 1:
            inside += 1
    return 4 * inside / n
```

Step 5: Metropolis-Hastings MCMC

```
def metropolis_hastings(target_log_pdf, proposal_sample, proposal_log_pdf, x0, n_sample
s, burn_in):
    samples = []
    x = x0
    for i in range(n_samples + burn_in):
        x_new = proposal_sample(x)
        log_alpha = (target_log_pdf(x_new) + proposal_log_pdf(x, x_new)
                     - target_log_pdf(x) - proposal_log_pdf(x_new, x))
        if math.log(random.random()) < log_alpha:
            x = x_new
        if i >= burn_in:
            samples.append(x)
    return samples
```

Sample from a bimodal distribution (mixture of two Gaussians). Visualize the chain's trajectory.

Step 6: Gibbs sampling

```
def gibbs_sampling_2d(conditional_x_given_y, conditional_y_given_x, x0, y0, n_samples,
burn_in):
    x, y = x0, y0
    samples = []
    for i in range(n_samples + burn_in):
        x = conditional_x_given_y(y)
        y = conditional_y_given_x(x)
        if i >= burn_in:
            samples.append((x, y))
    return samples
```

Step 7: Temperature sampling

```
def softmax(logits):
    max_l = max(logits)
    exps = [math.exp(z - max_l) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]
```

```
def temperature_sample(logits, temperature):
    scaled = [z / temperature for z in logits]
    probs = softmax(scaled)
    return sample_from_probs(probs)
```

Show how temperature changes the output distribution for a set of token logits.

Step 8: Top-k and top-p sampling

```
def top_k_sample(logits, k):
    indexed = sorted(enumerate(logits), key=lambda x: -x[1])
    top = indexed[:k]
    top_logits = [l for _, l in top]
    probs = softmax(top_logits)
    idx = sample_from_probs(probs)
    return top[idx][0]

def top_p_sample(logits, p):
    probs = softmax(logits)
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    cumsum = 0
    selected = []
    for token_idx, prob in indexed:
        cumsum += prob
        selected.append((token_idx, prob))
        if cumsum >= p:
            break
    sel_probs = [pr for _, pr in selected]
    total = sum(sel_probs)
    sel_probs = [pr / total for pr in sel_probs]
    idx = sample_from_probs(sel_probs)
    return selected[idx][0]
```

Step 9: Reparameterization trick

```
def reparam_sample(mu, sigma):
    epsilon = random.gauss(0, 1)
    return mu + sigma * epsilon

def reparam_gradient(mu, sigma, epsilon):
    dz_dmu = 1.0
    dz_dsigma = epsilon
    return dz_dmu, dz_dsigma
```

Demonstrate that gradients flow through the reparameterized sample but not through direct sampling.

Step 10: Gumbel-Softmax

```
def gumbel_sample():
    u = random.random()
    return -math.log(-math.log(u))
```

```
def gumbel_softmax(logits, temperature):
    gumbels = [math.log(p) + gumbel_sample() for p in logits]
    return softmax([g / temperature for g in gumbels])
```

Show how decreasing temperature makes the output approach a one-hot vector.

Full implementations with all visualizations are in `code/sampling.py`.

Use It

With NumPy and SciPy, the production versions:

```
import numpy as np

rng = np.random.default_rng(42)

exponential_samples = rng.exponential(scale=2.0, size=10000)
print(f"Exponential mean: {exponential_samples.mean():.4f} (expected 2.0)")

from scipy import stats
normal = stats.norm(loc=0, scale=1)
print(f"CDF at 1.96: {normal.cdf(1.96):.4f}")
print(f"Inverse CDF at 0.975: {normal.ppf(0.975):.4f}")

logits = np.array([2.0, 1.0, 0.5, 0.1, -1.0])
temperature = 0.7
scaled = logits / temperature
probs = np.exp(scaled - scaled.max()) / np.exp(scaled - scaled.max()).sum()
token = rng.choice(len(logits), p=probs)
print(f"Sampled token index: {token}")
```

For MCMC at scale, use dedicated libraries: - PyMC: full Bayesian modeling with NUTS (adaptive HMC) - emcee: ensemble MCMC sampler - NumPyro/JAX: GPU-accelerated MCMC

You built these from scratch. Now you know what the library calls are doing.

Exercises

1. Implement inverse CDF sampling for the Cauchy distribution. The CDF is $F(x) = 0.5 + \arctan(x)/\pi$. Generate 10,000 samples and plot the histogram against the true PDF. Notice the heavy tails (extreme values far from center).
2. Use rejection sampling to generate samples from a Beta(2, 5) distribution using a Uniform(0, 1) proposal. Plot the accepted samples against the true Beta PDF. What is the theoretical acceptance rate?
3. Estimate the integral of $\sin(x)$ from 0 to π using Monte Carlo with 1,000, 10,000, and 100,000 samples. Compare the error at each level. Verify that the error scales as $O(1/\sqrt{N})$.

4. Implement Metropolis-Hastings to sample from a 2D distribution $p(x, y)$ proportional to $\exp(-(x^2 + y^2 + x^2 + y^2 - 8x - 8y) / 2)$. Plot the samples and the chain trajectory. Experiment with different proposal standard deviations.
5. Build a complete text generation demo: given a vocabulary of 10 words with logits, generate sequences of 20 tokens using (a) greedy, (b) temperature=0.7, (c) top-k=3, (d) top-p=0.9. Compare the diversity of outputs across 5 runs.

Key Terms

Term	What people say	What it actually means
Sampling	"Drawing random values"	Generating values according to a probability distribution. The mechanism behind all generative AI
Uniform distribution	"All equally likely"	Every value in $[a, b]$ has equal probability density $1/(b-a)$. The starting point for all sampling methods
Inverse CDF	"Probability transform"	$F_{\text{inverse}}(U)$ converts a uniform sample into a sample from any distribution with known CDF. Exact and efficient
Rejection sampling	"Propose and accept/reject"	Generate from a simple proposal, accept with probability proportional to target/proposal ratio. Exact but wastes samples
Importance sampling	"Reweight samples"	Estimate expectations under $p(x)$ using samples from $q(x)$ by weighting each sample by $p(x)/q(x)$. Core to PPO in RL
Monte Carlo	"Average random samples"	Approximate integrals as sample averages. Error $O(1/\sqrt{N})$ regardless of dimension
MCMC	"Random walk that converges"	Construct a Markov chain whose stationary distribution is the target. Metropolis-Hastings is the foundational algorithm
Metropolis-Hastings	"Accept uphill, sometimes downhill"	Propose moves, accept based on density ratio. Detailed balance ensures convergence to target distribution
Gibbs sampling	"One variable at a time"	Update each variable from its conditional distribution holding others fixed. 100% acceptance rate
Temperature	"Confidence knob"	Divides logits by T before softmax. $T < 1$ sharpens (more confident), $T > 1$ flattens (more diverse)
Top-k sampling	"Keep the k best"	Zero out all but the k highest-probability tokens, renormalize, sample. Fixed candidate set size

Term	What people say	What it actually means
Nucleus sampling (top-p)	“Keep the probable ones”	Keep the smallest set of tokens whose cumulative probability exceeds p. Adaptive candidate set size
Reparameterization trick	“Move randomness outside”	Write $z = \mu + \sigma * \epsilon$ where $\epsilon \sim N(0,1)$. Makes sampling differentiable. Essential for VAE training
Gumbel-Softmax	“Soft categorical sampling”	Differentiable approximation to categorical sampling using Gumbel noise + softmax with temperature
Stratified sampling	“Forced coverage”	Divide sample space into strata, sample from each. Always lower variance than naive Monte Carlo
Burn-in	“Warm-up period”	Initial MCMC samples discarded before the chain reaches its stationary distribution
Detailed balance	“Reversibility condition”	$p(x) * T(x \rightarrow y) = p(y) * T(y \rightarrow x)$. Sufficient condition for p to be the stationary distribution of a Markov chain
Diffusion sampling	“Iterative denoising”	Generate data by starting from noise and applying learned denoising steps. Each step is a conditional sampling operation

Sampling Strategy Selection

How to pick the right sampling method for text generation, Bayesian inference, Monte Carlo estimation, and training.

Decision Checklist

1. Are you generating output (text, images) or estimating a quantity (integral, expectation)?
2. Can you sample directly from the target distribution, or only evaluate its density?
3. Is the target distribution discrete or continuous?
4. What dimension is the sample space? Low (< 5), medium (5-100), or high (> 100)?
5. Do you need exact samples or approximate ones?
6. Do you need gradients through the sampling operation?

When to use each method

Method	When to use	Complexity	Exact?
Direct sampling	You have the CDF or can use a library function	O(1) per sample	Yes

Method	When to use	Complexity	Exact?
Inverse CDF	Known closed-form CDF inverse (exponential, Cauchy)	$O(1)$ per sample	Yes
Box-Muller	Need normal samples without a library	$O(1)$ per sample	Yes
Rejection sampling	Can evaluate target PDF, low dimension (1-3)	$O(1/\text{acceptance})$ per sample	Yes
Importance sampling	Need expectations, not individual samples	$O(n)$ for n samples	Approximate
Stratified sampling	Monte Carlo estimation, want lower variance	$O(n)$ for n samples	Approximate
Metropolis-Hastings	High-dimensional, can evaluate unnormalized density	$O(1)$ per step + burn-in	Asymptotically
Gibbs sampling	Can sample from each conditional distribution	$O(d)$ per full sweep	Asymptotically
HMC/NUTS	High-dimensional continuous, smooth density	$O(L * d)$ per step	Asymptotically
Temperature sampling	LLM text generation, control creativity	$O(V)$ for vocab size V	N/A
Top-k sampling	LLM generation, remove unlikely tokens	$O(V \log k)$	N/A
Top-p (nucleus)	LLM generation, adaptive candidate set	$O(V \log V)$	N/A
Reparameterization	Need gradients through Gaussian sampling (VAEs)	$O(d)$	Yes
Gumbel-Softmax	Need gradients through categorical sampling	$O(k)$ for k classes	Approximate

LLM generation settings

Use case	Temperature	Top-p	Top-k	Notes
Factual Q&A	0.0 (greedy)	–	–	Deterministic, no randomness
Code generation	0.2-0.5	0.9	–	Low creativity, high coherence

Use case	Temperature	Top-p	Top-k	Notes
General chat	0.7	0.9	–	Balanced
Creative writing	0.9-1.2	0.95	–	Higher diversity
Brainstorming	1.0-1.5	0.95	–	Maximum diversity, may lose coherence

Temperature and top-p can be combined. Apply temperature first (scale logits), then apply top-p filtering.

MCMC method selection

Property	Metropolis-Hastings	Gibbs	HMC/NUTS
Dimension	Any	Any (best < 100)	High (100+)
Requires conditionals	No	Yes	No
Requires gradient	No	No	Yes
Acceptance rate	Tune to ~23%	Always 100%	Tune to ~65%
Correlation	High (random walk)	Moderate	Low
Burn-in	Long	Moderate	Short
Best for	Exploration, simple models	Conjugate models, Bayesian networks	Continuous posteriors, deep probabilistic models

Common mistakes

- Using rejection sampling in high dimensions. Acceptance rate drops exponentially with dimension. Above 5 dimensions, switch to MCMC.
- Setting MCMC proposal variance too high or too low. Too high: most proposals rejected, chain stuck. Too low: all proposals accepted, chain moves slowly. Target ~23% acceptance for random walk MH.
- Forgetting burn-in. The first N samples from MCMC are biased by the starting point. Discard at least 1000 steps (or more for complex distributions).
- Using importance sampling with a proposal very different from the target. A few samples get enormous weights, making the estimate unreliable. Monitor the effective sample size: $ESS = (\sum w_i)^2 / \sum (w_i^2)$.
- Using temperature > 0 for tasks that need deterministic output (e.g., classification, structured extraction). Use greedy (T=0) or beam search instead.

- Not combining top-p with temperature. Temperature alone does not remove garbage tokens from the long tail. Top-p does.
- Backpropagating through a standard sampling operation. Use reparameterization trick for continuous (Gaussian) and Gumbel-Softmax for discrete (categorical).

Quick reference: variance reduction techniques

Technique	How it works	Variance reduction
Stratified sampling	Divide space into strata, sample each	Always $\leq$ standard MC
Antithetic variates	Use both U and $1-U$	Works for monotone functions
Control variates	Subtract a known-mean variable	Proportional to correlation
Importance sampling	Reweight samples from a better proposal	Depends on proposal quality
Latin hypercube	Stratify each dimension independently	Better than stratified in high-d

Quiz

Pre-lesson check

1. What does temperature < 1.0 do to a language model's output distribution?
 - a) Makes the distribution more uniform (more random)
 - b) Sharpens the distribution, making the highest-probability token more likely (**correct**)
 - c) Removes all tokens except the top one
 - d) Has no effect on the output

Explanation: Temperature < 1.0 divides logits by a number less than 1, which amplifies differences between logits. After softmax, the highest-probability token gets an even larger share. Temperature approaches 0 gives greedy (argmax) decoding.
2. What is the key difference between top-k and top-p (nucleus) sampling?
 - a) Top-k is faster than top-p

- b) Top-k keeps a fixed number of tokens; top-p keeps a variable number based on cumulative probability **(correct)**
- c) Top-p only works with temperature = 1.0
- d) Top-k works on logits while top-p works on probabilities
Explanation: Top-k always keeps exactly k tokens regardless of the probability distribution. Top-p adaptively keeps the smallest set of tokens whose cumulative probability exceeds p. When the model is confident, top-p keeps few tokens; when uncertain, it keeps many.

Post-lesson check

3. Why can't you backpropagate through a standard sampling operation $z \sim N(\mu, \sigma^2)$?
 - a) Normal distributions don't have gradients
 - b) The sampling operation is non-deterministic and has no well-defined derivative with respect to μ and σ **(correct)**
 - c) PyTorch doesn't support normal distributions
 - d) The gradient is always exactly zero
*Explanation: Sampling introduces a stochastic discontinuity — you can't compute $d(\text{sample})/d(\mu)$ for a random draw. The reparameterization trick solves this by writing $z = \mu + \sigma \epsilon$ (where $\epsilon \sim N(0,1)$), making z a deterministic, differentiable function of μ and σ .**
4. In Metropolis-Hastings MCMC, what happens if the proposal standard deviation is set much too large?
 - a) The chain converges faster because it takes bigger steps
 - b) Most proposals land in low-probability regions and are rejected, so the chain barely moves **(correct)**
 - c) The stationary distribution changes to a uniform distribution
 - d) The burn-in period becomes zero
Explanation: With a large proposal standard deviation, proposed points are far from the current position and likely land in low-probability regions. These are rejected, causing the

chain to stay stuck at the current point. The optimal acceptance rate is about 23% for high-dimensional Gaussian proposals.

5. In rejection sampling, what happens to the acceptance rate as the dimensionality of the target distribution increases?

- a) It stays constant regardless of dimension
- b) It increases because there are more dimensions to accept in
- c) It drops exponentially because most of the proposal volume gets rejected (**correct**)
- d) It approaches 50% in all cases

Explanation: In high dimensions, the volume of the proposal distribution that overlaps with the target distribution shrinks exponentially. The bound M grows, and the acceptance rate ($1/M$) drops exponentially. This is the curse of dimensionality for rejection sampling.

Linear Systems

Solving $Ax = b$ is the oldest problem in mathematics that still runs your neural network.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors & Matrices), 03 (Matrix Transformations) **Time:** ~120 minutes

Learning Objectives

- Solve $Ax = b$ using Gaussian elimination with partial pivoting and back substitution
- Factor matrices with LU, QR, and Cholesky decompositions and explain when each is appropriate
- Derive the normal equations for least squares and connect them to linear and ridge regression
- Diagnose ill-conditioned systems using the condition number and apply regularization to stabilize them

The Problem

Every time you train a linear regression, you solve a linear system. Every time you compute a least-squares fit, you solve a linear system. Every time a neural network layer computes $y = Wx + b$, it is evaluating one side of a linear system. When you add regularization, you modify the system. When you use Gaussian processes, you factor a matrix. When you invert a covariance matrix for Mahalanobis distance, you solve a linear system.

The equation $Ax = b$ appears everywhere. A is a matrix of known coefficients. b is a vector of known outputs. x is the vector of unknowns you want to find. In linear regression, A is your data matrix, b is your target vector, and x is the weight vector. The entire model reduces to: find x such that Ax is as close to b as possible.

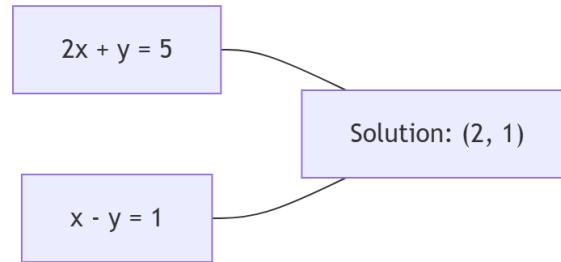
This lesson builds every major method for solving that equation from scratch. You will understand why some methods are fast and others are stable, why some work only for square systems and others handle overdetermined ones, and why the condition number of your matrix determines whether your answer means anything at all.

The Concept

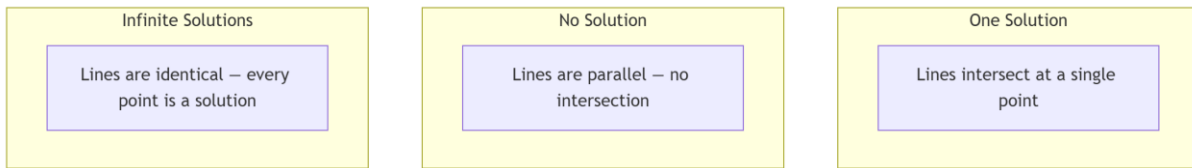
What $Ax = b$ means geometrically

A system of linear equations has a geometric interpretation. Each equation defines a hyperplane. The solution is the point (or set of points) where all hyperplanes intersect.

$2x + y = 5$	Two lines in 2D.
$x - y = 1$	They intersect at $x=2, y=1$.



Three things can happen:



In matrix form, “one solution” means A is invertible. “No solution” means the system is inconsistent. “Infinite solutions” means A has a null space. Most ML problems fall in the “no exact solution” category because you have more equations (data points) than unknowns (parameters). That is where least squares comes in.

Column picture vs row picture

There are two ways to read $Ax = b$.

Row picture. Each row of A defines one equation. Each equation is a hyperplane. The solution is where they all intersect.

Column picture. Each column of A is a vector. The question becomes: what linear combination of the columns of A produces b ?

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \quad b = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

Row picture: solve $2x + y = 5$ and $x - y = 1$ simultaneously.

Column picture: find x_1, x_2 such that:

$$x_1 * [2, 1] + x_2 * [1, -1] = [5, 1]$$

$$2 * [2, 1] + 1 * [1, -1] = [4+1, 2-1] = [5, 1] \quad \text{check.}$$

The column picture is more fundamental. If b lies in the column space of A , the system has a solution. If b does not, you find the closest point in the column space. That closest point is the least-squares solution.

Gaussian elimination

Gaussian elimination transforms $Ax = b$ into an upper triangular system $Ux = c$ that you solve by back substitution. It is the most direct method.

The algorithm:

1. For each column k (the pivot column):
 - a. Find the largest entry in column k at or below row k (partial pivoting).
 - b. Swap that row with row k .
 - c. For each row i below k :
 - Compute multiplier $m = A[i][k] / A[k][k]$
 - Subtract m times row k from row i .
2. Back substitute: solve from the last equation upward.

Example:

Original:

$$\begin{array}{ccc|c} 2 & 1 & 1 & 8 \\ 4 & 3 & 3 & 20 \\ 2 & 3 & 1 & 12 \end{array} \quad \begin{array}{l} R2 = R2 - (2)R1 \\ R3 = R3 - (1)R1 \end{array} \quad \begin{array}{ccc|c} 2 & 1 & 1 & 8 \\ 0 & 1 & 1 & 4 \\ 0 & 2 & 0 & 4 \end{array}$$
$$\begin{array}{ccc|c} 2 & 1 & 1 & 8 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & -2 & -4 \end{array} \quad \begin{array}{l} R3 = R3 - (2)R2 \end{array}$$

Back substitute:

$$\begin{array}{l} -2 * x_3 = -4 \quad \rightarrow x_3 = 2 \\ x_2 + 2 = 4 \quad \rightarrow x_2 = 2 \\ 2*x_1 + 2 + 2 = 8 \quad \rightarrow x_1 = 2 \end{array}$$

Gaussian elimination costs $O(n^3)$ operations. For a 1000x1000 system, that is about a billion floating-point operations. Fast, but you can do better if you need to solve multiple systems with the same A .

Partial pivoting: why it matters

Without pivoting, Gaussian elimination can fail or produce garbage. If a pivot element is zero, you divide by zero. If it is small, you amplify rounding errors.

Bad pivot:

$$\begin{array}{ccc|c} 0.001 & 1 & 1.001 & \\ 1 & 1 & 2 & \end{array}$$

$$m = 1/0.001 = 1000$$

With partial pivoting:

Swap rows first:

$$\begin{array}{ccc|c} 1 & 1 & 2 & \\ 0.001 & 1 & 1.001 & \end{array}$$

$$m = 0.001/1 = 0.001$$

$$R2 = R2 - 1000 \cdot R1$$

$$\begin{array}{cc|cc} 0.001 & 1 & 1.001 & \\ 0 & -999 & -999.0 & \end{array}$$

$$x2 = 1.000 \text{ (correct)}$$

$$\begin{aligned} x1 &= (1.001 - 1)/0.001 \\ &= 0.001/0.001 = 1.000 \end{aligned}$$

$$R2 = R2 - 0.001 \cdot R1$$

$$\begin{array}{cc|cc} 1 & 1 & 2 & \\ 0 & 0.999 & 0.999 & \end{array}$$

$$x2 = 1.000 \text{ (correct)}$$

$$\begin{aligned} x1 &= (2 - 1)/1 = 1.000 \text{ (correct)} \\ &\text{Stable because the multiplier is small.} \end{aligned}$$

In floating-point arithmetic with limited precision, the unpivoted version can lose significant digits. Partial pivoting always selects the largest available pivot to minimize error amplification.

LU decomposition

LU decomposition factors A into a lower triangular matrix L and an upper triangular matrix U: $A = LU$. The L matrix stores the multipliers from Gaussian elimination. The U matrix is the result of elimination.

$$A = L @ U$$

$$\begin{array}{ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 \\ 4 & 3 & 3 & 2 & 1 & 0 \\ 2 & 3 & 1 & 1 & 2 & 1 \end{array} = \begin{array}{ccc|ccc} 2 & 1 & 1 & 2 & 1 & 1 \\ 4 & 3 & 3 & 0 & 1 & 1 \\ 2 & 3 & 1 & 0 & 0 & -2 \end{array} @$$

Why factor instead of just eliminating? Because once you have L and U, solving $Ax = b$ for any new b costs only $O(n^2)$:

$$Ax = b$$

$$LUx = b$$

$$\text{Let } y = Ux:$$

$$Ly = b \quad (\text{forward substitution, } O(n^2))$$

$$Ux = y \quad (\text{back substitution, } O(n^2))$$

The $O(n^3)$ cost is paid once during factorization. Every subsequent solve is $O(n^2)$. If you need to solve 1000 systems with the same A but different b vectors, LU saves a factor of 1000/3 in total work.

With partial pivoting, you get $PA = LU$ where P is a permutation matrix recording the row swaps.

QR decomposition

QR decomposition factors A into an orthogonal matrix Q and an upper triangular matrix R: $A = QR$.

An orthogonal matrix has the property $Q^T Q = I$. Its columns are orthonormal vectors. Multiplying by Q preserves lengths and angles.

$$A = Q @ R$$

Q has orthonormal columns: $Q^T Q = I$

R is upper triangular

To solve $Ax = b$:

$QRx = b$

$Rx = Q^T b$ (just multiply by Q^T , no inversion needed)

Back substitute to get x.

QR is numerically more stable than LU for solving least-squares problems. The Gram-Schmidt process builds Q column by column:

Given columns $a_1, a_2, \dots$ of A:

$q_1 = a_1 / ||a_1||$

$q_2 = a_2 - (a_2 \cdot q_1) * q_1$ (subtract projection onto q_1)
 $q_2 = q_2 / ||q_2||$ (normalize)

$q_3 = a_3 - (a_3 \cdot q_1) * q_1 - (a_3 \cdot q_2) * q_2$
 $q_3 = q_3 / ||q_3||$

$R[i][j] = q_i \cdot a_j$ for $i \leq j$

Each step removes the component along all previous q vectors, leaving only the new orthogonal direction.

Cholesky decomposition

When A is symmetric ($A = A^T$) and positive definite (all eigenvalues positive), you can factor it as $A = L L^T$ where L is lower triangular. This is the Cholesky decomposition.

$A = L @ L^T$

$\begin{vmatrix} 4 & 2 \\ 2 & 5 \end{vmatrix} = \begin{vmatrix} 1 & 2 \\ 0 & 2 \end{vmatrix} @ \begin{vmatrix} 2 & 1 \\ 0 & 2 \end{vmatrix}$

$L[i][i] = \sqrt{A[i][i] - \sum(L[i][k]^2 \text{ for } k < i)}$
 $L[i][j] = (A[i][j] - \sum(L[i][k]*L[j][k] \text{ for } k < j)) / L[j][j]$ for $i > j$

Cholesky is twice as fast as LU and requires half the storage. It only works for symmetric positive definite matrices, but those show up constantly:

- Covariance matrices are symmetric positive semi-definite (positive definite with regularization).
- The kernel matrix in Gaussian processes is symmetric positive definite.
- The Hessian of a convex function at a minimum is symmetric positive definite.

- $A^T A$ is always symmetric positive semi-definite.

In Gaussian processes, you factor the kernel matrix K with Cholesky, then solve $K \alpha = y$ to get the predictive mean. The Cholesky factor also gives you the log-determinant for the marginal likelihood: $\log \det(K) = 2 * \sum(\log(\text{diag}(L)))$.

Least squares: when $Ax = b$ has no exact solution

If A is $m \times n$ with $m > n$ (more equations than unknowns), the system is overdetermined. There is no exact solution. Instead, you minimize the squared error:

```
minimize ||Ax - b||^2
```

This is the sum of squared residuals:

```
sum((A[i,:] @ x - b[i])^2 for i in range(m))
```

The minimizer satisfies the normal equations:

$$A^T A x = A^T b$$

Derivation: expand $||Ax - b||^2 = (Ax - b)^T (Ax - b) = x^T A^T A x - 2 x^T A^T b + b^T b$. Take the gradient with respect to x , set it to zero: $2 A^T A x - 2 A^T b = 0$.

Original system (overdetermined, 4 equations, 2 unknowns):

$$\begin{array}{cc|c} 1 & 1 & 3 \\ 1 & 2 & 5 \\ 1 & 3 & 6 \\ 1 & 4 & 8 \end{array} x = \begin{array}{c} 3 \\ 5 \\ 6 \\ 8 \end{array} \quad \text{No exact } x \text{ satisfies all 4 equations.}$$

Normal equations:

$$A^T A = \begin{array}{cc|c} 4 & 10 & \\ \hline 10 & 30 & \end{array} \quad A^T b = \begin{array}{c|c} 22 & \\ \hline 63 & \end{array}$$

Solve: $x = [1.5, 1.7]$

This is linear regression. $x[0]$ is the intercept, $x[1]$ is the slope.

Normal equations = linear regression

The connection is exact. In linear regression, your data matrix X has one row per sample and one column per feature. Your target vector y has one entry per sample. The weight vector w satisfies:

$$\begin{aligned} X^T X w &= X^T y \\ w &= (X^T X)^{-1} X^T y \end{aligned}$$

This is the closed-form solution to linear regression. Every call to `sklearn.linear_model.LinearRegression.fit()` computes this (or an equivalent via QR or SVD).

Add a regularization term $\lambda * I$ to the matrix and you get ridge regression:

$$\begin{aligned}(X^T X + \text{lambda} * I) w &= X^T y \\ w &= (X^T X + \text{lambda} * I)^{-1} X^T y\end{aligned}$$

The regularization makes the matrix better conditioned (easier to invert accurately) and prevents overfitting by shrinking the weights toward zero. The matrix $X^T X + \lambda * I$ is always symmetric positive definite when $\lambda > 0$, so you can use Cholesky to solve it.

Pseudoinverse (Moore-Penrose)

The pseudoinverse A^+ generalizes matrix inversion to non-square and singular matrices. For any matrix A :

$$\begin{aligned}x &= A^+ b \\ \text{where } A^+ &= V \Sigma^+ U^T \quad (\text{computed via SVD})\end{aligned}$$

Σ^+ is formed by taking the reciprocal of each nonzero singular value and transposing the result. If $A = U \Sigma V^T$, then $A^+ = V \Sigma^+ U^T$.

$$\begin{aligned}A &= U \Sigma V^T && (\text{SVD}) \\ \Sigma &= \begin{bmatrix} 5 & 0 & \\ 0 & 2 & \\ 0 & 0 & \end{bmatrix} && \Sigma^+ = \begin{bmatrix} 1/5 & 0 & 0 & \\ 0 & 1/2 & 0 & \end{bmatrix} \\ A^+ &= V \Sigma^+ U^T\end{aligned}$$

The pseudoinverse gives the minimum-norm least-squares solution. If the system has: - One solution: $A^+ b$ gives it. - No solution: $A^+ b$ gives the least-squares solution. - Infinite solutions: $A^+ b$ gives the one with the smallest $\|x\|$.

NumPy's `np.linalg.lstsq` and `np.linalg.pinv` both use the SVD internally.

Condition number

The condition number measures how sensitive the solution is to small changes in the input. For a matrix A , the condition number is:

$$\kappa(A) = \|A\| * \|A^{-1}\| = \text{sigma_max} / \text{sigma_min}$$

where `sigma_max` and `sigma_min` are the largest and smallest singular values.

Well-conditioned ($\kappa \sim 1$):	Ill-conditioned ($\kappa \sim 10^{15}$):
Small change in $b \rightarrow$	Small change in $b \rightarrow$
small change in x	huge change in x
$\begin{vmatrix} 2 & 0 \\ 0 & 1 \end{vmatrix}$ $\kappa = 2/1 = 2$ safe to solve	$\begin{vmatrix} 1 & 1 \\ 1 & 1+10^{-15} \end{vmatrix}$ $\kappa \sim 10^{15}$ solution is garbage

Rules of thumb: - $\kappa < 100$: safe, solution is accurate. - $\kappa \sim 10^k$: you lose about k digits of precision from your floating-point arithmetic. - $\kappa \sim 10^{16}$ (for float64): the solution is meaningless. The matrix is effectively singular.

In ML, ill-conditioning happens when features are nearly collinear. Regularization (adding $\lambda * I$) improves the condition number from $\sigma_{\max} / \sigma_{\min}$ to $(\sigma_{\max} + \lambda) / (\sigma_{\min} + \lambda)$.

Iterative methods: conjugate gradient

For very large sparse systems (millions of unknowns), direct methods like LU or Cholesky are too expensive. Iterative methods approximate the solution by improving a guess over many iterations.

Conjugate gradient (CG) solves $Ax = b$ when A is symmetric positive definite. It finds the exact solution in at most n iterations (in exact arithmetic), but typically converges much faster if the eigenvalues of A are clustered.

```

Algorithm sketch:
x0 = initial guess (often zero)
r0 = b - A x0          (residual)
p0 = r0                (search direction)

For k = 0, 1, 2, ...:
    alpha = (rk . rk) / (pk . A pk)
    x_{k+1} = xk + alpha * pk
    r_{k+1} = rk - alpha * A pk
    beta = (r_{k+1} . r_{k+1}) / (rk . rk)
    p_{k+1} = r_{k+1} + beta * pk
    if ||r_{k+1}|| < tolerance: stop

```

CG is used in: - Large-scale optimization (Newton-CG method) - Solving PDE discretizations - Kernel methods where the kernel matrix is too large to factor - Preconditioning for other iterative solvers

The convergence rate depends on the condition number. Better conditioned systems converge faster, which is another reason regularization helps.

The full picture: which method when

Method	Requirements	Cost	Use case
Gaussian elimination	Square, nonsingular A	$O(n^3)$	One-off solve of a square system
LU decomposition	Square, nonsingular A	$O(n^3)$ factor + $O(n^2)$ solve	Multiple solves with the same A
QR decomposition	Any A ($m \geq n$)	$O(mn^2)$	Least squares, numerically stable
Cholesky	Symmetric positive definite A	$O(n^3/3)$	Covariance matrices, Gaussian processes, ridge regression
Normal equations	Overdetermined ($m > n$)	$O(mn^2 + n^3)$	Linear regression (small n)
SVD / pseudoinverse	Any A	$O(mn^2)$	Rank-deficient systems, minimum-norm solutions
Conjugate gradient	Symmetric positive definite, sparse A	$O(n * k * \text{nnz})$	Large sparse systems, k = iterations

Connection to ML

Every method in this lesson appears in production ML:

Linear regression. The closed-form solution solves the normal equations $X^T X w = X^T y$. This is done via Cholesky (if n is small) or QR (if numerical stability matters) or SVD (if the matrix might be rank-deficient).

Ridge regression. Adds $\lambda * I$ to $X^T X$. The regularized system $(X^T X + \lambda * I) w = X^T y$ is always solvable via Cholesky because $X^T X + \lambda * I$ is symmetric positive definite for $\lambda > 0$.

Gaussian processes. The predictive mean requires solving $K \alpha = y$ where K is the kernel matrix. Cholesky factorization of K is the standard approach. The log marginal likelihood uses $\log \det(K) = 2 \sum(\log(\text{diag}(L)))$.

Neural network initialization. Orthogonal initialization uses QR decomposition to create weight matrices whose columns are orthonormal. This prevents signal collapse in deep networks.

Preconditioning. Large-scale optimizers use incomplete Cholesky or incomplete LU as preconditioners for conjugate gradient solvers.

Feature engineering. The condition number of $X^T X$ tells you if your features are collinear. If κ is large, drop features or add regularization.

Build It

Step 1: Gaussian elimination with partial pivoting

```
import numpy as np

def gaussian_elimination(A, b):
    n = len(b)
    Ab = np.hstack([A.astype(float), b.reshape(-1, 1).astype(float)])

    for k in range(n):
        max_row = k + np.argmax(np.abs(Ab[k:, k]))
        Ab[[k, max_row]] = Ab[[max_row, k]]

        if abs(Ab[k, k]) < 1e-12:
            raise ValueError(f"Matrix is singular or nearly singular at pivot {k}")

        for i in range(k + 1, n):
            m = Ab[i, k] / Ab[k, k]
            Ab[i, k:] -= m * Ab[k, k:]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (Ab[i, -1] - Ab[i, i+1:n] @ x[i+1:n]) / Ab[i, i]

    return x
```

Step 2: LU decomposition

```
def lu_decompose(A):
    n = A.shape[0]
    L = np.eye(n)
    U = A.astype(float).copy()
    P = np.eye(n)

    for k in range(n):
        max_row = k + np.argmax(np.abs(U[k:, k]))
        if max_row != k:
            U[[k, max_row]] = U[[max_row, k]]
            P[[k, max_row]] = P[[max_row, k]]
            if k > 0:
                L[[k, max_row], :k] = L[[max_row, k], :k]

        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k:] -= L[i, k] * U[k, k:]

    return P, L, U

def lu_solve(P, L, U, b):
    n = len(b)
```



```

Pb = P @ b.astype(float)

y = np.zeros(n)
for i in range(n):
    y[i] = Pb[i] - L[i, :i] @ y[:i]

x = np.zeros(n)
for i in range(n - 1, -1, -1):
    x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]

return x

```

Step 3: Cholesky decomposition

```

def cholesky(A):
    n = A.shape[0]
    L = np.zeros_like(A, dtype=float)

    for i in range(n):
        for j in range(i + 1):
            s = A[i, j] - L[i, :j] @ L[j, :j]
            if i == j:
                if s <= 0:
                    raise ValueError("Matrix is not positive definite")
                L[i, j] = np.sqrt(s)
            else:
                L[i, j] = s / L[j, j]

    return L

```

Step 4: Least squares via normal equations

```

def least_squares_normal(A, b):
    AtA = A.T @ A
    Atb = A.T @ b
    return gaussian_elimination(AtA, Atb)

def ridge_regression(A, b, lam):
    n = A.shape[1]
    AtA = A.T @ A + lam * np.eye(n)
    Atb = A.T @ b
    L = cholesky(AtA)
    y = np.zeros(n)
    for i in range(n):
        y[i] = (Atb[i] - L[i, :i] @ y[:i]) / L[i, i]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - L.T[i, i+1:] @ x[i+1:]) / L.T[i, i]
    return x

```

Step 5: Condition number

```
def condition_number(A):
    U, S, Vt = np.linalg.svd(A)
    return S[0] / S[-1]
```

Use It

Putting the pieces together for linear regression and ridge regression on real data:

```
np.random.seed(42)
X_raw = np.random.randn(100, 3)
w_true = np.array([2.0, -1.0, 0.5])
y = X_raw @ w_true + np.random.randn(100) * 0.1

X = np.column_stack([np.ones(100), X_raw])

w_ols = least_squares_normal(X, y)
print(f"OLS weights (ours): {w_ols}")

w_np = np.linalg.lstsq(X, y, rcond=None)[0]
print(f"OLS weights (numpy): {w_np}")
print(f"Max difference: {np.max(np.abs(w_ols - w_np)):.2e}")

w_ridge = ridge_regression(X, y, lam=1.0)
print(f"Ridge weights (ours): {w_ridge}")

from sklearn.linear_model import Ridge
ridge_sk = Ridge(alpha=1.0, fit_intercept=False)
ridge_sk.fit(X, y)
print(f"Ridge weights (sklearn): {ridge_sk.coef_}")
```

Exercises

1. Solve the system $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix} x = \begin{bmatrix} 6 \\ 15 \\ 27 \end{bmatrix}$ using your Gaussian elimination, your LU solver, and `np.linalg.solve`. Verify all three give the same answer within floating-point tolerance.
2. Generate a 50x5 random matrix X and target $y = X @ w_{\text{true}} + \text{noise}$. Solve for w using normal equations, QR (via `np.linalg.qr`), SVD (via `np.linalg.svd`), and `np.linalg.lstsq`. Compare all four solutions. Measure the condition number of $X^T X$ and explain how it affects which method you trust.
3. Create a nearly singular matrix by making two columns almost identical (e.g., column 2 = column 1 + $1e-10 * \text{noise}$). Compute its condition number. Solve $Ax = b$ with and without regularization (add $0.01 * I$). Compare the solutions and residuals. Explain why regularization helps.

4. Implement the conjugate gradient algorithm for a 100x100 random symmetric positive definite matrix. Count how many iterations it takes to converge to tolerance $1e-8$. Compare with the theoretical maximum of n iterations.
5. Time your Cholesky solver vs your LU solver vs `np.linalg.solve` on symmetric positive definite matrices of size 10, 50, 200, 500. Plot the results. Verify Cholesky is roughly 2x faster than LU.

Key Terms

Term	What people say	What it actually means
Linear system	"Solve for x"	A set of linear equations $Ax = b$. Finding x means finding the input that produces output b under transformation A.
Gaussian elimination	"Row reduce"	Systematically zero out entries below the diagonal using row operations, producing an upper triangular system solvable by back substitution. $O(n^3)$.
Partial pivoting	"Swap rows for stability"	Before eliminating in column k, swap the row with the largest absolute value in that column to the pivot position. Prevents division by small numbers.
LU decomposition	"Factor into triangles"	Write $A = LU$ where L is lower triangular (stores multipliers) and U is upper triangular (the eliminated matrix). Amortizes the $O(n^3)$ cost over multiple solves.
QR decomposition	"Orthogonal factorization"	Write $A = QR$ where Q has orthonormal columns and R is upper triangular. More stable than LU for least squares.
Cholesky decomposition	"Square root of a matrix"	For symmetric positive definite A, write $A = LL^T$. Half the cost of LU. Used for covariance matrices, kernel matrices, and ridge regression.
Least squares	"Best fit when exact is impossible"	Minimize the sum of squared residuals
Normal equations	"The calculus shortcut"	$A^T A x = A^T b$. Setting the gradient of
Pseudoinverse	"Inversion for non-square matrices"	$A^+ = V \Sigma^+ U^T$ via SVD. Gives the minimum-norm least-squares solution for any matrix, square or rectangular, singular or not.
Condition number	"How trustworthy is this answer"	$\kappa = \sigma_{\max} / \sigma_{\min}$. Measures sensitivity to input perturbations. Lose about $\log_{10}(\kappa)$ digits of precision.

Term	What people say	What it actually means
Ridge regression	“Regularized least squares”	Solve $(X^T X + \lambda I) w = X^T y$. Adding λI improves conditioning and shrinks weights toward zero. Prevents overfitting.
Conjugate gradient	“Iterative $Ax=b$ for big matrices”	An iterative solver for symmetric positive definite systems. Converges in at most n steps. Practical for large sparse systems where factorization is too expensive.
Overdetermined system	“More data than parameters”	$m > n$ in an m -by- n system. No exact solution exists. Least squares finds the best approximation. This is every regression problem.
Back substitution	“Solve from the bottom up”	Given an upper triangular system, solve the last equation first, then substitute backward. $O(n^2)$.
Forward substitution	“Solve from the top down”	Given a lower triangular system, solve the first equation first, then substitute forward. $O(n^2)$. Used in the L step of LU solves.

You are a linear algebra solver advisor. Your job is to recommend the best algorithm for solving $Ax = b$ based on the properties of matrix A .

When a user describes a linear system or provides a matrix, recommend the optimal solver.

Structure your response as:

1. **Classify the matrix.** Determine which properties apply:
 - Size: small ($n < 100$), medium (100-10,000), large ($> 10,000$)
 - Shape: square ($n \times n$), tall ($m > n$, overdetermined), wide ($m < n$, underdetermined)
 - Structure: dense, sparse, banded, triangular, diagonal
 - Symmetry: symmetric ($A = A^T$) or not
 - Definiteness: positive definite, positive semi-definite, indefinite, or unknown
 - Conditioning: well-conditioned ($\kappa < 100$) or ill-conditioned ($\kappa > 10^6$)
2. **Recommend the algorithm.** Pick from the decision tree below.
3. **State the cost.** Give the time complexity and whether it is a one-off solve or amortized across multiple right-hand sides.
4. **Warn about pitfalls.** Flag any numerical stability concerns for the given matrix type.

Use this decision framework:

```

Is the system square (m = n)?
  Yes --> Is A triangular?
    Yes --> Back/forward substitution.  $O(n^2)$ . Done.
  Is A diagonal?
    Yes --> Divide b by diagonal entries.  $O(n)$ . Done.
  Is A symmetric positive definite?
    Yes --> Cholesky ( $A = LL^T$ ).  $O(n^3/3)$ . Fastest for this class.
      Use for: covariance matrices, kernel matrices, ridge regression.
  Is A symmetric but indefinite?
    Yes -->  $LDL^T$  decomposition. Similar cost to Cholesky.
  Is A general dense?
    Yes --> LU with partial pivoting ( $PA = LU$ ).  $O(2n^3/3)$ .
      If solving for many b vectors, factor once, solve  $O(n^2)$  each .
  Is A large and sparse?
    Is A symmetric positive definite?
      Yes --> Conjugate gradient (CG).  $O(k * \text{nnz})$  where k = iterations.
    Is A general sparse?
      Yes --> GMRES or BiCGSTAB. Iterative, good with preconditioner.
      Alternative: Sparse LU (scipy.sparse.linalg.spsolve).

Is the system overdetermined (m > n)?
  Yes --> This is a least-squares problem: minimize  $\|Ax - b\|^2$ .
  Is  $A^T A$  well-conditioned?
    Yes --> Normal equations: solve  $A^T A x = A^T b$  via Cholesky.  $O(mn^2 + n^3/3)$ .
  Is  $A^T A$  ill-conditioned?
    Yes --> QR decomposition:  $A = QR$ , solve  $Rx = Q^T b$ .  $O(2mn^2)$ . More stable.
  Is A possibly rank-deficient?
    Yes --> SVD:  $A = USV^T$ , pseudoinverse.  $O(mn^2)$ . Most robust, slowest.
  Need regularization?
    Yes --> Ridge: solve  $(A^T A + \lambda I) x = A^T b$  via Cholesky. Always well-conditioned.

Is the system underdetermined (m < n)?
  Yes --> Infinite solutions. Use SVD pseudoinverse for minimum-norm solution.

```

Quick reference for the recommendation:

Matrix property	Recommended solver	Cost	Library call
Dense, square, general	LU (partial pivot)	$O(2n^3/3)$	<code>np.linalg.solve</code>
Dense, symmetric pos. def.	Cholesky	$O(n^3/3)$	<code>scipy.linalg.cho_solve</code>
Dense, overdetermined	QR	$O(2mn^2)$	<code>np.linalg.lstsq</code>
Dense, rank-deficient	SVD	$O(mn^2)$	<code>np.linalg.lstsq</code> or <code>pinv</code>
Sparse, sym. pos. def.	Conjugate gradient	$O(k * nnz)$	<code>scipy.sparse.linalg.cg</code>
Sparse, general	GMRES or SparseLU	$O(k * nnz)$	<code>scipy.sparse.linalg.gmres</code>
Banded	Banded LU	$O(n * bw^2)$	<code>scipy.linalg.solve_banded</code>
Multiple b, same A	Factor once (LU/Cholesky), solve many	$O(n^3) + O(n^2)$ each	<code>scipy.linalg.lu_factor</code> + <code>lu_solve</code>

Conditioning advice: - Check condition number first: `np.linalg.cond(A)`. If $\kappa > 10^{10}$, do not trust the raw solution. - Adding regularization ($\lambda * I$) improves κ from $\sigma_{\max}/\sigma_{\min}$ to $(\sigma_{\max} + \lambda)/(\sigma_{\min} + \lambda)$. - If κ is large, use QR or SVD instead of normal equations. Normal equations square the condition number.

Avoid: - Computing A^{-1} explicitly. Use a factorization and solve instead. Inversion is slower, less stable, and rarely necessary. - Using dense solvers on sparse matrices. A 100,000 x 100,000 sparse system fits in memory and solves in seconds with CG. Dense LU would need 80 GB and hours. - Using normal equations when $A^T A$ is ill-conditioned. The normal equations square the condition number: $\kappa(A^T A) = \kappa(A)^2$.

Quiz

Pre-lesson check

- What does it mean geometrically when a system $Ax = b$ has no exact solution?
 - The matrix A has all zero entries
 - The vector b does not lie in the column space of A (**correct**)
 - The system has more unknowns than equations

- d) The matrix A is symmetric

Explanation: In the column picture, $Ax = b$ asks: what linear combination of A 's columns produces b ? If b is not in the column space (span of A 's columns), no exact solution exists. This happens when the system is overdetermined (more equations than unknowns).

2. Why is partial pivoting used in Gaussian elimination?

- a) It reduces the time complexity from $O(n^3)$ to $O(n^2)$

- b) It selects the largest available pivot to minimize error amplification from dividing by small numbers **(correct)**

- c) It ensures the result is always an integer

- d) It eliminates the need for back substitution

Explanation: Without pivoting, dividing by a small pivot amplifies rounding errors. Partial pivoting swaps rows to place the largest absolute value in the pivot position, keeping the multipliers small and the computation numerically stable.

Post-lesson check

3. Why is Cholesky decomposition preferred over LU for solving $(X^T X + \lambda I) w = X^T y$ in ridge regression?

- a) Cholesky works on any matrix while LU requires square matrices

- b) The matrix $X^T X + \lambda I$ is symmetric positive definite, so Cholesky is twice as fast as LU and requires half the storage **(correct)**

- c) Cholesky gives a more accurate answer than LU

- d) LU decomposition cannot handle regularization terms

Explanation: When $\lambda > 0$, $X^T X + \lambda I$ is always symmetric positive definite. Cholesky factors $A = LL^T$ in $O(n^3/3)$ operations — roughly half the $O(2n^3/3)$ of LU — and needs only the lower triangle. It exploits the symmetry that LU does not.

4. A matrix has condition number $\kappa = 10^8$. You are using float64 (~15 digits of precision). How many digits of the solution can you trust?

- a) About 15 digits

- b) About 7 digits ($15 - \log_{10}(10^8) = 15 - 8$) **(correct)**
- c) About 8 digits
- d) Zero digits — the solution is meaningless
Explanation: You lose approximately $\log_{10}(\kappa)$ digits of precision. With $\kappa = 10^8$, you lose about 8 digits from float64's ~15 digits, leaving about 7 trustworthy digits. If κ approaches 10^{16} , the solution becomes meaningless in float64.
5. What is the main advantage of LU decomposition over Gaussian elimination when you need to solve $Ax = b$ for many different b vectors?
- a) LU decomposition is more numerically stable
- b) The $O(n^3)$ factorization is done once; each subsequent solve with a new b costs only $O(n^2)$ **(correct)**
- c) LU decomposition works on rectangular matrices
- d) LU always produces a unique solution
Explanation: LU factors $A = LU$ once in $O(n^3)$. Then for each new b , you solve $Ly = b$ (forward substitution) and $Ux = y$ (back substitution), each $O(n^2)$. Gaussian elimination would redo the full $O(n^3)$ for every new b .
-

Convex Optimization

Convex problems have one valley. Neural networks have millions. Knowing the difference matters.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 04 (Calculus for ML), 08 (Optimization)

Time: ~90 minutes

Learning Objectives

- Test whether a function is convex using the definition, second derivative, and Hessian criteria
- Implement Newton's method and compare its quadratic convergence against gradient descent
- Solve constrained optimization problems using Lagrange multipliers and interpret KKT conditions
- Explain why neural network loss landscapes are non-convex yet SGD still finds good solutions

The Problem

Lesson 08 taught you gradient descent, momentum, and Adam. Those optimizers walk downhill on any surface. But they come with no guarantees. Gradient descent on a non-convex landscape might land in a bad local minimum, get stuck on a saddle point, or oscillate forever. You used it anyway because neural networks are non-convex and there is no alternative.

But many problems in machine learning are convex. Linear regression, logistic regression, SVMs, LASSO, ridge regression. For these, something stronger exists: optimization with mathematical guarantees. A convex problem has exactly one valley. Any algorithm that walks downhill will reach the global minimum. No restarts needed. No learning rate schedules. No prayer.

Understanding convexity does three things. First, it tells you when your problem is easy (convex) versus hard (non-convex). Second, it gives you faster tools like Newton's method for convex problems. Third, it explains concepts that appear throughout ML: regularization as a constraint, duality in SVMs, and why deep learning works despite violating every nice property convexity gives you.

The Concept

Convex sets

A set S is convex if for any two points in S , the line segment between them also lies entirely in S .

Convex sets	Not convex
Rectangle: any two points inside can be connected by a line segment that stays inside	Star/crescent shape: a line between two interior points can pass outside the set

Convex sets	Not convex
Triangle: same property holds for all interior points	Donut/annulus: the hole means some line segments leave the set
The line segment between any two points stays within the set	The line segment between some pairs of points exits the set

Formal test: for any points x, y in S and any t in $[0, 1]$, the point $tx + (1-t)y$ is also in S .

Examples of convex sets: - A line, a plane, all of $\mathbb{R}^n$ - A ball (circle, sphere, hypersphere) - A halfspace: $\{x : a^T x \leq b\}$ - The intersection of any number of convex sets

Examples of non-convex sets: - A donut (annulus) - The union of two disjoint circles - Any set with a “dent” or “hole”

Convex functions

A function f is convex if its domain is a convex set and for any two points x, y in its domain and any t in $[0, 1]$:

$$f(tx + (1-t)y) \leq t*f(x) + (1-t)*f(y)$$

Geometrically: the line segment between any two points on the graph lies above or on the graph.

Property	Convex function	Non-convex function
Line segment test	The line between any two points on the graph lies above or on the curve	The line between some points on the graph dips below the curve
Shape	Single bowl/valley curving upward	Multiple peaks and valleys with mixed curvature
Local minima	Every local minimum is the global minimum	Multiple local minima may exist at different heights

Common convex functions: - $f(x) = x^2$ (parabola) - $f(x) = |x|$ (absolute value) - $f(x) = e^x$ (exponential) - $f(x) = \max(0, x)$ (ReLU, though piecewise linear) - $f(x) = -\log(x)$ for $x > 0$ (negative log) - Any linear function $f(x) = a^T x + b$ (both convex and concave)

Testing for convexity

Three practical tests, from easiest to most rigorous.

Test 1: Second derivative test (1D). If $f''(x) \geq 0$ for all x , then f is convex.

- $f(x) = x^2$: $f''(x) = 2 \geq 0$. Convex.
- $f(x) = x^3$: $f''(x) = 6x$. Negative for $x < 0$. Not convex.
- $f(x) = e^x$: $f''(x) = e^x > 0$. Convex.

Test 2: Hessian test (multivariate). If the Hessian matrix $H(x)$ is positive semidefinite for all x , then f is convex. The Hessian is the matrix of second partial derivatives.

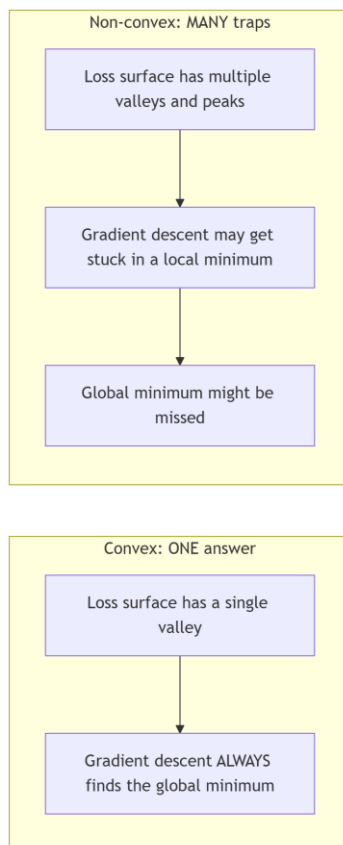
Test 3: Definition test. Check the inequality $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ directly. Useful for functions where derivatives are hard to compute.

Why convexity matters

The central theorem of convex optimization:

For a convex function, every local minimum is a global minimum.

This means gradient descent cannot get trapped. Any downhill path leads to the same answer. The algorithm is guaranteed to converge to the optimal solution.



Consequences: - No need for random restarts - No need for sophisticated learning rate schedules - Convergence proofs are possible (rate depends on function properties) - The solution is unique (up to flat regions)

Convex vs non-convex in ML

Problem	Convex?	Why
Linear regression (MSE)	Yes	Loss is quadratic in weights
Logistic regression	Yes	Log-loss is convex in weights
SVM (hinge loss)	Yes	Maximum of linear functions
LASSO (L1 regression)	Yes	Sum of convex functions is convex
Ridge regression (L2)	Yes	Quadratic + quadratic = convex
Neural network (any loss)	No	Nonlinear activations create non-convex landscape
k-means clustering	No	Discrete assignment step
Matrix factorization	No	Product of unknowns

Linear models with convex losses are convex. The moment you add hidden layers with nonlinear activations, convexity breaks.

The Hessian matrix

The Hessian H of a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is the $n \times n$ matrix of second partial derivatives.

$$H[i][j] = d^2 f / (dx_i dx_j)$$

For $f(x, y) = x^2 + 3xy + y^2$:

$$\begin{aligned} df/dx &= 2x + 3y & d^2f/dx^2 &= 2 & d^2f/dxdy &= 3 \\ df/dy &= 3x + 2y & d^2f/dydx &= 3 & d^2f/dy^2 &= 2 \end{aligned}$$
$$H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$$

The Hessian tells you about curvature: - Eigenvalues all positive: the function curves upward in every direction (convex at that point) - Eigenvalues all negative: curves downward in every direction (concave, a local max) - Mixed signs: saddle point (curves up in some directions, down in others) - Zero eigenvalue: flat in that direction (degenerate)

For convexity, the Hessian must be positive semidefinite (all eigenvalues ≥ 0) everywhere, not just at one point.

Newton's method

Gradient descent uses first-order information (the gradient). Newton's method uses second-order information (the Hessian). It fits a quadratic approximation at the current point and jumps directly to the minimum of that quadratic.

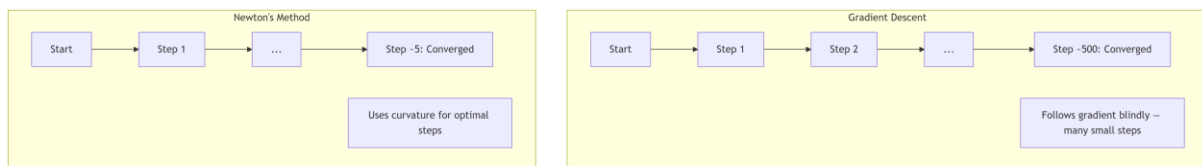
Update rule:

$$x_{\text{new}} = x - H^{-1} * \text{gradient}$$

Compare to gradient descent:

$$x_{\text{new}} = x - \text{lr} * \text{gradient}$$

Newton's method replaces the scalar learning rate with the inverse Hessian. This automatically adjusts the step size and direction based on local curvature.



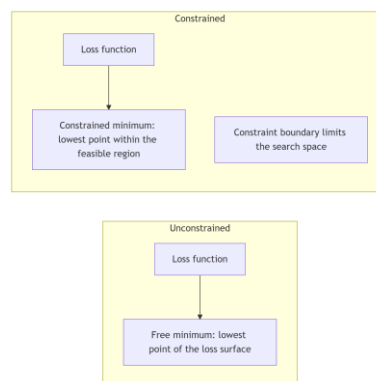
Advantages: - Quadratic convergence near the minimum (error squares each step) - No learning rate to tune - Scale-invariant (works regardless of how you parameterize the problem)

Disadvantages: - Computing the Hessian costs $O(n^2)$ memory and $O(n^3)$ to invert - For a neural network with 1 million weights, that is 10^{12} entries and 10^{18} operations - Not practical for deep learning

Constrained optimization

Unconstrained optimization: minimize $f(x)$ over all x . Constrained optimization: minimize $f(x)$ subject to constraints.

Real problems have constraints. You want to minimize cost but your budget is limited. You want to minimize error but your model complexity is bounded.



Lagrange multipliers

The method of Lagrange multipliers converts a constrained problem into an unconstrained one.

Problem: minimize $f(x)$ subject to $g(x) = 0$.

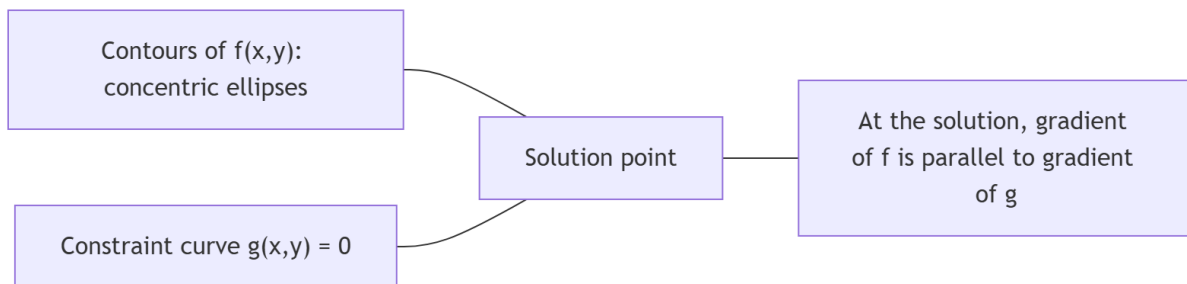
Solution: introduce a new variable (the Lagrange multiplier λ) and solve the unconstrained problem:

$$L(x, \lambda) = f(x) + \lambda * g(x)$$

At the solution, the gradient of L is zero:

$$\begin{aligned} dL/dx &= df/dx + \lambda * dg/dx = 0 \\ dL/d\lambda &= g(x) = 0 \end{aligned}$$

Geometric intuition: at the constrained minimum, the gradient of f must be parallel to the gradient of the constraint g . If they were not parallel, you could move along the constraint surface and reduce f further.



Example: minimize $f(x,y) = x^2 + y^2$ subject to $x + y = 1$.

$$\begin{aligned} L &= x^2 + y^2 + \lambda(x + y - 1) \\ dL/dx &= 2x + \lambda = 0 \Rightarrow x = -\lambda/2 \\ dL/dy &= 2y + \lambda = 0 \Rightarrow y = -\lambda/2 \\ dL/d\lambda &= x + y - 1 = 0 \\ \text{From first two: } x &= y \\ \text{Substituting: } 2x &= 1, \text{ so } x = y = 0.5, \lambda = -1 \end{aligned}$$

The closest point on the line $x + y = 1$ to the origin is $(0.5, 0.5)$.

KKT conditions

The Karush-Kuhn-Tucker conditions extend Lagrange multipliers to inequality constraints.

Problem: minimize $f(x)$ subject to $g_i(x) \leq 0$ for $i = 1, \dots, m$.

The KKT conditions (necessary for optimality):

1. Stationarity: $df/dx + \sum(\lambda_i * dg_i/dx) = 0$
2. Primal feasibility: $g_i(x) \leq 0$ for all i
3. Dual feasibility: $\lambda_i \geq 0$ for all i
4. Complementary slackness: $\lambda_i * g_i(x) = 0$ for all i

Complementary slackness is the key insight: either the constraint is active ($g_i = 0$, the solution sits on the boundary) or the multiplier is zero (the constraint does not matter). A constraint that does not affect the solution has $\lambda = 0$.

KKT conditions are central to SVMs. The support vectors are the data points where the constraint is active ($\lambda > 0$). All other data points have $\lambda = 0$ and do not affect the decision boundary.

Regularization as constrained optimization

L1 and L2 regularization are not arbitrary tricks. They are constrained optimization problems in disguise.

L2 regularization (Ridge):

```

minimize Loss(w) subject to ||w||^2 <= t

Equivalent unconstrained form:
minimize Loss(w) + lambda * ||w||^2

```

The constraint $||w||^2 \leq t$ defines a ball (circle in 2D, sphere in 3D). The solution is where the loss contours first touch this ball.

L1 regularization (LASSO):

```

minimize Loss(w) subject to ||w||_1 <= t

Equivalent unconstrained form:
minimize Loss(w) + lambda * ||w||_1

```

The constraint $||w||_1 \leq t$ defines a diamond (rotated square in 2D).

Property	L2 constraint (circle)	L1 constraint (diamond)
Constraint shape	Circle (sphere in higher dims)	Diamond (rotated square in 2D)
Where loss contour touches	Smooth boundary — any point on the circle	Corner — aligned with an axis
Solution behavior	Weights are small but nonzero	Some weights are exactly zero (sparse)

Property	L2 constraint (circle)	L1 constraint (diamond)
Result	Weight shrinkage	Feature selection

This explains why L1 produces sparse models (feature selection) while L2 only shrinks weights. The diamond has corners aligned with axes. Loss contours are more likely to touch a corner, setting one or more weights exactly to zero.

Duality

Every constrained optimization problem (the primal) has a companion problem (the dual). For convex problems, the primal and dual have the same optimal value. This is strong duality.

The Lagrangian dual function:

```
Primal: minimize f(x) subject to g(x) <= 0
Lagrangian: L(x, lambda) = f(x) + lambda * g(x)
Dual function: d(lambda) = min_x L(x, lambda)
Dual problem: maximize d(lambda) subject to lambda >= 0
```

Why duality matters: - The dual problem is sometimes easier to solve than the primal - SVMs are solved in their dual form, where the problem depends on dot products between data points (enabling the kernel trick) - The dual provides a lower bound on the primal optimum, useful for checking solution quality

For SVMs specifically:

```
Primal: find w, b that maximize the margin 2/||w|| subject to
        y_i(w^T x_i + b) >= 1 for all i

Dual:   maximize sum(alpha_i) - 0.5 * sum_ij(alpha_i * alpha_j * y_i *
        y_j * x_i^T x_j)
        subject to alpha_i >= 0 and sum(alpha_i * y_i) = 0

The dual only involves dot products x_i^T x_j.
Replace x_i^T x_j with K(x_i, x_j) to get the kernel trick.
```

Why deep learning works despite non-convexity

Neural network loss functions are wildly non-convex. By every classical measure, optimizing them should fail. Yet stochastic gradient descent finds good solutions reliably. Several factors explain this.

Most local minima are good enough. In high-dimensional spaces, random critical points (where the gradient is zero) are overwhelmingly saddle points, not local minima. The few local minima that exist tend to have loss values close to the global minimum. Getting trapped in a terrible local minimum is extremely unlikely when the parameter space has millions of dimensions.

Saddle points, not local minima, are the real obstacle. In a function with n parameters, a saddle point has a mix of positive and negative curvature directions. For a random critical point in high dimensions, the probability of all n eigenvalues being positive (local minimum) is roughly $2^{(-n)}$. Almost all critical points are saddle points. SGD's noise helps escape them.

Overparameterization smooths the landscape. Networks with more parameters than training examples have smoother, more connected loss surfaces. Wider networks have fewer bad local minima. This is counterintuitive but empirically consistent.

Loss landscape structure:

Property	Low-dimensional space	High-dimensional space
Landscape	Many isolated peaks and valleys	Smoothly connected valleys
Minima	Many isolated local minima	Few bad local minima; most are near-optimal
Navigation	Hard to find global minimum	Many paths lead to good solutions
Critical points	Mix of local minima and saddle points	Overwhelmingly saddle points, not local minima

Stochastic noise acts as implicit regularization. Mini-batch SGD adds noise that prevents settling into sharp minima. Sharp minima overfit; flat minima generalize. The noise biases optimization toward flat regions of the loss landscape.

Second-order methods in practice

Pure Newton's method is impractical for large models. Several approximations make second-order information usable.

L-BFGS (Limited-memory BFGS): Approximates the inverse Hessian using the last m gradient differences. Requires $O(mn)$ memory instead of $O(n^2)$. Works well for problems with up to $\sim 10,000$ parameters. Used in classical ML (logistic regression, CRFs) but not deep learning.

Natural gradient: Uses the Fisher information matrix (expected Hessian of the log-likelihood) instead of the standard Hessian. This accounts for the geometry of probability distributions. K-FAC (Kronecker-Factored Approximate Curvature) approximates the Fisher matrix as a Kronecker product, making it practical for neural networks.

Hessian-free optimization: Uses conjugate gradient to solve $Hx = g$ without ever forming H . Only requires Hessian-vector products, which can be computed in $O(n)$ time via automatic differentiation.

Diagonal approximations: Adam's second moment is a diagonal approximation of the Hessian's diagonal. AdaHessian extends this by using actual Hessian diagonal elements via Hutchinson's estimator.

Method	Memory	Per-step cost	When to use
Gradient descent	$O(n)$	$O(n)$	Baseline, large models
Newton's method	$O(n^2)$	$O(n^3)$	Small convex problems
L-BFGS	$O(mn)$	$O(mn)$	Medium convex problems
Adam	$O(n)$	$O(n)$	Deep learning default
K-FAC	$O(n)$	$O(n)$ per layer	Research, large-batch training

Build It

Step 1: Convexity checker

Build a function that tests convexity empirically by sampling points and checking the definition.

```
import random
import math

def check_convexity(f, dim, bounds=(-5, 5), samples=1000):
    violations = 0
    for _ in range(samples):
        x = [random.uniform(*bounds) for _ in range(dim)]
        y = [random.uniform(*bounds) for _ in range(dim)]
        t = random.uniform(0, 1)
        mid = [t * xi + (1 - t) * yi for xi, yi in zip(x, y)]
        lhs = f(mid)
        rhs = t * f(x) + (1 - t) * f(y)
        if lhs > rhs + 1e-10:
            violations += 1
    return violations == 0, violations
```

Step 2: Newton's method for 2D

Implement Newton's method using an explicit Hessian. Compare convergence speed against gradient descent.

```
def newtons_method(f, grad_f, hessian_f, x0, steps=50, tol=1e-12):
    x = list(x0)
    history = [x[:]]
    for _ in range(steps):
        g = grad_f(x)
        H = hessian_f(x)
        det = H[0][0] * H[1][1] - H[0][1] * H[1][0]
        if abs(det) < 1e-15:
            break
```

```

H_inv = [
    [H[1][1] / det, -H[0][1] / det],
    [-H[1][0] / det, H[0][0] / det],
]
dx = [
    H_inv[0][0] * g[0] + H_inv[0][1] * g[1],
    H_inv[1][0] * g[0] + H_inv[1][1] * g[1],
]
x = [x[0] - dx[0], x[1] - dx[1]]
history.append(x[:])
if sum(gi ** 2 for gi in g) < tol:
    break
return history

```

Step 3: Lagrange multiplier solver

Solve constrained optimization using gradient descent on the Lagrangian.

```

def lagrange_solve(f_grad, g_val, g_grad, x0, lr=0.01,
                  lr_lambda=0.01, steps=5000):
    x = list(x0)
    lam = 0.0
    history = []
    for _ in range(steps):
        fg = f_grad(x)
        gv = g_val(x)
        gg = g_grad(x)
        x = [
            xi - lr * (fgi + lam * ggi)
            for xi, fgi, ggi in zip(x, fg, gg)
        ]
        lam = lam + lr_lambda * gv
        history.append((x[:], lam, gv))
    return history

```

Step 4: Compare first-order vs second-order

Run gradient descent and Newton's method on the same quadratic function. Count the steps to convergence.

```

def quadratic(x):
    return 5 * x[0] ** 2 + x[1] ** 2

def quadratic_grad(x):
    return [10 * x[0], 2 * x[1]]

def quadratic_hessian(x):
    return [[10, 0], [0, 2]]

```

Newton's method will converge in 1 step (it is exact for quadratics). Gradient descent will take hundreds of steps because the eigenvalues of the Hessian differ by a factor of 5, creating an elongated valley.

Use It

Convexity analysis applies directly when choosing ML models and solvers.

For convex problems (logistic regression, SVMs, LASSO): - Use dedicated solvers (liblinear, CVXPY, `scipy.optimize.minimize` with `method='L-BFGS-B'`) - Expect a unique global solution - Second-order methods are practical and fast

For non-convex problems (neural networks): - Use first-order methods (SGD, Adam) - Accept that the solution depends on initialization and randomness - Use overparameterization, noise, and learning rate schedules as implicit regularization - Do not waste time searching for the global minimum. A good local minimum is sufficient.

```
from scipy.optimize import minimize

result = minimize(
    fun=lambda w: sum((y - X @ w) ** 2) + 0.1 * sum(w ** 2),
    x0=np.zeros(d),
    method='L-BFGS-B',
    jac=lambda w: -2 * X.T @ (y - X @ w) + 0.2 * w,
)
```

For SVMs, the dual formulation lets you use the kernel trick:

```
from sklearn.svm import SVC

svm = SVC(kernel='rbf', C=1.0)
svm.fit(X_train, y_train)
print(f"Support vectors: {svm.n_support_}")
```

Exercises

1. **Convexity gallery.** Test these functions for convexity using the checker: $f(x) = x^4$, $f(x) = \sin(x)$, $f(x,y) = x^2 + y^2$, $f(x,y) = x*y$, $f(x) = \max(x, 0)$. Explain why each result makes sense.
2. **Newton vs gradient descent race.** Run both methods on $f(x,y) = 50*x^2 + y^2$ from the starting point (10, 10). How many steps does each need to reach $\text{loss} < 1e-10$? What happens to gradient descent when the condition number (ratio of largest to smallest Hessian eigenvalue) increases?
3. **Lagrange multiplier geometry.** Minimize $f(x,y) = (x-3)^2 + (y-3)^2$ subject to $x + 2y = 4$. Verify the solution by checking that the gradient of f is parallel to the gradient of g at the solution.
4. **Regularization constraint.** Implement L1-constrained optimization: minimize $(x-3)^2 + (y-2)^2$ subject to $|x| + |y| \leq 1$. Show that the solution has one coordinate equal to zero (sparsity from the diamond constraint).

5. **Hessian eigenvalue analysis.** Compute the Hessian of the Rosenbrock function at (1,1) and at (-1,1). Compute eigenvalues at both points. What do the eigenvalues tell you about the curvature at the minimum versus far from it?

Key Terms

Term	What it means
Convex set	A set where the line segment between any two points in the set stays inside the set
Convex function	A function where the line between any two points on its graph lies above or on the graph. Equivalently, Hessian is positive semidefinite everywhere
Local minimum	A point lower than all nearby points. For convex functions, every local minimum is the global minimum
Global minimum	The lowest point of a function over its entire domain
Hessian matrix	The matrix of all second partial derivatives. Encodes curvature information
Positive semidefinite	A matrix whose eigenvalues are all non-negative. The multidimensional analogue of “second derivative ≥ 0 ”
Condition number	Ratio of largest to smallest eigenvalue of the Hessian. High condition number means elongated valleys and slow gradient descent
Newton’s method	Second-order optimizer that uses the inverse Hessian to determine step direction and size. Quadratic convergence near the minimum
Lagrange multiplier	A variable introduced to convert a constrained optimization problem into an unconstrained one
KKT conditions	Necessary conditions for optimality with inequality constraints. Generalize Lagrange multipliers
Complementary slackness	At the solution, either a constraint is active or its multiplier is zero. Never both nonzero
Duality	Every constrained problem has a companion dual problem. For convex problems, both have the same optimal value
Strong duality	Primal and dual optimal values are equal. Holds for convex problems satisfying Slater’s condition
L-BFGS	Approximate second-order method that stores the last m gradient differences instead of the full Hessian

Term	What it means
Saddle point	A point where the gradient is zero but it is a minimum in some directions and a maximum in others
Overparameterization	Using more parameters than training examples. Smooths the loss landscape and reduces bad local minima

Code Appendix: *convex.py*

```

import math
import random

def check_convexity(f, dim, bounds=(-5, 5), samples=2000, label=""):
    violations = 0
    worst_violation = 0.0
    for _ in range(samples):
        x = [random.uniform(*bounds) for _ in range(dim)]
        y = [random.uniform(*bounds) for _ in range(dim)]
        t = random.uniform(0, 1)
        mid = [t * xi + (1 - t) * yi for xi, yi in zip(x, y)]
        lhs = f(mid)
        rhs = t * f(x) + (1 - t) * f(y)
        gap = lhs - rhs
        if gap > 1e-10:
            violations += 1
            worst_violation = max(worst_violation, gap)
    is_convex = violations == 0
    status = "CONVEX" if is_convex else "NOT CONVEX"
    if label:
        print(f" {label:30s} {status:10s} violations: {violations}/{samples}"
              + (f" worst: {worst_violation:.6f}" if violations > 0 else ""))
    return is_convex, violations

def hessian_eigenvalues_2d(H):
    a, b = H[0][0], H[0][1]
    c, d = H[1][0], H[1][1]
    trace = a + d
    det = a * d - b * c
    discriminant = trace ** 2 - 4 * det
    if discriminant < 0:
        return None, None
    sqrt_disc = math.sqrt(discriminant)
    e1 = (trace + sqrt_disc) / 2
    e2 = (trace - sqrt_disc) / 2
    return e1, e2

```

```

def is_positive_semidefinite_2d(H):
    e1, e2 = hessian_eigenvalues_2d(H)
    if e1 is None:
        return False
    return e1 >= -1e-10 and e2 >= -1e-10

def invert_2x2(H):
    det = H[0][0] * H[1][1] - H[0][1] * H[1][0]
    if abs(det) < 1e-15:
        return None
    return [
        [H[1][1] / det, -H[0][1] / det],
        [-H[1][0] / det, H[0][0] / det],
    ]

def mat_vec_2d(M, v):
    return [
        M[0][0] * v[0] + M[0][1] * v[1],
        M[1][0] * v[0] + M[1][1] * v[1],
    ]

class GradientDescent:
    def __init__(self, lr=0.001):
        self.lr = lr

    def step(self, params, grads):
        return [p - self.lr * g for p, g in zip(params, grads)]

def optimize_gd(grad_f, x0, lr=0.01, steps=1000, tol=1e-12):
    x = list(x0)
    history = [x[:]]
    for _ in range(steps):
        g = grad_f(x)
        if sum(gi ** 2 for gi in g) < tol:
            break
        x = [xi - lr * gi for xi, gi in zip(x, g)]
        if any(math.isnan(xi) or math.isinf(xi) for xi in x):
            break
        history.append(x[:])
    return history

def newtons_method(grad_f, hessian_f, x0, steps=100, tol=1e-12):
    x = list(x0)
    history = [x[:]]
    for _ in range(steps):
        g = grad_f(x)
        if sum(gi ** 2 for gi in g) < tol:
            break

```

```

    H = hessian_f(x)
    H_inv = invert_2x2(H)
    if H_inv is None:
        break
    dx = mat_vec_2d(H_inv, g)
    x = [x[0] - dx[0], x[1] - dx[1]]
    if any(math.isnan(xi) or math.isinf(xi) for xi in x):
        break
    history.append(x[:])
return history

def lagrange_solve(f_grad, g_val, g_grad, x0, lr=0.01,
                   lr_lambda=0.01, steps=5000):
    x = list(x0)
    lam = 0.0
    history = []
    for _ in range(steps):
        fg = f_grad(x)
        gv = g_val(x)
        gg = g_grad(x)
        x = [
            xi - lr * (fgi + lam * ggi)
            for xi, fgi, ggi in zip(x, fg, gg)
        ]
        lam = lam + lr_lambda * gv
        if any(math.isnan(xi) or math.isinf(xi) for xi in x):
            break
        history.append((x[:], lam, gv))
    return history

def demo_convexity_checker():
    print("=" * 65)
    print("  CONVEXITY CHECKER")
    print("=" * 65)
    print()

    random.seed(42)

    check_convexity(lambda x: x[0] ** 2, 1, label="f(x) = x^2")
    check_convexity(lambda x: abs(x[0]), 1, label="f(x) = |x|")
    check_convexity(lambda x: math.exp(x[0]), 1, label="f(x) = e^x")
    check_convexity(lambda x: x[0] ** 2 + x[1] ** 2, 2, label="f(x,y) = x^2 + y^2")
    check_convexity(lambda x: max(x[0], 0), 1, label="f(x) = max(0, x) [ReLU]")

    check_convexity(lambda x: math.sin(x[0]), 1, label="f(x) = sin(x)")
    check_convexity(lambda x: x[0] ** 3, 1, label="f(x) = x^3")
    check_convexity(lambda x: -x[0] ** 2, 1, label="f(x) = -x^2")
    check_convexity(
        lambda x: math.sin(x[0]) * math.cos(x[1]),
        2,
        label="f(x,y) = sin(x)*cos(y)"
    )

```



```

)
check_convexity(
    lambda x: x[0] ** 2 - x[1] ** 2,
    2,
    label="f(x,y) = x^2 - y^2 [saddle]"
)

print()
print(" Top group: expected convex. Bottom group: expected non-convex.")

def demo_hessian_analysis():
    print()
    print()
    print("=" * 65)
    print(" HESSIAN ANALYSIS AND CURVATURE")
    print("=" * 65)

    print()
    print(" f(x,y) = 5x^2 + y^2 (elongated bowl)")
    H1 = [[10, 0], [0, 2]]
    e1, e2 = hessian_eigenvalues_2d(H1)
    psd = is_positive_semidefinite_2d(H1)
    print(f" Hessian: [{H1[0][0]}, {H1[0][1]}, [{H1[1][0]}, {H1[1][1]}]")
    print(f" Eigenvalues: {e1:.1f}, {e2:.1f}")
    print(f" Condition number: {e1 / e2:.1f}")
    print(f" Positive semidefinite: {psd}")
    print(f" Convex: {psd}")

    print()
    print(" f(x,y) = x^2 - y^2 (saddle)")
    H2 = [[2, 0], [0, -2]]
    e1, e2 = hessian_eigenvalues_2d(H2)
    psd = is_positive_semidefinite_2d(H2)
    print(f" Hessian: [{H2[0][0]}, {H2[0][1]}, [{H2[1][0]}, {H2[1][1]}]")
    print(f" Eigenvalues: {e1:.1f}, {e2:.1f}")
    print(f" Positive semidefinite: {psd}")
    print(f" Saddle point: mixed signs confirm saddle")

    print()
    print(" f(x,y) = x^2 + 3xy + y^2")
    H3 = [[2, 3], [3, 2]]
    e1, e2 = hessian_eigenvalues_2d(H3)
    psd = is_positive_semidefinite_2d(H3)
    print(f" Hessian: [{H3[0][0]}, {H3[0][1]}, [{H3[1][0]}, {H3[1][1]}]")
    print(f" Eigenvalues: {e1:.1f}, {e2:.1f}")
    print(f" Positive semidefinite: {psd}")
    print(f" Convex: {psd} (negative eigenvalue means indefinite)")

    print()

```

```

print(" Rosenbrock at minimum (1, 1)")
Hmin = [[802, -400], [-400, 200]]
e1, e2 = hessian_eigenvalues_2d(Hmin)
psd = is_positive_semidefinite_2d(Hmin)
print(f" Hessian: [{Hmin[0][0]}, {Hmin[0][1]}], [{Hmin[1][0]}, {Hmin[1][1]}]")
print(f" Eigenvalues: {e1:.2f}, {e2:.2f}")
print(f" Condition number: {e1 / e2:.1f}")
print(f" Positive semidefinite at (1,1): {psd}")

def demo_newtons_method():
    print()
    print()
    print("=" * 65)
    print(" NEWTON'S METHOD vs GRADIENT DESCENT")
    print("=" * 65)

    def f(x):
        return 50 * x[0] ** 2 + x[1] ** 2

    def grad_f(x):
        return [100 * x[0], 2 * x[1]]

    def hessian_f(x):
        return [[100, 0], [0, 2]]

    start = [10.0, 10.0]

    print()
    print(f" Function: f(x,y) = 50x^2 + y^2")
    print(f" Minimum at: (0, 0), f = 0")
    print(f" Starting point: ({start[0]}, {start[1]}), f = {f(start):.1f}")
    print(f" Condition number: {100 / 2:.0f} (elongated valley)")

    newton_hist = newtons_method(grad_f, hessian_f, start, steps=50)
    gd_hist = optimize_gd(grad_f, start, lr=0.015, steps=500)

    print()
    print(f" Newton's method: {len(newton_hist) - 1} steps to converge")
    print(f" {'Step':>6s} {'x':>12s} {'y':>12s} {'f(x,y)':>14s}")
    print(f" {'-' * 48}")
    for i, p in enumerate(newton_hist):
        print(f" {i:6d} {p[0]:12.8f} {p[1]:12.8f} {f(p):14.8f}")

    print()
    threshold = 1e-10
    gd_converged = len(gd_hist) - 1
    for i, p in enumerate(gd_hist):
        if f(p) < threshold:
            gd_converged = i
            break

```

```

print(f" Gradient descent (lr=0.015): {len(gd_hist) - 1} steps taken")
steps_to_show = [0, 1, 5, 10, 25, 50, 100, 200, 300, 400, 499]
steps_to_show = [s for s in steps_to_show if s < len(gd_hist)]
print(f" {'Step':>6s} {'x':>12s} {'y':>12s} {'f(x,y)':>14s}")
print(f" {'-' * 48}")
for i in steps_to_show:
    p = gd_hist[i]
    print(f" {i:6d} {p[0]:12.8f} {p[1]:12.8f} {f(p):14.8f}")

print()
print(f" Newton converged in {len(newton_hist) - 1} step(s)")
print(f" GD reached f < {threshold} at step {gd_converged}"
      + (" (did not converge)" if gd_converged == len(gd_hist) - 1 else ""))
print()
print(" Newton's method is exact for quadratic functions.")
print(" GD struggles with high condition number (elongated valleys).")

def demo_condition_number_effect():
    print()
    print()
    print("=" * 65)
    print(" CONDITION NUMBER EFFECT ON GRADIENT DESCENT")
    print("=" * 65)
    print()

    conditions = [1, 5, 10, 50, 100]

    print(f" Minimizing f(x,y) = c*x^2 + y^2, start = (10, 10)")
    print(f" Newton always converges in 1 step for any condition number.")
    print()
    print(f" {'Cond #':>8s} {'GD steps':>10s} {'Newton steps':>14s} {'GD final loss':>14s}")
    print(f" {'-' * 50}")

    for c in conditions:
        def grad_f(x, c=c):
            return [2 * c * x[0], 2 * x[1]]

        def hess_f(x, c=c):
            return [[2 * c, 0], [0, 2]]

        def f_val(x, c=c):
            return c * x[0] ** 2 + x[1] ** 2

        start = [10.0, 10.0]
        lr = 0.9 / (2 * c)

        gd_hist = optimize_gd(grad_f, start, lr=lr, steps=2000)
        newton_hist = newtons_method(grad_f, hess_f, start, steps=50)

```

```

gd_steps = len(gd_hist) - 1
newton_steps = len(newton_hist) - 1
gd_final = f_val(gd_hist[-1])

print(f" {c:8d} {gd_steps:10d} {newton_steps:14d} {gd_final:14.2e}")

def demo_lagrange_multipliers():
    print()
    print()
    print("=" * 65)
    print(" LAGRANGE MULTIPLIER SOLVER")
    print("=" * 65)

    print()
    print(" Problem: minimize f(x,y) = x^2 + y^2")
    print(" Subject to: g(x,y) = x + y - 1 = 0")
    print(" Analytical solution: x = 0.5, y = 0.5, lambda = -1")
    print()

    def f_grad(x):
        return [2 * x[0], 2 * x[1]]

    def g_val(x):
        return x[0] + x[1] - 1

    def g_grad(x):
        return [1.0, 1.0]

    history = lagrange_solve(f_grad, g_val, g_grad, [2.0, 2.0],
                             lr=0.01, lr_lambda=0.01, steps=5000)

    milestones = [0, 49, 499, 999, 2499, 4999]
    milestones = [m for m in milestones if m < len(history)]

    print(f" {'Step':>6s} {'x':>8s} {'y':>8s} {'lambda':>8s} {'g(x,y)':>10s}
} {'f(x,y)':>10s}")
    print(f" {'-' * 56}")

    for i in milestones:
        x, lam, gv = history[i]
        fv = x[0] ** 2 + x[1] ** 2
        print(f" {i + 1:6d} {x[0]:8.4f} {x[1]:8.4f} {lam:8.4f} {gv:10.6f}
{fv:10.6f}")

    final_x, final_lam, final_g = history[-1]
    print()
    print(f" Final: x = {final_x[0]:.6f}, y = {final_x[1]:.6f}")
    print(f" lambda = {final_lam:.6f}")
    print(f" Constraint violation: {abs(final_g):.2e}")

```

```

print(f" Objective value: {final_x[0] ** 2 + final_x[1] ** 2:.6f}")

print()
print()
print(" Problem: minimize f(x,y) = (x-3)^2 + (y-3)^2")
print(" Subject to: x + 2y = 4")
print()

def f_grad2(x):
    return [2 * (x[0] - 3), 2 * (x[1] - 3)]

def g_val2(x):
    return x[0] + 2 * x[1] - 4

def g_grad2(x):
    return [1.0, 2.0]

history2 = lagrange_solve(f_grad2, g_val2, g_grad2, [0.0, 0.0],
                          lr=0.002, lr_lambda=0.002, steps=20000)

final_x2, final_lam2, final_g2 = history2[-1]
print(f" Solution: x = {final_x2[0]:.6f}, y = {final_x2[1]:.6f}")
print(f" lambda = {final_lam2:.6f}")
print(f" Constraint x + 2y = {final_x2[0] + 2 * final_x2[1]:.6f} (target: 4)")
print(f" Objective: {(final_x2[0] - 3) ** 2 + (final_x2[1] - 3) ** 2:.6f}")

x_exact = 2.0
y_exact = 1.0
print()
print(f" Analytical: x = 2, y = 1, lambda = 2")
print(f" Error: {math.sqrt((final_x2[0] - x_exact) ** 2 + (final_x2[1] - y_exact)
** 2):.2e}")

def demo_regularization_geometry():
    print()
    print()
    print("=" * 65)
    print(" REGULARIZATION AS CONSTRAINED OPTIMIZATION")
    print("=" * 65)
    print()

    def unconstrained_min():
        return [3.0, 2.0]

    print(" Unconstrained minimum of (x-3)^2 + (y-2)^2: (3, 2)")
    print()

    print(" L2 constraint: x^2 + y^2 <= 1 (unit circle)")
    x_l2 = [3.0 / math.sqrt(13), 2.0 / math.sqrt(13)]
    print(f" Projected solution: ({x_l2[0]:.6f}, {x_l2[1]:.6f})")

```

```

print(f" ||w||^2 = {x_l2[0] ** 2 + x_l2[1] ** 2:.6f}")
print(f" Both weights nonzero: weights shrink but none eliminated")
print()

print(" L1 constraint: |x| + |y| <= 1 (unit diamond)")
print(" Solution sits at a corner of the diamond.")

best_val = float('inf')
best_x = None

for x_cand in [i * 0.001 for i in range(1001)]:
    y_cand = 1.0 - x_cand
    val = (x_cand - 3) ** 2 + (y_cand - 2) ** 2
    if val < best_val:
        best_val = val
        best_x = [x_cand, y_cand]

for y_cand_raw in [i * 0.001 for i in range(1001)]:
    x_cand = 1.0 - y_cand_raw
    val = (x_cand - 3) ** 2 + (y_cand_raw - 2) ** 2
    if val < best_val:
        best_val = val
        best_x = [x_cand, y_cand_raw]

corner_vals = [
    ([1.0, 0.0], (1 - 3) ** 2 + (0 - 2) ** 2),
    ([0.0, 1.0], (0 - 3) ** 2 + (1 - 2) ** 2),
    ([-1.0, 0.0], (-1 - 3) ** 2 + (0 - 2) ** 2),
    ([0.0, -1.0], (0 - 3) ** 2 + (-1 - 2) ** 2),
]

print(f" Scanning diamond boundary...")
print(f" Best on edges: ({best_x[0]:.4f}, {best_x[1]:.4f}), "
      f"objective = {best_val:.4f}")
print()
print(f" Diamond corners:")
for pt, val in corner_vals:
    marker = " <-- best corner" if val == min(v for _, v in corner_vals) else ""
    print(f" ({pt[0]:5.1f}, {pt[1]:5.1f}) objective = {val:.1f}{marker}")

print()
print(" L1 pushes solution toward corners (axis-aligned).")
print(" L2 pushes solution toward the nearest point on the circle.")
print(" L1 produces sparsity. L2 produces small but nonzero weights.")

def demo_first_vs_second_order():
    print()
    print()
    print("=" * 65)
    print(" FIRST-ORDER vs SECOND-ORDER: CONVERGENCE SPEED")
    print("=" * 65)

```

```

print()

def rosenbrock(x):
    return (1 - x[0]) ** 2 + 100 * (x[1] - x[0] ** 2) ** 2

def rosenbrock_grad(x):
    dx = -2 * (1 - x[0]) + 200 * (x[1] - x[0] ** 2) * (-2 * x[0])
    dy = 200 * (x[1] - x[0] ** 2)
    return [dx, dy]

def rosenbrock_hessian(x):
    h00 = 2 - 400 * x[1] + 1200 * x[0] ** 2
    h01 = -400 * x[0]
    h10 = -400 * x[0]
    h11 = 200
    return [[h00, h01], [h10, h11]]

start = [0.5, 0.5]

print(f" Rosenbrock function: f(x,y) = (1-x)^2 + 100(y-x^2)^2")
print(f" Minimum at (1, 1), f = 0")
print(f" Start: ({start[0]}, {start[1]}), f = {rosenbrock(start):.4f}")
print()

newton_hist = newtons_method(rosenbrock_grad, rosenbrock_hessian, start, steps=100)

gd_hist = optimize_gd(rosenbrock_grad, start, lr=0.001, steps=10000)

print(f" Newton's method ({len(newton_hist) - 1} steps):")
print(f" {'Step':>6s} {'x':>10s} {'y':>10s} {'f(x,y)':>14s}")
print(f" {'-' * 44}")
for i, p in enumerate(newton_hist[:15]):
    print(f" {i:6d} {p[0]:10.6f} {p[1]:10.6f} {rosenbrock(p):14.8f}")
if len(newton_hist) > 15:
    p = newton_hist[-1]
    print(f" {len(newton_hist) - 1:6d} {p[0]:10.6f} {p[1]:10.6f} {rosenbrock(p):14.8f}")

print()

gd_threshold = 1e-6
gd_converge_step = len(gd_hist) - 1
for i, p in enumerate(gd_hist):
    if rosenbrock(p) < gd_threshold:
        gd_converge_step = i
        break

print(f" Gradient descent (lr=0.001, {len(gd_hist) - 1} steps):")
show_steps = [0, 10, 100, 500, 1000, 2000, 5000, 9999]
show_steps = [s for s in show_steps if s < len(gd_hist)]
print(f" {'Step':>6s} {'x':>10s} {'y':>10s} {'f(x,y)':>14s}")

```

```

print(f" {'-' * 44}")
for i in show_steps:
    p = gd_hist[i]
    print(f" {i:6d} {p[0]:10.6f} {p[1]:10.6f} {rosenbrock(p):14.8f}")

print()
print(f" Newton converged (f < 1e-12) in {len(newton_hist) - 1} steps")
if gd_converge_step < len(gd_hist) - 1:
    print(f" GD converged (f < {gd_threshold}) in {gd_converge_step} steps")
else:
    final_gd = rosenbrock(gd_hist[-1])
    print(f" GD did not reach f < {gd_threshold} in {len(gd_hist) - 1} steps "
          f"(final: {final_gd:.2e})")

print()
print(" Newton uses O(n^3) per step but converges quadratically.")
print(" GD uses O(n) per step but converges linearly.")
print(" For small problems, Newton wins. For millions of parameters, GD wins.")

def demo_convex_vs_nonconvex_landscape():
    print()
    print()
    print("=" * 65)
    print(" CONVEX vs NON-CONVEX: ASCII LANDSCAPE")
    print("=" * 65)
    print()
    print(" Convex: f(x) = x^2")
    print()

    for y_level in range(10, -1, -1):
        threshold = y_level * 2.5
        line = " "
        for x_step in range(-20, 21):
            x = x_step * 0.25
            val = x ** 2
            if abs(val - threshold) < 1.3:
                line += "*"
            elif val < threshold:
                line += " "
            else:
                line += " "
        print(line)

    print(" " + "-" * 41)
    print(" " + " " * 18 + "x=0")
    print(" One valley. Gradient descent always finds the bottom.")

    print()
    print(" Non-convex: f(x) = sin(3x) + 0.1*x^2")
    print()

```



```

for y_level in range(10, -1, -1):
    threshold = -1.0 + y_level * 0.4
    line = " "
    for x_step in range(-25, 26):
        x = x_step * 0.2
        val = math.sin(3 * x) + 0.1 * x ** 2
        if abs(val - threshold) < 0.25:
            line += "*"
        else:
            line += " "
    print(line)

print(" " + "-" * 51)
print(" Multiple valleys. Gradient descent may get stuck.")

```

```

def demo_duality_intuition():
    print()
    print()
    print("=" * 65)
    print(" DUALITY: PRIMAL vs DUAL")
    print("=" * 65)
    print()
    print(" Primal: minimize x^2 + y^2 subject to x + y >= 1")
    print(" Rewrite constraint as: -(x + y - 1) <= 0")
    print()
    print(" Lagrangian: L = x^2 + y^2 + lambda * (1 - x - y)")
    print(" dL/dx = 2x - lambda = 0 => x = lambda/2")
    print(" dL/dy = 2y - lambda = 0 => y = lambda/2")
    print()
    print(" Dual function:")
    print("     d(lambda) = min_x,y [x^2 + y^2 + lambda(1 - x - y)]")
    print("                 = (lambda/2)^2 + (lambda/2)^2 + lambda(1 - lambda)")
    print("                 = lambda^2/2 + lambda - lambda^2")
    print("                 = lambda - lambda^2/2")
    print()
    print(" Dual problem: maximize lambda - lambda^2/2 s.t. lambda >= 0")
    print(" d'(lambda) = 1 - lambda = 0 => lambda* = 1")
    print()

    lam_star = 1.0
    x_star = lam_star / 2
    y_star = lam_star / 2
    primal_val = x_star ** 2 + y_star ** 2
    dual_val = lam_star - lam_star ** 2 / 2

    print(f" Primal solution: x = {x_star}, y = {y_star}")
    print(f" Primal objective: {primal_val}")
    print(f" Dual objective: {dual_val}")
    print(f" Strong duality: primal = dual = {primal_val}")
    print(f" Constraint: x + y = {x_star + y_star} >= 1 (active)")
    print(f" Complementary slackness: lambda * (1 - x - y) = {lam_star * (1 - x_star - y_star)}")

```

```

y_star)}}")

def print_summary():
    print()
    print()
    print("=" * 65)
    print(" SUMMARY")
    print("=" * 65)
    print()
    print(" 1. Convex functions have one valley. Every local min is global.")
    print(" 2. The Hessian encodes curvature. PSD Hessian = convex.")
    print(" 3. Newton's method uses curvature for faster convergence.")
    print(" 4. Lagrange multipliers handle equality constraints.")
    print(" 5. KKT conditions handle inequality constraints.")
    print(" 6. L1 regularization = diamond constraint = sparsity.")
    print(" 7. L2 regularization = circle constraint = weight shrinkage.")
    print(" 8. Duality converts hard primal problems into sometimes-easier duals.")
    print(" 9. Neural networks are non-convex, but overparameterization and")
    print("     stochastic noise make gradient descent work anyway.")
    print()

if __name__ == "__main__":
    demo_convexity_checker()
    demo_hessian_analysis()
    demo_newtons_method()
    demo_condition_number_effect()
    demo_lagrange_multipliers()
    demo_regularization_geometry()
    demo_first_vs_second_order()
    demo_convex_vs_nonconvex_landscape()
    demo_duality_intuition()
    print_summary()

```

Convexity Checker

How to verify whether an optimization problem is convex, and what to do with the answer.

Decision Checklist

1. Is the objective function convex? (Check Hessian positive semi-definiteness or use composition rules.)
2. Are all inequality constraints of the form $g_i(x) \leq 0$ where each g_i is convex?
3. Are all equality constraints affine (linear)?
4. If all three are yes, the problem is convex. Use a convex solver with convergence guarantees.
5. If any are no, the problem is non-convex. Use SGD/Adam and accept local optima.

How to test convexity of a function

Test	Applies to	Method
Second derivative ≥ 0	Scalar functions $f(x)$	Compute $f''(x)$. If $f''(x) \geq 0$ for all x , convex.
Hessian is PSD	Multivariate functions $f(x)$	Compute $H(x)$. If all eigenvalues ≥ 0 everywhere, convex.
Definition test	Any function	Check $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ for sampled x, y, t .
Composition rules	Composed functions	See composition table below.
Restriction to a line	Multivariate f	f is convex iff $g(t) = f(x + tv)$ is convex in t for all x, v .

Composition rules (preserving convexity)

Operation	Result
$f + g$ (both convex)	Convex
$c * f$ ($c > 0$, f convex)	Convex
$\max(f, g)$ (both convex)	Convex
$f(Ax + b)$ where f is convex	Convex
$g(f(x))$ where g is convex non-decreasing and f is convex	Convex
$g(f(x))$ where g is convex non-increasing and f is concave	Convex
sum of convex functions	Convex
pointwise supremum of convex functions	Convex

Common ML objectives: convex or not?

Objective	Convex?	Reason
MSE: $(1/n) \sum (y - Xw)^2$	Yes	Quadratic in w , Hessian = $(2/n) X^T X$ is PSD
Logistic loss: $\sum (\log(1 + \exp(-y_i * w^T x_i)))$	Yes	Sum of convex functions (log-sum-exp family)
Hinge loss: $\sum (\max(0, 1 - y_i * w^T x_i))$	Yes	Max of convex (linear) functions
L2 regularization: $\lambda * w ^2$	Yes	Quadratic, Hessian = $2\lambda I$

Objective	Convex?	Reason
L1 regularization: $\lambda \cdot w _1$	Yes	Sum of absolute values (convex but not differentiable)
Ridge regression: MSE + L2	Yes	Sum of two convex functions
LASSO: MSE + L1	Yes	Sum of two convex functions
Elastic net: MSE + L1 + L2	Yes	Sum of convex functions
SVM (primal): hinge + L2	Yes	Sum of convex functions
Cross-entropy with softmax	Yes (in logits)	Log-sum-exp is convex
Neural network (any loss)	No	Nonlinear activations create non-convex composition
k-means objective	No	Discrete assignment step
Matrix factorization: $ X - UV^T _2$	No	Bilinear in U and V
GAN loss	No	Minimax, non-convex in generator
Contrastive loss (InfoNCE)	No	Log of ratio of exponentials with negative samples

Solver selection based on convexity

Problem type	Solver	Convergence guarantee
Convex, smooth, unconstrained	Gradient descent	$O(1/k)$ to global minimum
Convex, smooth, unconstrained	L-BFGS	Superlinear to global minimum
Convex, smooth, unconstrained	Newton's method	Quadratic near minimum (if Hessian tractable)
Convex, smooth, constrained	Interior point method	Polynomial time
Convex, non-smooth (L1)	Proximal gradient / ISTA	$O(1/k)$ to global minimum
Convex, non-smooth (L1)	ADMM	Flexible, handles constraints
Convex, quadratic	Conjugate gradient	Exact in n steps
Non-convex, smooth	SGD / Adam	Converges to local minimum

Problem type	Solver	Convergence guarantee
Non-convex, smooth	SGD + restarts	Better local minimum on average
Non-convex, smooth	Overparameterize + SGD	Flat minima, good generalization

Common mistakes

- Assuming a problem is convex because the loss function is convex. The loss must be convex in the parameters you are optimizing. Cross-entropy is convex in the logits, but the full neural network mapping from inputs to logits is non-convex.
- Using Newton's method on a non-convex problem. The Hessian may have negative eigenvalues, causing Newton to move toward saddle points or maxima instead of minima.
- Forgetting that L1 regularization makes the objective non-differentiable at zero. Standard gradient descent does not work well. Use proximal gradient descent or subgradient methods.
- Squaring the condition number by forming $A^T A$. If you need to solve a least-squares problem and A is ill-conditioned, use QR or SVD instead of the normal equations.
- Declaring a problem non-convex without checking. Many ML problems (linear models, SVMs, logistic regression) are convex and benefit from stronger solvers.

Quick test: is my problem convex?

1. Write out the objective: minimize $f(w)$ subject to constraints
2. For each term in $f(w)$:
 - Is it quadratic with PSD matrix? -> Convex
 - Is it a norm? -> Convex
 - Is it log-sum-exp? -> Convex
 - Does it involve w nonlinearly (sigmoid(w), $w_1 w_2$)? -> Likely non-convex
3. Are all constraints linear or convex inequalities?
4. If ALL terms are convex and constraints are convex/linear -> problem is convex
5. If ANY term is non-convex -> problem is non-convex

Quiz

Pre-lesson check

1. What is the defining property of a convex function?

- a) It has exactly one critical point
- b) The line segment between any two points on its graph lies above or on the graph **(correct)**
- c) Its derivative is always positive
- d) It can only be defined on positive real numbers
Explanation: A function is convex if for any two points x, y and any t in $[0,1]$: $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$. Geometrically, the chord between any two points on the graph never dips below the graph itself.

2. Which of these ML problems has a convex loss landscape?

- a) Training a 3-layer neural network with ReLU activations
- b) Logistic regression with cross-entropy loss **(correct)**
- c) k-means clustering
- d) Matrix factorization for recommendation
Explanation: Logistic regression has a convex loss (log-loss is convex in the weights). Neural networks, k-means, and matrix factorization are all non-convex. For convex problems, any local minimum is the global minimum.

Post-lesson check

3. Newton's method converges to the minimum of $f(x) = 5x^2 + 3x + 1$ in how many steps?

- a) 1 step (it is exact for quadratic functions) **(correct)**
- b) About 10 steps
- c) About 100 steps
- d) It depends on the learning rate
Explanation: Newton's method fits a local quadratic approximation and jumps to its minimum. For an actual quadratic function, the approximation is exact, so Newton's method converges in a single step regardless of the starting point.

4. In the KKT conditions, what does 'complementary slackness' ($\lambda_i * g_i(x) = 0$) mean?
- a) All constraints must be active at the optimum
 - b) Either a constraint is active ($g_i = 0$) or its multiplier is zero ($\lambda_i = 0$) — an inactive constraint has no effect **(correct)**
 - c) The gradients of all constraints must be orthogonal
 - d) The Lagrangian is always zero at the optimum
Explanation: Complementary slackness means each constraint is either binding ($g_i = 0$, sitting on the boundary) or irrelevant ($\lambda_i = 0$, not affecting the solution). In SVMs, this is why only support vectors (active constraints with $\lambda_i > 0$) determine the decision boundary.
5. Why does SGD find good solutions in non-convex neural network landscapes despite the lack of convexity guarantees?
- a) Neural networks are secretly convex in high dimensions
 - b) SGD always finds the global minimum
 - c) In high dimensions, most critical points are saddle points (not bad local minima), and SGD noise helps escape them **(correct)**
 - d) The loss function is irrelevant to model performance
Explanation: In high-dimensional parameter spaces, random critical points are overwhelmingly saddle points. The few local minima that exist tend to have loss values close to the global minimum. SGD's stochastic noise helps escape saddle points, and overparameterization smooths the landscape.
-

Complex Numbers for AI

The square root of -1 is not imaginary. It is the key to rotations, frequencies, and half of signal processing.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 (linear algebra, calculus) **Time:** ~60 minutes

Learning Objectives

- Perform complex arithmetic (add, multiply, divide, conjugate) in both rectangular and polar form
- Apply Euler's formula to convert between complex exponentials and trigonometric functions
- Implement the Discrete Fourier Transform using complex roots of unity
- Explain how complex rotations underlie RoPE and sinusoidal positional encodings in transformers

The Problem

You open a paper on Fourier transforms and there is i everywhere. You look at transformer positional encodings and see $\sin$ and $\cos$ at different frequencies – the real and imaginary parts of complex exponentials. You read about quantum computing and find everything expressed in complex vector spaces.

Complex numbers seem abstract. A number system built on the square root of -1 feels like a mathematical trick. But it is not a trick. It is the natural language of rotations and oscillations. Every time something spins, vibrates, or oscillates, complex numbers are the right tool.

Without understanding complex numbers, you cannot understand the Discrete Fourier Transform. You cannot understand FFT. You cannot understand how RoPE (Rotary Position Embedding) works in modern language models. You cannot understand why sinusoidal positional encodings in the original Transformer paper use the frequencies they do.

This lesson builds complex arithmetic from scratch, connects it to geometry, and shows you exactly where complex numbers appear in machine learning.

The Concept

What is a complex number?

A complex number has two parts: a real part and an imaginary part.

$$z = a + bi$$

where:

a is the real part

b is the imaginary part
i is the imaginary unit, defined by $i^2 = -1$

That is it. You extend the number line into a plane. The real numbers sit on one axis. The imaginary numbers sit on the other. Every complex number is a point in this plane.

Complex arithmetic

Addition. Add the real parts together, add the imaginary parts together.

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

Example: $(3 + 2i) + (1 + 4i) = 4 + 6i$

Multiplication. Use the distributive law and remember that $i^2 = -1$.

$$\begin{aligned}(a + bi)(c + di) &= ac + adi + bci + bdi^2 \\ &= ac + adi + bci - bd \\ &= (ac - bd) + (ad + bc)i\end{aligned}$$

Example: $(3 + 2i)(1 + 4i) = 3 + 12i + 2i + 8i^2$
 $= 3 + 14i - 8$
 $= -5 + 14i$

Conjugate. Flip the sign of the imaginary part.

$$\text{conjugate of } (a + bi) = a - bi$$

The product of a complex number and its conjugate is always real:

$$(a + bi)(a - bi) = a^2 + b^2$$

Division. Multiply numerator and denominator by the conjugate of the denominator.

$$(a + bi) / (c + di) = (a + bi)(c - di) / (c^2 + d^2)$$

This eliminates the imaginary part from the denominator, giving you a clean complex number.

The complex plane

The complex plane maps every complex number to a 2D point. The horizontal axis is the real axis, the vertical axis is the imaginary axis.

```
z = 3 + 2i corresponds to the point (3, 2)
z = -1 + 0i corresponds to the point (-1, 0) on the real axis
z = 0 + 4i corresponds to the point (0, 4) on the imaginary axis
```

A complex number is simultaneously a point and a vector from the origin. This dual interpretation is what makes complex numbers useful for geometry.

Polar form

Any point in the plane can be described by its distance from the origin and its angle from the positive real axis.

```
z = r * (cos(theta) + i*sin(theta))

where:
  r = |z| = sqrt(a^2 + b^2)      (magnitude, or modulus)
  theta = atan2(b, a)           (phase, or argument)
```

Rectangular form ($a + bi$) is good for addition. Polar form (r, θ) is good for multiplication.

Multiplication in polar form. Multiply the magnitudes, add the angles.

```
z1 = r1 * e^(i*theta1)
z2 = r2 * e^(i*theta2)

z1 * z2 = (r1 * r2) * e^(i*(theta1 + theta2))
```

This is why complex numbers are perfect for rotations. Multiplying by a complex number with magnitude 1 is a pure rotation.

Euler's formula

The bridge between complex exponentials and trigonometry:

```
e^(i*theta) = cos(theta) + i*sin(theta)
```

This is the most important formula in this lesson. When $\theta = \pi$:

```
e^(i*pi) = cos(pi) + i*sin(pi) = -1 + 0i = -1

Therefore: e^(i*pi) + 1 = 0
```

Five fundamental constants ($e, i, \pi, 1, 0$) linked in one equation.

Why Euler's formula matters for ML

Euler's formula says that $e^{i\theta}$ traces the unit circle as θ varies. At $\theta = 0$, you are at $(1, 0)$. At $\theta = \pi/2$, you are at $(0, 1)$. At $\theta = \pi$, you are at $(-1, 0)$. At $\theta = 3\pi/2$, you are at $(0, -1)$. A full rotation is $\vartheta = 2\pi$.

This means complex exponentials ARE rotations. And rotations are everywhere in signal processing and ML.

Connection to 2D rotations

Multiplying the complex number $(x + yi)$ by $e^{i\theta}$ rotates the point (x, y) by angle θ around the origin.

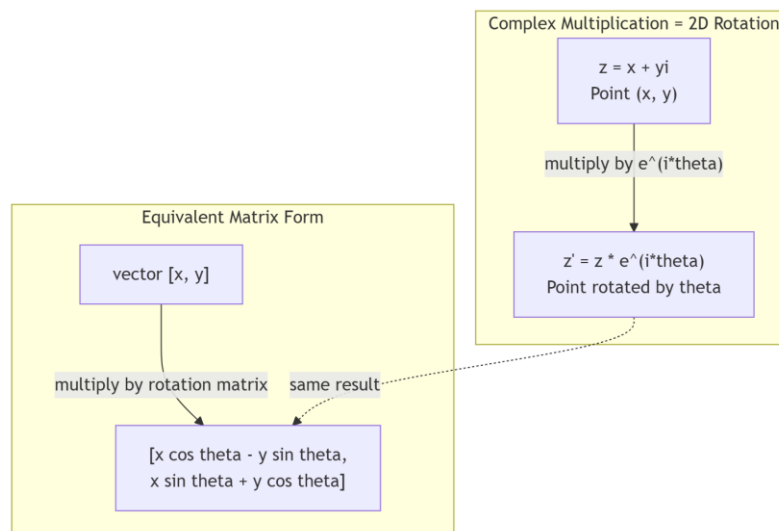
Rotation via complex multiplication:

$$(x + yi) * (\cos(\theta) + i\sin(\theta)) \\ = (x\cos(\theta) - y\sin(\theta)) + (x\sin(\theta) + y\cos(\theta))i$$

Rotation via matrix multiplication:

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos(\theta) - y\sin(\theta) \\ x\sin(\theta) + y\cos(\theta) \end{bmatrix}$$

They produce identical results. Complex multiplication IS 2D rotation. The rotation matrix is just complex multiplication written in matrix notation.



Phasors and rotating signals

A complex exponential $e^{i\omega t}$ is a point rotating around the unit circle at angular frequency ω . As t increases, the point traces the circle.

The real part of this rotating point is $\cos(\omega t)$. The imaginary part is $\sin(\omega t)$. A sinusoidal signal is the shadow of a rotating complex number.

$$e^{(i*\omega*t)} = \cos(\omega*t) + i*\sin(\omega*t)$$

Real part: $\cos(\omega*t)$ -- a cosine wave

Imaginary part: $\sin(\omega*t)$ -- a sine wave

This is the phasor representation. Instead of tracking a wiggly sine wave, you track a smoothly rotating arrow. Phase shifts become angle offsets. Amplitude changes become magnitude changes. Addition of signals becomes vector addition.

Roots of unity

The N-th roots of unity are N points equally spaced on the unit circle:

$$w_k = e^{(2*\pi*i*k/N)} \quad \text{for } k = 0, 1, 2, \dots, N-1$$

For N = 4, the roots are: 1, i, -1, -i (the four compass points). For N = 8, you get the four compass points plus the four diagonals.

Roots of unity are the foundation of the Discrete Fourier Transform. The DFT decomposes a signal into components at these N equally-spaced frequencies.

Connection to the DFT

The Discrete Fourier Transform of a signal $x[0], x[1], \dots, x[N-1]$ is:

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{(-2*\pi*i*k*n/N)}$$

Each $X[k]$ measures how much the signal correlates with the k-th root of unity – a complex sinusoid at frequency k. The DFT breaks a signal into N rotating phasors and tells you the amplitude and phase of each one.

Why i is not imaginary

The word “imaginary” is a historical accident. Descartes used it dismissively. But i is no more imaginary than negative numbers were when people first rejected them. Negative numbers answer “what do you subtract 5 from 3 to get?” The imaginary unit answers “what do you square to get -1?”

More usefully: i is a 90-degree rotation operator. Multiply a real number by i once, you rotate 90 degrees to the imaginary axis. Multiply by i again (i^2), you rotate another 90 degrees – now you are pointing in the negative real direction. That is why $i^2 = -1$. It is not mysterious. It is a half-turn built from two quarter-turns.

This is why complex numbers are everywhere in engineering. Anything that rotates – electromagnetic waves, quantum states, signal oscillations, positional encodings – is naturally described by complex numbers.

Complex exponentials vs trigonometric functions

Before Euler’s formula, engineers wrote signals as $A\cos(\omega t + \phi)$ – amplitude A , frequency ω , phase ϕ . This works but makes arithmetic painful. Adding two cosines with different phases requires trigonometric identities.

With complex exponentials, the same signal is $Ae^{i(\omega t + \phi)}$. Adding two signals is just adding two complex numbers. Multiplying (modulating) is just multiplying magnitudes and adding angles. Phase shifts become angle additions. Frequency shifts become multiplications by phasors.

The entire field of signal processing switched to complex exponential notation because the math is cleaner. The “real signal” is always just the real part of the complex representation. The imaginary part is carried along as bookkeeping, making all the algebra work out naturally.

Connection to transformers

Sinusoidal positional encodings (original Transformer paper):

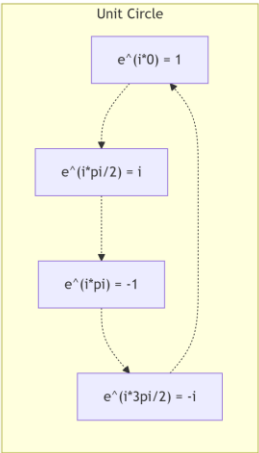
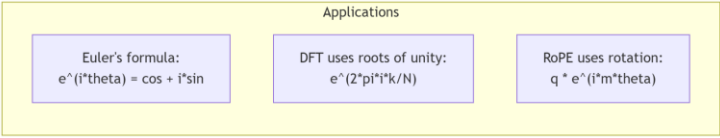
$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d)}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d)}) \end{aligned}$$

The sin and cos pairs are the real and imaginary parts of complex exponentials at different frequencies. Each frequency provides a different “resolution” for encoding position. Low frequencies change slowly (coarse position). High frequencies change quickly (fine position). Together they give each position a unique frequency fingerprint.

RoPE (Rotary Position Embedding) takes this further. It explicitly multiplies query and key vectors by complex rotation matrices. The relative position between two tokens becomes a rotation angle. Attention is computed using these rotated vectors, making the model sensitive to relative position through complex multiplication.

Operation	Algebraic Form	Geometric Meaning
Addition	$(a+c) + (b+d)i$	Vector addition in the plane
Multiplication	$(ac-bd) + (ad+bc)i$	Rotate and scale
Conjugate	$a - bi$	Reflect over real axis
Magnitude	$\sqrt{a^2 + b^2}$	Distance from origin
Phase	$\text{atan2}(b, a)$	Angle from positive real axis
Division	multiply by conjugate	Reverse rotation and rescale

Operation	Algebraic Form	Geometric Meaning
Power	$r^n * e^{in\theta}$	Rotate n times, scale by r^n



Build It

Step 1: Complex class

Build a Complex number class that supports arithmetic, magnitude, phase, and conversion between rectangular and polar forms.

```

import math

class Complex:
    def __init__(self, real, imag=0.0):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        return Complex(self.real + other.real, self.imag + other.imag)

    def __mul__(self, other):
        r = self.real * other.real - self.imag * other.imag
        i = self.real * other.imag + self.imag * other.real
        return Complex(r, i)

    def __truediv__(self, other):
        denom = other.real ** 2 + other.imag ** 2
        r = (self.real * other.real + self.imag * other.imag) / denom

```

```

        i = (self.imag * other.real - self.real * other.imag) / denom
        return Complex(r, i)

    def magnitude(self):
        return math.sqrt(self.real ** 2 + self.imag ** 2)

    def phase(self):
        return math.atan2(self.imag, self.real)

    def conjugate(self):
        return Complex(self.real, -self.imag)

```

Step 2: Polar conversion and Euler's formula

```

def to_polar(z):
    return z.magnitude(), z.phase()

def from_polar(r, theta):
    return Complex(r * math.cos(theta), r * math.sin(theta))

def euler(theta):
    return Complex(math.cos(theta), math.sin(theta))

```

Verify: `euler(0).magnitude()` should always be 1.0. `euler(0)` should give (1, 0). `euler( $\pi$ )` should give (-1, 0).

Step 3: Rotation

Rotating a point (x, y) by angle θ is one complex multiplication:

```

point = Complex(3, 4)
rotated = point * euler(math.pi / 4)

```

The magnitude stays the same. Only the angle changes.

Step 4: DFT from complex arithmetic

```

def dft(signal):
    N = len(signal)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N
            total = total + Complex(signal[n], 0) * euler(angle)
        result.append(total)
    return result

```

This is the $O(N^2)$ DFT. Each output $X[k]$ is the sum of the signal samples multiplied by roots of unity.

Step 5: Inverse DFT

The inverse DFT reconstructs the original signal from its spectrum. The only changes from the forward DFT: flip the sign in the exponent and divide by N.

```
def idft(spectrum):
    N = len(spectrum)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            total = total + spectrum[k] * euler(angle)
        result.append(Complex(total.real / N, total.imag / N))
    return result
```

This gives you perfect reconstruction. Apply DFT, then IDFT, and you get back the original signal to machine precision. No information is lost.

Step 6: Roots of unity

```
def roots_of_unity(N):
    return [euler(2 * math.pi * k / N) for k in range(N)]
```

Verify two properties: - Every root has magnitude exactly 1. - The sum of all N roots is zero (they cancel out by symmetry).

These properties are what make the DFT invertible. The roots of unity form an orthogonal basis for the frequency domain.

Use It

Python has built-in complex number support. The literal `j` represents the imaginary unit.

```
z = 3 + 2j
w = 1 + 4j

print(z + w)
print(z * w)
print(abs(z))

import cmath
print(cmath.phase(z))
print(cmath.exp(1j * cmath.pi))
```

For arrays, numpy handles complex numbers natively:

```
import numpy as np
```



```

z = np.array([1+2j, 3+4j, 5+6j])
print(np.abs(z))
print(np.angle(z))
print(np.conj(z))
print(np.real(z))
print(np.imag(z))

signal = np.sin(2 * np.pi * 5 * np.linspace(0, 1, 128))
spectrum = np.fft.fft(signal)
freqs = np.fft.fftfreq(128, d=1/128)

```

Exercises

1. **Complex arithmetic by hand.** Compute $(2 + 3i) * (4 - i)$ and verify with the code. Then compute $(5 + 2i) / (1 - 3i)$. Draw both results on the complex plane and check that multiplication rotated and scaled the first number.
2. **Rotation sequence.** Start with the point $(1, 0)$. Multiply by $e^{i\pi/6}$ twelve times. Verify that you return to $(1, 0)$ after 12 multiplications. Print the coordinates at each step and confirm they trace a regular 12-gon.
3. **DFT of a known signal.** Create a signal that is the sum of $\sin(2\pi 3t)$ and $0.5\sin(2\pi 7t)$ sampled at 32 points. Run your DFT. Verify that the magnitude spectrum has peaks at frequencies 3 and 7, with the peak at 7 being half the height of the peak at 3.
4. **Roots of unity visualization.** Compute the 8th roots of unity. Verify that they sum to zero. Verify that multiplying any root by the primitive root $e^{2\pi i/8}$ gives the next root.
5. **Rotation matrix equivalence.** For 10 random angles and 10 random points, verify that complex multiplication gives the same result as matrix-vector multiplication with the 2x2 rotation matrix. Print the maximum numerical difference.

Key Terms

Term	What it means
Complex number	A number $a + bi$ where a is the real part, b is the imaginary part, and $i^2 = -1$
Imaginary unit	The number i , defined by $i^2 = -1$. Not imaginary in the philosophical sense – it is a rotation operator
Complex plane	The 2D plane where the x-axis is real and the y-axis is imaginary. Also called the Argand plane
Magnitude (modulus)	The distance from the origin: $\sqrt{a^2 + b^2}$. Written as $ z $
Phase (argument)	The angle from the positive real axis: $\text{atan2}(b, a)$. Written as $\arg(z)$

Term	What it means
Conjugate	The mirror image across the real axis: conjugate of $a + bi$ is $a - bi$
Polar form	Expressing z as $r * e^{i\theta}$ instead of $a + bi$. Makes multiplication easy
Euler's formula	$e^{i\theta} = \cos(\theta) + i\sin(\theta)$. Connects exponentials to trigonometry
Phasor	A rotating complex number $e^{i\omega t}$ representing a sinusoidal signal
Roots of unity	The N complex numbers $e^{2\pi i k/N}$ for $k = 0$ to $N-1$. N equally spaced points on the unit circle
DFT	Discrete Fourier Transform. Decomposes a signal into complex sinusoidal components using roots of unity
RoPE	Rotary Position Embedding. Uses complex multiplication to encode relative position in transformer attention

Code Appendix: `complex_numbers.py`

```

import math
import os

class Complex:
    def __init__(self, real, imag=0.0):
        self.real = float(real)
        self.imag = float(imag)

    def __add__(self, other):
        if isinstance(other, (int, float)):
            other = Complex(other)
        return Complex(self.real + other.real, self.imag + other.imag)

    def __radd__(self, other):
        return self.__add__(Complex(other))

    def __sub__(self, other):
        if isinstance(other, (int, float)):
            other = Complex(other)
        return Complex(self.real - other.real, self.imag - other.imag)

    def __rsub__(self, other):
        return Complex(other - self.real, -self.imag)

    def __mul__(self, other):
        if isinstance(other, (int, float)):
            other = Complex(other)

```

```

    r = self.real * other.real - self.imag * other.imag
    i = self.real * other.imag + self.imag * other.real
    return Complex(r, i)

def __rmul__(self, other):
    return self.__mul__(Complex(other))

def __truediv__(self, other):
    if isinstance(other, (int, float)):
        other = Complex(other)
    denom = other.real ** 2 + other.imag ** 2
    if denom == 0:
        raise ZeroDivisionError("division by zero complex number")
    r = (self.real * other.real + self.imag * other.imag) / denom
    i = (self.imag * other.real - self.real * other.imag) / denom
    return Complex(r, i)

def __neg__(self):
    return Complex(-self.real, -self.imag)

def magnitude(self):
    return math.sqrt(self.real ** 2 + self.imag ** 2)

def phase(self):
    return math.atan2(self.imag, self.real)

def conjugate(self):
    return Complex(self.real, -self.imag)

def __repr__(self):
    if abs(self.imag) < 1e-12:
        return f"{self.real:.6f}"
    sign = "+" if self.imag >= 0 else "-"
    return f"{self.real:.6f} {sign} {abs(self.imag):.6f}i"

def __eq__(self, other):
    if isinstance(other, (int, float)):
        other = Complex(other)
    return (abs(self.real - other.real) < 1e-10 and
            abs(self.imag - other.imag) < 1e-10)

def to_polar(z):
    return z.magnitude(), z.phase()

def from_polar(r, theta):
    return Complex(r * math.cos(theta), r * math.sin(theta))

def euler(theta):
    return Complex(math.cos(theta), math.sin(theta))

```

```

def dft(signal):
    N = len(signal)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N
            xn = signal[n] if isinstance(signal[n], Complex) else Complex(signal[n])
            total = total + xn * euler(angle)
        result.append(total)
    return result

def idft(spectrum):
    N = len(spectrum)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            total = total + spectrum[k] * euler(angle)
        result.append(Complex(total.real / N, total.imag / N))
    return result

def roots_of_unity(N):
    return [euler(2 * math.pi * k / N) for k in range(N)]

def demo_arithmetic():
    print("=" * 65)
    print("  COMPLEX ARITHMETIC")
    print("=" * 65)
    print()

    z1 = Complex(3, 2)
    z2 = Complex(1, 4)

    print(f"  z1 = {z1}")
    print(f"  z2 = {z2}")
    print()

    print(f"  z1 + z2 = {z1 + z2}")
    print(f"  z1 - z2 = {z1 - z2}")
    print(f"  z1 * z2 = {z1 * z2}")
    print(f"  z1 / z2 = {z1 / z2}")
    print()

    print(f"  |z1|      = {z1.magnitude():.6f}")
    print(f"  phase(z1)= {z1.phase():.6f} rad ({math.degrees(z1.phase()):.2f} deg)")

```

```

print(f" conj(z1) = {z1.conjugate()}")
print()

product = z1 * z1.conjugate()
expected = z1.real ** 2 + z1.imag ** 2
print(f" z1 * conj(z1) = {product}")
print(f" a^2 + b^2 = {expected:.6f}")
print(f" Match: {abs(product.real - expected) < 1e-10}")
print()

z3 = Complex(5, 2)
z4 = Complex(1, -3)
quotient = z3 / z4
reconstructed = quotient * z4
print(f" Division check: (5+2i) / (1-3i) = {quotient}")
print(f" Reconstruct: result * (1-3i) = {reconstructed}")
print(f" Match original: {abs(reconstructed.real - 5) < 1e-10 and abs(reconstructed.imag - 2) < 1e-10}")

def demo_polar_conversion():
    print()
    print()
    print("=" * 65)
    print(" POLAR FORM AND CONVERSION")
    print("=" * 65)
    print()

    test_cases = [
        Complex(1, 0),
        Complex(0, 1),
        Complex(-1, 0),
        Complex(0, -1),
        Complex(3, 4),
        Complex(-2, 3),
    ]

    print(f" {'Rectangular':<25s} {'r':>8s} {'theta (deg)':>12s} {'Reconstructed':<25s}")
    print(f" {'-' * 25} {'-' * 8} {'-' * 12} {'-' * 25}")

    for z in test_cases:
        r, theta = to_polar(z)
        z_back = from_polar(r, theta)
        print(f" {str(z):<25s} {r:>8.4f} {math.degrees(theta):>12.2f} {str(z_back):<25s}")

def demo_euler_formula():
    print()

```

```

print()
print("=" * 65)
print(" EULER'S FORMULA:  $e^{i\theta} = \cos(\theta) + i\sin(\theta)$ ")
print("=" * 65)
print()

angles = [0, math.pi / 6, math.pi / 4, math.pi / 3, math.pi / 2,
          math.pi, 3 * math.pi / 2, 2 * math.pi]
labels = ["0", "pi/6", "pi/4", "pi/3", "pi/2", "pi", "3pi/2", "2pi"]

print(f" {'theta':<8s} {'cos(theta)':>12s} {'sin(theta)':>12s} "
      f"{'e^{i*theta}':>25s} {'|e^{i*theta}|':>14s}")
print(f" {'-' * 8} {'-' * 12} {'-' * 12} {'-' * 25} {'-' * 14}")

for label, theta in zip(labels, angles):
    e = euler(theta)
    print(f" {label:<8s} {math.cos(theta):>12.6f} {math.sin(theta):>12.6f} "
          f"{'e^{i*theta}':>23s} {e.magnitude():>14.10f}")

print()
e_pi = euler(math.pi)
result = e_pi + Complex(1, 0)
print(f" Euler's identity:  $e^{i\pi} + 1 = \{result\}$ ")
print(f"  $|e^{i\pi} + 1| = \{result.magnitude():.2e\}$  (should be ~0)")

def demo_rotation():
    print()
    print()
    print("=" * 65)
    print(" ROTATION VIA COMPLEX MULTIPLICATION")
    print("=" * 65)
    print()

    point = Complex(3, 4)
    print(f" Original point: {point}")
    print(f" Magnitude: {point.magnitude():.4f}")
    print(f" Phase: {math.degrees(point.phase()):.2f} deg")
    print()

    rotation_angles = [45, 90, 180, 270, 360]

    print(f" {'Rotation':<12s} {'Result':<30s} {'Magnitude':>10s} {'Phase (deg)':>12s}")
    print(f" {'-' * 12} {'-' * 30} {'-' * 10} {'-' * 12}")

    for deg in rotation_angles:
        rad = math.radians(deg)
        rotated = point * euler(rad)
        r, theta = to_polar(rotated)

```

```

        print(f" {deg:>3d} deg      {str(rotated):<30s} {r:>10.4f} {math.degrees(
theta):>12.2f}")

    print()
    print(" Magnitude is preserved through all rotations.")
    print(" 360 degrees returns to the original point.")
    print()

    print(" Rotation matrix equivalence check:")
    print()

    test_angles = [math.pi / 6, math.pi / 4, math.pi / 3, math.pi / 2, math.pi]
    test_points = [Complex(1, 0), Complex(3, 4), Complex(-2, 5)]

    max_error = 0.0
    for theta in test_angles:
        cos_t = math.cos(theta)
        sin_t = math.sin(theta)
        for p in test_points:
            complex_result = p * euler(theta)
            matrix_x = cos_t * p.real - sin_t * p.imag
            matrix_y = sin_t * p.real + cos_t * p.imag

            err = math.sqrt((complex_result.real - matrix_x) ** 2 +
                            (complex_result.imag - matrix_y) ** 2)
            max_error = max(max_error, err)

    print(f" Max difference between complex multiplication")
    print(f" and rotation matrix: {max_error:.2e}")

def demo_roots_of_unity():
    print()
    print()
    print("=" * 65)
    print(" ROOTS OF UNITY")
    print("=" * 65)
    print()

    for N in [4, 8]:
        roots = roots_of_unity(N)
        print(f" {N}-th roots of unity:")
        print(f" {'k':<4s} {'Root':<30s} {'|root|':>8s}")
        print(f" {'-' * 4} {'-' * 30} {'-' * 8}")

        total = Complex(0, 0)
        for k, root in enumerate(roots):
            total = total + root
            print(f" {k:<4d} {str(root):<30s} {root.magnitude():>8.6f}")

    print(f" Sum of all roots: {total}")

```

```

    print(f" |sum| = {total.magnitude():.2e} (should be ~0)")
    print()

    print(" Roots of unity always sum to zero.")
    print(" Each root has magnitude exactly 1.")

def demo_dft():
    print()
    print()
    print("=" * 65)
    print(" DFT OF A SIMPLE SIGNAL")
    print("=" * 65)
    print()

    N = 32
    freq1 = 3
    freq2 = 7
    amp1 = 1.0
    amp2 = 0.5

    signal = []
    for n in range(N):
        t = n / N
        val = amp1 * math.sin(2 * math.pi * freq1 * t) + amp2 * math.sin(2 * math.pi *
freq2 * t)
        signal.append(val)

    print(f" Signal: {amp1}*sin(2*pi*{freq1}*t) + {amp2}*sin(2*pi*{freq2}*t)")
    print(f" {N} samples")
    print()

    spectrum = dft(signal)

    print(f" {'Freq bin':<10s} {'|X[k]|':>10s} {'Phase (deg)':>12s}")
    print(f" {'-' * 10} {'-' * 10} {'-' * 12}")

    for k in range(N // 2 + 1):
        mag = spectrum[k].magnitude()
        if mag > 0.01:
            phase_deg = math.degrees(spectrum[k].phase())
            print(f" k={k:<6d} {mag:>10.4f} {phase_deg:>12.2f}")

    print()
    print(f" Expected peaks at k={freq1} (amplitude {amp1 * N / 2:.1f})")
    print(f" and k={freq2} (amplitude {amp2 * N / 2:.1f})")
    print()

    reconstructed = idft(spectrum)
    max_err = max(abs(reconstructed[n].real - signal[n]) for n in range(N))
    print(f" IDFT reconstruction error: {max_err:.2e}")

```



```

print(f" Perfect reconstruction: {max_err < 1e-10}")

def demo_phasor():
    print()
    print()
    print("=" * 65)
    print(" PHASORS: ROTATING COMPLEX NUMBERS AS SIGNALS")
    print("=" * 65)
    print()

    omega = 2 * math.pi * 3
    N = 16

    print(f" Phasor: e^{i*3*2*pi*t}, sampled at {N} points")
    print()
    print(f" {'t':>6s} {'Real (cos)':>12s} {'Imag (sin)':>12s} {'Magnitude':>10s
}")
    print(f" {'-' * 6} {'-' * 12} {'-' * 12} {'-' * 10}")

    for n in range(N):
        t = n / N
        phasor = euler(omega * t)
        print(f" {t:>6.3f} {phasor.real:>12.6f} {phasor.imag:>12.6f} {phasor.ma
gnitude():>10.6f}")

    print()
    print(" The real part traces cos(6*pi*t).")
    print(" The imaginary part traces sin(6*pi*t).")
    print(" Magnitude is always 1 -- the phasor stays on the unit circle.")

def demo_positional_encoding():
    print()
    print()
    print("=" * 65)
    print(" TRANSFORMER POSITIONAL ENCODING FREQUENCIES")
    print("=" * 65)
    print()

    d_model = 8
    max_pos = 10

    print(f" d_model = {d_model}, showing first {max_pos} positions")
    print()
    print(f" Frequencies (1/10000^(2i/d)):" )
    freqs = []
    for i in range(d_model // 2):
        freq = 1.0 / (10000 ** (2 * i / d_model))
        freqs.append(freq)
        print(f" dim pair {i}: freq = {freq:.6f}")

```

```

print()
print(f" PE matrix (sin/cos pairs for each position):")
print()

header = " pos"
for i in range(d_model // 2):
    header += f" sin_{i:d} cos_{i:d} "
print(header)
print(f" {'-' * (5 + d_model // 2 * 20)}")

for pos in range(max_pos):
    line = f" {pos:>3d}"
    for i in range(d_model // 2):
        angle = pos * freqs[i]
        line += f" {math.sin(angle):>7.4f} {math.cos(angle):>7.4f}"
    print(line)

print()
print(" Each (sin, cos) pair is the real and imaginary part")
print(" of  $e^{i * pos * freq}$ . Different frequencies give each")
print(" position a unique 'fingerprint' in the complex plane.")

def write_skill_output():
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_path = os.path.join(script_dir, "outputs", "skill-complex-arithmetic.md")
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    try:
        with open(output_path, "w") as f:
            f.write("---\n")
            f.write("name: skill-complex-arithmetic\n")
            f.write("description: Quick reference for complex number operations in ML a\n")
            f.write("nd signal processing contexts\n")
            f.write("phase: 1\n")
            f.write("lesson: 19\n")
            f.write("---\n\n")
            f.write("You are an expert in complex number arithmetic for machine learnin\n")
            f.write("g and signal processing.\n\n")
            f.write("When someone asks about complex numbers, Fourier transforms, rotat\n")
            f.write("ions, or positional encodings:\n\n")
            f.write("1. Identify which representation is best: rectangular ( $a + bi$ ) for\n")
            f.write("addition, polar ( $r * e^{i*theta}$ ) for multiplication and rotation.\n\n")
            f.write("2. Key conversions:\n")
            f.write("    - Rectangular to polar:  $r = \sqrt{a^2 + b^2}$ ,  $theta = \text{atan2}(b, a)$ \n")
            f.write(")\n")
            f.write("    - Polar to rectangular:  $a = r * \cos(theta)$ ,  $b = r * \sin(theta)$ \n")
            f.write("    - Euler's formula:  $e^{i*theta} = \cos(theta) + i * \sin(theta)$ \n\n")
    )

```

```

        f.write("3. Common operations and their geometric meaning:\n")
        f.write("    - Addition: vector addition in the complex plane\n")
        f.write("    - Multiplication: rotate by  $\arg(z_2)$  and scale by  $|z_2|$ \n")
        f.write("    - Conjugate: reflect over the real axis\n")
        f.write("    - Division: reverse rotation and rescale\n\n")
        f.write("4. ML connections:\n")
        f.write("    - DFT uses roots of unity:  $e^{(-2\pi i k n/N)}$ \n")
        f.write("    - Positional encodings: sin/cos pairs are real/imag parts of co
mplex exponentials\n")
        f.write("    - RoPE: explicit complex multiplication for position-dependent
rotation of query/key vectors\n")
        f.write("    - FFT: recursive DFT using symmetry of roots of unity,  $O(N \log
N)$ \n\n")

        f.write("5. Quick checks:\n")
        f.write("    -  $|e^{i\theta}| = 1$  always\n")
        f.write("    -  $z * \text{conj}(z) = |z|^2$  (always real)\n")
        f.write("    - Sum of N-th roots of unity = 0\n")
        f.write("    -  $e^{i\pi} + 1 = 0$  (Euler's identity)\n")
        f.write("    - Multiplying by  $e^{i\theta}$  rotates by  $\theta$  radians\n\n")
        f.write("6. Python quick reference:\n")
        f.write("    - Built-in:  $z = 3+2j$ ,  $\text{abs}(z)$ ,  $z.\text{conjugate}()$ ,  $z.\text{real}$ ,  $z.\text{imag}$ \n"
    )
        f.write("    - cmath:  $\text{cmath.phase}(z)$ ,  $\text{cmath.exp}(1j*\theta)$ ,  $\text{cmath.polar}(z)$ \n"
    )
        f.write("    - numpy:  $\text{np.abs}(z)$ ,  $\text{np.angle}(z)$ ,  $\text{np.conj}(z)$ ,  $\text{np.fft.fft}(\text{signal})$ 
\n")

    print(f"\n Skill output written to {output_path}")
except OSError:
    print("\n Could not write skill output (run from the lesson directory)")

def print_summary():
    print()
    print()
    print("=" * 65)
    print(" SUMMARY")
    print("=" * 65)
    print()
    print(" 1. A complex number  $z = a + bi$  is a point (a, b) in the plane.")
    print(" 2. Multiplication rotates and scales. Division reverses it.")
    print(" 3. Euler's formula:  $e^{i\theta} = \cos(\theta) + i\sin(\theta)$ ." )
    print(" 4. Multiplying by  $e^{i\theta}$  rotates by  $\theta$  radians.")
    print(" 5. Complex multiplication IS 2D rotation (same as rotation matrix).")
    print(" 6. DFT decomposes signals into rotating phasors (roots of unity).")
    print(" 7. Transformer positional encodings are complex exponentials")
    print("    at different frequencies.")
    print(" 8. RoPE uses explicit complex multiplication for position.")
    print()

```

```

if __name__ == "__main__":
    demo_arithmetic()
    demo_polar_conversion()
    demo_euler_formula()
    demo_rotation()
    demo_roots_of_unity()
    demo_dft()
    demo_phasor()
    demo_positional_encoding()
    write_skill_output()
    print_summary()

```

You are an expert in complex number arithmetic for machine learning and signal processing.

When someone asks about complex numbers, Fourier transforms, rotations, or positional encodings:

1. Identify which representation is best: rectangular ($a + bi$) for addition, polar ($r * e^{i\theta}$) for multiplication and rotation.
2. Key conversions:
 - Rectangular to polar: $r = \sqrt{a^2 + b^2}$, $\theta = \text{atan2}(b, a)$
 - Polar to rectangular: $a = r\cos(\theta)$, $b = r\sin(\theta)$
 - Euler's formula: $e^{i\theta} = \cos(\theta) + i\sin(\theta)$
3. Common operations and their geometric meaning:
 - Addition: vector addition in the complex plane
 - Multiplication: rotate by $\arg(z_2)$ and scale by $|z_2|$
 - Conjugate: reflect over the real axis
 - Division: reverse rotation and rescale
4. ML connections:
 - DFT uses roots of unity: $e^{-2\pi i k n / N}$
 - Positional encodings: sin/cos pairs are real/imag parts of complex exponentials
 - RoPE: explicit complex multiplication for position-dependent rotation of query/key vectors
 - FFT: recursive DFT using symmetry of roots of unity, $O(N \log N)$
5. Quick checks:
 - $|e^{i\theta}| = 1$ always
 - $z * \text{conj}(z) = |z|^2$ (always real)

- Sum of N-th roots of unity = 0
 - $e^{i\pi} + 1 = 0$ (Euler's identity)
 - Multiplying by $e^{i\theta}$ rotates by θ radians
6. Python quick reference:
- Built-in: $z = 3+2j$, `abs(z)`, `z.conjugate()`, `z.real`, `z.imag`
 - `cmath`: `cmath.phase(z)`, `cmath.exp(1j*θ)`, `cmath.polar(z)`
 - `numpy`: `np.abs(z)`, `np.angle(z)`, `np.conj(z)`, `np.fft.fft(signal)`

Quiz

Pre-lesson check

1. What is the imaginary unit i defined by?
 - a) $i = \sqrt{2}$
 - b) $i^2 = -1$ **(correct)**
 - c) $i = -1$
 - d) $i^2 = 1$

Explanation: The imaginary unit i is defined by the property $i^2 = -1$. It extends the real number line into a 2D plane. Geometrically, multiplying by i is a 90-degree rotation — two multiplications (i^2) give a 180-degree rotation, which is -1 .
2. What does Euler's formula $e^{i\theta}$ equal?
 - a) $\theta + i$
 - b) $\cos(\theta) + i\sin(\theta)$ **(correct)**
 - c) $\sin(\theta) + \cos(\theta)$
 - d) i^θ

*Explanation: Euler's formula states $e^{i\theta} = \cos(\theta) + i\sin(\theta)$. This connects complex exponentials to trigonometry and shows that $e^{i\theta}$ traces the unit circle as θ varies.**

Post-lesson check

3. What is the result of $(3 + 2i)(1 + 4i)$?

a) $3 + 8i$

b) $4 + 6i$

c) $-5 + 14i$ **(correct)**

d) $5 + 14i$

Explanation: Using FOIL: $(3)(1) + (3)(4i) + (2i)(1) + (2i)(4i) = 3 + 12i + 2i + 8i^2 = 3 + 14i + 8(-1) = 3 + 14i - 8 = -5 + 14i$.

4. Why are complex numbers used in Rotary Position Embedding (RoPE) for transformers?

a) Complex numbers compress the position encoding to use less memory

b) Multiplying query/key vectors by complex rotations encodes relative position as a rotation angle **(correct)**

c) Complex numbers are required by the attention softmax function

d) RoPE uses imaginary numbers to handle negative positions

Explanation: RoPE multiplies query and key vectors by complex rotation matrices $e^{im\theta}$ where m is the position. The relative position between two tokens becomes a rotation angle, and attention naturally becomes sensitive to relative (not absolute) position through complex multiplication.

5. The N-th roots of unity are N equally spaced points on the unit circle. What is their sum?

a) N

b) 1

c) 0 **(correct)**

d) $N/2$

*Explanation: The N roots of unity are $e^{2\pi i k/N}$ for $k = 0, \dots, N-1$. They are symmetrically distributed around the unit circle, so their vector sum cancels out to zero. This symmetry property is what makes the DFT invertible.**

The Fourier Transform

Every signal is a sum of sine waves. The Fourier transform tells you which ones.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04, 19 (complex numbers) **Time:** ~90 minutes

Learning Objectives

- Implement the DFT from scratch and verify it against the $O(N \log N)$ Cooley-Tukey FFT
- Interpret frequency coefficients: extract amplitude, phase, and power spectrum from a signal
- Apply the convolution theorem to perform convolution via FFT multiplication
- Connect Fourier frequency decomposition to transformer positional encodings and CNN convolution layers

The Problem

An audio recording is a sequence of pressure measurements over time. A stock price is a sequence of values over days. An image is a grid of pixel intensities over space. All of these are data in the time domain (or space domain). You see values changing over some index.

But many patterns are invisible in the time domain. Is this audio signal a pure tone or a chord? Does this stock price have a weekly cycle? Does this image have a repeating texture? These questions are about frequency content, and the time domain hides it.

The Fourier transform converts data from the time domain to the frequency domain. It takes a signal and decomposes it into sine waves of different frequencies. Each sine wave has an amplitude (how strong it is) and a phase (where it starts). The Fourier transform tells you both.

This matters for ML because frequency-domain thinking appears everywhere. Convolutional neural networks perform convolution, which is multiplication in the frequency domain. Transformer positional encodings use frequency decomposition to represent position. Audio models (speech recognition, music generation) operate on spectrograms – frequency representations of sound. Time series models look for periodic patterns. Understanding the Fourier transform gives you the vocabulary to work with all of these.

The Concept

The DFT definition

Given N samples $x[0], x[1], \dots, x[N-1]$, the Discrete Fourier Transform produces N frequency coefficients $X[0], X[1], \dots, X[N-1]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{(-2\pi i * k * n / N)}$$

for $k = 0, 1, \dots, N-1$

Each $X[k]$ is a complex number. Its magnitude $|X[k]|$ tells you the amplitude of frequency k . Its phase $\angle(X[k])$ tells you the phase offset of that frequency.

The key insight: $e^{(-2\pi i k n / N)}$ is a rotating phasor at frequency k . The DFT computes the correlation between the signal and each of N equally-spaced frequencies. If the signal contains energy at frequency k , the correlation is large. If not, it is near zero.

What each coefficient means

$X[0]$: the DC component. This is the sum of all samples – proportional to the mean. It represents the constant (zero-frequency) offset of the signal.

$$X[0] = \sum_{n=0}^{N-1} x[n] * e^{0} = \text{sum of all samples}$$

$X[k]$ for $1 \leq k \leq N/2$: positive frequencies. $X[k]$ represents frequency k cycles per N samples. Higher k means higher frequency (faster oscillation).

$X[N/2]$: the Nyquist frequency. The highest frequency you can represent with N samples. Above this, you get aliasing – high frequencies masquerading as low ones.

$X[k]$ for $N/2 < k < N$: negative frequencies. For real-valued signals, $X[N-k] = \text{conj}(X[k])$. The negative frequencies are mirror images of the positive ones. This is why the useful information is in the first $N/2 + 1$ coefficients.

Inverse DFT

The inverse DFT reconstructs the original signal from its frequency coefficients:

$$x[n] = (1/N) * \sum_{k=0}^{N-1} X[k] * e^{(2\pi i k n / N)}$$

for $n = 0, 1, \dots, N-1$

The only differences from the forward DFT: the sign in the exponent is positive (not negative), and there is a $1/N$ normalization factor.

The inverse DFT is perfect reconstruction. No information is lost. You can go from time domain to frequency domain and back without any error. The DFT is a change of basis – it re-expresses the same information in a different coordinate system.

The FFT: making it fast

The DFT as defined above is $O(N^2)$: for each of N output coefficients, you sum over N input samples. For $N = 1$ million, that is 10^{12} operations.

The Fast Fourier Transform (FFT) computes the same result in $O(N \log N)$. For $N = 1$ million, that is about 20 million operations instead of a trillion. This is what makes frequency analysis practical.

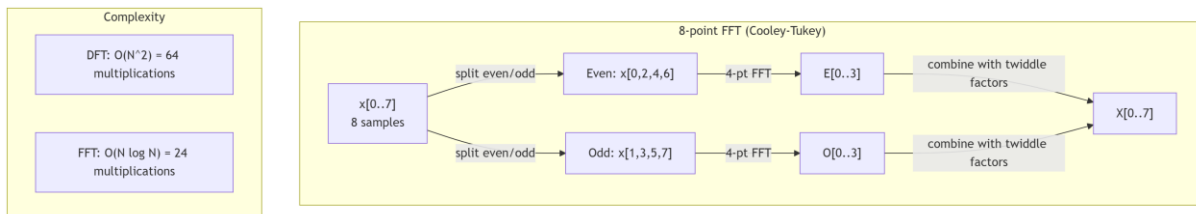
The Cooley-Tukey algorithm (the most common FFT) works by divide and conquer:

1. Split the signal into even-indexed and odd-indexed samples.
2. Compute the DFT of each half recursively.
3. Combine the two half-size DFTs using “twiddle factors” $e^{-2\pi i k/N}$.

$$\begin{aligned} X[k] &= E[k] + e^{-2\pi i k/N} * O[k] & \text{for } k = 0, \dots, N/2 - 1 \\ X[k + N/2] &= E[k] - e^{-2\pi i k/N} * O[k] & \text{for } k = 0, \dots, N/2 - 1 \end{aligned}$$

where E = DFT of even-indexed samples
 O = DFT of odd-indexed samples

The symmetry means each level of recursion does $O(N)$ work, and there are $\log_2(N)$ levels. Total: $O(N \log N)$.



The FFT requires the signal length to be a power of 2. In practice, signals are zero-padded to the next power of 2.

Spectral analysis

The **power spectrum** is $|X[k]|^2$ – the squared magnitude of each frequency coefficient. It shows how much energy is at each frequency.

The **phase spectrum** is $\angle(X[k])$ – the phase offset of each frequency. For most analysis tasks, you care about the power spectrum and ignore the phase.

$$\begin{aligned} \text{Power at frequency } k: & P[k] = |X[k]|^2 = X[k].\text{real}^2 + X[k].\text{imag}^2 \\ \text{Phase at frequency } k: & \phi[k] = \text{atan2}(X[k].\text{imag}, X[k].\text{real}) \end{aligned}$$

Frequency resolution

The frequency resolution of the DFT depends on the number of samples N and the sampling rate f_s .

$$\begin{aligned} \text{Frequency of bin } k: & f_k = k * f_s / N \\ \text{Frequency resolution:} & \Delta f = f_s / N \\ \text{Maximum frequency:} & f_{\text{max}} = f_s / 2 \quad (\text{Nyquist}) \end{aligned}$$

To resolve two frequencies that are close together, you need more samples. To capture high frequencies, you need a higher sampling rate.

The convolution theorem

This is one of the most important results in signal processing and directly relevant to CNNs.

Convolution in the time domain equals pointwise multiplication in the frequency domain.

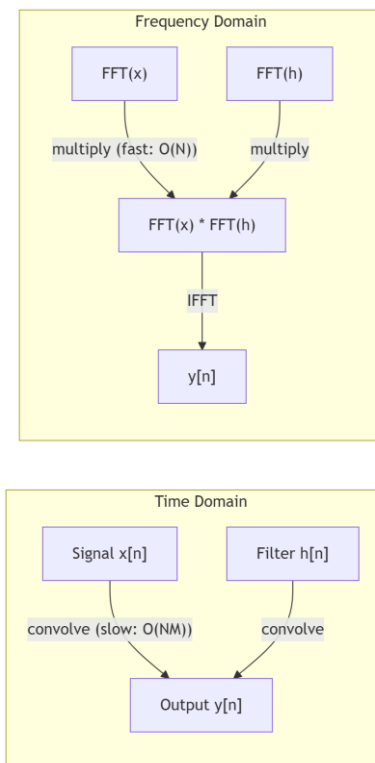
$$x * h = \text{IFFT}(\text{FFT}(x) \cdot \text{FFT}(h))$$

where $*$ is convolution and $\cdot$ is element-wise multiplication

Why this matters:

- Direct convolution of two signals of length N and M takes $O(N \cdot M)$ operations.
- FFT-based convolution takes $O(N \log N)$: transform both, multiply, transform back.
- For large kernels, FFT convolution is dramatically faster.
- This is exactly what happens in convolutional layers with large receptive fields.

Note: the DFT computes circular convolution (the signal wraps around). For linear convolution (no wraparound), zero-pad both signals to length $N + M - 1$ before computing.



Windowing

The DFT assumes the signal is periodic – it treats the N samples as one period of an infinitely repeating signal. If the signal does not start and end at the same value, this creates a discontinuity at the boundary, which shows up as spurious high-frequency content. This is called spectral leakage.

Windowing reduces leakage by tapering the signal to zero at both ends before computing the DFT.

Common windows:

Window	Shape	Main lobe width	Side lobe level	Use case
Rectangular	Flat (no window)	Narrowest	Highest (-13 dB)	When signal is exactly periodic in N samples
Hann	Raised cosine	Moderate	Low (-31 dB)	General purpose spectral analysis
Hamming	Modified cosine	Moderate	Lower (-42 dB)	Audio processing, speech analysis
Blackman	Triple cosine	Wide	Very low (-58 dB)	When side lobe suppression is critical

Hann window: $w[n] = 0.5 * (1 - \cos(2\pi n / (N-1)))$

Hamming window: $w[n] = 0.54 - 0.46 * \cos(2\pi n / (N-1))$

Apply the window by multiplying it element-wise with the signal before the DFT: $X = \text{DFT}(x * w)$.

DFT properties

Property	Time Domain	Frequency Domain
Linearity	$ax + by$	$aX + bY$
Time shift	$x[n - k]$	$X[f] * e^{(-2\pi jfk/N)}$
Frequency shift	$x[n] * e^{(2\pi jf_0n/N)}$	$X[f - f_0]$
Convolution	$x * h$	$X * H$ (pointwise)
Multiplication	$x * h$ (pointwise)	$X * H$ (circular convolution, scaled by $1/N$)
Parseval's theorem	$\sum x[n] ^2$	$(1/N) * \sum X[k] ^2$
Conjugate symmetry (real input)	$x[n]$ real	$X[k] = \text{conj}(X[N-k])$

Parseval's theorem says the total energy is the same in both domains. Energy is conserved through the transform.

Connection to positional encodings

The original Transformer uses sinusoidal positional encodings:

$$\begin{aligned}\text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})\end{aligned}$$

Each dimension pair $(2i, 2i+1)$ oscillates at a different frequency. The frequencies are geometrically spaced from high (dimension 0,1) to low (last dimensions). This gives each position a unique pattern across all frequency bands – similar to how Fourier coefficients uniquely identify a signal.

The key properties this provides:

- **Uniqueness:** No two positions have the same encoding.
- **Bounded values:** sin and cos are always in $[-1, 1]$.
- **Relative position:** The encoding of position $p+k$ can be expressed as a linear function of the encoding at position p . The model can learn to attend to relative positions.

Connection to CNNs

A convolution layer applies a learned filter (kernel) to the input by sliding it across the signal or image. Mathematically, this is the convolution operation.

By the convolution theorem, this is equivalent to: 1. FFT the input 2. FFT the kernel 3. Multiply in frequency domain 4. IFFT the result

Standard CNN implementations use direct convolution (faster for small 3×3 kernels). But for large kernels or global convolution, FFT-based approaches are significantly faster. Some architectures (like FNet) replace attention entirely with FFT, achieving competitive accuracy with $O(N \log N)$ instead of $O(N^2)$ complexity.

Spectrograms and the Short-Time Fourier Transform

A single FFT gives you the frequency content of the entire signal, but tells you nothing about when those frequencies occur. A chirp (a signal whose frequency increases over time) and a chord (all frequencies present simultaneously) can have the same magnitude spectrum.

The Short-Time Fourier Transform (STFT) solves this by computing FFTs on overlapping windows of the signal. The result is a spectrogram: a 2D representation with time on one axis and frequency on the other. The intensity at each point shows the energy at that frequency at that time.

STFT procedure:

1. Choose a window size (e.g., 1024 samples)
2. Choose a hop size (e.g., 256 samples -- 75% overlap)
3. For each window position:
 - a. Extract the windowed segment
 - b. Apply a Hann/Hamming window
 - c. Compute FFT
 - d. Store the magnitude spectrum as one column of the spectrogram

Spectrograms are the standard input representation for audio ML models. Speech recognition models (Whisper, DeepSpeech) operate on mel-spectrograms – spectrograms with frequencies mapped to the mel scale, which better matches human pitch perception.

Aliasing

If a signal contains frequencies above $f_s/2$ (the Nyquist frequency), sampling at rate f_s will create aliased copies. A 90 Hz signal sampled at 100 Hz looks identical to a 10 Hz signal. There is no way to distinguish them from the samples alone.

Example:

True signal: 90 Hz sine wave

Sampling rate: 100 Hz

Apparent frequency: $100 - 90 = 10$ Hz

The samples from the 90 Hz signal at 100 Hz sampling rate are identical to the samples from a 10 Hz signal.

No amount of math can recover the original 90 Hz.

This is why analog-to-digital converters include anti-aliasing filters that remove frequencies above Nyquist before sampling. In ML, aliasing appears when downsampling feature maps without proper low-pass filtering – some architectures address this with anti-aliased pooling layers.

Zero-padding does not increase resolution

A common misconception: zero-padding a signal before FFT improves frequency resolution. It does not. Zero-padding interpolates between existing frequency bins, giving you a smoother-looking spectrum. But it cannot reveal frequency detail that was not present in the original samples.

True frequency resolution depends only on the observation time $T = N / f_s$. To resolve two frequencies separated by Δf , you need at least $T = 1 / \Delta f$ seconds of data. No amount of zero-padding changes this fundamental limit.

Build It

Step 1: DFT from scratch

The $O(N^2)$ DFT follows directly from the definition.

```
import math

class Complex:
    ...

def dft(x):
    N = len(x)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N
            w = Complex(math.cos(angle), math.sin(angle))
            xn = x[n] if isinstance(x[n], Complex) else Complex(x[n])
            total = total + xn * w
        result.append(total)
    return result
```

Step 2: Inverse DFT

Same structure, positive exponent, divide by N.

```
def idft(X):
    N = len(X)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            w = Complex(math.cos(angle), math.sin(angle))
            total = total + X[k] * w
        result.append(Complex(total.real / N, total.imag / N))
    return result
```

Step 3: FFT (Cooley-Tukey)

The recursive FFT requires power-of-2 length. Split into even and odd, recurse, combine with twiddle factors.

```
def fft(x):
    N = len(x)
    if N <= 1:
        return [x[0] if isinstance(x[0], Complex) else Complex(x[0])]
    if N % 2 != 0:
```

```

    return dft(x)

even = fft([x[i] for i in range(0, N, 2)])
odd = fft([x[i] for i in range(1, N, 2)])

result = [Complex(0)] * N
for k in range(N // 2):
    angle = -2 * math.pi * k / N
    twiddle = Complex(math.cos(angle), math.sin(angle))
    t = twiddle * odd[k]
    result[k] = even[k] + t
    result[k + N // 2] = even[k] - t
return result

```

Step 4: Spectral analysis helpers

```

def power_spectrum(X):
    return [xk.real ** 2 + xk.imag ** 2 for xk in X]

def convolve_fft(x, h):
    N = len(x) + len(h) - 1
    padded_N = 1
    while padded_N < N:
        padded_N *= 2

    x_padded = x + [0.0] * (padded_N - len(x))
    h_padded = h + [0.0] * (padded_N - len(h))

    X = fft(x_padded)
    H = fft(h_padded)

    Y = [xk * hk for xk, hk in zip(X, H)]

    y = idft(Y)
    return [y[n].real for n in range(N)]

```

Use It

For real work, use numpy's FFT which is backed by highly optimized C libraries.

```

import numpy as np

signal = np.sin(2 * np.pi * 5 * np.arange(256) / 256)
spectrum = np.fft.fft(signal)
freqs = np.fft.fftfreq(256, d=1/256)

power = np.abs(spectrum) ** 2

positive_freqs = freqs[:len(freqs)//2]
positive_power = power[:len(power)//2]

```


For windowing and more advanced spectral analysis:

```
from scipy.signal import windows, stft

window = windows.hann(256)
windowed = signal * window
spectrum = np.fft.fft(windowed)
```

For convolution:

```
from scipy.signal import fftconvolve

result = fftconvolve(signal, kernel, mode='full')
```

For spectrograms:

```
from scipy.signal import stft

frequencies, times, Zxx = stft(signal, fs=sample_rate, nperseg=256)
spectrogram = np.abs(Zxx) ** 2
```

The spectrogram matrix has shape (n_frequencies, n_time_frames). Each column is the power spectrum at one time window. This is what audio ML models consume as input.

Exercises

1. **Pure tone identification.** Create a signal with a single sine wave at an unknown frequency (between 1 and 50 Hz), sampled at 128 Hz for 1 second. Use your DFT to identify the frequency. Verify the answer matches. Now add Gaussian noise with standard deviation 0.5 and repeat. How does noise affect the spectrum?
2. **FFT vs DFT verification.** Generate a random signal of length 64. Compute both DFT ($O(N^2)$) and FFT. Verify that all coefficients match to within $1e-10$. Time both functions on signals of length 256, 512, 1024, and 2048. Plot the ratio of DFT time to FFT time.
3. **Convolution theorem proof by example.** Create signal $x = [1, 2, 3, 4, 0, 0, 0, 0]$ and filter $h = [1, 1, 1, 0, 0, 0, 0, 0]$. Compute their circular convolution directly (nested loop). Then compute it via FFT (transform, multiply, inverse transform). Verify the results match. Now do linear convolution by zero-padding appropriately.
4. **Windowing effects.** Create a signal that is the sum of two sine waves at 10 Hz and 12 Hz (very close). Sample at 128 Hz for 1 second. Compute the power spectrum with no window, Hann window, and Hamming window. Which window makes it easiest to distinguish the two peaks? Why?
5. **Positional encoding analysis.** Generate the sinusoidal positional encodings for $d_{\text{model}} = 128$ and $\text{max_pos} = 512$. For each pair of positions (p_1, p_2), compute the dot product of their

encodings. Show that the dot product depends only on $|p_1 - p_2|$, not on the absolute positions. What happens to the dot product as the distance increases?

Key Terms

Term	What it means
DFT (Discrete Fourier Transform)	Converts N time-domain samples into N frequency-domain coefficients. Each coefficient is the correlation with a complex sinusoid at that frequency
FFT (Fast Fourier Transform)	An $O(N \log N)$ algorithm to compute the DFT. The Cooley-Tukey algorithm splits even/odd indices recursively
Inverse DFT	Reconstructs the time-domain signal from frequency coefficients. Same formula as DFT with flipped exponent sign and $1/N$ scaling
Frequency bin	Each index k in the DFT output represents frequency $k \cdot f_s / N$ Hz. The “bin” is the discrete frequency slot
DC component	$X[0]$, the zero-frequency coefficient. Proportional to the signal mean
Nyquist frequency	$f_s/2$, the maximum frequency representable at sampling rate f_s . Frequencies above this alias
Power spectrum	$ X[k] ^2$, the squared magnitude of each frequency coefficient. Shows energy distribution across frequencies
Phase spectrum	$\angle(X[k])$, the phase offset of each frequency component. Often ignored in analysis
Spectral leakage	Spurious frequency content caused by treating a non-periodic signal as periodic. Reduced by windowing
Window function	A tapering function (Hann, Hamming, Blackman) applied before DFT to reduce spectral leakage
Twiddle factor	The complex exponential $e^{-j2\pi k/N}$ used to combine sub-DFTs in the FFT butterfly computation
Convolution theorem	Convolution in time domain equals pointwise multiplication in frequency domain. Fundamental to signal processing and CNNs
Circular convolution	Convolution where the signal wraps around. This is what the DFT naturally computes
Linear convolution	Standard convolution without wraparound. Achieved by zero-padding before DFT

Term	What it means
Parseval's theorem	Total energy is preserved through the Fourier transform. $\sum x[n] ^2 = (1/N) \sum X[k] ^2$
Aliasing	When frequencies above Nyquist appear as lower frequencies due to insufficient sampling rate

Code Appendix: *fourier.py*

```
import math

class Complex:
    def __init__(self, real=0.0, imag=0.0):
        self.real = float(real)
        self.imag = float(imag)

    def __add__(self, other):
        if isinstance(other, (int, float)):
            return Complex(self.real + other, self.imag)
        return Complex(self.real + other.real, self.imag + other.imag)

    def __radd__(self, other):
        return self.__add__(other)

    def __sub__(self, other):
        if isinstance(other, (int, float)):
            return Complex(self.real - other, self.imag)
        return Complex(self.real - other.real, self.imag - other.imag)

    def __mul__(self, other):
        if isinstance(other, (int, float)):
            return Complex(self.real * other, self.imag * other)
        r = self.real * other.real - self.imag * other.imag
        i = self.real * other.imag + self.imag * other.real
        return Complex(r, i)

    def __rmul__(self, other):
        return self.__mul__(other)

    def magnitude(self):
        return math.sqrt(self.real ** 2 + self.imag ** 2)

    def phase(self):
        return math.atan2(self.imag, self.real)

    def conjugate(self):
        return Complex(self.real, -self.imag)

    def __repr__(self):
```

```

        if abs(self.imag) < 1e-12:
            return f"{self.real:.6f}"
        sign = "+" if self.imag >= 0 else "-"
        return f"{self.real:.6f} {sign} {abs(self.imag):.6f}i"

def euler(theta):
    return Complex(math.cos(theta), math.sin(theta))

def dft(x):
    N = len(x)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N
            xn = x[n] if isinstance(x[n], Complex) else Complex(x[n])
            total = total + xn * euler(angle)
        result.append(total)
    return result

def idft(X):
    N = len(X)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            xk = X[k] if isinstance(X[k], Complex) else Complex(X[k])
            total = total + xk * euler(angle)
        result.append(Complex(total.real / N, total.imag / N))
    return result

def fft(x):
    N = len(x)
    if N <= 1:
        return [x[0] if isinstance(x[0], Complex) else Complex(x[0])]
    if N % 2 != 0:
        return dft(x)

    even = fft([x[i] for i in range(0, N, 2)])
    odd = fft([x[i] for i in range(1, N, 2)])

    result = [Complex(0)] * N
    for k in range(N // 2):
        angle = -2 * math.pi * k / N
        twiddle = euler(angle)
        t = twiddle * odd[k]
        result[k] = even[k] + t
        result[k + N // 2] = even[k] - t

```

```

    return result

def ifft(X):
    N = len(X)
    conj_X = [xk.conjugate() if isinstance(xk, Complex) else Complex(xk) for xk in X]
    result = fft(conj_X)
    return [Complex(r.real / N, -r.imag / N) for r in result]

def power_spectrum(X):
    return [xk.real ** 2 + xk.imag ** 2 for xk in X]

def magnitude_spectrum(X):
    return [xk.magnitude() for xk in X]

def spectral_analysis(signal, sample_rate):
    N = len(signal)
    X = fft(signal)
    magnitudes = magnitude_spectrum(X)
    freqs = [k * sample_rate / N for k in range(N)]
    return freqs[:N // 2 + 1], magnitudes[:N // 2 + 1]

def hann_window(N):
    return [0.5 * (1 - math.cos(2 * math.pi * n / (N - 1))) for n in range(N)]

def hamming_window(N):
    return [0.54 - 0.46 * math.cos(2 * math.pi * n / (N - 1)) for n in range(N)]

def apply_window(signal, window):
    return [s * w for s, w in zip(signal, window)]

def convolve_direct(x, h):
    N = len(x)
    M = len(h)
    out_len = N + M - 1
    result = [0.0] * out_len
    for n in range(out_len):
        total = 0.0
        for k in range(M):
            if 0 <= n - k < N:
                total += x[n - k] * h[k]
        result[n] = total
    return result

def convolve_fft(x, h):

```

```

if len(x) == 0 or len(h) == 0:
    return []
N = len(x) + len(h) - 1
padded_N = 1
while padded_N < N:
    padded_N *= 2

x_padded = list(x) + [0.0] * (padded_N - len(x))
h_padded = list(h) + [0.0] * (padded_N - len(h))

X = fft(x_padded)
H = fft(h_padded)

Y = [xk * hk for xk, hk in zip(X, H)]

y = ifft(Y)
return [y[n].real for n in range(N)]

def generate_signal(frequencies, amplitudes, N, sample_rate):
    signal = [0.0] * N
    for freq, amp in zip(frequencies, amplitudes):
        for n in range(N):
            t = n / sample_rate
            signal[n] += amp * math.sin(2 * math.pi * freq * t)
    return signal

def positional_encoding(pos, d_model):
    pe = [0.0] * d_model
    for i in range(d_model // 2):
        freq = 1.0 / (10000 ** (2 * i / d_model))
        angle = pos * freq
        pe[2 * i] = math.sin(angle)
        pe[2 * i + 1] = math.cos(angle)
    return pe

def demo_pure_sine():
    print("=" * 65)
    print("  DFT OF A PURE SINE WAVE")
    print("=" * 65)
    print()

    N = 32
    sample_rate = 32
    freq = 5
    signal = generate_signal([freq], [1.0], N, sample_rate)

    print(f"  Signal: sin(2*pi*{freq}*t), {N} samples at {sample_rate} Hz")
    print()

    X = dft(signal)

```

```

mags = magnitude_spectrum(X)

print(f" {'Freq bin k':<12s} {'Frequency (Hz)':>14s} {'|X[k]|':>10s}")
print(f" {'-' * 12} {'-' * 14} {'-' * 10}")

for k in range(N // 2 + 1):
    f_hz = k * sample_rate / N
    if mags[k] > 0.01:
        print(f" k={k:<8d} {f_hz:>14.1f} {mags[k]:>10.4f}")

print()
print(f" Peak at k={freq}, corresponding to {freq} Hz.")
print(f" The DFT correctly identified the frequency.")

def demo_multi_frequency():
    print()
    print()
    print("=" * 65)
    print(" DFT OF SUMMED SINE WAVES")
    print("=" * 65)
    print()

    N = 64
    sample_rate = 64
    freqs = [3, 7, 15]
    amps = [1.0, 0.5, 0.3]

    signal = generate_signal(freqs, amps, N, sample_rate)

    print(f" Signal: {amps[0]}*sin(2*pi*{freqs[0]}*t) + "
          f"{amps[1]}*sin(2*pi*{freqs[1]}*t) + "
          f"{amps[2]}*sin(2*pi*{freqs[2]}*t)")
    print(f" {N} samples at {sample_rate} Hz")
    print()

    X = fft(signal)
    mags = magnitude_spectrum(X)

    print(f" Frequencies recovered (magnitude > 0.5):")
    print(f" {'Freq (Hz)':>10s} {'|X[k]|':>10s} {'Expected amp * N/2':>20s}")
    print(f" {'-' * 10} {'-' * 10} {'-' * 20}")

    for k in range(N // 2 + 1):
        if mags[k] > 0.5:
            f_hz = k * sample_rate / N
            expected = ""
            for freq, amp in zip(freqs, amps):
                if abs(f_hz - freq) < 0.1:
                    expected = f"{amp * N / 2:.1f}"

```

```

        print(f" {f_hz:>10.1f} {mags[k]:>10.4f} {expected:>20s}")

    print()
    print(" All three frequencies correctly recovered.")
    print(" Amplitudes match expected values (amplitude * N/2).")

def demo_fft_vs_dft():
    print()
    print()
    print("=" * 65)
    print(" FFT vs DFT: SAME RESULT, FASTER")
    print("=" * 65)
    print()

    N = 32
    import random
    random.seed(42)
    signal = [random.gauss(0, 1) for _ in range(N)]

    X_dft = dft(signal)
    X_fft = fft(signal)

    max_error = 0.0
    for k in range(N):
        diff_real = abs(X_dft[k].real - X_fft[k].real)
        diff_imag = abs(X_dft[k].imag - X_fft[k].imag)
        max_error = max(max_error, diff_real, diff_imag)

    print(f" Random signal, N = {N}")
    print(f" Max difference between DFT and FFT: {max_error:.2e}")
    print(f" Match: {max_error < 1e-10}")
    print()

    print(f" {'k':<6s} {'DFT |X[k]|':>14s} {'FFT |X[k]|':>14s} {'Diff':>12s}")
    print(f" {'-' * 6} {'-' * 14} {'-' * 14} {'-' * 12}")
    for k in range(8):
        d_mag = X_dft[k].magnitude()
        f_mag = X_fft[k].magnitude()
        diff = abs(d_mag - f_mag)
        print(f" {k:<6d} {d_mag:>14.8f} {f_mag:>14.8f} {diff:>12.2e}")

    print(f" ... ({N - 8} more coefficients)")
    print()

    print(f" DFT complexity:  $O(N^2)$  = {N * N} multiplications")
    print(f" FFT complexity:  $O(N \log_2(N))$  = {int(N * math.log2(N))} multiplications")
    print(f" Speedup: {N * N / (N * math.log2(N)):.1f}x")

def demo_reconstruction():

```



```

print()
print()
print("=" * 65)
print(" PERFECT RECONSTRUCTION: DFT -> IDFT")
print("=" * 65)
print()

import random
random.seed(99)
N = 16
signal = [random.gauss(0, 2) for _ in range(N)]

X = fft(signal)
reconstructed = ifft(X)

max_err = max(abs(reconstructed[n].real - signal[n]) for n in range(N))

print(f" Original and reconstructed signal (N={N}):")
print(f" {'n':<4s} {'Original':>12s} {'Reconstructed':>14s} {'Error':>12s}")
print(f" {'-' * 4} {'-' * 12} {'-' * 14} {'-' * 12}")

for n in range(N):
    err = abs(reconstructed[n].real - signal[n])
    print(f" {'n':<4d} {'signal[n]:>12.6f} {'reconstructed[n].real:>14.6f} {'err:'
>12.2e}")

print()
print(f" Max reconstruction error: {max_err:.2e}")
print(f" Perfect reconstruction: {max_err < 1e-10}")

def demo_convolution_theorem():
    print()
    print()
    print("=" * 65)
    print(" CONVOLUTION THEOREM")
    print("=" * 65)
    print()

    x = [1.0, 2.0, 3.0, 4.0, 5.0]
    h = [1.0, 1.0, 1.0]

    direct = convolve_direct(x, h)
    fft_result = convolve_fft(x, h)

    print(f" Signal x = {x}")
    print(f" Filter h = {h}")
    print(f" Linear convolution (x * h):")
    print()

    print(f" {'n':<4s} {'Direct':>10s} {'FFT-based':>10s} {'Diff':>12s}")

```

```

print(f" {'-' * 4} {'-' * 10} {'-' * 10} {'-' * 12}")

max_err = 0.0
for n in range(len(direct)):
    diff = abs(direct[n] - fft_result[n])
    max_err = max(max_err, diff)
    print(f" {n:<4d} {direct[n]:>10.4f} {fft_result[n]:>10.4f} {diff:>12.2e}
}")

print()
print(f" Max difference: {max_err:.2e}")
print(f" Match: {max_err < 1e-8}")
print()
print(" Convolution in time = multiplication in frequency.")
print(" Direct convolution:  $O(N*M) = O(15)$ ")
print(" FFT convolution:  $O(N*\log(N))$  for large  $N$ ")

def demo_windowing():
    print()
    print()
    print("=" * 65)
    print(" WINDOWING AND SPECTRAL LEAKAGE")
    print("=" * 65)
    print()

    N = 64
    sample_rate = 64
    freq = 7.5

    signal = [math.sin(2 * math.pi * freq * n / sample_rate) for n in range(N)]

    X_rect = fft(signal)
    mags_rect = magnitude_spectrum(X_rect)

    hann = hann_window(N)
    signal_hann = apply_window(signal, hann)
    X_hann = fft(signal_hann)
    mags_hann = magnitude_spectrum(X_hann)

    hamm = hamming_window(N)
    signal_hamm = apply_window(signal, hamm)
    X_hamm = fft(signal_hamm)
    mags_hamm = magnitude_spectrum(X_hamm)

    print(f" Signal:  $\sin(2\pi*\{freq\}*t)$  -- frequency is between bins")
    print(f" N = {N}, sample rate = {sample_rate} Hz")
    print(f" Frequency resolution: {sample_rate / N:.2f} Hz per bin")
    print(f" {freq} Hz falls between bin 7 and bin 8")
    print()

```

```

print(f" {'Freq (Hz)':>10s} {'No window':>12s} {'Hann':>12s} {'Hamming':>12s}
)")
print(f" {'-' * 10} {'-' * 12} {'-' * 12} {'-' * 12}")

for k in range(N // 2 + 1):
    f_hz = k * sample_rate / N
    if mags_rect[k] > 0.5 or (5 <= f_hz <= 11):
        print(f" {f_hz:>10.1f} {mags_rect[k]:>12.4f} "
              f"{mags_hann[k]:>12.4f} {mags_hamm[k]:>12.4f}")

print()
print(" Without windowing, energy leaks into neighboring bins.")
print(" Hann and Hamming windows concentrate energy near the true frequency.")
print(" Tradeoff: windows widen the main peak but suppress side lobes.")

def demo_parseval():
    print()
    print()
    print("=" * 65)
    print(" PARSEVAL'S THEOREM: ENERGY CONSERVATION")
    print("=" * 65)
    print()

    import random
    random.seed(7)
    N = 32
    signal = [random.gauss(0, 1) for _ in range(N)]

    time_energy = sum(s ** 2 for s in signal)

    X = fft(signal)
    freq_energy = sum(xk.real ** 2 + xk.imag ** 2 for xk in X) / N

    print(f" Signal: {N} random samples")
    print(f" Time-domain energy:  $\sum |x[n]|^2 = \{time\_energy:.6f\}$ ")
    print(f" Freq-domain energy:  $(1/N) \sum |X[k]|^2 = \{freq\_energy:.6f\}$ ")
    print(f" Difference:  $\{abs(time\_energy - freq\_energy):.2e\}$ ")
    print(f" Energy conserved:  $\{abs(time\_energy - freq\_energy) < 1e-10\}$ ")

def demo_positional_encoding():
    print()
    print()
    print("=" * 65)
    print(" POSITIONAL ENCODING FREQUENCIES")
    print("=" * 65)
    print()

    d_model = 16
    max_pos = 8

```

```

print(f" d_model = {d_model}, positions 0-{max_pos - 1}")
print()

print(f" Frequency at each dimension pair:")
for i in range(d_model // 2):
    freq = 1.0 / (10000 ** (2 * i / d_model))
    wavelength = 2 * math.pi / freq if freq > 0 else float('inf')
    print(f" dim ({2 * i:>2d},{2 * i + 1:>2d}): freq = {freq:.8f} "
          f"wavelength = {wavelength:.1f}")

print()
print(f" Dot product between position encodings:")
print(f" (depends only on distance, not absolute position)")
print()

print(f" {'pos_i':>6s} {'pos_j':>6s} {'dist':>6s} {'dot product':>12s}")
print(f" {'-' * 6} {'-' * 6} {'-' * 6} {'-' * 12}")

pairs = [(0, 0), (0, 1), (0, 2), (0, 4), (1, 2), (1, 3), (2, 4), (3, 7)]
for p1, p2 in pairs:
    pe1 = positional_encoding(p1, d_model)
    pe2 = positional_encoding(p2, d_model)
    dot = sum(a * b for a, b in zip(pe1, pe2))
    print(f" {p1:>6d} {p2:>6d} {abs(p2 - p1):>6d} {dot:>12.4f}")

print()
print(" Pairs with the same distance have similar dot products.")
print(" This lets the model learn relative position through attention.")

def demo_frequency_scaling():
    print()
    print()
    print("=" * 65)
    print(" FFT COMPLEXITY SCALING")
    print("=" * 65)
    print()

    print(f" {'N':>8s} {'DFT O(N^2)':>14s} {'FFT O(N logN)':>16s} {'Speedup':>10s}")
    print(f" {'-' * 8} {'-' * 14} {'-' * 16} {'-' * 10}")

    for exp in range(3, 14):
        N = 2 ** exp
        dft_ops = N * N
        fft_ops = int(N * math.log2(N))
        speedup = dft_ops / fft_ops
        print(f" {N:>8d} {dft_ops:>14,d} {fft_ops:>16,d} {speedup:>10.1f}x")

```

```

def write_prompt_output():
    output_path = "outputs/prompt-spectral-analyzer.md"
    try:
        with open(output_path, "w") as f:
            f.write("---\n")
            f.write("name: prompt-spectral-analyzer\n")
            f.write("description: Guides analysis of frequency content in signals using
Fourier transform techniques\n")
            f.write("phase: 1\n")
            f.write("lesson: 20\n")
            f.write("---\n\n")
            f.write("You are a spectral analysis expert. You help engineers analyze the
frequency content of signals using Fourier transform techniques.\n\n")
            f.write("When given a signal or signal description, guide the analysis step
by step:\n\n")
            f.write("1. **Determine sampling parameters.**\n")
            f.write("    - What is the sampling rate (fs)? This sets the maximum detecta
ble frequency (Nyquist = fs/2).\n")
            f.write("    - How many samples (N)? This sets the frequency resolution (del
ta_f = fs/N).\n")
            f.write("    - Is the signal length a power of 2? If not, recommend zero-pad
ding for FFT efficiency.\n\n")
            f.write("2. **Choose a window function.**\n")
            f.write("    - Is the signal exactly periodic in the analysis window? If yes
, no window needed.\n")
            f.write("    - For general analysis: use Hann window (good tradeoff between
resolution and leakage).\n")
            f.write("    - For audio/speech: Hamming window.\n")
            f.write("    - When side lobe suppression matters most: Blackman window.\n"
)
            f.write("    - Remember: windowing widens peaks but reduces leakage.\n\n")
            f.write("3. **Compute and interpret the spectrum.**\n")
            f.write("    - Power spectrum  $|X[k]|^2$  shows energy at each frequency.\n")
            f.write("    - Peaks in the power spectrum indicate dominant frequencies.\n"
")
            f.write("    -  $X[0]$  is the DC component (signal mean * N).\n")
            f.write("    - Only look at bins 0 to N/2 for real-valued signals (upper hal
f is the mirror).\n")
            f.write("    - Frequency of bin k:  $f_k = k * fs / N$ .\n\n")
            f.write("4. **Identify dominant frequencies.**\n")
            f.write("    - Find peaks above a noise threshold.\n")
            f.write("    - Convert bin index to Hz:  $freq = k * fs / N$ .\n")
            f.write("    - Check for harmonics (peaks at integer multiples of a fundamen
tal).\n")
            f.write("    - Check for aliased frequencies (actual frequency = fs - appare
nt frequency).\n\n")

```

```

        f.write("5. **Common pitfalls to watch for.**\n")
        f.write("    - Spectral leakage: non-integer number of cycles in the window
causes energy to spread across bins.\n")
        f.write("    - Aliasing: if signal contains frequencies above fs/2, they fol
d back into the spectrum.\n")
        f.write("    - DC offset: large X[0] can mask nearby low-frequency content.
Remove the mean before FFT.\n")
        f.write("    - Zero-padding increases bin density but does NOT improve actua
l frequency resolution.\n")
        f.write("    - Circular vs linear convolution: DFT gives circular convolutio
n. Zero-pad for linear.\n\n")
        f.write("6. **For convolution analysis.**\n")
        f.write("    - Time-domain convolution = frequency-domain multiplication.\n
")
        f.write("    - For large kernels, FFT-based convolution is faster: O(N log N
) vs O(N*M).\n")
        f.write("    - Zero-pad both signals to length N + M - 1 for correct linear
convolution.\n")
        print(f"\n Prompt output written to {output_path}")
    except OSError:
        print("\n Could not write prompt output (run from the lesson directory)")

def print_summary():
    print()
    print()
    print("=" * 65)
    print(" SUMMARY")
    print("=" * 65)
    print()
    print(" 1. The DFT converts N time samples to N frequency coefficients.")
    print(" 2. Each X[k] measures the signal's correlation with frequency k.")
    print(" 3. The FFT computes the DFT in O(N log N) instead of O(N^2).")
    print(" 4. DFT and IDFT are perfect inverses -- no information is lost.")
    print(" 5. The convolution theorem: convolution in time = multiplication")
    print("    in frequency. This is why FFT-based convolution is fast.")
    print(" 6. Windowing reduces spectral leakage for non-periodic signals.")
    print(" 7. Parseval's theorem: energy is conserved through the transform.")
    print(" 8. Transformer positional encodings use the same frequency")
    print("    decomposition idea -- each position gets a unique spectrum.")
    print()

if __name__ == "__main__":
    demo_pure_sine()
    demo_multi_frequency()
    demo_fft_vs_dft()
    demo_reconstruction()
    demo_convolution_theorem()
    demo_windowing()

```

```
demo_parseval()
demo_positional_encoding()
demo_frequency_scaling()
write_prompt_output()
print_summary()
```

You are a spectral analysis expert. You help engineers analyze the frequency content of signals using Fourier transform techniques.

When given a signal or signal description, guide the analysis step by step:

1. **Determine sampling parameters.**
 - What is the sampling rate (fs)? This sets the maximum detectable frequency (Nyquist = $fs/2$).
 - How many samples (N)? This sets the frequency resolution ($\Delta f = fs/N$).
 - Is the signal length a power of 2? If not, recommend zero-padding for FFT efficiency.
2. **Choose a window function.**
 - Is the signal exactly periodic in the analysis window? If yes, no window needed.
 - For general analysis: use Hann window (good tradeoff between resolution and leakage).
 - For audio/speech: Hamming window.
 - When side lobe suppression matters most: Blackman window.
 - Remember: windowing widens peaks but reduces leakage.
3. **Compute and interpret the spectrum.**
 - Power spectrum $|X[k]|^2$ shows energy at each frequency.
 - Peaks in the power spectrum indicate dominant frequencies.
 - $X[0]$ is the DC component (signal mean * N).
 - Only look at bins 0 to $N/2$ for real-valued signals (upper half is the mirror).
 - Frequency of bin k: $f_k = k * fs / N$.
4. **Identify dominant frequencies.**
 - Find peaks above a noise threshold.
 - Convert bin index to Hz: $freq = k * fs / N$.
 - Check for harmonics (peaks at integer multiples of a fundamental).
 - Check for aliased frequencies (apparent frequency = $f_{actual} \bmod fs$; if above $fs/2$, it folds to $fs - f_{apparent}$).
5. **Common pitfalls to watch for.**
 - Spectral leakage: non-integer number of cycles in the window causes energy to spread across bins.

- Aliasing: if signal contains frequencies above $f_s/2$, they fold back into the spectrum.
- DC offset: large $X[0]$ can mask nearby low-frequency content. Remove the mean before FFT.
- Zero-padding increases bin density but does NOT improve actual frequency resolution.
- Circular vs linear convolution: DFT gives circular convolution. Zero-pad for linear.

6. **For convolution analysis.**

- Time-domain convolution = frequency-domain multiplication.
- For large kernels, FFT-based convolution is faster: $O(N \log N)$ vs $O(N*M)$.
- Zero-pad both signals to length $N + M - 1$ for correct linear convolution.

Quiz

Pre-lesson check

- What does the Fourier transform do to a signal?
 - Compresses the signal to use less storage
 - Decomposes the signal into sine waves of different frequencies, amplitudes, and phases **(correct)**
 - Removes noise from the signal
 - Converts the signal from analog to digital

Explanation: The Fourier transform converts a signal from the time domain to the frequency domain. Each frequency coefficient $X[k]$ tells you the amplitude and phase of a sine wave at frequency k . The signal is re-expressed as a sum of these sine waves.
- What is the time complexity of the Fast Fourier Transform (FFT) compared to the direct DFT?
 - Both are $O(N^2)$
 - FFT is $O(N \log N)$, DFT is $O(N^2)$ **(correct)**
 - FFT is $O(N)$, DFT is $O(N \log N)$

- d) FFT is $O(\log N)$, DFT is $O(N)$

Explanation: The direct DFT computes N outputs, each summing over N inputs: $O(N^2)$. The Cooley-Tukey FFT splits the signal into even/odd halves recursively, doing $O(N)$ work at each of $\log_2(N)$ levels, giving $O(N \log N)$. For $N = 1$ million, this is 20 million vs 1 trillion operations.

Post-lesson check

3. What does the convolution theorem state?

- a) Convolution in the time domain equals addition in the frequency domain
- b) Convolution in the time domain equals pointwise multiplication in the frequency domain **(correct)**
- c) Convolution always increases the length of a signal
- d) Convolution and correlation are identical operations

Explanation: The convolution theorem states that convolution in the time domain equals pointwise multiplication in the frequency domain: $x \otimes h = \text{IFFT}(\text{FFT}(x) \cdot \text{FFT}(h))$. This is why FFT-based convolution is $O(N \log N)$ instead of $O(NM)$ for large kernels.

4. Why does zero-padding a signal before FFT NOT increase the true frequency resolution?

- a) Zero-padding introduces noise that cancels the improvement
- b) Zero-padding interpolates between existing frequency bins but cannot reveal frequency detail absent from the original samples **(correct)**
- c) Zero-padding only works for power-of-2 signal lengths
- d) The FFT algorithm ignores zero-padded samples

Explanation: True frequency resolution depends on the observation time $T = N/f_s$. Zero-padding adds more frequency bins (finer grid) but only interpolates the existing spectrum — it gives a smoother-looking result without resolving frequencies closer than $1/T$ Hz apart.

5. In the original Transformer's sinusoidal positional encodings, why are different dimension pairs assigned geometrically spaced frequencies?

- a) It reduces the computational cost of attention
 - b) Each frequency provides a different resolution — high frequencies encode fine position, low frequencies encode coarse position, giving each position a unique fingerprint
(correct)
 - c) Geometric spacing is required by the FFT algorithm
 - d) It ensures all encoding values are between 0 and 1
Explanation: High-frequency dimensions change rapidly with position (fine resolution), while low-frequency dimensions change slowly (coarse resolution). Together, the multi-frequency encoding gives every position a unique pattern — similar to how Fourier coefficients uniquely identify a signal.
-

Graph Theory for Machine Learning

Graphs are the data structure of relationships. If your data has connections, you need graph theory.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-03 (linear algebra, matrices) **Time:** ~90 minutes

Learning Objectives

- Build a graph class with adjacency matrix/list representations and implement BFS and DFS traversals
- Compute the graph Laplacian and use its eigenvalues to detect connected components and cluster nodes
- Implement one round of GNN-style message passing as a normalized adjacency matrix multiplication
- Apply spectral clustering to partition a graph using the Fiedler vector

The Problem

Social networks, molecules, knowledge bases, citation networks, road maps – all are graphs. Traditional ML treats data as flat tables. Each row is independent. Each feature is a column. But when the structure of connections matters, tables fail.

Consider a social network. You want to predict what product a user will buy. Their purchase history matters. But their friends' purchase history matters more. The connections carry signal.

Or consider a molecule. You want to predict if it binds to a protein. The atoms matter, but what really matters is how atoms are bonded to each other. The structure is the data.

Graph Neural Networks (GNNs) are the fastest-growing area in deep learning. They power drug discovery, social recommendation, fraud detection, and knowledge graph reasoning. Every GNN builds on the same foundation: basic graph theory.

You need four things: 1. A way to represent graphs as matrices (so you can multiply them) 2. Traversal algorithms to explore graph structure 3. The Laplacian – the single most important matrix in spectral graph theory 4. Message passing – the operation that makes GNNs work

The Concept

Graphs: Nodes and Edges

A graph $G = (V, E)$ consists of vertices (nodes) V and edges E . Each edge connects two nodes.

Directed vs undirected. In an undirected graph, edge (u, v) means u connects to v AND v connects to u . In a directed graph (digraph), edge (u, v) means u points to v , but not necessarily the reverse.

Weighted vs unweighted. In an unweighted graph, edges either exist or they don't. In a weighted graph, each edge has a numerical weight – a distance, a cost, a strength.

Graph type	Example
Undirected, unweighted	Facebook friendship network
Directed, unweighted	Twitter follow network
Undirected, weighted	Road map (distances)
Directed, weighted	Web page links (PageRank scores)

The Adjacency Matrix

The adjacency matrix A is the core representation. For a graph with n nodes:

$A[i][j] = 1$ if there is an edge from node i to node j
 $A[i][j] = 0$ otherwise

For undirected graphs, A is symmetric: $A[i][j] = A[j][i]$. For weighted graphs, $A[i][j]$ = weight of edge (i, j) .

Example – a triangle:

Nodes: 0, 1, 2
Edges: (0,1), (1,2), (0,2)

 $A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$

The adjacency matrix is the input to every GNN. Matrix operations on A correspond to operations on the graph.

Degree

The degree of a node is the number of edges connected to it. For directed graphs, you have in-degree (edges coming in) and out-degree (edges going out).

The degree matrix D is diagonal:

```
D[i][i] = degree of node i
D[i][j] = 0    for i != j
```

For the triangle example: $D = \text{diag}(2, 2, 2)$ because every node connects to two others.

Degree tells you about node importance. High degree = hub node. The degree distribution of a network reveals its structure. Social networks follow power laws (few hubs, many leaf nodes). Random graphs have Poisson-distributed degrees.

BFS and DFS

The two fundamental graph traversal algorithms. You need both.

Breadth-First Search (BFS): Explore all neighbors first, then neighbors' neighbors. Uses a queue (FIFO).

```
BFS from node 0:
Visit 0
Queue: [1, 2]      (neighbors of 0)
Visit 1
Queue: [2, 3]      (add neighbors of 1)
Visit 2
Queue: [3]          (neighbors of 2 already visited)
Visit 3
Queue: []           (done)
```

BFS finds shortest paths in unweighted graphs. The distance from the start to any node equals the BFS level at which that node is first discovered. This is why BFS is used for hop-count distances in social networks.

Depth-First Search (DFS): Go as deep as possible before backtracking. Uses a stack (LIFO) or recursion.

```
DFS from node 0:
Visit 0
Stack: [1, 2]      (neighbors of 0)
Visit 2             (pop from stack)
Stack: [1, 3]      (add neighbors of 2)
Visit 3             (pop from stack)
Stack: [1]
Visit 1             (pop from stack)
Stack: []           (done)
```

DFS is useful for: - Finding connected components (run DFS from unvisited nodes) - Cycle detection (back edges in DFS tree) - Topological sorting (reverse DFS finish order)

Algorithm	Data structure	Finds	Use case
BFS	Queue	Shortest paths	Social network distance, knowledge graph traversal
DFS	Stack	Components, cycles	Connectivity, topological sort

The Graph Laplacian

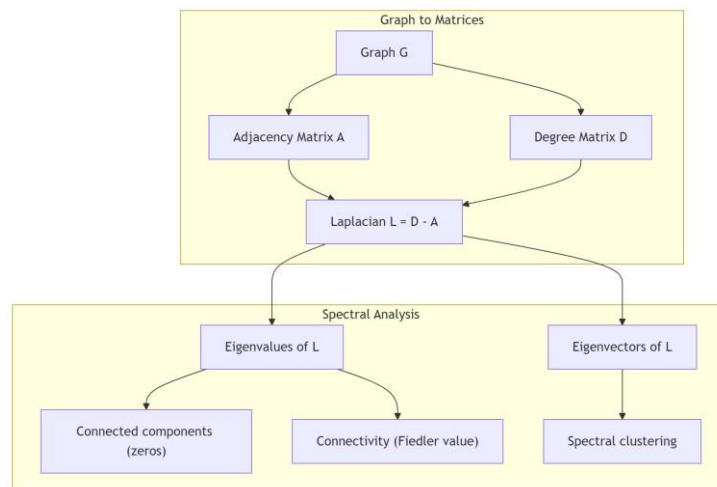
$L = D - A$. The most important matrix in spectral graph theory.

For the triangle:

$$D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \quad A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \quad L = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

The Laplacian has remarkable properties:

1. **L is positive semi-definite.** All eigenvalues are ≥ 0 .
2. **The number of zero eigenvalues equals the number of connected components.** A connected graph has exactly one zero eigenvalue. A graph with 3 disconnected components has three zero eigenvalues.
3. **The smallest non-zero eigenvalue (Fiedler value) measures connectivity.** A large Fiedler value means the graph is well-connected. A small Fiedler value means the graph has a weak point – a bottleneck.
4. **The eigenvector of the Fiedler value (Fiedler vector) reveals the best split.** Nodes with positive values go in one group, nodes with negative values go in the other. This is spectral clustering.



Spectral Properties

The eigenvalues of the adjacency matrix and Laplacian reveal structural properties without any traversal.

Spectral clustering works like this: 1. Compute the Laplacian L 2. Find the k smallest eigenvectors of L (skip the first, which is all-ones for connected graphs) 3. Use those eigenvectors as new coordinates for each node 4. Run k -means on those coordinates

Why does this work? The eigenvectors of L encode the “smoothest” functions on the graph. Nodes that are well-connected get similar eigenvector values. Nodes separated by a bottleneck get different values. The eigenvectors naturally separate clusters.

Random walk connection. The normalized Laplacian relates to random walks on the graph. The stationary distribution of a random walk is proportional to node degree. The mixing time (how fast the walk converges) depends on the spectral gap.

Message Passing

The core operation of Graph Neural Networks. Each node collects messages from its neighbors, aggregates them, and updates its own state.

$$h_v^{(k+1)} = \text{UPDATE}(h_v^{(k)}, \text{AGGREGATE}(\{h_u^{(k)} : u \text{ in neighbors}(v)\}))$$

In the simplest form, AGGREGATE = mean, and UPDATE = linear transform + activation:

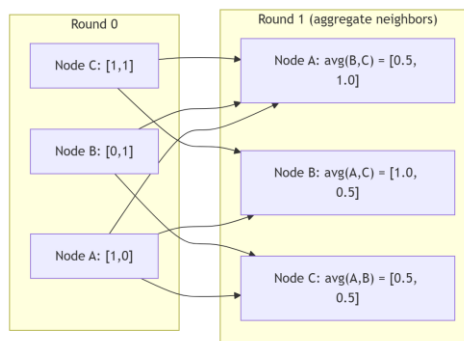
$$h_v^{(k+1)} = \text{sigma}(W * \text{mean}(\{h_u^{(k)} : u \text{ in neighbors}(v)\}))$$

This is matrix multiplication in disguise. If H is the matrix of all node features and A is the adjacency matrix:

$$H^{(k+1)} = \text{sigma}(A_{\text{norm}} * H^{(k)} * W)$$

where A_{norm} is the normalized adjacency matrix (each row sums to 1).

One round of message passing lets each node “see” its immediate neighbors. Two rounds let it see neighbors of neighbors. K rounds give each node information from its K -hop neighborhood.



Concepts and ML Applications

Concept	ML Application
Adjacency matrix	GNN input representation
Graph Laplacian	Spectral clustering, community detection
BFS/DFS	Knowledge graph traversal, path finding
Degree distribution	Node importance, feature engineering
Message passing	GNN layers (GCN, GAT, GraphSAGE)
Eigenvalues of L	Community detection, graph partitioning
Spectral clustering	Unsupervised node grouping
PageRank	Node importance, web search

Build It

Step 1: Graph class from scratch

```
class Graph:
    def __init__(self, n_nodes, directed=False):
        self.n = n_nodes
        self.directed = directed
        self.adj = {i: {} for i in range(n_nodes)}

    def add_edge(self, u, v, weight=1.0):
        self.adj[u][v] = weight
        if not self.directed:
            self.adj[v][u] = weight

    def neighbors(self, node):
        return list(self.adj[node].keys())

    def degree(self, node):
        return len(self.adj[node])

    def adjacency_matrix(self):
        import numpy as np
        A = np.zeros((self.n, self.n))
        for u in range(self.n):
            for v, w in self.adj[u].items():
                A[u][v] = w
        return A
```



```

def degree_matrix(self):
    import numpy as np
    D = np.zeros((self.n, self.n))
    for i in range(self.n):
        D[i][i] = self.degree(i)
    return D

def laplacian(self):
    return self.degree_matrix() - self.adjacency_matrix()

```

The adjacency list (`self.adj`) stores neighbors efficiently. The adjacency matrix conversion uses numpy because all the spectral operations need it.

Step 2: BFS and DFS

```

from collections import deque

def bfs(graph, start):
    visited = set()
    order = []
    distances = {}
    queue = deque([(start, 0)])
    visited.add(start)
    while queue:
        node, dist = queue.popleft()
        order.append(node)
        distances[node] = dist
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))
    return order, distances

def dfs(graph, start):
    visited = set()
    order = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for neighbor in reversed(graph.neighbors(node)):
            if neighbor not in visited:
                stack.append(neighbor)
    return order

```

BFS uses a deque (double-ended queue) for $O(1)$ `popleft`. DFS uses a list as a stack. Both visit every node exactly once – $O(V + E)$ time.

Step 3: Connected components and Laplacian eigenvalues

```
def connected_components(graph):
    visited = set()
    components = []
    for node in range(graph.n):
        if node not in visited:
            order, _ = bfs(graph, node)
            visited.update(order)
            components.append(order)
    return components

def laplacian_eigenvalues(graph):
    import numpy as np
    L = graph.laplacian()
    eigenvalues = np.linalg.eigvalsh(L)
    return eigenvalues
```

`eigvalsh` is for symmetric matrices – the Laplacian is always symmetric for undirected graphs. It returns eigenvalues in ascending order. Count the zeros to find the number of connected components.

Step 4: Spectral clustering

```
def spectral_clustering(graph, k=2):
    import numpy as np
    L = graph.laplacian()
    eigenvalues, eigenvectors = np.linalg.eigh(L)
    features = eigenvectors[:, 1:k+1]

    labels = np.zeros(graph.n, dtype=int)
    for i in range(graph.n):
        if features[i, 0] >= 0:
            labels[i] = 0
        else:
            labels[i] = 1
    return labels
```

For $k=2$, the sign of the Fiedler vector splits the graph into two clusters. For $k>2$, you would run k -means on the first k eigenvectors (excluding the trivial all-ones eigenvector).

Step 5: Message passing

```
def message_passing(graph, features, weight_matrix):
    import numpy as np
    A = graph.adjacency_matrix()
    row_sums = A.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1
    A_norm = A / row_sums
    aggregated = A_norm @ features
```

```

output = aggregated @ weight_matrix
return output

```

This is one round of GNN message passing. Each node's new features are the weighted average of its neighbors' features, transformed by the weight matrix. Stack multiple rounds to propagate information further.

Use It

With networkx and numpy, the same operations are one-liners:

```

import networkx as nx
import numpy as np

G = nx.karate_club_graph()

A = nx.adjacency_matrix(G).toarray()
L = nx.laplacian_matrix(G).toarray()

eigenvalues = np.linalg.eigvalsh(L.astype(float))
print(f"Smallest eigenvalues: {eigenvalues[:5]}")
print(f"Connected components: {nx.number_connected_components(G)}")

communities = nx.community.greedy_modularity_communities(G)
print(f"Communities found: {len(communities)}")

pr = nx.pagerank(G)
top_nodes = sorted(pr.items(), key=lambda x: x[1], reverse=True)[:5]
print(f"Top 5 PageRank nodes: {top_nodes}")

```

networkx handles graphs of any size with optimized C backends. Use it in production. Use your from-scratch implementation to understand what it does.

numpy spectral analysis

```

import numpy as np

A = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 1, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 1, 0, 1],
    [0, 0, 0, 1, 0]
])

D = np.diag(A.sum(axis=1))
L = D - A

eigenvalues, eigenvectors = np.linalg.eigh(L)

```

```

print(f"Eigenvalues: {np.round(eigenvalues, 4)}")
print(f"Fiedler value: {eigenvalues[1]:.4f}")
print(f"Fiedler vector: {np.round(eigenvectors[:, 1], 4)}")

fiedler = eigenvectors[:, 1]
group_a = np.where(fiedler >= 0)[0]
group_b = np.where(fiedler < 0)[0]
print(f"Cluster A: {group_a}")
print(f"Cluster B: {group_b}")

```

The Fiedler vector does the heavy lifting. Positive entries in one cluster, negative in the other. No iterative optimization needed – just one eigendecomposition.

Connections

Concept	Where it shows up
Adjacency matrix	GCN, GAT, GraphSAGE input
Laplacian	Spectral clustering, ChebNet filters
BFS	Knowledge graph traversal, shortest path queries
Message passing	Every GNN layer, neural message passing
Spectral gap	Graph connectivity, mixing time of random walks
Degree distribution	Power-law networks, node feature engineering
Connected components	Preprocessing, handling disconnected graphs
PageRank	Node importance ranking, attention initialization

GNNs deserve special mention. The graph convolution operation in GCN (Kipf & Welling, 2017) uses the adjacency matrix with added self-loops, $A_{\text{hat}} = A + I$:

$$H^{(l+1)} = \text{sigma}(D_{\text{hat}}^{-1/2} * A_{\text{hat}} * D_{\text{hat}}^{-1/2} * H^{(l)} * W^{(l)})$$

where $A_{\text{hat}} = A + I$ (adjacency plus self-loops) and D_{hat} is the degree matrix of A_{hat} . The self-loops ensure each node includes its own features during aggregation. This is exactly message passing with symmetric normalization. $D_{\text{hat}}^{-1/2} * A_{\text{hat}} * D_{\text{hat}}^{-1/2}$ is the normalized adjacency matrix. The Laplacian shows up because this normalization is related to $L_{\text{sym}} = I - D^{-1/2} * A * D^{-1/2}$. Understanding the Laplacian means understanding why GCNs work.

Exercises

1. **Implement PageRank from scratch.** Start with uniform scores. At each step: $\text{score}(v) = (1-d)/n + d * \sum(\text{score}(u)/\text{out_degree}(u))$ for all u pointing to v . Use $d=0.85$. Run until convergence (change $< 1e-6$). Test on a small web graph.
2. **Find communities using spectral clustering.** Create a graph with two clearly separated clusters (e.g., two cliques connected by a single edge). Run spectral clustering and verify it finds the right split. What happens as you add more cross-cluster edges?
3. **Implement Dijkstra's algorithm** for shortest paths in weighted graphs. Compare results to BFS on the same graph with uniform weights.
4. **Build a 2-layer message passing network.** Apply message passing twice with different weight matrices. Show that after 2 rounds, each node has information from its 2-hop neighborhood.
5. **Analyze a real-world graph.** Use the Karate Club graph (34 nodes, 78 edges). Compute degree distribution, Laplacian eigenvalues, and spectral clustering. Compare the spectral clustering result to the known ground truth split.

Key Terms

Term	What people say	What it actually means
Graph	"Nodes and edges"	A mathematical structure $G=(V,E)$ encoding pairwise relationships
Adjacency matrix	"The connection table"	An $n \times n$ matrix where $A[i][j] = 1$ if nodes i and j are connected
Degree	"How connected a node is"	The number of edges touching a node
Laplacian	"D minus A"	$L = D - A$, the matrix whose eigenvalues reveal graph structure
Fiedler value	"The algebraic connectivity"	The smallest non-zero eigenvalue of L , measuring how well-connected the graph is
BFS	"Level-by-level search"	Traversal that visits all neighbors before going deeper, finds shortest paths
DFS	"Go deep first"	Traversal that follows one path to its end before backtracking
Message passing	"Nodes talk to neighbors"	Each node aggregates information from its neighbors, the core of GNNs
Spectral clustering	"Cluster by eigenvectors"	Partition a graph using eigenvectors of its Laplacian

Term	What people say	What it actually means
Connected component	"A separate piece"	A maximal subgraph where every node can reach every other node

You are a graph analysis advisor for ML engineers. Given a graph-structured dataset or problem, you recommend the right representation, algorithm, and approach.

When to use which algorithm

Finding shortest paths: - Unweighted graph: BFS ($O(V + E)$, guaranteed optimal) - Weighted graph, non-negative weights: Dijkstra ($O((V + E) \log V)$) - Weighted graph, negative weights: Bellman-Ford ($O(VE)$)

Finding clusters/communities: - Know the number of clusters: Spectral clustering (compute Laplacian eigenvectors, run k-means) - Don't know the number: Modularity optimization (Louvain algorithm) - Need overlapping communities: Node2Vec embeddings + soft clustering

Measuring node importance: - Directed graph (web/citation): PageRank - Undirected graph (social): Degree centrality, betweenness centrality - Information flow: Eigenvector centrality

Checking structure: - Is the graph connected? BFS from any node, check if all visited - How many components? Repeated BFS on unvisited nodes - Any cycles? DFS, check for back edges - Is it a tree? Connected + exactly $V-1$ edges

Quick reference for graph properties

Property	How to compute	What it tells you
Degree distribution	Count neighbors per node	Hub structure, scale-free vs random
Diameter	BFS from every node, take max	How "wide" the graph is
Clustering coefficient	Triangle count / possible triangles per node	Local density of connections
Fiedler value	Second smallest eigenvalue of Laplacian	Graph connectivity strength
Spectral gap	Difference between first two Laplacian eigenvalues	How fast random walks mix
Average path length	All-pairs BFS, take mean	Small-world property ($< \log(n)$)

Graph representation checklist

1. **Define nodes.** What are the entities? Users, atoms, words, pages?

2. **Define edges.** What relationship? Friendship, bond, co-occurrence, hyperlink?
3. **Directed or undirected?** Is the relationship symmetric?
4. **Weighted or unweighted?** Does edge strength vary?
5. **Node features?** What attributes does each node have?
6. **Edge features?** What attributes does each edge have?
7. **Dynamic or static?** Does the graph change over time?

When to use GNNs vs traditional graph algorithms

Use **traditional algorithms** when: - You need exact answers (shortest paths, connectivity) - The graph is small (< 10K nodes) - You don't have node features - Interpretability matters

Use **GNNs** when: - You have node/edge features - You need to generalize to unseen graphs - The task is node classification, link prediction, or graph classification - The graph is large and you need scalable approximate solutions

Common mistakes

- Forgetting to handle disconnected graphs (run connected components first)
- Using dense adjacency matrices for sparse graphs (wastes memory)
- Ignoring self-loops in GNNs (add identity to adjacency: $A + I$)
- Not normalizing the adjacency matrix (causes feature scale explosion in message passing)
- Running too many message passing rounds (over-smoothing – all nodes converge to same representation)

Quiz

Pre-lesson check

1. What does the adjacency matrix $A[i][j] = 1$ represent?

- a) Node i has degree j
- b) There is an edge from node i to node j (**correct**)
- c) Node i and node j have the same label
- d) The shortest path from i to j has length 1

Explanation: The adjacency matrix is the core representation of a graph. $A[i][j] = 1$ means

there is an edge connecting node i to node j . For undirected graphs, the matrix is symmetric ($A[i][j] = A[j][i]$).

2. What data structure does BFS use and what does it find?

- a) Stack; finds connected components
- b) Queue; finds shortest paths in unweighted graphs **(correct)**
- c) Priority queue; finds minimum spanning tree
- d) Hash map; finds duplicate nodes

Explanation: BFS uses a queue (FIFO) to explore all neighbors at distance k before moving to distance $k+1$. This guarantees that the first time a node is discovered, it is via a shortest path from the source.

Post-lesson check

3. The graph Laplacian $L = D - A$ of a connected graph has how many zero eigenvalues?

- a) Zero
- b) Exactly one **(correct)**
- c) Equal to the number of nodes
- d) Equal to the number of edges

Explanation: The number of zero eigenvalues of the Laplacian equals the number of connected components. A connected graph has exactly one connected component, so exactly one zero eigenvalue. A graph with k disconnected pieces has k zero eigenvalues.

4. In GNN message passing, what does $h_v^{(k+1)} = \sigma(W * \text{mean}(\{h_u^{(k)} : u \text{ in neighbors}(v)\}))$ compute?

- a) The shortest path from v to all other nodes
- b) A new feature vector for node v by aggregating neighbor features, transforming with learned weights, and applying a nonlinearity **(correct)**

- c) The degree of node v at layer $k+1$
 - d) The PageRank score of node v
Explanation: Each node collects features from its neighbors (mean aggregation), multiplies by a learned weight matrix W , and applies an activation function σ . After k rounds, each node has information from its k -hop neighborhood.
5. How does spectral clustering use the Fiedler vector (eigenvector of the second-smallest eigenvalue of L)?
- a) Nodes with the largest Fiedler vector entries form one cluster
 - b) Nodes with positive Fiedler vector values go in one group, nodes with negative values go in the other (**correct**)
 - c) The Fiedler vector is used as edge weights
 - d) The Fiedler vector determines the number of clusters
Explanation: The Fiedler vector encodes the smoothest non-trivial function on the graph. Nodes in the same tightly-connected cluster get similar values, while nodes separated by a bottleneck get values with opposite signs. The sign split partitions the graph into two clusters.
-

Stochastic Processes

Randomness with structure. The math behind random walks, Markov chains, and diffusion models.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 06-07 (probability, Bayes) **Time:** ~75 minutes

Learning Objectives

- Simulate 1D and 2D random walks and verify the $\sqrt{n}$ scaling of displacement
- Build a Markov chain simulator and compute its stationary distribution via eigendecomposition
- Implement Metropolis-Hastings MCMC and Langevin dynamics for sampling from target distributions
- Connect the forward diffusion process to Brownian motion and explain how the reverse process generates data

The Problem

Many AI systems involve randomness that evolves over time. Not static randomness – structured, sequential randomness where each step depends on what came before.

Language models generate tokens one at a time. Each token depends on the previous context. The model outputs a probability distribution, samples from it, and moves on. That is a stochastic process.

Diffusion models add noise to an image step by step until it becomes pure static. Then they reverse the process, denoising step by step until a new image emerges. The forward process is a Markov chain. The reverse process is a learned Markov chain running backward.

Reinforcement learning agents take actions in an environment. Each action leads to a new state with some probability. The agent follows a random policy in a random world. The whole thing is a Markov decision process.

MCMC sampling – the backbone of Bayesian inference – constructs a Markov chain whose stationary distribution is the posterior you want to sample from.

All of these build on four foundational ideas: 1. Random walks – the simplest stochastic process 2. Markov chains – structured randomness with a transition matrix 3. Langevin dynamics – gradient descent with noise 4. Metropolis-Hastings – sampling from any distribution

The Concept

Random Walks

Start at position 0. At each step, flip a fair coin. Heads: move right (+1). Tails: move left (-1).

After n steps, your position is the sum of n random ± 1 values. The expected position is 0 (the walk is unbiased). But the expected distance from the origin grows as $\sqrt{n}$.

This is counterintuitive. The walk is fair – no drift in either direction. But over time, it wanders further and further from where it started. The standard deviation after n steps is $\sqrt{n}$.

```
Step 0: Position = 0
Step 1: Position = +1 or -1
Step 2: Position = +2, 0, or -2
...
Step 100: Expected distance from origin ~ 10 (sqrt(100))
Step 10000: Expected distance from origin ~ 100 (sqrt(10000))
```

In **2D**, the walk moves up, down, left, or right with equal probability. The same $\sqrt{n}$ scaling applies to the distance from the origin. The path traces a fractal-like pattern.

Why $\sqrt{n}$? Each step is ± 1 with equal probability. After n steps, the position $S_n = X_1 + X_2 + \dots + X_n$ where each X_i is ± 1 . The variance of each step is 1, and the steps are independent, so $\text{Var}(S_n) = n$. Standard deviation = $\sqrt{n}$. By the central limit theorem, $S_n / \sqrt{n}$ converges to a standard normal distribution.

This $\sqrt{n}$ scaling shows up everywhere in ML. SGD noise scales as $1/\sqrt{\text{batch_size}}$. Embedding dimensions scale as $\sqrt{d}$. The square root is the signature of independent random additions.

Connection to Brownian motion. Take a random walk with step size $1/\sqrt{n}$ and n steps per unit time. As n goes to ∞ , the walk converges to Brownian motion $B(t)$ – a continuous-time process where $B(t)$ is normally distributed with mean 0 and variance t .

Brownian motion is the mathematical foundation of diffusion. It models the random jiggling of particles in a fluid, the fluctuations of stock prices, and – crucially – the noise process in diffusion models.

Gambler's ruin. A random walker starting at position k , with absorbing barriers at 0 and N . What is the probability of reaching N before 0? For a fair walk: $P(\text{reach } N) = k/N$. This is surprisingly simple and elegant. It connects to the theory of martingales – the fair random walk is a martingale (expected future value = current value).

Markov Chains

A Markov chain is a system that transitions between states according to fixed probabilities. The key property: the next state depends only on the current state, not on the history.

$$P(X_{t+1} = j \mid X_t = i, X_{t-1} = \dots) = P(X_{t+1} = j \mid X_t = i)$$

This is the Markov property. It means you can describe the entire dynamics with a transition matrix P :

Convergence conditions. A Markov chain converges to a unique stationary distribution if it is: -
Irreducible: every state is reachable from every other state - **Aperiodic:** the chain does not cycle with a fixed period

Most chains you encounter in ML satisfy both conditions.

Absorbing states. A state is absorbing if once you enter it, you never leave ($P[i][i] = 1$). Absorbing Markov chains model processes with terminal states – a game that ends, a customer who churns, a token sequence that hits the end-of-text token.

Mixing time. How many steps until the chain is “close” to the stationary distribution? Formally, the number of steps until the total variation distance from stationarity drops below some threshold. Fast mixing = few steps needed. The spectral gap of P (1 minus the second-largest eigenvalue) controls the mixing time. Larger gap = faster mixing.

Connection to Language Models

Token generation in a language model is approximately a Markov process. Given the current context, the model outputs a distribution over the next token. Temperature controls the sharpness:

$$P(\text{token}_i) = \exp(\text{logit}_i / \text{temperature}) / \sum(\exp(\text{logit}_j / \text{temperature}))$$

- Temperature = 1.0: standard distribution
- Temperature < 1.0: sharper (more deterministic)
- Temperature > 1.0: flatter (more random)
- Temperature $\rightarrow 0$: argmax (greedy)

Top-k sampling truncates to the k highest-probability tokens. Top-p (nucleus) sampling truncates to the smallest set of tokens whose cumulative probability exceeds p. Both modify the Markov transition probabilities.

Brownian Motion

The continuous-time limit of the random walk. Position $B(t)$ has three properties: 1. $B(0) = 0$ 2. $B(t) - B(s)$ is normally distributed with mean 0 and variance $t - s$ (for $t > s$) 3. Increments on non-overlapping intervals are independent

Brownian motion is continuous but nowhere differentiable – it jiggles at every scale. The path has fractal dimension 2 in the plane.

In discrete simulation, you approximate Brownian motion by:

$$B(t + dt) = B(t) + \sqrt{dt} * z, \quad \text{where } z \sim N(0, 1)$$

The $\sqrt{dt}$ scaling is important. It comes from the central limit theorem applied to random walks.

Langevin Dynamics

Gradient descent finds the minimum of a function. Langevin dynamics finds the probability distribution proportional to $\exp(-U(x)/T)$, where U is an energy function and T is temperature.

$$x_{t+1} = x_t - dt * \text{gradient}(U(x_t)) + \sqrt{2 * T * dt} * z_t$$

Two forces act on the particle: 1. **Gradient force** ($-dt * \text{gradient}(U)$): pushes toward low energy (like gradient descent) 2. **Random force** ($\sqrt{2Tdt} * z$): pushes in random directions (exploration)

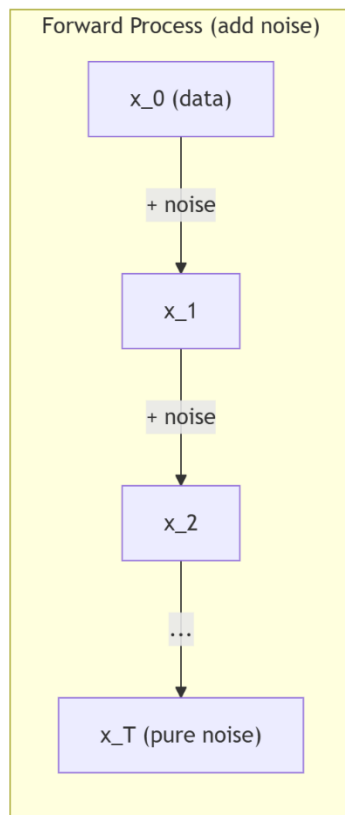
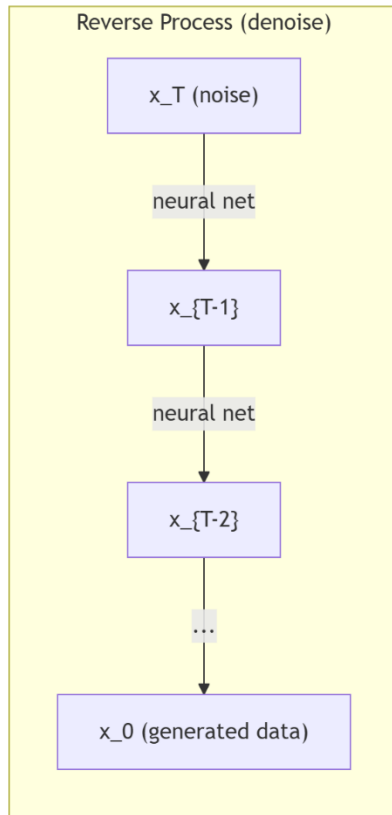
At temperature $T = 0$, this is pure gradient descent. At high temperature, it is nearly a random walk. At the right temperature, the particle explores the energy landscape and spends more time in low-energy regions.

Connection to diffusion models. The forward process of a diffusion model is:

$$x_t = \sqrt{\alpha_t} * x_{t-1} + \sqrt{1 - \alpha_t} * \text{noise}$$

This is a Markov chain that gradually mixes the data with noise. After enough steps, x_T is pure Gaussian noise.

The reverse process – going from noise back to data – is also a Markov chain, but its transition probabilities are learned by a neural network. The network learns to predict the noise that was added at each step, then subtracts it.



MCMC: Markov Chain Monte Carlo

Sometimes you need to sample from a distribution $p(x)$ that you can evaluate (up to a constant) but cannot sample from directly. Bayesian posteriors are the classic example – you know the likelihood times the prior, but the normalizing constant is intractable.

Metropolis-Hastings constructs a Markov chain whose stationary distribution is $p(x)$:

1. Start at some position x
2. Propose a new position x' from a proposal distribution $Q(x' | x)$
3. Compute acceptance ratio: $a = p(x') * Q(x | x') / (p(x) * Q(x' | x))$
4. Accept x' with probability $\min(1, a)$. Otherwise stay at x .
5. Repeat.

If Q is symmetric (e.g., $Q(x' | x) = Q(x | x') = N(x, \sigma^2)$), the ratio simplifies to $a = p(x') / p(x)$. You only need the ratio of probabilities – the normalizing constant cancels.

The chain is guaranteed to converge to $p(x)$ under mild conditions. But convergence can be slow if the proposal is too small (random walk) or too large (high rejection). Tuning the proposal is the art of MCMC.

Why it works. The acceptance ratio ensures detailed balance: the probability of being at x and moving to x' equals the probability of being at x' and moving to x . Detailed balance implies that $p(x)$ is the stationary distribution of the chain. So after enough steps, the samples come from $p(x)$.

Practical considerations:

- **Burn-in:** discard the first N samples. The chain needs time to reach the stationary distribution from its starting point.
- **Thinning:** keep every k -th sample to reduce autocorrelation.
- **Multiple chains:** run several chains from different starting points. If they converge to the same distribution, you have evidence of convergence.
- **Acceptance rate:** for Gaussian proposals in d dimensions, the optimal acceptance rate is about 23% (Roberts & Rosenthal, 2001). Too high means the chain barely moves. Too low means it rejects everything.

Stochastic Processes in AI

Process	AI Application
Random walk	Exploration in RL, Node2Vec embeddings
Markov chain	Text generation, MCMC sampling
Brownian motion	Diffusion models (forward process)
Langevin dynamics	Score-based generative models, SGLD
Markov decision process	Reinforcement learning
Metropolis-Hastings	Bayesian inference, posterior sampling

Build It

Step 1: Random walk simulator

```
import numpy as np

def random_walk_1d(n_steps, seed=None):
    rng = np.random.RandomState(seed)
    steps = rng.choice([-1, 1], size=n_steps)
    positions = np.concatenate([[0], np.cumsum(steps)])
    return positions

def random_walk_2d(n_steps, seed=None):
    rng = np.random.RandomState(seed)
    directions = rng.choice(4, size=n_steps)
    dx = np.zeros(n_steps)
    dy = np.zeros(n_steps)
    dx[directions == 0] = 1 # right
    dx[directions == 1] = -1 # left
    dy[directions == 2] = 1 # up
    dy[directions == 3] = -1 # down
    x = np.concatenate([[0], np.cumsum(dx)])
    y = np.concatenate([[0], np.cumsum(dy)])
    return x, y
```

The 1D walk stores cumulative sums. Each step is +1 or -1. After n steps, the position is the sum. The variance grows linearly with n , so the standard deviation grows as $\sqrt{n}$.

Step 2: Markov chain

```
class MarkovChain:
    def __init__(self, transition_matrix, state_names=None):
```



```

self.P = np.array(transition_matrix, dtype=float)
self.n_states = len(self.P)
self.state_names = state_names or [str(i) for i in range(self.n_states)]

def step(self, current_state, rng=None):
    if rng is None:
        rng = np.random.RandomState()
    probs = self.P[current_state]
    return rng.choice(self.n_states, p=probs)

def simulate(self, start_state, n_steps, seed=None):
    rng = np.random.RandomState(seed)
    states = [start_state]
    current = start_state
    for _ in range(n_steps):
        current = self.step(current, rng)
        states.append(current)
    return states

def stationary_distribution(self):
    eigenvalues, eigenvectors = np.linalg.eig(self.P.T)
    idx = np.argmax(np.abs(eigenvalues - 1.0))
    stationary = np.real(eigenvectors[:, idx])
    stationary = stationary / stationary.sum()
    return np.abs(stationary)

```

The stationary distribution is the left eigenvector of P with eigenvalue 1. We find it by computing eigenvectors of P^T (transposing turns left eigenvectors into right eigenvectors).

Step 3: Langevin dynamics

```

def langevin_dynamics(grad_U, x0, dt, temperature, n_steps, seed=None):
    rng = np.random.RandomState(seed)
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    for _ in range(n_steps):
        noise = rng.randn(*x.shape)
        x = x - dt * grad_U(x) + np.sqrt(2 * temperature * dt) * noise
        trajectory.append(x.copy())
    return np.array(trajectory)

```

The gradient pushes x toward low energy. The noise prevents it from getting stuck. At equilibrium, the distribution of samples is proportional to $\exp(-U(x)/\text{temperature})$.

Step 4: Metropolis-Hastings

```

def metropolis_hastings(target_log_prob, proposal_std, x0, n_samples, seed=None):
    rng = np.random.RandomState(seed)
    x = np.array(x0, dtype=float)
    samples = [x.copy()]
    accepted = 0

```

```

for _ in range(n_samples - 1):
    x_proposed = x + rng.randn(*x.shape) * proposal_std
    log_ratio = target_log_prob(x_proposed) - target_log_prob(x)
    if np.log(rng.rand()) < log_ratio:
        x = x_proposed
        accepted += 1
    samples.append(x.copy())
acceptance_rate = accepted / (n_samples - 1)
return np.array(samples), acceptance_rate

```

The algorithm proposes a new point, checks if it has higher probability (or accepts with probability proportional to the ratio), and repeats. The acceptance rate should be around 23-50% for good mixing.

Use It

In practice, you use established libraries for these algorithms. But understanding the mechanics matters for debugging and tuning.

```

import numpy as np

rng = np.random.RandomState(42)
walk = np.cumsum(rng.choice([-1, 1], size=10000))
print(f"Final position: {walk[-1]}")
print(f"Expected distance: {np.sqrt(10000):.1f}")
print(f"Actual distance: {abs(walk[-1])}")

```

numpy for transition matrices

```

import numpy as np

P = np.array([[0.7, 0.1, 0.2],
              [0.3, 0.4, 0.3],
              [0.4, 0.2, 0.4]])

distribution = np.array([1.0, 0.0, 0.0])
for _ in range(100):
    distribution = distribution @ P

print(f"Stationary distribution: {np.round(distribution, 4)}")

```

Multiply the initial distribution by P repeatedly. After enough iterations, it converges to the stationary distribution regardless of where you started. This is the power method for finding the dominant left eigenvector.

Connections to real frameworks

- **PyTorch diffusion:** The DDPM Scheduler in Hugging Face `diffusers` implements the forward and reverse Markov chains

- **NumPyro / PyMC:** Use MCMC (NUTS sampler, which improves on Metropolis-Hastings) for Bayesian inference
- **Gymnasium (RL):** The environment step function defines a Markov decision process

Verifying Markov chain convergence

```
import numpy as np

P = np.array([[0.9, 0.1], [0.3, 0.7]])

eigenvalues = np.linalg.eigvals(P)
spectral_gap = 1 - sorted(np.abs(eigenvalues))[-2]
print(f"Eigenvalues: {eigenvalues}")
print(f"Spectral gap: {spectral_gap:.4f}")
print(f"Approximate mixing time: {1/spectral_gap:.1f} steps")
```

The spectral gap tells you how fast the chain forgets its initial state. A gap of 0.2 means roughly 5 steps to mix. A gap of 0.01 means roughly 100 steps. Always check this before running long simulations – a slowly mixing chain wastes compute.

Connections

Concept	Where it shows up
Random walk	Node2Vec graph embeddings, exploration in RL
Markov chain	Token generation in LLMs, MCMC sampling
Brownian motion	Forward diffusion process in DDPM, SDE-based models
Langevin dynamics	Score-based generative models, stochastic gradient Langevin dynamics (SGLD)
Stationary distribution	MCMC convergence target, PageRank
Metropolis-Hastings	Bayesian posterior sampling, simulated annealing
Temperature	LLM sampling, Boltzmann exploration in RL, simulated annealing
Mixing time	Convergence speed of MCMC, spectral gap analysis
Absorbing state	End-of-sequence token, terminal states in RL
Detailed balance	Correctness guarantee for MCMC samplers

Diffusion models deserve special attention. DDPM (Ho et al., 2020) defines a forward Markov chain:

$$q(x_t \mid x_{t-1}) = N(x_t; \sqrt{1-\beta_t} * x_{t-1}, \beta_t * I)$$

where β_t is a noise schedule. After T steps, x_T is approximately $N(0, I)$. The reverse process is parameterized by a neural network that predicts the noise:

$$p_\theta(x_{t-1} \mid x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 * I)$$

Every step of generation is a step in a learned Markov chain. Understanding Markov chains means understanding how and why diffusion models generate data.

SGLD (Stochastic Gradient Langevin Dynamics) combines mini-batch gradient descent with Langevin noise. Instead of computing the full gradient, you use a stochastic estimate and add calibrated noise. As learning rate decays, SGLD transitions from optimization to sampling – you get approximate Bayesian posterior samples for free. This is one of the simplest ways to get uncertainty estimates from a neural network.

The key insight across all these connections: stochastic processes are not just theoretical tools. They are the computational mechanisms inside modern AI systems. When you tune the temperature of an LLM, you are adjusting a Markov chain. When you train a diffusion model, you are learning to reverse a Brownian-motion-like process. When you run Bayesian inference, you are constructing a chain that converges to the posterior.

Exercises

1. **Simulate 1000 random walks of 10000 steps.** Plot the distribution of final positions. Verify it is approximately Gaussian with mean 0 and standard deviation $\sqrt{10000} = 100$.
2. **Build a text generator using a Markov chain.** Train on a small corpus: for each word, count transitions to the next word. Build the transition matrix. Generate new sentences by sampling from the chain.
3. **Implement simulated annealing** using Metropolis-Hastings. Start at high temperature (accept almost everything) and gradually cool down (accept only improvements). Use it to find the minimum of a function with many local minima.
4. **Compare Langevin dynamics at different temperatures.** Sample from a double-well potential $U(x) = (x^2 - 1)^2$. At low temperature, samples cluster in one well. At high temperature, they spread across both. Find the critical temperature where the chain mixes between wells.
5. **Implement the forward diffusion process.** Start with a 1D signal (e.g., a sine wave). Add noise progressively over 100 steps with a linear noise schedule. Show how the signal degrades to pure noise. Then implement a simple denoiser that reverses the process (even a naive one that just subtracts the estimated noise).

Key Terms

Term	What people say	What it actually means
Random walk	“Coin-flip movement”	A process where position changes by random increments at each step
Markov property	“Memoryless”	The future depends only on the present state, not on the history
Transition matrix	“The probability table”	$P[i][j]$ = probability of moving from state i to state j
Stationary distribution	“The long-run average”	The distribution π where $\pi * P = \pi$ – the chain’s equilibrium
Brownian motion	“Random jiggling”	The continuous-time limit of a random walk, $B(t) \sim N(0, t)$
Langevin dynamics	“Gradient descent with noise”	Update rule that combines deterministic gradient and random perturbation
MCMC	“Walking toward the target”	Constructing a Markov chain whose stationary distribution is the one you want
Metropolis-Hastings	“Propose and accept/reject”	MCMC algorithm that uses acceptance ratios to ensure convergence
Temperature	“The randomness knob”	Parameter controlling the tradeoff between exploration and exploitation
Diffusion process	“Noise in, noise out”	Forward: gradually add noise. Reverse: gradually remove it. Generates data.

You are a stochastic processes advisor for ML engineers. Given a problem description, you identify the right stochastic process framework and recommend an implementation approach.

Decision framework

When the user describes a problem, classify it:

Is the system discrete or continuous in time? - Discrete: Markov chain, random walk - Continuous: Brownian motion, diffusion, Langevin dynamics

Does the system have a finite set of states? - Yes, finite states: Markov chain (use transition matrix) - No, continuous state: Random walk, Brownian motion, Langevin dynamics

What is the goal? - Sample from a distribution: MCMC (Metropolis-Hastings, Langevin) - Generate new data: Diffusion model - Find optimal actions: Markov decision process (RL) - Model a sequence: Markov chain - Simulate random motion: Random walk / Brownian motion

Process selection guide

Problem type	Process	Key parameters
"I need to sample from a posterior"	Metropolis-Hastings	proposal_std, burn-in, chain length
"I want to generate images/audio"	Diffusion (forward + reverse chains)	noise schedule, number of steps
"I need to model state transitions"	Markov chain	transition matrix P, state space
"I want to find an optimal policy"	MDP + RL	states, actions, rewards, discount
"I need to explore a graph"	Random walk on graph	walk length, restart probability
"I need to optimize with noise"	Langevin dynamics / SGLD	step size, temperature, gradient
"I want to model time series"	Hidden Markov model	emission + transition matrices

Implementation checklist

For **Markov chains**: 1. Define the state space (finite, enumerate all states) 2. Build the transition matrix (rows sum to 1) 3. Verify irreducibility (every state reachable from every other) 4. Check aperiodicity (no fixed cycle length) 5. Compute stationary distribution (eigenvalue method or power iteration) 6. Validate: run a long simulation, compare empirical to theoretical

For **MCMC sampling**: 1. Define the target log-probability (up to a constant is fine) 2. Choose proposal distribution (Gaussian with tunable std) 3. Run chain with burn-in (discard first 10-25% of samples) 4. Check acceptance rate (target 23-50%) 5. Check convergence (multiple chains from different starting points) 6. Compute effective sample size (account for autocorrelation)

For **Langevin dynamics**: 1. Define the energy function $U(x)$ and its gradient 2. Choose step size dt (too large = unstable, too small = slow) 3. Choose temperature (determines exploration vs exploitation) 4. Run with burn-in 5. Verify: samples should match $\exp(-U(x)/T)$ up to normalization

For **diffusion models**: 1. Define the noise schedule ($\beta_1, \dots, \beta_T$) 2. Implement forward process: $x_t = \sqrt{1-\beta_t} * x_{t-1} + \sqrt{\beta_t} * \text{noise}$ 3. Train a neural network to predict the noise at each step 4. Implement reverse process using the trained network 5. Generate by starting from pure noise and running reverse

Common pitfalls

- **MCMC not mixing:** Proposal too small (acceptance too high, chain barely moves) or too large (acceptance too low, chain stays put). Target 23-50% acceptance.
- **Langevin instability:** Step size dt too large. Reduce dt or use adaptive step sizes.
- **Markov chain not converging:** Check that the chain is irreducible and aperiodic. Periodic chains oscillate instead of converging.
- **Diffusion model quality:** Too few steps = blurry outputs. Too many = slow generation. Typical: 50-1000 steps.
- **Forgetting burn-in:** Early samples are biased toward the starting point. Always discard the first portion of the chain.

Quick diagnostics

When something goes wrong: - **Acceptance rate < 10%:** Proposal too aggressive, reduce `proposal_std` -

Acceptance rate > 90%: Proposal too timid, increase `proposal_std` - **Samples stuck in one mode:**

Temperature too low or proposal too small - **Samples everywhere (no structure):** Temperature too high -

Langevin diverges to ∞ : dt too large, reduce by 10x - **Markov chain oscillates:** Check for periodicity, add self-loops

Quiz

Pre-lesson check

1. What is the Markov property?
 - a) The process always returns to its starting state
 - b) The next state depends only on the current state, not on the history of previous states
(correct)
 - c) All states are equally likely at every step
 - d) The process must have a finite number of states

Explanation: The Markov property (memorylessness) means $P(X_{t+1} = j \mid X_t = i, X_{t-1}, \dots) = P(X_{t+1} = j \mid X_t = i)$. The future depends only on where you are, not how you got there. This enables compact representation via a transition matrix.
2. In a 1D random walk, how does the expected distance from the origin scale with the number of steps n ?

- a) Linearly: proportional to n
- b) As the square root: proportional to $\sqrt{n}$ (**correct**)
- c) Logarithmically: proportional to $\log(n)$
- d) It stays constant regardless of n
Explanation: Each step is ± 1 with equal probability. The variance after n steps is n , so the standard deviation (typical distance from origin) is $\sqrt{n}$. After 10,000 steps, the expected distance is about 100, not 10,000.

Post-lesson check

3. What is the stationary distribution of a Markov chain?
 - a) The initial distribution of states
 - b) The distribution that does not change under the transition matrix: $\pi * P = \pi$ (**correct**)
 - c) The distribution of states after exactly one transition
 - d) The uniform distribution over all states
*Explanation: The stationary distribution π satisfies $\pi * P = \pi$ — applying the transition matrix leaves it unchanged. It represents the long-run fraction of time spent in each state. For an irreducible, aperiodic chain, any initial distribution converges to π .*
4. In Langevin dynamics $x_{t+1} = x_t - dt * \text{grad}(U) + \sqrt{2Tdt} * z$, what happens as temperature T approaches 0?
 - a) The process becomes a pure random walk
 - b) The process becomes pure gradient descent (deterministic optimization) (**correct**)
 - c) The process diverges to ∞
 - d) The process freezes at the initial position
*Explanation: At $T = 0$, the noise term $\sqrt{2Tdt}z$ vanishes, leaving $x_{t+1} = x_t - dt * \text{grad}(U)$, which is standard gradient descent. At high T , the noise dominates and the process is nearly a random walk. Intermediate T balances exploration and exploitation.**

5. In a diffusion model, what does the forward process do to a data sample over T steps?

- a) It sharpens the image by removing noise at each step
- b) It gradually adds Gaussian noise until the sample becomes pure noise, following a Markov chain **(correct)**
- c) It compresses the image to a lower resolution

- d) It applies learned transformations to generate new data

*Explanation: The forward process is a Markov chain: $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \cdot \text{noise}$. After T steps, x_T is approximately $N(0, I)$ — pure Gaussian noise. The reverse process (learned by a neural network) then denoises step-by-step to generate new data.**

AI Engineering From Scratch

A Complete Reference for AI Engineers

18 Lessons

AI ENGINEERING FROM SCRATCH

Phase 2: Machine Learning Fundamentals

What Is Machine Learning

Machine learning is teaching computers to find patterns in data instead of writing rules by hand.

Type: Learn **Languages:** Python **Prerequisites:** Phase 1 (Math Foundations) **Time:** ~45 minutes

Learning Objectives

- Explain the difference between supervised, unsupervised, and reinforcement learning and identify which type applies to a given problem
- Implement a nearest centroid classifier from scratch and evaluate it against a random baseline
- Distinguish between classification and regression tasks and select the appropriate loss function for each
- Evaluate whether a given business problem is suitable for ML or better solved with deterministic rules

The Problem

You want to build a spam filter. The traditional approach: sit down and write hundreds of rules. “If the email contains ‘FREE MONEY’, mark it spam. If it has more than 3 exclamation marks, mark it spam.” You spend weeks writing rules. Then spammers change their wording. Your rules break. You write more rules. The cycle never ends.

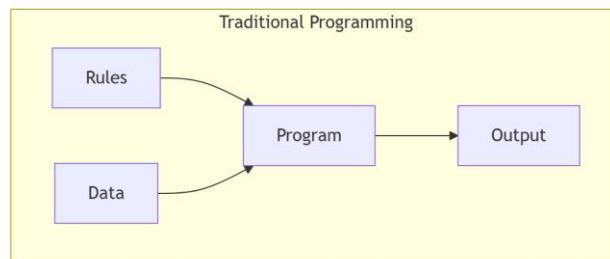
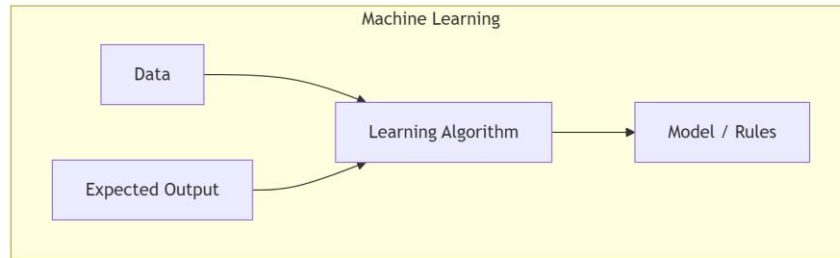
Machine learning flips this. Instead of writing rules, you give the computer thousands of labeled emails (“spam” or “not spam”) and let it figure out the rules on its own. The computer finds patterns you never would have thought of. When spammers change tactics, you retrain on new data instead of rewriting code.

This shift from “programming rules” to “learning from data” is the core of machine learning. Every recommendation engine, voice assistant, self-driving car, and language model works this way.

The Concept

Learning From Data, Not Rules

Traditional programming and machine learning solve problems in opposite directions.

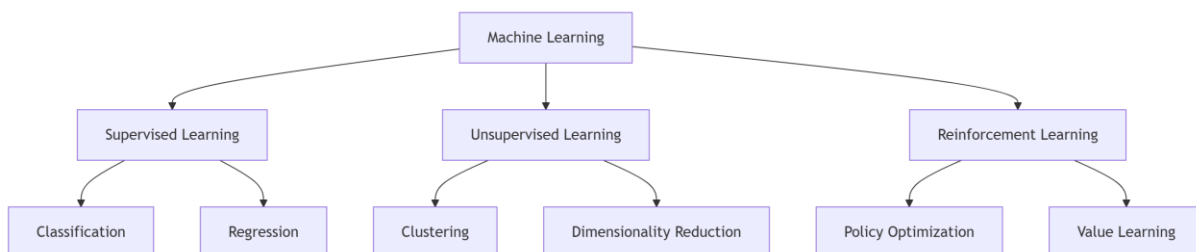


Traditional programming: you write the rules. The program applies them to data to produce output.

Machine learning: you provide data and expected outputs. The algorithm discovers the rules.

The “model” that comes out of training IS the rules, encoded as numbers (weights, parameters). It generalizes from examples it has seen to make predictions on data it has never seen.

The Three Types of Machine Learning



Supervised Learning: You have input-output pairs. The model learns to map inputs to outputs. - “Here are 10,000 photos labeled cat or dog. Learn to tell them apart.” - “Here are house features and prices. Learn to predict the price.”

Unsupervised Learning: You have inputs only. No labels. The model finds structure on its own. - “Here are 10,000 customer purchase histories. Find natural groupings.” - “Here are 1,000 dimensional data points. Reduce to 2 dimensions while keeping structure.”

Reinforcement Learning: An agent takes actions in an environment and receives rewards or penalties. It learns a strategy (policy) to maximize total reward. - “Play this game. +1 for winning, -1 for losing. Figure out a strategy.” - “Control this robot arm. +1 for picking up the object, -0.01 for each second wasted.”

Most of what you will build in practice uses supervised learning. Unsupervised learning is common for preprocessing and exploration. Reinforcement learning powers game AI, robotics, and RLHF for language models.

Beyond the Big Three

The three categories above are clean, but real-world ML often blurs the lines.

Semi-supervised learning uses a small set of labeled data and a large set of unlabeled data. You might have 100 labeled medical images and 100,000 unlabeled ones. Techniques include:

3. **Label propagation:** Build a graph connecting similar data points. Labels spread from labeled nodes to unlabeled neighbors through the graph.
4. **Pseudo-labeling:** Train a model on the labeled data, use it to predict labels for unlabeled data, then retrain on everything. The model bootstraps its own training set.
5. **Consistency regularization:** The model should give the same prediction for an input and a slightly perturbed version of that input. This works even without labels.

Self-supervised learning creates supervision from the data itself. No human labels needed at all. The model creates its own prediction task from the structure of the data.

6. **Masked language modeling (BERT):** Hide 15% of words in a sentence, train the model to predict the missing words. The “labels” come from the original text.
7. **Contrastive learning (SimCLR):** Take an image, create two augmented versions. Train the model to recognize they came from the same image while distinguishing them from augmented versions of other images.
8. **Next-token prediction (GPT):** Predict the next word given all previous words. Every text document becomes a training example.

These are not separate categories from the big three. They are strategies that combine supervised and unsupervised ideas. Self-supervised learning is technically supervised (the model predicts something), but the labels are generated automatically, not by humans.

Classification vs Regression

These are the two main supervised learning tasks.

Aspect	Classification	Regression
Output	Discrete categories	Continuous numbers
Example	“Is this email spam?”	“What will the house price be?”
Output space	{cat, dog, bird}	Any real number

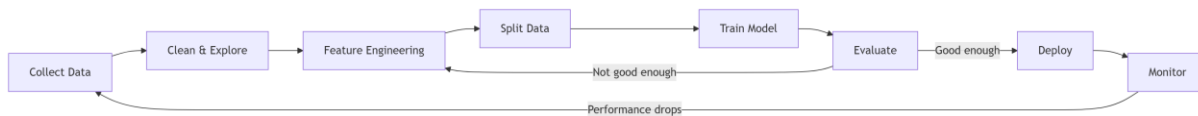
Aspect	Classification	Regression
Loss function	Cross-entropy, accuracy	Mean squared error, MAE
Decision	Boundaries between classes	A curve that fits the data

Classification answers “which category?” Regression answers “how much?”

Some problems can be framed either way. Predicting if a stock goes up or down is classification. Predicting the exact price is regression.

The ML Workflow

Every machine learning project follows the same pipeline, regardless of the algorithm.



Collect Data: Gather raw data. More data is almost always better, but quality matters more than quantity.

Clean & Explore: Handle missing values, remove duplicates, visualize distributions, spot anomalies. This step often takes 60-80% of total project time.

Feature Engineering: Transform raw data into features the model can use. Turn dates into day-of-week. Normalize numerical columns. Encode categorical variables. Good features matter more than fancy algorithms.

Split Data: Divide into training, validation, and test sets. The model trains on training data, you tune hyperparameters on validation data, and you report final performance on test data.

Train Model: Feed training data into an algorithm. The algorithm adjusts internal parameters to minimize a loss function.

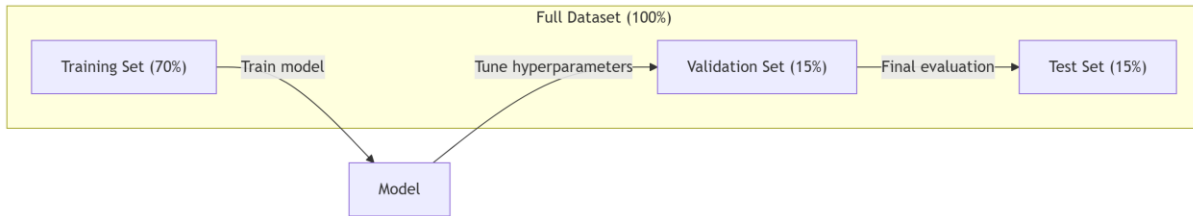
Evaluate: Measure performance on validation/test data. If performance is not acceptable, go back and try different features, algorithms, or hyperparameters.

Deploy: Put the model into production where it makes predictions on new data.

Monitor: Track performance over time. Data distributions change (data drift), and models degrade. When performance drops, retrain.

Training, Validation, and Test Splits

This is the most important concept beginners get wrong. You must evaluate your model on data it has never seen during training. Otherwise you are measuring memorization, not learning.



Split	Purpose	When used	Typical size
Training	Model learns from this data	During training	60-80%
Validation	Tune hyperparameters, compare models	After each training run	10-20%
Test	Final unbiased performance estimate	Once, at the very end	10-20%

The test set is sacred. You look at it exactly once. If you keep adjusting your model based on test performance, you are effectively training on the test set and your reported numbers are meaningless.

For small datasets, use k-fold cross-validation: split data into k parts, train on k-1 parts, validate on the remaining part, rotate, and average results.

Overfitting vs Underfitting



Underfitting: The model is too simple to capture the patterns in the data. A straight line trying to fit a curved relationship. Training error is high. Test error is high.

Overfitting: The model is too complex and memorizes the training data, including its noise. A wiggly curve that passes through every training point but fails on new data. Training error is low. Test error is high.

Good fit: The model captures real patterns without memorizing noise. Training error and test error are both reasonably low.

Signs of overfitting: - Training accuracy is much higher than validation accuracy - The model performs well on training data but poorly on new data - Adding more training data improves performance (the model was memorizing, not learning)

Fixes for overfitting: - Get more training data - Reduce model complexity (fewer parameters, simpler architecture) - Regularization (add a penalty for large weights) - Dropout (randomly zero out neurons during training) - Early stopping (stop training when validation error starts increasing)

Fixes for underfitting: - Use a more complex model - Add more features - Reduce regularization - Train longer

The Bias-Variance Tradeoff

This is the mathematical framework behind overfitting and underfitting.

Bias: Error from wrong assumptions in the model. A linear model has high bias when the true relationship is nonlinear. High bias leads to underfitting.

Variance: Error from sensitivity to small fluctuations in the training data. A model with high variance gives very different predictions when trained on different subsets of data. High variance leads to overfitting.

Model complexity	Bias	Variance	Result
Too low (linear model for curved data)	High	Low	Underfitting
Just right	Medium	Medium	Good generalization
Too high (degree-20 polynomial for 10 points)	Low	High	Overfitting

Total error = Bias² + Variance + Irreducible noise

You cannot reduce irreducible noise (it is randomness in the data itself). You want to find the sweet spot where bias² + variance is minimized.

No Free Lunch Theorem

There is no single algorithm that works best for every problem. An algorithm that performs well on one class of problems will perform poorly on another. This is why data scientists try multiple algorithms and compare results.

In practice, the choice depends on: - How much data you have - How many features there are - Whether the relationship is linear or nonlinear - Whether you need interpretability - How much compute you can afford

When NOT to Use Machine Learning

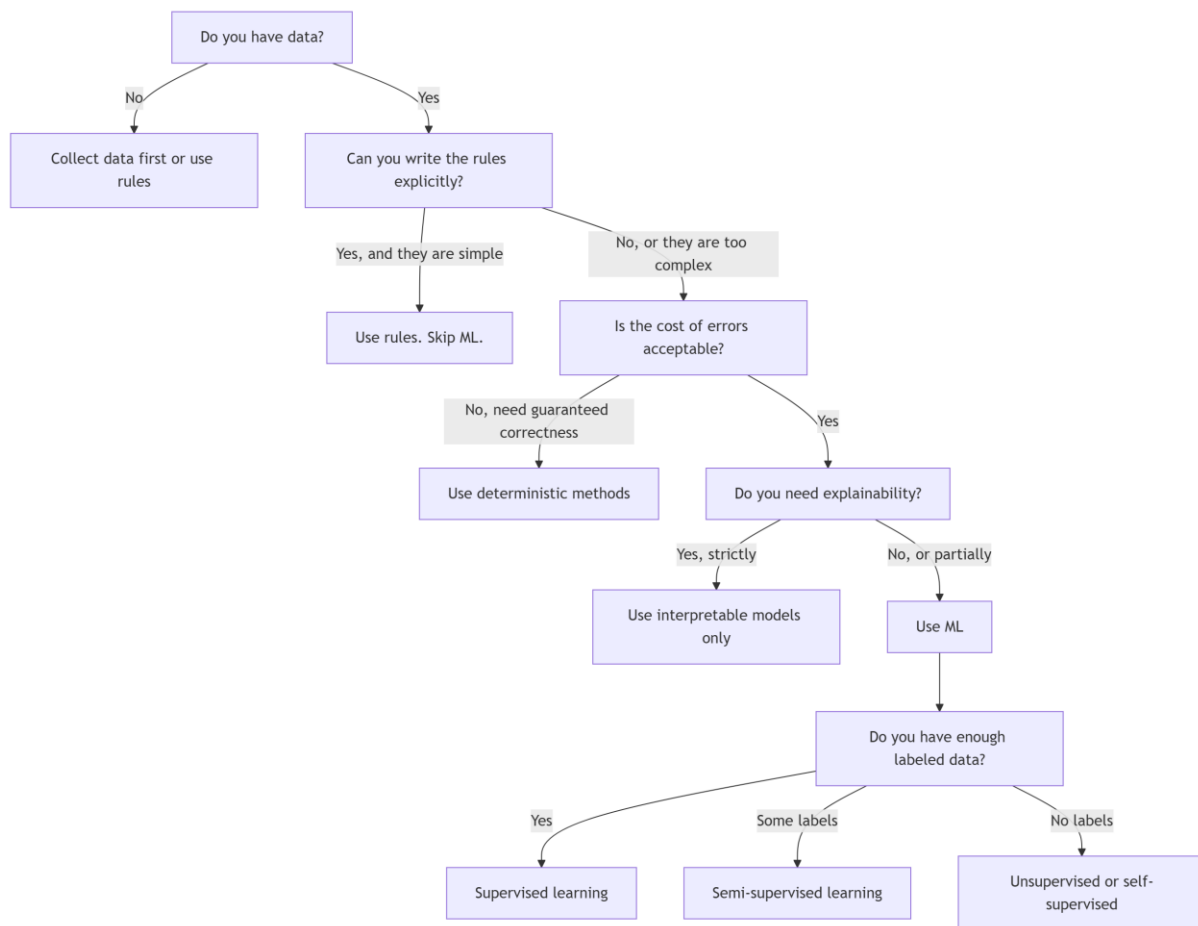
ML is powerful but not always the right tool. Before reaching for a model, ask whether you actually need one.

Do not use ML when:

- **Rules are simple and well-defined.** Tax calculation, sorting algorithms, unit conversions. If you can write the logic in a few if-statements, a model adds complexity for no benefit.
- **You have no data or very little data.** ML needs examples to learn from. With 10 data points, you cannot train anything meaningful. Collect data first.

- **The cost of being wrong is catastrophic and you need guaranteed correctness.** Medical dosage calculation, nuclear reactor control, cryptographic verification. ML models are probabilistic. They will sometimes be wrong. If “sometimes wrong” is unacceptable, use deterministic methods.
- **A lookup table or heuristic solves the problem.** If a simple threshold or table covers 99% of cases, adding ML increases maintenance cost without meaningful improvement.
- **You cannot explain the decision and explainability is required.** Regulated industries (lending, insurance, criminal justice) sometimes require that every decision be fully explainable. Some ML models are interpretable (linear regression, small decision trees). Most are not.
- **The problem changes faster than you can retrain.** If the rules change daily and retraining takes a week, the model is always stale.

Use this decision flowchart:



Build It

The code in `code/ml_intro.py` implements a nearest centroid classifier from scratch, the simplest possible ML algorithm. It demonstrates the core idea: learn from data, then predict on new data.

Step 1: Nearest Centroid Classifier from Scratch

The nearest centroid classifier computes the center (mean) of each class in the training data. To predict, it assigns each new point to the class whose center is closest.

```
class NearestCentroid:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.centroids = np.array([
            X[y == c].mean(axis=0) for c in self.classes
        ])

    def predict(self, X):
        distances = np.array([
            np.sqrt((X - c) ** 2).sum(axis=1)
            for c in self.centroids
        ])
        return self.classes[distances.argmin(axis=0)]
```

That is the entire algorithm. Fit computes two means. Predict computes distances. No gradient descent, no iteration, no hyperparameters.

Step 2: Train on Synthetic Data

We generate a 2D classification dataset with two classes that overlap slightly. The centroid classifier draws a linear decision boundary between the class centers.

```
rng = np.random.RandomState(42)
X_class0 = rng.randn(100, 2) + np.array([1.0, 1.0])
X_class1 = rng.randn(100, 2) + np.array([-1.0, -1.0])
X = np.vstack([X_class0, X_class1])
y = np.array([0] * 100 + [1] * 100)
```

Step 3: Compare Against a Baseline

Every ML model should be compared against a trivial baseline. Here, the baseline predicts a random class. If your ML model does not beat random guessing, something is wrong.

```
baseline_preds = rng.choice([0, 1], size=len(y_test))
baseline_acc = np.mean(baseline_preds == y_test)
```

The centroid classifier should get around 90%+ accuracy on this clean dataset. Random baseline gets around 50%.

Why This Matters

The nearest centroid classifier is trivially simple. It has no hyperparameters, no iteration, no gradient descent. Yet it captures the fundamental ML pattern:

9. **Learn** a representation from training data (the centroids)
10. **Predict** on new data using that representation (nearest distance)
11. **Evaluate** against a baseline (random guessing)

Every ML algorithm, from logistic regression to transformers, follows this same three-step pattern. The representation gets more complex, but the workflow stays the same.

Step 4: What the Centroid Classifier Cannot Do

The nearest centroid classifier assumes each class forms a single blob. It draws linear decision boundaries. It fails when:

- b) Classes have multiple clusters (e.g., the digit “1” can be written in several different ways)
- c) The decision boundary is nonlinear (e.g., one class wraps around another)
- d) Features have very different scales (distance is dominated by the largest-scale feature)

These limitations motivate every other algorithm you will learn. K-nearest neighbors handles multiple clusters. Decision trees handle nonlinear boundaries. Feature scaling fixes the scale problem. Each lesson builds on the limitations of the previous one.

Use It

sklearn provides NearestCentroid and synthetic data generators:

```
from sklearn.neighbors import NearestCentroid
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=500, n_features=2, n_redundant=0,
    n_clusters_per_class=1, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

clf = NearestCentroid()
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.3f}")
```

Key Terms

Term	What people say	What it actually means
Model	“The AI”	A mathematical function with learnable parameters that maps inputs to outputs

Term	What people say	What it actually means
Training	“Teaching the AI”	Running an optimization algorithm to adjust model parameters so predictions match known outputs
Feature	“An input column”	A measurable property of the data that the model uses to make predictions
Label	“The answer”	The known output for a training example, used to compute the error signal
Hyperparameter	“A setting you tweak”	A parameter set before training that controls the learning process (learning rate, number of layers)
Loss function	“How wrong the model is”	A function that measures the gap between predicted and actual outputs, which training tries to minimize
Overfitting	“It memorized the test”	The model learned training-specific noise instead of general patterns, so it fails on new data
Underfitting	“It didn’t learn anything”	The model is too simple to capture the real patterns in the data
Generalization	“It works on new data”	The model’s ability to make accurate predictions on data it was not trained on
Cross-validation	“Testing on different chunks”	Repeatedly splitting data into train/test folds and averaging results, giving a more robust performance estimate
Regularization	“Keeping weights small”	Adding a penalty term to the loss function that discourages overly complex models
Data drift	“The world changed”	The statistical distribution of incoming data shifts over time, degrading model performance

Exercises

- e) Take any dataset (e.g., Iris, Titanic). Split it 70/15/15 into train/validation/test. Explain why you should not tune hyperparameters on the test set.
- f) List three real-world problems. For each one, identify whether it is classification, regression, or clustering, and whether it is supervised or unsupervised.
- g) A model gets 99% accuracy on training data but 60% on test data. Diagnose the problem and list three things you would try to fix it.

You are a machine learning problem framer. Your job is to take a vague business problem and turn it into a concrete ML task with clear inputs, outputs, and success criteria.

When a user describes a business problem, work through each of these steps:

Step 1: Determine the learning type

Ask: do you have labeled data (input-output pairs)? - Yes, with categorical outputs: supervised classification - Yes, with numeric outputs: supervised regression - No labels, looking for structure: unsupervised (clustering or dimensionality reduction) - Some labels, mostly unlabeled: semi-supervised - Agent taking actions in an environment: reinforcement learning

Step 2: Define the prediction target

State exactly what the model predicts. Be specific: - Bad: “predict customer behavior” - Good: “predict whether a customer will cancel their subscription in the next 30 days (binary classification)”

Step 3: Identify features and labels

List the input features the model would use. For each feature, state: - Name and data type (numeric, categorical, text, date) - Whether it would be available at prediction time (no data leakage) - Expected signal strength (high, medium, low)

State the label column and how it is defined.

Step 4: Choose a success metric

Pick the right metric based on the problem: - Classification with balanced classes: accuracy or F1 - Classification with imbalanced classes: precision, recall, F1, or AUC-ROC - Classification where false negatives are costly (medical, fraud): recall - Classification where false positives are costly (spam filter): precision - Regression: MAE if outliers should not dominate, MSE if large errors are especially bad, R-squared for explained variance

Step 5: Establish a baseline

Every ML model must beat a trivial baseline: - Classification: majority class predictor (always predict the most common class) - Regression: predict the mean of the training target - Time series: predict the last observed value

State the expected baseline performance.

Step 6: Flag potential pitfalls

Check for these common issues: - Data leakage: features that encode the target or come from the future - Class imbalance: one class is 10x or more common than the other - Small dataset: fewer than a few hundred labeled examples - Non-stationarity: the data distribution changes over time - Missing a

feedback loop: the model's predictions affect future training data - Not actually needing ML: simple rules or a lookup table would work

Output format

Structure your response as:

- h) **Problem type:** [supervised/unsupervised] [classification/regression/clustering]
- i) **Target variable:** [what exactly the model predicts]
- j) **Features:** [bulleted list with types]
- k) **Success metric:** [metric and why]
- l) **Baseline:** [trivial baseline and expected score]
- m) **Pitfalls:** [any red flags]
- n) **Recommendation:** [start with algorithm X because Y]

Avoid: - Recommending deep learning when the dataset is small or tabular - Skipping the baseline step - Framing a problem as ML when simple rules would suffice - Using jargon without explaining its relevance to the specific problem

Quiz

Pre-lesson check

- o) In supervised learning, what does the model receive during training?
 - a) Only input data with no labels
 - b) Input-output pairs where the correct answer is provided (**correct**)
 - c) A reward signal for each action taken
 - d) A set of rules written by a human expert
Explanation: Supervised learning trains on input-output pairs (labeled data). The model learns to map inputs to known correct outputs.
- p) What is the purpose of splitting data into training and test sets?
 - To make training faster by using less data

- To have backup data in case the training data is lost
- To evaluate whether the model generalizes to data it has never seen during training **(correct)**
- To balance the classes in the dataset
Explanation: The test set measures generalization. If you evaluate on training data, you measure memorization, not learning.

Post-lesson check

12. A model gets 98% accuracy on training data but 55% on test data. What is this an example of?

- Underfitting: the model is too simple
- Overfitting: the model memorized training noise instead of learning general patterns **(correct)**
- Data drift: the test distribution changed
- Good generalization: the model learned the true patterns
Explanation: A large gap between training accuracy (high) and test accuracy (low) is the hallmark of overfitting. The model memorized the training data.

13. An e-commerce site wants to group customers into segments based on purchase behavior without any predefined labels. Which type of ML is this?

1. Supervised learning (classification)
2. Supervised learning (regression)
3. Unsupervised learning (clustering) **(correct)**
4. Reinforcement learning

Explanation: Finding natural groupings in data without predefined labels is clustering, which is a form of unsupervised learning.

14. Which scenario is NOT a good use case for machine learning?

- Predicting customer churn from historical behavior data
 - Detecting fraudulent transactions in a stream of millions of payments
 - Converting temperatures from Celsius to Fahrenheit (**correct**)
 - Classifying images of skin lesions as benign or malignant
- Explanation: Celsius to Fahrenheit is a fixed formula ($F = 9/5 C + 32$). Simple, well-defined rules do not need ML. ML adds complexity with no benefit.**
-

Linear Regression

Linear regression draws the best straight line through your data. It is the “hello world” of machine learning.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Linear Algebra, Calculus, Optimization), Phase 2 Lesson 1 **Time:** ~90 minutes

Learning Objectives

- Derive the gradient descent update rules for mean squared error and implement linear regression from scratch
- Compare gradient descent and the normal equation in terms of computational complexity and when to use each
- Build a multiple linear regression model with feature standardization and interpret the learned weights
- Explain how Ridge regression (L2 regularization) prevents overfitting by penalizing large weights

The Problem

You have data: house sizes and their sale prices. You want to predict the price of a new house given its size. You could eyeball it on a scatter plot, but you need a formula. You need a line that best fits the data so you can plug in any size and get a price prediction.

Linear regression gives you that line. More importantly, it introduces the entire ML training loop: define a model, define a cost function, optimize the parameters. Every ML algorithm follows this same pattern. Master it here with the simplest case, and you will recognize it everywhere.

This is not just for simple problems. Linear regression is used in production systems for demand forecasting, A/B test analysis, financial modeling, and as a baseline for every regression task.

The Model

Linear regression assumes a linear relationship between input (x) and output (y):

$$y = wx + b$$

15. w (weight/slope): how much y changes when x increases by 1

16. b (bias/intercept): the value of y when $x = 0$

For multiple inputs (features), this extends to:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Or in vector form: $y = \mathbf{w}^T * \mathbf{x} + b$

The goal: find the values of w and b that make the predicted y as close as possible to the actual y across all training examples.

The Cost Function (Mean Squared Error)

How do you measure “as close as possible”? You need a single number that captures how wrong your predictions are. The most common choice is Mean Squared Error (MSE):

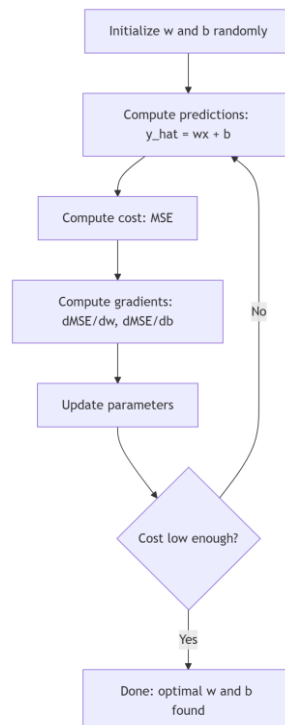
$$\text{MSE} = (1/n) * \sum (y_{\text{predicted}} - y_{\text{actual}})^2$$

Why squared? Two reasons. First, it penalizes large errors more than small errors (an error of 10 is 100x worse than an error of 1, not 10x). Second, the squared function is smooth and differentiable everywhere, which makes optimization straightforward.

The cost function creates a surface. For a single weight w and bias b , the MSE surface looks like a bowl (a convex paraboloid). The bottom of the bowl is where MSE is minimized. Training means finding that bottom.

Gradient Descent

Gradient descent finds the bottom of the bowl by taking steps downhill.



The gradients tell you two things: which direction to move each parameter, and how much to move.

For MSE with $y_{\text{hat}} = wx + b$:

$$\begin{aligned}dMSE/dw &= (2/n) * \text{sum}((y_hat - y) * x) \\dMSE/db &= (2/n) * \text{sum}(y_hat - y)\end{aligned}$$

The update rule:

$$\begin{aligned}w &= w - \text{learning_rate} * dMSE/dw \\b &= b - \text{learning_rate} * dMSE/db\end{aligned}$$

The learning rate controls step size. Too large: you overshoot the minimum and diverge. Too small: training takes forever. Typical starting values: 0.01, 0.001, or 0.0001.

The Normal Equation (Closed-Form Solution)

For linear regression specifically, there is a direct formula that gives the optimal weights without any iteration:

$$w = (X^T * X)^{-1} * X^T * y$$

This inverts a matrix to solve for w in one step. It works perfectly for small datasets. For large datasets (millions of rows or thousands of features), gradient descent is preferred because matrix inversion is $O(n^3)$ in the number of features.

Multiple Linear Regression

With multiple features, the model becomes:

$$y = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$$

Everything works the same: MSE is the cost function, gradient descent updates all weights simultaneously. The only difference is that you are fitting a hyperplane instead of a line.

Feature scaling matters here. If one feature ranges from 0 to 1 and another ranges from 0 to 1,000,000, gradient descent will struggle because the cost surface becomes elongated. Standardize features (subtract mean, divide by standard deviation) before training.

Polynomial Regression

What if the relationship is not linear? You can still use linear regression by creating polynomial features:

$$y = w_1 * x + w_2 * x^2 + w_3 * x^3 + b$$

This is still “linear” regression because the model is linear in the weights (w_1, w_2, w_3). You are just using nonlinear features of x .

Higher-degree polynomials can fit more complex curves but risk overfitting. A degree-10 polynomial will pass through every point in a 10-point dataset but predict poorly on new data.

R-Squared Score

MSE tells you how wrong you are, but the number depends on the scale of y . R-squared (R^2) gives a scale-independent measure:

$$\begin{aligned} R^2 &= 1 - (\text{sum of squared residuals}) / (\text{sum of squared deviations from mean}) \\ &= 1 - SS_{\text{res}} / SS_{\text{tot}} \end{aligned}$$

- q) $R^2 = 1.0$: perfect predictions
- r) $R^2 = 0.0$: the model is no better than predicting the mean every time
- s) $R^2 < 0.0$: the model is worse than predicting the mean

Regularization Preview (Ridge Regression)

When you have many features, the model can overfit by assigning large weights. Ridge regression (L2 regularization) adds a penalty:

$$\text{Cost} = \text{MSE} + \text{lambda} * \text{sum}(w_i^2)$$

The penalty term discourages large weights. The hyperparameter λ controls the tradeoff: higher λ means smaller weights and more regularization. This is covered in depth in a later lesson. For now, know that it exists and why it helps.

Build It

Step 1: Generate sample data

```
import random
import math

random.seed(42)

TRUE_W = 3.0
TRUE_B = 7.0
N_SAMPLES = 100

X = [random.uniform(0, 10) for _ in range(N_SAMPLES)]
y = [TRUE_W * x + TRUE_B + random.gauss(0, 2.0) for x in X]

print(f"Generated {N_SAMPLES} samples")
print(f"True relationship: y = {TRUE_W}x + {TRUE_B} (+ noise)")
print(f"First 5 points: {[round(X[i], 2), round(y[i], 2)] for i in range(5)}")
```

Step 2: Linear regression from scratch with gradient descent

```
class LinearRegression:
    def __init__(self, learning_rate=0.01):
        self.w = 0.0
        self.b = 0.0
        self.lr = learning_rate
        self.cost_history = []

    def predict(self, X):
        return [self.w * x + self.b for x in X]

    def compute_cost(self, X, y):
        predictions = self.predict(X)
        n = len(y)
        cost = sum((pred - actual) ** 2 for pred, actual in zip(predictions, y)) / n
        return cost

    def compute_gradients(self, X, y):
        predictions = self.predict(X)
        n = len(y)
        dw = (2 / n) * sum((pred - actual) * x for pred, actual, x in zip(predictions,
y, X))
        db = (2 / n) * sum(pred - actual for pred, actual in zip(predictions, y))
        return dw, db

    def fit(self, X, y, epochs=1000, print_every=200):
        for epoch in range(epochs):
            dw, db = self.compute_gradients(X, y)
            self.w -= self.lr * dw
            self.b -= self.lr * db
            cost = self.compute_cost(X, y)
            self.cost_history.append(cost)
            if epoch % print_every == 0:
                print(f" Epoch {epoch:4d} | Cost: {cost:.4f} | w: {self.w:.4f} | b
: {self.b:.4f}")
            return self

    def r_squared(self, X, y):
        predictions = self.predict(X)
        y_mean = sum(y) / len(y)
        ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
        ss_tot = sum((actual - y_mean) ** 2 for actual in y)
        return 1 - (ss_res / ss_tot)

print("=== Training Linear Regression (Gradient Descent) ===")
model = LinearRegression(learning_rate=0.005)
model.fit(X, y, epochs=1000, print_every=200)
print(f"\nLearned: y = {model.w:.4f}x + {model.b:.4f}")
```

```
print(f"True:    y = {TRUE_W}x + {TRUE_B}")
print(f"R-squared: {model.r_squared(X, y):.4f}")
```

Step 3: Normal equation (closed-form solution)

```
class LinearRegressionNormal:
    def __init__(self):
        self.w = 0.0
        self.b = 0.0

    def fit(self, X, y):
        n = len(X)
        x_mean = sum(X) / n
        y_mean = sum(y) / n
        numerator = sum((X[i] - x_mean) * (y[i] - y_mean) for i in range(n))
        denominator = sum((X[i] - x_mean) ** 2 for i in range(n))
        self.w = numerator / denominator
        self.b = y_mean - self.w * x_mean
        return self

    def predict(self, X):
        return [self.w * x + self.b for x in X]

    def r_squared(self, X, y):
        predictions = self.predict(X)
        y_mean = sum(y) / len(y)
        ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
        ss_tot = sum((actual - y_mean) ** 2 for actual in y)
        return 1 - (ss_res / ss_tot)

print("\n=== Normal Equation (Closed-Form) ===")
model_normal = LinearRegressionNormal()
model_normal.fit(X, y)
print(f"Learned: y = {model_normal.w:.4f}x + {model_normal.b:.4f}")
print(f"R-squared: {model_normal.r_squared(X, y):.4f}")
```

Step 4: Multiple linear regression

```
class MultipleLinearRegression:
    def __init__(self, n_features, learning_rate=0.01):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.cost_history = []

    def predict_single(self, x):
        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

    def predict(self, X):
```

```

        return [self.predict_single(x) for x in X]

def compute_cost(self, X, y):
    predictions = self.predict(X)
    n = len(y)
    return sum((pred - actual) ** 2 for pred, actual in zip(predictions, y)) / n

def fit(self, X, y, epochs=1000, print_every=200):
    n = len(y)
    n_features = len(X[0])
    for epoch in range(epochs):
        predictions = self.predict(X)
        errors = [pred - actual for pred, actual in zip(predictions, y)]
        for j in range(n_features):
            grad = (2 / n) * sum(errors[i] * X[i][j] for i in range(n))
            self.weights[j] -= self.lr * grad
        grad_b = (2 / n) * sum(errors)
        self.bias -= self.lr * grad_b
        cost = self.compute_cost(X, y)
        self.cost_history.append(cost)
        if epoch % print_every == 0:
            print(f" Epoch {epoch:4d} | Cost: {cost:.4f}")
    return self

def r_squared(self, X, y):
    predictions = self.predict(X)
    y_mean = sum(y) / len(y)
    ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
    ss_tot = sum((actual - y_mean) ** 2 for actual in y)
    return 1 - (ss_res / ss_tot)

random.seed(42)
N = 100
X_multi = []
y_multi = []
for _ in range(N):
    size = random.uniform(500, 3000)
    bedrooms = random.randint(1, 5)
    age = random.uniform(0, 50)
    price = 50 * size + 10000 * bedrooms - 1000 * age + 50000 + random.gauss(0, 20000)
    X_multi.append([size, bedrooms, age])
    y_multi.append(price)

def standardize(X):
    n_features = len(X[0])
    means = [sum(X[i][j] for i in range(len(X))) / len(X) for j in range(n_features)]
    stds = []
    for j in range(n_features):
        variance = sum((X[i][j] - means[j]) ** 2 for i in range(len(X))) / len(X)
        stds.append(variance ** 0.5)
    X_scaled = []

```



```

    for i in range(len(X)):
        row = [(X[i][j] - means[j]) / stds[j] if stds[j] > 0 else 0 for j in range(n_features)]
        X_scaled.append(row)
    return X_scaled, means, stds

y_mean_val = sum(y_multi) / len(y_multi)
y_std_val = (sum((yi - y_mean_val) ** 2 for yi in y_multi) / len(y_multi)) ** 0.5
y_scaled = [(yi - y_mean_val) / y_std_val for yi in y_multi]

X_scaled, x_means, x_stds = standardize(X_multi)

print("\n=== Multiple Linear Regression (3 features) ===")
print("Features: house size, bedrooms, age")
multi_model = MultipleLinearRegression(n_features=3, learning_rate=0.01)
multi_model.fit(X_scaled, y_scaled, epochs=1000, print_every=200)

print(f"\nWeights (standardized): {[round(w, 4) for w in multi_model.weights]}")
print(f"Bias (standardized): {multi_model.bias:.4f}")
print(f"R-squared: {multi_model.r_squared(X_scaled, y_scaled):.4f}")

```

Step 5: Polynomial regression

```

class PolynomialRegression:
    def __init__(self, degree, learning_rate=0.01):
        self.degree = degree
        self.weights = [0.0] * degree
        self.bias = 0.0
        self.lr = learning_rate

    def make_features(self, X):
        return [[x ** (d + 1) for d in range(self.degree)] for x in X]

    def predict(self, X):
        features = self.make_features(X)
        return [sum(w * f for w, f in zip(self.weights, row)) + self.bias for row in features]

    def fit(self, X, y, epochs=1000, print_every=200):
        features = self.make_features(X)
        n = len(y)
        for epoch in range(epochs):
            predictions = [sum(w * f for w, f in zip(self.weights, row)) + self.bias for row in features]
            errors = [pred - actual for pred, actual in zip(predictions, y)]
            for j in range(self.degree):
                grad = (2 / n) * sum(errors[i] * features[i][j] for i in range(n))
                self.weights[j] -= self.lr * grad
            grad_b = (2 / n) * sum(errors)
            self.bias -= self.lr * grad_b

```

```

        if epoch % print_every == 0:
            cost = sum(e ** 2 for e in errors) / n
            print(f" Epoch {epoch:4d} | Cost: {cost:.6f}")
        return self

    def r_squared(self, X, y):
        predictions = self.predict(X)
        y_mean = sum(y) / len(y)
        ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
        ss_tot = sum((actual - y_mean) ** 2 for actual in y)
        return 1 - (ss_res / ss_tot)

random.seed(42)
X_poly = [x / 10.0 for x in range(0, 50)]
y_poly = [0.5 * x ** 2 - 2 * x + 3 + random.gauss(0, 1.0) for x in X_poly]

x_max = max(abs(x) for x in X_poly)
X_poly_norm = [x / x_max for x in X_poly]
y_poly_mean = sum(y_poly) / len(y_poly)
y_poly_std = (sum((yi - y_poly_mean) ** 2 for yi in y_poly) / len(y_poly)) ** 0.5
y_poly_norm = [(yi - y_poly_mean) / y_poly_std for yi in y_poly]

print("\n=== Polynomial Regression (degree 2 vs degree 5) ===")
print("True relationship: y = 0.5x^2 - 2x + 3")

print("\nDegree 2:")
poly2 = PolynomialRegression(degree=2, learning_rate=0.1)
poly2.fit(X_poly_norm, y_poly_norm, epochs=2000, print_every=500)
print(f" R-squared: {poly2.r_squared(X_poly_norm, y_poly_norm):.4f}")

print("\nDegree 5:")
poly5 = PolynomialRegression(degree=5, learning_rate=0.1)
poly5.fit(X_poly_norm, y_poly_norm, epochs=2000, print_every=500)
print(f" R-squared: {poly5.r_squared(X_poly_norm, y_poly_norm):.4f}")

print("\nDegree 2 fits the true curve well. Degree 5 fits training data slightly better")
print("but risks overfitting on new data.")

```

Step 6: Ridge regression (L2 regularization)

```

class RidgeRegression:
    def __init__(self, n_features, learning_rate=0.01, alpha=1.0):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.alpha = alpha

    def predict_single(self, x):
        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

```

```

def predict(self, X):
    return [self.predict_single(x) for x in X]

def fit(self, X, y, epochs=1000, print_every=200):
    n = len(y)
    n_features = len(X[0])
    for epoch in range(epochs):
        predictions = self.predict(X)
        errors = [pred - actual for pred, actual in zip(predictions, y)]
        mse = sum(e ** 2 for e in errors) / n
        reg_term = self.alpha * sum(w ** 2 for w in self.weights)
        cost = mse + reg_term
        for j in range(n_features):
            grad = (2 / n) * sum(errors[i] * X[i][j] for i in range(n))
            grad += 2 * self.alpha * self.weights[j]
            self.weights[j] -= self.lr * grad
        grad_b = (2 / n) * sum(errors)
        self.bias -= self.lr * grad_b
        if epoch % print_every == 0:
            print(f" Epoch {epoch:4d} | Cost: {cost:.4f} | L2 penalty: {reg_term:.4f}")
    return self

print("\n=== Ridge Regression (L2 Regularization) ===")
print("Same data as multiple regression, with alpha=0.1")
ridge = RidgeRegression(n_features=3, learning_rate=0.01, alpha=0.1)
ridge.fit(X_scaled, y_scaled, epochs=1000, print_every=200)
print(f"\nRidge weights: {[round(w, 4) for w in ridge.weights]}")
print(f"Plain weights: {[round(w, 4) for w in multi_model.weights]}")
print("Ridge weights are smaller (shrunk toward zero) due to the L2 penalty.")

```

Use It

Now the same thing with scikit-learn, which is what you will actually use in production.

```

from sklearn.linear_model import LinearRegression as SklearnLR
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

np.random.seed(42)
X_sk = np.random.uniform(0, 10, (100, 1))
y_sk = 3.0 * X_sk.squeeze() + 7.0 + np.random.normal(0, 2.0, 100)

X_train, X_test, y_train, y_test = train_test_split(X_sk, y_sk, test_size=0.2, random_state=42)

```

```

lr = SklearnLR()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

print("=== Scikit-learn Linear Regression ===")
print(f"Coefficient (w): {lr.coef_[0]:.4f}")
print(f"Intercept (b): {lr.intercept_:.4f}")
print(f"R-squared (test): {r2_score(y_test, y_pred):.4f}")
print(f"MSE (test): {mean_squared_error(y_test, y_pred):.4f}")

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly_sk = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

lr_poly = SklearnLR()
lr_poly.fit(X_poly_sk, y_train)
print(f"\nPolynomial degree 2 R-squared: {r2_score(y_test, lr_poly.predict(X_poly_test)):.4f}")

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
print(f"Ridge R-squared: {r2_score(y_test, ridge.predict(X_test_scaled)):.4f}")
print(f"Ridge coefficient: {ridge.coef_[0]:.4f}")

```

Your from-scratch implementation and scikit-learn produce the same results. The difference: scikit-learn handles edge cases, numerical stability, and performance optimizations. Use the library for production. Use the from-scratch version to understand what is happening.

Exercises

- t) Implement batch gradient descent, stochastic gradient descent (SGD), and mini-batch gradient descent. Compare convergence speed on the same dataset. Which converges fastest? Which has the smoothest cost curve?
- u) Generate data from a cubic function ($y = ax^3 + bx^2 + cx + d + \text{noise}$). Fit polynomials of degree 1, 3, and 10. Compare training R^2 and test R^2 . At what degree does overfitting become obvious?
- v) Implement Lasso regression (L1 regularization: $\text{penalty} = \alpha * \sum(|w_i|)$). Train on the multi-feature housing data. Compare which weights go to zero vs Ridge. Why does L1 produce sparse solutions while L2 does not?

Key Terms

Term	What people say	What it actually means
Linear regression	“Draw a line through data”	Find weight w and bias b that minimize the sum of squared differences between $wx+b$ and actual y values
Cost function	“How bad the model is”	A function that maps model parameters to a single number measuring prediction error, which optimization minimizes
Mean squared error	“Average of squared errors”	$(1/n) * \text{sum of } (\text{predicted} - \text{actual})^2$, penalizing large errors disproportionately
Gradient descent	“Walk downhill”	Iteratively adjust parameters in the direction that reduces the cost function, using partial derivatives
Learning rate	“Step size”	A scalar that controls how much parameters change per gradient descent step
Normal equation	“Solve it directly”	The closed-form solution $w = (X^T X)^{-1} X^T y$ that gives optimal weights without iteration
R-squared	“How good the fit is”	The fraction of variance in y explained by the model, ranging from negative ∞ to 1.0
Feature scaling	“Make features comparable”	Transforming features to similar ranges (e.g., zero mean, unit variance) so gradient descent converges faster
Regularization	“Penalize complexity”	Adding a term to the cost function that shrinks weights, preventing overfitting
Ridge regression	“L2 regularization”	Linear regression with a penalty of $\lambda * \text{sum}(w_i^2)$ added to MSE
Polynomial regression	“Fitting curves with linear math”	Linear regression on polynomial features $(x, x^2, x^3, \dots)$, still linear in the weights
Overfitting	“Memorizing training data”	Using a model so complex that it fits noise in training data and fails on new data

Regression Strategy Guide

Regression predicts continuous values. The right approach depends on the relationship between features and target, the number of features, and the risk of overfitting.

Decision Checklist

Is the relationship between features and target approximately linear?

- a) Yes: start with ordinary linear regression
- b) No: try polynomial features or a nonlinear model

How many features do you have relative to samples?

- c) Few features, many samples: ordinary linear regression works fine
- d) Many features, few samples: use regularization (Ridge or Lasso)
- e) More features than samples: Lasso (L1) to select features, or Ridge (L2) to shrink all weights

Do you need interpretability?

- Yes: linear regression with few features, or Lasso for automatic feature selection
- No: polynomial features, or move to tree-based models or neural networks

Is your dataset small (under 10,000 rows)?

1. Use the normal equation (closed-form solution) for speed
2. Cross-validation is essential for reliable evaluation

Is your dataset large (millions of rows)?

- Use stochastic gradient descent (SGD) or mini-batch gradient descent
- The normal equation is too slow due to $O(n^3)$ matrix inversion

When to use each approach

Ordinary Linear Regression: baseline for any regression task. Start here. If R-squared is acceptable and the model is simple, stop here.

Polynomial Regression: the scatter plot shows a curve, not a line. Start with degree 2. Increase only if justified by validation performance. Degree > 5 almost always overfits.

Ridge Regression (L2): many correlated features. All weights shrink toward zero but none become exactly zero. Good when you believe all features contribute.

Lasso Regression (L1): many features and you suspect only a few matter. Lasso drives irrelevant feature weights to exactly zero, performing automatic feature selection.

Elastic Net: combines L1 and L2 penalties. Use when you have many correlated features and want some feature selection.

Common mistakes

17. Skipping feature scaling before gradient descent (convergence becomes extremely slow)
18. Using test set performance to tune hyperparameters (use validation set or cross-validation)
19. Fitting high-degree polynomials without checking validation error (training R^2 always increases with degree)

20. Ignoring residual plots (R^2 can be misleading if residuals show patterns)
21. Treating R^2 as the only metric (check residual distribution, MAE, and domain-specific thresholds)

Quick reference

Method	When to use	Regularization	Feature selection
OLS	Baseline, few features	None	Manual
Ridge	Many features, all relevant	L2 (shrink)	No
Lasso	Many features, few relevant	L1 (zero out)	Automatic
Elastic Net	Many correlated features	L1 + L2	Partial
Polynomial	Nonlinear relationship	Add Ridge/Lasso on top	Manual degree choice

Quiz

Pre-lesson check

- What does the learning rate control in gradient descent?
 1. The number of features used by the model
 2. How many epochs the model trains for
 3. The size of each parameter update step (**correct**)
 4. The ratio of training to test data

Explanation: The learning rate is a scalar that controls how much weights change per gradient descent step. Too large causes divergence, too small causes slow convergence.
- What does $R^2 = 0$ mean for a regression model?
 - f) The model makes perfect predictions
 - g) The model is no better than always predicting the mean of the target (**correct**)
 - h) The model has negative error

- i) The model has not been trained yet
Explanation: $R\text{-squared} = 0$ means the model explains none of the variance in the target. It performs exactly as well as simply predicting the mean every time.

Post-lesson check

Why is feature scaling important for gradient descent in multiple linear regression?

- 3. It makes the model more interpretable
- 4. It prevents the cost surface from being elongated, allowing faster convergence **(correct)**
- 5. It reduces the number of features needed
- 6. It guarantees the model will find the global minimum
Explanation: When features have very different scales, the cost surface becomes elongated. Gradient descent takes many more steps to converge. Standardizing features makes the surface more spherical.

The normal equation gives optimal weights directly. Why would you prefer gradient descent instead?

- j) Gradient descent always gives more accurate results
- k) The normal equation does not work for linear regression
- l) Matrix inversion in the normal equation is $O(n^3)$ in features, which is too slow for thousands of features **(correct)**
- m) Gradient descent requires less memory than storing the data
*Explanation: The normal equation requires inverting $X^T X$, which is $O(n^3)$ in the number of features. For large feature counts, gradient descent is more efficient.**

A degree-10 polynomial regression model fits training data perfectly ($R^2 = 1.0$) but has $R^2 = 0.3$ on test data. What should you do?

- n) Increase the polynomial degree to 20 for even better training fit
- o) Reduce model complexity (lower degree) or add regularization (Ridge) to prevent overfitting **(correct)**

p) Collect less training data so the model cannot memorize

q) Remove the test set and report training R^2 only

Explanation: Perfect training fit with poor test fit is overfitting. The fix is to reduce complexity (lower polynomial degree) or add regularization to penalize large weights.

Logistic Regression

Logistic regression bends a straight line into an S-curve to answer yes-or-no questions with probabilities.

Type: Build **Languages:** Python **Prerequisites:** Phase 2 Lesson 1-2 (What Is ML, Linear Regression) **Time:** ~90 minutes

Learning Objectives

Implement logistic regression from scratch using the sigmoid function and binary cross-entropy loss

Compute and interpret precision, recall, F1 score, and the confusion matrix for binary classification

Explain why MSE fails for classification and why binary cross-entropy produces a convex cost surface

Build a softmax regression model for multi-class classification and evaluate threshold tuning tradeoffs

The Problem

You want to predict whether a tumor is malignant or benign given its size. You try linear regression. It outputs numbers like 0.3 or 1.7 or -0.5. What do those mean? Is 1.7 “very malignant”? Is -0.5 “very benign”? Linear regression outputs unbounded numbers. Classification needs bounded probabilities between 0 and 1, and a clear decision: yes or no.

Logistic regression solves this. It takes the same linear combination ($wx + b$) and passes it through the sigmoid function, which squashes any number into the range (0, 1). The output is a probability. You set a threshold (usually 0.5) and make a decision.

This is one of the most widely used algorithms in practice. Despite its name, logistic regression is a classification algorithm, not a regression algorithm. The name comes from the logistic (sigmoid) function it uses.

The Concept

Why Linear Regression Fails for Classification

Imagine predicting pass/fail (1/0) based on study hours. Linear regression fits a line through the data:

hours:	1	2	3	4	5	6	7	8	9	10
actual:	0	0	0	0	1	1	1	1	1	1

A linear fit might produce predictions like -0.2 at hour 1 and 1.3 at hour 10. These values are not probabilities. They go below 0 and above 1. Worse, a single outlier (someone who studied 50 hours) would drag the entire line, changing predictions for everyone.

Classification needs a function that: - Outputs values between 0 and 1 (probabilities) - Creates a sharp transition (a decision boundary) - Is not distorted by outliers far from the boundary

The Sigmoid Function

The sigmoid function does exactly this:

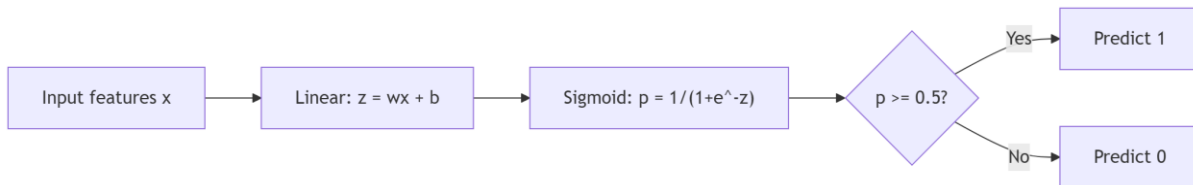
$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

Properties: - When z is large and positive, $\text{sigmoid}(z)$ approaches 1 - When z is large and negative, $\text{sigmoid}(z)$ approaches 0 - When $z = 0$, $\text{sigmoid}(z) = 0.5$ - The output is always between 0 and 1 - The function is smooth and differentiable everywhere

The derivative has a convenient form: $\text{sigmoid}'(z) = \text{sigmoid}(z) * (1 - \text{sigmoid}(z))$. This makes gradient computation efficient.

Logistic Regression = Linear Model + Sigmoid

The model computes $z = wx + b$ (same as linear regression), then applies sigmoid:



The output p is interpreted as $P(y=1 | x)$, the probability that the input belongs to class 1. The decision boundary is where $w x + b = 0$, which makes sigmoid output exactly 0.5.

Binary Cross-Entropy Loss

You cannot use MSE for logistic regression. MSE with a sigmoid creates a non-convex cost surface with many local minima. Instead, use binary cross-entropy (log loss):

$$\text{Loss} = -(1/n) * \sum(y * \log(p) + (1-y) * \log(1-p))$$

Why this works: - When $y=1$ and p is close to 1: $\log(1) = 0$, so loss is near 0 (correct, low cost) - When $y=1$ and p is close to 0: $\log(0)$ approaches negative ∞ , so loss is huge (wrong, high cost) - When $y=0$ and p is close to 0: $\log(1) = 0$, so loss is near 0 (correct, low cost) - When $y=0$ and p is close to 1: $\log(0)$ approaches negative ∞ , so loss is huge (wrong, high cost)

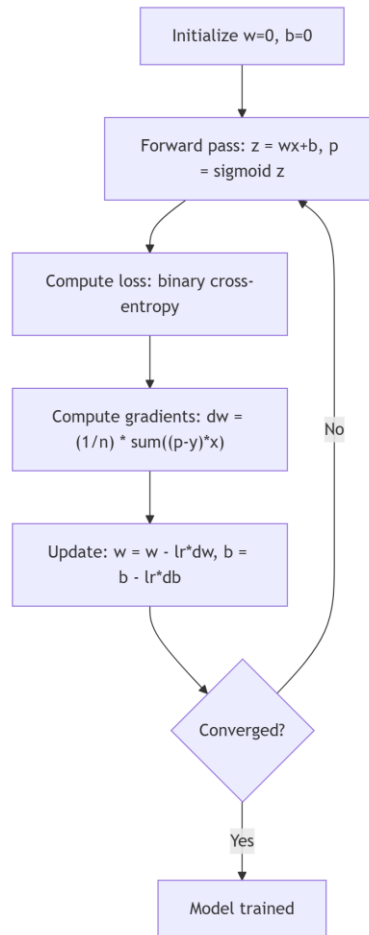
This loss function is convex for logistic regression, guaranteeing a single global minimum.

Gradient Descent for Logistic Regression

The gradients for binary cross-entropy with sigmoid have a clean form:

$$\begin{aligned} dL/dw &= (1/n) * \sum((p - y) * x) \\ dL/db &= (1/n) * \sum(p - y) \end{aligned}$$

These look identical to the linear regression gradients. The difference is that $p = \text{sigmoid}(wx + b)$ instead of $p = wx + b$. The sigmoid introduces the nonlinearity, but the gradient update rule stays the same.



The Decision Boundary

For a 2D input (two features), the decision boundary is the line where:

$$w_1 * x_1 + w_2 * x_2 + b = 0$$

Points on one side get classified as 1, points on the other side as 0. Logistic regression always produces a linear decision boundary. If you need a curved boundary, you either add polynomial features or use a nonlinear model.

Multi-Class Classification with Softmax

Binary logistic regression handles two classes. For k classes, use the softmax function:

$$\text{softmax}(z_i) = e^{(z_i)} / \sum(e^{(z_j)} \text{ for all } j)$$

Each class has its own weight vector. The model computes a score z_i for each class, then softmax converts scores to probabilities that sum to 1. The predicted class is the one with the highest probability.

The loss function becomes categorical cross-entropy:

$$\text{Loss} = -(1/n) * \sum(\sum(y_k * \log(p_k)))$$

where y_k is 1 for the true class and 0 for all others (one-hot encoding).

Evaluation Metrics

Accuracy alone is not enough. For a dataset with 95% negative and 5% positive, a model that always predicts negative gets 95% accuracy but is useless.

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

Precision: Of all predicted positives, how many are actually positive?

$$\text{Precision} = TP / (TP + FP)$$

Recall (Sensitivity): Of all actual positives, how many did we catch?

$$\text{Recall} = TP / (TP + FN)$$

F1 Score: Harmonic mean of precision and recall. Balances both metrics.

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

When to prioritize: - **Precision:** when false positives are costly (spam filter, you do not want to block legitimate email) - **Recall:** when false negatives are costly (cancer screening, you do not want to miss a tumor) - **F1:** when you need a single balanced metric

Build It

Step 1: Sigmoid function and data generation

```
import random
import math

def sigmoid(z):
```

```

    z = max(-500, min(500, z))
    return 1.0 / (1.0 + math.exp(-z))

random.seed(42)
N = 200
X = []
y = []

for _ in range(N // 2):
    X.append([random.gauss(2, 1), random.gauss(2, 1)])
    y.append(0)

for _ in range(N // 2):
    X.append([random.gauss(5, 1), random.gauss(5, 1)])
    y.append(1)

combined = list(zip(X, y))
random.shuffle(combined)
X, y = zip(*combined)
X = list(X)
y = list(y)

print(f"Generated {N} samples (2 classes, 2 features)")
print(f"Class 0 center: (2, 2), Class 1 center: (5, 5)")
print(f"First 5 samples:")
for i in range(5):
    print(f"  Features: [{X[i][0]:.2f}, {X[i][1]:.2f}], Label: {y[i]}")

```

Step 2: Logistic regression from scratch

```

class LogisticRegression:
    def __init__(self, n_features, learning_rate=0.01):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.loss_history = []

    def predict_proba(self, x):
        z = sum(w * xi for w, xi in zip(self.weights, x)) + self.bias
        return sigmoid(z)

    def predict(self, x, threshold=0.5):
        return 1 if self.predict_proba(x) >= threshold else 0

    def compute_loss(self, X, y):
        n = len(y)
        total = 0.0
        for i in range(n):
            p = self.predict_proba(X[i])
            p = max(1e-15, min(1 - 1e-15, p))
            total += y[i] * math.log(p) + (1 - y[i]) * math.log(1 - p)

```

```

        return -total / n

    def fit(self, X, y, epochs=1000, print_every=200):
        n = len(y)
        n_features = len(X[0])
        for epoch in range(epochs):
            dw = [0.0] * n_features
            db = 0.0
            for i in range(n):
                p = self.predict_proba(X[i])
                error = p - y[i]
                for j in range(n_features):
                    dw[j] += error * X[i][j]
                db += error
            for j in range(n_features):
                self.weights[j] -= self.lr * (dw[j] / n)
            self.bias -= self.lr * (db / n)
            loss = self.compute_loss(X, y)
            self.loss_history.append(loss)
            if epoch % print_every == 0:
                print(f" Epoch {epoch:4d} | Loss: {loss:.4f} | w: [{self.weights[0]:.3f}, {self.weights[1]:.3f}] | b: {self.bias:.3f}")
            return self

    def accuracy(self, X, y):
        correct = sum(1 for i in range(len(y)) if self.predict(X[i]) == y[i])
        return correct / len(y)

split = int(0.8 * N)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print("\n=== Training Logistic Regression ===")
model = LogisticRegression(n_features=2, learning_rate=0.1)
model.fit(X_train, y_train, epochs=1000, print_every=200)

print(f"\nTrain accuracy: {model.accuracy(X_train, y_train):.4f}")
print(f"Test accuracy: {model.accuracy(X_test, y_test):.4f}")
print(f"Weights: [{model.weights[0]:.4f}, {model.weights[1]:.4f}]")
print(f"Bias: {model.bias:.4f}")

```

Step 3: Confusion matrix and metrics from scratch

```

class ClassificationMetrics:
    def __init__(self, y_true, y_pred):
        self.tp = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 1)
        self.tn = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 0)
        self.fp = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 1)
        self.fn = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 0)

```

```

def accuracy(self):
    total = self.tp + self.tn + self.fp + self.fn
    return (self.tp + self.tn) / total if total > 0 else 0

def precision(self):
    denom = self.tp + self.fp
    return self.tp / denom if denom > 0 else 0

def recall(self):
    denom = self.tp + self.fn
    return self.tp / denom if denom > 0 else 0

def f1(self):
    p = self.precision()
    r = self.recall()
    return 2 * p * r / (p + r) if (p + r) > 0 else 0

def print_confusion_matrix(self):
    print(f"\n Confusion Matrix:")
    print(f"                Predicted")
    print(f"                Pos    Neg")
    print(f" Actual Pos    {self.tp:4d} {self.fn:4d}")
    print(f" Actual Neg    {self.fp:4d} {self.tn:4d}")

def print_report(self):
    self.print_confusion_matrix()
    print(f"\n Accuracy: {self.accuracy():.4f}")
    print(f" Precision: {self.precision():.4f}")
    print(f" Recall:    {self.recall():.4f}")
    print(f" F1 Score:  {self.f1():.4f}")

y_pred_test = [model.predict(x) for x in X_test]
print("\n=== Classification Report (Test Set) ===")
metrics = ClassificationMetrics(y_test, y_pred_test)
metrics.print_report()

```

Step 4: Decision boundary analysis

```

print("\n=== Decision Boundary ===")
w1, w2 = model.weights
b = model.bias
print(f"Decision boundary: {w1:.4f}*x1 + {w2:.4f}*x2 + {b:.4f} = 0")
if abs(w2) > 1e-10:
    print(f"Solved for x2:      x2 = {-w1/w2:.4f}*x1 + {-b/w2:.4f}")

print("\nSample predictions near the boundary:")
test_points = [
    [3.0, 3.0],
    [3.5, 3.5],

```



```

[4.0, 4.0],
[2.5, 2.5],
[5.0, 5.0],
]
for point in test_points:
    prob = model.predict_proba(point)
    pred = model.predict(point)
    print(f" [{point[0]}, {point[1]}] -> prob={prob:.4f}, class={pred}")

```

Step 5: Multi-class with softmax

```

class SoftmaxRegression:
    def __init__(self, n_features, n_classes, learning_rate=0.01):
        self.n_features = n_features
        self.n_classes = n_classes
        self.lr = learning_rate
        self.weights = [[0.0] * n_features for _ in range(n_classes)]
        self.biases = [0.0] * n_classes

    def softmax(self, scores):
        max_score = max(scores)
        exp_scores = [math.exp(s - max_score) for s in scores]
        total = sum(exp_scores)
        return [e / total for e in exp_scores]

    def predict_proba(self, x):
        scores = [
            sum(self.weights[k][j] * x[j] for j in range(self.n_features)) + self.biases[k]
            for k in range(self.n_classes)
        ]
        return self.softmax(scores)

    def predict(self, x):
        probs = self.predict_proba(x)
        return probs.index(max(probs))

    def fit(self, X, y, epochs=1000, print_every=200):
        n = len(y)
        for epoch in range(epochs):
            grad_w = [[0.0] * self.n_features for _ in range(self.n_classes)]
            grad_b = [0.0] * self.n_classes
            total_loss = 0.0
            for i in range(n):
                probs = self.predict_proba(X[i])
                for k in range(self.n_classes):
                    target = 1.0 if y[i] == k else 0.0
                    error = probs[k] - target
                    for j in range(self.n_features):
                        grad_w[k][j] += error * X[i][j]
                    grad_b[k] += error
            true_prob = max(probs[y[i]], 1e-15)

```

```

        total_loss -= math.log(true_prob)
    for k in range(self.n_classes):
        for j in range(self.n_features):
            self.weights[k][j] -= self.lr * (grad_w[k][j] / n)
            self.biases[k] -= self.lr * (grad_b[k] / n)
    if epoch % print_every == 0:
        print(f" Epoch {epoch:4d} | Loss: {total_loss / n:.4f}")
    return self

def accuracy(self, X, y):
    correct = sum(1 for i in range(len(y)) if self.predict(X[i]) == y[i])
    return correct / len(y)

random.seed(42)
X_3class = []
y_3class = []

centers = [(1, 1), (5, 1), (3, 5)]
for label, (cx, cy) in enumerate(centers):
    for _ in range(50):
        X_3class.append([random.gauss(cx, 0.8), random.gauss(cy, 0.8)])
        y_3class.append(label)

combined = list(zip(X_3class, y_3class))
random.shuffle(combined)
X_3class, y_3class = zip(*combined)
X_3class = list(X_3class)
y_3class = list(y_3class)

split_3 = int(0.8 * len(X_3class))
X_train_3 = X_3class[:split_3]
y_train_3 = y_3class[:split_3]
X_test_3 = X_3class[split_3:]
y_test_3 = y_3class[split_3:]

print("\n=== Multi-class Softmax Regression (3 classes) ===")
softmax_model = SoftmaxRegression(n_features=2, n_classes=3, learning_rate=0.1)
softmax_model.fit(X_train_3, y_train_3, epochs=1000, print_every=200)
print(f"\nTrain accuracy: {softmax_model.accuracy(X_train_3, y_train_3):.4f}")
print(f"Test accuracy: {softmax_model.accuracy(X_test_3, y_test_3):.4f}")

print("\nSample predictions:")
for i in range(5):
    probs = softmax_model.predict_proba(X_test_3[i])
    pred = softmax_model.predict(X_test_3[i])
    print(f" True: {y_test_3[i]}, Predicted: {pred}, Probs: [{', '.join(f'{p:.3f}'
    for p in probs)}]")

```

Step 6: Threshold tuning

```
print("\n=== Threshold Tuning ===")
print("Default threshold: 0.5. Adjusting the threshold trades precision for recall.\n")

thresholds = [0.3, 0.4, 0.5, 0.6, 0.7]
print(f"{'Threshold':>10} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1':>10}")
print("-" * 52)

for t in thresholds:
    y_pred_t = [1 if model.predict_proba(x) >= t else 0 for x in X_test]
    m = ClassificationMetrics(y_test, y_pred_t)
    print(f"{'t':>10.1f} {'m.accuracy():>10.4f} {'m.precision():>10.4f} {'m.recall():>10.4f} {'m.f1():>10.4f}")
```

Use It

Now the same thing with scikit-learn.

```
from sklearn.linear_model import LogisticRegression as SklearnLR
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

np.random.seed(42)
X_0 = np.random.randn(100, 2) + [2, 2]
X_1 = np.random.randn(100, 2) + [5, 5]
X_sk = np.vstack([X_0, X_1])
y_sk = np.array([0] * 100 + [1] * 100)

X_tr, X_te, y_tr, y_te = train_test_split(X_sk, y_sk, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_tr)
X_te_sc = scaler.transform(X_te)

lr = SklearnLR()
lr.fit(X_tr_sc, y_tr)
y_pred = lr.predict(X_te_sc)

print("=== Scikit-learn Logistic Regression ===")
print(f"Accuracy: {accuracy_score(y_te, y_pred):.4f}")
print(f"Precision: {precision_score(y_te, y_pred):.4f}")
print(f"Recall: {recall_score(y_te, y_pred):.4f}")
print(f"F1: {f1_score(y_te, y_pred):.4f}")
```

```
print(f"\nConfusion Matrix:\n{confusion_matrix(y_te, y_pred)}")
print(f"\nClassification Report:\n{classification_report(y_te, y_pred)}")
```

Your from-scratch implementation produces the same decision boundary and metrics. Scikit-learn adds solver options (liblinear, lbfgs, saga), automatic regularization, multi-class strategies (one-vs-rest, multinomial), and numerical stability optimizations.

Exercises

- Generate a dataset that is NOT linearly separable (e.g., two concentric circles). Train logistic regression and observe its failure. Then add polynomial features (x_1^2 , x_2^2 , $x_1 \cdot x_2$) and train again. Show that the accuracy improves.
- Implement a multi-class confusion matrix for the 3-class softmax model. Compute per-class precision and recall. Which class is hardest to classify?
- Build an ROC curve from scratch. For 100 threshold values from 0 to 1, compute the true positive rate and false positive rate. Calculate the AUC (area under the curve) using the trapezoidal rule.

Key Terms

Term	What people say	What it actually means
Logistic regression	“Regression for classification”	A linear model followed by a sigmoid function that outputs class probabilities
Sigmoid function	“The S-curve”	The function $1/(1+e^{-z})$ that maps any real number to the range (0, 1)
Binary cross-entropy	“Log loss”	The loss function $-[y \log(p) + (1-y) \log(1-p)]$ that penalizes confident wrong predictions severely
Decision boundary	“The dividing line”	The surface where the model’s output probability equals 0.5, separating predicted classes
Softmax	“Multi-class sigmoid”	A function that converts a vector of scores into probabilities that sum to 1
Precision	“How many selected are relevant”	$TP / (TP + FP)$, the fraction of positive predictions that are actually positive
Recall	“How many relevant are selected”	$TP / (TP + FN)$, the fraction of actual positives that the model correctly identifies
F1 score	“Balanced accuracy”	The harmonic mean of precision and recall: $2PR / (P+R)$
Confusion matrix	“The error breakdown”	A table showing TP, TN, FP, FN counts for each class pair

Term	What people say	What it actually means
Threshold	"The cutoff"	The probability value above which the model predicts class 1 (default 0.5, tunable)
One-hot encoding	"Binary columns for categories"	Representing class k as a vector of zeros with a 1 at position k
Categorical cross-entropy	"Multi-class log loss"	The extension of binary cross-entropy to k classes using one-hot encoded labels

Classification Baseline Guide

Before trying complex models, establish a baseline with logistic regression. It trains in seconds, produces probabilities, and is fully interpretable. A surprising number of real-world problems never need anything fancier.

Decision Checklist

- Is the decision boundary likely linear?
 5. Yes: logistic regression will probably be sufficient
 6. No: you still want it as a baseline to measure improvement
- How many features do you have?
 - Under 50: standard logistic regression works fine
 - 50 to 10,000: add L2 regularization (Ridge)
 - Over 10,000 (e.g., TF-IDF text features): use L1 regularization (Lasso) or LinearSVC
- Is the dataset imbalanced?
 7. Under 5:1 ratio: probably fine without adjustment
 8. 5:1 to 50:1: use `class_weight="balanced"` in sklearn
 9. Over 50:1: combine class weighting with appropriate metric (precision, recall, or F1)
- Are features on different scales?
 - Always standardize before logistic regression. It uses gradient-based optimization, and unscaled features slow convergence or distort the decision boundary.
- Are there missing values?
 10. Impute before fitting. Logistic regression cannot handle NaNs.
 11. Use median imputation for numeric columns, mode for categorical.

When logistic regression is good enough

Binary classification with mostly linear feature relationships

You need probability outputs (not just class labels)

Interpretability is required (coefficients indicate feature importance direction and relative magnitude after standardization)

Training data is small (hundreds to low thousands of samples)

You need a fast model for real-time serving (single dot product at inference)

Regulatory or compliance requirements demand explainability

When to upgrade

Accuracy plateaus well below the target and you have tried feature engineering

The relationship between features and target is clearly nonlinear (check residual plots)

You have large tabular data (10k+ rows): try gradient boosting (XGBoost or LightGBM)

Features have complex interactions that polynomial features cannot capture

You have image, text, or sequential data: logistic regression on raw inputs will not work

Preprocessing steps for a classification baseline

6. **Train/test split** first, before any preprocessing. This prevents data leakage.
7. **Handle missing values**: median impute numeric, mode impute categorical.
8. **Encode categoricals**: one-hot for low cardinality (under 10 values), target encoding for higher. Fit target encoding only on training folds (use out-of-fold encoding to prevent leakage).
9. **Scale numerics**: StandardScaler (zero mean, unit variance). Fit on train, transform both.
10. **Fit logistic regression** with $C=1.0$ (default regularization).
11. **Evaluate**: confusion matrix, precision, recall, F1. Not just accuracy.
12. **Tune threshold**: default 0.5 is rarely optimal. Sweep 0.1 to 0.9 and pick the threshold that matches your precision/recall priority.

Common mistakes

Evaluating only accuracy on imbalanced data (a model predicting the majority class scores high but is useless)

Forgetting to scale features (logistic regression with unscaled features trains slowly and converges to a worse solution)

Using the test set to tune the decision threshold (use validation or cross-validation)

Skipping the baseline and jumping straight to XGBoost (you lose interpretability and have no reference point)

Not checking for multicollinearity (highly correlated features inflate coefficient variance)

Quick reference

Scenario	Model	Regularization	Key setting
Few features, interpretable	LogisticRegression	L2 (default)	C=1.0
Many features, some irrelevant	LogisticRegression	L1	penalty="l1", solver="saga"
High-dim sparse (text)	SGDClassifier	L1 or ElasticNet	loss="log_loss"
Imbalanced classes	LogisticRegression	L2	class_weight="balanced"
Need probabilities	LogisticRegression	L2	predict_proba()
Need class labels only	LinearSVC	L2	Faster than LR for large data

Quiz

Pre-lesson check

What is the range of the sigmoid function's output?

- r) Negative ∞ to positive ∞
- s) 0 to 1 (exclusive) **(correct)**
- t) -1 to 1
- u) 0 to positive ∞

Explanation: The sigmoid function $1/(1+e^{(-z)})$ outputs values strictly between 0 and 1, which can be interpreted as probabilities.

Why is logistic regression called 'regression' even though it is used for classification?

- It predicts continuous values that are then rounded
- The name comes from the logistic (sigmoid) function it uses, not from regression analysis **(correct)**
- It was originally designed for regression and later adapted

- It minimizes mean squared error like linear regression
Explanation: The name comes from the logistic function (sigmoid). Despite the name, logistic regression is a classification algorithm that outputs class probabilities.

Post-lesson check

22. Why is binary cross-entropy used instead of MSE for logistic regression?

1. Cross-entropy is faster to compute
2. MSE with sigmoid creates a non-convex cost surface with local minima, while cross-entropy is convex **(correct)**
3. MSE can only be used with linear models

4. Cross-entropy works only when the dataset is balanced
Explanation: MSE combined with the sigmoid activation creates a non-convex cost surface with many local minima. Binary cross-entropy with sigmoid is convex, guaranteeing a single global minimum.

23. A spam filter has precision = 0.95 and recall = 0.60. What does this mean in practical terms?

1. 95% of all emails are correctly classified, and 60% of spam is caught
2. When it flags an email as spam, it is correct 95% of the time, but it only catches 60% of actual spam **(correct)**
3. 60% of flagged emails are spam, and 95% of all spam is caught

4. The model is 95% accurate on the test set and 60% on the training set
Explanation: Precision = 0.95 means 95% of emails predicted as spam are actually spam (few false alarms). Recall = 0.60 means only 60% of actual spam is caught (40% slips through).

24. In softmax regression for 4 classes, what is true about the output probabilities?

- Each class gets an independent probability between 0 and 1
- The four output probabilities sum to 1, and the class with the highest probability is the prediction **(correct)**

- Only the top-2 classes receive nonzero probabilities

- The outputs are raw scores, not probabilities

Explanation: Softmax converts a vector of raw scores into probabilities that sum to 1. The predicted class is the one with the highest probability.

Decision Trees and Random Forests

A decision tree is just a flowchart. But a forest of them is one of the most powerful tools in ML.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lessons 09 Information Theory, 06 Probability)

Time: ~90 minutes

Learning Objectives

25. Implement Gini impurity, entropy, and information gain calculations to find optimal decision tree splits
26. Build a decision tree classifier from scratch with pre-pruning controls (max depth, min samples)
27. Construct a random forest using bootstrap sampling and feature randomization, and explain why it reduces variance
28. Compare MDI feature importance with permutation importance and identify when MDI is biased

The Problem

You have tabular data. Rows are samples, columns are features, and there is a target column you want to predict. You could throw a neural network at it. But for tabular data, tree-based models (decision trees, random forests, gradient boosted trees) consistently outperform deep learning. Kaggle competitions on structured data are dominated by XGBoost and LightGBM, not transformers.

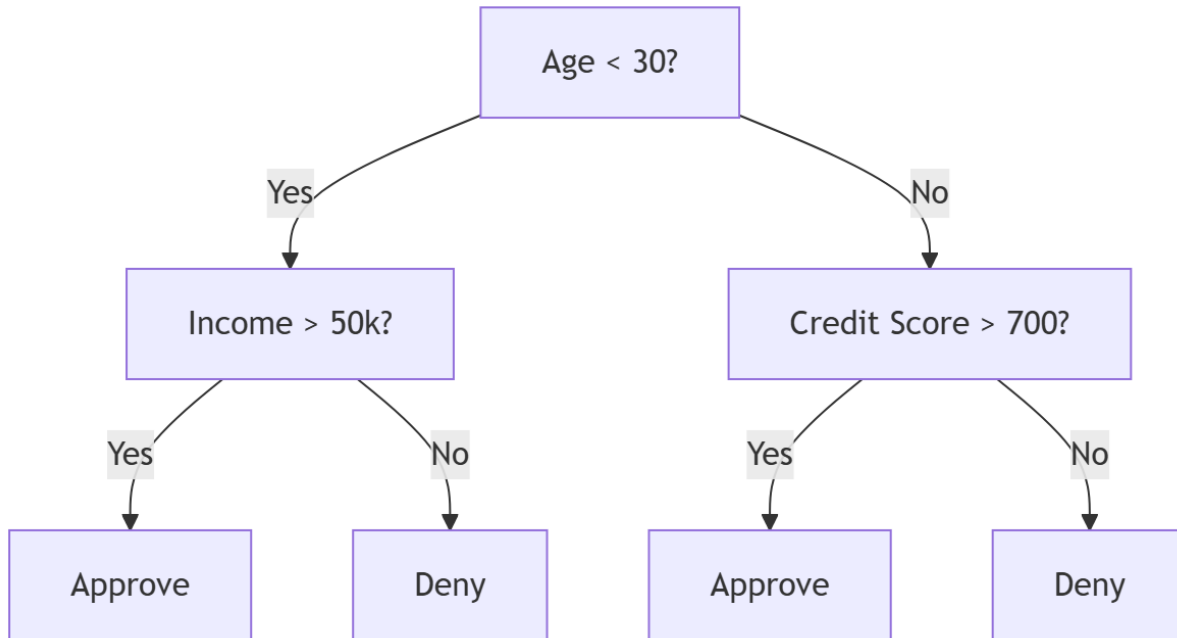
Why? Trees handle mixed feature types (numeric and categorical) without preprocessing. They handle nonlinear relationships without feature engineering. They are interpretable: you can look at the tree and see exactly why a prediction was made. And random forests, which average many trees, are highly resistant to overfitting on moderate-sized datasets.

This lesson builds decision trees from scratch using recursive splitting, then builds a random forest on top. You will implement the math behind split criteria (Gini impurity, entropy, information gain) and understand why an ensemble of weak learners becomes a strong one.

The Concept

What a decision tree does

A decision tree partitions the feature space into rectangular regions by asking a sequence of yes/no questions.



Each internal node tests a feature against a threshold. Each leaf node makes a prediction. To classify a new data point, you start at the root and follow the branches until you reach a leaf.

The tree is built top-down by choosing, at each node, the feature and threshold that best separate the data. “Best” is defined by a split criterion.

Split criteria: measuring impurity

At each node, we have a set of samples. We want to split them so that the resulting child nodes are as “pure” as possible, meaning each child contains mostly one class.

Gini impurity measures the probability that a randomly chosen sample would be misclassified if it were labeled according to the class distribution at that node.

$$\text{Gini}(S) = 1 - \sum(p_k^2)$$

where p_k is the proportion of class k in set S .

For a pure node (all one class), Gini = 0. For a binary split with 50/50 classes, Gini = 0.5. Lower is better.

Example: 6 cats, 4 dogs

$$\text{Gini} = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$$

Entropy measures the information content (disorder) in a node. Covered in Phase 1 Lesson 09.

$$\text{Entropy}(S) = -\sum(p_k * \log_2(p_k))$$

For a pure node, entropy = 0. For a 50/50 binary split, entropy = 1.0. Lower is better.

Example: 6 cats, 4 dogs

$$\begin{aligned}\text{Entropy} &= -(0.6 * \log_2(0.6) + 0.4 * \log_2(0.4)) \\ &= -(0.6 * -0.737 + 0.4 * -1.322) \\ &= 0.442 + 0.529 \\ &= 0.971 \text{ bits}\end{aligned}$$

Information gain is the reduction in impurity (entropy or Gini) after a split.

$$\text{IG}(S, \text{feature}, \text{threshold}) = \text{Impurity}(S) - \text{weighted_avg}(\text{Impurity}(S_{\text{left}}), \text{Impurity}(S_{\text{right}}))$$

where the weights are the proportions of samples in each child.

The greedy algorithm at each node: try every feature and every possible threshold. Pick the (feature, threshold) pair that maximizes information gain.

How splitting works

For a dataset with n features and m samples at the current node:

- For each feature j (j = 1 to n):
 - Sort the samples by feature j
 - Try every midpoint between consecutive distinct values as a threshold
 - Compute the information gain for each threshold
- Select the feature and threshold with the highest information gain
- Split the data into left (feature <= threshold) and right (feature > threshold)
- Recurse on each child

This greedy approach does not guarantee the globally optimal tree. Finding the optimal tree is NP-hard. But greedy splitting works well in practice.

Stopping conditions

Without stopping conditions, the tree grows until every leaf is pure (one sample per leaf). This perfectly memorizes the training data and generalizes terribly.

Pre-pruning stops the tree before it fully grows: - Maximum depth: stop splitting when the tree reaches a set depth - Minimum samples per leaf: stop if a node has fewer than k samples - Minimum information gain: stop if the best split improves impurity by less than a threshold - Maximum leaf nodes: limit the total number of leaves

Post-pruning grows the full tree, then trims it back: - Cost-complexity pruning (used by scikit-learn): adds a penalty proportional to the number of leaves. Increase the penalty to get smaller trees - Reduced error pruning: remove a subtree if the validation error does not increase

Pre-pruning is simpler and faster. Post-pruning often produces better trees because it does not prematurely stop splits that might lead to useful further splits.

Decision trees for regression

For regression, the leaf prediction is the mean of the target values in that leaf. The split criterion changes too:

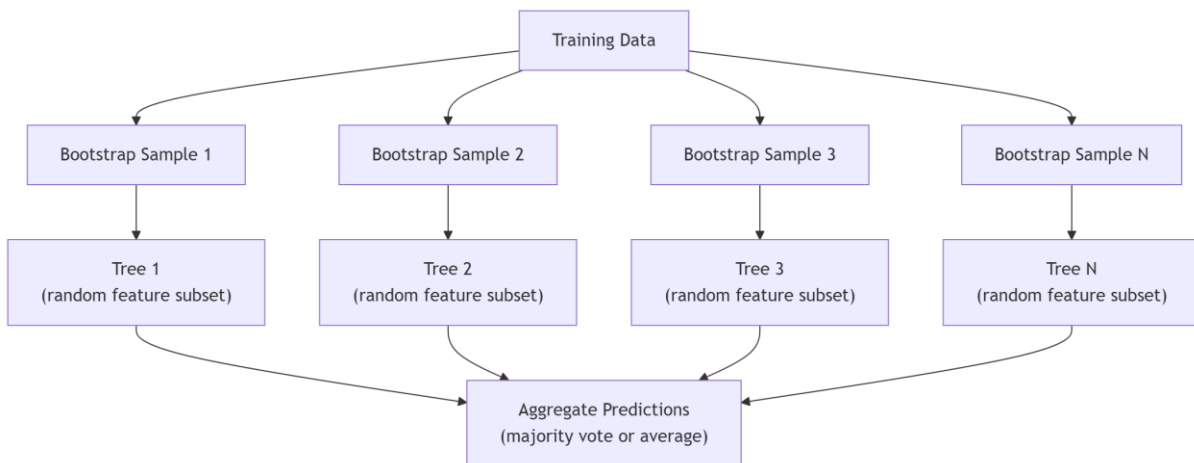
Variance reduction replaces information gain:

$$VR(S, \text{feature}, \text{threshold}) = \text{Var}(S) - \text{weighted_avg}(\text{Var}(S_{\text{left}}), \text{Var}(S_{\text{right}}))$$

Pick the split that reduces variance the most. The tree partitions the input space into regions, and predicts a constant (the mean) in each region.

Random forests: the power of ensembles

A single decision tree is high variance. Small changes in the data can produce completely different trees. Random forests fix this by averaging many trees.



Two sources of randomness make the trees diverse:

Bagging (bootstrap aggregating): Each tree is trained on a bootstrap sample, a random sample with replacement from the training data. About 63% of the original samples appear in each bootstrap (the rest are out-of-bag samples that can be used for validation).

Feature randomization: At each split, only a random subset of features is considered. For classification, the default is $\sqrt{n_features}$. For regression, $n_features/3$. This prevents all trees from splitting on the same dominant feature.

The key insight: averaging many decorrelated trees reduces variance without increasing bias. Each individual tree may be mediocre. The ensemble is strong.

Feature importance

Random forests naturally provide feature importance scores. The most common method:

Mean Decrease in Impurity (MDI): For each feature, sum the total reduction in impurity across all trees and all nodes where that feature is used. Features that produce bigger impurity reductions at earlier splits are more important.

```
importance(feature_j) = sum over all nodes where feature_j is used:  
    (n_samples_at_node / n_total_samples) * impurity_decrease
```

This is fast (computed during training) but biased toward high-cardinality features and features with many possible split points.

Permutation importance is the alternative: shuffle one feature's values and measure how much the model's accuracy drops. More reliable but slower.

When trees beat neural networks

Trees and forests dominate neural networks on tabular data. Several reasons:

Factor	Trees	Neural networks
Mixed types (numeric + categorical)	Native support	Need encoding
Small datasets (< 10k rows)	Work well	Overfit
Feature interactions	Found by splitting	Need architecture design
Interpretability	Full transparency	Black box
Training time	Minutes	Hours
Hyperparameter sensitivity	Low	High

Neural networks win when the data has spatial or sequential structure (images, text, audio). For flat tables of features, trees are the default.

Build It

Step 1: Gini impurity and entropy

Build both split criteria from scratch and verify they agree on which splits are good.

```
import math

def gini_impurity(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    return 1.0 - sum((c / n) ** 2 for c in counts.values())

def entropy(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    return -sum(
        (c / n) * math.log2(c / n) for c in counts.values() if c > 0
    )
```

Step 2: Find the best split

Try every feature and every threshold. Return the one with the highest information gain.

```
def information_gain(parent_labels, left_labels, right_labels, criterion="gini"):
    measure = gini_impurity if criterion == "gini" else entropy
    n = len(parent_labels)
    n_left = len(left_labels)
    n_right = len(right_labels)
    if n_left == 0 or n_right == 0:
        return 0.0
    parent_impurity = measure(parent_labels)
    child_impurity = (
        (n_left / n) * measure(left_labels) +
        (n_right / n) * measure(right_labels)
    )
    return parent_impurity - child_impurity
```

Step 3: Build the DecisionTree class

Recursive splitting, prediction, and feature importance tracking.

```

class DecisionTree:
    def __init__(self, max_depth=None, min_samples_split=2,
                  min_samples_leaf=1, criterion="gini",
                  max_features=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion
        self.max_features = max_features
        self.tree = None
        self.feature_importances_ = None

    def fit(self, X, y):
        self.n_features = len(X[0])
        self.feature_importances_ = [0.0] * self.n_features
        self.n_samples = len(X)
        self.tree = self._build(X, y, depth=0)
        total = sum(self.feature_importances_)
        if total > 0:
            self.feature_importances_ = [
                fi / total for fi in self.feature_importances_
            ]

    def predict(self, X):
        return [self._predict_one(x, self.tree) for x in X]

```

Step 4: Build the RandomForest class

Bootstrap sampling, feature randomization, and majority voting.

```

class RandomForest:
    def __init__(self, n_trees=100, max_depth=None,
                  min_samples_split=2, max_features="sqrt",
                  criterion="gini"):
        self.n_trees = n_trees
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.criterion = criterion
        self.trees = []

    def fit(self, X, y):
        n = len(X)
        for _ in range(self.n_trees):
            indices = [random.randint(0, n - 1) for _ in range(n)]
            X_boot = [X[i] for i in indices]
            y_boot = [y[i] for i in indices]
            tree = DecisionTree(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split,
                max_features=self.max_features,
                criterion=self.criterion,

```



```

    )
    tree.fit(X_boot, y_boot)
    self.trees.append(tree)

def predict(self, X):
    all_preds = [tree.predict(X) for tree in self.trees]
    predictions = []
    for i in range(len(X)):
        votes = {}
        for preds in all_preds:
            v = preds[i]
            votes[v] = votes.get(v, 0) + 1
        predictions.append(max(votes, key=votes.get))
    return predictions

```

See `code/trees.py` for the complete implementation with all helper methods.

Use It

With scikit-learn, training a random forest is three lines:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print(f"Accuracy: {rf.score(X_test, y_test):.4f}")
print(f"Feature importances: {rf.feature_importances_}")

```

In practice, gradient boosted trees (XGBoost, LightGBM, CatBoost) are often stronger than random forests because they build trees sequentially, with each tree correcting the errors of the previous ones. But random forests are harder to misconfigure and require almost no hyperparameter tuning.

Exercises

- Train a single decision tree on a 2D dataset with 3 classes. Manually trace the splits and draw the rectangular decision boundaries. Compare the boundaries at `max_depth=2` vs `max_depth=10`.
- Implement variance reduction splitting for regression trees. Generate $y = \sin(x) + \text{noise}$ for 200 points and fit your regression tree. Plot the tree's piecewise-constant predictions against the true curve.
- Build a random forest with 1, 5, 10, 50, and 200 trees. Plot training accuracy and test accuracy vs number of trees. Observe that test accuracy plateaus but does not decrease (forests resist overfitting).

- Compare Gini impurity vs entropy as split criteria on 5 different datasets. Measure accuracy and tree depth. In most cases, they produce nearly identical results. Explain why.
- Implement permutation importance. Compare it with MDI importance on a dataset where one feature is random noise but has high cardinality. MDI will rank the noise feature highly. Permutation importance will not.

Key Terms

Term	What people say	What it actually means
Decision tree	“A flowchart for predictions”	A model that partitions feature space into rectangular regions by learning a sequence of if/else splits
Gini impurity	“How mixed the node is”	Probability of misclassifying a random sample at a node. 0 = pure, 0.5 = maximum impurity for binary
Entropy	“The disorder in a node”	Information content at a node. 0 = pure, 1.0 = maximum uncertainty for binary. From information theory
Information gain	“How good a split is”	Reduction in impurity after a split. The greedy criterion for choosing splits
Pre-pruning	“Stop the tree early”	Stopping tree growth early by setting max depth, min samples, or min gain thresholds
Post-pruning	“Trim the tree after”	Growing the full tree, then removing subtrees that do not improve validation performance
Bagging	“Train on random subsets”	Bootstrap aggregating. Train each model on a different random sample with replacement
Random forest	“A bunch of trees”	Ensemble of decision trees, each trained on a bootstrap sample with random feature subsets at each split
Feature importance (MDI)	“Which features matter”	Total impurity decrease contributed by each feature, summed across all trees and nodes
Permutation importance	“Shuffle and check”	Accuracy drop when a feature’s values are randomly shuffled. More reliable than MDI for noisy features
Variance reduction	“The regression version of info gain”	The regression tree analogue of information gain. Picks the split that reduces target variance the most
Bootstrap sample	“Random sample with repeats”	A random sample drawn with replacement from the original dataset. Same size, but with duplicates

You are a decision tree interpreter. Given information about a trained decision tree (depth, features used, split points, accuracy), you explain what the model learned, identify the most important features, and flag potential problems.

When a user provides decision tree results, work through each section below.

Step 1: Summarize the tree structure

State: - Total depth of the tree - Number of leaf nodes - Which features appear in the top 3 levels of splits (these are the most influential) - The root split: which feature and threshold the model found most informative overall

If the tree is deeper than 6 levels on a dataset with fewer than 1,000 samples, flag this as likely overfitting.

Step 2: Identify the most important features

Rank features by their contribution. Two methods:

By split position: features used at the root and early levels have the highest information gain across the entire dataset. Later splits act on smaller subsets and contribute less.

By impurity decrease (MDI): if feature importance scores are provided, rank them. Note that MDI is biased toward high-cardinality features (features with many unique values get more split opportunities).

State which features the model relies on most and whether this makes domain sense.

Step 3: Explain what the model learned

Translate the tree into plain language rules. For example: - "The strongest signal is age. Customers under 30 with income above 50k are predicted to buy." - "The model splits on feature X first, then refines using Y. Feature Z appears only in deep leaves and likely captures noise."

Highlight any splits that seem counterintuitive or domain-questionable.

Step 4: Diagnose potential issues

Check for each of these problems:

Overfitting signals: - Training accuracy much higher than test accuracy (gap > 10%) - Tree depth exceeds $\sqrt{n_{\text{samples}}}$ - Many leaves contain just 1-2 samples - Fix: reduce max_depth, increase min_samples_leaf, or use pruning

Underfitting signals: - Both training and test accuracy are low - Tree is too shallow (depth 1-2) for a complex problem - Fix: increase max_depth, reduce min_samples constraints

Class imbalance effects: - The tree may ignore the minority class entirely - Check per-class accuracy, not just overall accuracy - Fix: use class_weight="balanced" or resample the data

Feature leakage: - One feature has near-perfect splits at the root - If a single feature gives 99% accuracy, verify it is not encoding the target

High-cardinality bias: - If a feature with many unique values (like an ID column or zip code) appears important, MDI importance may be misleading - Verify with permutation importance: shuffle the feature and measure accuracy drop

Step 5: Recommend next steps

Based on the diagnosis: - If overfitting: suggest random forest (reduces variance through bagging) - If underfitting: suggest deeper tree or gradient boosting - If accuracy is good: suggest comparing with a random forest to see if the ensemble improves further - If interpretability matters: keep the pruned tree and document the rules

Output format

Structure your response as: 1. **Tree summary:** depth, leaves, top features 2. **Key rules:** 2-3 plain-language decision rules the tree learned 3. **Feature ranking:** ordered list with importance scores or split positions 4. **Issues found:** any overfitting, leakage, or imbalance concerns 5. **Recommendation:** what to try next

Avoid: - Reporting only overall accuracy without per-class breakdown - Ignoring the possibility of data leakage when a single feature dominates - Treating deep, unpruned trees as the final model - Trusting MDI importance without questioning high-cardinality bias

Quiz

Pre-lesson check

What does Gini impurity measure at a decision tree node?

- v) The depth of the node in the tree
- w) The probability of misclassifying a randomly chosen sample given the class distribution at that node (**correct**)
- x) The total number of samples at the node
- y) The correlation between features

Explanation: Gini impurity = $1 - \sum(p_k^2)$. It measures how often a randomly chosen sample would be misclassified if labeled according to the class distribution at that node. Pure node = 0.

What is the main advantage of tree-based models over neural networks for tabular data?

- 3. Trees can process images and text natively
- 4. Trees always have lower bias than neural networks
- 5. Trees handle mixed feature types, require less preprocessing, and are more interpretable **(correct)**
- 6. Trees train faster on GPU hardware
Explanation: Trees natively handle numeric and categorical features without encoding, require minimal preprocessing, and produce interpretable rules. Neural networks excel at spatial/sequential data, not flat tables.

Post-lesson check

Why does a random forest use both bootstrap sampling AND random feature subsets at each split?

- z) To speed up training by reducing the dataset size
- aa) To create diverse, decorrelated trees so that averaging reduces variance without increasing bias **(correct)**
- bb) To ensure each tree sees every data point at least once
- cc) To reduce the depth of individual trees
Explanation: Both sources of randomness make the trees diverse. Without feature randomization, all trees would split on the same dominant feature. Diversity is what makes averaging effective at reducing variance.

A node contains 8 dogs and 2 cats. What is its Gini impurity?

- dd) 0.0
- ee) 0.20
- ff) 0.32 **(correct)**

gg) 0.50

Explanation: $Gini = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 0.32$. The node is mostly dogs but not pure, so Gini is between 0 (pure) and 0.5 (max for binary).

MDI (Mean Decrease in Impurity) feature importance is biased toward which type of feature?

- Binary features with only two values
- Features with low variance
- High-cardinality features with many possible split points **(correct)**
- Features that are highly correlated with the target

Explanation: MDI is biased toward high-cardinality features because they offer more possible split points, giving them more chances to reduce impurity by luck. Permutation importance is more reliable.

Support Vector Machines

Find the widest street between two classes. That is the entire idea.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lessons 08 Optimization, 14 Norms and Distances, 18 Convex Optimization) **Time:** ~90 minutes

Learning Objectives

29. Implement a linear SVM from scratch using hinge loss and gradient descent on the primal formulation
30. Explain the maximum margin principle and identify support vectors from a trained model
31. Compare linear, polynomial, and RBF kernels and explain how the kernel trick avoids explicit high-dimensional mapping
32. Evaluate the tradeoff controlled by the C parameter between margin width and classification errors

The Problem

You have two classes of data points and need to draw a line (or hyperplane) separating them. Infinitely many lines could work. Which one should you pick?

The one with the biggest margin. The margin is the distance between the decision boundary and the nearest data points on each side. A wider margin means the classifier is more confident and generalizes better to unseen data.

This intuition leads to Support Vector Machines, one of the most mathematically elegant algorithms in ML. SVMs were the dominant classification method before deep learning and remain the best choice for small datasets, high-dimensional data, and problems where you need a principled, well-understood model with theoretical guarantees.

SVMs connect directly to Phase 1: the optimization is convex (Lesson 18), the margin is measured with norms (Lesson 14), and the kernel trick exploits dot products to handle nonlinear boundaries without ever computing in the high-dimensional space.

The Concept

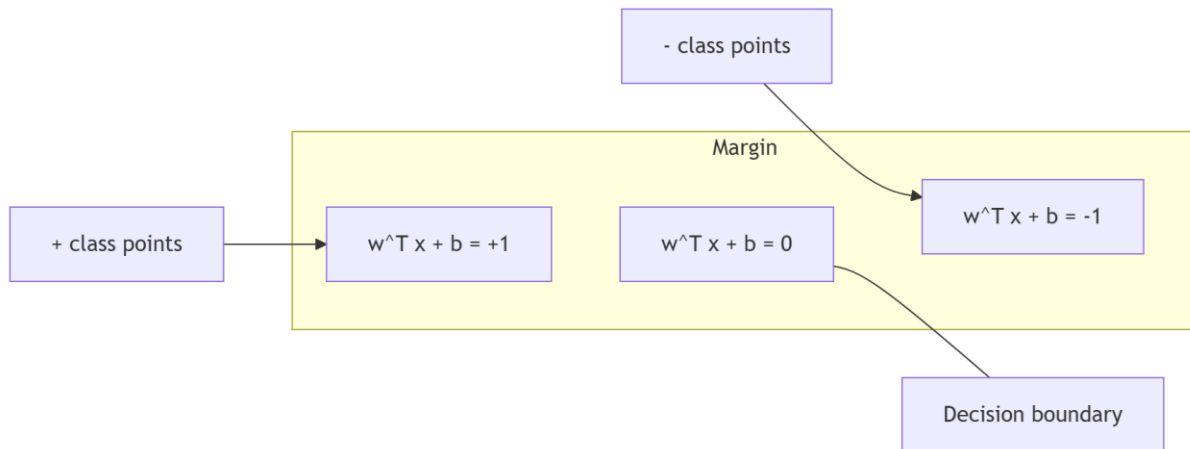
The maximum margin classifier

Given linearly separable data with labels y_i in $\{-1, +1\}$ and feature vectors x_i , we want a hyperplane $w^T x + b = 0$ that separates the classes.

The distance from a point x_i to the hyperplane is:

$$\text{distance} = |w^T x_i + b| / \|w\|$$

For a correctly classified point: $y_i * (w^T x_i + b) > 0$. The margin is twice the distance from the hyperplane to the nearest point on either side.



The optimization problem:

```

maximize    2 / ||w||      (the margin width)
subject to  y_i * (w^T x_i + b) >= 1  for all i
  
```

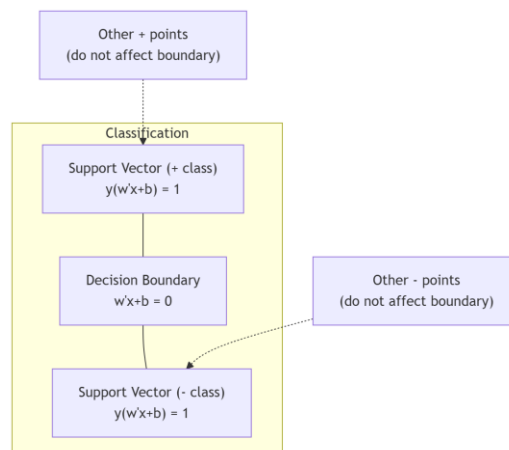
Equivalently (minimizing $||w||^2$ is easier to optimize):

```

minimize    (1/2) ||w||^2
subject to  y_i * (w^T x_i + b) >= 1  for all i
  
```

This is a convex quadratic program. It has a unique global solution. The data points that sit exactly on the margin boundaries (where $y_i * (w^T x_i + b) = 1$) are the support vectors. They are the only points that determine the decision boundary. Move or remove any non-support-vector point, and the boundary does not change.

Support vectors: the critical few



Most training points are irrelevant. Only the support vectors matter. This is why SVMs are memory-efficient at prediction time: you only need to store the support vectors, not the entire training set.

The number of support vectors also gives a bound on generalization error. Fewer support vectors relative to the dataset size means better generalization.

Soft margin: handling noise with the C parameter

Real data is rarely perfectly separable. Some points may be on the wrong side of the boundary, or inside the margin. The soft margin formulation allows violations by introducing slack variables.

```

minimize    (1/2) ||w||^2 + C * sum(xi_i)
subject to  y_i * (w^T x_i + b) >= 1 - xi_i
            xi_i >= 0 for all i

```

The slack variable ξ_i measures how much point i violates the margin. C controls the trade-off:

C value	Behavior
Large C	Penalizes violations heavily. Narrow margin, fewer misclassifications. Overfits
Small C	Allows more violations. Wide margin, more misclassifications. Underfits

C is the regularization strength, inverted. Large C = less regularization. Small C = more regularization.

Hinge loss: the SVM loss function

The soft margin SVM can be rewritten as an unconstrained optimization:

```

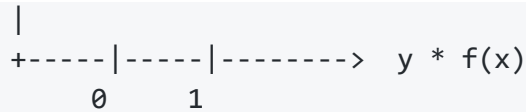
minimize    (1/2) ||w||^2 + C * sum(max(0, 1 - y_i * (w^T x_i + b)))

```

The term $\max(0, 1 - y_i * f(x_i))$ is the hinge loss. It is zero when the point is correctly classified and beyond the margin. It is linear when the point is inside the margin or misclassified.

Hinge loss for a single point:





Zero loss when $y \cdot f(x) \geq 1$ (correctly classified, outside margin).
Linear penalty when $y \cdot f(x) < 1$.

Compare with logistic loss (logistic regression):

Hinge:	$\max(0, 1 - y \cdot f(x))$	Hard cutoff at margin
Logistic:	$\log(1 + \exp(-y \cdot f(x)))$	Smooth, never exactly zero

Hinge loss produces sparse solutions (only support vectors have nonzero contribution). Logistic loss uses all data points. This makes SVMs more memory-efficient at prediction time.

Training a linear SVM with gradient descent

You can train a linear SVM using gradient descent on the hinge loss plus L2 regularization, without solving the constrained QP:

$$L(w, b) = (\lambda/2) * ||w||^2 + (1/n) * \sum(\max(0, 1 - y_i * (w^T x_i + b)))$$

Gradient with respect to w :

If $y_i * (w^T x_i + b) \geq 1$: $dL/dw = 0$

If $y_i * (w^T x_i + b) < 1$: $dL/dw = \lambda w - y_i * x_i$

Gradient with respect to b :

If $y_i * (w^T x_i + b) \geq 1$: $dL/db = 0$

If $y_i * (w^T x_i + b) < 1$: $dL/db = -y_i$

This is called the primal formulation. It runs in $O(n * d)$ per epoch, where n is the number of samples and d is the number of features. For large, sparse, high-dimensional data (text classification), this is fast.

The dual formulation and the kernel trick

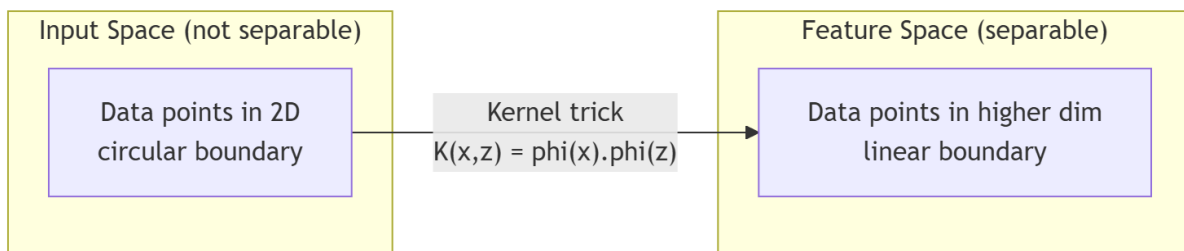
The Lagrangian dual of the SVM problem (from Phase 1 Lesson 18, KKT conditions) is:

$$\begin{aligned} &\text{maximize} && \sum(\alpha_i) - (1/2) * \sum_{ij}(\alpha_i * \alpha_j * y_i * y_j * (x_i \cdot x_j)) \\ &\text{subject to} && 0 \leq \alpha_i \leq C \\ &&& \sum(\alpha_i * y_i) = 0 \end{aligned}$$

The dual only involves dot products $x_i \cdot x_j$ between data points. This is the key insight. Replace every dot product with a kernel function $K(x_i, x_j)$ and the SVM can learn nonlinear boundaries without ever computing the transformation explicitly.

Linear kernel: $K(x, z) = x \cdot z$
 Polynomial kernel: $K(x, z) = (x \cdot z + c)^d$
 RBF (Gaussian): $K(x, z) = \exp(-\gamma \|x - z\|^2)$

The RBF kernel maps data into an infinite-dimensional space. Points that are close in input space have kernel value near 1. Points that are far apart have kernel value near 0. It can learn any smooth decision boundary.



The kernel trick computes the dot product in the high-dimensional space without ever going there. For the polynomial kernel of degree d in D dimensions, the explicit feature space has $O(D^d)$ dimensions. But $K(x, z)$ is computed in $O(D)$ time.

SVM for regression (SVR)

Support Vector Regression fits a tube of width ϵ around the data. Points inside the tube have zero loss. Points outside the tube are penalized linearly.

```

minimize    (1/2) ||w||^2 + C * sum(xi_i + xi_i*)
subject to  y_i - (w^T x_i + b) <= epsilon + xi_i
            (w^T x_i + b) - y_i <= epsilon + xi_i*
            xi_i, xi_i* >= 0
    
```

The ϵ parameter controls the tube width. Wider tube = fewer support vectors = smoother fit. Narrower tube = more support vectors = tighter fit.

Why SVMs lost to deep learning (and when they still win)

SVMs dominated ML from the late 1990s through the early 2010s. Deep learning surpassed them for several reasons:

Factor	SVMs	Deep learning
Feature engineering	Requires it	Learns features
Scalability	$O(n^2)$ to $O(n^3)$ for kernel	$O(n)$ per epoch with SGD
Image/text/audio	Needs handcrafted features	Learns from raw data
Large datasets (>100k)	Slow	Scales well
GPU acceleration	Limited benefit	Massive speedup

SVMs still win in these situations: - Small datasets (hundreds to low thousands of samples) - High-dimensional sparse data (text with TF-IDF features) - When you need mathematical guarantees (margin bounds) - When training time must be minimal (linear SVM is very fast) - Binary classification with clear margin structure - Anomaly detection (one-class SVM)

Build It

Step 1: Hinge loss and gradient

The foundation. Compute hinge loss for a batch and its gradient.

```
def hinge_loss(X, y, w, b):
    n = len(X)
    total_loss = 0.0
    for i in range(n):
        margin = y[i] * (dot(w, X[i]) + b)
        total_loss += max(0.0, 1.0 - margin)
    return total_loss / n
```

Step 2: Linear SVM via gradient descent

Train by minimizing regularized hinge loss. No QP solver needed.

```
class LinearSVM:
    def __init__(self, lr=0.001, lambda_param=0.01, n_epochs=1000):
        self.lr = lr
        self.lambda_param = lambda_param
        self.n_epochs = n_epochs
        self.w = None
        self.b = 0.0

    def fit(self, X, y):
        n_features = len(X[0])
        self.w = [0.0] * n_features
        self.b = 0.0
```

```

    for epoch in range(self.n_epochs):
        for i in range(len(X)):
            margin = y[i] * (dot(self.w, X[i]) + self.b)
            if margin >= 1:
                self.w = [wj - self.lr * self.lambda_param * wj
                           for wj in self.w]
            else:
                self.w = [wj - self.lr * (self.lambda_param * wj - y[i] * X[i][j])
                           for j, wj in enumerate(self.w)]
                self.b -= self.lr * (-y[i])

    def predict(self, X):
        return [1 if dot(self.w, x) + self.b >= 0 else -1 for x in X]

```

Step 3: Kernel functions

Implement linear, polynomial, and RBF kernels.

```

def linear_kernel(x, z):
    return dot(x, z)

def polynomial_kernel(x, z, degree=3, c=1.0):
    return (dot(x, z) + c) ** degree

def rbf_kernel(x, z, gamma=0.5):
    diff = [xi - zi for xi, zi in zip(x, z)]
    return math.exp(-gamma * dot(diff, diff))

```

Step 4: Margin and support vector identification

After training, identify which points are support vectors and compute the margin width.

```

def find_support_vectors(X, y, w, b, tol=1e-3):
    support_vectors = []
    for i in range(len(X)):
        margin = y[i] * (dot(w, X[i]) + b)
        if abs(margin - 1.0) < tol:
            support_vectors.append(i)
    return support_vectors

```

See code/svm.py for the complete implementation with all demos.

Use It

With scikit-learn:

```

from sklearn.svm import SVC, LinearSVC, SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

```

```

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=1.0, gamma="scale")),
])
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.4f}")
print(f"Support vectors: {clf['svm'].n_support_}")

```

Important: always scale your features before training an SVM. SVMs are sensitive to feature magnitudes because the margin depends on $\|w\|$, and unscaled features distort the geometry.

For large datasets, use `LinearSVC` (primal formulation, $O(n)$ per epoch) instead of `SVC` (dual formulation, $O(n^2)$ to $O(n^3)$):

```

from sklearn.svm import LinearSVC

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", LinearSVC(C=1.0, max_iter=10000)),
])

```

Exercises

- Generate a 2D linearly separable dataset. Train your LinearSVM and identify the support vectors. Verify that the support vectors are the points closest to the decision boundary.
- Vary C from 0.001 to 1000 on a noisy dataset. Plot the decision boundary for each C value. Observe the transition from wide margin (underfitting) to narrow margin (overfitting).
- Create a dataset where class boundaries are circular (not linear). Show that a linear SVM fails. Compute the RBF kernel matrix and show that the classes become separable in the kernel-induced feature space.
- Compare hinge loss vs logistic loss on the same dataset. Train a linear SVM and logistic regression. Count how many training points contribute to each model's decision boundary (support vectors vs all points).
- Implement SVR (ϵ -insensitive loss). Fit it to $y = \sin(x) + \text{noise}$. Plot the ϵ tube around the predictions and highlight the support vectors (points outside the tube).

Key Terms

Term	What it actually means
Support vectors	The training points closest to the decision boundary. The only points that determine the hyperplane

Term	What it actually means
Margin	The distance between the decision boundary and the nearest support vectors. SVMs maximize this
Hinge loss	$\max(0, 1 - y \cdot f(x))$. Zero when correctly classified and outside the margin. Linear penalty otherwise
C parameter	Trade-off between margin width and classification errors. Large C = narrow margin, small C = wide margin
Soft margin	SVM formulation that allows margin violations via slack variables. Handles non-separable data
Kernel trick	Computing dot products in a high-dimensional feature space without explicitly mapping to that space
Linear kernel	$K(x, z) = x \cdot z$. Equivalent to standard dot product. For linearly separable data
RBF kernel	$K(x, z) = \exp(-\gamma \cdot x-z ^2)$. Maps to infinite dimensions. Learns any smooth boundary
Polynomial kernel	$K(x, z) = (x \cdot z + c)^d$. Maps to a feature space of polynomial combinations
Dual formulation	Reformulation of the SVM problem that depends only on dot products between data points. Enables kernels
SVR	Support Vector Regression. Fits an ϵ -tube around the data. Points inside the tube have zero loss
Slack variables	ξ_i : measures how much a point violates the margin. Zero for correctly classified points outside margin
Maximum margin	The principle of choosing the hyperplane that maximizes the distance to the nearest points of each class

Code Appendix: *svm.py*

```

import math
import random

def dot(a, b):
    return sum(ai * bi for ai, bi in zip(a, b))

def vec_add(a, b):

```

```

        return [ai + bi for ai, bi in zip(a, b)]

def vec_sub(a, b):
    return [ai - bi for ai, bi in zip(a, b)]

def vec_scale(a, s):
    return [ai * s for ai in a]

def vec_norm(a):
    return math.sqrt(dot(a, a))

def linear_kernel(x, z):
    return dot(x, z)

def polynomial_kernel(x, z, degree=3, c=1.0):
    return (dot(x, z) + c) ** degree

def rbf_kernel(x, z, gamma=0.5):
    diff = vec_sub(x, z)
    return math.exp(-gamma * dot(diff, diff))

def hinge_loss(X, y, w, b):
    n = len(X)
    total = 0.0
    for i in range(n):
        margin = y[i] * (dot(w, X[i]) + b)
        total += max(0.0, 1.0 - margin)
    return total / n

def svm_objective(X, y, w, b, lambda_param):
    reg = 0.5 * lambda_param * dot(w, w)
    loss = hinge_loss(X, y, w, b)
    return reg + loss

class LinearSVM:
    def __init__(self, lr=0.001, lambda_param=0.01, n_epochs=1000):
        self.lr = lr
        self.lambda_param = lambda_param
        self.n_epochs = n_epochs
        self.w = None
        self.b = 0.0
        self.loss_history = []

    def fit(self, X, y):

```



```

n_features = len(X[0])
n_samples = len(X)
self.w = [0.0] * n_features
self.b = 0.0
self.loss_history = []

for epoch in range(self.n_epochs):
    indices = list(range(n_samples))
    random.shuffle(indices)

    for i in indices:
        margin = y[i] * (dot(self.w, X[i]) + self.b)

        if margin >= 1:
            self.w = [
                wj - self.lr * self.lambda_param * wj
                for wj in self.w
            ]
        else:
            self.w = [
                wj - self.lr * (self.lambda_param * wj - y[i] * X[i][j])
                for j, wj in enumerate(self.w)
            ]
            self.b -= self.lr * (-y[i])

    if epoch % 100 == 0 or epoch == self.n_epochs - 1:
        loss = svm_objective(X, y, self.w, self.b, self.lambda_param)
        self.loss_history.append((epoch, loss))

def predict(self, X):
    return [1 if dot(self.w, x) + self.b >= 0 else -1 for x in X]

def decision_function(self, X):
    return [dot(self.w, x) + self.b for x in X]

def margin_width(self):
    w_norm = vec_norm(self.w)
    if w_norm == 0:
        return 0.0
    return 2.0 / w_norm

def find_support_vectors(self, X, y, tol=0.1):
    svcs = []
    for i in range(len(X)):
        margin = y[i] * (dot(self.w, X[i]) + self.b)
        if abs(margin - 1.0) < tol:
            svcs.append(i)
    return svcs

def accuracy(y_true, y_pred):
    correct = sum(1 for a, b in zip(y_true, y_pred) if a == b)
    return correct / len(y_true)

```

```

def generate_linear_data(n_samples=100, margin=1.0, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n_samples):
        x1 = random.uniform(-3, 3)
        x2 = random.uniform(-3, 3)
        val = x1 + x2
        if val > margin / 2:
            X.append([x1, x2])
            y.append(1)
        elif val < -margin / 2:
            X.append([x1, x2])
            y.append(-1)
    return X, y

def generate_noisy_data(n_samples=200, noise=0.5, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n_samples):
        x1 = random.uniform(-3, 3)
        x2 = random.uniform(-3, 3)
        val = x1 - 0.5 * x2 + random.gauss(0, noise)
        label = 1 if val > 0 else -1
        X.append([x1, x2])
        y.append(label)
    return X, y

def generate_circular_data(n_samples=200, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n_samples):
        r = random.uniform(0, 3)
        angle = random.uniform(0, 2 * math.pi)
        x1 = r * math.cos(angle) + random.gauss(0, 0.1)
        x2 = r * math.sin(angle) + random.gauss(0, 0.1)
        label = 1 if r > 1.5 else -1
        X.append([x1, x2])
        y.append(label)
    return X, y

def train_test_split(X, y, test_ratio=0.2, seed=42):
    random.seed(seed)
    n = len(X)
    indices = list(range(n))
    random.shuffle(indices)

```

```

split = int(n * (1 - test_ratio))
train_idx = indices[:split]
test_idx = indices[split:]
return (
    [X[i] for i in train_idx],
    [y[i] for i in train_idx],
    [X[i] for i in test_idx],
    [y[i] for i in test_idx],
)

def compute_kernel_matrix(X, kernel_fn, **kwargs):
    n = len(X)
    K = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            val = kernel_fn(X[i], X[j], **kwargs)
            K[i][j] = val
            K[j][i] = val
    return K

def demo_hinge_loss():
    print("=" * 65)
    print("HINGE LOSS: THE SVM LOSS FUNCTION")
    print("=" * 65)
    print()

    margins = [-2.0, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2.0, 3.0]
    print(f" {'y * f(x)':>10s} {'Hinge loss':>12s} {'Logistic loss':>14s} {'Visual':>20s}")
    print(f" {'-' * 10} {'-' * 12} {'-' * 14} {'-' * 20}")

    for m in margins:
        h_loss = max(0.0, 1.0 - m)
        l_loss = math.log(1 + math.exp(-m))
        bar_len = int(h_loss * 5)
        bar = "#" * bar_len
        print(f" {m:>10.1f} {h_loss:>12.3f} {l_loss:>14.3f} {bar}")

    print()
    print(" Hinge loss is exactly zero when y*f(x) >= 1 (outside margin).")
    print(" Logistic loss is never exactly zero. Always uses all data points.")
    print()

def demo_linear_svm():
    print("=" * 65)
    print("LINEAR SVM: MAXIMUM MARGIN CLASSIFIER")
    print("=" * 65)
    print()

```

```

X, y = generate_linear_data(200, margin=1.0, seed=42)
X_train, y_train, X_test, y_test = train_test_split(X, y)

print(f" Dataset: {len(X)} samples, linearly separable")
print(f" Train: {len(X_train)} Test: {len(X_test)}")
print()

svm = LinearSVM(lr=0.001, lambda_param=0.01, n_epochs=500)
svm.fit(X_train, y_train)

train_pred = svm.predict(X_train)
test_pred = svm.predict(X_test)
train_acc = accuracy(y_train, train_pred)
test_acc = accuracy(y_test, test_pred)

print(f" Weights: [{svm.w[0]:.4f}, {svm.w[1]:.4f}]")
print(f" Bias: {svm.b:.4f}")
print(f" Margin width: {svm.margin_width():.4f}")
print(f" Train accuracy: {train_acc:.4f}")
print(f" Test accuracy: {test_acc:.4f}")

svs = svm.find_support_vectors(X_train, y_train, tol=0.3)
print(f" Support vectors: {len(svs)} / {len(X_train)} training points")
print()

print(" Training loss progression:")
print(f" {'Epoch':>8s} {'Loss':>10s}")
print(f" {'-' * 8} {'-' * 10}")
for epoch, loss in svm.loss_history:
    print(f" {epoch:>8d} {loss:>10.4f}")
print()

def demo_c_parameter():
    print("=" * 65)
    print("C PARAMETER: REGULARIZATION TRADE-OFF")
    print("=" * 65)
    print()

    X, y = generate_noisy_data(300, noise=0.8, seed=42)
    X_train, y_train, X_test, y_test = train_test_split(X, y)

    print(f" Dataset: {len(X)} samples with noise (not perfectly separable)")
    print(f" Train: {len(X_train)} Test: {len(X_test)}")
    print()

    c_values = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
    print(f" {'C':>8s} {'lambda':>8s} {'Train Acc':>10s} {'Test Acc':>10s} {'Margin':>8s} {'SVs':>6s}")

```

```

print(f" {'-' * 8} {'-' * 8} {'-' * 10} {'-' * 10} {'-' * 8} {'-' * 6}")

for c in c_values:
    lam = 1.0 / (c * len(X_train))
    svm = LinearSVM(lr=0.001, lambda_param=lam, n_epochs=500)
    svm.fit(X_train, y_train)

    train_acc = accuracy(y_train, svm.predict(X_train))
    test_acc = accuracy(y_test, svm.predict(X_test))
    margin = svm.margin_width()
    n_sv = len(svm.find_support_vectors(X_train, y_train, tol=0.3))

    print(f" {c:>8.3f} {lam:>8.5f} {train_acc:>10.4f} {test_acc:>10.4f}
"
        f"{margin:>8.4f} {n_sv:>6d}")

print()
print(" Small C (large lambda): wide margin, more errors, better generalization.")
print(" Large C (small lambda): narrow margin, fewer errors, risk of overfitting."
)
print()

def demo_kernel_functions():
    print("=" * 65)
    print("KERNEL FUNCTIONS: SIMILARITY IN DIFFERENT SPACES")
    print("=" * 65)
    print()

    x = [1.0, 0.0]
    points = [
        ("same direction", [2.0, 0.0]),
        ("perpendicular", [0.0, 1.0]),
        ("close", [1.1, 0.1]),
        ("far same dir", [5.0, 0.0]),
        ("opposite", [-1.0, 0.0]),
    ]

    print(f" Reference point: {x}")
    print()
    print(f" {'Point':<20s} {'Linear':>8s} {'Poly(d=2)':>10s} {'Poly(d=3)':>1
0s} {'RBF(g=0.5)':>10s}")
    print(f" {'-' * 20} {'-' * 8} {'-' * 10} {'-' * 10} {'-' * 10}")

    for name, z in points:
        k_lin = linear_kernel(x, z)
        k_p2 = polynomial_kernel(x, z, degree=2)
        k_p3 = polynomial_kernel(x, z, degree=3)
        k_rbf = rbf_kernel(x, z, gamma=0.5)
        print(f" {name:<20s} {k_lin:>8.3f} {k_p2:>10.3f} {k_p3:>10.3f} {k_
rbf:>10.4f}")

```

```

print()
print("  Linear kernel: raw dot product. Measures projection.")
print("  Polynomial kernel: captures feature interactions up to degree d.")
print("  RBF kernel: locality-based. High for nearby points, near zero for distant.
")
print()

def demo_kernel_matrix():
    print("=" * 65)
    print("KERNEL MATRIX: RBF ON CIRCULAR DATA")
    print("=" * 65)
    print()

    X, y = generate_circular_data(20, seed=42)

    K_linear = compute_kernel_matrix(X, linear_kernel)
    K_rbf = compute_kernel_matrix(X, rbf_kernel, gamma=1.0)

    print(f"  Generated {len(X)} points with circular decision boundary")
    print()

    pos_pos_lin = []
    pos_neg_lin = []
    neg_neg_lin = []
    pos_pos_rbf = []
    pos_neg_rbf = []
    neg_neg_rbf = []

    for i in range(len(X)):
        for j in range(i + 1, len(X)):
            if y[i] == 1 and y[j] == 1:
                pos_pos_lin.append(K_linear[i][j])
                pos_pos_rbf.append(K_rbf[i][j])
            elif y[i] == -1 and y[j] == -1:
                neg_neg_lin.append(K_linear[i][j])
                neg_neg_rbf.append(K_rbf[i][j])
            else:
                pos_neg_lin.append(K_linear[i][j])
                pos_neg_rbf.append(K_rbf[i][j])

    def safe_mean(lst):
        return sum(lst) / len(lst) if lst else 0.0

    print(f"  Average kernel values between classes:")
    print(f"  {'Pair':<15s} {'Linear':>10s} {'RBF(g=1)':>10s}")
    print(f"  {'-' * 15} {'-' * 10} {'-' * 10}")
    print(f"  {'Same (+/+)':<15s} {safe_mean(pos_pos_lin):>10.4f} {safe_mean(pos_
pos_rbf):>10.4f}")
    print(f"  {'Same (-/-)':<15s} {safe_mean(neg_neg_lin):>10.4f} {safe_mean(neg_

```

```

neg_rbf):>10.4f}")
    print(f" {'Different':<15s} {safe_mean(pos_neg_lin):>10.4f} {safe_mean(pos_n
eg_rbf):>10.4f}")
    print()
    print(" Linear kernel: cannot separate circular classes well.")
    print(" RBF kernel: creates separation by measuring local similarity.")
    print()

def demo_linear_vs_nonlinear():
    print("=" * 65)
    print("LINEAR SVM vs NONLINEAR BOUNDARY")
    print("=" * 65)
    print()

    X, y = generate_circular_data(200, seed=42)
    X_train, y_train, X_test, y_test = train_test_split(X, y)

    svm = LinearSVM(lr=0.001, lambda_param=0.01, n_epochs=500)
    svm.fit(X_train, y_train)

    train_acc = accuracy(y_train, svm.predict(X_train))
    test_acc = accuracy(y_test, svm.predict(X_test))

    print(f" Circular data (not linearly separable)")
    print(f" Linear SVM: train acc = {train_acc:.4f}, test acc = {test_acc:.4f}")
    print()

    X_train_aug = [
        [x[0], x[1], x[0] ** 2, x[1] ** 2, x[0] * x[1]]
        for x in X_train
    ]
    X_test_aug = [
        [x[0], x[1], x[0] ** 2, x[1] ** 2, x[0] * x[1]]
        for x in X_test
    ]

    svm_aug = LinearSVM(lr=0.0005, lambda_param=0.01, n_epochs=1000)
    svm_aug.fit(X_train_aug, y_train)

    train_acc_aug = accuracy(y_train, svm_aug.predict(X_train_aug))
    test_acc_aug = accuracy(y_test, svm_aug.predict(X_test_aug))

    print(f" After polynomial feature mapping (x1, x2) -> (x1, x2, x1^2, x2^2, x1*x2):
")
    print(f" Linear SVM on augmented features: train acc = {train_acc_aug:.4f}, "
          f"test acc = {test_acc_aug:.4f}")
    print()
    print(" The kernel trick does this feature mapping implicitly.")
    print(" You compute K(x, z) instead of explicitly constructing the features.")
    print()

```

```

def demo_support_vectors():
    print("=" * 65)
    print("SUPPORT VECTORS: THE CRITICAL FEW")
    print("=" * 65)
    print()

    X, y = generate_linear_data(200, margin=1.5, seed=42)
    X_train, y_train, X_test, y_test = train_test_split(X, y)

    svm = LinearSVM(lr=0.001, lambda_param=0.01, n_epochs=1000)
    svm.fit(X_train, y_train)

    margins = []
    for i in range(len(X_train)):
        m = y_train[i] * (dot(svm.w, X_train[i]) + svm.b)
        margins.append((i, m))

    margins.sort(key=lambda x: x[1])

    print(f" Trained on {len(X_train)} points")
    print(f" Weights: [{svm.w[0]:.4f}, {svm.w[1]:.4f}], bias: {svm.b:.4f}")
    print()
    print(" Points sorted by margin (y * f(x)):" )
    print(f" {'Index':>6s} {'y':>4s} {'Margin':>8s} {'Role':<20s}")
    print(f" {'-' * 6} {'-' * 4} {'-' * 8} {'-' * 20}")

    for i, (idx, m) in enumerate(margins[:8]):
        if m < 0:
            role = "MISCLASSIFIED"
        elif m < 1.0:
            role = "inside margin"
        elif m < 1.2:
            role = "SUPPORT VECTOR"
        else:
            role = "safely classified"
        print(f" {idx:>6d} {y_train[idx]:>4d} {m:>8.4f} {role:<20s}")

    print(f" ...")
    for i, (idx, m) in enumerate(margins[-3:]):
        role = "safely classified"
        print(f" {idx:>6d} {y_train[idx]:>4d} {m:>8.4f} {role:<20s}")

    n_sv = sum(1 for _, m in margins if 0.7 < m < 1.3)
    n_safe = sum(1 for _, m in margins if m >= 1.3)
    n_inside = sum(1 for _, m in margins if 0 < m < 0.7)

    print()
    print(f" Support vectors (margin ~ 1.0): {n_sv}")
    print(f" Safely classified (margin >> 1): {n_safe}")
    print(f" Inside margin (0 < margin < 1): {n_inside}")

```



```

print(f" Only {n_sv} out of {len(X_train)} points define the boundary.")
print()

def demo_svm_vs_logistic():
    print("=" * 65)
    print("SVM vs LOGISTIC REGRESSION: LOSS COMPARISON")
    print("=" * 65)
    print()

    X, y = generate_noisy_data(200, noise=0.3, seed=42)
    X_train, y_train, X_test, y_test = train_test_split(X, y)

    svm = LinearSVM(lr=0.001, lambda_param=0.01, n_epochs=500)
    svm.fit(X_train, y_train)
    svm_test_acc = accuracy(y_test, svm.predict(X_test))

    w_lr = [0.0, 0.0]
    b_lr = 0.0
    lr_rate = 0.01
    for epoch in range(500):
        for i in range(len(X_train)):
            z = dot(w_lr, X_train[i]) + b_lr
            z = max(-500, min(500, z))
            p = 1.0 / (1.0 + math.exp(-z))
            y_01 = (y_train[i] + 1) / 2
            error = p - y_01
            for j in range(len(w_lr)):
                w_lr[j] -= lr_rate * error * X_train[i][j]
            b_lr -= lr_rate * error

    lr_pred = [1 if dot(w_lr, x) + b_lr >= 0 else -1 for x in X_test]
    lr_test_acc = accuracy(y_test, lr_pred)

    svm_svs = len(svm.find_support_vectors(X_train, y_train, tol=0.5))

    print(f" SVM test accuracy: {svm_test_acc:.4f}")
    print(f" Logistic regression test acc: {lr_test_acc:.4f}")
    print()
    print(f" SVM support vectors: {svm_svs} / {len(X_train)}")
    print(f" Logistic regression: ALL {len(X_train)} points used")
    print()
    print(" SVM: sparse model, only support vectors matter at prediction time.")
    print(" Logistic: dense model, all training points contribute.")
    print()

def demo_margin_effect():
    print("=" * 65)
    print("MARGIN WIDTH AND GENERALIZATION")
    print("=" * 65)
    print()

```

```

    margins = [0.5, 1.0, 2.0, 3.0]
    print(f" {'Data margin':>12s} {'SVM margin':>12s} {'Train Acc':>10s} {'Test Acc':>10s}")
    print(f" {'-' * 12} {'-' * 12} {'-' * 10} {'-' * 10}")

    for data_margin in margins:
        X, y = generate_linear_data(200, margin=data_margin, seed=42)
        X_train, y_train, X_test, y_test = train_test_split(X, y)

        svm = LinearSVM(lr=0.001, lambda_param=0.01, n_epochs=500)
        svm.fit(X_train, y_train)

        train_acc = accuracy(y_train, svm.predict(X_train))
        test_acc = accuracy(y_test, svm.predict(X_test))

        print(f" {data_margin:>12.1f} {svm.margin_width():>12.4f} "
              f"{train_acc:>10.4f} {test_acc:>10.4f}")

    print()
    print(" Wider data separation = wider learned margin = better generalization.")
    print()

def print_summary():
    print()
    print("=" * 65)
    print("SUMMARY")
    print("=" * 65)
    print()
    print(" 1. SVMs find the maximum margin hyperplane between classes.")
    print(" 2. Only support vectors determine the boundary.")
    print(" 3. Hinge loss produces sparse models (zero loss outside margin).")
    print(" 4. The C parameter trades off margin width vs classification errors.")
    print(" 5. The kernel trick enables nonlinear boundaries via dot products.")
    print(" 6. RBF kernel maps to infinite dimensions using local similarity.")
    print(" 7. Linear SVMs train in O(n*d) per epoch using gradient descent.")
    print(" 8. SVMs still win on small datasets and high-dimensional sparse data.")
    print()

if __name__ == "__main__":
    demo_hinge_loss()
    demo_linear_svm()
    demo_c_parameter()
    demo_kernel_functions()
    demo_kernel_matrix()
    demo_linear_vs_nonlinear()
    demo_support_vectors()
    demo_svm_vs_logistic()
    demo_margin_effect()
    print_summary()

```

SVM Kernel Selection Guide

SVMs are defined by two choices: the kernel (which determines the shape of the decision boundary) and the regularization parameters (which control the tradeoff between margin width and classification errors). Getting these right is the difference between a useless model and a strong one.

Decision Checklist

- Is the data linearly separable (or close to it)?
 - Yes: use linear kernel. It is faster and more interpretable.
 - No: go to step 2.
- How many features vs samples?
 1. Features $\gg$ samples (e.g., text with TF-IDF): use linear kernel. High-dimensional data is often linearly separable. RBF adds complexity for no gain.
 2. Samples $\gg$ features (e.g., tabular data with 10-50 features): RBF kernel is the default choice.
- Is the decision boundary expected to be smooth?

Smooth, continuous boundary: RBF kernel

Polynomial-shaped boundary: polynomial kernel (start with degree 2 or 3)

Domain knowledge suggests specific interaction terms: polynomial kernel with matching degree
- How large is the dataset?

Under 10,000 samples: any kernel works, RBF is the safe default

10,000 to 100,000: linear kernel or LinearSVC (primal formulation, $O(n)$ per epoch)

Over 100,000: do not use kernel SVM. Switch to linear SVM, gradient boosting, or neural networks.
- Did you scale the features?
 7. SVMs require feature scaling. Always standardize (zero mean, unit variance) before fitting. Unscaled features distort the margin geometry.

Kernel selection flowchart

```
graph TD
    Start[Start] --> Q1{Features > 1000 or features >> samples?}
    Q1 -- Yes --> A1[Linear kernel (LinearSVC for speed)]
    Q1 -- No --> Q2{Dataset < 10k samples?}
```

Yes --> Try RBF first (best general-purpose kernel)
 No --> Linear kernel (kernel SVMs are $O(n^2)$ to $O(n^3)$)

If RBF does not work well, try polynomial degree 2-3. If that fails, the problem may not be suited to SVMs.

Tuning C (regularization)

C controls the penalty for misclassifications. It is inversely related to regularization strength.

C value	Effect	When to use
0.001 - 0.01	Wide margin, many violations allowed	Noisy data, want generalization
0.1 - 1.0	Balanced	Good starting range
10 - 1000	Narrow margin, few violations	Clean data, need high accuracy

Tuning strategy: - Start with $C=1.0$ - Search on a log scale: [0.001, 0.01, 0.1, 1, 10, 100, 1000] - Use cross-validation to pick the best value - If best C is at the edge of your range, extend the range in that direction

Tuning γ (RBF kernel)

Gamma controls how far the influence of a single training point reaches. It defines the width of the Gaussian.

γ value	Effect	When to use
Small (0.001)	Each point influences a large area. Smooth, simple boundary	Underfitting or few features
Medium (auto: $1/n_{\text{features}}$)	sklearn default. Reasonable starting point	General use
Large (10+)	Each point influences only nearby points. Complex, wiggly boundary	Risk of overfitting

Tuning strategy: - Start with $\gamma = \text{"scale"}$ ($1 / (n_{\text{features}} * X.\text{var}())$, the sklearn default) - Search on a log scale: [0.001, 0.01, 0.1, 1, 10] - Low γ + high C tends to overfit - High γ + low C tends to underfit

Joint C and γ tuning

C and γ interact. Always tune them together, not independently.

Recommended approach: 1. Coarse grid search: C in [0.01, 0.1, 1, 10, 100], γ in [0.001, 0.01, 0.1, 1, 10] (25 combos) 2. Find the best region 3. Fine grid search around the best region (e.g., C in [5, 10, 20, 50], γ in [0.05, 0.1, 0.2]) 4. Use 5-fold cross-validation throughout

Common mistakes

- Using RBF kernel on high-dimensional sparse data (linear is better and 100x faster)
- Forgetting to scale features (the single most common SVM mistake)
- Setting C too high on noisy data (memorizes noise instead of learning the boundary)
- Using kernel SVM on datasets over 50k samples (training time is prohibitive)
- Not tuning C and γ together (they compensate for each other)
- Defaulting to polynomial degree 5+ (overfits aggressively, try 2 or 3 first)

Quick reference

Kernel	When to use	Key parameters	Training complexity
Linear	Text/TF-IDF, many features, large data	C only	$O(n)$ per epoch
RBF	General-purpose, under 10k samples	C, γ	$O(n^2)$ to $O(n^3)$
Polynomial	Known polynomial relationships	C, degree, coef0	$O(n^2)$ to $O(n^3)$
Sigmoid	Rarely useful (equivalent to two-layer neural net)	C, γ , coef0	$O(n^2)$ to $O(n^3)$

Quiz

Pre-lesson check

- w) What are support vectors in an SVM?
- a) All data points in the training set
 - b) The training points closest to the decision boundary that determine the hyperplane **(correct)**
 - c) The feature vectors after kernel transformation
 - d) The weight vectors learned during training
- Explanation: Support vectors are the training points that lie exactly on the margin boundaries. They are the only points that determine the decision hyperplane. Removing non-support-vector points does not change the boundary.*

What does the SVM maximize when finding the decision boundary?

- The number of correctly classified training points
- The margin – the distance between the decision boundary and the nearest points of each class **(correct)**
- The total distance from all points to the boundary
- The complexity of the decision boundary
Explanation: SVMs find the hyperplane that maximizes the margin between the two classes. A wider margin leads to better generalization on unseen data.

Post-lesson check

33. What happens when you increase the C parameter in an SVM?

- The margin gets wider and more misclassifications are allowed
- The margin gets narrower, fewer misclassifications are tolerated, and the model may overfit **(correct)**
- The kernel function changes from linear to RBF
- The number of support vectors always increases
Explanation: Large C penalizes misclassifications heavily, producing a narrow margin that closely fits the training data. This can lead to overfitting. Small C allows more violations for a wider, more regularized margin.

34. How does the kernel trick enable SVMs to learn nonlinear boundaries?

1. It replaces the SVM with a neural network
2. It computes dot products in a high-dimensional space without explicitly mapping data to that space **(correct)**
3. It removes outliers from the dataset before training
4. It adds polynomial features to the input data directly
Explanation: The kernel trick replaces every dot product $x_i \cdot x_j$ with $K(x_i, x_j)$, computing the dot product in a high-dimensional (even infinite-dimensional for RBF) feature space without ever constructing it.

35. Hinge loss is zero when $y * f(x) \geq 1$. What does this mean in terms of classification?

- The point is misclassified
- The point is correctly classified and lies outside the margin **(correct)**
- The point is exactly on the decision boundary
- The point is a noise sample that should be ignored

*Explanation: When $y * f(x) \geq 1$, the point is correctly classified AND lies on or beyond the margin boundary. Only points inside the margin or misclassified ($y * f(x) < 1$) contribute to the hinge loss.**

K-Nearest Neighbors and Distances

Store everything. Predict by looking at your neighbors. The simplest algorithm that actually works.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lesson 14 Norms and Distances) **Time:** ~90 minutes

Learning Objectives

36. Implement KNN classification and regression from scratch with configurable K and distance-weighted voting
37. Compare L1, L2, cosine, and Minkowski distance metrics and select the appropriate one for a given data type
38. Explain the curse of dimensionality and demonstrate why KNN degrades in high-dimensional spaces
39. Build a KD-tree for efficient nearest neighbor search and analyze when it outperforms brute-force

The Problem

You have a dataset. A new data point arrives. You need to classify it or predict its value. Instead of learning parameters from the data (like linear regression or SVMs), you just find the K training points closest to the new point and let them vote.

This is K-nearest neighbors. There is no training phase. No parameters to learn. No loss function to minimize. You store the entire training set and compute distances at prediction time.

It sounds too simple to work. But KNN is surprisingly competitive for many problems, especially with small to medium datasets, and understanding it deeply reveals fundamental concepts: the choice of distance metric (connecting to Phase 1 Lesson 14), the curse of dimensionality, and the difference between lazy and eager learning.

KNN also shows up everywhere in modern AI, just under different names. Vector databases do KNN search over embeddings. Retrieval-augmented generation (RAG) finds the K nearest document chunks. Recommendation systems find similar users or items. The algorithm is the same. The scale and the data structures are different.

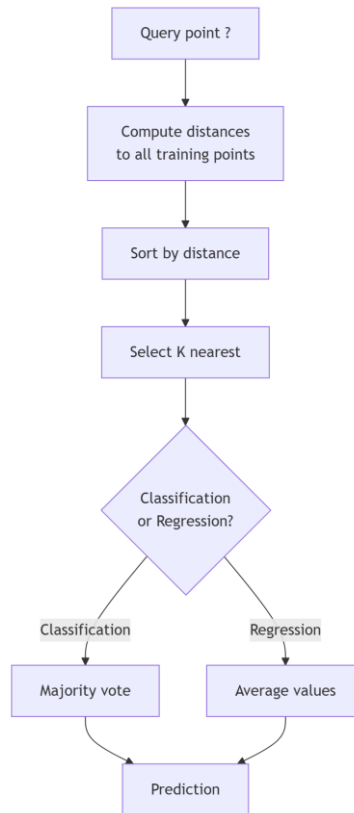
The Concept

How KNN works

Given a dataset of labeled points and a new query point:

40. Compute the distance from the query to every point in the dataset
41. Sort by distance

42. Take the K closest points
43. For classification: majority vote among the K neighbors
44. For regression: average (or weighted average) of the K neighbors' values



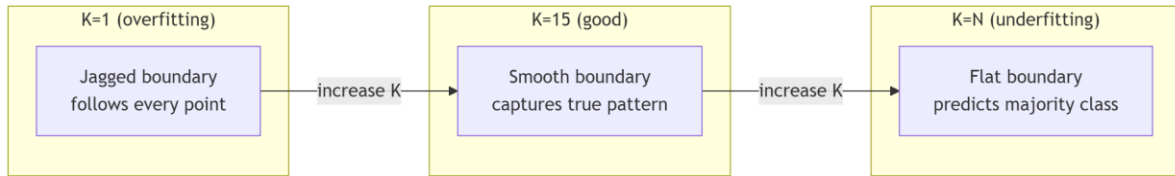
That is the entire algorithm. No fitting. No gradient descent. No epochs.

Choosing K

K is the single hyperparameter. It controls the bias-variance trade-off:

K	Behavior
K = 1	Decision boundary follows every point. Zero training error. High variance. Overfits
Small K (3-5)	Sensitive to local structure. Can capture complex boundaries
Large K	Smoother boundaries. More robust to noise. May underfit
K = N	Predicts the majority class for every point. Maximum bias

A common starting point is $K = \sqrt{N}$ for a dataset of N points. Use odd K for binary classification to avoid ties.



Distance metrics

The distance function defines what “near” means. Different metrics produce different neighbors, different predictions.

L2 (Euclidean) is the default. Straight-line distance.

$$d(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

Sensitive to feature scale. Always standardize features before using L2 with KNN.

L1 (Manhattan) sums absolute differences. More robust to outliers than L2 because it does not square the differences.

$$d(a, b) = \sum |a_i - b_i|$$

Cosine distance measures the angle between vectors, ignoring magnitude. Essential for text and embedding data.

$$d(a, b) = 1 - (a \cdot b) / (||a|| * ||b||)$$

Minkowski generalizes L1 and L2 with parameter p.

$$d(a, b) = (\sum |a_i - b_i|^p)^{1/p}$$

p=1: Manhattan
 p=2: Euclidean
 p->inf: Chebyshev (max absolute difference)

Which metric to use depends on the data:

Data type	Best metric	Why
Numeric features, similar scale	L2 (Euclidean)	Default, works for spatial data
Numeric features, outliers	L1 (Manhattan)	Robust, does not amplify large differences

Data type	Best metric	Why
Text embeddings	Cosine	Magnitude is noise, direction is meaning
High-dimensional sparse	Cosine or L1	L2 suffers from curse of dimensionality
Mixed types	Custom distance	Combine metrics per feature type

Weighted KNN

Standard KNN gives equal weight to all K neighbors. But a neighbor at distance 0.1 should matter more than one at distance 5.0.

Distance-weighted KNN weights each neighbor inversely by distance:

```
weight_i = 1 / (distance_i + epsilon)
```

For classification: weighted vote

For regression: $\text{weighted average} = \frac{\sum(w_i * y_i)}{\sum(w_i)}$

The ϵ prevents division by zero when a query point exactly matches a training point.

Weighted KNN is less sensitive to the choice of K because distant neighbors contribute very little regardless.

The curse of dimensionality

KNN performance degrades in high dimensions. This is not a vague concern. It is a mathematical fact.

Problem 1: distances converge. As dimensionality increases, the ratio of the maximum distance to the minimum distance approaches 1. All points become equally “far” from the query.

In d dimensions, for random uniform points:

d=2: $\text{max_dist} / \text{min_dist} = \text{varies widely}$

d=100: $\text{max_dist} / \text{min_dist} \sim 1.01$

d=1000: $\text{max_dist} / \text{min_dist} \sim 1.001$

When all distances are nearly equal, “nearest” is meaningless.

Problem 2: volume explodes. To capture K neighbors within a fixed fraction of the data, you need to extend your search radius to cover a much larger fraction of the feature space. The “neighborhood” in high dimensions encompasses most of the space.

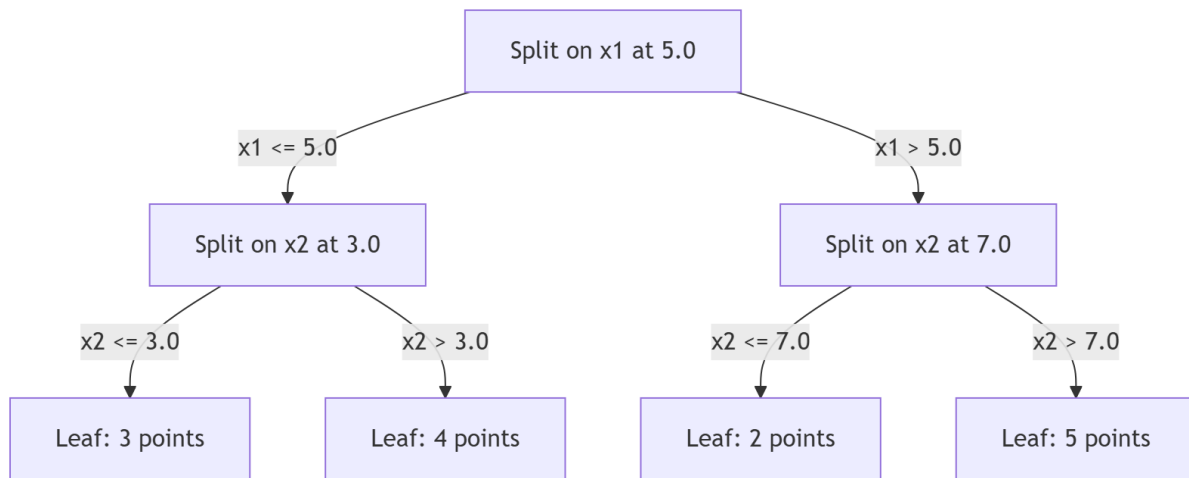
Problem 3: corners dominate. In a unit hypercube in d dimensions, most of the volume is concentrated near the corners, not the center. A sphere inscribed in the cube contains a vanishing fraction of the volume as d grows.

Practical consequence: KNN works well up to about 20-50 features. Beyond that, you need dimensionality reduction (PCA, UMAP, t-SNE) before applying KNN, or you need to use tree-based search structures that exploit the data's intrinsic lower dimensionality.

KD-trees: fast nearest neighbor search

Brute-force KNN computes the distance from the query to every training point. That is $O(n * d)$ per query. For large datasets, this is too slow.

A KD-tree recursively partitions the space along feature axes. At each level, it splits along one dimension at the median value.



To find the nearest neighbor, traverse the tree to the leaf containing the query, then backtrack and check neighboring partitions only if they could contain closer points.

Average query time: $O(\log n)$ for low dimensions. But KD-trees degrade to $O(n)$ in high dimensions ($d > 20$) because the backtracking eliminates fewer and fewer branches.

Ball trees: better for moderate dimensions

Ball trees partition data into nested hyperspheres instead of axis-aligned boxes. Each node defines a ball (center + radius) that contains all points in that subtree.

Advantages over KD-trees: - Work better in moderate dimensions (up to ~50) - Handle non-axis-aligned structure - Tighter bounding volumes mean more branches are pruned during search

Both KD-trees and ball trees are exact algorithms. For truly large-scale search (millions of points, hundreds of dimensions), approximate nearest neighbor methods (HNSW, IVF, product quantization) are used instead. These are covered in Phase 1 Lesson 14.

Lazy learning vs eager learning

KNN is a lazy learner: it does no work at training time and all work at prediction time. Most other algorithms (linear regression, SVMs, neural networks) are eager learners: they do heavy computation at training time to build a compact model, then predictions are fast.

Aspect	Lazy (KNN)	Eager (SVM, neural net)
Training time	$O(1)$ just store data	$O(n * \text{epochs})$
Prediction time	$O(n * d)$ per query	$O(d)$ or $O(\text{parameters})$
Memory at prediction	Store entire training set	Store model parameters only
Adapts to new data	Add points instantly	Retrain the model
Decision boundary	Implicit, computed on the fly	Explicit, fixed after training

Lazy learning is ideal when: - The dataset changes frequently (add/remove points without retraining) - You need predictions for very few queries - You want zero training time - The dataset is small enough that brute-force search is fast

KNN for regression

Instead of majority voting, KNN for regression averages the target values of the K neighbors.

```
prediction = (1/K) * sum(y_i for i in K nearest neighbors)
```

Or with distance weighting:

```
prediction = sum(w_i * y_i) / sum(w_i)
```

```
where w_i = 1 / distance_i
```

KNN regression produces piecewise-constant (or piecewise-smooth with weighting) predictions. It cannot extrapolate beyond the range of the training data. If the training targets are all between 0 and 100, KNN will never predict 200.

Build It

Step 1: Distance functions

Implement L1, L2, cosine, and Minkowski distances. These connect directly to Phase 1 Lesson 14.

```

import math

def l2_distance(a, b):
    return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))

def l1_distance(a, b):
    return sum(abs(ai - bi) for ai, bi in zip(a, b))

def cosine_distance(a, b):
    dot_val = sum(ai * bi for ai, bi in zip(a, b))
    norm_a = math.sqrt(sum(ai ** 2 for ai in a))
    norm_b = math.sqrt(sum(bi ** 2 for bi in b))
    if norm_a == 0 or norm_b == 0:
        return 1.0
    return 1.0 - dot_val / (norm_a * norm_b)

def minkowski_distance(a, b, p=2):
    if p == float('inf'):
        return max(abs(ai - bi) for ai, bi in zip(a, b))
    return sum(abs(ai - bi) ** p for ai, bi in zip(a, b)) ** (1 / p)

```

Step 2: KNN classifier and regressor

Build the full KNN with configurable K, distance metric, and optional distance weighting.

```

class KNN:
    def __init__(self, k=5, distance_fn=l2_distance, weighted=False,
                 task="classification"):
        self.k = k
        self.distance_fn = distance_fn
        self.weighted = weighted
        self.task = task
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        return [self._predict_one(x) for x in X]

```

Step 3: KD-tree for efficient search

Build a KD-tree from scratch that recursively splits on the median of each dimension.

```

class KDTree:
    def __init__(self, X, indices=None, depth=0):
        # Recursively partition the data
        self.axis = depth % len(X[0])
        # Split on median of the current axis

```

```

...

def query(self, point, k=1):
    # Traverse to leaf, then backtrack
    ...

```

See code/knn.py for the complete implementation with all helper methods and demos.

Step 4: Feature scaling

KNN requires feature scaling because distances are sensitive to feature magnitudes. A feature ranging from 0 to 1000 will dominate a feature ranging from 0 to 1.

```

def standardize(X):
    n = len(X)
    d = len(X[0])
    means = [sum(X[i][j] for i in range(n)) / n for j in range(d)]
    stds = [
        max(1e-10, (sum((X[i][j] - means[j]) ** 2 for i in range(n)) / n) ** 0.5)
        for j in range(d)
    ]
    return [[(X[i][j] - means[j]) / stds[j] for j in range(d)) for i in range(n)], means, stds

```

Use It

With scikit-learn:

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5, metric="euclidean")),
])
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.4f}")

```

Scikit-learn automatically uses KD-trees or ball trees when the dataset is large enough and the dimensionality is low enough. For high-dimensional data, it falls back to brute force. You can control this with the `algorithm` parameter.

For large-scale nearest neighbor search (millions of vectors), use FAISS, Annoy, or a vector database:

```

import faiss

index = faiss.IndexFlatL2(dimension)

```

```
index.add(embeddings)
distances, indices = index.search(query_vectors, k=5)
```

Exercises

- x) Implement KNN classification on a 2D dataset with 3 classes. Plot the decision boundary for $K=1$, $K=5$, $K=15$, and $K=N$. Observe the transition from overfitting to underfitting.
- y) Generate 1000 random points in 2, 5, 10, 50, 100, and 500 dimensions. For each dimensionality, compute the ratio of the maximum pairwise distance to the minimum pairwise distance. Plot the ratio vs dimensionality to visualize the curse of dimensionality.
- z) Compare L1, L2, and cosine distance for KNN on a text classification problem (use TF-IDF vectors). Which metric gives the best accuracy? Why does cosine tend to win for text?
- aa) Implement a KD-tree and measure query time vs brute force for datasets of 1k, 10k, and 100k points in 2D, 10D, and 50D. At what dimensionality does the KD-tree stop being faster than brute force?
- bb) Build a weighted KNN regressor for $y = \sin(x) + \text{noise}$. Compare it with unweighted KNN for $K=3$, 10, 30. Show that weighting produces smoother predictions, especially for large K .

Key Terms

Term	What it actually means
K-nearest neighbors	Non-parametric algorithm that predicts by finding the K closest training points to a query
Lazy learning	No computation at training time. All work happens at prediction time. KNN is the canonical example
Eager learning	Heavy computation at training time to build a compact model. Most ML algorithms are eager
Curse of dimensionality	In high dimensions, distances converge and neighborhoods expand to cover most of the space, making KNN ineffective
KD-tree	Binary tree that recursively partitions space along feature axes. $O(\log n)$ queries in low dimensions
Ball tree	Tree of nested hyperspheres. Works better than KD-trees in moderate dimensions (up to ~ 50)
Weighted KNN	Neighbors weighted inversely by distance. Closer neighbors have more influence on the prediction

Term	What it actually means
Feature scaling	Normalizing features to comparable ranges. Required for distance-based methods like KNN
Majority vote	Classification by counting which class is most common among K neighbors
Brute force search	Computing distance to every training point. $O(n*d)$ per query. Exact but slow for large n
Approximate nearest neighbor	Algorithms (HNSW, LSH, IVF) that find approximately nearest points much faster than exact search
Voronoi diagram	The partition of space where each region contains all points closer to one training point than any other. $K=1$ KNN produces Voronoi boundaries

You are a distance metric advisor. Given a description of a dataset (feature types, scale, domain), you recommend the most appropriate distance metric and explain why alternatives would fail.

When a user describes their data, work through this process:

Step 1: Identify the data type

Determine what kind of features the dataset contains: - Pure numerical (continuous values) - Pure categorical (discrete labels or categories) - Mixed (both numerical and categorical) - Text (documents, sentences, words) - Embeddings (dense vectors from a neural network) - Binary (presence/absence features) - Time series (sequences of values)

Step 2: Recommend the primary metric

Use this decision framework:

Numerical, similar scale, no extreme outliers: - Use Euclidean (L2) distance - The default for most spatial and tabular problems - Assumes all dimensions contribute equally

Numerical, outliers present or sparse data: - Use Manhattan (L1) distance - Does not square differences, so a single large deviation does not dominate - More robust in practice than Euclidean for noisy real-world data

Text embeddings, document vectors, or TF-IDF: - Use Cosine distance (1 minus cosine similarity) - Ignores vector magnitude, measures only direction - A long document and a short document about the same topic will be “close” in cosine but far in Euclidean

Binary features (0/1 vectors): - Use Hamming distance (fraction of positions that differ) - Directly interpretable: “these two items differ in 3 out of 10 attributes” - Jaccard distance is the alternative when you only care about shared presences, not shared absences

Categorical features: - Use Hamming distance or a custom overlap metric - Euclidean is meaningless on one-hot encoded categories unless combined with numerical features

Mixed types: - Use Gower distance: normalizes each feature type appropriately and combines them - Alternatively, compute separate distances per type and weight them

High-dimensional data (100+ features): - Euclidean distance concentrates (all pairwise distances converge to similar values) - Cosine distance or Manhattan tend to work better - Consider dimensionality reduction (PCA, UMAP) before computing distances

Time series: - Dynamic Time Warping (DTW) for sequences that may be shifted or stretched in time - Euclidean on raw values only if sequences are perfectly aligned

Step 3: Check prerequisites

Before applying the chosen metric: - **Scaling:** Euclidean and Manhattan require features on comparable scales. Standardize (zero mean, unit variance) or min-max normalize. - **Dimensionality:** above 50 dimensions, consider reducing dimensionality first. Distance metrics become less discriminative in high dimensions (the curse of dimensionality). - **Missing values:** most distance metrics cannot handle NaN. Impute first, or use a metric that supports missing data (like Gower distance).

Step 4: Suggest validation

Recommend the user verify the metric choice: - Run KNN with 2-3 candidate metrics and compare accuracy via cross-validation - For clustering, compare silhouette scores across metrics - Spot-check: find the 5 nearest neighbors of a few known points and confirm they make domain sense

Output format

Structure your response as: 1. **Recommended metric:** [name] with formula 2. **Why this metric:** [1-2 sentence justification tied to the data properties] 3. **Why not alternatives:** [explain why the obvious alternative would be worse] 4. **Preprocessing needed:** [scaling, imputation, or dimensionality reduction] 5. **Validation step:** [how to confirm the choice]

Avoid: - Recommending Euclidean distance for text or embedding data without justification - Ignoring feature scaling when recommending L1 or L2 distances - Suggesting exotic metrics without explaining the tradeoff (computation cost, interpretability) - Defaulting to Euclidean when data is high-dimensional sparse (cosine or L1 are almost always better)

Quiz

Pre-lesson check

13. KNN is called a 'lazy learner.' What does that mean?

- a) It converges slowly during training
- b) It does no computation at training time and computes everything at prediction time **(correct)**
- c) It uses a simplified version of the loss function
- d) It only works on small datasets
Explanation: Lazy learning means KNN stores the training data and does no work during 'training.' All computation (distance calculations, voting) happens when a prediction is requested.

14. Why is feature scaling critical for KNN?

- e) KNN cannot handle negative numbers without scaling
- f) Distance calculations are dominated by features with larger ranges, making scaling necessary for fair comparisons **(correct)**
- g) Feature scaling reduces the number of neighbors needed
- h) KNN uses gradient descent which requires normalized inputs
Explanation: KNN relies on distances. A feature ranging 0-1000 will dominate a feature ranging 0-1 in distance calculations. Scaling puts all features on comparable ranges.

Post-lesson check

- cc) In 100 dimensions with uniform random points, what happens to the ratio of max distance to min distance?
 - It increases dramatically, making neighbors more distinct
 - It approaches 1, making all points nearly equidistant from each other **(correct)**
 - It stays the same as in 2 dimensions
 - It becomes negative due to numerical overflow
Explanation: This is the curse of dimensionality. In high dimensions, distances converge: $\text{max_dist} / \text{min_dist}$ approaches 1. When all points are equidistant, 'nearest' becomes meaningless.

dd) Which distance metric is most appropriate for comparing text documents represented as TF-IDF vectors?

1. L2 (Euclidean) distance
2. L1 (Manhattan) distance
3. Cosine distance **(correct)**

4. Chebyshev distance

Explanation: Cosine distance measures the angle between vectors, ignoring magnitude. For text, document length (magnitude) is noise – direction captures meaning. Cosine consistently outperforms L1/L2 for text.

ee) What happens to the KNN decision boundary as K increases from 1 to N (the full dataset size)?

- The boundary becomes more complex and detailed
- The boundary stays the same regardless of K
- The boundary smooths out, eventually predicting the majority class for every point **(correct)**

- The boundary becomes circular

Explanation: K=1 produces jagged boundaries that follow every point (overfitting). As K increases, boundaries smooth out. K=N means every query considers all points, always predicting the majority class (maximum bias).

Unsupervised Learning

No labels, no teacher. The algorithm finds structure on its own.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Norms & Distances, Probability & Distributions), Phase 2 Lessons 1-6 **Time:** ~90 minutes

Learning Objectives

45. Implement K-Means, DBSCAN, and Gaussian Mixture Models from scratch and compare their clustering behavior
46. Evaluate cluster quality using the silhouette score and the elbow method to select the optimal K
47. Explain when DBSCAN outperforms K-Means and identify which algorithm handles non-spherical clusters and outliers
48. Build an anomaly detection pipeline using clustering methods to flag points that deviate from normal patterns

The Problem

Every ML lesson so far has assumed labeled data: “here is an input, here is the correct output.” In the real world, labels are expensive. A hospital has millions of patient records but no one has manually tagged each one with a disease category. An e-commerce site has millions of user sessions but no one has hand-labeled customer segments. A security team has network logs but nobody has flagged every anomaly.

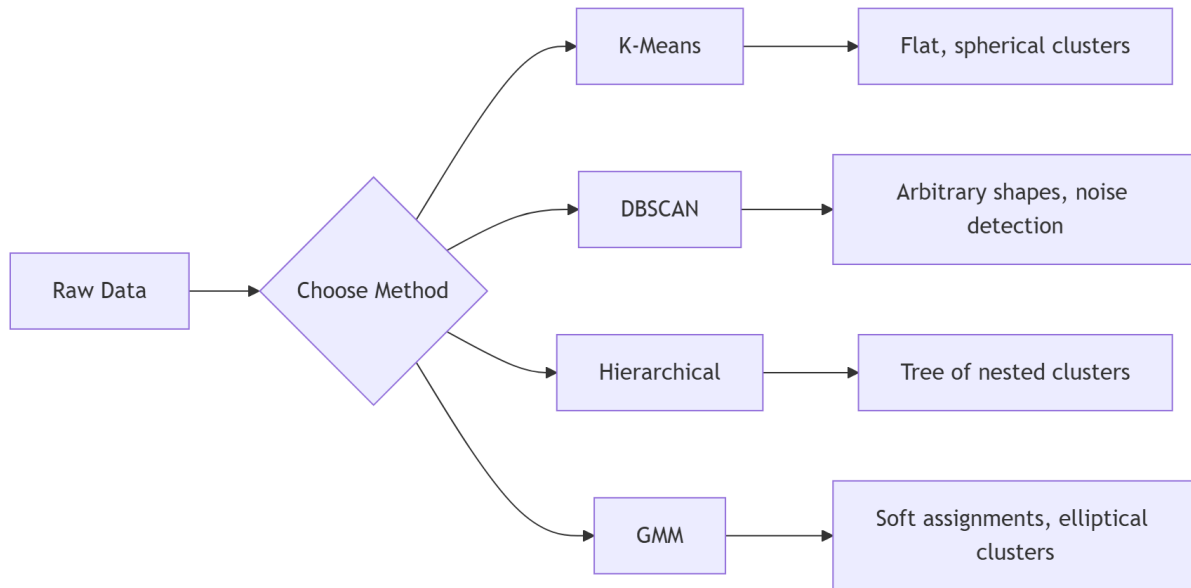
Unsupervised learning finds patterns without being told what to look for. It groups similar data points, discovers hidden structures, and surfaces anomalies. If supervised learning is learning from a textbook with an answer key, unsupervised learning is staring at raw data until the patterns reveal themselves.

The catch: without labels, you cannot directly measure “right” or “wrong.” You need different tools to evaluate whether the structure your algorithm found is meaningful.

The Concept

Clustering: Grouping Similar Things Together

Clustering assigns each data point to a group (cluster) so that points within the same group are more similar to each other than to points in other groups. The question is always: what does “similar” mean?



K-Means: The Workhorse

K-Means partitions data into exactly K clusters. Each cluster has a centroid (its center of mass), and every point belongs to the nearest centroid.

Lloyd's algorithm:

- Pick K random points as initial centroids
- Assign each data point to the nearest centroid
- Recompute each centroid as the mean of its assigned points
- Repeat steps 2-3 until assignments stop changing

The objective function (inertia) measures the total squared distance from each point to its assigned centroid. K-Means minimizes this, but only finds a local minimum. Different initializations can give different results.

Choosing K

Two standard methods:

Elbow method: Run K-Means for $K = 1, 2, 3, \dots, n$. Plot inertia vs K. Look for the “elbow” where adding more clusters stops reducing inertia significantly.

Silhouette score: For each point, measure how similar it is to its own cluster (a) versus the nearest other cluster (b). The silhouette coefficient is $(b - a) / \max(a, b)$, ranging from -1 (wrong cluster) to +1 (well-clustered). Average across all points for a global score.

DBSCAN: Density-Based Clustering

K-Means assumes clusters are spherical and requires you to pick K upfront. DBSCAN makes neither assumption. It finds clusters as dense regions separated by sparse regions.

Two parameters: - **eps**: the radius of a neighborhood - **min_samples**: the minimum number of points needed to form a dense region

Three types of points: - **Core point**: has at least min_samples points within eps distance - **Border point**: within eps of a core point but not itself a core point - **Noise point**: neither core nor border. These are outliers.

DBSCAN connects core points that are within eps of each other into the same cluster. Border points join the cluster of a nearby core point. Noise points belong to no cluster.

Strengths: finds clusters of any shape, automatically determines the number of clusters, identifies outliers. Weakness: struggles with clusters of varying densities.

Hierarchical Clustering

Builds a tree (dendrogram) of nested clusters.

Agglomerative (bottom-up): 1. Start with each point as its own cluster 2. Merge the two closest clusters 3. Repeat until only one cluster remains 4. Cut the dendrogram at the desired level to get K clusters

The “closeness” between clusters can be measured as: - **Single linkage**: minimum distance between any two points in the two clusters - **Complete linkage**: maximum distance between any two points - **Average linkage**: average distance between all pairs - **Ward’s method**: the merge that causes the smallest increase in total within-cluster variance

Gaussian Mixture Models (GMM)

K-Means gives hard assignments: each point belongs to exactly one cluster. GMM gives soft assignments: each point has a probability of belonging to each cluster.

GMM assumes the data is generated from a mixture of K Gaussian distributions, each with its own mean and covariance. The Expectation-Maximization (EM) algorithm alternates between:

49. **E-step**: compute the probability that each point belongs to each Gaussian
50. **M-step**: update the mean, covariance, and mixing weight of each Gaussian to maximize the likelihood of the data

GMM can model elliptical clusters (not just spherical like K-Means) and naturally handles overlapping clusters.

When to Use Which

Method	Best for	Avoid when
K-Means	Large datasets, spherical clusters, known K	Irregular shapes, outliers present
DBSCAN	Unknown K, arbitrary shapes, outlier detection	Varying densities, very high dimensions
Hierarchical	Small datasets, need dendrogram, unknown K	Large datasets ($O(n^2)$ memory)
GMM	Overlapping clusters, soft assignments needed	Very large datasets, too many dimensions

Anomaly Detection with Clustering

Clustering naturally supports anomaly detection: - **K-Means**: points far from any centroid are anomalies - **DBSCAN**: noise points are anomalies by definition - **GMM**: points with low probability under all Gaussians are anomalies

Build It

Step 1: K-Means from scratch

```
import math
import random

def euclidean_distance(a, b):
    return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))

def kmeans(data, k, max_iterations=100, seed=42):
    random.seed(seed)
    n_features = len(data[0])

    centroids = random.sample(data, k)

    for iteration in range(max_iterations):
        clusters = [[] for _ in range(k)]
        assignments = []

        for point in data:
            distances = [euclidean_distance(point, c) for c in centroids]
            nearest = distances.index(min(distances))
            clusters[nearest].append(point)
            assignments.append(nearest)

        new_centroids = []
        for cluster in clusters:
            if len(cluster) == 0:
                new_centroids.append(random.choice(data))
```



```

        continue
    centroid = [
        sum(point[j] for point in cluster) / len(cluster)
        for j in range(n_features)
    ]
    new_centroids.append(centroid)

if all(
    euclidean_distance(old, new) < 1e-6
    for old, new in zip(centroids, new_centroids)
):
    print(f" Converged at iteration {iteration + 1}")
    break

centroids = new_centroids

return assignments, centroids

```

Step 2: Elbow method and silhouette score

```

def compute_inertia(data, assignments, centroids):
    total = 0.0
    for point, cluster_id in zip(data, assignments):
        total += euclidean_distance(point, centroids[cluster_id]) ** 2
    return total

def silhouette_score(data, assignments):
    n = len(data)
    if n < 2:
        return 0.0

    clusters = {}
    for i, c in enumerate(assignments):
        clusters.setdefault(c, []).append(i)

    if len(clusters) < 2:
        return 0.0

    scores = []
    for i in range(n):
        own_cluster = assignments[i]
        own_members = [j for j in clusters[own_cluster] if j != i]

        if len(own_members) == 0:
            scores.append(0.0)
            continue

        a = sum(euclidean_distance(data[i], data[j]) for j in own_members) / len(own_me
members)

        b = float("inf")

```

```

        for cluster_id, members in clusters.items():
            if cluster_id == own_cluster:
                continue
            avg_dist = sum(euclidean_distance(data[i], data[j]) for j in members) / len
(members)
            b = min(b, avg_dist)

        if max(a, b) == 0:
            scores.append(0.0)
        else:
            scores.append((b - a) / max(a, b))

    return sum(scores) / len(scores)

def find_best_k(data, max_k=10):
    print("Elbow method:")
    inertias = []
    for k in range(1, max_k + 1):
        assignments, centroids = kmeans(data, k)
        inertia = compute_inertia(data, assignments, centroids)
        inertias.append(inertia)
        print(f" K={k}: inertia={inertia:.2f}")

    print("\nSilhouette scores:")
    for k in range(2, max_k + 1):
        assignments, centroids = kmeans(data, k)
        score = silhouette_score(data, assignments)
        print(f" K={k}: silhouette={score:.4f}")

    return inertias

```

Step 3: DBSCAN from scratch

```

def dbscan(data, eps, min_samples):
    n = len(data)
    labels = [-1] * n
    cluster_id = 0

    def region_query(point_idx):
        neighbors = []
        for i in range(n):
            if euclidean_distance(data[point_idx], data[i]) <= eps:
                neighbors.append(i)
        return neighbors

    visited = [False] * n

    for i in range(n):
        if visited[i]:
            continue
        visited[i] = True

```

```

neighbors = region_query(i)

if len(neighbors) < min_samples:
    labels[i] = -1
    continue

labels[i] = cluster_id
seed_set = list(neighbors)
seed_set.remove(i)

j = 0
while j < len(seed_set):
    q = seed_set[j]

    if not visited[q]:
        visited[q] = True
        q_neighbors = region_query(q)
        if len(q_neighbors) >= min_samples:
            for nb in q_neighbors:
                if nb not in seed_set:
                    seed_set.append(nb)

    if labels[q] == -1:
        labels[q] = cluster_id

    j += 1

cluster_id += 1

return labels

```

Step 4: Gaussian Mixture Model (EM algorithm)

```

def gmm(data, k, max_iterations=100, seed=42):
    random.seed(seed)
    n = len(data)
    d = len(data[0])

    indices = random.sample(range(n), k)
    means = [list(data[i]) for i in indices]
    variances = [1.0] * k
    weights = [1.0 / k] * k

    def gaussian_pdf(x, mean, variance):
        d = len(x)
        coeff = 1.0 / ((2 * math.pi * variance) ** (d / 2))
        exponent = -sum((xi - mi) ** 2 for xi, mi in zip(x, mean)) / (2 * variance)
        return coeff * math.exp(max(exponent, -500))

    for iteration in range(max_iterations):
        responsibilities = []

```

```

    for i in range(n):
        probs = []
        for j in range(k):
            probs.append(weights[j] * gaussian_pdf(data[i], means[j], variances[j]))
    )

    total = sum(probs)
    if total == 0:
        total = 1e-300
    responsibilities.append([p / total for p in probs])

old_means = [list(m) for m in means]

for j in range(k):
    r_sum = sum(responsibilities[i][j] for i in range(n))
    if r_sum < 1e-10:
        continue

    weights[j] = r_sum / n

    for dim in range(d):
        means[j][dim] = sum(
            responsibilities[i][j] * data[i][dim] for i in range(n)
        ) / r_sum

    variances[j] = sum(
        responsibilities[i][j]
        * sum((data[i][dim] - means[j][dim]) ** 2 for dim in range(d))
        for i in range(n)
    ) / (r_sum * d)
    variances[j] = max(variances[j], 1e-6)

shift = sum(
    euclidean_distance(old_means[j], means[j]) for j in range(k)
)
if shift < 1e-6:
    print(f" GMM converged at iteration {iteration + 1}")
    break

assignments = []
for i in range(n):
    assignments.append(responsibilities[i].index(max(responsibilities[i])))

return assignments, means, weights, responsibilities

```

Step 5: Generate test data and run everything

```

def make_blobs(centers, n_per_cluster=50, spread=0.5, seed=42):
    random.seed(seed)
    data = []
    true_labels = []
    for label, (cx, cy) in enumerate(centers):
        for _ in range(n_per_cluster):

```

```

        x = cx + random.gauss(0, spread)
        y = cy + random.gauss(0, spread)
        data.append([x, y])
        true_labels.append(label)
    return data, true_labels

def make_moons(n_samples=200, noise=0.1, seed=42):
    random.seed(seed)
    data = []
    labels = []
    n_half = n_samples // 2
    for i in range(n_half):
        angle = math.pi * i / n_half
        x = math.cos(angle) + random.gauss(0, noise)
        y = math.sin(angle) + random.gauss(0, noise)
        data.append([x, y])
        labels.append(0)
    for i in range(n_half):
        angle = math.pi * i / n_half
        x = 1 - math.cos(angle) + random.gauss(0, noise)
        y = 1 - math.sin(angle) - 0.5 + random.gauss(0, noise)
        data.append([x, y])
        labels.append(1)
    return data, labels

if __name__ == "__main__":
    centers = [[2, 2], [8, 3], [5, 8]]
    data, true_labels = make_blobs(centers, n_per_cluster=50, spread=0.8)

    print("=== K-Means on 3 blobs ===")
    assignments, centroids = kmeans(data, k=3)
    print(f" Centroids: {[round(c, 2) for c in cent] for cent in centroids}")
    sil = silhouette_score(data, assignments)
    print(f" Silhouette score: {sil:.4f}")

    print("\n=== Elbow Method ===")
    find_best_k(data, max_k=6)

    print("\n=== DBSCAN on 3 blobs ===")
    db_labels = dbscan(data, eps=1.5, min_samples=5)
    n_clusters = len(set(db_labels) - {-1})
    n_noise = db_labels.count(-1)
    print(f" Found {n_clusters} clusters, {n_noise} noise points")

    print("\n=== GMM on 3 blobs ===")
    gmm_assignments, gmm_means, gmm_weights, _ = gmm(data, k=3)
    print(f" Means: {[round(m, 2) for m in mean] for mean in gmm_means}")
    print(f" Weights: {[round(w, 3) for w in gmm_weights]}")
    gmm_sil = silhouette_score(data, gmm_assignments)

```

```

print(f" Silhouette score: {gmm_sil:.4f}")

print("\n=== DBSCAN on moons (non-spherical clusters) ===")
moon_data, moon_labels = make_moons(n_samples=200, noise=0.1)
moon_db = dbscan(moon_data, eps=0.3, min_samples=5)
n_moon_clusters = len(set(moon_db) - {-1})
n_moon_noise = moon_db.count(-1)
print(f" Found {n_moon_clusters} clusters, {n_moon_noise} noise points")

print("\n=== K-Means on moons (will fail to separate) ===")
moon_km, moon_centroids = kmeans(moon_data, k=2)
moon_sil = silhouette_score(moon_data, moon_km)
print(f" Silhouette score: {moon_sil:.4f}")
print(" K-Means splits moons poorly because they are not spherical")

print("\n=== Anomaly detection with DBSCAN ===")
anomaly_data = list(data)
anomaly_data.append([20.0, 20.0])
anomaly_data.append([-5.0, -5.0])
anomaly_data.append([15.0, 0.0])
anomaly_labels = dbscan(anomaly_data, eps=1.5, min_samples=5)
anomalies = [
    anomaly_data[i]
    for i in range(len(anomaly_labels))
    if anomaly_labels[i] == -1
]
print(f" Detected {len(anomalies)} anomalies")
for a in anomalies[-3:]:
    print(f" Point {[round(v, 2) for v in a]}")

```

Use It

With scikit-learn, the same algorithms are one-liners:

```

from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score as sklearn_silhouette

km = KMeans(n_clusters=3, random_state=42).fit(data)
db = DBSCAN(eps=1.5, min_samples=5).fit(data)
agg = AgglomerativeClustering(n_clusters=3).fit(data)
gmm_model = GaussianMixture(n_components=3, random_state=42).fit(data)

```

The from-scratch versions show you exactly what these libraries compute. K-Means iterates between assigning and recomputing. DBSCAN grows clusters from dense seeds. GMM alternates between expectation and maximization. The library versions add numerical stability, smarter initialization (K-Means++), and GPU acceleration, but the core logic is the same.

Exercises

51. Implement K-Means++ initialization: instead of picking random centroids, pick the first randomly and each subsequent centroid with probability proportional to its squared distance from the nearest existing centroid. Compare convergence speed to random initialization.
52. Add hierarchical agglomerative clustering to the code. Implement Ward's linkage and produce a dendrogram (as a nested list of merges). Cut it at different levels and compare to K-Means results.
53. Build a simple anomaly detection pipeline: run DBSCAN and GMM on the same data, flag points that both methods agree are outliers (noise in DBSCAN, low probability in GMM). Measure the overlap and discuss when the methods disagree.

Key Terms

Term	What people say	What it actually means
Clustering	"Grouping similar things"	Partitioning data into subsets where within-group similarity exceeds between-group similarity, measured by a specific distance metric
Centroid	"The center of a cluster"	The mean of all points assigned to a cluster; used by K-Means as the cluster representative
Inertia	"How tight the clusters are"	Sum of squared distances from each point to its assigned centroid; lower is tighter
Silhouette score	"How well-separated clusters are"	For each point, $(b - a) / \max(a, b)$ where a is mean intra-cluster distance and b is mean nearest-cluster distance
Core point	"A point in a dense region"	A point with at least <code>min_samples</code> neighbors within <code>eps</code> distance, in DBSCAN
EM algorithm	"Soft K-Means"	Expectation-Maximization: iteratively compute membership probabilities (E-step) and update distribution parameters (M-step)
Dendrogram	"A tree of clusters"	A tree diagram showing the order and distance at which clusters were merged in hierarchical clustering
Anomaly	"An outlier"	A data point that does not conform to the expected pattern, identified as noise by DBSCAN or low-probability by GMM

Clustering Algorithm Selection Guide

Clustering has no single best algorithm. The right choice depends on cluster shape, whether you know the number of clusters, how much noise is in the data, and how large the dataset is.

Decision Checklist

Do you know the number of clusters?

3. Yes: K-Means or GMM
4. No: DBSCAN (finds clusters automatically), or hierarchical (cut the dendrogram at different levels)

What shape are the clusters?

- a) Roughly spherical (blob-like): K-Means
- b) Elliptical with different sizes: GMM
- c) Arbitrary shapes (crescents, rings, chains): DBSCAN
- d) Nested or hierarchical: hierarchical clustering

Does the data contain noise or outliers?

- e) Yes: DBSCAN (labels noise points explicitly) or GMM (low-probability points are outliers)
- f) No: K-Means is fine

Do you need soft assignments (probabilities)?

- g) Yes: GMM gives $P(\text{cluster} \mid \text{data point})$ for each cluster
- h) No: K-Means or DBSCAN give hard assignments

How large is the dataset?

- Under 10,000: any algorithm works
- 10,000 to 1,000,000: K-Means (fast), Mini-Batch K-Means (faster)
- Over 1,000,000: Mini-Batch K-Means or BIRCH. Hierarchical is too slow.

When to use each approach

K-Means: the default starting point. Fast ($O(n * k * \text{iterations})$), simple, and good enough for many problems. Use the elbow method or silhouette score to pick K. Limitations: assumes spherical clusters, sensitive to initialization (use K-Means++ or run multiple times), cannot handle varying cluster sizes well.

DBSCAN: best for discovering clusters of arbitrary shape and automatically detecting outliers. Two parameters: `eps` (neighborhood radius) and `min_samples` (minimum density). Does not require specifying K. Limitations: struggles when clusters have very different densities, and tuning `eps` can be tricky. Use a k-distance plot to estimate `eps`: compute the distance to each point's k-th nearest neighbor, sort, and look for an elbow.

Hierarchical (Agglomerative): builds a tree of merges. Useful when you want to explore cluster structure at multiple granularities (cut the dendrogram at different heights). Ward's linkage works best for compact clusters. Single linkage finds elongated clusters but is sensitive to noise. Limitations: $O(n^2)$ memory and $O(n^3)$ time, so impractical for large datasets.

GMM (Gaussian Mixture Models): soft clustering with probabilistic assignments. Models each cluster as a Gaussian distribution with its own mean and covariance. Better than K-Means when clusters are elliptical or overlapping. Use BIC (Bayesian Information Criterion) to select the number of components. Limitations: assumes Gaussian distributions, can fail on non-convex shapes, sensitive to initialization.

Evaluating cluster quality (no labels)

Metric	What it measures	Range	Use when
Silhouette score	Cohesion vs separation	-1 to 1 (higher is better)	Comparing K values or algorithms
Inertia (within-cluster SS)	Tightness of clusters	0 to ∞ (lower is better)	Elbow method for K-Means
BIC / AIC	Model fit with complexity penalty	Lower is better	Choosing number of GMM components
Calinski-Harabasz index	Ratio of between to within variance	Higher is better	Quick comparison
Davies-Bouldin index	Average similarity between clusters	Lower is better	Penalizes overlapping clusters

Common mistakes

54. Running K-Means without scaling features (features on larger scales dominate the distance calculation)
55. Picking K by eyeballing data in 2D when the actual data is high-dimensional (use silhouette scores)
56. Using K-Means on non-spherical clusters (crescent or ring-shaped data needs DBSCAN)
57. Setting DBSCAN eps too large (everything in one cluster) or too small (everything is noise)
58. Treating cluster labels as ground truth (clustering is exploratory; validate with domain knowledge)
59. Running hierarchical clustering on datasets with more than 20,000 points (memory and time explode)

Quick reference

Algorithm	Cluster shape	Finds K	Handles noise	Soft assignments	Scalability
K-Means	Spherical	No (you set K)	No	No	Millions
Mini-Batch K-Means	Spherical	No	No	No	Tens of millions
DBSCAN	Arbitrary	Yes	Yes	No	Hundreds of thousands
Hierarchical	Any (linkage-dependent)	Flexible (cut dendrogram)	Depends on linkage	No	Under 20k
GMM	Elliptical	No (you set K)	Partial (low probability)	Yes	Under 100k
HDBSCAN	Arbitrary	Yes	Yes	Partial	Hundreds of thousands

Quiz

Pre-lesson check

60. What distinguishes unsupervised learning from supervised learning?
- Unsupervised learning uses more data
 - Unsupervised learning has no labeled outputs – the algorithm finds structure on its own **(correct)**
 - Unsupervised learning only works with text data
 - Unsupervised learning always produces better results
Explanation: In unsupervised learning, there are no labels. The algorithm discovers patterns, groupings, or structure in the data without being told what the correct output should be.
61. What does K-Means require you to specify before training?

1. The exact cluster centers
2. The number of clusters K **(correct)**
3. The labels for each data point
4. The distance metric to use

Explanation: K-Means requires the number of clusters K as input. It then iteratively assigns points to the nearest centroid and recomputes centroids until convergence.

Post-lesson check

- K-Means fails on two interlocking half-moon shapes but DBSCAN succeeds. Why?
 5. DBSCAN uses more data than K-Means
 6. DBSCAN finds clusters based on density, so it can discover arbitrary shapes, while K-Means assumes spherical clusters **(correct)**
 7. DBSCAN always outperforms K-Means on every dataset
 8. K-Means cannot handle 2D data

Explanation: K-Means assigns points to the nearest centroid, producing spherical (convex) clusters. DBSCAN grows clusters from dense regions, discovering any shape as long as the cluster is connected by density.
- What is the silhouette score measuring?
 - i) The total number of clusters found
 - j) How similar each point is to its own cluster compared to the nearest other cluster **(correct)**
 - k) The speed of the clustering algorithm
 - l) The percentage of outliers in the data

Explanation: Silhouette score = $(b - a) / \max(a, b)$, where a is mean intra-cluster distance and b is mean nearest-cluster distance. It ranges from -1 (wrong cluster) to +1 (well-clustered).

- How does a Gaussian Mixture Model differ from K-Means in its cluster assignments?
 - m) GMM uses hard assignments where each point belongs to exactly one cluster
 - n) GMM gives soft (probabilistic) assignments where each point has a probability of belonging to each cluster **(correct)**
 - o) GMM does not use centroids at all
 - p) GMM only works with one-dimensional data
Explanation: K-Means assigns each point to exactly one cluster (hard). GMM computes the probability that each point belongs to each Gaussian component (soft), and can model elliptical, overlapping clusters.
-

Feature Engineering & Selection

A good feature is worth a thousand data points.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Statistics for ML, Linear Algebra), Phase 2 Lessons 1-7 **Time:** ~90 minutes

Learning Objectives

15. Implement numerical transforms (standardization, min-max scaling, log transform, binning) and explain when each is appropriate
16. Build one-hot, label, and target encoding for categorical features and identify the data leakage risk in target encoding
17. Construct a TF-IDF vectorizer from scratch and explain why it outperforms raw word counts for text classification
18. Apply filter-based feature selection (variance threshold, correlation, mutual information) to reduce dimensionality

The Problem

You have a dataset. You pick an algorithm. You train it. The results are mediocre. You try a fancier algorithm. Still mediocre. You spend a week tuning hyperparameters. Marginal improvement.

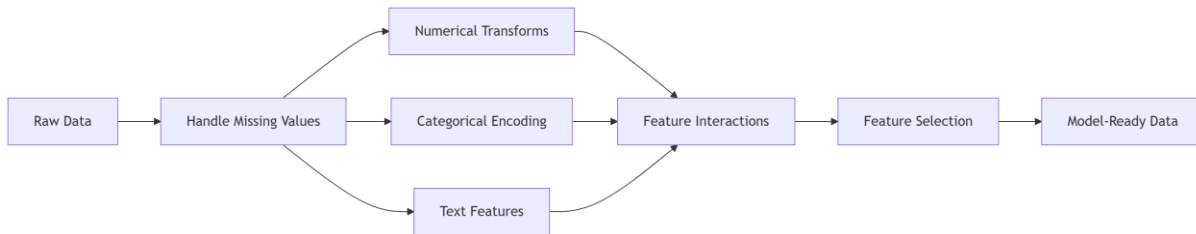
Then someone transforms the raw data into better features and a simple logistic regression beats your tuned gradient-boosted ensemble.

This happens constantly. In classical ML, the representation of the data matters more than the choice of algorithm. A house price model with “square footage” and “number of bedrooms” will beat a model with “address as a raw string” no matter how sophisticated the learner is. The algorithm can only work with what you give it.

Feature engineering is the process of transforming raw data into representations that make patterns easier for models to find. Feature selection is the process of throwing away features that add noise without adding signal. Together, they are the highest-leverage activity in classical ML.

The Concept

The Feature Pipeline



Numerical Features

Raw numbers are rarely model-ready. Common transforms:

Scaling: Put features on the same range so distance-based algorithms (K-Means, KNN, SVM) treat all features equally. Min-max scaling maps to [0, 1]. Standardization (z-score) maps to mean=0, std=1.

Log transform: Compresses right-skewed distributions (income, population, word counts). Turns multiplicative relationships into additive ones.

Binning: Converts continuous values into categories. Useful when the relationship between feature and target is non-linear but step-wise (e.g., age groups).

Polynomial features: Creates x^2 , x^3 , $x_1 * x_2$ terms. Lets linear models capture non-linear relationships at the cost of more features.

Categorical Features

Models need numbers. Categories need encoding.

One-hot encoding: Creates a binary column for each category. “color = red/blue/green” becomes three columns: `is_red`, `is_blue`, `is_green`. Works well for low-cardinality features but explodes with many categories.

Label encoding: Maps each category to an integer: red=0, blue=1, green=2. Introduces false ordering (the model might think green > blue > red). Only appropriate for tree-based models that split on individual values.

Target encoding: Replaces each category with the mean of the target variable for that category. Powerful but dangerous: high risk of data leakage. Must be computed only on training data and applied to test data.

Text Features

Count vectorizer: Counts how many times each word appears in a document. “the cat sat on the mat” becomes {the: 2, cat: 1, sat: 1, on: 1, mat: 1}.

TF-IDF: Term Frequency-Inverse Document Frequency. Weighs words by how unique they are across documents. Common words like “the” get low weight. Rare, distinctive words get high weight.

```
TF(word, doc) = count(word in doc) / total words in doc
IDF(word) = log(total docs / docs containing word)
TF-IDF = TF * IDF
```

Missing Values

Real data has holes. Strategies:

Drop rows: Only when missing data is rare and random

Mean/median imputation: Simple, preserves distribution shape (median is more robust to outliers)

Mode imputation: For categorical features

Indicator column: Add a binary column “was_this_missing” before imputing. The fact that data is missing can itself be informative

Forward/backward fill: For time series data

Feature Interaction

Sometimes the relationship is in the combination. “Height” and “weight” alone are less predictive than “BMI = weight / height²”. Feature interactions multiply the feature space, so use domain knowledge to pick the right ones.

Feature Selection

More features is not always better. Irrelevant features add noise, increase training time, and can cause overfitting.

Filter methods (pre-model): - Correlation: remove features highly correlated with each other (redundant)
- Mutual information: measures how much knowing a feature reduces uncertainty about the target -
Variance threshold: remove features that barely vary

Wrapper methods (model-based): - L1 regularization (Lasso): drives irrelevant feature weights to exactly zero - Recursive feature elimination: train, remove least important feature, repeat

Why selection matters: A model with 10 good features will usually outperform a model with 10 good features and 90 noisy ones. The noisy features give the model opportunities to overfit on training data patterns that do not generalize.

Build It

Step 1: Numerical transforms from scratch

```
import math

def min_max_scale(values):
    min_val = min(values)
    max_val = max(values)
    if max_val == min_val:
        return [0.0] * len(values)
    return [(v - min_val) / (max_val - min_val) for v in values]

def standardize(values):
    n = len(values)
    mean = sum(values) / n
    variance = sum((v - mean) ** 2 for v in values) / n
    std = math.sqrt(variance) if variance > 0 else 1.0
    return [(v - mean) / std for v in values]

def log_transform(values):
    return [math.log(v + 1) for v in values]

def bin_values(values, n_bins=5):
    min_val = min(values)
    max_val = max(values)
    bin_width = (max_val - min_val) / n_bins
    if bin_width == 0:
        return [0] * len(values)
    result = []
    for v in values:
        bin_idx = int((v - min_val) / bin_width)
        bin_idx = min(bin_idx, n_bins - 1)
        result.append(bin_idx)
    return result

def polynomial_features(row, degree=2):
    n = len(row)
    result = list(row)
    if degree >= 2:
        for i in range(n):
            result.append(row[i] ** 2)
        for i in range(n):
            for j in range(i + 1, n):
                result.append(row[i] * row[j])
    return result
```


Step 2: Categorical encoding from scratch

```
def one_hot_encode(values):
    categories = sorted(set(values))
    cat_to_idx = {cat: i for i, cat in enumerate(categories)}
    n_cats = len(categories)

    encoded = []
    for v in values:
        row = [0] * n_cats
        row[cat_to_idx[v]] = 1
        encoded.append(row)

    return encoded, categories

def label_encode(values):
    categories = sorted(set(values))
    cat_to_int = {cat: i for i, cat in enumerate(categories)}
    return [cat_to_int[v] for v in values], cat_to_int

def target_encode(feature_values, target_values, smoothing=10):
    global_mean = sum(target_values) / len(target_values)

    category_stats = {}
    for feat, target in zip(feature_values, target_values):
        if feat not in category_stats:
            category_stats[feat] = {"sum": 0.0, "count": 0}
        category_stats[feat]["sum"] += target
        category_stats[feat]["count"] += 1

    encoding = {}
    for cat, stats in category_stats.items():
        cat_mean = stats["sum"] / stats["count"]
        weight = stats["count"] / (stats["count"] + smoothing)
        encoding[cat] = weight * cat_mean + (1 - weight) * global_mean

    return [encoding[v] for v in feature_values], encoding
```

Step 3: Text features from scratch

```
def count_vectorize(documents):
    vocab = {}
    idx = 0
    for doc in documents:
        for word in doc.lower().split():
            if word not in vocab:
                vocab[word] = idx
                idx += 1

    vectors = []
```

```

for doc in documents:
    vec = [0] * len(vocab)
    for word in doc.lower().split():
        vec[vocab[word]] += 1
    vectors.append(vec)

return vectors, vocab

def tfidf(documents):
    n_docs = len(documents)

    vocab = {}
    idx = 0
    for doc in documents:
        for word in doc.lower().split():
            if word not in vocab:
                vocab[word] = idx
                idx += 1

    doc_freq = {}
    for doc in documents:
        seen = set()
        for word in doc.lower().split():
            if word not in seen:
                doc_freq[word] = doc_freq.get(word, 0) + 1
                seen.add(word)

    vectors = []
    for doc in documents:
        words = doc.lower().split()
        word_count = len(words)
        tf_map = {}
        for word in words:
            tf_map[word] = tf_map.get(word, 0) + 1

        vec = [0.0] * len(vocab)
        for word, count in tf_map.items():
            tf = count / word_count
            idf = math.log(n_docs / doc_freq[word])
            vec[vocab[word]] = tf * idf
        vectors.append(vec)

    return vectors, vocab

```

Step 4: Missing value imputation from scratch

```

def impute_mean(values):
    present = [v for v in values if v is not None]
    if not present:
        return [0.0] * len(values), 0.0
    mean = sum(present) / len(present)

```

```

    return [v if v is not None else mean for v in values], mean

def impute_median(values):
    present = sorted(v for v in values if v is not None)
    if not present:
        return [0.0] * len(values), 0.0
    n = len(present)
    if n % 2 == 0:
        median = (present[n // 2 - 1] + present[n // 2]) / 2
    else:
        median = present[n // 2]
    return [v if v is not None else median for v in values], median

def impute_mode(values):
    present = [v for v in values if v is not None]
    if not present:
        return values, None
    counts = {}
    for v in present:
        counts[v] = counts.get(v, 0) + 1
    mode = max(counts, key=counts.get)
    return [v if v is not None else mode for v in values], mode

def add_missing_indicator(values):
    return [0 if v is not None else 1 for v in values]

```

Step 5: Feature selection from scratch

```

def correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / n
    std_x = math.sqrt(sum((xi - mean_x) ** 2 for xi in x) / n)
    std_y = math.sqrt(sum((yi - mean_y) ** 2 for yi in y) / n)
    if std_x == 0 or std_y == 0:
        return 0.0
    return cov / (std_x * std_y)

def mutual_information(feature, target, n_bins=10):
    feat_min = min(feature)
    feat_max = max(feature)
    bin_width = (feat_max - feat_min) / n_bins if feat_max != feat_min else 1.0
    feat_binned = [
        min(int((f - feat_min) / bin_width), n_bins - 1) for f in feature
    ]
    n = len(feature)

```

```

target_classes = sorted(set(target))

feat_bins = sorted(set(feat_binned))
p_feat = {}
for b in feat_bins:
    p_feat[b] = feat_binned.count(b) / n

p_target = {}
for t in target_classes:
    p_target[t] = target.count(t) / n

mi = 0.0
for b in feat_bins:
    for t in target_classes:
        joint_count = sum(
            1 for fb, tv in zip(feat_binned, target) if fb == b and tv == t
        )
        p_joint = joint_count / n
        if p_joint > 0:
            mi += p_joint * math.log(p_joint / (p_feat[b] * p_target[t]))

return mi

def variance_threshold(features, threshold=0.01):
    n_features = len(features[0])
    n_samples = len(features)
    selected = []

    for j in range(n_features):
        col = [features[i][j] for i in range(n_samples)]
        mean = sum(col) / n_samples
        var = sum((v - mean) ** 2 for v in col) / n_samples
        if var >= threshold:
            selected.append(j)

    return selected

def remove_correlated(features, threshold=0.9):
    n_features = len(features[0])
    n_samples = len(features)

    to_remove = set()
    for i in range(n_features):
        if i in to_remove:
            continue
        col_i = [features[r][i] for r in range(n_samples)]
        for j in range(i + 1, n_features):
            if j in to_remove:
                continue
            col_j = [features[r][j] for r in range(n_samples)]
            corr = abs(correlation(col_i, col_j))

```

```

        if corr >= threshold:
            to_remove.add(j)

    return [i for i in range(n_features) if i not in to_remove]

```

Step 6: Full pipeline and demo

```

import random

def make_housing_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        sqft = random.uniform(500, 5000)
        bedrooms = random.choice([1, 2, 3, 4, 5])
        age = random.uniform(0, 50)
        neighborhood = random.choice(["downtown", "suburbs", "rural"])
        has_pool = random.choice([True, False])

        sqft_with_missing = sqft if random.random() > 0.05 else None
        age_with_missing = age if random.random() > 0.08 else None

        price = (
            50 * sqft
            + 20000 * bedrooms
            - 1000 * age
            + (50000 if neighborhood == "downtown" else 10000 if neighborhood == "suburbs" else 0)
            + (15000 if has_pool else 0)
            + random.gauss(0, 20000)
        )

        data.append({
            "sqft": sqft_with_missing,
            "bedrooms": bedrooms,
            "age": age_with_missing,
            "neighborhood": neighborhood,
            "has_pool": has_pool,
            "price": price,
        })
    return data

if __name__ == "__main__":
    data = make_housing_data(200)

    print("=== Raw Data Sample ===")
    for row in data[:3]:
        print(f" {row}")

    sqft_raw = [d["sqft"] for d in data]

```

```

age_raw = [d["age"] for d in data]
prices = [d["price"] for d in data]

print("\n=== Missing Value Handling ===")
sqft_missing = sum(1 for v in sqft_raw if v is None)
age_missing = sum(1 for v in age_raw if v is None)
print(f" sqft missing: {sqft_missing}/{len(sqft_raw)}")
print(f" age missing: {age_missing}/{len(age_raw)}")

sqft_indicator = add_missing_indicator(sqft_raw)
age_indicator = add_missing_indicator(age_raw)
sqft_imputed, sqft_fill = impute_median(sqft_raw)
age_imputed, age_fill = impute_mean(age_raw)
print(f" sqft filled with median: {sqft_fill:.0f}")
print(f" age filled with mean: {age_fill:.1f}")

print("\n=== Numerical Transforms ===")
sqft_scaled = standardize(sqft_imputed)
age_scaled = min_max_scale(age_imputed)
sqft_log = log_transform(sqft_imputed)
age_binned = bin_values(age_imputed, n_bins=5)
print(f" sqft standardized: mean={sum(sqft_scaled)/len(sqft_scaled):.4f}, std={math.sqrt(sum(v**2 for v in sqft_scaled)/len(sqft_scaled)):.4f}")
print(f" age min-max: [{min(age_scaled):.2f}, {max(age_scaled):.2f}]")
print(f" age bins: {sorted(set(age_binned))}")

print("\n=== Categorical Encoding ===")
neighborhoods = [d["neighborhood"] for d in data]

ohe, ohe_cats = one_hot_encode(neighborhoods)
print(f" One-hot categories: {ohe_cats}")
print(f" Sample encoding: {neighborhoods[0]} -> {ohe[0]}")

le, le_map = label_encode(neighborhoods)
print(f" Label encoding map: {le_map}")

te, te_map = target_encode(neighborhoods, prices, smoothing=10)
print(f" Target encoding: {{(k: round(v) for k, v in te_map.items())}}")

print("\n=== Text Features ===")
descriptions = [
    "large modern house with pool",
    "small cozy cottage near downtown",
    "spacious family home with large yard",
    "modern apartment downtown with view",
    "rustic cabin in rural area",
]
cv, cv_vocab = count_vectorize(descriptions)
print(f" Vocabulary size: {len(cv_vocab)}")

```

```

print(f" Doc 0 non-zero features: {sum(1 for v in cv[0] if v > 0)}")

tf, tf_vocab = tfidf(descriptions)
print(f" TF-IDF vocabulary size: {len(tf_vocab)}")
top_words = sorted(tf_vocab.keys(), key=lambda w: tf[0][tf_vocab[w]], reverse=True)
[:3]
print(f" Doc 0 top TF-IDF words: {top_words}")

print("\n=== Polynomial Features ===")
sample_row = [sqft_scaled[0], age_scaled[0]]
poly = polynomial_features(sample_row, degree=2)
print(f" Input: {[round(v, 4) for v in sample_row]}")
print(f" Polynomial: {[round(v, 4) for v in poly]}")
print(f" Features: [x1, x2, x1^2, x2^2, x1*x2]")

print("\n=== Feature Selection ===")
feature_matrix = [
    [sqft_scaled[i], age_scaled[i], float(sqft_indicator[i]), float(age_indicator[i
))]
    + ohe[i]
    for i in range(len(data))
]

print(f" Total features: {len(feature_matrix[0])}")

surviving_var = variance_threshold(feature_matrix, threshold=0.01)
print(f" After variance threshold (0.01): {len(surviving_var)} features kept")

surviving_corr = remove_correlated(feature_matrix, threshold=0.9)
print(f" After correlation filter (0.9): {len(surviving_corr)} features kept")

binary_prices = [1 if p > sum(prices) / len(prices) else 0 for p in prices]
print("\n Mutual information with target:")
feature_names = ["sqft", "age", "sqft_missing", "age_missing"] + [f"neigh_{c}" for
c in ohe_cats]
for j in range(len(feature_matrix[0])):
    col = [feature_matrix[i][j] for i in range(len(feature_matrix))]
    mi = mutual_information(col, binary_prices, n_bins=10)
    print(f" {feature_names[j]}: MI={mi:.4f}")

print("\n Correlation with price:")
for j in range(len(feature_matrix[0])):
    col = [feature_matrix[i][j] for i in range(len(feature_matrix))]
    corr = correlation(col, prices)
    print(f" {feature_names[j]}: r={corr:.4f}")

```

Use It

With scikit-learn, these transforms are composable pipelines:

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.impute import SimpleImputer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import mutual_info_classif, VarianceThreshold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("encoder", OneHotEncoder(sparse_output=False)),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, ["sqft", "age"]),
    ("cat", categorical_pipe, ["neighborhood"]),
])

```

The from-scratch versions show exactly what happens inside each transform. The library versions add edge-case handling, sparse matrix support, and pipeline composition, but the math is the same.

Exercises

Add robust scaling (using median and interquartile range instead of mean and standard deviation) to the numerical transforms. Compare it to standard scaling on data with extreme outliers.

Implement leave-one-out target encoding: for each row, compute the target mean excluding that row's own target value. Show how this reduces overfitting compared to naive target encoding.

Build an automated feature selection pipeline that combines variance threshold, correlation filtering, and mutual information ranking. Apply it to the housing dataset and compare model performance (use a simple linear regression) with all features vs selected features.

Key Terms

Term	What people say	What it actually means
Feature engineering	"Making new columns"	Transforming raw data into representations that expose patterns to the model
Standardization	"Making it normal"	Subtracting the mean and dividing by standard deviation so the feature has mean=0 and std=1
One-hot encoding	"Making dummy variables"	Creating one binary column per category, where exactly one column is 1 for each row

Term	What people say	What it actually means
Target encoding	“Using the answer to encode”	Replacing each category with the average target value for that category, with smoothing to prevent overfitting
TF-IDF	“Fancy word counts”	Term Frequency times Inverse Document Frequency: words weighted by how distinctive they are across the corpus
Imputation	“Filling in blanks”	Replacing missing values with estimated values (mean, median, mode, or model-predicted)
Feature selection	“Throwing out bad columns”	Removing features that add noise or redundancy, keeping only those with signal about the target
Mutual information	“How much one thing tells you about another”	A measure of the reduction in uncertainty about variable Y gained by observing variable X
Data leakage	“Accidentally cheating”	Using information during training that would not be available at prediction time, giving falsely optimistic results

Feature Engineering Prompt

You are a feature engineering specialist. Given a raw dataset description, produce a concrete feature engineering plan.

Input

Describe the dataset: column names, types, sample values, and the prediction target.

Process

For each column in the dataset, work through this checklist:

1. Missing values

- What percentage is missing?
- Is missingness random or informative?
- Choose strategy: drop, impute (mean/median/mode), or add a missing indicator column

2. Numerical columns

- Is the distribution skewed? If so, apply log transform
- Are units comparable across features? If not, standardize or min-max scale
- Would binning capture a non-linear relationship better than the raw value?
- Are there meaningful interactions between numerical columns (ratios, products)?

3. Categorical columns

62. How many unique values (cardinality)?
 1. Low (under 10): one-hot encode
 2. Medium (10-100): target encode with smoothing
 3. High (100+): consider hashing, embeddings, or grouping rare categories
63. Is there a natural order? If so, ordinal encoding may be appropriate

4. Text columns

- Is the text short and structured? Use TF-IDF
- Is the text long and semantic? Consider embeddings (out of scope for classical ML)
- Extract length, word count, and character count as additional features

5. Date/time columns

- Extract: year, month, day of week, hour, is_weekend
- Compute: days since a reference date, time between events
- Cyclical encoding for periodic features (hour, day of week)

6. Feature interactions

- Domain-specific combinations (e.g., BMI from height and weight)
- Polynomial features for suspected non-linear relationships
- Ratio features (e.g., price per square foot)

7. Feature selection

- Remove zero-variance features
- Remove features correlated above 0.95 with another feature
- Rank remaining features by mutual information with the target
- Keep the top N features or use L1 regularization for automatic selection

Output format

For each feature, state: 1. Original column name and type 2. Transform applied (and why) 3. New feature name(s) 4. Expected impact (high/medium/low signal)

Quiz

Pre-lesson check

64. Why is feature engineering often more impactful than choosing a fancier algorithm?

- q) Feature engineering makes the code run faster
- r) Good features expose patterns to the model that raw data hides, making even simple models effective **(correct)**
- s) Feature engineering eliminates the need for a test set
- t) Fancy algorithms cannot process raw data
Explanation: The representation of data matters more than the algorithm. A well-engineered feature like BMI (weight/height²) directly exposes the relevant pattern, making even logistic regression competitive with complex ensembles.

65. What is one-hot encoding?

- u) Replacing each category with its frequency in the dataset
- v) Creating one binary column per category, with exactly one column set to 1 per row **(correct)**
- w) Converting all features to values between 0 and 1
- x) Encoding the target variable as a probability
Explanation: One-hot encoding creates a binary column for each unique category. For a color feature with values red/blue/green, it produces three columns: is_red, is_blue, is_green.

Post-lesson check

19. What is the data leakage risk with target encoding?

- y) It makes the model too slow to train
- z) It replaces categories with the mean target value, which can leak information from the test set if not computed on training data only **(correct)**
- aa) It creates too many features

bb) It only works with binary targets

Explanation: Target encoding replaces each category with the mean target for that category. If computed on the full dataset (including test data), test labels leak into training features, inflating performance estimates.

20. TF-IDF weights a word by its inverse document frequency. What is the effect?

cc) Common words like 'the' get high weight because they appear frequently

dd) Rare, distinctive words get higher weight while common words get lower weight
(correct)

ee) All words get equal weight regardless of frequency

ff) Only the most frequent word in each document is kept

Explanation: $IDF = \log(\text{total docs} / \text{docs containing word})$. Common words (appearing in many documents) get low IDF. Rare, distinctive words get high IDF, making them more influential in the representation.

21. You have two features with correlation 0.98. Why might you remove one?

gg) Highly correlated features always cause the model to crash

hh) They are redundant – both carry nearly the same information, and keeping both increases overfitting risk without adding signal **(correct)**

ii) Correlated features make the data non-stationary

jj) Correlation above 0.5 means the features are measuring different things

Explanation: Features with $r=0.98$ are nearly redundant. Keeping both adds a noisy duplicate that increases dimensionality, overfitting risk, and multicollinearity without providing new information about the target.

Model Evaluation

A model is only as good as the way you measure it.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Probability & Distributions, Statistics for ML), Phase 2 Lessons 1-8 **Time:** ~90 minutes

Learning Objectives

- Implement K-fold and stratified K-fold cross-validation from scratch and explain why stratification matters for imbalanced data
- Compute precision, recall, F1, AUC-ROC, and regression metrics (MSE, RMSE, MAE, R-squared) from scratch
- Interpret learning curves to diagnose whether a model suffers from high bias or high variance
- Identify common evaluation mistakes including data leakage, wrong metric selection, and test set contamination

The Problem

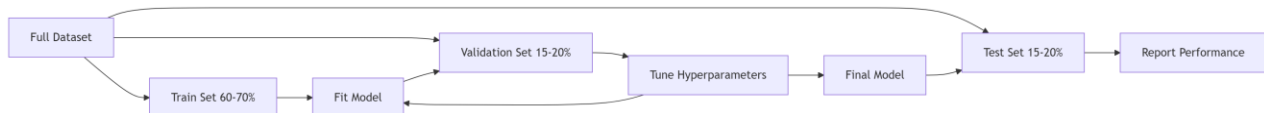
You trained a model. It gets 95% accuracy on your data. Is it good?

Maybe. Maybe not. If 95% of your data belongs to one class, a model that always predicts that class gets 95% accuracy while being completely useless. If you evaluated on the same data you trained on, the 95% number is meaningless because the model just memorized the answers. If your dataset has a time component and you randomly shuffled before splitting, your model might be using future data to predict the past.

Model evaluation is where most ML projects go wrong. The wrong metric makes a bad model look good. The wrong split lets a model cheat. The wrong comparison makes you pick the worse model. Getting evaluation right is not optional. It is the difference between a model that works in production and one that fails the moment it sees real data.

The Concept

Train, Validation, Test



Three splits, three purposes:

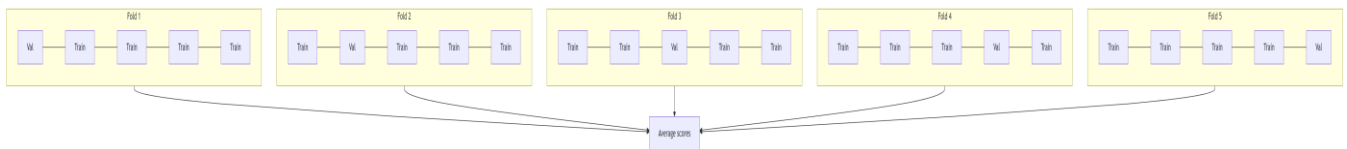
66. **Training set:** the model learns from this data. It sees these examples during training.
67. **Validation set:** used to tune hyperparameters and select between models. The model never trains on this data, but your decisions are influenced by it.

68. **Test set:** touched exactly once, at the very end, to report final performance. If you look at test performance and then go back to change your model, it is no longer a test set. It has become a second validation set.

The test set is your hold-out guarantee that the reported performance reflects how the model will do on truly unseen data.

K-Fold Cross-Validation

With small datasets, a single train/validation split wastes data and gives noisy estimates. K-fold cross-validation uses all the data for both training and validation:



- Split data into K equal-sized folds
- For each fold, train on K-1 folds and validate on the remaining fold
- Average the K validation scores

K=5 or K=10 are standard choices. Every data point gets used for validation exactly once. The average score is a more stable estimate than any single split.

Stratified K-fold: preserves the class distribution in each fold. If your dataset is 70% class A and 30% class B, each fold will have roughly the same ratio. This is important for imbalanced datasets where a random split might put all minority samples in one fold.

Classification Metrics

Confusion matrix: the foundation. For binary classification:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

From this matrix, all other metrics follow:

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$. Fraction of correct predictions. Misleading when classes are imbalanced.
- **Precision** = $TP / (TP + FP)$. Of all things predicted positive, how many actually were? Use when false positives are costly (e.g., spam filter marking real email as spam).

- **Recall** (sensitivity) = $TP / (TP + FN)$. Of all actual positives, how many did we catch? Use when false negatives are costly (e.g., cancer screening missing a tumor).
- **F1 score** = $2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$. Harmonic mean of precision and recall. Balances both when neither clearly dominates.
- **AUC-ROC**: Area Under the Receiver Operating Characteristic curve. Plots true positive rate vs false positive rate at various classification thresholds. AUC = 0.5 means random guessing, AUC = 1.0 means perfect separation. Threshold-independent: it measures how well the model ranks positives above negatives, regardless of the cutoff you pick.

Regression Metrics

- **MSE** (Mean Squared Error) = $\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)$. Penalizes large errors quadratically. Sensitive to outliers.
- **RMSE** (Root Mean Squared Error) = $\sqrt{\text{MSE}}$. Same units as the target variable. Easier to interpret than MSE.
- **MAE** (Mean Absolute Error) = $\text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$. Treats all errors linearly. More robust to outliers than MSE.
- **R-squared** = $1 - SS_{\text{res}} / SS_{\text{tot}}$, where $SS_{\text{res}} = \text{sum}((y_{\text{true}} - y_{\text{pred}})^2)$ and $SS_{\text{tot}} = \text{sum}((y_{\text{true}} - y_{\text{mean}})^2)$. Fraction of variance explained by the model. $R^2 = 1.0$ is perfect. $R^2 = 0.0$ means the model is no better than always predicting the mean. R^2 can be negative if the model is worse than the mean.

Learning Curves

Plot training and validation scores as a function of training set size:

- **High bias (underfitting)**: both curves converge to a low score. Adding more data will not help. You need a more complex model.
- **High variance (overfitting)**: training score is high but validation score is much lower. The gap between them is large. Adding more data should help.

Validation Curves

Plot training and validation scores as a function of a hyperparameter:

- At low complexity: both scores are low (underfitting)
- At the right complexity: both scores are high and close together
- At high complexity: training score stays high but validation score drops (overfitting)

The optimal hyperparameter value is where the validation score peaks.

Common Evaluation Mistakes

Data leakage: information from the test set leaks into training. Examples: fitting a scaler on the full dataset before splitting, including future data in time series prediction, using a feature that is derived from the target. Always split first, then preprocess.

Class imbalance: 99% of transactions are legitimate, 1% are fraud. A model that always predicts “legitimate” gets 99% accuracy. Use precision, recall, F1, or AUC-ROC instead.

Wrong metric: optimizing accuracy when you should optimize recall (medical diagnosis), or optimizing RMSE when your data has heavy outliers (use MAE instead).

Not using stratified splits: with imbalanced data, a random split might put very few minority samples in the validation fold, giving unstable estimates.

Testing too often: every time you look at test performance and adjust, you overfit to the test set. The test set is single-use.

Build It

Step 1: Train/validation/test split

```
import random
import math

def train_val_test_split(X, y, train_ratio=0.6, val_ratio=0.2, seed=42):
    random.seed(seed)
    n = len(X)
    indices = list(range(n))
    random.shuffle(indices)

    train_end = int(n * train_ratio)
    val_end = int(n * (train_ratio + val_ratio))

    train_idx = indices[:train_end]
    val_idx = indices[train_end:val_end]
    test_idx = indices[val_end:]

    X_train = [X[i] for i in train_idx]
    y_train = [y[i] for i in train_idx]
    X_val = [X[i] for i in val_idx]
    y_val = [y[i] for i in val_idx]
    X_test = [X[i] for i in test_idx]
    y_test = [y[i] for i in test_idx]

    return X_train, y_train, X_val, y_val, X_test, y_test
```


Step 2: K-fold and stratified K-fold cross-validation

```
def kfold_split(n, k=5, seed=42):
    random.seed(seed)
    indices = list(range(n))
    random.shuffle(indices)

    fold_size = n // k
    folds = []

    for i in range(k):
        start = i * fold_size
        end = start + fold_size if i < k - 1 else n
        val_idx = indices[start:end]
        train_idx = indices[:start] + indices[end:]
        folds.append((train_idx, val_idx))

    return folds

def stratified_kfold_split(y, k=5, seed=42):
    random.seed(seed)

    class_indices = {}
    for i, label in enumerate(y):
        class_indices.setdefault(label, []).append(i)

    for label in class_indices:
        random.shuffle(class_indices[label])

    folds = [{"train": [], "val": []} for _ in range(k)]

    for label, indices in class_indices.items():
        fold_size = len(indices) // k
        for i in range(k):
            start = i * fold_size
            end = start + fold_size if i < k - 1 else len(indices)
            val_part = indices[start:end]
            train_part = indices[:start] + indices[end:]
            folds[i]["val"].extend(val_part)
            folds[i]["train"].extend(train_part)

    return [(f["train"], f["val"]) for f in folds]

def cross_validate(X, y, model_fn, k=5, metric_fn=None, stratified=False):
    n = len(X)

    if stratified:
        folds = stratified_kfold_split(y, k)
    else:
        folds = kfold_split(n, k)
```

```

scores = []
for train_idx, val_idx in folds:
    X_train = [X[i] for i in train_idx]
    y_train = [y[i] for i in train_idx]
    X_val = [X[i] for i in val_idx]
    y_val = [y[i] for i in val_idx]

    model = model_fn()
    model.fit(X_train, y_train)
    predictions = [model.predict(x) for x in X_val]

    if metric_fn:
        score = metric_fn(y_val, predictions)
    else:
        score = sum(1 for yt, yp in zip(y_val, predictions) if yt == yp) / len(y_val)
1)    scores.append(score)

return scores

```

Step 3: Confusion matrix and classification metrics

```

def confusion_matrix(y_true, y_pred):
    tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
    tn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 0)
    fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)
    fn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 0)
    return tp, tn, fp, fn

def accuracy(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    total = tp + tn + fp + fn
    return (tp + tn) / total if total > 0 else 0.0

def precision(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    return tp / (tp + fp) if (tp + fp) > 0 else 0.0

def recall(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    return tp / (tp + fn) if (tp + fn) > 0 else 0.0

def f1_score(y_true, y_pred):
    p = precision(y_true, y_pred)
    r = recall(y_true, y_pred)
    return 2 * p * r / (p + r) if (p + r) > 0 else 0.0

```

```

def roc_curve(y_true, y_scores):
    thresholds = sorted(set(y_scores), reverse=True)
    tpr_list = []
    fpr_list = []

    total_positives = sum(y_true)
    total_negatives = len(y_true) - total_positives

    for threshold in thresholds:
        y_pred = [1 if s >= threshold else 0 for s in y_scores]
        tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
        fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)

        tpr = tp / total_positives if total_positives > 0 else 0.0
        fpr = fp / total_negatives if total_negatives > 0 else 0.0

        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return fpr_list, tpr_list, thresholds

def auc_roc(y_true, y_scores):
    fpr_list, tpr_list, _ = roc_curve(y_true, y_scores)

    pairs = sorted(zip(fpr_list, tpr_list))
    fpr_sorted = [p[0] for p in pairs]
    tpr_sorted = [p[1] for p in pairs]

    area = 0.0
    for i in range(1, len(fpr_sorted)):
        width = fpr_sorted[i] - fpr_sorted[i - 1]
        height = (tpr_sorted[i] + tpr_sorted[i - 1]) / 2
        area += width * height

    return area

```

Step 4: Regression metrics

```

def mse(y_true, y_pred):
    n = len(y_true)
    return sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred)) / n

def rmse(y_true, y_pred):
    return math.sqrt(mse(y_true, y_pred))

def mae(y_true, y_pred):
    n = len(y_true)
    return sum(abs(yt - yp) for yt, yp in zip(y_true, y_pred)) / n

```

```
def r_squared(y_true, y_pred):
    mean_y = sum(y_true) / len(y_true)
    ss_res = sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred))
    ss_tot = sum((yt - mean_y) ** 2 for yt in y_true)
    if ss_tot == 0:
        return 0.0
    return 1.0 - ss_res / ss_tot
```

Step 5: Learning curves

```
def learning_curve(X, y, model_fn, metric_fn, train_sizes=None, val_ratio=0.2, seed=42):
    :
    random.seed(seed)
    n = len(X)
    indices = list(range(n))
    random.shuffle(indices)

    val_size = int(n * val_ratio)
    val_idx = indices[:val_size]
    pool_idx = indices[val_size:]

    X_val = [X[i] for i in val_idx]
    y_val = [y[i] for i in val_idx]

    if train_sizes is None:
        train_sizes = [int(len(pool_idx) * r) for r in [0.1, 0.2, 0.4, 0.6, 0.8, 1.0]]

    train_scores = []
    val_scores = []

    for size in train_sizes:
        subset = pool_idx[:size]
        X_train = [X[i] for i in subset]
        y_train = [y[i] for i in subset]

        model = model_fn()
        model.fit(X_train, y_train)

        train_pred = [model.predict(x) for x in X_train]
        val_pred = [model.predict(x) for x in X_val]

        train_scores.append(metric_fn(y_train, train_pred))
        val_scores.append(metric_fn(y_val, val_pred))

    return train_sizes, train_scores, val_scores
```

Step 6: A simple classifier for testing, plus the full demo

```
class SimpleLogistic:
    def __init__(self, lr=0.1, epochs=100):
```

```

        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def sigmoid(self, z):
        z = max(-500, min(500, z))
        return 1.0 / (1.0 + math.exp(-z))

    def fit(self, X, y):
        n_features = len(X[0])
        self.weights = [0.0] * n_features
        self.bias = 0.0

        for _ in range(self.epochs):
            for xi, yi in zip(X, y):
                z = sum(w * x for w, x in zip(self.weights, xi)) + self.bias
                pred = self.sigmoid(z)
                error = yi - pred
                for j in range(n_features):
                    self.weights[j] += self.lr * error * xi[j]
                self.bias += self.lr * error

    def predict_proba(self, x):
        z = sum(w * xi for w, xi in zip(self.weights, x)) + self.bias
        return self.sigmoid(z)

    def predict(self, x):
        return 1 if self.predict_proba(x) >= 0.5 else 0

class SimpleLinearRegression:
    def __init__(self, lr=0.001, epochs=200):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def fit(self, X, y):
        n_features = len(X[0])
        self.weights = [0.0] * n_features
        self.bias = 0.0
        n = len(X)

        for _ in range(self.epochs):
            for xi, yi in zip(X, y):
                pred = sum(w * x for w, x in zip(self.weights, xi)) + self.bias
                error = yi - pred
                for j in range(n_features):
                    self.weights[j] += self.lr * error * xi[j] / n
                self.bias += self.lr * error / n

    def predict(self, x):

```

```

        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

def standardize(values):
    n = len(values)
    mean = sum(values) / n
    var = sum((v - mean) ** 2 for v in values) / n
    std = math.sqrt(var) if var > 0 else 1.0
    return [(v - mean) / std for v in values], mean, std

def make_classification_data(n=300, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        x1 = random.gauss(0, 1)
        x2 = random.gauss(0, 1)
        label = 1 if (x1 + x2 + random.gauss(0, 0.5)) > 0 else 0
        X.append([x1, x2])
        y.append(label)
    return X, y

def make_regression_data(n=200, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        x1 = random.uniform(0, 10)
        x2 = random.uniform(0, 5)
        target = 3 * x1 + 2 * x2 + random.gauss(0, 2)
        X.append([x1, x2])
        y.append(target)
    return X, y

def make_imbalanced_data(n=300, minority_ratio=0.05, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        if random.random() < minority_ratio:
            x1 = random.gauss(3, 0.5)
            x2 = random.gauss(3, 0.5)
            label = 1
        else:
            x1 = random.gauss(0, 1)
            x2 = random.gauss(0, 1)
            label = 0
        X.append([x1, x2])
        y.append(label)
    return X, y

```

```

if __name__ == "__main__":
    X_clf, y_clf = make_classification_data(300)

    print("=== Train/Validation/Test Split ===")
    X_train, y_train, X_val, y_val, X_test, y_test = train_val_test_split(X_clf, y_clf)
    print(f" Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
    print(f" Train class distribution: {sum(y_train)}/{len(y_train)} positive")
    print(f" Val class distribution: {sum(y_val)}/{len(y_val)} positive")

    model = SimpleLogistic(lr=0.1, epochs=200)
    model.fit(X_train, y_train)

    print("\n=== Classification Metrics ===")
    y_pred = [model.predict(x) for x in X_test]
    tp, tn, fp, fn = confusion_matrix(y_test, y_pred)
    print(f" Confusion matrix: TP={tp}, TN={tn}, FP={fp}, FN={fn}")
    print(f" Accuracy: {accuracy(y_test, y_pred):.4f}")
    print(f" Precision: {precision(y_test, y_pred):.4f}")
    print(f" Recall: {recall(y_test, y_pred):.4f}")
    print(f" F1 Score: {f1_score(y_test, y_pred):.4f}")

    y_scores = [model.predict_proba(x) for x in X_test]
    auc = auc_roc(y_test, y_scores)
    print(f" AUC-ROC: {auc:.4f}")

    print("\n=== K-Fold Cross-Validation (K=5) ===")
    cv_scores = cross_validate(
        X_clf, y_clf,
        model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
        k=5,
        metric_fn=accuracy,
    )
    mean_cv = sum(cv_scores) / len(cv_scores)
    std_cv = math.sqrt(sum((s - mean_cv) ** 2 for s in cv_scores) / len(cv_scores))
    print(f" Fold scores: {[round(s, 4) for s in cv_scores]}")
    print(f" Mean: {mean_cv:.4f} (+/- {std_cv:.4f})")

    print("\n=== Stratified K-Fold Cross-Validation (K=5) ===")
    strat_scores = cross_validate(
        X_clf, y_clf,
        model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
        k=5,
        metric_fn=accuracy,
        stratified=True,
    )
    strat_mean = sum(strat_scores) / len(strat_scores)
    strat_std = math.sqrt(sum((s - strat_mean) ** 2 for s in strat_scores) / len(strat_scores))

```

```

print(f" Fold scores: {[round(s, 4) for s in strat_scores]}")
print(f" Mean: {strat_mean:.4f} (+/- {strat_std:.4f})")

print("\n=== Imbalanced Data: Why Accuracy Lies ===")
X_imb, y_imb = make_imbalanced_data(300, minority_ratio=0.05)
positives = sum(y_imb)
print(f" Class distribution: {positives} positive, {len(y_imb) - positives} negative ({positives/len(y_imb)*100:.1f}% positive)")

always_negative = [0] * len(y_imb)
print(f" Always-negative baseline:")
print(f" Accuracy: {accuracy(y_imb, always_negative):.4f}")
print(f" Precision: {precision(y_imb, always_negative):.4f}")
print(f" Recall: {recall(y_imb, always_negative):.4f}")
print(f" F1 Score: {f1_score(y_imb, always_negative):.4f}")

X_tr_i, y_tr_i, X_v_i, y_v_i, X_te_i, y_te_i = train_val_test_split(X_imb, y_imb)
model_imb = SimpleLogistic(lr=0.5, epochs=500)
model_imb.fit(X_tr_i, y_tr_i)
y_pred_imb = [model_imb.predict(x) for x in X_te_i]
print(f"\n Trained model on imbalanced data:")
print(f" Accuracy: {accuracy(y_te_i, y_pred_imb):.4f}")
print(f" Precision: {precision(y_te_i, y_pred_imb):.4f}")
print(f" Recall: {recall(y_te_i, y_pred_imb):.4f}")
print(f" F1 Score: {f1_score(y_te_i, y_pred_imb):.4f}")

print("\n=== Regression Metrics ===")
X_reg, y_reg = make_regression_data(200)

col0 = [x[0] for x in X_reg]
col1 = [x[1] for x in X_reg]
col0_s, m0, s0 = standardize(col0)
col1_s, m1, s1 = standardize(col1)
X_reg_scaled = [[col0_s[i], col1_s[i]] for i in range(len(X_reg))]

X_tr_r, y_tr_r, X_v_r, y_v_r, X_te_r, y_te_r = train_val_test_split(X_reg_scaled, y_reg)
reg_model = SimpleLinearRegression(lr=0.01, epochs=500)
reg_model.fit(X_tr_r, y_tr_r)
y_pred_r = [reg_model.predict(x) for x in X_te_r]

print(f" MSE: {mse(y_te_r, y_pred_r):.4f}")
print(f" RMSE: {rmse(y_te_r, y_pred_r):.4f}")
print(f" MAE: {mae(y_te_r, y_pred_r):.4f}")
print(f" R-squared: {r_squared(y_te_r, y_pred_r):.4f}")

mean_baseline = [sum(y_tr_r) / len(y_tr_r)] * len(y_te_r)
print(f"\n Mean baseline:")

```



```

print(f"    MSE:        {mse(y_te_r, mean_baseline):.4f}")
print(f"    R-squared: {r_squared(y_te_r, mean_baseline):.4f}")

print("\n=== Learning Curve ===")
sizes, train_sc, val_sc = learning_curve(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
    metric_fn=accuracy,
)
print(f"  {'Size':>6} {'Train':>8} {'Val':>8}")
for s, tr, va in zip(sizes, train_sc, val_sc):
    print(f"  {s:>6} {tr:>8.4f} {va:>8.4f}")

print("\n=== Statistical Model Comparison ===")
model_a_scores = cross_validate(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=100),
    k=5, metric_fn=accuracy,
)
model_b_scores = cross_validate(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=500),
    k=5, metric_fn=accuracy,
)
diffs = [a - b for a, b in zip(model_a_scores, model_b_scores)]
mean_diff = sum(diffs) / len(diffs)
std_diff = math.sqrt(sum((d - mean_diff) ** 2 for d in diffs) / len(diffs))
t_stat = mean_diff / (std_diff / math.sqrt(len(diffs))) if std_diff > 0 else 0.0
print(f"  Model A (100 epochs) mean: {sum(model_a_scores)/len(model_a_scores):.4f
}")
print(f"  Model B (500 epochs) mean: {sum(model_b_scores)/len(model_b_scores):.4f
}")
print(f"  Mean difference: {mean_diff:.4f}")
print(f"  Paired t-statistic: {t_stat:.4f}")
print(f"  (|t| > 2.78 for significance at p<0.05 with df=4)")

```

Use It

With scikit-learn, evaluation is built into the workflow:

```

from sklearn.model_selection import cross_val_score, StratifiedKFold, learning_curve
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, mean_squared_error, r2_score,
)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=StratifiedKFold(5), scoring="f1")

```

The from-scratch versions show exactly what cross-validation does (no magic, just for-loops and index tracking), how each metric is computed (just counting TP/FP/TN/FN), and why stratification matters (preserving class ratios in each fold). The library versions add parallelism, more scoring options, and integration with pipelines.

Exercises

- Implement precision-recall curves: plot precision vs recall at different thresholds. Compute the average precision (area under the PR curve). Compare the PR curve to the ROC curve on an imbalanced dataset and explain when each is more informative.
- Build a nested cross-validation loop: the outer loop evaluates model performance, the inner loop tunes hyperparameters. Use it to compare two models fairly without leaking validation data into the evaluation.
- Implement a permutation test for model comparison: shuffle the labels, retrain, and measure performance. Repeat 100 times to build a null distribution. Compute the p-value for the observed model performance against this distribution.

Key Terms

Term	What people say	What it actually means
Overfitting	“Memorizing the training data”	The model captures noise in the training data, performing well on training but poorly on unseen data
Cross-validation	“Testing on different subsets”	Systematically rotating which portion of data is used for validation, averaging results across all rotations
Precision	“How many predicted positives are correct”	$TP / (TP + FP)$: the fraction of positive predictions that are actually positive
Recall	“How many actual positives we found”	$TP / (TP + FN)$: the fraction of actual positives that were correctly identified
AUC-ROC	“How well the model separates classes”	The area under the curve of true positive rate vs false positive rate across all thresholds, from 0.5 (random) to 1.0 (perfect)
R-squared	“How much variance is explained”	$1 - (\text{sum of squared residuals} / \text{total sum of squares})$: the fraction of target variance captured by the model
Data leakage	“The model cheated”	Using information during training that would not be available at prediction time, leading to optimistic evaluation
Learning curve	“How performance changes with more data”	A plot of training and validation scores vs training set size, revealing underfitting or overfitting

Term	What people say	What it actually means
Stratified split	“Keeping class ratios balanced”	Splitting data so each subset has the same proportion of each class as the full dataset

Model Evaluation Strategy

A checklist for correctly evaluating any ML model. Follow this sequence to avoid the most common evaluation mistakes.

Step 1: Split the data correctly

Split before any preprocessing (scaling, imputation, encoding)

Use stratified splits for classification tasks

Reserve a test set that you touch exactly once at the end

For small datasets, use 5-fold or 10-fold cross-validation instead of a single split

For time series, use time-based splits (never shuffle)

Step 2: Pick the right metric

Classification

Situation	Use this metric	Why
Balanced classes, simple comparison	Accuracy	Easy to interpret, meaningful when classes are equal
False positives are costly (spam filter, fraud alerts)	Precision	Measures how many flagged items are actually positive
False negatives are costly (cancer screening, security)	Recall	Measures how many actual positives you catch
Need to balance precision and recall	F1 Score	Harmonic mean, punishes extreme imbalance
Comparing models across thresholds	AUC-ROC	Threshold-independent ranking quality
Imbalanced data	F1, AUC-ROC, or PR-AUC	Accuracy is misleading with imbalanced classes

Regression

Situation	Use this metric	Why
Standard regression, outliers acceptable	RMSE	Same units as target, penalizes large errors
Outlier-robust evaluation	MAE	Treats all errors equally, not dominated by outliers
Comparing models on different scales	R-squared	Normalized 0-1 scale (fraction of variance explained)
Business requires dollar amounts	MAE or RMSE	Directly interpretable as error magnitude

Step 3: Establish baselines

Before evaluating your model, compute baseline performance: - Classification: majority class predictor (always predict the most common class) - Regression: always predict the mean of the training target - Any model that cannot beat these baselines is not learning

Step 4: Cross-validate

Use K-fold (K=5 or K=10) for stable estimates

Use stratified K-fold for classification

Report mean and standard deviation across folds

A model with mean=0.85 and std=0.02 is more trustworthy than mean=0.87 and std=0.10

Step 5: Compare models statistically

Do not pick the model with the highest average score without checking significance

Use a paired t-test across cross-validation folds

If $|t| < 2.78$ (for K=5, df=4, $p < 0.05$), the difference may be due to chance

Consider the simpler model when performance differences are not significant

Step 6: Check for common mistakes

Data leakage: did any test data information flow into training? (scaling before splitting, target-derived features)

Class imbalance: is accuracy hiding poor minority-class performance?

Overfitting: is the gap between training and validation performance large?

Too many evaluations: have you looked at the test set more than once?

Step 7: Report final performance

Train on train + validation combined

Evaluate on the held-out test set exactly once

Report the chosen metric with confidence intervals if possible

State the baseline comparison (how much better than random/mean)

Quiz

Pre-lesson check

Why should you never tune hyperparameters based on test set performance?

- The test set is too small for reliable estimates
- Adjusting the model based on test results effectively trains on the test set, making reported performance meaningless **(correct)**
- Hyperparameters cannot be changed after training
- The test set always has different features than the training set
Explanation: Every time you adjust your model based on test performance, you leak test information into your modeling decisions. The test set must be used exactly once at the end for an unbiased estimate.

A dataset has 95% negative and 5% positive samples. A model predicts 'negative' for every sample. What is its accuracy?

1. 50%
2. 5%
3. 95% **(correct)**
4. 0%

Explanation: Accuracy = correct predictions / total = 950/1000 = 95%. This shows why accuracy is misleading for imbalanced data – a useless model looks great.

Post-lesson check

69. In K-fold cross-validation with K=5, how many times is each data point used for validation?

- 5 times
- Exactly once **(correct)**
- It depends on the random seed
- Never – all data is used for training

Explanation: In K-fold CV, data is split into K equal folds. Each fold is used as the validation set exactly once while the remaining K-1 folds are used for training.

70. A learning curve shows training score = 0.95 and validation score = 0.60 that does not improve with more data. What should you try?

1. Collect more training data
2. Use a simpler model or add regularization to reduce variance (overfitting) **(correct)**
3. Remove the validation set to give the model more training data
4. Increase the learning rate

Explanation: A large gap between training (high) and validation (low) scores is high variance (overfitting). The fix is a simpler model, more regularization, or techniques like dropout – not more data if the gap persists.

71. AUC-ROC = 0.5 for a binary classifier. What does this indicate?

- The model perfectly separates the two classes
- The model performs no better than random guessing at ranking positives above negatives **(correct)**
- The model has 50% accuracy
- The model has equal precision and recall

Explanation: AUC-ROC = 0.5 means the model's ranking of positive and negative examples is no better than random. AUC = 1.0 would be perfect separation. The metric is threshold-independent.

Bias-Variance Tradeoff

Every model error comes from one of three sources: bias, variance, or noise. You can only control the first two.

Type: Learn **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 (ML basics, regression, classification, evaluation) **Time:** ~75 minutes

Learning Objectives

72. Derive the bias-variance decomposition of expected prediction error and explain the role of irreducible noise
73. Diagnose whether a model suffers from high bias or high variance using training and test error patterns
74. Explain how regularization techniques (L1, L2, dropout, early stopping) trade bias for variance
75. Implement experiments that visualize the bias-variance tradeoff across models of increasing complexity

The Problem

You trained a model. It has some error on test data. Where does that error come from?

If your model is too simple (linear regression on a curved dataset), it will consistently miss the true pattern. That is bias. If your model is too complex (degree-20 polynomial on 15 data points), it will fit the training data perfectly but give wildly different predictions on new data. That is variance.

You cannot minimize both at the same time for a fixed model capacity. Push bias down and variance goes up. Push variance down and bias goes up. Understanding this tradeoff is the single most useful diagnostic skill in machine learning. It tells you whether to make your model more complex or less complex, whether to get more data or engineer better features, whether to regularize more or less.

The Concept

Bias: Systematic Error

Bias measures how far off your model's average prediction is from the true value. If you trained the same model on many different training sets drawn from the same distribution and averaged the predictions, bias is the gap between that average and the truth.

High bias means the model is too rigid to capture the real pattern. A straight line fit to a parabola will always miss the curve, no matter how much data you give it. This is underfitting.

High bias (underfitting):
Model always predicts roughly the same wrong thing.
Training error: HIGH

Test error: HIGH
Gap between them: SMALL

Variance: Sensitivity to Training Data

Variance measures how much your predictions change when you train on different subsets of data. If small changes in the training set cause large changes in the model, variance is high.

High variance means the model is fitting noise in the training data, not the underlying signal. A degree-20 polynomial will thread through every training point but oscillate wildly between them. This is overfitting.

High variance (overfitting):
Model fits training data perfectly but fails on new data.
Training error: LOW
Test error: HIGH
Gap between them: LARGE

The Decomposition

For any point x , the expected prediction error under squared loss decomposes exactly:

Expected Error = Bias² + Variance + Irreducible Noise

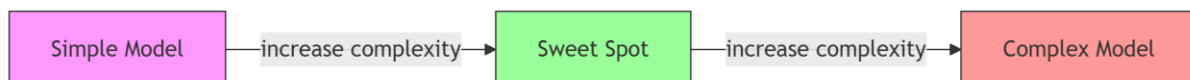
where:

Bias² = $(E[f_{\text{hat}}(x)] - f(x))^2$
Variance = $E[(f_{\text{hat}}(x) - E[f_{\text{hat}}(x)])^2]$
Noise = $E[(y - f(x))^2]$ (σ^2)

- ff) $f(x)$ is the true function
- gg) $f_{\text{hat}}(x)$ is your model's prediction
- hh) $E[\dots]$ is the expectation over different training sets
- ii) y is the observed label (true function plus noise)

The noise term is irreducible. No model can do better than σ^2 on noisy data. Your job is to find the right balance between bias² and variance.

Model Complexity vs Error



The classic U-shaped curve:

Complexity	Bias	Variance	Total Error
Too low	HIGH	LOW	HIGH (underfitting)
Just right	MODERATE	MODERATE	LOWEST
Too high	LOW	HIGH	HIGH (overfitting)

Regularization as Bias-Variance Control

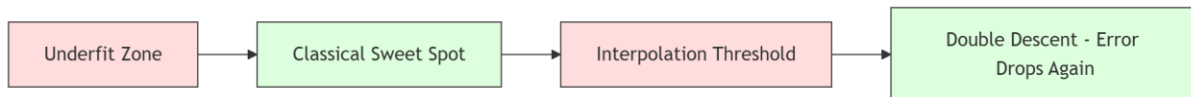
Regularization deliberately increases bias to reduce variance. It constrains the model so it cannot chase noise.

- jj) **L2 (Ridge)**: Shrinks all weights toward zero. Keeps all features but reduces their influence.
- kk) **L1 (Lasso)**: Pushes some weights exactly to zero. Performs feature selection.
- ll) **Dropout**: Randomly disables neurons during training. Forces redundant representations.
- mm) **Early stopping**: Stops training before the model fully fits the training data.

The regularization strength (λ , dropout rate, number of epochs) directly controls where you sit on the bias-variance curve. More regularization means more bias, less variance.

Double Descent: The Modern Perspective

Classical theory says: after the sweet spot, more complexity always hurts. But research since 2019 has shown something unexpected. If you keep increasing model capacity far past the interpolation threshold (where the model has enough parameters to perfectly fit training data), test error can decrease again.



This “double descent” phenomenon explains why massively overparameterized neural networks (with far more parameters than training examples) still generalize well. The classical bias-variance tradeoff is not wrong, but it is incomplete for the modern regime.

Key observations about double descent: - It happens in linear models, decision trees, and neural networks - More data can actually hurt in the interpolation region (sample-wise double descent) - More training epochs can cause it too (epoch-wise double descent) - Regularization smooths out the peak but does not eliminate it

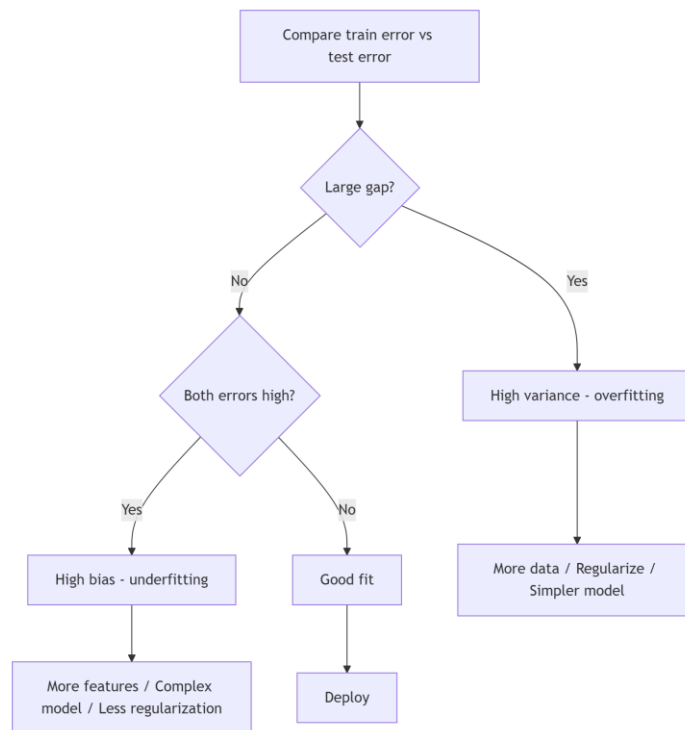
Why does this happen? At the interpolation threshold, the model has just enough capacity to fit all training points. It is forced into a very specific solution that threads through every point, and small perturbations in the data cause large changes in the fit. This is where variance peaks. Past the threshold, the model has many possible solutions that fit the data perfectly. The learning algorithm (e.g., gradient

descent with implicit regularization) tends to pick the simplest one among them. This implicit bias toward simple solutions is why overparameterized models generalize.

Regime	Parameters vs Samples	Behavior
Underparameterized	$p \ll n$	Classical tradeoff applies
Interpolation threshold	$p \sim n$	Variance peaks, test error spikes
Overparameterized	$p \gg n$	Implicit regularization kicks in, test error drops

For practical purposes: if you are using neural networks or large tree ensembles, do not stop at the interpolation threshold. Either stay well below it (with explicit regularization) or go well past it. The worst place to be is right at the threshold.

Diagnosing Your Model



Symptom	Diagnosis	Fix
High train error, high test error	Bias	More features, complex model, less regularization
Low train error, high test error	Variance	More data, regularization, simpler model, dropout

Symptom	Diagnosis	Fix
Low train error, low test error	Good fit	Ship it
Train error decreasing, test error increasing	Overfitting in progress	Early stopping

Practical Strategies

When bias is the problem: - Add polynomial or interaction features - Use a more flexible model (tree ensemble instead of linear) - Reduce regularization strength - Train longer (if not yet converged)

When variance is the problem: - Get more training data - Use bagging (random forests) - Increase regularization (higher λ , more dropout) - Feature selection (remove noisy features) - Use cross-validation to detect it early

Ensemble Methods and Variance Reduction

Ensemble methods are the most practical tool for fighting variance.

Bagging (Bootstrap Aggregating) trains multiple models on different bootstrap samples of the training data, then averages their predictions. Each individual model has high variance, but the average has much lower variance. Random forests are bagging applied to decision trees.

Why it works mathematically: if you average N independent predictions, each with variance σ^2 , the variance of the average is σ^2 / N . The models are not truly independent (they all see similar data), so the reduction is less than $1/N$, but it is still substantial.

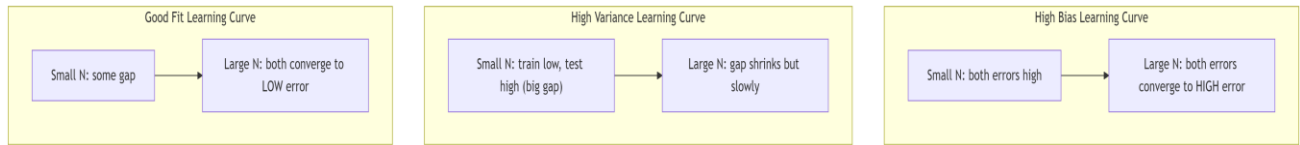
Boosting reduces bias by building models sequentially, where each new model focuses on the errors of the ensemble so far. Gradient boosting and AdaBoost are the main examples. Boosting can overfit if you add too many models, so you need early stopping or regularization.

Method	Primary Effect	Bias Change	Variance Change
Bagging	Reduces variance	No change	Decreases
Boosting	Reduces bias	Decreases	Can increase
Stacking	Reduces both	Depends on meta-learner	Depends on base models
Dropout	Implicit bagging	Slight increase	Decreases

Practical rule: if your base model has high variance (deep trees, high-degree polynomials), use bagging. If your base model has high bias (shallow stumps, simple linear models), use boosting.

Learning Curves

Learning curves plot training and validation error as a function of training set size. They are the most practical diagnostic tool you have. Unlike a single train/test comparison, learning curves show you the trajectory of your model and tell you whether more data will help.



How to read them:

Scenario	Training Error	Validation Error	Gap	What It Means	What to Do
High bias	High	High	Small	Model cannot capture the pattern	More features, complex model, less regularization
High variance	Low	High	Large	Model memorizes training data	More data, regularization, simpler model
Good fit	Moderate	Moderate	Small	Model generalizes well	Ship it
High variance, improving	Low	Decreasing with more data	Shrinking	Variance problem that data can fix	Collect more data
High bias, flat	High	High and flat	Small and flat	More data will NOT help	Change model architecture

The critical insight: if both curves have plateaued and the gap is small but both errors are high, more data is useless. You need a better model. If the gap is large and still shrinking, more data will help.

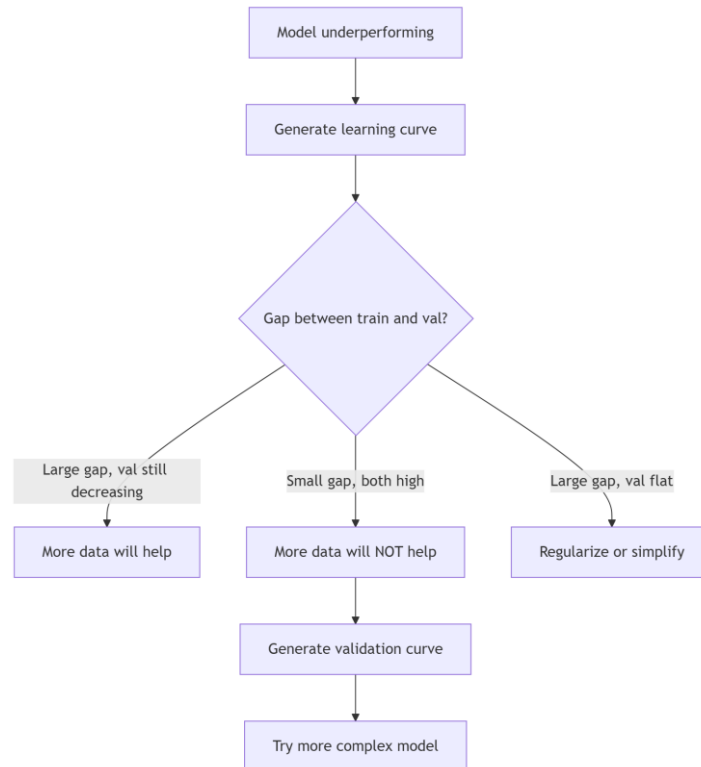
How to Generate Learning Curves

There are two approaches:

Approach 1: Vary training set size, fixed model. Hold the model and hyperparameters constant. Train on increasingly large subsets of the training data. Measure training error and validation error at each size. This is the standard learning curve.

Approach 2: Vary model complexity, fixed data. Hold the data constant. Sweep a complexity parameter (polynomial degree, tree depth, number of layers). Measure training error and validation error at each complexity. This is a validation curve and shows the bias-variance tradeoff directly.

Both approaches complement each other. The first tells you if more data will help. The second tells you if a different model will help. Run both before making decisions about your next step.



Build It

The code in `code/bias_variance.py` runs the full bias-variance decomposition experiment. Here is the approach, step by step.

Step 1: Generate Synthetic Data from a Known Function

We use $f(x) = \sin(1.5x) + 0.5x$ with Gaussian noise. Knowing the true function lets us compute exact bias and variance.

```
def true_function(x):  
    return np.sin(1.5 * x) + 0.5 * x  
  
def generate_data(n_samples=30, noise_std=0.5, x_range=(-3, 3), seed=None):  
    rng = np.random.RandomState(seed)  
    x = rng.uniform(x_range[0], x_range[1], n_samples)  
    y = true_function(x) + rng.normal(0, noise_std, n_samples)  
    return x, y
```

Step 2: Bootstrap Sampling and Polynomial Fitting

For each polynomial degree, we draw many bootstrap training sets, fit the polynomial, and record predictions on a fixed test grid. This gives us a distribution of predictions at each test point.

```
def fit_polynomial(x_train, y_train, degree, lam=0.0):
    X = np.column_stack([x_train ** d for d in range(degree + 1)])
    if lam > 0:
        penalty = lam * np.eye(X.shape[1])
        penalty[0, 0] = 0
        w = np.linalg.solve(X.T @ X + penalty, X.T @ y_train)
    else:
        w = np.linalg.lstsq(X, y_train, rcond=None)[0]
    return w
```

We fit on 200 different bootstrap samples. Each bootstrap sample is drawn from the same underlying distribution but contains different points.

Step 3: Computing Bias², Variance Decomposition

With 200 sets of predictions at each test point, we can compute the decomposition directly from the definition:

```
mean_pred = predictions.mean(axis=0)
bias_sq = np.mean((mean_pred - y_true) ** 2)
variance = np.mean(predictions.var(axis=0))
total_error = np.mean(np.mean((predictions - y_true) ** 2, axis=1))
```

- 22. `mean_pred` is $E[\hat{f}(x)]$ estimated from bootstrap samples
- 23. `bias_sq` is the squared gap between average prediction and truth
- 24. `variance` is the average spread of predictions across bootstrap samples
- 25. `total_error` should approximately equal $\text{bias}^2 + \text{variance} + \text{noise}$

Step 4: Learning Curves

Learning curves sweep training set size while holding model complexity fixed. They show whether your model is data-limited or capacity-limited.

```
def demo_learning_curves():
    sizes = [10, 15, 20, 30, 50, 75, 100, 150, 200, 300]
    degree = 5

    for n in sizes:
        train_errors = []
        test_errors = []
        for seed in range(50):
            x_train, y_train = generate_data(n_samples=n, seed=seed * 100)
```

```

w = fit_polynomial(x_train, y_train, degree)
train_pred = predict_polynomial(x_train, w)
train_mse = np.mean((train_pred - y_train) ** 2)
test_pred = predict_polynomial(x_test, w)
test_mse = np.mean((test_pred - y_test) ** 2)
train_errors.append(train_mse)
test_errors.append(test_mse)
# Average over runs gives the Learning curve point

```

For a high-variance model (degree 5 with small data), you see: - Training error starts low and increases as more data makes memorization harder - Test error starts high and decreases as the model gets more signal - The gap shrinks with more data

For a high-bias model (degree 1), both errors converge quickly to the same high value and more data does not help.

Step 5: Regularization Sweep

The code also includes `demo_regularization_sweep()`, which fixes a high-degree polynomial (degree 15) and sweeps Ridge regularization strength from 0.001 to 100. This shows the bias-variance tradeoff from a different angle: instead of varying model complexity, we vary the constraint strength.

```

def demo_regularization_sweep():
    alphas = [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
    for alpha in alphas:
        results = bias_variance_decomposition([15], lam=alpha)
        r = results[15]
        print(f"alpha={alpha:.3f}  bias={r['bias_sq']:.4f}  var={r['variance']:.4f}")

```

At low α , the degree-15 polynomial is nearly unconstrained. Variance dominates because the model chases noise in each bootstrap sample. At high α , the penalty is so strong that the model effectively becomes a near-constant function. Bias dominates. The optimal α sits between these extremes.

This is the same U-curve from varying polynomial degree, but controlled by a continuous knob instead of a discrete one. In practice, regularization is the preferred way to control the tradeoff because it allows fine-grained control without changing the feature set.

Use It

sklearn provides `learning_curve` and `validation_curve` to automate these diagnostics without writing bootstrap loops.

Validation Curve: Sweep Model Complexity

```

from sklearn.model_selection import validation_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

```



```

from sklearn.linear_model import Ridge

degrees = list(range(1, 16))
train_scores_all = []
val_scores_all = []

for d in degrees:
    pipe = make_pipeline(PolynomialFeatures(d), Ridge(alpha=0.01))
    train_scores, val_scores = validation_curve(
        pipe, X, y, param_name="polynomialfeatures__degree",
        param_range=[d], cv=5, scoring="neg_mean_squared_error"
    )
    train_scores_all.append(-train_scores.mean())
    val_scores_all.append(-val_scores.mean())

```

This gives you the bias-variance tradeoff curve directly. Where the validation score is worst relative to train score, variance dominates. Where both are bad, bias dominates.

Learning Curve: Sweep Training Set Size

```

from sklearn.model_selection import learning_curve

pipe = make_pipeline(PolynomialFeatures(5), Ridge(alpha=0.01))
train_sizes, train_scores, val_scores = learning_curve(
    pipe, X, y, train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5, scoring="neg_mean_squared_error"
)
train_mse = -train_scores.mean(axis=1)
val_mse = -val_scores.mean(axis=1)

```

Plot `train_mse` and `val_mse` against `train_sizes`. The shape tells you everything about your model.

Cross-Validation with Regularization Sweep

```

from sklearn.model_selection import cross_val_score

alphas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
for alpha in alphas:
    pipe = make_pipeline(PolynomialFeatures(10), Ridge(alpha=alpha))
    scores = cross_val_score(pipe, X, y, cv=5, scoring="neg_mean_squared_error")
    print(f"alpha={alpha:>7.3f} MSE={-scores.mean():.4f} +/- {scores.std():.4f}")

```

This sweeps regularization strength for a fixed model complexity. You will see the same bias-variance tradeoff: low α means high variance, high α means high bias.

Putting It All Together: A Complete Diagnostic Workflow

In practice, you run these diagnostics in sequence:

Train your model. Compute train and test error.

If both are high: you have a bias problem. Skip to step 4.

If train is low but test is high: you have a variance problem. Generate a learning curve to see if more data will help. If not, regularize.

Generate a validation curve sweeping your main complexity parameter. Find the sweet spot.

At the sweet spot, generate a learning curve. If the gap is still large, you need more data or regularization.

Try Ridge/Lasso with different α values using `cross_val_score`. Pick the α where cross-validated error is lowest.

This takes 10-15 minutes of compute for most tabular datasets and saves hours of guessing.

Exercises

Run the decomposition with `noise_std=0` (no noise). What happens to the irreducible error term? Does the optimal complexity change?

Increase the training set size from 30 to 300. How does this affect the variance component? Does the optimal polynomial degree shift?

Add L2 regularization (Ridge regression) to the experiment. For a fixed high-degree polynomial (degree 15), sweep λ from 0 to 100. Plot bias^2 and variance as functions of λ .

Modify the true function from a polynomial to $\sin(x)$. How does the bias-variance decomposition change? Is there still a clear optimal degree?

Implement a simple bootstrap aggregating (bagging) wrapper: train 10 models on bootstrap samples and average predictions. Show that this reduces variance without increasing bias much.

Key Terms

Term	What people say	What it actually means
Bias	"The model is too simple"	Systematic error from wrong assumptions. The gap between the average model prediction and truth.
Variance	"The model is overfitting"	Error from sensitivity to training data. How much predictions change across different training sets.
Irreducible error	"Noise in the data"	Error from randomness in the true data-generating process. No model can eliminate it.
Underfitting	"Not learning enough"	Model has high bias. It misses the real pattern even on training data.

Term	What people say	What it actually means
Overfitting	“Memorizing the data”	Model has high variance. It fits noise in training data that does not generalize.
Regularization	“Constraining the model”	Adding a penalty to reduce model complexity, trading bias for lower variance.
Double descent	“More parameters can help”	Test error decreases again when model capacity far exceeds the interpolation threshold.
Model complexity	“How flexible the model is”	The capacity of a model to fit arbitrary patterns. Controlled by architecture, features, or regularization.

You are a model diagnostics specialist. Given a model’s training and test metrics (and optionally a learning curve), you identify whether the problem is high bias, high variance, or something else, and recommend specific fixes.

When a user provides model metrics, work through each step:

Step 1: Compare train and test performance

Ask the user for: - Training set metric (accuracy, MSE, F1, etc.) - Test/validation set metric (same metric) - Dataset size (number of samples) - Model type and complexity (e.g., “random forest with max_depth=20” or “linear regression with 5 features”)

Step 2: Diagnose the problem

Use this framework:

High bias (underfitting): - Training error is high - Test error is high - Gap between them is small - The model is too simple to capture the pattern

High variance (overfitting): - Training error is low - Test error is high - Gap between them is large (more than 10-15% relative) - The model is memorizing the training data

Good fit: - Training error is reasonably low - Test error is close to training error - Both are at an acceptable level for the problem

Data quality issue: - Training error is suspiciously low (close to 0) but the model is simple - Possible data leakage: a feature is encoding the target - Check for duplicate rows between train and test

Noise floor: - Both errors are moderate, gap is small, and no model improvement seems to help - You may have hit the irreducible error from noise in the data - Better features or more data are the only paths forward

Step 3: Interpret the learning curve (if provided)

A learning curve plots train and test error vs training set size.

High bias learning curve: - Both curves converge quickly to a high error - They are close together - Meaning: more data will not help. The model needs more capacity.

High variance learning curve: - Large gap between train (low) and test (high) - The gap shrinks as data increases - Meaning: more data will help. Alternatively, regularize or simplify.

Good fit learning curve: - Both curves converge to a low error - Small gap that stabilizes

If train error increases and test error decreases as data grows: - This is normal. With more data, the model cannot memorize as easily (train error rises), but it learns the true pattern better (test error drops).

Step 4: Recommend specific fixes

For high bias: 1. Add polynomial or interaction features 2. Use a more flexible model (e.g., tree ensemble instead of linear model) 3. Reduce regularization strength (lower α/λ) 4. Engineer domain-specific features 5. Train longer (if optimization has not converged)

For high variance: 1. Get more training data (most reliable fix) 2. Increase regularization (higher α/λ , add dropout) 3. Reduce model complexity (shallower trees, fewer features) 4. Use bagging or a random forest (averaging reduces variance) 5. Feature selection (remove noisy or irrelevant features) 6. Use cross-validation to get a more stable performance estimate

For noise floor: 1. Collect better features (new data sources, domain expertise) 2. Clean existing data (fix labeling errors, remove contradictory samples) 3. Accept the current performance as the best achievable

Output format

Structure your response as: 1. **Diagnosis:** [high bias / high variance / good fit / data issue / noise floor] 2. **Evidence:** [specific numbers from the metrics that support this] 3. **Root cause:** [why this is happening given the model and data] 4. **Fixes (ranked):** [ordered list from most impactful to least] 5. **What NOT to do:** [common wrong response to this diagnosis]

Avoid: - Recommending “get more data” as the first fix for high bias (it will not help) - Suggesting a more complex model for high variance (it will make things worse) - Diagnosing overfitting when both train and test errors are high (that is underfitting) - Ignoring the possibility of data leakage when training accuracy is near 100%

Quiz

Pre-lesson check

- A linear model is used to fit a clearly curved (quadratic) relationship. Which error component dominates?
 1. Variance: the model changes too much with different training data
 2. Bias: the model is too rigid to capture the true nonlinear pattern **(correct)**
 3. Irreducible noise: the data is too noisy
 4. None: the model should fit perfectly

Explanation: A linear model cannot capture a quadratic curve no matter how much data it sees. This systematic error from wrong model assumptions is bias. The model underfits.
- The bias-variance decomposition of expected error has three terms. Which one cannot be reduced by any model?
 - Bias squared
 - Variance
 - Irreducible noise (σ squared) **(correct)**
 - All three can be reduced to zero

Explanation: Irreducible noise comes from randomness in the data itself (measurement error, missing variables). No model can predict noise. Expected error = bias² + variance + irreducible noise.

Post-lesson check

76. Adding L2 regularization to a model increases bias and decreases variance. Why is this useful?
- It always improves both training and test accuracy
 - The reduction in variance can outweigh the increase in bias, lowering total error **(correct)**
 - L2 regularization eliminates irreducible noise
 - It makes the model faster to train

Explanation: Regularization trades a small increase in bias for a larger decrease in

variance. When a model is overfitting (high variance), this tradeoff reduces total error even though bias goes up slightly.

77. A model has training error = 2% and test error = 25%. What is the most likely diagnosis?

1. High bias (underfitting): the model is too simple
2. High variance (overfitting): the model memorized training data and fails to generalize **(correct)**
3. High irreducible noise: the data is too noisy

4. The model is perfectly calibrated

Explanation: Low training error + high test error + large gap = high variance (overfitting). The model fits training-specific noise. Remedies: regularize, reduce complexity, get more data.

78. You train the same model architecture on 50 different random training subsets and observe that predictions vary wildly between them. What does this indicate?

- High bias: the model consistently misses the true pattern
- High variance: the model is sensitive to which specific training data it sees **(correct)**
- High irreducible noise: the target variable is random

- The learning rate is too high

Explanation: Variance measures how much predictions change when trained on different data subsets. Wildly different predictions across subsets is the definition of high variance.

Ensemble Methods

A group of weak learners, combined correctly, becomes a strong learner. This is not a metaphor. It is a theorem.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 10 (Bias-Variance Tradeoff) **Time:** ~120 minutes

Learning Objectives

- Implement AdaBoost and gradient boosting from scratch and explain how boosting sequentially reduces bias
- Build a bagging ensemble and demonstrate how averaging decorrelated models reduces variance without increasing bias
- Compare bagging, boosting, and stacking in terms of what error component each method targets
- Evaluate ensemble diversity and explain why majority voting accuracy improves with more independent weak learners

The Problem

A single decision tree is fast to train and easy to interpret, but it overfits. A single linear model underfits on complex boundaries. You could spend days engineering the perfect model architecture. Or you could combine a bunch of imperfect models and get something better than any of them individually.

Ensemble methods do exactly this. They are the most reliable technique for winning Kaggle competitions on tabular data, they power most production ML systems, and they illustrate the bias-variance tradeoff in action. Bagging reduces variance. Boosting reduces bias. Stacking learns which models to trust on which inputs.

The Concept

Why Ensembles Work

Suppose you have N independent classifiers, each with accuracy $p > 0.5$. The majority vote has accuracy:

$$P(\text{majority correct}) = \sum_{k > N/2} C(N, k) * p^k * (1-p)^{(N-k)}$$

For 21 classifiers each with 60% accuracy, majority vote accuracy is about 74%. With 101 classifiers, it rises to 84%. The errors cancel out when the models make different mistakes.

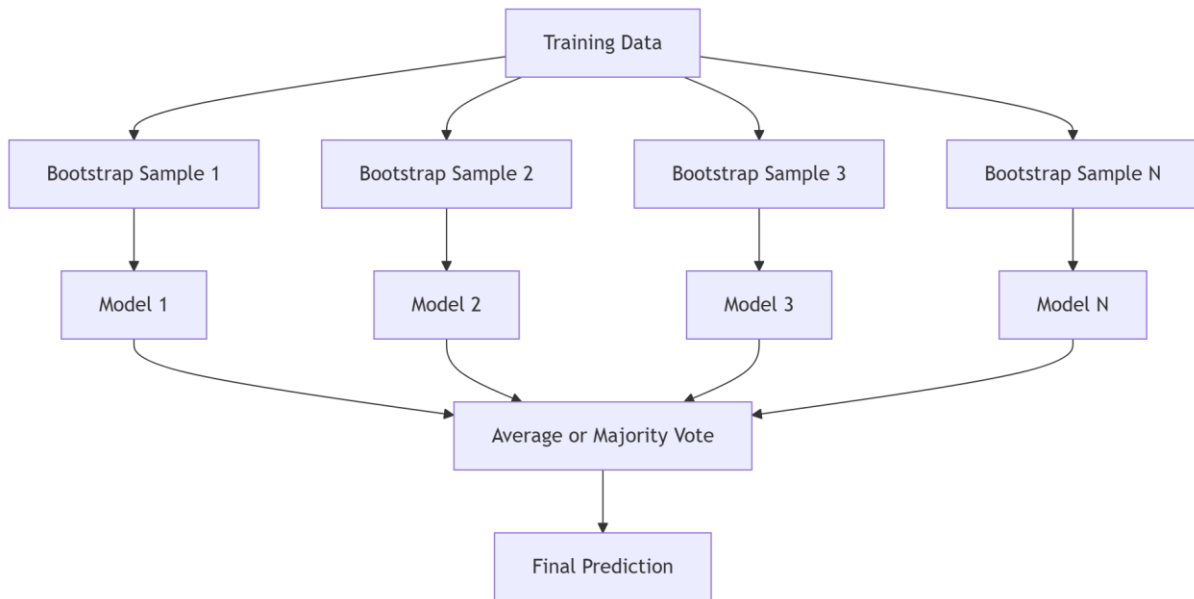
The key requirement is **diversity**. If all models make the same errors, combining them helps nothing. Ensembles work because they produce diverse models through:

79. Different training subsets (bagging)

- 80. Different feature subsets (random forests)
- 81. Sequential error correction (boosting)
- 82. Different model families (stacking)

Bagging (Bootstrap Aggregating)

Bagging creates diversity by training each model on a different bootstrap sample of the training data.



A bootstrap sample is drawn with replacement from the original data, same size as the original. About 63.2% of unique samples appear in each bootstrap. The remaining 36.8% (out-of-bag samples) provide a free validation set.

Bagging reduces variance without increasing bias much. Each individual tree overfits to its bootstrap sample, but the overfitting is different for each tree, so averaging cancels out the noise.

Random Forests are bagging with an extra twist: at each split, only a random subset of features is considered. This forces even more diversity among trees. The typical number of candidate features is $\sqrt{n_features}$ for classification and $n_features / 3$ for regression.

Boosting (Sequential Error Correction)

Boosting trains models sequentially. Each new model focuses on the examples that previous models got wrong.



Boosting reduces bias. Each new model corrects the systematic errors of the ensemble so far. The final prediction is a weighted sum of all models, where better models get higher weights.

The tradeoff: boosting can overfit if you run too many rounds, because it keeps fitting harder examples, some of which may be noise.

AdaBoost

AdaBoost (Adaptive Boosting) was the first practical boosting algorithm. It works with any base learner, typically decision stumps (depth-1 trees).

The algorithm:

1. Initialize sample weights: $w_i = 1/N$ for all i
2. For $t = 1$ to T :
 - a. Train weak learner h_t on weighted data
 - b. Compute weighted error:
 $err_t = \sum(w_i * I(h_t(x_i) \neq y_i)) / \sum(w_i)$
 - c. Compute model weight:
 $alpha_t = 0.5 * \ln((1 - err_t) / err_t)$
 - d. Update sample weights:
 $w_i = w_i * \exp(-alpha_t * y_i * h_t(x_i))$
 - e. Normalize weights to sum to 1
3. Final prediction: $H(x) = \text{sign}(\sum(alpha_t * h_t(x)))$

Models with lower error get higher α . Misclassified samples get higher weights so the next model focuses on them.

Gradient Boosting

Gradient boosting generalizes boosting to arbitrary loss functions. Instead of reweighting samples, it fits each new model to the residuals (negative gradient of the loss) of the current ensemble.

1. Initialize: $F_0(x) = \text{argmin}_c \sum(L(y_i, c))$
2. For $t = 1$ to T :
 - a. Compute pseudo-residuals:
 $r_i = -dL(y_i, F_{t-1}(x_i)) / dF_{t-1}(x_i)$
 - b. Fit a tree h_t to the residuals r_i
 - c. Find optimal step size:
 $gamma_t = \text{argmin}_{gamma} \sum(L(y_i, F_{t-1}(x_i) + gamma * h_t(x_i)))$
 - d. Update:
 $F_t(x) = F_{t-1}(x) + \text{learning_rate} * gamma_t * h_t(x)$
3. Final prediction: $F_T(x)$

For squared error loss, the pseudo-residuals are just the actual residuals: $r_i = y_i - F_{t-1}(x_i)$. Each tree literally fits the errors of the previous ensemble.

The learning rate (shrinkage) controls how much each tree contributes. Smaller learning rates require more trees but generalize better. Typical values: 0.01 to 0.3.

XGBoost: Why It Dominates Tabular Data

XGBoost (eXtreme Gradient Boosting) is gradient boosting with engineering optimizations that make it fast, accurate, and resistant to overfitting:

Regularized objective: L1 and L2 penalties on leaf weights prevent individual trees from being too confident

Second-order approximation: Uses both first and second derivatives of the loss, giving better split decisions

Sparsity-aware splits: Handles missing values natively by learning the best direction for missing data at each split

Column subsampling: Like random forests, samples features at each split for diversity

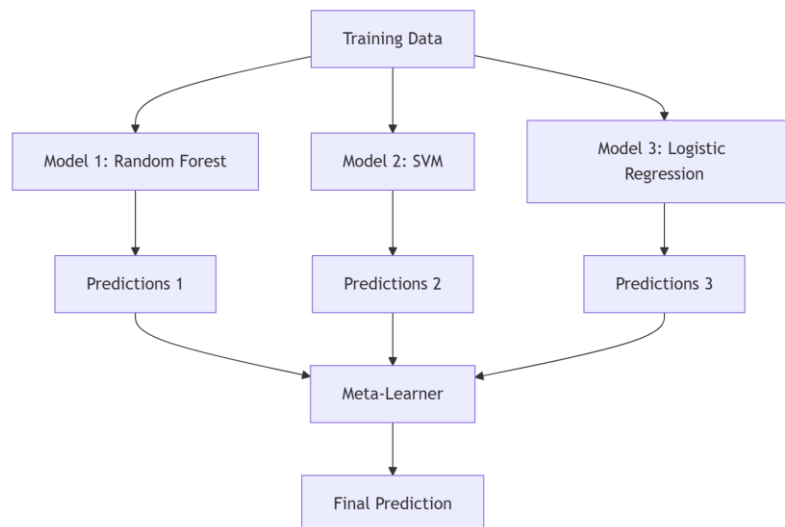
Weighted quantile sketch: Efficiently finds split points for continuous features on distributed data

Cache-aware block structure: Memory layout optimized for CPU cache lines

For tabular data, XGBoost (and its successor LightGBM) consistently outperforms neural networks. This is not changing anytime soon. If your data fits in a table with rows and columns, start with gradient boosting.

Stacking (Meta-Learning)

Stacking uses the predictions of multiple base models as features for a meta-learner.



The meta-learner learns which base model to trust for which inputs. If the random forest is better at certain regions and the SVM at others, the meta-learner will learn to route accordingly.

To avoid data leakage, base model predictions must be generated via cross-validation on the training set. You never train base models and generate meta-features on the same data.

Voting

The simplest ensemble. Just combine predictions directly.

Hard voting: Majority vote on class labels.

Soft voting: Average predicted probabilities, pick the class with highest average probability. Usually better because it uses confidence information.

Build It

Step 1: Decision Stump (Base Learner)

The code in `code/ensembles.py` implements everything from scratch. We start with a decision stump: a tree with a single split.

```
class DecisionStump:
    def __init__(self):
        self.feature_idx = None
        self.threshold = None
        self.polarity = 1
        self.alpha = None

    def fit(self, X, y, weights):
        n_samples, n_features = X.shape
        best_error = float("inf")

        for f in range(n_features):
            thresholds = np.unique(X[:, f])
            for thresh in thresholds:
                for polarity in [1, -1]:
                    pred = np.ones(n_samples)
                    pred[polarity * X[:, f] < polarity * thresh] = -1
                    error = np.sum(weights[pred != y])
                    if error < best_error:
                        best_error = error
                        self.feature_idx = f
                        self.threshold = thresh
                        self.polarity = polarity

    def predict(self, X):
        n = X.shape[0]
        pred = np.ones(n)
        idx = self.polarity * X[:, self.feature_idx] < self.polarity * self.threshold
```

```

pred[idx] = -1
return pred

```

Step 2: AdaBoost from Scratch

```

class AdaBoostScratch:
    def __init__(self, n_estimators=50):
        self.n_estimators = n_estimators
        self.stumps = []
        self.alphas = []

    def fit(self, X, y):
        n = X.shape[0]
        weights = np.full(n, 1 / n)

        for _ in range(self.n_estimators):
            stump = DecisionStump()
            stump.fit(X, y, weights)
            pred = stump.predict(X)

            err = np.sum(weights[pred != y])
            err = np.clip(err, 1e-10, 1 - 1e-10)

            alpha = 0.5 * np.log((1 - err) / err)
            weights *= np.exp(-alpha * y * pred)
            weights /= weights.sum()

            stump.alpha = alpha
            self.stumps.append(stump)
            self.alphas.append(alpha)

    def predict(self, X):
        total = sum(a * s.predict(X) for a, s in zip(self.alphas, self.stumps))
        return np.sign(total)

```

Step 3: Gradient Boosting from Scratch

```

class GradientBoostingScratch:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3):
        self.n_estimators = n_estimators
        self.lr = learning_rate
        self.max_depth = max_depth
        self.trees = []
        self.initial_pred = None

    def fit(self, X, y):
        self.initial_pred = np.mean(y)
        current_pred = np.full(len(y), self.initial_pred)

        for _ in range(self.n_estimators):
            residuals = y - current_pred
            tree = SimpleRegressionTree(max_depth=self.max_depth)

```

```

        tree.fit(X, residuals)
        update = tree.predict(X)
        current_pred += self.lr * update
        self.trees.append(tree)

    def predict(self, X):
        pred = np.full(X.shape[0], self.initial_pred)
        for tree in self.trees:
            pred += self.lr * tree.predict(X)
        return pred

```

Step 4: Compare against sklearn

The code verifies that our from-scratch implementations produce similar accuracy to sklearn's `AdaBoostClassifier` and `GradientBoostingClassifier`, and compares all methods side by side.

Use It

When to Use Each Method

Method	Reduces	Best for	Watch out for
Bagging / Random Forest	Variance	Noisy data, many features	Does not help with bias
AdaBoost	Bias	Clean data, simple base learners	Sensitive to outliers and noise
Gradient Boosting	Bias	Tabular data, competitions	Slow to train, easy to overfit without tuning
XGBoost / LightGBM	Both	Production tabular ML	Many hyperparameters
Stacking	Both	Getting last 1-2% accuracy	Complex, risk of overfitting meta-learner
Voting	Variance	Quick combination of diverse models	Only helps if models are diverse

The Production Stack for Tabular Data

For most tabular prediction problems, this is the order to try:

26. **LightGBM or XGBoost** with default parameters
27. Tune `n_estimators`, `learning_rate`, `max_depth`, `min_child_weight`
28. If you need the last 0.5%, build a stacking ensemble with 3-5 diverse models

29. Use cross-validation throughout

Neural networks on tabular data are almost always worse than gradient boosting, despite continued research attempts. TabNet, NODE, and similar architectures occasionally match but rarely beat a well-tuned XGBoost.

Exercises

Modify the AdaBoost implementation to track training accuracy after each round. Plot accuracy vs. number of estimators. When does it converge?

Implement a random forest from scratch by adding random feature subsampling to the regression tree. Train 100 trees with `max_features=sqrt(n_features)` and average predictions. Compare variance reduction to a single tree.

In the gradient boosting implementation, add early stopping: track validation loss after each round and stop when it has not improved for 10 consecutive rounds. How many trees does it actually need?

Build a stacking ensemble with three base models (logistic regression, decision tree, k-nearest neighbors) and a logistic regression meta-learner. Use 5-fold cross-validation to generate meta-features. Compare to each base model alone.

Run XGBoost on the same dataset with default parameters. Compare its accuracy to your from-scratch gradient boosting. Time both. How large is the speed difference?

Key Terms

Term	What people say	What it actually means
Bagging	“Train on random subsets”	Bootstrap aggregating: train models on bootstrap samples, average predictions to reduce variance
Boosting	“Focus on hard examples”	Train models sequentially, each correcting errors of the ensemble so far, to reduce bias
AdaBoost	“Reweight the data”	Boosting via sample weight updates; misclassified points get higher weight for the next learner
Gradient boosting	“Fit the residuals”	Boosting via fitting each new model to the negative gradient of the loss function
XGBoost	“The Kaggle weapon”	Gradient boosting with regularization, second-order optimization, and systems-level speed tricks
Stacking	“Models on top of models”	Use predictions of base models as input features for a meta-learner

Term	What people say	What it actually means
Random forest	"Many randomized trees"	Bagging with decision trees, adding random feature subsampling at each split for diversity
Ensemble diversity	"Make different mistakes"	Models must be uncorrelated in their errors for the ensemble to improve over individuals
Out-of-bag error	"Free validation"	Samples not in a bootstrap draw (~36.8%) serve as a validation set without needing a holdout

Code Appendix: *ensembLes.py*

```

import numpy as np
from collections import Counter

def make_classification_data(n_samples=300, n_features=5, noise=0.1, seed=42):
    rng = np.random.RandomState(seed)
    X = rng.randn(n_samples, n_features)
    boundary = 0.5 * X[:, 0] + 0.3 * X[:, 1] ** 2 - 0.2 * X[:, 2]
    y = np.where(boundary + rng.normal(0, noise, n_samples) > 0, 1, -1)
    return X, y

def make_regression_data(n_samples=300, n_features=5, noise=0.3, seed=42):
    rng = np.random.RandomState(seed)
    X = rng.randn(n_samples, n_features)
    y = 2.0 * X[:, 0] + np.sin(3 * X[:, 1]) - 0.5 * X[:, 2] ** 2 + rng.normal(0, noise,
n_samples)
    return X, y

def train_test_split(X, y, test_ratio=0.2, seed=42):
    rng = np.random.RandomState(seed)
    idx = rng.permutation(len(y))
    split = int(len(y) * (1 - test_ratio))
    return X[idx[:split]], X[idx[split:]], y[idx[:split]], y[idx[split:]]

class DecisionStump:
    def __init__(self):
        self.feature_idx = None
        self.threshold = None
        self.polarity = 1
        self.alpha = None

    def fit(self, X, y, weights):
        n_samples, n_features = X.shape
        best_error = float("inf")

```

```

for f in range(n_features):
    thresholds = np.unique(X[:, f])
    for thresh in thresholds:
        for polarity in [1, -1]:
            pred = np.ones(n_samples)
            pred[polarity * X[:, f] < polarity * thresh] = -1
            error = np.sum(weights[pred != y])
            if error < best_error:
                best_error = error
                self.feature_idx = f
                self.threshold = thresh
                self.polarity = polarity

def predict(self, X):
    n = X.shape[0]
    pred = np.ones(n)
    idx = self.polarity * X[:, self.feature_idx] < self.polarity * self.threshold
    pred[idx] = -1
    return pred

class AdaBoostScratch:
    def __init__(self, n_estimators=50):
        self.n_estimators = n_estimators
        self.stumps = []
        self.alphas = []

    def fit(self, X, y):
        n = X.shape[0]
        weights = np.full(n, 1 / n)

        for t in range(self.n_estimators):
            stump = DecisionStump()
            stump.fit(X, y, weights)
            pred = stump.predict(X)

            err = np.sum(weights[pred != y])
            err = np.clip(err, 1e-10, 1 - 1e-10)

            alpha = 0.5 * np.log((1 - err) / err)
            weights *= np.exp(-alpha * y * pred)
            weights /= weights.sum()

            stump.alpha = alpha
            self.stumps.append(stump)
            self.alphas.append(alpha)

    def predict(self, X):
        total = sum(a * s.predict(X) for a, s in zip(self.alphas, self.stumps))
        return np.sign(total)

    def accuracy(self, X, y):

```



```

        return np.mean(self.predict(X) == y)

class TreeNode:
    def __init__(self, value=None):
        self.feature_idx = None
        self.threshold = None
        self.left = None
        self.right = None
        self.value = value

class SimpleRegressionTree:
    def __init__(self, max_depth=3, min_samples_split=2):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.root = None

    def fit(self, X, y):
        self.root = self._build(X, y, depth=0)

    def _build(self, X, y, depth):
        n_samples, n_features = X.shape

        if depth >= self.max_depth or n_samples < self.min_samples_split:
            return TreeNode(value=np.mean(y))

        best_gain = -float("inf")
        best_feature = None
        best_threshold = None

        current_var = np.var(y) * n_samples

        for f in range(n_features):
            thresholds = np.unique(X[:, f])
            if len(thresholds) > 20:
                thresholds = np.percentile(X[:, f], np.linspace(0, 100, 20))

            for thresh in thresholds:
                left_mask = X[:, f] <= thresh
                right_mask = ~left_mask

                if left_mask.sum() < 1 or right_mask.sum() < 1:
                    continue

                left_var = np.var(y[left_mask]) * left_mask.sum()
                right_var = np.var(y[right_mask]) * right_mask.sum()
                gain = current_var - left_var - right_var

                if gain > best_gain:
                    best_gain = gain
                    best_feature = f
                    best_threshold = thresh

```

```

    if best_feature is None or best_gain <= 0:
        return TreeNode(value=np.mean(y))

    left_mask = X[:, best_feature] <= best_threshold
    node = TreeNode()
    node.feature_idx = best_feature
    node.threshold = best_threshold
    node.left = self._build(X[left_mask], y[left_mask], depth + 1)
    node.right = self._build(X[~left_mask], y[~left_mask], depth + 1)
    return node

def predict(self, X):
    return np.array([self._predict_one(x, self.root) for x in X])

def _predict_one(self, x, node):
    if node.value is not None:
        return node.value
    if x[node.feature_idx] <= node.threshold:
        return self._predict_one(x, node.left)
    return self._predict_one(x, node.right)

class GradientBoostingScratch:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3):
        self.n_estimators = n_estimators
        self.lr = learning_rate
        self.max_depth = max_depth
        self.trees = []
        self.initial_pred = None

    def fit(self, X, y):
        self.initial_pred = np.mean(y)
        current_pred = np.full(len(y), self.initial_pred)

        for _ in range(self.n_estimators):
            residuals = y - current_pred
            tree = SimpleRegressionTree(max_depth=self.max_depth)
            tree.fit(X, residuals)
            update = tree.predict(X)
            current_pred += self.lr * update
            self.trees.append(tree)

    def predict(self, X):
        pred = np.full(X.shape[0], self.initial_pred)
        for tree in self.trees:
            pred += self.lr * tree.predict(X)
        return pred

    def mse(self, X, y):
        return np.mean((self.predict(X) - y) ** 2)

```

```

class BaggingClassifier:
    def __init__(self, n_estimators=20, max_depth=5):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.trees = []

    def fit(self, X, y):
        rng = np.random.RandomState(42)
        n = len(y)

        for _ in range(self.n_estimators):
            idx = rng.choice(n, size=n, replace=True)
            tree = SimpleRegressionTree(max_depth=self.max_depth)
            tree.fit(X[idx], y[idx])
            self.trees.append(tree)

    def predict(self, X):
        predictions = np.array([tree.predict(X) for tree in self.trees])
        return np.sign(np.mean(predictions, axis=0))

    def accuracy(self, X, y):
        return np.mean(self.predict(X) == y)

class StackingClassifier:
    def __init__(self, base_models, meta_lr=0.1, n_folds=5):
        self.base_models = base_models
        self.meta_lr = meta_lr
        self.n_folds = n_folds
        self.meta_weights = None
        self.meta_bias = None
        self.fitted_models = []

    def fit(self, X, y):
        n = len(y)
        meta_features = np.zeros((n, len(self.base_models)))

        fold_size = n // self.n_folds
        indices = np.arange(n)

        for fold in range(self.n_folds):
            val_start = fold * fold_size
            val_end = val_start + fold_size if fold < self.n_folds - 1 else n
            val_idx = indices[val_start:val_end]
            train_idx = np.concatenate([indices[:val_start], indices[val_end:]])

            for m_idx, model_class in enumerate(self.base_models):
                model = model_class()
                model.fit(X[train_idx], y[train_idx])
                meta_features[val_idx, m_idx] = model.predict(X[val_idx])

        self.meta_weights = np.zeros(len(self.base_models))
        self.meta_bias = 0.0

```

```

        for _ in range(200):
            logits = meta_features @ self.meta_weights + self.meta_bias
            preds = np.tanh(logits)
            errors = y - preds
            grad_w = -2 * meta_features.T @ errors / n
            grad_b = -2 * np.sum(errors) / n
            self.meta_weights -= self.meta_lr * grad_w
            self.meta_bias -= self.meta_lr * grad_b

        self.fitted_models = []
        for model_class in self.base_models:
            model = model_class()
            model.fit(X, y)
            self.fitted_models.append(model)

    def predict(self, X):
        meta_features = np.column_stack([m.predict(X) for m in self.fitted_models])
        logits = meta_features @ self.meta_weights + self.meta_bias
        return np.sign(logits)

    def accuracy(self, X, y):
        return np.mean(self.predict(X) == y)

def demo_adaboost():
    print("=" * 60)
    print("ADABOOST FROM SCRATCH")
    print("=" * 60)

    X, y = make_classification_data(n_samples=400, n_features=5)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    for n_est in [1, 5, 10, 25, 50]:
        model = AdaBoostScratch(n_estimators=n_est)
        model.fit(X_train, y_train)
        train_acc = model.accuracy(X_train, y_train)
        test_acc = model.accuracy(X_test, y_test)
        print(f"  n_estimators={n_est:>3d}  train_acc={train_acc:.3f}  test_acc={tes
t_acc:.3f}")

    print()

    stump = DecisionStump()
    stump.fit(X_train, y_train, np.full(len(y_train), 1 / len(y_train)))
    stump_acc = np.mean(stump.predict(X_test) == y_test)
    print(f"  Single stump accuracy: {stump_acc:.3f}")
    print(f"  AdaBoost 50 accuracy: {model.accuracy(X_test, y_test):.3f}")
    print(f"  Improvement: {model.accuracy(X_test, y_test) - stump_acc:.3f}")
    print()

```

```

def demo_gradient_boosting():
    print("=" * 60)
    print("GRADIENT BOOSTING FROM SCRATCH")
    print("=" * 60)

    X, y = make_regression_data(n_samples=400, n_features=5)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    for n_est in [1, 10, 50, 100, 200]:
        model = GradientBoostingScratch(n_estimators=n_est, learning_rate=0.1)
        model.fit(X_train, y_train)
        train_mse = model.mse(X_train, y_train)
        test_mse = model.mse(X_test, y_test)
        print(f"  n_estimators={n_est:>3d}  train_mse={train_mse:.4f}  test_mse={tes
t_mse:.4f}")

    print()

    single_tree = SimpleRegressionTree(max_depth=3)
    single_tree.fit(X_train, y_train)
    tree_mse = np.mean((single_tree.predict(X_test) - y_test) ** 2)
    print(f"  Single tree MSE: {tree_mse:.4f}")
    print(f"  GBM 100 MSE:      {model.mse(X_test, y_test):.4f}")
    print()

def demo_learning_rate_effect():
    print("=" * 60)
    print("LEARNING RATE vs NUMBER OF TREES")
    print("=" * 60)

    X, y = make_regression_data(n_samples=400)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    configs = [
        (0.5, 20),
        (0.1, 100),
        (0.05, 200),
        (0.01, 500),
    ]

    for lr, n_est in configs:
        model = GradientBoostingScratch(n_estimators=n_est, learning_rate=lr)
        model.fit(X_train, y_train)
        test_mse = model.mse(X_test, y_test)
        print(f"  lr={lr:.2f}, n_trees={n_est:>3d}  test_mse={test_mse:.4f}")

    print()
    print("Lower learning rates need more trees but often generalize better.")
    print()

```

```

def demo_bagging():
    print("=" * 60)
    print("BAGGING CLASSIFIER")
    print("=" * 60)

    X, y = make_classification_data(n_samples=400)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    single_tree = SimpleRegressionTree(max_depth=5)
    single_tree.fit(X_train, y_train)
    single_acc = np.mean(np.sign(single_tree.predict(X_test)) == y_test)

    bagging = BaggingClassifier(n_estimators=20, max_depth=5)
    bagging.fit(X_train, y_train)
    bag_acc = bagging.accuracy(X_test, y_test)

    print(f" Single tree accuracy: {single_acc:.3f}")
    print(f" Bagging (20 trees):    {bag_acc:.3f}")
    print(f" Variance reduction:      {bag_acc - single_acc:+.3f}")
    print()

def demo_stacking():
    print("=" * 60)
    print("STACKING ENSEMBLE")
    print("=" * 60)

    X, y = make_classification_data(n_samples=400)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    def make_tree_d3():
        return SimpleRegressionTree(max_depth=3)

    def make_tree_d5():
        return SimpleRegressionTree(max_depth=5)

    def make_tree_d7():
        return SimpleRegressionTree(max_depth=7)

    make_tree_d3.fit = None
    make_tree_d5.fit = None
    make_tree_d7.fit = None

    class TreeWrapper:
        def __init__(self, max_depth):
            self.max_depth = max_depth
            self.tree = None

        def fit(self, X, y):
            self.tree = SimpleRegressionTree(max_depth=self.max_depth)
            self.tree.fit(X, y)

```

```

    def predict(self, X):
        return np.sign(self.tree.predict(X))

base_models = [
    lambda: TreeWrapper(3),
    lambda: TreeWrapper(5),
    lambda: TreeWrapper(7),
]

stack = StackingClassifier(base_models=base_models, meta_lr=0.05)
stack.fit(X_train, y_train)

for depth, model_fn in zip([3, 5, 7], base_models):
    m = model_fn()
    m.fit(X_train, y_train)
    acc = np.mean(m.predict(X_test) == y_test)
    print(f" Tree depth={depth} accuracy: {acc:.3f}")

stack_acc = stack.accuracy(X_test, y_test)
print(f" Stacking accuracy: {stack_acc:.3f}")
print(f" Meta-learner weights: {stack.meta_weights}")
print()

def demo_comparison():
    print("=" * 60)
    print("FULL COMPARISON")
    print("=" * 60)

    X, y = make_classification_data(n_samples=500)
    X_train, X_test, y_train, y_test = train_test_split(X, y)

    single = SimpleRegressionTree(max_depth=5)
    single.fit(X_train, y_train)
    print(f" Single tree (d=5): {np.mean(np.sign(single.predict(X_test)) == y_test) :.3f}")

    bag = BaggingClassifier(n_estimators=20, max_depth=5)
    bag.fit(X_train, y_train)
    print(f" Bagging (20, d=5): {bag.accuracy(X_test, y_test):.3f}")

    ada = AdaBoostScratch(n_estimators=50)
    ada.fit(X_train, y_train)
    print(f" AdaBoost (50 stumps): {ada.accuracy(X_test, y_test):.3f}")

    print()
    print("Bagging reduces variance (better than single tree).")
    print("Boosting reduces bias (learns complex boundaries from weak learners).")
    print()

```

```

def demo_sklearn_comparison():
    print("=" * 60)
    print("SKLEARN COMPARISON")
    print("=" * 60)

    try:
        from sklearn.ensemble import (
            AdaBoostClassifier,
            GradientBoostingClassifier,
            RandomForestClassifier,
        )
        from sklearn.metrics import accuracy_score
    except ImportError:
        print(" sklearn not installed, skipping comparison.")
        print()
        return

    X, y = make_classification_data(n_samples=500)
    y_01 = (y + 1) // 2
    X_train, X_test, y_train, y_test = train_test_split(X, y)
    X_train_01, X_test_01, y_train_01, y_test_01 = train_test_split(X, y_01)

    ada_ours = AdaBoostScratch(n_estimators=50)
    ada_ours.fit(X_train, y_train)
    print(f" Our AdaBoost:      {ada_ours.accuracy(X_test, y_test):.3f}")

    ada_sk = AdaBoostClassifier(n_estimators=50, random_state=42, algorithm="SAMME")
    ada_sk.fit(X_train_01, y_train_01)
    print(f" sklearn AdaBoost: {accuracy_score(y_test_01, ada_sk.predict(X_test_01))
:.3f}")

    rf = RandomForestClassifier(n_estimators=100, random_state=42)
    rf.fit(X_train_01, y_train_01)
    print(f" sklearn RF:      {accuracy_score(y_test_01, rf.predict(X_test_01)):.3f
}")

    gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
    gb.fit(X_train_01, y_train_01)
    print(f" sklearn GBM:      {accuracy_score(y_test_01, gb.predict(X_test_01)):.3f
}")

    print()

if __name__ == "__main__":
    demo_adaboost()
    demo_gradient_boosting()
    demo_learning_rate_effect()
    demo_bagging()
    demo_stacking()
    demo_comparison()

```



```
demo_sklearn_comparison()
print("All ensemble demos complete.")
```

You are an ensemble method selector. Given a description of a dataset and a prediction problem, you recommend the best ensemble approach with specific configuration advice.

When a user describes their data and problem, work through each section below.

Step 1: Understand the data

Ask about and summarize: - Number of rows (under 1k, 1k-100k, over 100k) - Number of features and their types (numeric, categorical, mixed) - Class balance (for classification) or target distribution (for regression) - Noise level: is the data clean or noisy with outliers? - Whether there are missing values

Step 2: Identify the core issue

Determine the primary modeling challenge: - High variance (model overfits, large gap between train and test scores): bagging territory - High bias (model underfits, both train and test scores are low): boosting territory - Need maximum accuracy with compute to spare: stacking territory - Quick baseline needed with minimal tuning risk: Random Forest

Step 3: Recommend a method

Based on the data profile and core issue, recommend one primary method and one alternative:

Small data (under 1k rows): Random Forest. Boosting methods overfit easily on small data. Random Forest is nearly impossible to misconfigure.

Medium data (1k-100k rows), clean: XGBoost or LightGBM. Start with `learning_rate=0.1` and use early stopping on a validation set. These give the best accuracy-to-effort ratio.

Medium data, noisy with outliers: Random Forest. Bagging is robust to noise because outliers affect individual trees differently and averaging cancels out their influence.

Large data (100k+ rows): LightGBM. Its histogram-based splits and leaf-wise growth make it the fastest gradient boosting implementation. XGBoost works too but is slower at this scale.

Many categorical features: CatBoost. It handles categoricals natively without one-hot encoding, which avoids the curse of dimensionality from high-cardinality features.

Need the last 1-2% accuracy: Stacking with 3-5 diverse base models (e.g., Random Forest + XGBoost + logistic regression + SVM). Always generate base model predictions via cross-validation.

Quick combination of existing models: Soft voting. Average predicted probabilities from 2-3 already-trained models. No meta-learner needed.

Step 4: Suggest starting hyperparameters

For the recommended method, provide specific starting values:

Random Forest: - n_estimators: 200 - max_depth: None (let trees grow fully) - max_features: “v” for classification, n_features/3 for regression - min_samples_leaf: 1-5

XGBoost / LightGBM: - learning_rate: 0.1 - n_estimators: 1000 with early_stopping_rounds=50 - max_depth: 6 - subsample: 0.8 - colsample_bytree: 0.8

Stacking: - Base models: at least 3, from different families - Meta-learner: logistic regression (classification) or ridge regression (regression) - Use 5-fold cross-validation for generating meta-features

Step 5: Warn about pitfalls

Flag the most common mistakes for the recommended method: - Gradient boosting without early stopping will overfit - Random Forest will not fix underfitting (it reduces variance, not bias) - Stacking with similar base models provides no diversity benefit - AdaBoost on noisy data amplifies outliers each round - Setting learning_rate above 0.3 in gradient boosting causes instability

Output format

Structure your response as: 1. **Data profile:** size, types, noise, balance 2. **Core issue:** variance, bias, or both 3. **Recommended method:** primary choice and why 4. **Alternative:** backup option if the primary does not work 5. **Starting config:** specific hyperparameters to try first 6. **Pitfalls:** what to watch out for with this method 7. **Next step:** the single most important thing to do first

Ensemble Method Selection Guide

Ensembles combine multiple models to produce better predictions than any single model. The question is always: which kind of ensemble, and when?

Decision Checklist

What is the main problem with your current model?

- High variance (overfitting): use bagging (Random Forest)
- High bias (underfitting): use boosting (Gradient Boosting, XGBoost)
- Both, or you want maximum accuracy: use stacking

How much data do you have?

1. Under 1,000 rows: Random Forest (robust, hard to misconfigure)
2. 1,000 to 100,000: XGBoost or LightGBM (best overall for tabular)
3. Over 100,000: LightGBM (fastest gradient boosting, handles large data well)

How much tuning time can you invest?

- Minimal: Random Forest with defaults (almost always works)
- Moderate: XGBoost with `learning_rate=0.1`, tune `n_estimators` with early stopping
- Maximum: LightGBM or XGBoost with Bayesian hyperparameter search

Do you need interpretability?

4. Yes: single decision tree or small Random Forest with feature importance
5. Partial: gradient boosting with SHAP values
6. No: stacking or deep ensembles

Is the data noisy with many outliers?

7. Yes: Random Forest (bagging is robust to noise)
8. No: gradient boosting (can push accuracy further on clean data)

When to use each method

Random Forest (Bagging): your safe first choice. Trains many trees on bootstrap samples and averages. Reduces variance without increasing bias. Nearly impossible to overfit on moderate data. Minimal tuning needed: set `n_estimators=100-500` and leave defaults.

AdaBoost: sequential boosting with sample reweighting. Works well with simple base learners (decision stumps). Sensitive to outliers and noisy labels because it upweights misclassified points. Largely replaced by gradient boosting in practice.

Gradient Boosting: fits each new tree to the residuals of the ensemble so far. Reduces bias. The most powerful method for tabular data. Requires tuning: `learning_rate`, `n_estimators`, `max_depth`, `min_child_weight`, `subsample`.

XGBoost: gradient boosting with regularization, second-order optimization, and systems-level speedups. Handles missing values natively. The default for Kaggle competitions and production ML on tabular data.

LightGBM: gradient boosting with leaf-wise growth (instead of level-wise). Faster than XGBoost on large datasets. Uses histogram-based splits. Best for datasets over 50k rows.

CatBoost: gradient boosting with native categorical feature handling. No need to one-hot encode. Good when you have many categorical features.

Stacking: trains a meta-learner on the predictions of multiple diverse base models. Use when you need the absolute best accuracy and have compute to spare. Always generate base model predictions via cross-validation to avoid leakage.

Voting: simplest ensemble. Hard voting (majority class) or soft voting (average probabilities). Quick way to combine 2-3 diverse models without a meta-learner.

Common mistakes

Using gradient boosting without early stopping (it will overfit if you let it run too many rounds)

Setting learning_rate too high (above 0.3 usually causes instability)

Not tuning max_depth for gradient boosting (default of unlimited or very deep trees overfit)

Stacking with models that are all the same type (diversity is the point of stacking)

Using AdaBoost on noisy data (outliers get higher and higher weight each round)

Expecting Random Forest to fix underfitting (it reduces variance, not bias)

Tuning priorities by method

Random Forest: 1. n_estimators: 100-500 (more is rarely worse, just slower) 2. max_depth: None (let trees grow fully) or cap at 10-20 for speed 3. max_features: "v" for classification, "log2" or n/3 for regression

XGBoost / LightGBM: 1. learning_rate: 0.01-0.3 (lower is better if you have compute for more trees) 2. n_estimators: use early stopping on a validation set instead of guessing 3. max_depth: 3-8 (start with 6) 4. min_child_weight / min_data_in_leaf: 1-20 (higher prevents overfitting) 5. subsample: 0.7-1.0 6. colsample_bytree: 0.7-1.0 7. reg_alpha (L1) and reg_lambda (L2): 0-10

Quick reference

Method	Reduces	Speed	Tuning effort	Best for
Random Forest	Variance	Fast	Low	Noisy data, quick baseline
AdaBoost	Bias	Fast	Low	Simple base learners, clean data
Gradient Boosting	Bias	Medium	High	Tabular data, competitions
XGBoost	Both	Fast	High	Production tabular ML
LightGBM	Both	Fastest	High	Large datasets (50k+ rows)
CatBoost	Both	Medium	Medium	Many categorical features
Stacking	Both	Slow	High	Maximum accuracy, diverse models
Voting	Variance	Fast	None	Quick combination of 2-3 models

Quiz

Pre-lesson check

Why does combining multiple weak classifiers into an ensemble improve accuracy?

- a) Each weak classifier memorizes a different part of the test set
- b) If the classifiers make different errors, majority voting cancels out individual mistakes **(correct)**
- c) Ensembles always use more training data than single models
- d) Weak classifiers are always faster than strong classifiers
Explanation: The key is diversity. If classifiers make independent errors, majority voting means a wrong answer must fool more than half the models. Errors cancel out, and the ensemble accuracy exceeds any individual.

What is the main difference between bagging and boosting?

- a) Bagging trains models in parallel on random subsets; boosting trains models sequentially, focusing on previous errors **(correct)**
- b) Bagging uses deep neural networks; boosting uses decision trees
- c) Bagging reduces bias; boosting reduces variance
- d) Bagging requires labeled data; boosting works unsupervised
Explanation: Bagging trains models independently on bootstrap samples (parallel, reduces variance). Boosting trains models sequentially, with each new model focusing on the mistakes of the ensemble so far (reduces bias).

Post-lesson check

In AdaBoost, what happens to the sample weight of a misclassified training point after each round?

- a) It stays the same
- b) It increases, so the next weak learner focuses more on this hard example **(correct)**
- c) It decreases, so the next learner ignores it

- d) It is removed from the training set

Explanation: AdaBoost increases the weights of misclassified samples after each round. This forces the next weak learner to pay more attention to the examples the ensemble currently gets wrong.

A random forest with 100 trees has the same test accuracy as 200 trees. Adding more trees to 500 also shows no improvement. Why?

- The random forest is underfitting and needs a different algorithm
- After enough trees, variance reduction plateaus and adding more trees provides diminishing returns without increasing overfitting **(correct)**
- 500 trees is the maximum allowed
- The trees are all identical so adding more has no effect
Explanation: Random forests do not overfit with more trees (unlike boosting). However, variance reduction plateaus once enough diverse trees have been averaged. More trees just add compute cost without improving accuracy.

Gradient boosting fits each new tree to what quantity?

- a) The original target values
 - b) The residuals (errors) of the current ensemble's predictions **(correct)**
 - c) Random subsets of features
 - d) The predictions of the previous tree
Explanation: In gradient boosting, each new tree is trained to predict the residuals (negative gradient of the loss) of the current ensemble. This sequentially reduces the remaining error.
-

Hyperparameter Tuning

Hyperparameters are the knobs you turn before training starts. Turning them well is the difference between a mediocre model and a great one.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 11 (Ensemble Methods) **Time:** ~90 minutes

Learning Objectives

83. Implement grid search, random search, and Bayesian optimization from scratch and compare their sample efficiency
84. Explain why random search outperforms grid search when most hyperparameters have low effective dimensionality
85. Build a Bayesian optimization loop using a surrogate model and acquisition function to guide the search
86. Design a hyperparameter tuning strategy that avoids overfitting the validation set through proper cross-validation

The Problem

Your gradient boosting model has a learning rate, number of trees, max depth, min samples per leaf, subsample ratio, and column sample ratio. That is six hyperparameters. If each has 5 reasonable values, the grid has $5^6 = 15,625$ combinations. Training each takes 10 seconds. That is 43 hours of compute to try them all.

Grid search is the obvious approach and the worst one at scale. Random search does better with less compute. Bayesian optimization does even better by learning from past evaluations. Knowing which strategy to use, and which hyperparameters actually matter, saves days of wasted GPU time.

The Concept

Parameters vs Hyperparameters

Parameters are learned during training (weights, biases, split thresholds). Hyperparameters are set before training starts and control how learning happens.

Hyperparameter	What it controls	Typical range
Learning rate	Step size per update	0.001 to 1.0
Number of trees/epochs	How long to train	10 to 10,000
Max depth	Model complexity	1 to 30

Hyperparameter	What it controls	Typical range
Regularization (λ)	Overfitting prevention	0.0001 to 100
Batch size	Gradient estimation noise	16 to 512
Dropout rate	Fraction of neurons dropped	0.0 to 0.5

Grid Search

Grid search evaluates every combination of specified values. It is exhaustive and easy to understand, but scales exponentially with the number of hyperparameters.

Grid for 2 hyperparameters:

```
learning_rate: [0.01, 0.1, 1.0]
max_depth:     [3, 5, 7]
```

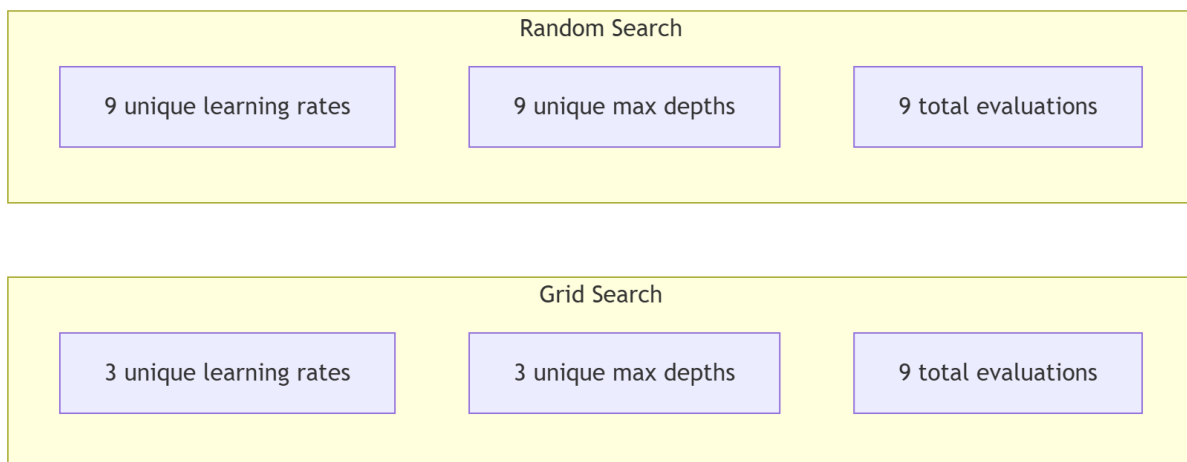
Evaluations: $3 \times 3 = 9$ combinations

```
(0.01, 3) (0.01, 5) (0.01, 7)
(0.1, 3)  (0.1, 5)  (0.1, 7)
(1.0, 3)  (1.0, 5)  (1.0, 7)
```

Grid search has a fundamental flaw: if one hyperparameter matters and the other does not, most evaluations are wasted. You get only 3 unique values of the important parameter from 9 evaluations.

Random Search

Random search samples hyperparameters from distributions instead of a grid. With the same budget of 9 evaluations, you get 9 unique values of each hyperparameter.

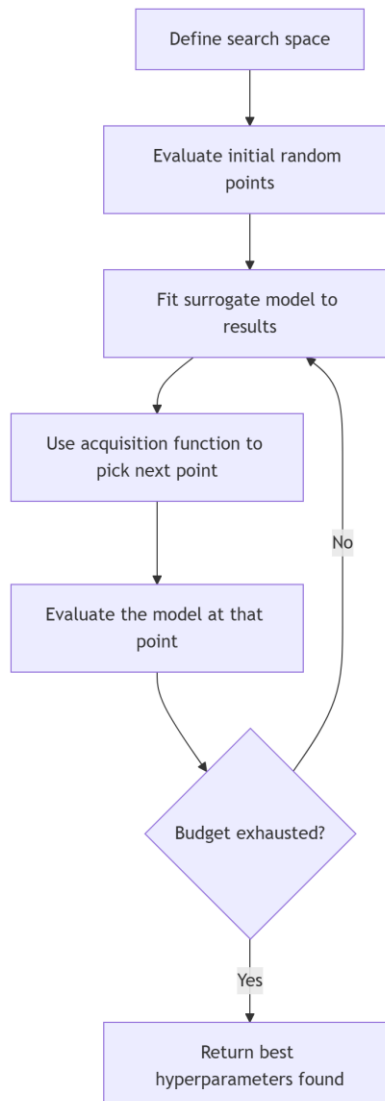


Why random beats grid (Bergstra & Bengio, 2012):

- 87. Most hyperparameters have low effective dimensionality. Only 1-2 of 6 hyperparameters usually matter for a given problem.
- 88. Grid search wastes evaluations on unimportant dimensions.
- 89. Random search covers the important dimensions more densely for the same budget.
- 90. At 60 random trials, you have a 95% chance of finding a point within 5% of the optimum (if one exists in the search space).

Bayesian Optimization

Random search ignores results. It does not learn that high learning rates cause divergence or that depth 3 consistently outperforms depth 10. Bayesian optimization uses past evaluations to decide where to search next.



The two key components:

Surrogate model: A cheap-to-evaluate model (usually a Gaussian process) that approximates the expensive objective function. It gives both a prediction and an uncertainty estimate at any point in the search space.

Acquisition function: Decides where to evaluate next by balancing exploitation (search near known good points) and exploration (search where uncertainty is high). Common choices:

91. **Expected Improvement (EI):** How much improvement over the current best do we expect at this point?
92. **Upper Confidence Bound (UCB):** Prediction plus a multiple of uncertainty. Higher UCB means either promising or unexplored.
93. **Probability of Improvement (PI):** What is the probability this point beats the current best?

Bayesian optimization typically finds better hyperparameters than random search with 2-5x fewer evaluations. The overhead of fitting the surrogate model is negligible compared to training the actual model.

Early Stopping

Not every training run needs to finish. If a configuration is clearly bad after 10 epochs, stop it and move on. This is early stopping in the context of hyperparameter search.

Strategies: - **Patience-based:** Stop if validation loss has not improved for N consecutive epochs - **Median pruning:** Stop if the trial's intermediate result is worse than the median of completed trials at the same step - **Hyperband:** Allocate small budgets to many configurations, then progressively increase budget for the best ones

Hyperband is particularly effective. It starts 81 configurations with 1 epoch each, keeps the top third, gives them 3 epochs, keeps the top third, and so on. This finds good configurations 10-50x faster than evaluating all configs for the full budget.

Learning Rate Schedulers

The learning rate is almost always the most important hyperparameter. Rather than keeping it fixed, schedulers adjust it during training.

Scheduler	Formula	When to use
Step decay	Multiply by 0.1 every N epochs	Classic CNN training
Cosine annealing	$lr * 0.5 * (1 + \cos(\pi * t / T))$	Modern default
Warmup + decay	Linear increase then cosine decay	Transformers

Scheduler	Formula	When to use
One-cycle	Increase then decrease over one cycle	Fast convergence
Reduce on plateau	Reduce by factor when metric stalls	Safe default

Hyperparameter Importance

Not all hyperparameters matter equally. Research on random forests (Probst et al., 2019) and gradient boosting shows consistent patterns:

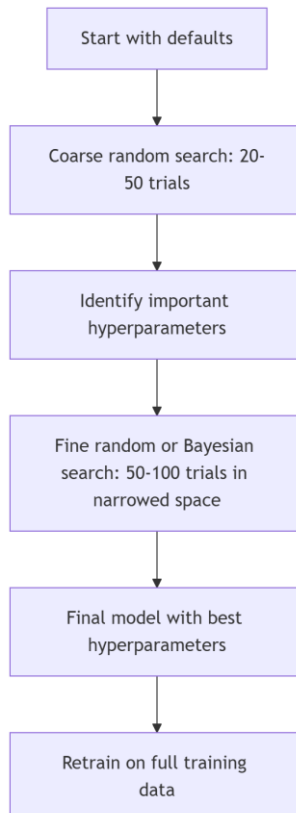
High importance: - Learning rate (always tune first) - Number of estimators / epochs (use early stopping instead of tuning) - Regularization strength

Medium importance: - Max depth / number of layers - Min samples per leaf / weight decay - Subsample ratio

Low importance: - Max features (for random forests) - Specific activation function choice - Batch size (within reasonable range)

Tune the important ones first, leave the rest at defaults.

Practical Strategy



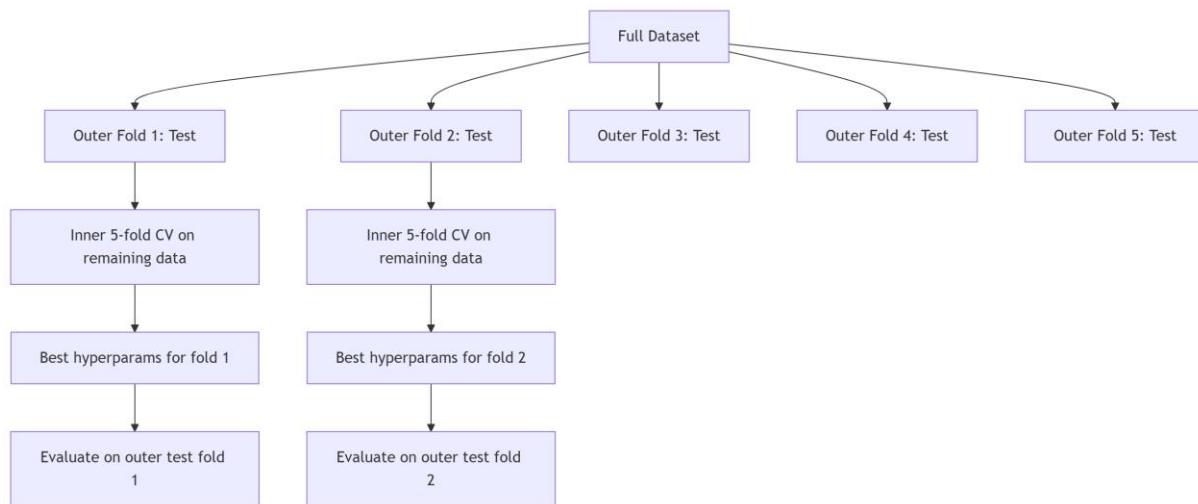
The concrete workflow:

- **Start with library defaults.** They are chosen by experienced practitioners and are often 80% of the way there.
- **Coarse random search.** Wide ranges, 20-50 trials. Use early stopping to kill bad runs fast.
- **Analyze results.** Which hyperparameters correlate with performance? Narrow the search space.
- **Fine search.** Bayesian optimization or focused random search in the narrowed space. 50-100 trials.
- **Retrain on all training data** with the best hyperparameters found.

Cross-Validation Integration

Tuning hyperparameters on a single validation split is risky. The best hyperparameters might overfit to the specific validation fold. Nested cross-validation solves this by using two loops:

94. **Outer loop** (evaluation): splits data into train+val and test. Reports unbiased performance.
95. **Inner loop** (tuning): splits train+val into train and val. Finds best hyperparameters.



Each outer fold finds its own best hyperparameters independently. The outer scores are an unbiased estimate of generalization performance.

With sklearn:

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor

inner_cv = GridSearchCV(
    GradientBoostingRegressor(),
    param_grid={
        "learning_rate": [0.01, 0.05, 0.1],
        "max_depth": [2, 3, 5],
```

```

        "n_estimators": [50, 100, 200],
    },
    cv=5,
    scoring="neg_mean_squared_error",
)

outer_scores = cross_val_score(
    inner_cv, X, y, cv=5, scoring="neg_mean_squared_error"
)

print(f"Nested CV MSE: {-outer_scores.mean():.4f} +/- {outer_scores.std():.4f}")

```

This is expensive (5 outer folds x 5 inner folds x 27 grid points = 675 model fits), but it gives you a trustworthy performance estimate. Use it when reporting final results in papers or when the stake of the decision is high.

Practical Tips

Start with the learning rate. It is always the most important hyperparameter for gradient-based methods. A bad learning rate makes everything else irrelevant. Fix other hyperparameters at defaults and sweep learning rate first.

Use log-uniform distributions for learning rate and regularization. The difference between 0.001 and 0.01 matters as much as the difference between 0.1 and 1.0. Searching linearly wastes budget on the large end.

Use early stopping instead of tuning n_estimators. For boosting and neural networks, set n_estimators or epochs high and let early stopping decide when to stop. This removes one hyperparameter from the search.

Budget allocation. Spend 60% of your tuning budget on the top 2 most important hyperparameters. Spend the remaining 40% on everything else. The top 2 account for most of the performance variation.

Scale matters. Never search batch size on a log scale (16, 32, 64 are fine). Always search learning rate on a log scale. Match the search distribution to how the hyperparameter affects the model.

Model Type	Top Hyperparameters	Recommended Search	Budget
Random Forest	n_estimators, max_depth, min_samples_leaf	Random search, 50 trials	Low (fast training)
Gradient Boosting	learning_rate, n_estimators, max_depth	Bayesian, 100 trials + early stopping	Medium
Neural Network	learning_rate, weight_decay, batch_size	Bayesian or random, 100+ trials	High (slow training)

Model Type	Top Hyperparameters	Recommended Search	Budget
SVM	C, γ (RBF kernel)	Grid on log scale, 25-50 trials	Low (2 params)
Lasso/Ridge	α	1D search on log scale, 20 trials	Very low
XGBoost	learning_rate, max_depth, subsample, colsample	Bayesian, 100-200 trials + early stopping	Medium

When in doubt: random search with 2x the number of hyperparameters as trials (e.g., 6 hyperparameters = 12+ trials minimum). You will be surprised how often random search with 50 trials beats carefully designed grid search.

Build It

Step 1: Grid Search from Scratch

The code in `code/tuning.py` implements grid search, random search, and a simple Bayesian optimizer from scratch.

```
def grid_search(model_fn, param_grid, X_train, y_train, X_val, y_val):
    keys = list(param_grid.keys())
    values = list(param_grid.values())
    best_score = -float("inf")
    best_params = None
    n_evals = 0

    for combo in itertools.product(*values):
        params = dict(zip(keys, combo))
        model = model_fn(**params)
        model.fit(X_train, y_train)
        score = evaluate(model, X_val, y_val)
        n_evals += 1

        if score > best_score:
            best_score = score
            best_params = params

    return best_params, best_score, n_evals
```

Step 2: Random Search from Scratch

```
def random_search(model_fn, param_distributions, X_train, y_train,
                  X_val, y_val, n_iter=50, seed=42):
    rng = np.random.RandomState(seed)
    best_score = -float("inf")
    best_params = None
```

```

for _ in range(n_iter):
    params = {k: sample(v, rng) for k, v in param_distributions.items()}
    model = model_fn(**params)
    model.fit(X_train, y_train)
    score = evaluate(model, X_val, y_val)

    if score > best_score:
        best_score = score
        best_params = params

return best_params, best_score, n_iter

```

Step 3: Bayesian Optimization (Simplified)

The core idea: fit a Gaussian process to observed (hyperparameter, score) pairs, then use an acquisition function to decide where to look next.

```

class SimpleBayesianOptimizer:
    def __init__(self, search_space, n_initial=5):
        self.search_space = search_space
        self.n_initial = n_initial
        self.X_observed = []
        self.y_observed = []

    def _kernel(self, x1, x2, length_scale=1.0):
        dists = np.sum((x1[:, None, :] - x2[None, :, :]) ** 2, axis=2)
        return np.exp(-0.5 * dists / length_scale ** 2)

    def _fit_gp(self, X_new):
        X_obs = np.array(self.X_observed)
        y_obs = np.array(self.y_observed)
        y_mean = y_obs.mean()
        y_centered = y_obs - y_mean

        K = self._kernel(X_obs, X_obs) + 1e-4 * np.eye(len(X_obs))
        K_star = self._kernel(X_new, X_obs)

        L = np.linalg.cholesky(K)
        alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_centered))
        mu = K_star @ alpha + y_mean

        v = np.linalg.solve(L, K_star.T)
        var = 1.0 - np.sum(v ** 2, axis=0)
        var = np.maximum(var, 1e-6)

        return mu, var

    def _expected_improvement(self, mu, var, best_y):
        sigma = np.sqrt(var)
        z = (mu - best_y) / (sigma + 1e-10)
        ei = sigma * (z * norm_cdf(z) + norm_pdf(z))
        return ei

```

```

def suggest(self):
    if len(self.X_observed) < self.n_initial:
        return sample_random(self.search_space)

    candidates = [sample_random(self.search_space) for _ in range(500)]
    X_cand = np.array([to_vector(c) for c in candidates])
    mu, var = self._fit_gp(X_cand)
    ei = self._expected_improvement(mu, var, max(self.y_observed))
    return candidates[np.argmax(ei)]

def observe(self, params, score):
    self.X_observed.append(to_vector(params))
    self.y_observed.append(score)

```

The GP surrogate gives two things at each candidate point: a predicted score (μ) and an uncertainty (var). Expected Improvement balances these: it favors points where the model predicts high scores OR where uncertainty is high. Early on, most points have high uncertainty so the optimizer explores. Later, it focuses on the most promising region.

Step 4: Compare All Methods

Run all three methods on the same synthetic objective and compare. This comparison uses a simplified wrapper that calls each optimizer with a direct objective function (no model training), so the API differs from the model-based implementations above:

```

def synthetic_objective(params):
    lr = params["learning_rate"]
    depth = params["max_depth"]
    return -(np.log10(lr) + 2) ** 2 - (depth - 4) ** 2 + 10

param_grid = {
    "learning_rate": [0.001, 0.01, 0.1, 1.0],
    "max_depth": [2, 3, 4, 5, 6, 7, 8],
}

grid_best = None
grid_score = -float("inf")
grid_history = []
for combo in itertools.product(*param_grid.values()):
    params = dict(zip(param_grid.keys(), combo))
    score = synthetic_objective(params)
    grid_history.append((params, score))
    if score > grid_score:
        grid_score = score
        grid_best = params

param_dist = {
    "learning_rate": ("log_float", 0.001, 1.0),
    "max_depth": ("int", 2, 8),
}

```



```

rand_best = None
rand_score = -float("inf")
rand_history = []
rng = np.random.RandomState(42)
for _ in range(28):
    params = {k: sample(v, rng) for k, v in param_dist.items()}
    score = synthetic_objective(params)
    rand_history.append((params, score))
    if score > rand_score:
        rand_score = score
        rand_best = params

optimizer = SimpleBayesianOptimizer(param_dist, n_initial=5)
bayes_history = []
for _ in range(28):
    params = optimizer.suggest()
    score = synthetic_objective(params)
    optimizer.observe(params, score)
    bayes_history.append((params, score))
bayes_score = max(s for _, s in bayes_history)

print(f"{'Method':<20} {'Best Score':>12} {'Evaluations':>12}")
print("-" * 50)
print(f"{'Grid Search':<20} {grid_score:>12.4f} {len(grid_history):>12}")
print(f"{'Random Search':<20} {rand_score:>12.4f} {len(rand_history):>12}")
print(f"{'Bayesian Opt':<20} {bayes_score:>12.4f} {len(bayes_history):>12}")

```

With the same budget, Bayesian optimization usually finds the best score fastest because it does not waste evaluations in clearly bad regions. Random search covers more ground than grid search. Grid search only wins when you have very few hyperparameters and can afford to be exhaustive.

Use It

Optuna in Practice

Optuna is the recommended library for serious hyperparameter tuning. It supports pruning, distributed search, and visualization out of the box.

```

import optuna

def objective(trial):
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
    n_est = trial.suggest_int("n_estimators", 50, 500)
    max_depth = trial.suggest_int("max_depth", 2, 10)

    model = GradientBoostingRegressor(
        learning_rate=lr,
        n_estimators=n_est,
        max_depth=max_depth,

```

```

    )
    model.fit(X_train, y_train)
    return mean_squared_error(y_val, model.predict(X_val))

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=100)

print(f"Best params: {study.best_params}")
print(f"Best MSE: {study.best_value:.4f}")

```

Key Optuna features: - `suggest_float(..., log=True)` for parameters best searched on log scale (learning rate, regularization) - `suggest_int` for integer parameters - `suggest_categorical` for discrete choices - Built-in MedianPruner for early stopping of bad trials - `study.trials_dataframe()` for analysis

Optuna with Pruning

Pruning stops unpromising trials early, saving massive compute. Here is the pattern:

```

import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("lr", 1e-4, 0.5, log=True),
        "max_depth": trial.suggest_int("max_depth", 2, 10),
        "n_estimators": trial.suggest_int("n_estimators", 50, 500),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    }

    model = GradientBoostingRegressor(**params)
    scores = cross_val_score(model, X_train, y_train, cv=3,
                             scoring="neg_mean_squared_error")
    mean_score = -scores.mean()

    trial.report(mean_score, step=0)
    if trial.should_prune():
        raise optuna.TrialPruned()

    return mean_score

pruner = optuna.pruners.MedianPruner(n_startup_trials=10, n_warmup_steps=5)
study = optuna.create_study(direction="minimize", pruner=pruner)
study.optimize(objective, n_trials=200)

```

The MedianPruner stops a trial if its intermediate value is worse than the median of all completed trials at the same step. Pruning requires calling `trial.report()` to report intermediate metrics and `trial.should_prune()` to check whether the trial should be stopped. The `n_startup_trials=10`

ensures at least 10 trials complete fully before pruning kicks in. This typically saves 40-60% of total compute.

sklearn's Built-in Tuners

For quick experiments, sklearn provides GridSearchCV, RandomizedSearchCV, and HalvingRandomSearchCV:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

param_dist = {
    "learning_rate": loguniform(1e-4, 0.5),
    "max_depth": randint(2, 10),
    "n_estimators": randint(50, 500),
}

search = RandomizedSearchCV(
    GradientBoostingRegressor(),
    param_dist,
    n_iter=100,
    cv=5,
    scoring="neg_mean_squared_error",
    random_state=42,
    n_jobs=-1,
)
search.fit(X_train, y_train)
print(f"Best params: {search.best_params_}")
print(f"Best CV MSE: {-search.best_score_: .4f}")
```

Use loguniform from scipy for learning rate and regularization. Use randint for integer hyperparameters. The n_jobs=-1 flag parallelizes across all CPU cores.

Common Mistakes in Hyperparameter Tuning

Data leakage through preprocessing. If you fit a scaler on the full dataset before cross-validation, information from the validation fold leaks into training. Always put preprocessing inside a Pipeline so it is fit only on the training fold.

Overfitting to the validation set. Running thousands of trials effectively trains on the validation set. Use nested cross-validation for final performance estimates, or hold out a separate test set that you never touch during tuning.

Searching too narrow a range. If your best value is at the boundary of your search space, you have not searched widely enough. The optimal value might be outside your range. Always check if the best parameters are at the edges.

Ignoring interaction effects. Learning rate and number of estimators interact strongly in boosting. A low learning rate needs more estimators. Tuning them independently gives worse results than tuning them together.

Not using early stopping for iterative models. For gradient boosting and neural networks, set `n_estimators` or epochs to a high value and use early stopping. This is strictly better than tuning the number of iterations as a hyperparameter.

Exercises

Run grid search and random search with the same total budget (e.g., 50 evaluations). Compare the best scores found. Run the experiment 10 times with different seeds. How often does random search win?

Implement Hyperband from scratch. Start with 81 configurations, each trained for 1 epoch. Keep the top 1/3 at each round and triple their budget. Compare total compute (sum of all epochs across all configs) to running 81 configs for the full budget.

Add a learning rate scheduler (cosine annealing) to the gradient boosting implementation from Lesson 11. Does it help compared to a fixed learning rate?

Use Optuna to tune a `RandomForestClassifier` on a real dataset (e.g., sklearn's breast cancer dataset). Use `optuna.visualization.plot_param_importances(study)` to see which hyperparameters matter most. Does it match the importance ranking from this lesson?

Implement a simple acquisition function (Expected Improvement) and demonstrate exploration vs exploitation. Plot the surrogate model's mean and uncertainty, and show where EI chooses to evaluate next.

Key Terms

Term	What people say	What it actually means
Hyperparameter	"A setting you choose"	A value set before training that controls the learning process, not learned from data
Grid search	"Try every combination"	Exhaustive search over a specified parameter grid. Exponential cost.
Random search	"Just sample randomly"	Sample hyperparameters from distributions. Covers important dimensions better than grid search.
Bayesian optimization	"Smart search"	Uses a surrogate model of the objective to decide where to evaluate next, balancing exploration and exploitation
Surrogate model	"A cheap approximation"	A model (usually Gaussian process) that approximates the expensive objective function from observed evaluations

Term	What people say	What it actually means
Acquisition function	“Where to look next”	Scores candidate points by balancing expected improvement with uncertainty. EI and UCB are common choices.
Early stopping	“Stop wasting time”	Terminate training early when validation performance stops improving
Hyperband	“Tournament bracket for configs”	Adaptive resource allocation: start many configs with small budgets, keep the best and increase their budgets
Learning rate scheduler	“Change lr during training”	A function that adjusts the learning rate over the course of training for better convergence

Code Appendix: *tuning.py*

```

import numpy as np
import itertools
import time

def make_data(n_samples=400, n_features=8, seed=42):
    rng = np.random.RandomState(seed)
    X = rng.randn(n_samples, n_features)
    y = (
        2.0 * X[:, 0]
        + np.sin(3 * X[:, 1])
        - 0.5 * X[:, 2] ** 2
        + 0.3 * X[:, 3] * X[:, 4]
        + rng.normal(0, 0.3, n_samples)
    )
    split = int(n_samples * 0.6)
    val_split = int(n_samples * 0.8)
    return (
        X[:split], y[:split],
        X[split:val_split], y[split:val_split],
        X[val_split:], y[val_split:],
    )

class SimpleTree:
    def __init__(self, max_depth=3, min_samples_split=5):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.root = None

    def fit(self, X, y):
        self.root = self._build(X, y, 0)

    def _build(self, X, y, depth):

```

```

        if depth >= self.max_depth or len(y) < self.min_samples_split:
            return {"value": np.mean(y)}

        best_gain = -1
        best_f, best_t = 0, 0
        var_total = np.var(y) * len(y)

        n_features = X.shape[1]
        for f in range(n_features):
            thresholds = np.percentile(X[:, f], np.linspace(10, 90, 10))
            for t in thresholds:
                left = y[X[:, f] <= t]
                right = y[X[:, f] > t]
                if len(left) < 2 or len(right) < 2:
                    continue
                gain = var_total - np.var(left) * len(left) - np.var(right) * len(right)

                if gain > best_gain:
                    best_gain = gain
                    best_f, best_t = f, t

        if best_gain <= 0:
            return {"value": np.mean(y)}

        mask = X[:, best_f] <= best_t
        return {
            "feature": best_f,
            "threshold": best_t,
            "left": self._build(X[mask], y[mask], depth + 1),
            "right": self._build(X[~mask], y[~mask], depth + 1),
        }

    def predict(self, X):
        return np.array([self._predict_one(x, self.root) for x in X])

    def _predict_one(self, x, node):
        if "value" in node:
            return node["value"]
        if x[node["feature"]] <= node["threshold"]:
            return self._predict_one(x, node["left"])
        return self._predict_one(x, node["right"])

class GBMForTuning:
    def __init__(self, n_estimators=50, learning_rate=0.1, max_depth=3,
                  min_samples_split=5, subsample=1.0):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.subsample = subsample
        self.trees = []
        self.init_pred = None

```

```

def fit(self, X, y):
    rng = np.random.RandomState(42)
    self.init_pred = np.mean(y)
    pred = np.full(len(y), self.init_pred)

    for _ in range(self.n_estimators):
        residuals = y - pred

        if self.subsample < 1.0:
            n_sub = max(1, int(len(y) * self.subsample))
            idx = rng.choice(len(y), n_sub, replace=False)
            X_sub, r_sub = X[idx], residuals[idx]
        else:
            X_sub, r_sub = X, residuals

        tree = SimpleTree(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
        )
        tree.fit(X_sub, r_sub)
        pred += self.learning_rate * tree.predict(X)
        self.trees.append(tree)

def predict(self, X):
    pred = np.full(X.shape[0], self.init_pred)
    for tree in self.trees:
        pred += self.learning_rate * tree.predict(X)
    return pred

def neg_mse(model, X, y):
    pred = model.predict(X)
    return -np.mean((pred - y) ** 2)

def grid_search(param_grid, X_train, y_train, X_val, y_val):
    keys = list(param_grid.keys())
    values = list(param_grid.values())
    best_score = -float("inf")
    best_params = None
    history = []

    for combo in itertools.product(*values):
        params = dict(zip(keys, combo))
        model = GBMForTuning(**params)
        model.fit(X_train, y_train)
        score = neg_mse(model, X_val, y_val)
        history.append((params.copy(), score))

        if score > best_score:
            best_score = score
            best_params = params.copy()

```

```

    return best_params, best_score, history

def sample_param(spec, rng):
    if isinstance(spec, list):
        return rng.choice(spec)
    name, low, high = spec[0], spec[1], spec[2]
    if name == "int":
        return rng.randint(low, high + 1)
    if name == "float":
        return rng.uniform(low, high)
    if name == "log_float":
        return np.exp(rng.uniform(np.log(low), np.log(high)))
    return low

def random_search(param_distributions, X_train, y_train, X_val, y_val,
                  n_iter=50, seed=42):
    rng = np.random.RandomState(seed)
    best_score = -float("inf")
    best_params = None
    history = []

    for _ in range(n_iter):
        params = {k: sample_param(v, rng) for k, v in param_distributions.items()}
        int_params = {}
        for k, v in params.items():
            if k in ("n_estimators", "max_depth", "min_samples_split"):
                int_params[k] = int(v)
            else:
                int_params[k] = v

        model = GBMForTuning(**int_params)
        model.fit(X_train, y_train)
        score = neg_mse(model, X_val, y_val)
        history.append((int_params.copy(), score))

        if score > best_score:
            best_score = score
            best_params = int_params.copy()

    return best_params, best_score, history

class SimpleBayesianOptimizer:
    def __init__(self, param_space, n_initial=10, seed=42):
        self.param_space = param_space
        self.n_initial = n_initial
        self.rng = np.random.RandomState(seed)
        self.X_observed = []
        self.y_observed = []
        self.param_names = list(param_space.keys())

```



```

def _sample_random(self):
    return {k: sample_param(v, self.rng) for k, v in self.param_space.items()}

def _params_to_vec(self, params):
    vec = []
    for k in self.param_names:
        v = params[k]
        spec = self.param_space[k]
        if isinstance(spec, list):
            vec.append(spec.index(v) / max(1, len(spec) - 1))
        elif spec[0] == "log_float":
            vec.append(
                (np.log(v) - np.log(spec[1])) / (np.log(spec[2]) - np.log(spec[1]))
            )
        else:
            vec.append((v - spec[1]) / max(1e-10, spec[2] - spec[1]))
    return np.array(vec)

def _rbf_kernel(self, X1, X2, length_scale=0.3):
    dists = np.sum((X1[:, None, :] - X2[None, :, :]) ** 2, axis=2)
    return np.exp(-0.5 * dists / length_scale ** 2)

def _predict(self, X_new):
    if len(self.X_observed) == 0:
        return np.zeros(len(X_new)), np.ones(len(X_new))

    X_obs = np.array(self.X_observed)
    y_obs = np.array(self.y_observed)
    y_mean = y_obs.mean()
    y_centered = y_obs - y_mean

    K = self._rbf_kernel(X_obs, X_obs) + 1e-4 * np.eye(len(X_obs))
    K_star = self._rbf_kernel(X_new, X_obs)

    try:
        L = np.linalg.cholesky(K)
        alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_centered))
        mu = K_star @ alpha + y_mean
        v = np.linalg.solve(L, K_star.T)
        var = 1.0 - np.sum(v ** 2, axis=0)
        var = np.maximum(var, 1e-6)
    except np.linalg.LinAlgError:
        K_inv = np.linalg.pinv(K)
        mu = K_star @ K_inv @ y_centered + y_mean
        var = np.ones(len(X_new)) * 0.1

    return mu, var

def _expected_improvement(self, mu, var, best_y):
    sigma = np.sqrt(var)
    z = (mu - best_y) / (sigma + 1e-10)
    ei = sigma * (z * self._norm_cdf(z) + self._norm_pdf(z))

```

```

        return ei

    @staticmethod
    def _norm_cdf(x):
        return 0.5 * (1 + np.vectorize(lambda v: np.tanh(v * 0.7978845608))(x))

    @staticmethod
    def _norm_pdf(x):
        return np.exp(-0.5 * x ** 2) / np.sqrt(2 * np.pi)

    def suggest(self):
        if len(self.X_observed) < self.n_initial:
            return self._sample_random()

        n_candidates = 500
        candidates = [self._sample_random() for _ in range(n_candidates)]
        X_cand = np.array([self._params_to_vec(c) for c in candidates])

        mu, var = self._predict(X_cand)
        best_y = max(self.y_observed)
        ei = self._expected_improvement(mu, var, best_y)

        best_idx = np.argmax(ei)
        return candidates[best_idx]

    def observe(self, params, score):
        self.X_observed.append(self._params_to_vec(params))
        self.y_observed.append(score)

    def optimize(self, objective, n_iter=50):
        best_score = -float("inf")
        best_params = None
        history = []

        for i in range(n_iter):
            params = self.suggest()
            score = objective(params)
            self.observe(params, score)
            history.append((params.copy(), score))

            if score > best_score:
                best_score = score
                best_params = params.copy()

        return best_params, best_score, history

    def convergence_curve(history):
        best_so_far = -float("inf")
        curve = []
        for _, score in history:
            best_so_far = max(best_so_far, score)
            curve.append(best_so_far)

```

```

return curve

def demo_grid_search():
    print("=" * 60)
    print("GRID SEARCH")
    print("=" * 60)

    X_tr, y_tr, X_val, y_val, X_te, y_te = make_data()

    param_grid = {
        "n_estimators": [20, 50, 100],
        "learning_rate": [0.01, 0.05, 0.1, 0.3],
        "max_depth": [2, 3, 5],
    }

    start = time.time()
    best_params, best_score, history = grid_search(param_grid, X_tr, y_tr, X_val, y_val
    )
    elapsed = time.time() - start

    total_combos = 1
    for v in param_grid.values():
        total_combos *= len(v)

    print(f" Total combinations: {total_combos}")
    print(f" Best params: {best_params}")
    print(f" Best val neg_mse: {best_score:.4f} (MSE = {-best_score:.4f})")
    print(f" Time: {elapsed:.1f}s")

    model = GBMForTuning(**best_params)
    model.fit(X_tr, y_tr)
    test_mse = -neg_mse(model, X_te, y_te)
    print(f" Test MSE: {test_mse:.4f}")
    print()

def demo_random_search():
    print("=" * 60)
    print("RANDOM SEARCH")
    print("=" * 60)

    X_tr, y_tr, X_val, y_val, X_te, y_te = make_data()

    param_distributions = {
        "n_estimators": ("int", 10, 200),
        "learning_rate": ("log_float", 0.005, 0.5),
        "max_depth": ("int", 2, 8),
        "min_samples_split": ("int", 2, 20),
        "subsample": ("float", 0.5, 1.0),
    }

```

```

budgets = [10, 25, 50, 100]

for n_iter in budgets:
    start = time.time()
    best_params, best_score, history = random_search(
        param_distributions, X_tr, y_tr, X_val, y_val, n_iter=n_iter
    )
    elapsed = time.time() - start
    print(f" n_iter={n_iter:>3d} best_mse={-best_score:.4f} time={elapsed:.1
f}s")

print()

best_params, best_score, _ = random_search(
    param_distributions, X_tr, y_tr, X_val, y_val, n_iter=100
)
print(f" Best params (100 trials):")
for k, v in best_params.items():
    if isinstance(v, float):
        print(f" {k}: {v:.4f}")
    else:
        print(f" {k}: {v}")

model = GBMForTuning(**best_params)
model.fit(X_tr, y_tr)
test_mse = -neg_mse(model, X_te, y_te)
print(f" Test MSE: {test_mse:.4f}")
print()

def demo_bayesian():
    print("=" * 60)
    print("BAYESIAN OPTIMIZATION")
    print("=" * 60)

    X_tr, y_tr, X_val, y_val, X_te, y_te = make_data()

    param_space = {
        "n_estimators": ("int", 10, 200),
        "learning_rate": ("log_float", 0.005, 0.5),
        "max_depth": ("int", 2, 8),
        "min_samples_split": ("int", 2, 20),
        "subsample": ("float", 0.5, 1.0),
    }

    def objective(params):
        int_params = {}
        for k, v in params.items():
            if k in ("n_estimators", "max_depth", "min_samples_split"):
                int_params[k] = int(v)
            else:
                int_params[k] = v

```

```

        model = GBMForTuning(**int_params)
        model.fit(X_tr, y_tr)
        return neg_mse(model, X_val, y_val)

optimizer = SimpleBayesianOptimizer(param_space, n_initial=10, seed=42)
best_params, best_score, history = optimizer.optimize(objective, n_iter=50)

print(f" Best params (50 trials):")
for k, v in best_params.items():
    if isinstance(v, float):
        print(f"    {k}: {v:.4f}")
    else:
        print(f"    {k}: {v}")
print(f" Best val MSE: {-best_score:.4f}")

int_params = {}
for k, v in best_params.items():
    if k in ("n_estimators", "max_depth", "min_samples_split"):
        int_params[k] = int(v)
    else:
        int_params[k] = v
model = GBMForTuning(**int_params)
model.fit(X_tr, y_tr)
test_mse = -neg_mse(model, X_te, y_te)
print(f" Test MSE: {test_mse:.4f}")
print()

def demo_comparison():
    print("=" * 60)
    print("HEAD-TO-HEAD: GRID vs RANDOM vs BAYESIAN")
    print("=" * 60)

    X_tr, y_tr, X_val, y_val, X_te, y_te = make_data()

    param_grid = {
        "n_estimators": [20, 50, 100],
        "learning_rate": [0.01, 0.05, 0.1, 0.3],
        "max_depth": [2, 3, 5],
    }

    _, grid_score, grid_history = grid_search(param_grid, X_tr, y_tr, X_val, y_val)
    n_grid = len(grid_history)

    param_dist = {
        "n_estimators": ("int", 10, 200),
        "learning_rate": ("log_float", 0.005, 0.5),
        "max_depth": ("int", 2, 8),
    }

    _, rand_score, rand_history = random_search(
        param_dist, X_tr, y_tr, X_val, y_val, n_iter=n_grid

```

```

)

param_space = {
    "n_estimators": ("int", 10, 200),
    "learning_rate": ("log_float", 0.005, 0.5),
    "max_depth": ("int", 2, 8),
}

def objective(params):
    int_params = {k: int(v) if k in ("n_estimators", "max_depth") else v
                  for k, v in params.items()}
    model = GBMForTuning(**int_params)
    model.fit(X_tr, y_tr)
    return neg_mse(model, X_val, y_val)

optimizer = SimpleBayesianOptimizer(param_space, n_initial=10, seed=42)
_, bayes_score, bayes_history = optimizer.optimize(objective, n_iter=n_grid)

print(f" Budget: {n_grid} evaluations each")
print(f" Grid search best MSE: {-grid_score:.4f}")
print(f" Random search best MSE: {-rand_score:.4f}")
print(f" Bayesian opt best MSE: {-bayes_score:.4f}")
print()

grid_curve = convergence_curve(grid_history)
rand_curve = convergence_curve(rand_history)
bayes_curve = convergence_curve(bayes_history)

checkpoints = [5, 10, 20, n_grid - 1]
print(f" {'Eval':>6} {'Grid MSE':>10} {'Random MSE':>10} {'Bayes MSE':>10}
")
print(f" {'-'*6} {'-'*10} {'-'*10} {'-'*10}")
for cp in checkpoints:
    if cp < len(grid_curve):
        print(
            f" {cp+1:>6d} {-grid_curve[cp]:>10.4f} "
            f" {-rand_curve[cp]:>10.4f} {-bayes_curve[cp]:>10.4f}"
        )

print()
print("Random search explores the full continuous space (better coverage).")
print("Bayesian optimization learns from past results (better convergence).")
print("Grid search is only competitive with few hyperparameters.")
print()

def demo_optuna():
    print("=" * 60)
    print("OPTUNA (if installed)")
    print("=" * 60)

```

```

try:
    import optuna
    optuna.logging.set_verbosity(optuna.logging.WARNING)
except ImportError:
    print(" Optuna not installed. Install with: pip install optuna")
    print(" Skipping Optuna demo.")
    print()
    return

X_tr, y_tr, X_val, y_val, X_te, y_te = make_data()

def objective(trial):
    lr = trial.suggest_float("learning_rate", 0.005, 0.5, log=True)
    n_est = trial.suggest_int("n_estimators", 10, 200)
    max_depth = trial.suggest_int("max_depth", 2, 8)
    min_split = trial.suggest_int("min_samples_split", 2, 20)
    subsample = trial.suggest_float("subsample", 0.5, 1.0)

    model = GBMForTuning(
        n_estimators=n_est,
        learning_rate=lr,
        max_depth=max_depth,
        min_samples_split=min_split,
        subsample=subsample,
    )
    model.fit(X_tr, y_tr)
    return np.mean((model.predict(X_val) - y_val) ** 2)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=50)

print(f" Best params:")
for k, v in study.best_params.items():
    if isinstance(v, float):
        print(f"    {k}: {v:.4f}")
    else:
        print(f"    {k}: {v}")
print(f" Best val MSE: {study.best_value:.4f}")

best = study.best_params
model = GBMForTuning(
    n_estimators=best["n_estimators"],
    learning_rate=best["learning_rate"],
    max_depth=best["max_depth"],
    min_samples_split=best["min_samples_split"],
    subsample=best["subsample"],
)
model.fit(X_tr, y_tr)
test_mse = np.mean((model.predict(X_te) - y_te) ** 2)
print(f" Test MSE: {test_mse:.4f}")

try:

```

```

importances = optuna.importance.get_param_importances(study)
print(f"\n Hyperparameter importances:")
for k, v in importances.items():
    bar = "#" * int(v * 40)
    print(f"    {k:>20s}: {v:.3f} {bar}")
except Exception:
    pass

print()

if __name__ == "__main__":
    demo_grid_search()
    demo_random_search()
    demo_bayesian()
    demo_comparison()
    demo_optuna()
    print("All tuning demos complete.")

```

You are a hyperparameter tuning strategist. Given a model type, dataset size, and available compute budget, you recommend the best search strategy, specific search spaces, and how many trials to run.

When a user describes their setup, work through each step:

Step 1: Gather context

Ask for: - Model type (e.g., random forest, XGBoost, neural network, SVM) - Dataset size (rows and features) - Compute budget (how long can tuning run? minutes, hours, or days?) - Current performance (what is the baseline score?) - Metric being optimized (accuracy, F1, MSE, AUC-ROC, etc.)

Step 2: Choose a search strategy

Use this decision framework:

Grid search: - Use only when you have 1-2 hyperparameters and fewer than 50 total combinations - Appropriate for: final fine-tuning in a narrow range around a known good region - Never use for initial exploration with 3+ hyperparameters

Random search: - Use when you have 3+ hyperparameters and 20-100 trial budget - Better than grid because it covers important dimensions more densely - With 60 random trials, you have a 95% chance of landing within the top 5% of the search space - Appropriate for: most tuning tasks as the first pass

Bayesian optimization (Optuna, Hyperopt): - Use when each evaluation is expensive (more than 30 seconds per trial) - Learns from past trials to propose better candidates - Typically finds better results than random search with 2-5x fewer trials - Appropriate for: neural networks, gradient boosting with large data, any model where training is slow

Hyperband / ASHA: - Use when early stopping is meaningful (models that train iteratively) - Starts many configs with small budgets, keeps the best, increases their budget - 10-50x faster than running all configs to completion - Appropriate for: neural networks, gradient boosting, any iterative learner

Step 3: Define search spaces by model type

Random Forest:

```
n_estimators: [100, 200, 500] (or use early stopping via OOB score)
max_depth: [None, 10, 20, 30]
min_samples_split: [2, 5, 10]
min_samples_leaf: [1, 2, 4]
max_features: ["sqrt", "log2", 0.5]
```

Priority: $\text{max_depth} > \text{min_samples_leaf} > \text{max_features}$. n_estimators is rarely the bottleneck (more is generally better).

XGBoost / LightGBM:

```
learning_rate: log-uniform [0.005, 0.3]
n_estimators: use early stopping (set high, e.g., 2000, let it stop)
max_depth: uniform int [3, 10]
min_child_weight: uniform int [1, 20]
subsample: uniform [0.6, 1.0]
colsample_bytree: uniform [0.6, 1.0]
reg_alpha: log-uniform [1e-4, 10]
reg_lambda: log-uniform [1e-4, 10]
```

Priority: $\text{learning_rate} > \text{max_depth} > \text{min_child_weight} > \text{subsample}$.

SVM (RBF kernel):

```
C: log-uniform [0.01, 1000]
gamma: log-uniform [0.001, 10]
```

Always search on log scale. Only 2 parameters, so even grid search works ($7 \times 7 = 49$ combos).

Neural Network:

```
learning_rate: log-uniform [1e-5, 1e-2]
batch_size: [32, 64, 128, 256]
hidden_layers: [1, 2, 3]
hidden_units: [64, 128, 256, 512]
dropout: uniform [0.0, 0.5]
weight_decay: log-uniform [1e-6, 1e-2]
```

Priority: learning_rate > architecture > regularization. Use Hyperband with epoch budget.

Step 4: Recommend number of trials

Budget	Strategy	Trials
Under 10 minutes	Random search	10-20
10 min to 1 hour	Random search	30-60
1 to 8 hours	Bayesian (Optuna)	50-200
Over 8 hours	Bayesian + Hyperband	200-1000

Rule of thumb: with random search, $10 * (\text{number of hyperparameters})$ trials covers the space reasonably. With Bayesian optimization, $5 * (\text{number of hyperparameters})$ is often sufficient.

Step 5: Recommend the workflow

30. **Start with library defaults.** Train once. Record the baseline.
31. **Coarse search.** Wide ranges, 20-50 trials with random search. Use 3-fold CV for speed.
32. **Analyze.** Which hyperparameters correlated with good performance? Narrow ranges.
33. **Fine search.** Bayesian optimization in the narrowed space, 50-100 trials. Use 5-fold CV.
34. **Retrain.** Take the best hyperparameters, retrain on the full training set.
35. **Evaluate.** Test on the held-out test set exactly once. Report final metric.

Output format

Structure your response as: 1. **Search strategy:** [grid / random / Bayesian / Hyperband] 2. **Search space:** [table of hyperparameters with ranges and distributions] 3. **Number of trials:** [with justification] 4. **Cross-validation folds:** [3 or 5, with reasoning] 5. **Expected runtime:** [estimate based on per-trial time and number of trials] 6. **Early stopping:** [whether to use it and how]

Avoid: - Recommending grid search with more than 3 hyperparameters (exponential blowup) - Using uniform distributions for learning rates or regularization (always log-uniform) - Tuning `n_estimators` for gradient boosting (use early stopping instead) - Running more trials than necessary for simple models (random forest with defaults is already 90% of the way there) - Skipping cross-validation to save time (you will overfit to the validation set)

Quiz

Pre-lesson check

What is the difference between a parameter and a hyperparameter?

- a) Parameters are set by the user; hyperparameters are learned during training
- b) Parameters are learned during training (weights, biases); hyperparameters are set before training starts (learning rate, max depth) **(correct)**
- c) There is no difference; they are synonyms
- d) Parameters apply to neural networks only; hyperparameters apply to tree models
Explanation: Parameters are learned by the optimization algorithm during training (e.g., weights). Hyperparameters are set before training and control how learning happens (e.g., learning rate, regularization strength).

Grid search over 4 hyperparameters with 5 values each requires how many evaluations?

- a) 20
- b) 25
- c) 625 **(correct)**
- d) 4
Explanation: Grid search evaluates every combination: $5^4 = 625$. This exponential scaling is why grid search becomes impractical with many hyperparameters.

Post-lesson check

- Why does random search often outperform grid search with the same evaluation budget?
 1. Random search uses a better optimization algorithm
 2. Most hyperparameters have low effective dimensionality, so random search covers the important ones more densely **(correct)**
 3. Random search always finds the global optimum
 4. Grid search cannot handle continuous hyperparameters
Explanation: Usually only 1-2 hyperparameters matter for a given problem. Grid search

wastes evaluations varying unimportant ones. Random search gives unique values for every parameter per trial, covering important dimensions more densely.

- In Bayesian optimization, what does the acquisition function balance?
 - Training speed and model accuracy
 - Exploitation (searching near known good points) and exploration (searching uncertain regions) **(correct)**
 - The number of features and the number of samples
 - Bias and variance in the surrogate model
Explanation: The acquisition function decides where to evaluate next by balancing exploitation (near known good results) and exploration (where the surrogate model is uncertain). This directs search more efficiently than random.
 - You tune hyperparameters using the test set and report the best test performance. What is wrong with this approach?
 5. Nothing – this is standard practice
 6. You overfitted to the test set; the reported performance is optimistic and will not generalize **(correct)**
 7. The test set should be used for training, not tuning
 8. Hyperparameters should only be integers
Explanation: Tuning on the test set means you selected hyperparameters that happen to perform well on those specific samples. This is overfitting to the test set. Use a validation set for tuning and reserve the test set for final evaluation.
-

ML Pipelines

A model is not a product. A pipeline is. The pipeline is everything from raw data to deployed prediction, and every step must be reproducible.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 12 (Hyperparameter Tuning) **Time:** ~120 minutes

Learning Objectives

- Build an ML pipeline from scratch that chains imputation, scaling, encoding, and model training into a single reproducible object
- Identify data leakage scenarios and explain how pipelines prevent them by fitting transformers only on training data
- Construct a ColumnTransformer that applies different preprocessing to numeric and categorical features
- Implement pipeline serialization and demonstrate that the same fitted pipeline produces identical results in training and production

The Problem

You have a notebook that loads data, fills missing values with the median, scales features, trains a model, and prints accuracy. It works. You ship it.

A month later, someone retrains the model and gets different results. The median was computed on the full dataset including test data (data leakage). The scaling parameters were not saved, so inference uses different statistics. The feature engineering code was copy-pasted between training and serving, and the copies diverged. A categorical column gained a new value in production that the encoder has never seen.

These are not hypothetical. They are the most common reasons ML systems fail in production. Pipelines solve all of them by packaging every transformation step into a single, ordered, reproducible object.

The Concept

What a Pipeline Is

A pipeline is an ordered sequence of data transformations followed by a model. Each step takes the output of the previous step as input. The entire pipeline is fitted once on training data. At inference time, the same fitted pipeline transforms new data and produces predictions.



The pipeline guarantees: - Transformations are fitted only on training data (no leakage) - The same transformations are applied at inference time - The entire object can be serialized and deployed as one artifact - Cross-validation applies the pipeline per fold, preventing subtle leakage

Data Leakage: The Silent Killer

Data leakage happens when information from the test set or future data contaminates training. Pipelines prevent the most common forms.

Leaky (wrong):

```
X = df.drop("target", axis=1)
y = df["target"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test = X_scaled[:800], X_scaled[800:]
y_train, y_test = y[:800], y[800:]
```

The scaler saw test data. The mean and standard deviation include test samples. This inflates accuracy estimates.

Correct:

```
X_train, X_test = X[:800], X[800:]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

With a pipeline, you do not need to think about this. The pipeline handles it automatically.

sklearn Pipeline

sklearn's Pipeline chains transformers and an estimator. It exposes `.fit()`, `.predict()`, and `.score()` that apply all steps in order.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression()),
])

pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
```

When you call `pipe.fit(X_train, y_train)`: 1. Scaler calls `fit_transform` on `X_train` 2. Model calls `fit` on the scaled `X_train`

When you call `pipe.predict(X_test)`: 1. Scaler calls `transform` (not `fit_transform`) on `X_test` 2. Model calls `predict` on the scaled `X_test`

The scaler never sees test data during fitting. This is the whole point.

ColumnTransformer: Different Pipelines for Different Columns

Real datasets have numeric and categorical columns that need different preprocessing. `ColumnTransformer` handles this.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("encode", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, ["age", "income", "score"]),
    ("cat", categorical_pipe, ["city", "gender", "plan"]),
])

full_pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", GradientBoostingClassifier()),
])
```

The `handle_unknown="ignore"` in `OneHotEncoder` is critical for production. When a new category appears (a city the model has never seen), it produces a zero vector instead of crashing.

Experiment Tracking

A pipeline makes training reproducible, but you also need to track what happened across experiments: which hyperparameters were used, which dataset version, what the metrics were, which code was running.

MLflow is the most common open-source solution:

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("max_depth", 5)
    mlflow.log_param("n_estimators", 100)
```

```

mlflow.log_param("learning_rate", 0.1)

pipe.fit(X_train, y_train)
accuracy = pipe.score(X_test, y_test)

mlflow.log_metric("accuracy", accuracy)
mlflow.sklearn.log_model(pipe, "model")

```

Every run is recorded with parameters, metrics, artifacts, and the full model. You can compare runs, reproduce any experiment, and deploy any model version.

Weights & Biases (wandb) provides the same functionality with a hosted dashboard:

```

import wandb

wandb.init(project="my-pipeline")
wandb.config.update({"max_depth": 5, "n_estimators": 100})

pipe.fit(X_train, y_train)
accuracy = pipe.score(X_test, y_test)

wandb.log({"accuracy": accuracy})

```

Model Versioning

After experiment tracking, you need to manage model versions. Which model is in production? Which is staging? Which was last week's?

MLflow's Model Registry provides: - **Version tracking**: Every saved model gets a version number - **Stage transitions**: "Staging", "Production", "Archived" - **Approval workflow**: Models must be explicitly promoted to production - **Rollback**: Switch back to a previous version instantly

Data Versioning with DVC

Code is versioned with git. Data should be versioned too, but git cannot handle large files. DVC (Data Version Control) solves this.

```

dvc init
dvc add data/training.csv
git add data/training.csv.dvc data/.gitignore
git commit -m "Track training data"
dvc push

```

DVC stores the actual data in remote storage (S3, GCS, Azure) and keeps a small .dvc file in git that records the hash. When you checkout a git commit, `dvc checkout` restores the exact data that was used.

This means every git commit pins both the code and the data. Full reproducibility.

Reproducible Experiments

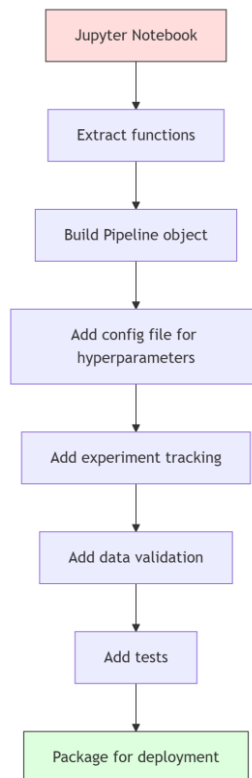
A reproducible experiment requires four things:

96. **Fixed random seeds:** Set seeds for numpy, random, and the framework (torch, sklearn)
97. **Pinned dependencies:** requirements.txt or poetry.lock with exact versions
98. **Versioned data:** DVC or similar
99. **Config files:** All hyperparameters in a config, not hardcoded

```
import numpy as np
import random

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
    except ImportError:
        pass
```

From Notebook to Production Pipeline



The typical progression:

Notebook exploration: Quick experiments, visualizations, feature ideas

Extract functions: Move preprocessing, feature engineering, evaluation into modules

Build Pipeline: Chain transformations into a sklearn Pipeline or custom class

Config management: Move all hyperparameters into a YAML/JSON config

Experiment tracking: Add MLflow or wandb logging

Data validation: Check schema, distributions, and missing value patterns before training

Tests: Unit tests for transformers, integration tests for the full pipeline

Deployment: Serialize the pipeline, wrap in an API (FastAPI, Flask), containerize

Common Pipeline Mistakes

Mistake	Why it is bad	Fix
Fitting on full data before splitting	Data leakage	Use Pipeline with cross_val_score
Feature engineering outside pipeline	Different transforms at train vs serve	Put all transforms in the Pipeline
Not handling unknown categories	Production crash on new values	OneHotEncoder(handle_unknown="ignore")
Hardcoded column names	Breaks when schema changes	Use column name lists from config
No data validation	Silently wrong predictions on bad data	Add schema checks before prediction
Training/serving skew	Model sees different features in prod	One Pipeline object for both

Build It

The code in `code/pipeline.py` builds a complete ML pipeline from scratch:

Step 1: Custom Transformer

```
class CustomTransformer:
    def __init__(self):
        self.means = None
        self.stds = None

    def fit(self, X):
```

```

self.means = np.mean(X, axis=0)
self.stds = np.std(X, axis=0)
self.stds[self.stds == 0] = 1.0
return self

def transform(self, X):
    return (X - self.means) / self.stds

def fit_transform(self, X):
    return self.fit(X).transform(X)

```

Step 2: Pipeline from Scratch

```

class PipelineFromScratch:
    def __init__(self, steps):
        self.steps = steps

    def fit(self, X, y=None):
        X_current = X.copy()
        for name, step in self.steps[::-1]:
            X_current = step.fit_transform(X_current)
        name, model = self.steps[-1]
        model.fit(X_current, y)
        return self

    def predict(self, X):
        X_current = X.copy()
        for name, step in self.steps[::-1]:
            X_current = step.transform(X_current)
        name, model = self.steps[-1]
        return model.predict(X_current)

```

Step 3: Cross-Validation with Pipeline

The code demonstrates how cross-validation with a pipeline prevents data leakage: the scaler is fit separately on each fold's training data.

Step 4: Full Production Pipeline with sklearn

A complete pipeline with `ColumnTransformer`, multiple preprocessing paths, and a model, trained with proper cross-validation and experiment logging.

Exercises

Build a pipeline that handles a dataset with 3 numeric columns and 2 categorical columns. Use `ColumnTransformer` to apply median imputation + scaling to numerics and most-frequent imputation + one-hot encoding to categoricals. Train with 5-fold cross-validation.

Deliberately introduce data leakage: fit the scaler on the full dataset before splitting. Compare the cross-validation score (leaky) to the pipeline cross-validation score (clean). How large is the difference?

Serialize your pipeline with `joblib.dump`. Load it in a separate script and run predictions. Verify the predictions are identical.

Add a custom transformer to the pipeline that creates polynomial features (degree 2) for the two most important numeric columns. Where should it go in the pipeline?

Set up MLflow tracking for the pipeline. Run 5 experiments with different hyperparameters. Use the MLflow UI (`mlflow ui`) to compare runs and pick the best model.

Key Terms

Term	What people say	What it actually means
Pipeline	“Chain of transforms + model”	An ordered sequence of fitted transformers and a model, applied as one unit to prevent leakage
Data leakage	“Test info leaked into training”	Using information from outside the training set to build the model, inflating performance estimates
ColumnTransformer	“Different preprocessing per column”	Applies different pipelines to different subsets of columns, combining results
Experiment tracking	“Logging your runs”	Recording parameters, metrics, artifacts, and code versions for every training run
MLflow	“Track and deploy models”	Open-source platform for experiment tracking, model registry, and deployment
DVC	“Git for data”	Version control system for large data files, storing hashes in git and data in remote storage
Model registry	“Model version catalog”	A system that tracks model versions with stage labels (staging, production, archived)
Training/serving skew	“It worked in the notebook”	Differences between how data is processed during training versus inference, causing silent errors
Reproducibility	“Same code, same result”	The ability to get identical results from the same code, data, and configuration

Code Appendix: `pipeline.py`

```
import numpy as np
import warnings
warnings.filterwarnings("ignore")
```

```

def make_mixed_data(n_samples=500, seed=42):
    rng = np.random.RandomState(seed)

    age = rng.normal(35, 12, n_samples).clip(18, 80)
    income = rng.lognormal(10.5, 0.8, n_samples)
    score = rng.uniform(300, 850, n_samples)

    cities = np.array(["new_york", "chicago", "la", "houston", "phoenix"])
    city = rng.choice(cities, n_samples)

    plans = np.array(["free", "basic", "premium"])
    plan = rng.choice(plans, n_samples, p=[0.5, 0.3, 0.2])

    mask = rng.random(n_samples) < 0.05
    age_with_missing = age.copy()
    age_with_missing[mask] = np.nan

    mask2 = rng.random(n_samples) < 0.03
    income_with_missing = income.copy()
    income_with_missing[mask2] = np.nan

    boundary = (
        0.01 * (age - 35)
        + 0.00001 * (income - 40000)
        + 0.002 * (score - 600)
        + 0.5 * (plan == "premium").astype(float)
        - 0.3 * (plan == "free").astype(float)
        + rng.normal(0, 0.5, n_samples)
    )
    target = (boundary > 0).astype(int)

    return {
        "age": age_with_missing,
        "income": income_with_missing,
        "score": score,
        "city": city,
        "plan": plan,
        "target": target,
    }

def train_test_split_dict(data, test_ratio=0.2, seed=42):
    rng = np.random.RandomState(seed)
    n = len(data["target"])
    idx = rng.permutation(n)
    split = int(n * (1 - test_ratio))
    train_idx, test_idx = idx[:split], idx[split:]

    train = {k: v[train_idx] for k, v in data.items()}
    test = {k: v[test_idx] for k, v in data.items()}
    return train, test

```

```

class MedianImputer:
    def __init__(self):
        self.medians = None

    def fit(self, X):
        self.medians = np.nanmedian(X, axis=0)
        return self

    def transform(self, X):
        X_out = X.copy()
        for col in range(X.shape[1]):
            mask = np.isnan(X_out[:, col])
            X_out[mask, col] = self.medians[col]
        return X_out

    def fit_transform(self, X):
        return self.fit(X).transform(X)

class StandardScaler:
    def __init__(self):
        self.means = None
        self.stds = None

    def fit(self, X):
        self.means = np.nanmean(X, axis=0)
        self.stds = np.nanstd(X, axis=0)
        self.stds[self.stds == 0] = 1.0
        return self

    def transform(self, X):
        return (X - self.means) / self.stds

    def fit_transform(self, X):
        return self.fit(X).transform(X)

class OneHotEncoder:
    def __init__(self, handle_unknown="ignore"):
        self.categories = None
        self.handle_unknown = handle_unknown

    def fit(self, X):
        self.categories = []
        for col in range(X.shape[1]):
            self.categories.append(sorted(set(X[:, col])))
        return self

    def transform(self, X):
        encoded_cols = []
        for col in range(X.shape[1]):
            cats = self.categories[col]
            for cat in cats:

```

```

        encoded_cols.append((X[:, col] == cat).astype(float))
    return np.column_stack(encoded_cols) if encoded_cols else np.zeros((X.shape[0],
0))

def fit_transform(self, X):
    return self.fit(X).transform(X)

class PipelineFromScratch:
    def __init__(self, steps):
        self.steps = steps

    def fit(self, X, y=None):
        X_current = X.copy() if isinstance(X, np.ndarray) else X
        for name, step in self.steps[:-1]:
            X_current = step.fit_transform(X_current)
        name, model = self.steps[-1]
        model.fit(X_current, y)
        return self

    def predict(self, X):
        X_current = X.copy() if isinstance(X, np.ndarray) else X
        for name, step in self.steps[:-1]:
            X_current = step.transform(X_current)
        name, model = self.steps[-1]
        return model.predict(X_current)

    def score(self, X, y):
        pred = self.predict(X)
        return np.mean(pred == y)

class ColumnTransformerScratch:
    def __init__(self, transformers):
        self.transformers = transformers

    def fit(self, data):
        for name, pipeline, columns in self.transformers:
            X_subset = np.column_stack([data[c] for c in columns])
            pipeline.fit(X_subset)
        return self

    def transform(self, data):
        outputs = []
        for name, pipeline, columns in self.transformers:
            X_subset = np.column_stack([data[c] for c in columns])
            outputs.append(pipeline.transform(X_subset))
        return np.hstack(outputs)

    def fit_transform(self, data):
        return self.fit(data).transform(data)

```

```

class TransformerPipeline:
    def __init__(self, steps):
        self.steps = steps

    def fit(self, X):
        X_current = X
        for name, step in self.steps:
            X_current = step.fit_transform(X_current)
        return self

    def transform(self, X):
        X_current = X
        for name, step in self.steps:
            X_current = step.transform(X_current)
        return X_current

    def fit_transform(self, X):
        return self.fit(X).transform(X)

class LogisticRegressionSimple:
    def __init__(self, lr=0.01, n_iter=1000):
        self.lr = lr
        self.n_iter = n_iter
        self.weights = None
        self.bias = None

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for _ in range(self.n_iter):
            z = X @ self.weights + self.bias
            pred = self._sigmoid(z)
            dw = (1 / n_samples) * X.T @ (pred - y)
            db = (1 / n_samples) * np.sum(pred - y)
            self.weights -= self.lr * dw
            self.bias -= self.lr * db

    def predict(self, X):
        z = X @ self.weights + self.bias
        return (self._sigmoid(z) >= 0.5).astype(int)

    def predict_proba(self, X):
        z = X @ self.weights + self.bias
        p = self._sigmoid(z)
        return np.column_stack([1 - p, p])

```



```

class DecisionTreeSimple:
    def __init__(self, max_depth=5):
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, y):
        self.root = self._build(X, y, 0)

    def _gini(self, y):
        if len(y) == 0:
            return 0
        p = np.mean(y)
        return 1 - p ** 2 - (1 - p) ** 2

    def _build(self, X, y, depth):
        if depth >= self.max_depth or len(np.unique(y)) == 1 or len(y) < 4:
            return {"value": round(np.mean(y))}

        best_gain = -1
        best_f, best_t = 0, 0
        gini_parent = self._gini(y) * len(y)

        for f in range(X.shape[1]):
            thresholds = np.percentile(X[:, f], np.linspace(10, 90, 10))
            for t in thresholds:
                left = y[X[:, f] <= t]
                right = y[X[:, f] > t]
                if len(left) < 2 or len(right) < 2:
                    continue
                gain = gini_parent - self._gini(left) * len(left) - self._gini(right) *
len(right)
                if gain > best_gain:
                    best_gain = gain
                    best_f, best_t = f, t

        if best_gain <= 0:
            return {"value": round(np.mean(y))}

        mask = X[:, best_f] <= best_t
        return {
            "feature": best_f,
            "threshold": best_t,
            "left": self._build(X[mask], y[mask], depth + 1),
            "right": self._build(X[~mask], y[~mask], depth + 1),
        }

    def predict(self, X):
        return np.array([self._pred(x, self.root) for x in X])

    def _pred(self, x, node):
        if "value" in node:
            return node["value"]
        if x[node["feature"]] <= node["threshold"]:

```

```

        return self._pred(x, node["left"])
    return self._pred(x, node["right"])

def cross_validate_pipeline(pipeline_factory, data, n_folds=5, seed=42):
    rng = np.random.RandomState(seed)
    n = len(data["target"])
    idx = rng.permutation(n)
    fold_size = n // n_folds
    scores = []

    for fold in range(n_folds):
        val_start = fold * fold_size
        val_end = val_start + fold_size if fold < n_folds - 1 else n

        val_idx = idx[val_start:val_end]
        train_idx = np.concatenate([idx[:val_start], idx[val_end:]])

        train_data = {k: v[train_idx] for k, v in data.items()}
        val_data = {k: v[val_idx] for k, v in data.items()}

        pipe = pipeline_factory()
        pipe.fit(train_data)
        score = pipe.score(val_data)
        scores.append(score)

    return scores

class FullPipeline:
    def __init__(self, model, numeric_cols, categorical_cols):
        self.model = model
        self.numeric_cols = numeric_cols
        self.categorical_cols = categorical_cols
        self.num_pipeline = TransformerPipeline([
            ("impute", MedianImputer()),
            ("scale", StandardScaler()),
        ])
        self.cat_encoder = OneHotEncoder(handle_unknown="ignore")

    def fit(self, data):
        X_num = np.column_stack([data[c] for c in self.numeric_cols])
        X_cat = np.column_stack([data[c] for c in self.categorical_cols])

        X_num_processed = self.num_pipeline.fit_transform(X_num)
        X_cat_processed = self.cat_encoder.fit_transform(X_cat)

        X = np.hstack([X_num_processed, X_cat_processed])
        y = data["target"]

        self.model.fit(X, y)
        return self

```

```

def predict(self, data):
    X_num = np.column_stack([data[c] for c in self.numeric_cols])
    X_cat = np.column_stack([data[c] for c in self.categorical_cols])

    X_num_processed = self.num_pipeline.transform(X_num)
    X_cat_processed = self.cat_encoder.transform(X_cat)

    X = np.hstack([X_num_processed, X_cat_processed])
    return self.model.predict(X)

def score(self, data):
    pred = self.predict(data)
    return np.mean(pred == data["target"])

def demo_data_leakage():
    print("=" * 60)
    print("DATA LEAKAGE DEMONSTRATION")
    print("=" * 60)

    rng = np.random.RandomState(42)
    X = rng.randn(200, 5)
    y = (X[:, 0] + 0.5 * X[:, 1] > 0).astype(int)

    scaler_leaky = StandardScaler()
    X_scaled_leaky = scaler_leaky.fit_transform(X)
    X_train_leaky = X_scaled_leaky[:160]
    X_test_leaky = X_scaled_leaky[160:]
    y_train, y_test = y[:160], y[160:]

    model_leaky = LogisticRegressionSimple(lr=0.1, n_iter=500)
    model_leaky.fit(X_train_leaky, y_train)
    acc_leaky = np.mean(model_leaky.predict(X_test_leaky) == y_test)

    X_train = X[:160]
    X_test = X[160:]
    scaler_clean = StandardScaler()
    X_train_clean = scaler_clean.fit_transform(X_train)
    X_test_clean = scaler_clean.transform(X_test)

    model_clean = LogisticRegressionSimple(lr=0.1, n_iter=500)
    model_clean.fit(X_train_clean, y_train)
    acc_clean = np.mean(model_clean.predict(X_test_clean) == y_test)

    print(f" Leaky (scaler fit on all data):    {acc_leaky:.3f}")
    print(f" Clean (scaler fit on train only):    {acc_clean:.3f}")
    print(f" Difference:                          {acc_leaky - acc_clean:+.3f}")
    print()
    print(" On this simple case the difference may be small,")
    print(" but on real data with target encoding or feature")
    print(" selection, leakage can inflate accuracy by 10-30%.")
    print()

```

```

def demo_pipeline_from_scratch():
    print("=" * 60)
    print("PIPELINE FROM SCRATCH")
    print("=" * 60)

    rng = np.random.RandomState(42)
    X = rng.randn(300, 5)
    y = (X[:, 0] + 0.5 * X[:, 1] - 0.3 * X[:, 2] > 0).astype(int)

    X_train, X_test = X[:240], X[240:]
    y_train, y_test = y[:240], y[240:]

    pipe = PipelineFromScratch([
        ("scaler", StandardScaler()),
        ("model", LogisticRegressionSimple(lr=0.1, n_iter=500)),
    ])

    pipe.fit(X_train, y_train)
    train_acc = pipe.score(X_train, y_train)
    test_acc = pipe.score(X_test, y_test)

    print(f" Pipeline (scaler + logistic regression):")
    print(f" Train accuracy: {train_acc:.3f}")
    print(f" Test accuracy: {test_acc:.3f}")
    print()

def demo_full_pipeline():
    print("=" * 60)
    print("FULL PIPELINE WITH MIXED DATA TYPES")
    print("=" * 60)

    data = make_mixed_data(n_samples=500)
    train, test = train_test_split_dict(data)

    pipe = FullPipeline(
        model=DecisionTreeSimple(max_depth=5),
        numeric_cols=["age", "income", "score"],
        categorical_cols=["city", "plan"],
    )

    pipe.fit(train)
    train_acc = pipe.score(train)
    test_acc = pipe.score(test)

    print(f" Full pipeline (impute + scale + encode + tree):")
    print(f" Train accuracy: {train_acc:.3f}")
    print(f" Test accuracy: {test_acc:.3f}")
    print()

```

```

def demo_cross_validation():
    print("=" * 60)
    print("CROSS-VALIDATION WITH PIPELINE")
    print("=" * 60)

    data = make_mixed_data(n_samples=500)

    def make_pipeline():
        return FullPipeline(
            model=DecisionTreeSimple(max_depth=5),
            numeric_cols=["age", "income", "score"],
            categorical_cols=["city", "plan"],
        )

    scores = cross_validate_pipeline(make_pipeline, data, n_folds=5)

    print(f" 5-fold CV scores: {[f'{s:.3f}' for s in scores]}")
    print(f" Mean: {np.mean(scores):.3f} +/- {np.std(scores):.3f}")
    print()
    print(" Each fold fits the preprocessor on its own training split.")
    print(" No data leakage across folds.")
    print()

def demo_unknown_categories():
    print("=" * 60)
    print("HANDLING UNKNOWN CATEGORIES")
    print("=" * 60)

    train_cats = np.array([["new_york"], ["chicago"], ["la"], ["houston"]])
    encoder = OneHotEncoder(handle_unknown="ignore")
    encoder.fit(train_cats)

    print(f" Known categories: {encoder.categories[0]}")

    train_encoded = encoder.transform(train_cats)
    print(f" 'new_york' encoded: {train_encoded[0]}")

    unknown = np.array([["seattle"]])
    unknown_encoded = encoder.transform(unknown)
    print(f" 'seattle' (unknown) encoded: {unknown_encoded[0]}")
    print(f" Unknown category produces zero vector (no crash).")
    print()

def demo_model_comparison():
    print("=" * 60)
    print("MODEL COMPARISON VIA PIPELINE")
    print("=" * 60)

    data = make_mixed_data(n_samples=500)

```

```

models = [
    ("Logistic Regression", lambda: LogisticRegressionSimple(lr=0.05, n_iter=1000))
    ,
    ("Decision Tree d=3", lambda: DecisionTreeSimple(max_depth=3)),
    ("Decision Tree d=5", lambda: DecisionTreeSimple(max_depth=5)),
    ("Decision Tree d=10", lambda: DecisionTreeSimple(max_depth=10)),
]

for name, model_fn in models:
    def make_pipe(m=model_fn):
        return FullPipeline(
            model=m(),
            numeric_cols=["age", "income", "score"],
            categorical_cols=["city", "plan"],
        )

    scores = cross_validate_pipeline(make_pipe, data, n_folds=5)
    print(f" {name:>25s}: {np.mean(scores):.3f} +/- {np.std(scores):.3f}")

print()

def demo_sklearn_pipeline():
    print("=" * 60)
    print("SKLEARN PIPELINE (if installed)")
    print("=" * 60)

    try:
        from sklearn.pipeline import Pipeline as SkPipeline
        from sklearn.compose import ColumnTransformer as SkColumnTransformer
        from sklearn.preprocessing import StandardScaler as SkScaler
        from sklearn.preprocessing import OneHotEncoder as SkOHE
        from sklearn.impute import SimpleImputer
        from sklearn.ensemble import GradientBoostingClassifier
        from sklearn.model_selection import cross_val_score
    except ImportError:
        print(" sklearn not installed, skipping.")
        print()
        return

    data = make_mixed_data(n_samples=500)

    import pandas as pd
    df = pd.DataFrame({
        "age": data["age"],
        "income": data["income"],
        "score": data["score"],
        "city": data["city"],
        "plan": data["plan"],
    })
    y = data["target"]

```

```

numeric_pipe = SkPipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", SkScaler()),
])

cat_pipe = SkPipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("encode", SkOHE(handle_unknown="ignore", sparse_output=False)),
])

preprocessor = SkColumnTransformer([
    ("num", numeric_pipe, ["age", "income", "score"]),
    ("cat", cat_pipe, ["city", "plan"]),
])

full_pipe = SkPipeline([
    ("preprocess", preprocessor),
    ("model", GradientBoostingClassifier(n_estimators=100, max_depth=3)),
])

scores = cross_val_score(full_pipe, df, y, cv=5, scoring="accuracy")
print(f" sklearn GBM pipeline:")
print(f" 5-fold CV: {scores.mean():.3f} +/- {scores.std():.3f}")
print(f" Per fold: {[f'{s:.3f}' for s in scores]}")
print()

full_pipe.fit(df, y)
print(f" Pipeline steps: {[name for name, _ in full_pipe.steps]}")
print(f" Preprocessor transformers: {[name for name, _, _ in preprocessor.transfor
mers]}")
print()

def demo_experiment_tracking():
    print("=" * 60)
    print("EXPERIMENT TRACKING (manual log)")
    print("=" * 60)

    data = make_mixed_data(n_samples=500)
    experiments = []

    configs = [
        {"model": "tree", "max_depth": 3},
        {"model": "tree", "max_depth": 5},
        {"model": "tree", "max_depth": 10},
        {"model": "logistic", "lr": 0.01, "n_iter": 500},
        {"model": "logistic", "lr": 0.1, "n_iter": 1000},
    ]

    for i, config in enumerate(configs):
        if config["model"] == "tree":
            model_fn = lambda c=config: DecisionTreeSimple(max_depth=c["max_depth"])

```

```

else:
    model_fn = lambda c=config: LogisticRegressionSimple(lr=c["lr"], n_iter=c["
n_iter"])

def make_pipe(m=model_fn):
    return FullPipeline(
        model=m(),
        numeric_cols=["age", "income", "score"],
        categorical_cols=["city", "plan"],
    )

scores = cross_validate_pipeline(make_pipe, data, n_folds=5)
result = {
    "run_id": i + 1,
    "config": config,
    "mean_accuracy": np.mean(scores),
    "std_accuracy": np.std(scores),
}
experiments.append(result)

print(f" {'Run':>4} {'Config':>40} {'Accuracy':>10} {'Std':>8}")
print(f" {'-'*4} {'-'*40} {'-'*10} {'-'*8}")
for exp in experiments:
    config_str = str(exp["config"])[40]
    print(
        f" {exp['run_id']:>4d} {config_str:>40} "
        f"{exp['mean_accuracy']:>10.3f} {exp['std_accuracy']:>8.3f}"
    )

best = max(experiments, key=lambda e: e["mean_accuracy"])
print(f"\n Best run: #{best['run_id']}")
print(f" Config: {best['config']}")
print(f" Accuracy: {best['mean_accuracy']:.3f} +/- {best['std_accuracy']:.3f}")
print()

def demo_reproducibility():
    print("=" * 60)
    print("REPRODUCIBILITY CHECK")
    print("=" * 60)

    data = make_mixed_data(n_samples=500, seed=42)

    def make_pipe():
        return FullPipeline(
            model=DecisionTreeSimple(max_depth=5),
            numeric_cols=["age", "income", "score"],
            categorical_cols=["city", "plan"],
        )

    run1 = cross_validate_pipeline(make_pipe, data, n_folds=5, seed=42)

```



```

run2 = cross_validate_pipeline(make_pipe, data, n_folds=5, seed=42)
run3 = cross_validate_pipeline(make_pipe, data, n_folds=5, seed=99)

print(f" Run 1 (seed=42): {[f'{s:.4f}' for s in run1]}")
print(f" Run 2 (seed=42): {[f'{s:.4f}' for s in run2]}")
print(f" Run 3 (seed=99): {[f'{s:.4f}' for s in run3]}")
print(f" Run 1 == Run 2: {all(abs(a-b) < 1e-10 for a,b in zip(run1, run2))}")
print(f" Run 1 == Run 3: {all(abs(a-b) < 1e-10 for a,b in zip(run1, run3))}")
print()
print(" Same seed, same data, same results. That is reproducibility.")
print()

if __name__ == "__main__":
    demo_data_leakage()
    demo_pipeline_from_scratch()
    demo_full_pipeline()
    demo_cross_validation()
    demo_unknown_categories()
    demo_model_comparison()
    demo_sklearn_pipeline()
    demo_experiment_tracking()
    demo_reproducibility()
    print("All pipeline demos complete.")

```

You are an expert in building production ML pipelines. You help engineers avoid data leakage, structure reproducible experiments, and deploy models reliably.

When someone asks about ML pipelines, preprocessing, or deployment:

Check for data leakage first. The most common forms:

- a) Fitting transformers (scaler, imputer, encoder) on the full dataset before splitting
- b) Target encoding without proper cross-validation
- c) Feature selection using the test set
- d) Time-series data shuffled before splitting (future leaking into past)
- e) Validation metrics computed on data the model saw during training

Verify the pipeline structure:

- a) All preprocessing steps are inside the Pipeline object, not outside
- b) ColumnTransformer handles different column types correctly
- c) handle_unknown="ignore" is set for categorical encoders
- d) Cross-validation wraps the entire pipeline, not just the model

Check for training/serving skew:

- Is the same Pipeline object used for training and inference?

- Are feature engineering steps duplicated between training and serving code?
- Does the serving code handle missing values the same way as training?
- Are there any features that are available at training time but not at inference time?

Verify reproducibility:

- Random seeds set for all sources of randomness
- Dependencies pinned to exact versions
- Data versioned (DVC or similar)
- Hyperparameters in config files, not hardcoded

Common debugging checklist:

100. Model accuracy drops in production: check for training/serving skew, data drift, or leakage in the original evaluation
101. Cross-validation scores are much higher than holdout: data leakage in preprocessing
102. Model works on notebook but not in production: missing preprocessing steps, different library versions, or hardcoded paths
103. Predictions are NaN: missing value handling failed, check imputation step
104. New categories crash the model: OneHotEncoder without `handle_unknown="ignore"`

Pipeline design patterns:

- Always use sklearn Pipeline for sklearn models
- For deep learning, create a data module that encapsulates all preprocessing
- Log the full pipeline configuration with every experiment (MLflow, wandb)
- Serialize the entire pipeline, not just the model weights
- Version the pipeline artifact alongside the code that created it

Quiz

Pre-lesson check

105. What is data leakage in the context of ML pipelines?
 - Data being accidentally deleted during preprocessing
 - Information from the test set or future data contaminating the training process (**correct**)

- The model being too slow to process the data
- Features being dropped during encoding
Explanation: Data leakage occurs when information that would not be available at prediction time is used during training. Example: computing the scaler's mean/std on the full dataset including test data.

106. Why is fitting a scaler on the full dataset before splitting into train/test considered leaky?

1. Scaling changes the data distribution
2. The scaler's statistics (mean, std) include test data, so the model indirectly sees test information during training **(correct)**
3. Scaling should only be done on the test set
4. Fitting the scaler takes too long on large datasets
Explanation: When the scaler is fit on all data, its mean and standard deviation encode information from test samples. Training features are then shifted using test statistics, leaking future information into training.

Post-lesson check

In sklearn, what is the difference between calling fit_transform and transform on a pipeline step?

- a) They are identical – both fit and transform
- b) fit_transform learns parameters from the data and applies the transform; transform only applies previously learned parameters **(correct)**
- c) transform is for training data; fit_transform is for test data
- d) fit_transform is slower but more accurate
Explanation: fit_transform computes statistics (e.g., mean, std) from the data AND transforms it. transform applies the already-learned statistics without recomputing. On test data, you must use transform to avoid leakage.

Why is a ColumnTransformer necessary for real-world datasets?

3. It makes the pipeline run in parallel on multiple CPUs

4. It applies different preprocessing steps to numeric and categorical columns within the same pipeline **(correct)**
5. It removes columns with missing values automatically
6. It converts all columns to the same data type
Explanation: Real datasets have mixed types. Numeric columns need scaling, categorical columns need encoding. ColumnTransformer routes each column subset to the appropriate transformer within a single pipeline.

A production model receives a categorical value it never saw during training ('new_category'). What should the pipeline handle?

- a) Ignore the row entirely and return no prediction
 - b) Retrain the entire model from scratch
 - c) Handle unknown categories gracefully, e.g., by using an 'unknown' bucket or the global mean in target encoding **(correct)**
 - d) Convert the unknown category to the number zero
Explanation: Robust pipelines anticipate unseen categories in production. Solutions include an 'unknown' fallback for one-hot encoding, or defaulting to the global mean for target encoding.
-

Naive Bayes

The “naive” assumption is wrong, and it works anyway. That’s the beauty of it.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-07 (classification, Bayes’ theorem) **Time:** ~75 minutes

Learning Objectives

Implement Multinomial Naive Bayes from scratch with Laplace smoothing for text classification

Explain why the naive independence assumption is mathematically wrong but produces correct class rankings in practice

Compare Multinomial, Bernoulli, and Gaussian Naive Bayes variants and select the right one for a given feature type

Evaluate Naive Bayes against logistic regression on high-dimensional sparse data and explain the bias-variance tradeoff at work

The Problem

You need to classify text. Emails into spam or not-spam. Customer reviews into positive or negative. Support tickets into categories. You have thousands of features (one per word) and limited training data.

Most classifiers choke here. Logistic regression needs enough samples to estimate thousands of weights reliably. Decision trees split on one word at a time and overfit wildly. KNN in 10,000 dimensions is meaningless because every point is equally far from every other point.

Naive Bayes handles this. It makes a mathematically wrong assumption (that every feature is independent of every other feature given the class), and it still outperforms “smarter” models on text classification, especially with small training sets. It trains in a single pass through the data. It scales to millions of features. It produces probability estimates (though often poorly calibrated due to the independence assumption).

Understanding why a wrong assumption leads to good predictions teaches you something fundamental about machine learning: the best model is not the most correct one, it is the one with the best bias-variance tradeoff for your data.

The Concept

Bayes’ Theorem (Quick Review)

Bayes’ theorem flips conditional probabilities:

$$P(\text{class} \mid \text{features}) = P(\text{features} \mid \text{class}) * P(\text{class}) / P(\text{features})$$

We want $P(\text{class} \mid \text{features})$ – the probability that a document belongs to a class given the words in it. We can compute this from: - $P(\text{features} \mid \text{class})$ – the likelihood of seeing these words in documents of this class - $P(\text{class})$ – the prior probability of the class (how common is spam in general?) - $P(\text{features})$ – the evidence, same for all classes, so we can ignore it when comparing

The class with the highest $P(\text{class} \mid \text{features})$ wins.

The Naive Independence Assumption

Computing $P(\text{features} \mid \text{class})$ exactly requires estimating the joint probability of all features together. With a vocabulary of 10,000 words, you would need to estimate a distribution over $2^{10,000}$ possible combinations. Impossible.

The naive assumption: every feature is conditionally independent given the class.

$$P(w_1, w_2, \dots, w_n \mid \text{class}) = P(w_1 \mid \text{class}) * P(w_2 \mid \text{class}) * \dots * P(w_n \mid \text{class})$$

Instead of one impossible joint distribution, you estimate n simple per-feature distributions. Each one needs only a count.

This assumption is obviously wrong. The words “machine” and “learning” are not independent in any document. But the classifier does not need correct probability estimates. It needs correct rankings – which class has the highest probability. The independence assumption introduces systematic errors, but those errors affect all classes similarly, so the ranking stays correct.

Why It Still Works

Three reasons:

Ranking over calibration. Classification only needs the top-ranked class to be correct. Even if $P(\text{spam}) = 0.99999$ when the true probability is 0.7, the classifier still picks spam correctly. We do not need correct probabilities. We need the correct winner.

High bias, low variance. The independence assumption is a strong prior. It constrains the model heavily, which prevents overfitting. With limited training data, a model that is slightly wrong but stable beats a model that is theoretically right but wildly unstable. This is the bias-variance tradeoff in action.

Feature redundancy cancels out. Correlated features provide redundant evidence. The classifier double-counts this evidence, but it double-counts it for the correct class too. If “machine” and “learning” always appear together, both provide evidence for the “tech” class. NB counts them twice, but it counts them twice for the right class.

A fourth, practical reason: Naive Bayes is extremely fast. Training is a single pass through the data counting frequencies. Prediction is a matrix multiplication. You can train on a million documents in

seconds. This speed means you can iterate faster, try more feature sets, and run more experiments than with slower models.

The Math Step by Step

Let us trace through a concrete example. Suppose we have two classes: spam and not-spam. Our vocabulary has three words: “free”, “money”, “meeting”.

Training data: - Spam emails mention “free” 80 times, “money” 60 times, “meeting” 10 times (150 total words) - Not-spam emails mention “free” 5 times, “money” 10 times, “meeting” 100 times (115 total words) - 40% of emails are spam, 60% are not-spam

With Laplace smoothing ($\alpha=1$):

$$\begin{aligned} P(\text{free} \mid \text{spam}) &= (80 + 1) / (150 + 3) = 81/153 = 0.529 \\ P(\text{money} \mid \text{spam}) &= (60 + 1) / (150 + 3) = 61/153 = 0.399 \\ P(\text{meeting} \mid \text{spam}) &= (10 + 1) / (150 + 3) = 11/153 = 0.072 \\ \\ P(\text{free} \mid \text{not-spam}) &= (5 + 1) / (115 + 3) = 6/118 = 0.051 \\ P(\text{money} \mid \text{not-spam}) &= (10 + 1) / (115 + 3) = 11/118 = 0.093 \\ P(\text{meeting} \mid \text{not-spam}) &= (100 + 1) / (115 + 3) = 101/118 = 0.856 \end{aligned}$$

New email contains: “free” (2 times), “money” (1 time), “meeting” (0 times).

$$\begin{aligned} \log P(\text{spam} \mid \text{email}) &= \log(0.4) + 2*\log(0.529) + 1*\log(0.399) + 0*\log(0.072) \\ &= -0.916 + 2*(-0.637) + (-0.919) + 0 \\ &= -3.109 \\ \\ \log P(\text{not-spam} \mid \text{email}) &= \log(0.6) + 2*\log(0.051) + 1*\log(0.093) + 0*\log(0.856) \\ &= -0.511 + 2*(-2.976) + (-2.375) + 0 \\ &= -8.838 \end{aligned}$$

Spam wins by a large margin. The word “free” appearing twice is strong evidence for spam. Note that “meeting” not appearing contributes zero to both log sums ($0 * \log(P)$) – in Multinomial NB, absent words have no effect. It is Bernoulli NB that explicitly models word absence.

Three Variants

Naive Bayes comes in three flavors. Each models $P(\text{feature} \mid \text{class})$ differently.

Multinomial Naive Bayes

Models each feature as a count. Best for text data where features are word frequencies or TF-IDF values.

$$P(\text{word}_i \mid \text{class}) = (\text{count of word}_i \text{ in class} + \alpha) / (\text{total words in class} + \alpha * \text{vocab_size})$$

The α is Laplace smoothing (explained below). This variant is the workhorse for text classification.

Gaussian Naive Bayes

Models each feature as a normal distribution. Best for continuous features.

$$P(x_i \mid \text{class}) = (1 / \sqrt{2 * \pi * \text{var}}) * \exp(-(x_i - \text{mean})^2 / (2 * \text{var}))$$

Each class gets its own mean and variance per feature. This works well when features genuinely follow a bell curve within each class.

Bernoulli Naive Bayes

Models each feature as binary (present or absent). Best for short text or binary feature vectors.

$$P(\text{word}_i \mid \text{class}) = (\text{docs in class containing word}_i + \alpha) / (\text{total docs in class} + 2 * \alpha)$$

Unlike Multinomial, Bernoulli explicitly penalizes the absence of a word. If “free” typically appears in spam but is absent from this email, Bernoulli counts that as evidence against spam.

When to Use Each Variant

Variant	Feature Type	Best For	Example
Multinomial	Counts or frequencies	Text classification, bag-of-words	Email spam, topic classification
Gaussian	Continuous values	Tabular data with normal-ish features	Iris classification, sensor data
Bernoulli	Binary (0/1)	Short text, binary feature vectors	SMS spam, presence/absence features

Laplace Smoothing

What happens when a word appears in the test data but never appeared in the training data for a particular class?

Without smoothing: $P(\text{word} \mid \text{class}) = 0/N = 0$. One zero multiplied through the entire product makes $P(\text{class} \mid \text{features}) = 0$, regardless of all other evidence. A single unseen word destroys the entire prediction, no matter how much other evidence supports it.

Laplace smoothing adds a small count α (usually 1) to every feature count:

$$P(\text{word}_i \mid \text{class}) = (\text{count}(\text{word}_i, \text{class}) + \alpha) / (\text{total_words_in_class} + \alpha * \text{vocab_size})$$

With $\alpha=1$, every word gets at least a tiny probability. The word “discombobulate” appearing in a test email no longer kills the spam probability. The smoothing has a Bayesian interpretation: it is equivalent to placing a uniform Dirichlet prior on the word distributions.

Higher α means stronger smoothing (more uniform distributions). Lower α means the model trusts the data more. Alpha is a hyperparameter you tune.

The effect of α :

Alpha	Effect	When to use
0.001	Almost no smoothing, trust the data	Very large training set, no unseen features expected
0.1	Light smoothing	Large training set
1.0	Standard Laplace smoothing	Default starting point
10.0	Heavy smoothing, flattens distributions	Very small training set, many unseen features expected

Log-Space Computation

Multiplying hundreds of probabilities (each less than 1) causes floating-point underflow. The product becomes zero in floating point even though the true value is a very small positive number.

The solution: work in log space. Instead of multiplying probabilities, add their logarithms:

$$\log P(\text{class} \mid x_1, x_2, \dots, x_n) = \log P(\text{class}) + \sum_i \log P(x_i \mid \text{class})$$

This turns the prediction into a dot product:

```
log_scores = X @ log_feature_probs.T + log_class_priors
prediction = argmax(log_scores)
```

Matrix multiplication. That is why Naive Bayes prediction is so fast – it is the same operation as a single-layer linear model.

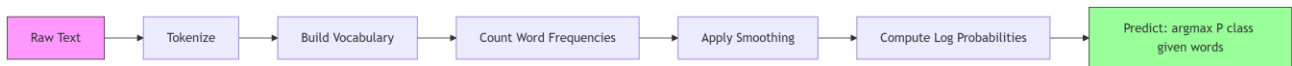
Naive Bayes vs Logistic Regression

Both are linear classifiers for text. The difference is in what they model.

Aspect	Naive Bayes	Logistic Regression
Type	Generative (models $P(X Y)$)	Discriminative (models $P(Y X)$)
Training	Count frequencies	Optimize loss function
Small data	Better (strong prior helps)	Worse (not enough to estimate weights)
Large data	Worse (wrong assumption hurts)	Better (flexible boundary)
Features	Assumes independence	Handles correlations
Speed	Single pass, very fast	Iterative optimization
Calibration	Poor probabilities	Better probabilities

Rule of thumb: start with Naive Bayes. If you have enough data and NB plateaus, switch to logistic regression.

Classification Pipeline



In practice, we work in log space to avoid floating-point underflow. Instead of multiplying many small probabilities, we add their logarithms:

$$\log P(\text{class} \mid \text{features}) = \log P(\text{class}) + \sum_i \log P(\text{feature}_i \mid \text{class})$$

Build It

The code in `code/naive_bayes.py` implements both MultinomialNB and GaussianNB from scratch.

MultinomialNB

The from-scratch implementation:

- **fit(X, y):** For each class, count the frequency of each feature. Add Laplace smoothing. Compute log probabilities. Store class priors (log of class frequencies).
- **predict_log_proba(X):** For each sample, compute $\log P(\text{class}) + \text{sum of } \log P(\text{feature}_i \mid \text{class})$ for all classes. This is a matrix multiplication: $X @ \text{log_probs.T} + \text{log_priors}$.
- **predict(X):** Return the class with highest log probability.

```

class MultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha

    def fit(self, X, y):
        classes = np.unique(y)
        n_classes = len(classes)
        n_features = X.shape[1]

        self.classes_ = classes
        self.class_log_prior_ = np.zeros(n_classes)
        self.feature_log_prob_ = np.zeros((n_classes, n_features))

        for i, c in enumerate(classes):
            X_c = X[y == c]
            self.class_log_prior_[i] = np.log(X_c.shape[0] / X.shape[0])
            counts = X_c.sum(axis=0) + self.alpha
            self.feature_log_prob_[i] = np.log(counts / counts.sum())

        return self

```

The key insight: after fitting, prediction is just matrix multiplication plus a bias. This is why Naive Bayes is so fast.

GaussianNB

For continuous features, we estimate mean and variance per class per feature:

```

class GaussianNB:
    def __init__(self):
        pass

    def fit(self, X, y):
        classes = np.unique(y)
        self.classes_ = classes
        self.means_ = np.zeros((len(classes), X.shape[1]))
        self.vars_ = np.zeros((len(classes), X.shape[1]))
        self.priors_ = np.zeros(len(classes))

        for i, c in enumerate(classes):
            X_c = X[y == c]
            self.means_[i] = X_c.mean(axis=0)
            self.vars_[i] = X_c.var(axis=0) + 1e-9
            self.priors_[i] = X_c.shape[0] / X.shape[0]

        return self

```

Prediction uses the Gaussian PDF per feature, multiplied across features (added in log space).

Demo: Text Classification

The code generates synthetic bag-of-words data simulating two classes (tech articles vs sports articles). Each class has a different word frequency distribution. MultinomialNB classifies them using word counts.

The synthetic data works like this: we create 200 “words” (feature columns). Words 0-39 have high frequency in tech articles and low in sports. Words 80-119 have high frequency in sports and low in tech. Words 40-79 are medium frequency in both. This creates a realistic scenario where some words are strong class indicators and others are noise.

Demo: Continuous Features

The code generates Iris-like data (3 classes, 4 features, Gaussian clusters). GaussianNB classifies using per-class mean and variance. Each class has a different center (mean vector) and different spread (variance), mimicking real-world data where measurements differ systematically between categories.

The code also demonstrates: - **Smoothing comparison:** Training MultinomialNB with different α values to show the effect of smoothing strength on accuracy. - **Training size experiment:** How NB accuracy improves as training data grows from 20 to 1600 samples. NB reaches decent accuracy even with very few samples – this is its main advantage. - **Confusion matrix:** Per-class precision, recall, and F1 score to show where NB makes mistakes.

Prediction Speed

Naive Bayes prediction is a matrix multiplication. For n samples with d features and k classes: - MultinomialNB: one matrix multiply $(n \times d) @ (d \times k) = O(n * d * k)$ - GaussianNB: $n * k$ Gaussian PDF evaluations, each over d features $= O(n * d * k)$

Both are linear in every dimension. Compare this to KNN (which requires distance computation to all training points) or SVM with RBF kernel (which requires kernel evaluation against all support vectors). NB is faster by orders of magnitude at prediction time.

Use It

With sklearn, both variants are one-liners:

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB

gnb = GaussianNB()
gnb.fit(X_train, y_train)
print(f"GaussianNB accuracy: {gnb.score(X_test, y_test):.3f}")

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_train_counts, y_train)
print(f"MultinomialNB accuracy: {mnb.score(X_test_counts, y_test):.3f}")
```

For text classification with sklearn:

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

text_clf = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("classifier", MultinomialNB(alpha=1.0)),
])

text_clf.fit(train_texts, train_labels)
accuracy = text_clf.score(test_texts, test_labels)

```

The code in `naive_bayes.py` compares from-scratch implementations against sklearn on the same data to verify correctness.

TF-IDF with Naive Bayes

Raw word counts give every word equal weight per occurrence. But common words like “the” and “is” appear frequently in every class – they carry no information. TF-IDF (Term Frequency - Inverse Document Frequency) downweights common words and upweights rare, discriminative words.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

text_clf = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", MultinomialNB(alpha=0.1)),
])

```

TF-IDF values are non-negative, so they work with MultinomialNB. The combination of TF-IDF + MultinomialNB is one of the strongest baselines for text classification. It frequently beats more complex models on datasets with fewer than 10,000 training samples.

BernoulliNB for Short Text

For short text (tweets, SMS, chat messages), BernoulliNB can outperform MultinomialNB. Short texts have low word counts, so the frequency information that MultinomialNB relies on is noisy. BernoulliNB only cares about presence or absence, which is more reliable with short text.

```

from sklearn.naive_bayes import BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer

text_clf = Pipeline([
    ("vectorizer", CountVectorizer(binary=True)),
    ("classifier", BernoulliNB(alpha=1.0)),
])

```

The `binary=True` flag in `CountVectorizer` converts all counts to 0/1. Without it, `BernoulliNB` still works but is seeing counts that it was not designed for.

Calibrating NB Probabilities

NB probabilities are poorly calibrated. When NB says $P(\text{spam}) = 0.95$, the true probability might be 0.7. If you need reliable probability estimates (for example, to set a threshold or to combine with other models), use `sklearn`'s `CalibratedClassifierCV`:

```
from sklearn.calibration import CalibratedClassifierCV

calibrated_nb = CalibratedClassifierCV(MultinomialNB(), cv=5, method="sigmoid")
calibrated_nb.fit(X_train, y_train)
proba = calibrated_nb.predict_proba(X_test)
```

This fits a logistic regression on top of NB's raw scores using cross-validation. The resulting probabilities are much closer to the true class frequencies.

Common Gotchas

- **Negative feature values.** `MultinomialNB` requires non-negative features. If you have negative values (like TF-IDF with certain settings or standardized features), use `GaussianNB` instead, or shift the features to be positive.
- **Zero variance features.** `GaussianNB` divides by variance. If a feature has zero variance for a class (all values identical), the probability computation breaks. The code adds a small smoothing term ($1e-9$) to all variances to prevent this.
- **Class imbalance.** If 99% of emails are not-spam, the prior $P(\text{not-spam}) = 0.99$ is so strong that it overwhelms the likelihood evidence. You can set class priors manually or use `class_prior` parameter in `sklearn`.
- **Feature scaling.** `MultinomialNB` does not need scaling (it works on counts). `GaussianNB` does not need scaling either (it estimates per-feature statistics). This is an advantage over logistic regression and SVM, which are sensitive to feature scales.

When Naive Bayes Fails

NB fails when the independence assumption causes incorrect rankings (not just incorrect probabilities). This happens when:

107. **Strong feature interactions.** If the class depends on the combination of two features but not either alone (XOR-like patterns), NB will miss it entirely. Each feature alone provides no evidence, and NB cannot combine them nonlinearly.

108. **Highly correlated features with opposing evidence.** If feature A says “spam” and feature B says “not-spam”, but A and B are perfectly correlated (they always agree in reality), NB will see conflicting evidence where there is none.
109. **Very large training sets.** With enough data, discriminative models like logistic regression learn the true decision boundary and outperform NB. The independence assumption that helped with small data now holds the model back.

In practice, these failure modes are rare for text classification. Text features are numerous, individually weak, and the independence assumption’s errors tend to cancel out. For tabular data with few strongly correlated features, consider logistic regression or tree-based models first.

Exercises

- **Smoothing experiment.** Train MultinomialNB on text data with α values of 0.01, 0.1, 1.0, 10.0, and 100.0. Plot accuracy vs α . Where does performance peak? Why does very high α hurt?
- **Feature independence test.** Take a real text dataset. Pick two words that are obviously correlated (“machine” and “learning”). Compute $P(\text{word1} \mid \text{class}) * P(\text{word2} \mid \text{class})$ and compare to $P(\text{word1 AND word2} \mid \text{class})$. How wrong is the independence assumption? Does it affect classification accuracy?
- **Bernoulli implementation.** Extend the code with a BernoulliNB class. Convert bag-of-words to binary (present/absent) and compare accuracy against MultinomialNB on text data. When does Bernoulli win?
- **NB vs Logistic Regression.** Train both on text data. Start with 100 training samples and increase to 10,000. Plot accuracy vs training set size for both. At what point does Logistic Regression overtake Naive Bayes?
- **Spam filter.** Build a complete spam classifier: tokenize raw email text, build vocabulary, create bag-of-words features, train MultinomialNB, evaluate with precision and recall (not just accuracy – why?).

Key Terms

Term	What people say	What it actually means
Naive Bayes	“Simple probabilistic classifier”	A classifier that applies Bayes’ theorem with the assumption that features are conditionally independent given the class
Conditional independence	“Features don’t affect each other”	$P(A, B \mid C) = P(A \mid C) * P(B \mid C)$ – knowing B tells you nothing new about A once you know C
Laplace smoothing	“Add-one smoothing”	Adding a small count to every feature to prevent zero probabilities from dominating the prediction

Term	What people say	What it actually means
Prior	“What you believed before seeing data”	$P(\text{class})$ – the probability of each class before observing any features
Likelihood	“How well the data fits”	$P(\text{features} \mid \text{class})$ – the probability of observing these features if the class is known
Posterior	“What you believe after seeing data”	$P(\text{class} \mid \text{features})$ – the updated probability of the class after observing the features
Generative model	“Models how data is generated”	A model that learns $P(X \mid Y)$ and $P(Y)$, then uses Bayes’ theorem to get $P(Y \mid X)$
Discriminative model	“Models the decision boundary”	A model that directly learns $P(Y \mid X)$ without modeling how X is generated
Log probability	“Avoid underflow”	Working with $\log P$ instead of P to prevent the product of many small numbers from becoming zero in floating point

Code Appendix: *naive_bayes.py*

```
import numpy as np

class MultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.classes_ = None
        self.class_log_prior_ = None
        self.feature_log_prob_ = None

    def fit(self, X, y):
        if np.any(X < 0):
            raise ValueError("MultinomialNB requires non-negative feature values")
        self.classes_ = np.unique(y)
        n_classes = len(self.classes_)
        n_features = X.shape[1]

        self.class_log_prior_ = np.zeros(n_classes)
        self.feature_log_prob_ = np.zeros((n_classes, n_features))

        for i, c in enumerate(self.classes_):
            X_c = X[y == c]
            self.class_log_prior_[i] = np.log(X_c.shape[0] / X.shape[0])
            counts = X_c.sum(axis=0) + self.alpha
            total = counts.sum()
            self.feature_log_prob_[i] = np.log(counts / total)
```



```

        return self

    def predict_log_proba(self, X):
        return X @ self.feature_log_prob_.T + self.class_log_prior_

    def predict_proba(self, X):
        log_proba = self.predict_log_proba(X)
        log_proba -= log_proba.max(axis=1, keepdims=True)
        proba = np.exp(log_proba)
        proba /= proba.sum(axis=1, keepdims=True)
        return proba

    def predict(self, X):
        log_proba = self.predict_log_proba(X)
        return self.classes_[np.argmax(log_proba, axis=1)]

    def score(self, X, y):
        return np.mean(self.predict(X) == y)

class GaussianNB:
    def __init__(self, var_smoothing=1e-9):
        self.var_smoothing = var_smoothing
        self.classes_ = None
        self.means_ = None
        self.vars_ = None
        self.priors_ = None

    def fit(self, X, y):
        self.classes_ = np.unique(y)
        n_classes = len(self.classes_)
        n_features = X.shape[1]

        self.means_ = np.zeros((n_classes, n_features))
        self.vars_ = np.zeros((n_classes, n_features))
        self.priors_ = np.zeros(n_classes)

        for i, c in enumerate(self.classes_):
            X_c = X[y == c]
            self.means_[i] = X_c.mean(axis=0)
            self.vars_[i] = X_c.var(axis=0) + self.var_smoothing
            self.priors_[i] = X_c.shape[0] / X.shape[0]

        return self

    def _log_likelihood(self, X):
        n_classes = len(self.classes_)
        n_samples = X.shape[0]
        log_proba = np.zeros((n_samples, n_classes))

        for i in range(n_classes):
            diff = X - self.means_[i]
            log_prob_features = (

```

```

        -0.5 * np.log(2 * np.pi * self.vars_[i])
        - 0.5 * (diff ** 2) / self.vars_[i]
    )
    log_proba[:, i] = log_prob_features.sum(axis=1) + np.log(self.priors_[i])

    return log_proba

def predict(self, X):
    log_proba = self._log_likelihood(X)
    return self.classes_[np.argmax(log_proba, axis=1)]

def predict_proba(self, X):
    log_proba = self._log_likelihood(X)
    log_proba -= log_proba.max(axis=1, keepdims=True)
    proba = np.exp(log_proba)
    proba /= proba.sum(axis=1, keepdims=True)
    return proba

def score(self, X, y):
    return np.mean(self.predict(X) == y)

def make_text_data(n_samples=1000, n_features=200, seed=42):
    rng = np.random.RandomState(seed)

    tech_words_weight = np.zeros(n_features)
    tech_words_weight[:40] = rng.uniform(3, 10, 40)
    tech_words_weight[40:80] = rng.uniform(0.5, 2, 40)
    tech_words_weight[80:] = rng.uniform(0.1, 1, 120)

    sports_words_weight = np.zeros(n_features)
    sports_words_weight[:40] = rng.uniform(0.1, 1, 40)
    sports_words_weight[40:80] = rng.uniform(0.5, 2, 40)
    sports_words_weight[80:120] = rng.uniform(3, 10, 40)
    sports_words_weight[120:] = rng.uniform(0.1, 1, 80)

    n_tech = n_samples // 2
    n_sports = n_samples - n_tech

    X_tech = rng.poisson(tech_words_weight, (n_tech, n_features)).astype(float)
    X_sports = rng.poisson(sports_words_weight, (n_sports, n_features)).astype(float)

    X = np.vstack([X_tech, X_sports])
    y = np.array([0] * n_tech + [1] * n_sports)

    shuffle_idx = rng.permutation(n_samples)
    return X[shuffle_idx], y[shuffle_idx]

def make_continuous_data(n_samples=300, seed=42):
    rng = np.random.RandomState(seed)
    n_per_class = n_samples // 3

```

```

class_0 = rng.multivariate_normal(
    [5.0, 3.4, 1.4, 0.2],
    np.diag([0.12, 0.14, 0.03, 0.01]),
    n_per_class,
)
class_1 = rng.multivariate_normal(
    [5.9, 2.8, 4.3, 1.3],
    np.diag([0.27, 0.10, 0.22, 0.04]),
    n_per_class,
)
class_2 = rng.multivariate_normal(
    [6.6, 3.0, 5.6, 2.0],
    np.diag([0.40, 0.10, 0.30, 0.08]),
    n_per_class,
)

X = np.vstack([class_0, class_1, class_2])
y = np.array([0] * n_per_class + [1] * n_per_class + [2] * n_per_class)

shuffle_idx = rng.permutation(len(y))
return X[shuffle_idx], y[shuffle_idx]

def train_test_split(X, y, test_ratio=0.2, seed=42):
    rng = np.random.RandomState(seed)
    n = len(y)
    idx = rng.permutation(n)
    split = int(n * (1 - test_ratio))
    train_idx, test_idx = idx[:split], idx[split:]
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]

def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred)

def print_separator(title):
    print(f"\n{'=' * 60}")
    print(f" {title}")
    print(f"{'=' * 60}\n")

def demo_multinomial():
    print_separator("MULTINOMIAL NAIVE BAYES -- TEXT CLASSIFICATION")

    X, y = make_text_data(n_samples=1200, n_features=200, seed=42)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_ratio=0.25, seed=42)

    print(f"Training samples: {X_train.shape[0]}")
    print(f"Test samples: {X_test.shape[0]}")
    print(f"Features (words): {X_train.shape[1]}")
    print(f"Classes: tech (0), sports (1)")

```

```

print()

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_train, y_train)

train_acc = mnb.score(X_train, y_train)
test_acc = mnb.score(X_test, y_test)
print(f"From-scratch MultinomialNB:")
print(f"  Train accuracy: {train_acc:.4f}")
print(f"  Test accuracy:  {test_acc:.4f}")

proba = mnb.predict_proba(X_test[:5])
print(f"\nPredicted probabilities (first 5 samples):")
for i in range(5):
    print(f"  Sample {i}: P(tech)={proba[i, 0]:.4f}, P(sports)={proba[i, 1]:.4f}")
-> {'tech' if proba[i, 0] > proba[i, 1] else 'sports'})

print(f"\nSmoothing (alpha) comparison:")
for alpha in [0.01, 0.1, 1.0, 5.0, 10.0]:
    model = MultinomialNB(alpha=alpha)
    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)
    print(f"  alpha={alpha:5.2f} -> test accuracy: {acc:.4f}")

def demo_gaussian():
    print_separator("GAUSSIAN NAIVE BAYES -- CONTINUOUS FEATURES")

    X, y = make_continuous_data(n_samples=450, seed=42)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_ratio=0.25, seed=42)

    print(f"Training samples: {X_train.shape[0]}")
    print(f"Test samples:      {X_test.shape[0]}")
    print(f"Features:           {X_train.shape[1]}")
    print(f"Classes:           0, 1, 2 (Iris-like)")
    print()

    gnb = GaussianNB()
    gnb.fit(X_train, y_train)

    train_acc = gnb.score(X_train, y_train)
    test_acc = gnb.score(X_test, y_test)
    print(f"From-scratch GaussianNB:")
    print(f"  Train accuracy: {train_acc:.4f}")
    print(f"  Test accuracy:  {test_acc:.4f}")

    print(f"\nLearned parameters:")
    for i, c in enumerate(gnb.classes_):
        print(f"  Class {c}:")
        print(f"    Means: {gnb.means_[i].round(3)}")

```

```

print(f"    Vars: {gnb.vars_[i].round(4)}")
print(f"    Prior: {gnb.priors_[i]:.3f}")

proba = gnb.predict_proba(X_test[:5])
print(f"\nPredicted probabilities (first 5 samples):")
for i in range(5):
    pred = gnb.classes_[np.argmax(proba[i])]
    probs_str = ", ".join(f"P({c})={proba[i, j]:.4f}" for j, c in enumerate(gnb.classes_))
    print(f"    Sample {i}: {probs_str} -> class {pred}")

def demo_comparison():
    print_separator("COMPARISON: MULTINOMIAL vs GAUSSIAN")

    print("Task 1: Text data (bag-of-words counts)")
    X, y = make_text_data(n_samples=1000, seed=99)
    X_train, X_test, y_train, y_test = train_test_split(X, y, seed=99)

    mnb = MultinomialNB(alpha=1.0)
    mnb.fit(X_train, y_train)
    mnb_acc = mnb.score(X_test, y_test)

    gnb = GaussianNB()
    gnb.fit(X_train, y_train)
    gnb_acc = gnb.score(X_test, y_test)

    print(f"    MultinomialNB: {mnb_acc:.4f}")
    print(f"    GaussianNB:      {gnb_acc:.4f}")
    print(f"    Winner: {'MultinomialNB' if mnb_acc >= gnb_acc else 'GaussianNB'}")

    print(f"\nTask 2: Continuous features (Iris-like)")
    X, y = make_continuous_data(n_samples=450, seed=99)
    X_train, X_test, y_train, y_test = train_test_split(X, y, seed=99)

    X_train_pos = X_train - X_train.min(axis=0) + 0.01
    X_test_pos = X_test - X_train.min(axis=0) + 0.01

    mnb2 = MultinomialNB(alpha=1.0)
    mnb2.fit(X_train_pos, y_train)
    mnb_acc2 = mnb2.score(X_test_pos, y_test)

    gnb2 = GaussianNB()
    gnb2.fit(X_train, y_train)
    gnb_acc2 = gnb2.score(X_test, y_test)

    print(f"    MultinomialNB: {mnb_acc2:.4f} (shifted to positive)")
    print(f"    GaussianNB:      {gnb_acc2:.4f}")
    print(f"    Winner: {'MultinomialNB' if mnb_acc2 >= gnb_acc2 else 'GaussianNB'}")

```

```

def demo_training_size():
    print_separator("NAIVE BAYES vs TRAINING SET SIZE")

    X_full, y_full = make_text_data(n_samples=2000, n_features=200, seed=42)
    X_test_full = X_full[1600:]
    y_test_full = y_full[1600:]

    print(f"{'Train Size':>12} {'Accuracy':>10}")
    print(f"{'-' * 24}")

    for n_train in [20, 50, 100, 200, 500, 1000, 1600]:
        X_train = X_full[:n_train]
        y_train = y_full[:n_train]

        mnb = MultinomialNB(alpha=1.0)
        mnb.fit(X_train, y_train)
        acc = mnb.score(X_test_full, y_test_full)
        print(f"{'n_train':>12} {'acc:>10.4f}")

def demo_confusion_matrix():
    print_separator("CONFUSION MATRIX AND PER-CLASS METRICS")

    X, y = make_text_data(n_samples=800, seed=42)
    X_train, X_test, y_train, y_test = train_test_split(X, y, seed=42)

    mnb = MultinomialNB(alpha=1.0)
    mnb.fit(X_train, y_train)
    y_pred = mnb.predict(X_test)

    classes = np.unique(y_test)
    n_classes = len(classes)
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for true, pred in zip(y_test, y_pred):
        cm[int(true), int(pred)] += 1

    class_names = ["tech", "sports"]
    print("Confusion Matrix:")
    print(f"{'':>12} {'Pred tech':>12} {'Pred sports':>12}")
    for i, name in enumerate(class_names):
        row = "".join(f"{'cm[i, j]:>12}' for j in range(n_classes))
        print(f"{'True ' + name:>12}{row}")

    print(f"\nPer-class metrics:")
    for i, name in enumerate(class_names):
        tp = cm[i, i]
        fp = cm[:, i].sum() - tp
        fn = cm[i, :].sum() - tp
        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0

```

```

        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0
    else 0
    print(f" {name:>8}: precision={precision:.4f}, recall={recall:.4f}, f1={f1:.4f}")

if __name__ == "__main__":
    demo_multinomial()
    demo_gaussian()
    demo_comparison()
    demo_training_size()
    demo_confusion_matrix()

```

You are an expert in probabilistic classification. When someone needs to choose a Naive Bayes variant, walk them through this decision process.

Decision Checklist

Step 1: What are your features?

- **Word counts or TF-IDF values** -> MultinomialNB
- **Continuous measurements (temperature, height, sensor readings)** -> GaussianNB
- **Binary indicators (word present/absent, checkbox states)** -> BernoulliNB
- **Mixed types** -> Split into subsets, or convert all to one type

Step 2: How much data do you have?

Under 1,000 samples: Naive Bayes is a strong choice. Its strong prior (independence assumption) prevents overfitting.

1,000 to 50,000 samples: NB is still competitive. Compare against logistic regression.

Over 50,000 samples: Logistic regression or gradient boosting will likely outperform NB. Use NB as a baseline.

Step 3: Tune smoothing

Start with $\alpha=1.0$ (Laplace smoothing).

If accuracy is low and you have enough data, try $\alpha=0.1$ or 0.01 .

If the model is overfitting (train >> test accuracy), increase α to 5.0 or 10.0 .

Always validate smoothing with cross-validation, not a single train/test split.

Step 4: Check assumptions

MultinomialNB: Features must be non-negative. If you have negative values, shift or use GaussianNB.

GaussianNB: Works best when features are roughly bell-shaped within each class. Check with histograms.

BernoulliNB: Binarize your features first. Choose the threshold carefully (for text: present=1, absent=0).

Common Mistakes

- 36. **Using GaussianNB on text data.** Word counts are not Gaussian. Use MultinomialNB.
- 37. **Forgetting Laplace smoothing.** A single unseen word zeros out the entire probability. Always smooth.
- 38. **Trusting the probability outputs.** NB probabilities are poorly calibrated. Use them for ranking, not as confidence scores. If you need calibrated probabilities, use CalibratedClassifierCV.
- 39. **Ignoring class imbalance.** NB priors reflect class frequencies. With 99% negative and 1% positive, the prior overwhelms the likelihood. Adjust priors manually or resample.

Quick Reference

Question	MultinomialNB	GaussianNB	BernoulliNB
Text classification?	Yes	No	Maybe (short text)
Continuous features?	No	Yes	No
Binary features?	No	No	Yes
Very fast training needed?	Yes	Yes	Yes
Small training set?	Good	Good	Good
Need calibrated probabilities?	No	No	No

When NOT to Use Naive Bayes

Features are highly correlated and you have enough data for a model that handles correlations (logistic regression, gradient boosting)

You need the best possible accuracy and have plenty of data

Your features are images, sequences, or graphs (use neural networks)

You need a model that captures feature interactions (use tree-based methods)

Quiz

Pre-lesson check

What is the 'naive' assumption in Naive Bayes?

- a) The prior probability of each class is equal
- b) All features are conditionally independent given the class label **(correct)**
- c) The data is normally distributed
- d) The model has no parameters to learn
Explanation: Naive Bayes assumes every feature is independent of every other feature, conditioned on the class. This is mathematically wrong (e.g., 'machine' and 'learning' co-occur) but works well in practice.

What does Laplace smoothing prevent in Naive Bayes?

- Overfitting to large datasets
- Zero probabilities for words never seen in a class during training **(correct)**
- Slow training on high-dimensional data
- Class imbalance in the dataset
Explanation: Without smoothing, a word that never appeared in 'spam' training emails would get $P(\text{word}|\text{spam}) = 0$, making the entire product zero regardless of other strong evidence. Laplace smoothing adds 1 to each count.

Post-lesson check

110. The naive independence assumption is clearly wrong for text. Why does Naive Bayes still classify well?

- It only works on very small vocabularies where independence holds
- Classification only needs correct class rankings, not correct probability estimates, and the assumption introduces stable errors that affect all classes similarly **(correct)**
- Modern implementations secretly remove the independence assumption
- It only works when features are truly independent
Explanation: NB needs to rank classes correctly, not estimate exact probabilities. The independence assumption is high bias but low variance, making it stable with limited data. Correlated features double-count evidence for the correct class too.

111. When should you use Multinomial NB versus Gaussian NB?

1. Multinomial for regression, Gaussian for classification
2. Multinomial for word count/frequency features, Gaussian for continuous real-valued features **(correct)**
3. Multinomial for binary data, Gaussian for multi-class problems

4. They are interchangeable – always use whichever is faster

Explanation: Multinomial NB models feature counts (word frequencies in text). Gaussian NB assumes features follow normal distributions, suitable for continuous features like measurements or sensor readings.

112. An email contains 'free' twice and 'money' once. In Multinomial NB with log probabilities, how is the spam score computed?

- $\log P(\text{spam}) + \log P(\text{free} | \text{spam}) + \log P(\text{money} | \text{spam})$
- $\log P(\text{spam}) + 2 * \log P(\text{free} | \text{spam}) + 1 * \log P(\text{money} | \text{spam})$ **(correct)**
- $P(\text{spam}) * P(\text{free} | \text{spam}) * P(\text{money} | \text{spam})$
- $\log P(\text{spam}) * 2 * \log P(\text{free} | \text{spam})$

Explanation: Multinomial NB multiplies the word likelihoods raised to their count. In log space: $\log P(\text{spam}) + 2\log P(\text{free} | \text{spam}) + 1\log P(\text{money} | \text{spam})$. The word count acts as an exponent.

Time Series Fundamentals

Past performance does predict future results – if you check for stationarity first.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 **Time:** ~90 minutes

Learning Objectives

- Decompose a time series into trend, seasonality, and residual components and test for stationarity
- Implement lag features and rolling statistics to convert a time series into a supervised learning problem
- Build a walk-forward validation framework that prevents future data from leaking into training
- Explain why random train/test splits are invalid for time series and demonstrate the performance gap versus proper temporal splits

The Problem

You have data ordered by time. Daily sales, hourly temperature, per-minute CPU usage, weekly stock prices. You want to predict the next value, the next week, the next quarter.

You reach for your standard ML toolkit: random train/test split, cross-validation, feature matrix in, prediction out. Every step is wrong.

Time series breaks the assumptions that standard ML relies on. Samples are not independent – today's temperature depends on yesterday's. Random splits leak future information into the past. Features that look great in backtest fail in production because they rely on patterns that shift over time.

A model that gets 95% accuracy with random cross-validation might get 55% with proper time-based evaluation. The difference is not a technicality. It is the difference between a model that works on paper and one that works in production.

This lesson covers the fundamentals: what makes time data different, how to evaluate models honestly, and how to turn a time series into features that standard ML models can consume.

The Concept

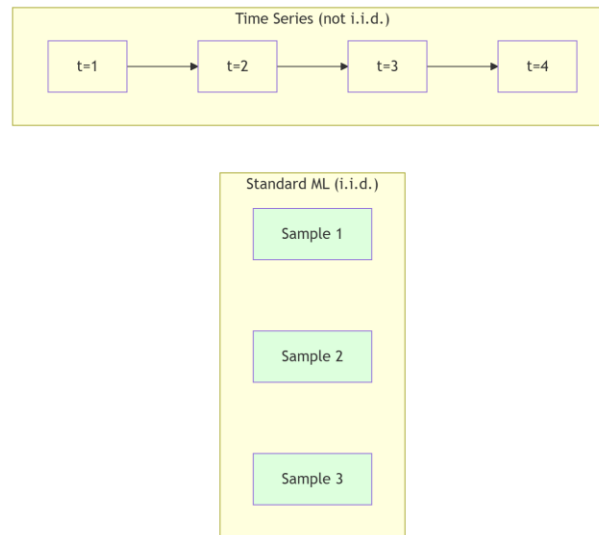
What Makes Time Series Different

Standard ML assumes i.i.d. – independent and identically distributed. Each sample is drawn from the same distribution, independently of other samples. Time series violates both:

- **Not independent.** Today's stock price depends on yesterday's. This week's sales correlate with last week's.

- **Not identically distributed.** The distribution shifts over time. Sales in December look different from sales in March.

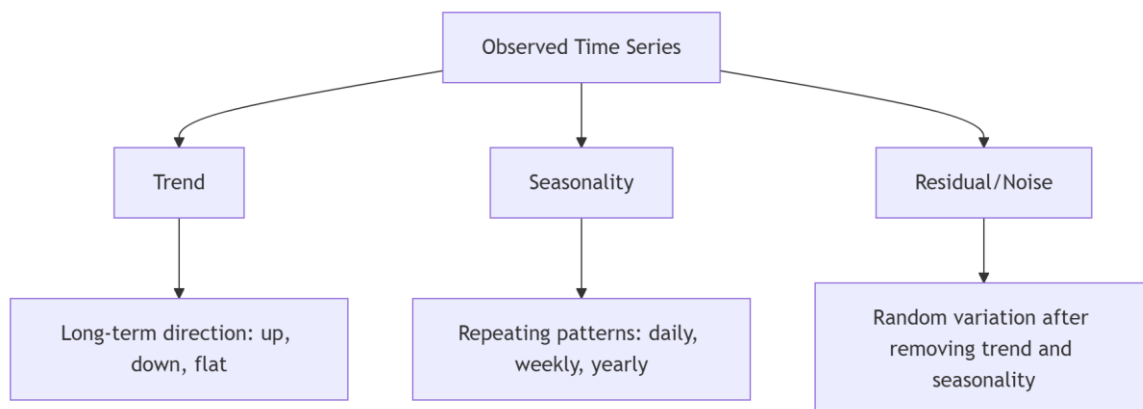
These violations are not minor. They change how you build features, how you evaluate models, and which algorithms work.



In standard ML, samples are interchangeable. Shuffling them changes nothing. In time series, order is everything. Shuffling destroys the signal.

Components of a Time Series

Every time series is a combination of:



- **Trend:** The long-term direction. Revenue growing 10% per year. Global temperature rising.
- **Seasonality:** Repeating patterns at fixed intervals. Retail sales spike in December. Air conditioning usage peaks in July.
- **Residual:** Whatever is left after removing trend and seasonality. If the residual looks like white noise, the decomposition captured the signal.

Stationarity

A time series is stationary if its statistical properties (mean, variance, autocorrelation) do not change over time. Most forecasting methods assume stationarity.

Why it matters: A non-stationary series has a mean that drifts. A model trained on data from January has learned a different mean than what February will show. It will be systematically wrong.

How to check: Compute rolling mean and rolling standard deviation over windows. If they drift, the series is non-stationary.

How to fix: Differencing. Instead of modeling the raw values, model the change between consecutive values:

```
diff[t] = value[t] - value[t-1]
```

If one round of differencing does not make the series stationary, apply it again (second-order differencing). Most real-world series need at most two rounds.

Example:

Original series: [100, 102, 106, 112, 120] First difference: [2, 4, 6, 8] (still trending upward) Second difference: [2, 2, 2] (constant – stationary)

The original series had a quadratic trend. First differencing turned it into a linear trend. Second differencing made it flat. In practice, you rarely need more than two rounds.

Formal test: The Augmented Dickey-Fuller (ADF) test is the standard statistical test for stationarity. The null hypothesis is “the series is non-stationary.” A p-value below 0.05 means you can reject the null and conclude stationarity. We do not implement ADF from scratch (it requires asymptotic distribution tables), but the rolling statistics approach in our code gives a practical visual check.

Autocorrelation

Autocorrelation measures how much a value at time t correlates with the value at time $t-k$ (k steps in the past). The autocorrelation function (ACF) plots this correlation for each lag k .

ACF tells you: - How far back the series remembers. If ACF drops to zero after lag 5, values more than 5 steps ago are irrelevant. - Whether seasonality exists. If ACF spikes at lag 12 (monthly data), there is yearly seasonality. - How many lag features to create. Use lags up to where ACF becomes negligible.

PACF (Partial Autocorrelation Function) removes indirect correlations. If today correlates with 3 days ago only because both correlate with yesterday, PACF at lag 3 will be zero while ACF at lag 3 will not.

Lag Features: Turning Time Series into Supervised Learning

Standard ML models need a feature matrix X and a target y. Time series gives you a single column of values. The bridge is lag features.

Take the series [10, 12, 14, 13, 15] and create lag-1 and lag-2 features:

lag_2	lag_1	target
10	12	14
12	14	13
14	13	15

Now you have a standard regression problem. Any ML model (linear regression, random forest, gradient boosting) can predict the target from the lags.

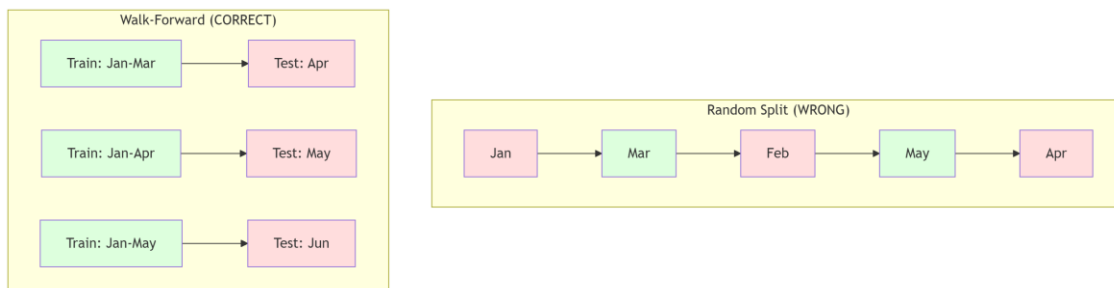
Additional features you can engineer: - **Rolling statistics:** mean, std, min, max over the last k values - **Calendar features:** day of week, month, is_holiday, is_weekend - **Differenced values:** change from previous step - **Expanding statistics:** cumulative mean, cumulative sum - **Ratio features:** current value / rolling mean (how far from recent average) - **Interaction features:** lag_1 * day_of_week (weekday effects on momentum)

How many lags? Use the autocorrelation function. If ACF is significant up to lag 10, use at least 10 lags. If there is weekly seasonality, include lag 7 (and possibly 14). More lags give the model more history but also more features to fit, increasing the risk of overfitting.

The target alignment trap. When creating lag features, the target must be the value at time t, and all features must use values at time t-1 or earlier. If you accidentally include the value at time t as a feature, you have a perfect predictor – and a completely useless model. This is the most common bug in time series feature engineering.

Walk-Forward Validation

This is the most important concept in this lesson. Standard k-fold cross-validation randomly assigns samples to train and test. For time series, this leaks future information.



Walk-forward validation: 1. Train on data up to time t 2. Predict at time $t+1$ (or $t+1$ to $t+k$ for multi-step) 3. Slide the window forward 4. Repeat

Each test fold only contains data that comes after all training data. No future leakage. This gives you an honest estimate of how the model will perform when deployed.

Expanding window uses all historical data for training (window grows). **Sliding window** uses a fixed-size training window (window slides). Use expanding when you believe older data is still relevant. Use sliding when the world changes and old data hurts.

ARIMA Intuition

ARIMA is the classical time series model. It has three components:

- **AR (Autoregressive):** Predict from past values. $AR(p)$ uses the last p values.
- **I (Integrated):** Differencing to achieve stationarity. $I(d)$ applies d rounds of differencing.
- **MA (Moving Average):** Predict from past forecast errors. $MA(q)$ uses the last q errors.

$ARIMA(p, d, q)$ combines all three. You choose p, d, q based on ACF/PACF analysis or automated search (auto-ARIMA).

We will not implement ARIMA from scratch – it requires numerical optimization that is beyond the scope of this lesson. The key insight is understanding what each component does so you can interpret ARIMA results and know when to use it.

When to Use What

Approach	Best For	Handles Seasonality	Handles External Features
Lag features + ML	Tabular with many external features	With calendar features	Yes
ARIMA	Single univariate series, short-term	SARIMA variant	No (ARIMAX for limited)
Exponential smoothing	Simple trend + seasonality	Yes (Holt-Winters)	No
Prophet	Business forecasting, holidays	Yes (Fourier terms)	Limited
Neural networks (LSTM, Transformer)	Long sequences, many series	Learned	Yes

For most practical problems, lag features + gradient boosting is the strongest starting point. It handles external features naturally, does not require stationarity, and is easy to debug.

Forecasting Horizons and Strategies

Single-step forecasting predicts one time step ahead. Multi-step forecasting predicts multiple steps. There are three strategies:

Recursive (iterated): Predict one step ahead, use the prediction as input for the next step. Simple but errors accumulate – each prediction uses the previous prediction, so mistakes compound.

Direct: Train a separate model for each horizon. Model-1 predicts $t+1$, Model-5 predicts $t+5$. No error accumulation, but each model has fewer training samples and they do not share information.

Multi-output: Train one model that outputs all horizons simultaneously. Shares information across horizons but requires a model that supports multiple outputs (or a custom loss function).

For most practical problems, start with recursive for short horizons (1-5 steps) and direct for longer horizons.

Common Mistakes in Time Series

Mistake	Why it happens	How to fix
Random train/test split	Habit from standard ML	Use walk-forward or temporal split
Using future features	Feature at time t included by mistake	Audit every feature for temporal alignment
Overfitting to seasonality	Model memorizes calendar patterns	Hold out a full seasonal cycle in the test set
Ignoring scale changes	Revenue doubles but patterns stay	Model percentage change instead of absolute
Too many lag features	“More history is better”	Use ACF to determine relevant lags
Not differencing	“The model will figure it out”	Tree models handle trends; linear models need stationarity

Build It

The code in `code/time_series.py` implements the core building blocks from scratch.

Lag Feature Creator

```
def make_lag_features(series, n_lags):  
    n = len(series)  
    X = np.full((n, n_lags), np.nan)
```



```

for lag in range(1, n_lags + 1):
    X[lag:, lag - 1] = series[:-lag]
valid = ~np.isnan(X).any(axis=1)
return X[valid], series[valid]

```

This converts a 1D series into a feature matrix where each row has the last `n_lags` values as features, and the current value as the target.

Walk-Forward Cross-Validation

```

def walk_forward_split(n_samples, n_splits=5, min_train=50):
    assert min_train < n_samples, "min_train must be less than n_samples"
    step = max(1, (n_samples - min_train) // n_splits)
    for i in range(n_splits):
        train_end = min_train + i * step
        test_end = min(train_end + step, n_samples)
        if train_end >= n_samples:
            break
        yield slice(0, train_end), slice(train_end, test_end)

```

Each split ensures training data comes strictly before test data. The training window expands with each fold.

Simple Autoregressive Model

A pure AR model is just linear regression on lag features:

```

class SimpleAR:
    def __init__(self, n_lags=5):
        self.n_lags = n_lags
        self.weights = None
        self.bias = None

    def fit(self, series):
        X, y = make_lag_features(series, self.n_lags)
        # Solve via normal equations
        X_b = np.column_stack([np.ones(len(X)), X])
        theta = np.linalg.lstsq(X_b, y, rcond=None)[0]
        self.bias = theta[0]
        self.weights = theta[1:]
        return self

```

This is conceptually identical to linear regression from Lesson 02, but applied to time-lagged versions of the same variable.

Stationarity Check

The code computes rolling statistics to visually and numerically assess stationarity:

```
def check_stationarity(series, window=50):
    rolling_mean = np.array([
        series[max(0, i - window):i].mean()
        for i in range(1, len(series) + 1)
    ])
    rolling_std = np.array([
        series[max(0, i - window):i].std()
        for i in range(1, len(series) + 1)
    ])
    return rolling_mean, rolling_std
```

If the rolling mean drifts or the rolling std changes, the series is non-stationary. Apply differencing and check again.

The code also checks stationarity by comparing the first half and second half of the series. If the means differ by more than half a standard deviation or the variance ratio exceeds 2x, the series is flagged as non-stationary.

Autocorrelation

```
def autocorrelation(series, max_lag=20):
    n = len(series)
    mean = series.mean()
    var = series.var()
    acf = np.zeros(max_lag + 1)
    for k in range(max_lag + 1):
        cov = np.mean((series[:n-k] - mean) * (series[k:] - mean))
        acf[k] = cov / var if var > 0 else 0
    return acf
```

Use It

With sklearn, you use lag features directly with any regressor:

```
from sklearn.linear_model import Ridge
from sklearn.ensemble import GradientBoostingRegressor

X, y = make_lag_features(series, n_lags=10)

for train_idx, test_idx in walk_forward_split(len(X)):
    model = Ridge(alpha=1.0)
    model.fit(X[train_idx], y[train_idx])
    predictions = model.predict(X[test_idx])
```

For ARIMA, use statsmodels:

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(train_series, order=(5, 1, 2))
```

```
fitted = model.fit()
forecast = fitted.forecast(steps=30)
```

The code in `time_series.py` demonstrates both approaches and compares them using walk-forward validation.

sklearn TimeSeriesSplit

sklearn provides `TimeSeriesSplit` which implements walk-forward validation:

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for train_index, test_index in tscv.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
```

This is equivalent to our from-scratch `walk_forward_split` but integrated into sklearn's cross-validation framework. You can use it with `cross_val_score`:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=TimeSeriesSplit(n_splits=5))
print(f"Mean score: {scores.mean():.4f} +/- {scores.std():.4f}")
```

Evaluation Metrics

Time series forecasting uses regression metrics, but with time-aware context:

113. **MAE (Mean Absolute Error):** Average of $|y_{\text{true}} - y_{\text{pred}}|$. Easy to interpret in original units. "On average, predictions are off by 3.2 degrees."
114. **RMSE (Root Mean Squared Error):** Square root of mean squared error. Penalizes large errors more than MAE. Use when big errors are worse than many small errors.
115. **MAPE (Mean Absolute Percentage Error):** Average of $|error / true_value| * 100$. Scale-independent, useful for comparing across different series. But undefined when true values are zero.
116. **Naive baseline comparison:** Always compare against simple baselines. The seasonal naive baseline predicts the value from one period ago (yesterday, last week). If your model cannot beat naive, something is wrong.

Rolling Features

The code demonstrates adding rolling statistics (mean, std, min, max over windows of 7 and 14 days) to lag features. These give the model information about recent trends and volatility that lag features alone do not capture.

For example, if the rolling mean is rising, it suggests an upward trend. If the rolling std is increasing, it suggests growing volatility. These are the kinds of patterns that tree-based models can learn from but linear models cannot.

Baselines You Must Beat

Before building any model, establish baselines:

Last value (persistence). Predict that tomorrow will be the same as today. For many series, this is surprisingly hard to beat.

Seasonal naive. Predict that today will be the same as the same day last week (or last year). If your model cannot beat this, it has not learned any useful pattern beyond seasonality.

Moving average. Predict the average of the last k values. Smooths noise but cannot capture sudden changes.

If your fancy ML model loses to the seasonal naive baseline, you have a bug. Most commonly: future leakage in features, wrong evaluation method, or the series is truly random and unpredictable.

Practical Tips

Start with plotting. Before any modeling, plot the raw series. Look for trends, seasonality, outliers, structural breaks (sudden changes in behavior). A 30-second visual inspection often tells you more than an hour of automated analysis.

Difference first, model second. If the series has a clear trend, difference it before creating lag features. Tree-based models can handle trends, but linear models cannot, and differencing never hurts.

Hold out at least one full seasonal cycle. If you have weekly seasonality, your test set needs at least one full week. If monthly, at least one full month. Otherwise you cannot evaluate whether the model captured the seasonal pattern.

Monitor in production. Time series models degrade over time as the world changes. Track prediction errors on a rolling basis. When errors start increasing, retrain the model on recent data.

Beware of regime changes. A model trained on pre-pandemic data will not predict post-pandemic behavior. Include indicators of known regime changes as features, or use a sliding window that forgets old data.

Log-transform skewed series. Revenue, prices, and counts are often right-skewed. Taking the log stabilizes variance and makes multiplicative patterns additive, which linear models can handle. Forecast in log space, then exponentiate to get back to original units.

Exercises

40. **Stationarity experiment.** Generate a series with a linear trend. Check stationarity with rolling statistics. Apply first differencing. Check again. How many rounds of differencing does it take for a quadratic trend?
41. **Lag selection.** Compute ACF on a seasonal series (period=7). Which lags have the highest autocorrelation? Create lag features using only those lags (not consecutive lags). Does accuracy improve compared to using lags 1 through 7?
42. **Walk-forward vs random split.** Train a Ridge regression on lag features. Evaluate with random 80/20 split and with walk-forward validation. How much does the random split overestimate performance?
43. **Feature engineering.** Add rolling mean (window=7), rolling std (window=7), and day-of-week features to the lag features. Compare accuracy with and without these extras using walk-forward validation.
44. **Multi-step forecasting.** Modify the AR model to predict 5 steps ahead instead of 1. Compare two strategies: (a) predict one step, use the prediction as input for the next step (recursive), and (b) train separate models for each horizon (direct). Which is more accurate?

Key Terms

Term	What people say	What it actually means
Stationarity	“The stats don’t change over time”	A series whose mean, variance, and autocorrelation structure are constant over time
Differencing	“Subtract consecutive values”	Computing $y[t] - y[t-1]$ to remove trends and achieve stationarity
Autocorrelation (ACF)	“How a series correlates with itself”	The correlation between a time series and a lagged copy of itself, as a function of the lag
Partial autocorrelation (PACF)	“Direct correlation only”	Autocorrelation at lag k after removing the effect of all shorter lags
Lag features	“Past values as inputs”	Using $y[t-1]$, $y[t-2]$, ..., $y[t-k]$ as features to predict $y[t]$

Term	What people say	What it actually means
Walk-forward validation	“Time-respecting cross-validation”	Evaluation where training data always precedes test data chronologically
ARIMA	“The classic time series model”	AutoRegressive Integrated Moving Average: combines past values (AR), differencing (I), and past errors (MA)
Seasonality	“Repeating calendar patterns”	Regular, predictable cycles in a time series tied to calendar periods (daily, weekly, yearly)
Trend	“The long-term direction”	A persistent increase or decrease in the series level over time
Expanding window	“Use all history”	Walk-forward validation where the training set grows with each fold
Sliding window	“Fixed-size history”	Walk-forward validation where the training set is a fixed-length window that slides forward

Code Appendix: *time_series.py*

```

import numpy as np

def make_synthetic_series(n=500, seed=42):
    rng = np.random.RandomState(seed)
    t = np.arange(n, dtype=float)

    trend = 0.05 * t
    seasonality = 10 * np.sin(2 * np.pi * t / 30)
    noise = rng.normal(0, 2, n)
    series = 50 + trend + seasonality + noise

    return series

def make_seasonal_series(n=365, period=7, seed=42):
    rng = np.random.RandomState(seed)
    t = np.arange(n, dtype=float)

    trend = 0.02 * t
    weekly = 5 * np.sin(2 * np.pi * t / period)
    monthly = 3 * np.sin(2 * np.pi * t / 30)
    noise = rng.normal(0, 1.5, n)
    series = 100 + trend + weekly + monthly + noise

    return series

```

```

def difference(series, order=1):
    result = series.copy()
    for _ in range(order):
        result = result[1:] - result[:-1]
    return result

def check_stationarity(series, window=50):
    n = len(series)
    rolling_mean = np.zeros(n)
    rolling_std = np.zeros(n)

    for i in range(n):
        start = max(0, i - window + 1)
        segment = series[start:i + 1]
        rolling_mean[i] = segment.mean()
        rolling_std[i] = segment.std() if len(segment) > 1 else 0.0

    first_half_mean = series[:n // 2].mean()
    second_half_mean = series[n // 2:].mean()
    first_half_var = series[:n // 2].var()
    second_half_var = series[n // 2:].var()

    mean_shift = abs(first_half_mean - second_half_mean)
    var_ratio = max(first_half_var, second_half_var) / max(min(first_half_var, second_half_var), 1e-10)

    is_stationary = mean_shift < 0.5 * series.std() and var_ratio < 2.0

    return rolling_mean, rolling_std, is_stationary

def autocorrelation(series, max_lag=20):
    n = len(series)
    mean = series.mean()
    var = series.var()
    acf = np.zeros(max_lag + 1)

    for k in range(max_lag + 1):
        if k >= n:
            break
        cov = np.mean((series[:n - k] - mean) * (series[k:] - mean)) if k < n else 0
        acf[k] = cov / var if var > 0 else 0.0

    return acf

def make_lag_features(series, n_lags):
    n = len(series)
    X = np.full((n, n_lags), np.nan)

    for lag in range(1, n_lags + 1):

```

```

        X[lag:, lag - 1] = series[:-lag]

    valid_mask = ~np.isnan(X).any(axis=1)
    X_valid = X[valid_mask]
    y_valid = series[valid_mask]

    return X_valid, y_valid

def walk_forward_split(n_samples, n_splits=5, min_train=50):
    if n_samples <= min_train:
        return

    step = max(1, (n_samples - min_train) // n_splits)

    for i in range(n_splits):
        train_end = min_train + i * step
        test_start = train_end
        test_end = min(train_end + step, n_samples)

        if test_start >= n_samples:
            break

        yield slice(0, train_end), slice(test_start, test_end)

class SimpleAR:
    def __init__(self, n_lags=5):
        self.n_lags = n_lags
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        X_b = np.column_stack([np.ones(len(X)), X])
        theta = np.linalg.lstsq(X_b, y, rcond=None)[0]
        self.bias = theta[0]
        self.weights = theta[1:]
        return self

    def predict(self, X):
        return X @ self.weights + self.bias

    def fit_series(self, series):
        X, y = make_lag_features(series, self.n_lags)
        return self.fit(X, y)

    def forecast(self, last_values, n_steps):
        if len(last_values) < self.n_lags:
            raise ValueError(
                f"Need at least {self.n_lags} history points, got {len(last_values)}"
            )
        history = list(last_values[-self.n_lags:])
        predictions = []

```



```

        for _ in range(n_steps):
            features = np.array(history[-self.n_lags:]).reshape(1, -1)
            pred = self.predict(features)[0]
            predictions.append(pred)
            history.append(pred)

        return np.array(predictions)

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

def mape(y_true, y_pred):
    mask = y_true != 0
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def print_separator(title):
    print(f"\n{'=' * 60}")
    print(f" {title}")
    print(f"{'=' * 60}\n")

def demo_stationarity():
    print_separator("STATIONARITY CHECK")

    series = make_synthetic_series(n=300, seed=42)
    _, _, is_stat = check_stationarity(series)
    print(f"Original series (trend + seasonality):")
    print(f" Mean: {series.mean():.2f}, Std: {series.std():.2f}")
    print(f" Stationary: {is_stat}")

    diff1 = difference(series, order=1)
    _, _, is_stat1 = check_stationarity(diff1)
    print(f"\nAfter first differencing:")
    print(f" Mean: {diff1.mean():.4f}, Std: {diff1.std():.2f}")
    print(f" Stationary: {is_stat1}")

def demo_autocorrelation():
    print_separator("AUTOCORRELATION ANALYSIS")

    series = make_seasonal_series(n=365, period=7, seed=42)
    diff_series = difference(series, order=1)
    acf = autocorrelation(diff_series, max_lag=30)

```

```

print("ACF of differenced series (first 15 lags):")
print(f"{'Lag':>5} {'ACF':>8} {'Significance':>14}")
print(f"{'-' * 28}")
threshold = 1.96 / np.sqrt(len(diff_series))
for k in range(15):
    sig = "****" if abs(acf[k]) > threshold else ""
    bar = "#" * int(abs(acf[k]) * 30)
    print(f"{'k':>5} {'acf[k]:>8.4f} {'sig:>4} {'bar}")

print(f"\nSignificance threshold (95%): +/-{threshold:.4f}")
print(f"Lags 7 and 14 should show spikes (weekly seasonality)")

def demo_lag_features():
    print_separator("LAG FEATURES AND AR MODEL")

    series = make_synthetic_series(n=400, seed=42)
    n_lags = 10

    X, y = make_lag_features(series, n_lags)
    print(f"Series length: {len(series)}")
    print(f"Feature matrix: {X.shape} (samples x lag features)")
    print(f"Target vector: {y.shape}")

    print(f"\nFirst 3 samples:")
    for i in range(3):
        lags_str = ", ".join(f"{v:.1f}" for v in X[i, :5])
        print(f"  Lags: [{lags_str}, ...] -> Target: {y[i]:.1f}")

    ar = SimpleAR(n_lags=n_lags)
    ar.fit(X, y)

    print(f"\nAR({n_lags}) weights:")
    for i, w in enumerate(ar.weights):
        print(f"  Lag {i+1}: {w:+.4f}")
    print(f"  Bias: {ar.bias:+.4f}")

def demo_walk_forward():
    print_separator("WALK-FORWARD VALIDATION")

    series = make_synthetic_series(n=400, seed=42)
    n_lags = 10
    X, y = make_lag_features(series, n_lags)

    n_splits = 5
    fold_scores = []

    print(f"Walk-forward with {n_splits} splits:")

```

```

print(f"{'Fold':>6} {'Train':>10} {'Test':>10} {'MSE':>10} {'MAE':>10}")
print(f"{'-' * 48}")

for fold, (train_sl, test_sl) in enumerate(walk_forward_split(len(X), n_splits=n_splits, min_train=100)):
    X_train, y_train = X[train_sl], y[train_sl]
    X_test, y_test = X[test_sl], y[test_sl]

    ar = SimpleAR(n_lags=n_lags)
    ar.fit(X_train, y_train)
    y_pred = ar.predict(X_test)

    fold_mse = mse(y_test, y_pred)
    fold_mae = mae(y_test, y_pred)
    fold_scores.append(fold_mse)

    print(f"{'fold+1':>6} {'X_train.shape[0]':>10} {'X_test.shape[0]':>10} {'fold_mse':>10.4f} {'fold_mae':>10.4f}")

print(f"\nMean MSE: {np.mean(fold_scores):.4f}")
print(f"Std MSE: {np.std(fold_scores):.4f}")

def demo_random_vs_walk_forward():
    print_separator("RANDOM SPLIT vs WALK-FORWARD")

    series = make_synthetic_series(n=500, seed=42)
    n_lags = 10
    X, y = make_lag_features(series, n_lags)

    rng = np.random.RandomState(42)
    idx = rng.permutation(len(X))
    split = int(len(X) * 0.8)
    train_idx, test_idx = idx[:split], idx[split:]

    ar_random = SimpleAR(n_lags=n_lags)
    ar_random.fit(X[train_idx], y[train_idx])
    random_mse = mse(y[test_idx], ar_random.predict(X[test_idx]))

    wf_scores = []
    for train_sl, test_sl in walk_forward_split(len(X), n_splits=5, min_train=100):
        ar_wf = SimpleAR(n_lags=n_lags)
        ar_wf.fit(X[train_sl], y[train_sl])
        y_pred = ar_wf.predict(X[test_sl])
        wf_scores.append(mse(y[test_sl], y_pred))

    wf_mse = np.mean(wf_scores)

    print(f"Random 80/20 split MSE: {random_mse:.4f}")
    print(f"Walk-forward mean MSE: {wf_mse:.4f}")
    print(f"Ratio (random/wf): {random_mse / wf_mse:.4f}")

```

```

print()
if random_mse < wf_mse:
    print("Random split gives lower MSE -- this is the optimistic bias from future
leakage.")
    print("The walk-forward score is the honest estimate of production performance.
")
else:
    print("Walk-forward gives similar or lower MSE -- the series may be stationary
enough")
    print("that future leakage is not a major factor here.")

def demo_lag_comparison():
    print_separator("LAG COUNT COMPARISON")

    series = make_seasonal_series(n=365, period=7, seed=42)

    print(f"{'n_lags':>8} {'Mean MSE':>12} {'Mean MAE':>12}")
    print(f"{'-' * 34}")

    for n_lags in [1, 3, 5, 7, 10, 14, 21, 30]:
        X, y = make_lag_features(series, n_lags)

        scores_mse = []
        scores_mae = []

        for train_sl, test_sl in walk_forward_split(len(X), n_splits=5, min_train=max(6
0, n_lags + 20)):
            ar = SimpleAR(n_lags=n_lags)
            ar.fit(X[train_sl], y[train_sl])
            y_pred = ar.predict(X[test_sl])
            scores_mse.append(mse(y[test_sl], y_pred))
            scores_mae.append(mae(y[test_sl], y_pred))

        if scores_mse:
            print(f"{'n_lags':>8} {np.mean(scores_mse):>12.4f} {np.mean(scores_mae):>
12.4f}")

def demo_forecasting():
    print_separator("MULTI-STEP FORECASTING")

    series = make_synthetic_series(n=300, seed=42)
    train_series = series[:250]
    true_future = series[250:270]

    n_lags = 10
    ar = SimpleAR(n_lags=n_lags)
    X, y = make_lag_features(train_series, n_lags)
    ar.fit(X, y)

    forecast = ar.forecast(train_series, n_steps=20)

```

```

    print(f"Training on {len(train_series)} points, forecasting {len(true_future)} steps ahead")
    print()
    print(f"{'Step':>6} {'True':>10} {'Predicted':>10} {'Error':>10}")
    print(f"{'-' * 38}")

    for i in range(len(true_future)):
        error = true_future[i] - forecast[i]
        print(f"{i+1:>6} {true_future[i]:>10.2f} {forecast[i]:>10.2f} {error:>+10.2f}")

    print(f"\nForecast MSE: {mse(true_future, forecast):.4f}")
    print(f"Forecast MAE: {mae(true_future, forecast):.4f}")
    print(f"Forecast MAPE: {mape(true_future, forecast):.2f}%")

if __name__ == "__main__":
    demo_stationarity()
    demo_autocorrelation()
    demo_lag_features()
    demo_walk_forward()
    demo_random_vs_walk_forward()
    demo_lag_comparison()
    demo_forecasting()

```

You are an expert in time series analysis and forecasting. When someone describes a prediction problem involving temporal data, help them frame it correctly and choose the right approach.

Step 1: Understand the Problem

Ask these questions:

What is the target? A single numeric value (regression) or a category (classification)?

What is the forecast horizon? Next hour, next day, next month, next year?

How many time series? One (univariate), a few (multivariate), or thousands (many-series)?

Are there external features? Holidays, promotions, weather, economic indicators?

What is the frequency? Minute, hourly, daily, weekly, monthly?

How much history? Months, years, decades?

Step 2: Check for Common Pitfalls

Before recommending a model, verify:

No random train/test split. Time series must use chronological splits. Walk-forward validation is the standard.

No future features. If a feature is not available at prediction time, it cannot be used. Example: using today’s closing price to predict today’s closing price.

Stationarity check. If the mean or variance drifts over time, either difference the series or use a model that handles non-stationarity (tree-based models, or ARIMA with $d > 0$).

Seasonality identification. Check ACF for spikes at regular intervals. If present, include seasonal features or use a seasonal model.

Scale of target. Percentage errors (MAPE) matter more for business metrics. Absolute errors (MAE, MSE) are easier to optimize.

Step 3: Recommend an Approach

Situation	Recommended Approach
Simple univariate, short history	Exponential smoothing or ARIMA
Univariate with strong seasonality	SARIMA or Prophet
Many external features available	Lag features + gradient boosting (XGBoost, LightGBM)
Hundreds of related series	LightGBM with series ID as feature, or global neural model
Very long sequences, complex patterns	LSTM or Temporal Fusion Transformer
Quick baseline needed	Seasonal naive (predict same value from one period ago)

Step 4: Feature Engineering Checklist

For lag-feature-based approaches:

- ☐ Lag values ($t-1$, $t-2$, ..., $t-k$), where k is guided by ACF
- ☐ Rolling statistics (mean, std, min, max over recent windows)
- ☐ Differenced values (change from previous step)
- ☐ Calendar features (day of week, month, quarter, is_holiday)
- ☐ Expanding features (cumulative mean, running count)
- ☐ External features aligned by timestamp

Step 5: Evaluation Protocol

Always use walk-forward (expanding or sliding window) cross-validation.

Metrics to report: - **MAE** (Mean Absolute Error) – interpretable in original units - **MAPE** (Mean Absolute Percentage Error) – relative, comparable across scales - **RMSE** (Root Mean Squared Error) – penalizes

large errors more - **Baseline comparison** – always compare against seasonal naive and simple moving average

Red flags in results: - Model is worse than naive baseline: feature leakage or wrong evaluation - Random split gives much better results than walk-forward: future leakage - Performance degrades sharply at longer horizons: model relies on short-term autocorrelation only

Quiz

Pre-lesson check

117. Why is a random train/test split invalid for time series data?

- Random splits make the dataset too small
- Random splits leak future information into the training set, allowing the model to cheat **(correct)**
- Time series data cannot be split at all
- Random splits only work for classification problems
Explanation: In a random split, future data points can end up in the training set while past points are in the test set. The model then uses future information to predict the past, giving falsely optimistic results.

118. What does it mean for a time series to be stationary?

- The values never change over time
- Its statistical properties (mean, variance, autocorrelation) do not change over time **(correct)**
- It has no seasonal patterns
- It always trends upward
Explanation: A stationary series has constant mean, variance, and autocorrelation structure over time. Most forecasting methods assume stationarity. Non-stationary series need differencing or detrending first.

Post-lesson check

119. What is the purpose of differencing a time series?

- To increase the number of data points
 - To remove trend and make the series stationary by modeling changes between consecutive values **(correct)**
 - To convert regression into classification
 - To normalize the values between 0 and 1
- Explanation: Differencing replaces each value with the change from the previous value: $\text{diff}[t] = \text{value}[t] - \text{value}[t-1]$. This removes trend, making the series closer to stationary.*

120. Lag features convert a time series into a supervised learning problem. What is a lag-3 feature for predicting $y[t]$?

1. The average of the next 3 values: $(y[t+1] + y[t+2] + y[t+3]) / 3$
 2. The value 3 time steps ago: $y[t-3]$ **(correct)**
 3. The 3rd derivative of the series
 4. The difference between the current value and 3 steps ahead
- Explanation: A lag-3 feature is $y[t-3]$: the value 3 time steps in the past. Using past values as input features lets standard ML models (regression, trees) make time series predictions without specialized algorithms.*

121. Walk-forward validation splits time data into expanding or sliding windows. Why is this better than K-fold CV for time series?

- Walk-forward is faster to compute
 - Walk-forward respects temporal order, training only on past data and testing on future data, preventing lookahead bias **(correct)**
 - K-fold CV cannot be used on numeric data
 - Walk-forward uses more data for training
- Explanation: Walk-forward validation always trains on past data and tests on future*

data, mimicking real-world deployment. K-fold CV shuffles data, potentially training on future to predict past (temporal leakage).

Anomaly Detection

Normal is easy to define. Abnormal is whatever doesn't fit.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 **Time:** ~75 minutes

Learning Objectives

- Implement Z-score, IQR, and Isolation Forest anomaly detection methods from scratch
- Distinguish between point, contextual, and collective anomalies and select the appropriate detection method for each
- Explain why anomaly detection is framed as modeling normal data rather than classifying anomalies
- Compare unsupervised anomaly detection with supervised classification and evaluate the tradeoff between novel anomaly coverage and precision

The Problem

A credit card is used in New York at 2pm, then in Tokyo at 2:05pm. A factory sensor reads 150 degrees when the normal range is 80-120. A server sends 50,000 requests per second when the daily average is 200.

These are anomalies. Finding them matters. Fraud costs billions. Equipment failures cost downtime. Network intrusions cost data.

The challenge: you rarely have labeled examples of anomalies. Fraud makes up 0.1% of transactions. Equipment failures happen a few times per year. You cannot train a standard classifier because there is almost nothing in the “anomaly” class to learn from. Even if you have some labels, the anomalies you have seen are not the only types you will encounter. Tomorrow’s fraud scheme looks different from today’s.

Anomaly detection flips the problem. Instead of learning what is abnormal, learn what is normal. Anything that deviates from normal is suspicious. This works without labels, adapts to new types of anomalies, and scales to massive datasets.

The Concept

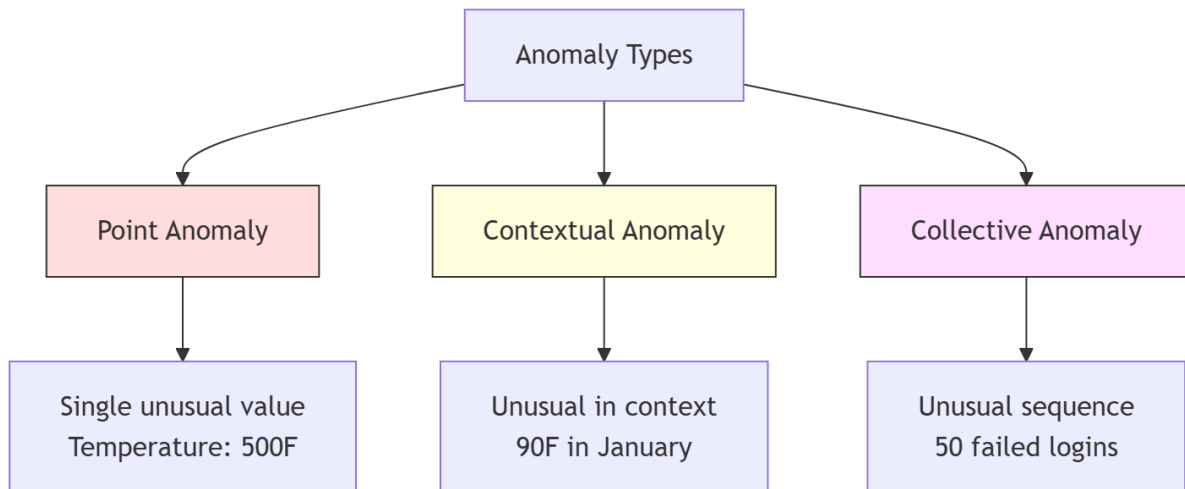
Types of Anomalies

Not all anomalies are the same:

- **Point anomalies.** A single data point that is unusual regardless of context. A temperature reading of 500 degrees. A transaction of \$50,000 from an account that normally spends \$50.
- **Contextual anomalies.** A data point that is unusual given its context. A temperature of 90 degrees is normal in summer, anomalous in winter. Same value, different context.

- **Collective anomalies.** A sequence of data points that is unusual as a group, even though each individual point might be normal. Five login failures is normal. Fifty in a row is a brute-force attack.

Most methods detect point anomalies. Contextual anomalies need time or location features. Collective anomalies need sequence-aware methods.



The Unsupervised Framing

In standard classification, you have labels for both classes. In anomaly detection, you typically have one of three situations:

- **Fully unsupervised.** No labels at all. You fit the detector on all data and hope anomalies are rare enough not to corrupt the “normal” model.
- **Semi-supervised.** You have a clean dataset of normal data only. You fit on this clean set and score everything else. This is the strongest setup when possible.
- **Weakly supervised.** You have a few labeled anomalies. Use them for evaluation, not training. Train unsupervised, then measure precision/recall on the labeled subset.

The key insight: anomaly detection is fundamentally different from classification. You are modeling the distribution of normal data, not the decision boundary between two classes.

Supervised vs Unsupervised: The Tradeoff

If you do have labeled anomalies, should you use them for training (supervised classification) or for evaluation only (unsupervised detection)?

Supervised (treat as classification): - Catches the exact types of anomalies you have seen before - Higher precision on known anomaly types - Misses novel anomaly types entirely - Requires retraining when new anomaly types emerge - Needs enough anomaly examples (often too few)

Unsupervised (model normal, flag deviations): - Catches any deviation from normal, including novel types - Does not require labeled anomalies - Higher false positive rate (not everything unusual is bad) - More robust to distribution shift

In practice, the best systems combine both: unsupervised detection for broad coverage, supervised models for known high-priority anomaly types, and human review for ambiguous cases.

Z-Score Method

The simplest approach. Compute the mean and standard deviation of each feature. Flag any point more than k standard deviations from the mean.

```
z_score = (x - mean) / std
anomaly if |z_score| > threshold
```

The default threshold is 3.0 (99.7% of normal data falls within 3 standard deviations for a Gaussian distribution).

Strengths: Simple. Fast. Interpretable (“this value is 4.5 standard deviations from normal”).

Weaknesses: Assumes data is normally distributed. Sensitive to outliers in the training data (the outliers shift the mean and inflate the std, making them harder to detect). Fails on multimodal distributions.

When it works well: Single-feature monitoring where data is roughly bell-shaped. Server response times, manufacturing tolerances, sensor readings with stable baselines.

When it fails: Multi-cluster data (two office locations with different baseline temperatures), skewed data (transaction amounts where \$1000 is rare but not anomalous), data with outliers in the training set.

IQR Method

More robust than Z-score. Uses the interquartile range instead of mean and standard deviation.

```
Q1 = 25th percentile
Q3 = 75th percentile
IQR = Q3 - Q1
lower_bound = Q1 - factor * IQR
upper_bound = Q3 + factor * IQR
anomaly if x < lower_bound or x > upper_bound
```

The default factor is 1.5.

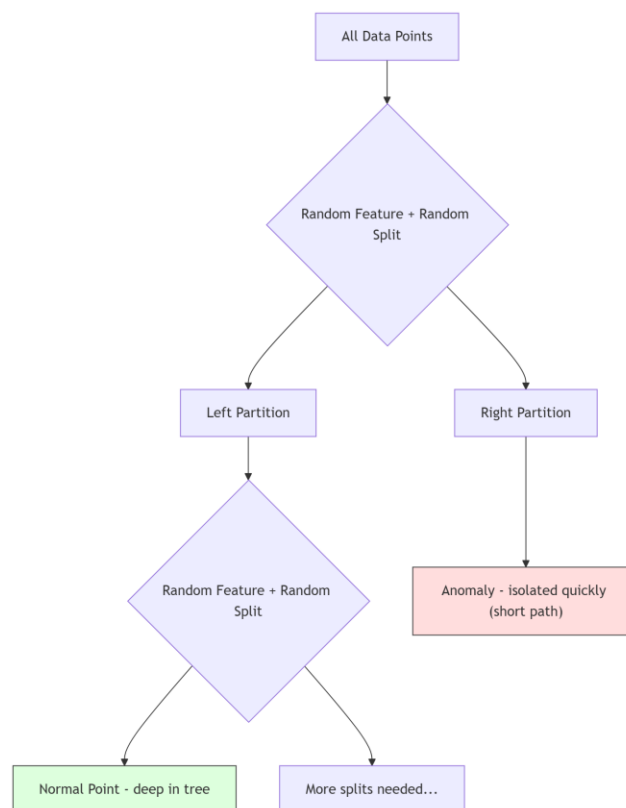
Strengths: Robust to outliers (percentiles are not affected by extreme values). Works on skewed distributions. No normality assumption.

Weaknesses: Univariate only (applies per feature independently). Cannot detect anomalies that are unusual only when features are considered together (a point might be normal in each feature individually but anomalous in the joint space).

Practical note: The 1.5 factor in IQR corresponds to the whiskers in a box plot. Points outside the whiskers are potential outliers. Using 3.0 instead of 1.5 makes the detector more conservative (fewer flags, fewer false positives). The right factor depends on your tolerance for false alarms.

Isolation Forest

The key insight: anomalies are few and different. In a random partitioning of the data, anomalies are easier to isolate – they need fewer random splits to be separated from the rest.



How it works: 1. Build many random trees (an isolation forest) 2. At each node, pick a random feature and a random split value between the feature's min and max 3. Keep splitting until every point is isolated (in its own leaf) 4. Anomalies have shorter average path lengths across all trees

Why it works: Normal points live in dense regions. Many random splits are needed to isolate one from its neighbors. Anomalies live in sparse regions. One or two random splits are enough to isolate them.

The anomaly score is based on the average path length across all trees, normalized by the expected path length of a random binary search tree:

$$\text{score}(x) = 2^{(-\text{average_path_length}(x) / c(n))}$$

Where $c(n)$ is the expected path length for n samples. Score near 1 means anomaly. Score near 0.5 means normal. Score near 0 means very normal (deep in dense clusters).

Strengths: No distribution assumptions. Works in high dimensions. Scales well (sublinear in sample size because each tree uses a subsample). Handles mixed feature types.

Weaknesses: Struggles with anomalies in dense regions (masking effect). Random splitting is less effective when many features are irrelevant.

Key hyperparameters: - `n_estimators`: Number of trees. 100 is usually enough. More trees give more stable scores but slower computation. - `max_samples`: Number of samples per tree. 256 is the default in the original paper. Smaller values make individual trees less accurate but increase diversity. The subsampling is what makes Isolation Forest fast – each tree sees a small fraction of the data. - `contamination`: Expected fraction of anomalies. Used only for setting the threshold. Does not affect the scores themselves.

Local Outlier Factor (LOF)

LOF compares the local density around a point to the density around its neighbors. A point in a sparse region surrounded by dense regions is anomalous.

How it works: 1. For each point, find its k nearest neighbors 2. Compute the local reachability density (how dense is the neighborhood) 3. Compare each point's density to its neighbors' densities 4. If a point has much lower density than its neighbors, it is an outlier

LOF score: - LOF close to 1.0 means similar density as neighbors (normal) - LOF greater than 1.0 means lower density than neighbors (potentially anomalous) - LOF much greater than 1.0 (e.g., 2.0+) means significantly lower density (likely anomaly)

The “local” part is critical. Consider a dataset with two clusters: a dense cluster of 1000 points and a sparse cluster of 50 points. A point on the edge of the sparse cluster is not globally unusual – it has 50 neighbors. But it is locally unusual if its immediate neighbors are denser than it is. LOF captures this nuance that global methods miss.

Strengths: Detects local anomalies (points that are unusual in their neighborhood, even if they are not globally unusual). Works on clusters of different densities.

Weaknesses: Slow on large datasets ($O(n^2)$ for naive implementation). Sensitive to the choice of k . Does not work well in very high dimensions (curse of dimensionality affects distance calculations).

Comparison

Method	Assumptions	Speed	Handles High Dims	Detects Local Anomalies
Z-score	Normal distribution	Very fast	Yes (per feature)	No
IQR	None (per feature)	Very fast	Yes (per feature)	No
Isolation Forest	None	Fast	Yes	Partially
LOF	Distance is meaningful	Slow	Poorly	Yes

Evaluation Challenges

Evaluating anomaly detectors is harder than evaluating classifiers:

- **Extreme class imbalance.** With 0.1% anomalies, predicting “normal” for everything gives 99.9% accuracy. Accuracy is useless.
- **AUROC is misleading.** With heavy imbalance, AUROC can look good even when the model misses most anomalies at practical thresholds.
- **Better metrics:** Precision@k (of the top k flagged items, how many are real anomalies), AUPRC (area under precision-recall curve), and recall at a fixed false positive rate.



Anomaly Detection Pipeline

In practice, anomaly detection follows this workflow:

- **Collect baseline data.** Ideally, a period where you know there are no (or very few) anomalies.
- **Feature engineering.** Raw features plus derived features (rolling statistics, time features, ratios).
- **Train the detector.** Fit on the baseline data. The model learns what “normal” looks like.
- **Score new data.** Each new observation gets an anomaly score.
- **Threshold selection.** Choose the score cutoff. This is a business decision: higher threshold means fewer false alarms but more missed anomalies.
- **Alert and investigate.** Flagged points go to human review or automated response.
- **Feedback collection.** Record whether flagged items were true anomalies or false alarms. Use this data to evaluate the detector and tune the threshold over time.

The pipeline is never “done.” Data distributions shift, new anomaly types emerge, and thresholds need adjustment. Treat anomaly detection as a living system, not a one-time model.

Build It

The code in `code/anomaly_detection.py` implements Z-score, IQR, and Isolation Forest from scratch.

Z-Score Detector

```
def zscore_detect(X, threshold=3.0):
    mean = X.mean(axis=0)
    std = X.std(axis=0)
    std[std == 0] = 1.0
    z = np.abs((X - mean) / std)
    return z.max(axis=1) > threshold
```

Simple and vectorized. Flags a point if any feature exceeds the threshold.

IQR Detector

```
def iqr_detect(X, factor=1.5):
    q1 = np.percentile(X, 25, axis=0)
    q3 = np.percentile(X, 75, axis=0)
    iqr = q3 - q1
    iqr[iqr == 0] = 1.0
    lower = q1 - factor * iqr
    upper = q3 + factor * iqr
    outside = (X < lower) | (X > upper)
    return outside.any(axis=1)
```

Isolation Forest from Scratch

The from-scratch implementation builds isolation trees that randomly partition the feature space:

```
class IsolationTree:
    def __init__(self, max_depth):
        self.max_depth = max_depth

    def fit(self, X, depth=0):
        n, p = X.shape
        if depth >= self.max_depth or n <= 1:
            self.is_leaf = True
            self.size = n
            return self
        self.is_leaf = False
        self.feature = np.random.randint(p)
        x_min = X[:, self.feature].min()
        x_max = X[:, self.feature].max()
        if x_min == x_max:
            self.is_leaf = True
            self.size = n
            return self
```



```

self.threshold = np.random.uniform(x_min, x_max)
left_mask = X[:, self.feature] < self.threshold
self.left = IsolationTree(self.max_depth).fit(X[left_mask], depth + 1)
self.right = IsolationTree(self.max_depth).fit(X[~left_mask], depth + 1)
return self

```

The path length to isolate a point determines its anomaly score. Shorter paths mean more anomalous.

The `IsolationForest` class wraps multiple trees:

```

class IsolationForest:
    def __init__(self, n_estimators=100, max_samples=256, seed=42):
        self.n_estimators = n_estimators
        self.max_samples = max_samples

    def fit(self, X):
        sample_size = min(self.max_samples, X.shape[0])
        max_depth = int(np.ceil(np.log2(sample_size)))
        for _ in range(self.n_estimators):
            idx = rng.choice(X.shape[0], size=sample_size, replace=False)
            tree = IsolationTree(max_depth=max_depth)
            tree.fit(X[idx])
            self.trees.append(tree)

    def anomaly_score(self, X):
        avg_path = average path length across all trees
        scores = 2.0 ** (-avg_path / c(max_samples))
        return scores

```

The normalization factor $c(n)$ is the expected path length of an unsuccessful search in a binary search tree with n elements. It equals $2 * H(n-1) - 2*(n-1)/n$ where H is the harmonic number. This normalization ensures scores are comparable across datasets of different sizes.

Demo Scenarios

The code generates multiple test scenarios:

122. **Single cluster with outliers.** A 2D Gaussian cluster with anomalies injected far from the center. All methods should work here.
123. **Multimodal data.** Three clusters of different sizes and densities. Points between clusters are anomalous. Z-score struggles because the per-feature ranges are wide.
124. **High-dimensional data.** 50 features, but anomalies differ in only 5 of them. Tests whether methods can find anomalies in a subset of features.

Each demo compares all methods using precision, recall, F1, and Precision@k.

Use It

With sklearn (using library implementations, not from-scratch):

```

from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

iso = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
iso.fit(X_train)
predictions = iso.predict(X_test)

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05, novelty=True)
lof.fit(X_train)
predictions = lof.predict(X_test)

```

Note `contamination` sets the expected fraction of anomalies. Setting it correctly matters – too low misses anomalies, too high creates false alarms.

The code in `anomaly_detection.py` compares from-scratch implementations against sklearn on the same data.

sklearn Contamination Parameter

The `contamination` parameter in sklearn determines the threshold for converting continuous anomaly scores into binary predictions. It does not change the underlying scores.

```

iso_5 = IsolationForest(contamination=0.05)
iso_10 = IsolationForest(contamination=0.10)

```

Both produce the same anomaly scores. But `iso_5` flags the top 5% while `iso_10` flags the top 10%. If you do not know the true anomaly rate (you usually do not), set `contamination` to “auto” and work with the raw scores directly. Set your own threshold based on the cost tradeoff between false positives and false negatives.

One-Class SVM

Another unsupervised anomaly detector worth knowing. One-Class SVM fits a boundary around normal data in a high-dimensional feature space (using the kernel trick).

```

from sklearn.svm import OneClassSVM

oc_svm = OneClassSVM(kernel="rbf", gamma="auto", nu=0.05)
oc_svm.fit(X_train)
predictions = oc_svm.predict(X_test)

```

The `nu` parameter approximates the fraction of anomalies. One-Class SVM works well on small to medium datasets but does not scale to very large data (the kernel matrix grows quadratically).

Autoencoder Approach (Preview)

Autoencoders are neural networks that learn to compress and reconstruct data. Train on normal data. At test time, anomalies have high reconstruction error because the network learned to reconstruct normal patterns only.

This is covered in Phase 3 (Deep Learning), but the principle is the same: model what is normal, flag what deviates.

Ensemble Anomaly Detection

Just as ensemble methods improve classification (Lesson 11), combining multiple anomaly detectors improves detection. The simplest approach:

- Run multiple detectors (Z-score, IQR, Isolation Forest, LOF)

- Normalize each detector's scores to [0, 1]

- Average the normalized scores

- Flag points above the threshold on the average score

This reduces false positives because different methods have different failure modes. A point flagged by all four methods is almost certainly anomalous. A point flagged by only one might be a quirk of that method.

More sophisticated ensembles weight each detector by its estimated reliability (measured on a validation set with known anomalies, if available).

Production Considerations

Threshold drift. As data distribution shifts, a fixed threshold becomes outdated. Monitor the distribution of anomaly scores and adjust periodically.

Alert fatigue. Too many false alarms and operators stop paying attention. Start with a high threshold (fewer, more reliable alerts) and lower it as trust builds.

Ensemble approach. In production, combine multiple detectors. Flag a point only if multiple methods agree it is anomalous. This reduces false positives significantly.

Feature engineering. Raw features are rarely enough. Add rolling statistics, ratios, time-since-last-event, and domain-specific features. A good feature set matters more than the choice of detector.

Feedback loop. When operators investigate flagged items and confirm or dismiss them, feed this back into the system. Accumulate labeled data over time to evaluate and improve the detector.

Choosing a Threshold

The anomaly score is continuous. You need a threshold to make binary decisions. This is a business decision, not a technical one.

Consider two scenarios: - **Fraud detection.** Missing fraud is expensive (chargebacks, customer trust). False alarms cost a human analyst 5 minutes to investigate. Set the threshold low to catch more fraud, accept more false alarms. - **Equipment maintenance.** A false alarm means an unnecessary shutdown costing \$50,000. A missed failure means a \$500,000 repair. Set the threshold to balance these costs.

In both cases, the optimal threshold depends on the cost ratio between false positives and false negatives. Plot precision and recall at different thresholds, overlay the cost function, and pick the minimum-cost point.

Scaling to Production

For real-time anomaly detection in production:

45. **Batch training, online scoring.** Train the model periodically (daily, weekly) on recent normal data. Score each new observation as it arrives.
46. **Feature computation must match.** If you trained with rolling statistics over 30 days, you need 30 days of history to compute features for a new observation. Buffer the required history.
47. **Score distribution monitoring.** Track the distribution of anomaly scores over time. If the median score drifts upward, either the data is changing or the model is stale.
48. **Explainability.** When you flag an anomaly, say why. Z-score: “Feature X is 4.2 standard deviations above normal.” Isolation Forest: “This point was isolated in 3.1 splits on average (normal points take 8.5).”

Exercises

Threshold tuning. Run the Z-score detector with thresholds from 1.0 to 5.0 in steps of 0.5. Plot precision and recall at each threshold. Where is the sweet spot for your data?

Multivariate anomalies. Create 2D data where each feature individually looks normal, but the combination is anomalous (e.g., points far from the main cluster diagonal). Show that Z-score per feature misses these but Isolation Forest catches them.

LOF from scratch. Implement Local Outlier Factor using k-nearest neighbors. Compare against sklearn’s LocalOutlierFactor on the same data. Use $k=10$ and $k=50$ – how does the choice of k affect results?

Streaming anomaly detection. Modify the Z-score detector to work in a streaming setting: update the running mean and variance as new points arrive (Welford’s online algorithm). Compare to batch Z-score on the same data.

Real-world evaluation. Take a dataset with known anomalies (credit card fraud from Kaggle, for example). Evaluate all four methods using precision@100, precision@500, and AUPRC. Which method works best? Why?

Key Terms

Term	What people say	What it actually means
Anomaly	"Outlier, unusual point"	A data point that deviates significantly from the expected pattern of normal data
Point anomaly	"A single weird value"	An individual observation that is unusual regardless of context
Contextual anomaly	"Normal value, wrong context"	An observation that is unusual given its context (time, location, etc.) but might be normal in another context
Isolation Forest	"Random splits to find outliers"	An ensemble of random trees that isolates anomalies with fewer splits than normal points
Local Outlier Factor	"Compare density to neighbors"	A method that flags points whose local density is much lower than their neighbors' density
Z-score	"Standard deviations from mean"	$(x - \text{mean}) / \text{std}$, measuring how far a point is from the center in units of standard deviation
IQR	"Interquartile range"	$Q3 - Q1$, measuring the spread of the middle 50% of data, used for robust outlier detection
Contamination	"Expected fraction of anomalies"	A hyperparameter telling the detector what proportion of the data it should flag as anomalous
Precision@k	"Of the top k flags, how many are real"	Precision computed on only the k most suspicious points, useful for imbalanced anomaly detection
AUPRC	"Area under precision-recall curve"	A metric that summarizes precision-recall performance across all thresholds, better than AUROC for imbalanced data

You are an expert in anomaly detection. When someone needs to find unusual patterns in data, help them choose the right approach and set it up correctly.

Decision Framework

Step 1: What kind of anomalies?

Point anomalies (single unusual values) -> Z-score, IQR, Isolation Forest, or LOF

Contextual anomalies (unusual given context like time) -> Add context features, then use any method

Collective anomalies (unusual sequences) -> Sliding window features + any method, or sequence models

Step 2: Do you have labels?

- **No labels at all** -> Unsupervised: Isolation Forest, LOF, Z-score, IQR, autoencoders
- **Some labels (few anomaly examples)** -> Semi-supervised: train on normal data only, test on everything
- **Many labels** -> Supervised: treat as imbalanced classification (but the anomaly types you trained on are the only ones you will catch)

Step 3: What are your constraints?

Constraint	Best Method
Must explain why it is anomalous	Z-score (which feature, how many stds) or IQR (which feature, how far from bounds)
Very high-dimensional data (50+ features)	Isolation Forest (handles irrelevant features)
Multiple clusters of different densities	LOF (local density comparison)
Real-time, single-pass processing	Z-score with running statistics (Welford's algorithm)
Large dataset (millions of rows)	Isolation Forest (subsamples) or Z-score ($O(n)$)
Must minimize false alarms	Higher thresholds, tune on precision, use ensemble of methods

Step 4: How to evaluate

125. Do NOT use accuracy. With 0.1% anomalies, always predicting "normal" gives 99.9% accuracy.
126. Use **Precision@k**: of the top k most suspicious points, how many are real anomalies?
127. Use **AUPRC**: area under the precision-recall curve.
128. Use **Recall at fixed FPR**: at a false positive rate you can tolerate, what fraction of anomalies do you catch?
129. Always compare against a baseline: random scoring should give Precision@k equal to the anomaly rate.

Step 5: Common Mistakes

130. **Training on contaminated data.** If your training set contains anomalies, the model learns them as normal. Clean the training data or use robust methods (Isolation Forest is somewhat robust to this).
131. **Using AUROC with extreme imbalance.** AUROC can be 0.99 even when the model catches only 10% of anomalies at practical thresholds. Use AUPRC instead.

132. **Ignoring temporal context.** A CPU usage of 90% is normal during deployment, anomalous at 3am. Add time features.
133. **Fixed thresholds in production.** The data distribution drifts. A threshold that works today may not work next month. Monitor the score distribution and adjust.
134. **Univariate detection on multivariate data.** Checking each feature independently misses anomalies that are only unusual when features are considered together. Use Isolation Forest or LOF for multivariate detection.

Quick Reference

Method	Speed	Interpretability	Multivariate	Robust to Outliers in Training
Z-score	Very fast	High	Per-feature only	No
IQR	Very fast	High	Per-feature only	Somewhat
Isolation Forest	Fast	Low	Yes	Somewhat
LOF	Slow	Medium	Yes	No
Autoencoder	Medium	Low	Yes	No
One-Class SVM	Medium	Low	Yes	No

Quiz

Pre-lesson check

- Why is anomaly detection typically framed as an unsupervised problem rather than classification?
 1. Anomaly detection does not require any data
 2. Labeled anomalies are extremely rare, and novel anomaly types differ from previously seen ones (**correct**)
 3. Supervised classification is always less accurate
 4. Anomaly detection only works on time series data
Explanation: Anomalies are rare (often <0.1% of data), so there are too few labeled examples to train a classifier. Also, future anomalies may be of types never seen before. Modeling 'normal' and flagging deviations handles both problems.

- A temperature of 90F is normal in summer but anomalous in winter. What type of anomaly is this?
 - Point anomaly
 - Contextual anomaly **(correct)**
 - Collective anomaly
 - Statistical anomaly

Explanation: A contextual anomaly is a value that is unusual given its context (time, location). The value itself might be normal in a different context. Same data point, different interpretation based on surrounding conditions.

Post-lesson check

135. The Z-score method flags points more than 3 standard deviations from the mean. When does this approach fail?

- e) When the data is perfectly normally distributed
 - f) When the data is multimodal, skewed, or when outliers in the training data inflate the mean and std **(correct)**
 - g) When there are exactly 3 anomalies in the dataset
 - h) When the features are standardized
- Explanation: Z-score assumes a single Gaussian distribution. It fails on multimodal data (multiple clusters), skewed distributions, and when outliers in training data shift the mean/std, making real anomalies harder to detect.*

136. How does Isolation Forest detect anomalies differently from distance-based methods?

- i) It uses neural networks instead of trees
- j) It isolates points using random splits; anomalies require fewer splits to isolate because they are few and different **(correct)**
- k) It computes distances to every other point in the dataset

- l) It only works on text data

Explanation: Isolation Forest randomly partitions data with tree splits. Anomalies, being rare and different, end up isolated (in their own leaf) with fewer splits. Short average path length = more anomalous.

137. You build both an unsupervised anomaly detector and a supervised fraud classifier. When should you prefer the unsupervised approach?

- 3. Always – unsupervised is always better for anomaly detection
- 4. When you need to detect novel fraud patterns that differ from historical labeled examples **(correct)**
- 5. When you have millions of labeled fraud examples

- 6. When you only care about precision, not recall

Explanation: Supervised classifiers only catch fraud types present in training data.

Unsupervised detectors flag any deviation from normal, catching novel fraud schemes.

The tradeoff is higher false positive rate.

Handling Imbalanced Data

When 99% of your data is “normal,” accuracy is a lie.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 (especially evaluation metrics) **Time:** ~90 minutes

Learning Objectives

Implement SMOTE from scratch and explain how synthetic oversampling differs from random duplication

Evaluate imbalanced classifiers using F1, AUPRC, and Matthews Correlation Coefficient instead of accuracy

Compare class weighting, threshold tuning, and resampling strategies and select the right approach for a given imbalance ratio

Build a complete imbalanced data pipeline that combines SMOTE, class weights, and threshold optimization

The Problem

You build a fraud detection model. It gets 99.9% accuracy. You celebrate. Then you realize it predicts “not fraud” for every single transaction.

This is not a bug. It is the rational thing to do when only 0.1% of transactions are fraudulent. The model learns that always guessing the majority class minimizes overall error. It is technically correct and completely useless.

This happens everywhere real classification matters. Disease diagnosis: 1% positive rate. Network intrusion: 0.01% attacks. Manufacturing defects: 0.5% defective. Spam filtering: 20% spam. Churn prediction: 5% churners. The more consequential the minority class, the rarer it tends to be.

Accuracy fails because it treats all correct predictions equally. Correctly labeling a legitimate transaction and correctly catching fraud both count as one point of accuracy. But catching fraud is the entire reason the model exists. We need metrics, techniques, and training strategies that force the model to pay attention to the rare but important class.

The Concept

Why Accuracy Fails

Consider a dataset with 1000 samples: 990 negative, 10 positive. A model that always predicts negative:

	Predicted Positive	Predicted Negative
Actually Positive	0 (TP)	10 (FN)
Actually Negative	0 (FP)	990 (TN)

Accuracy = $(0 + 990) / 1000 = 99.0\%$

The model catches zero fraud. Zero disease. Zero defects. But accuracy says 99%. This is why accuracy is dangerous for imbalanced problems.

Better Metrics

Precision = $TP / (TP + FP)$. Of everything flagged as positive, how many actually are? High precision means few false alarms.

Recall = $TP / (TP + FN)$. Of everything actually positive, how many did we catch? High recall means few missed positives.

F1 Score = $2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$. The harmonic mean. Penalizes extreme imbalance between precision and recall more than the arithmetic mean would.

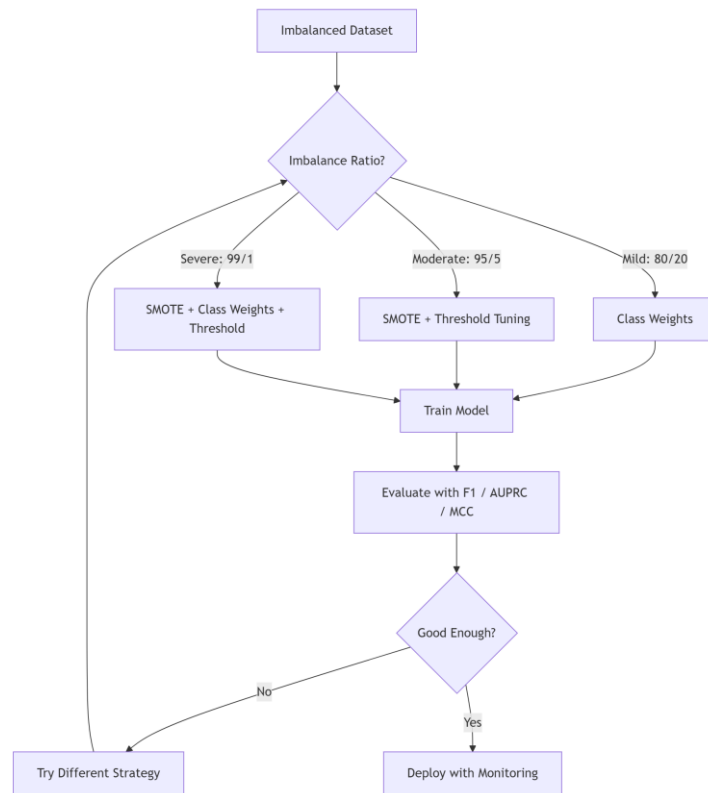
F- β Score = $(1 + \beta^2) * \text{precision} * \text{recall} / (\beta^2 * \text{precision} + \text{recall})$. When $\beta > 1$, recall matters more. When $\beta < 1$, precision matters more. F2 is common in fraud detection (missing fraud is worse than a false alarm).

AUPRC (Area Under Precision-Recall Curve). Like AUC-ROC but more informative for imbalanced data. A random classifier has AUPRC equal to the positive class rate (not 0.5 like ROC). This makes improvements easier to see.

Matthews Correlation Coefficient = $(TP * TN - FP * FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$. Ranges from -1 to +1. Only gives a high score when the model does well on both classes. Balanced even when classes are very different sizes.

For the “always predict negative” model above: precision = $0/0$ (undefined, often set to 0), recall = $0/10 = 0$, F1 = 0, MCC = 0. These metrics correctly identify the model as worthless.

The Imbalanced Data Pipeline



SMOTE: Synthetic Minority Oversampling Technique

Random oversampling duplicates existing minority samples. This works but risks overfitting because the model sees identical points repeatedly.

SMOTE creates new synthetic minority samples that are plausible but not copies. The algorithm:

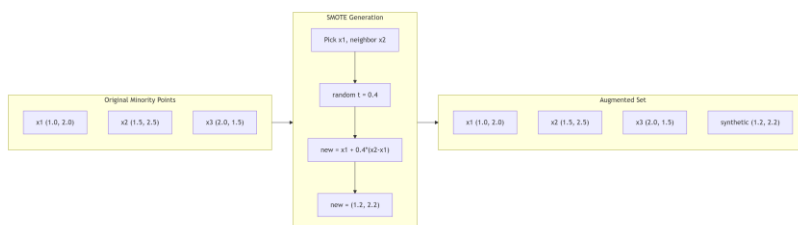
For each minority sample x , find its k nearest neighbors among other minority samples

Pick one neighbor at random

Create a new sample on the line segment between x and that neighbor

The formula: $\text{new_sample} = x + \text{random}(0, 1) * (\text{neighbor} - x)$

This interpolates between real minority points, creating samples in the same region of feature space without just copying existing data.



Sampling Strategies Compared

Random Oversampling: duplicate minority samples to match majority count. - Pros: simple, no information loss - Cons: exact duplicates cause overfitting, increases training time

Random Undersampling: remove majority samples to match minority count. - Pros: fast training, simple - Cons: throws away potentially useful majority data, higher variance

SMOTE: create synthetic minority samples via interpolation. - Pros: generates new data points, reduces overfitting compared to random oversampling - Cons: can create noisy samples near the decision boundary, does not account for majority class distribution

Strategy	Data Changed	Risk	When to Use
Oversample	Minority duplicated	Overfitting	Small datasets, moderate imbalance
Undersample	Majority removed	Information loss	Large datasets, want fast training
SMOTE	Synthetic minority added	Boundary noise	Moderate imbalance, enough minority samples for k-NN

Class Weights

Instead of changing the data, change how the model treats errors. Assign higher weight to misclassifying the minority class.

For a binary problem with 950 negative and 50 positive samples: - Weight for negative class = $n_samples / (2 * n_negative) = 1000 / (2 * 950) = 0.526$ - Weight for positive class = $n_samples / (2 * n_positive) = 1000 / (2 * 50) = 10.0$

The positive class gets 19x the weight. Misclassifying one positive sample costs as much as misclassifying 19 negative samples. The model is forced to pay attention to the minority class.

In logistic regression, this modifies the loss function:

$$\text{weighted_loss} = -\sum(w_i * [y_i * \log(p_i) + (1-y_i) * \log(1-p_i)])$$

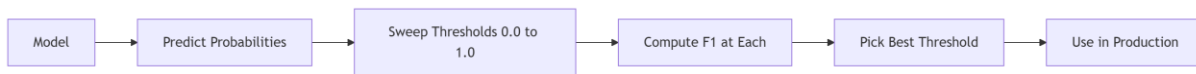
where w_i depends on the class of sample i .

Class weights are mathematically equivalent to oversampling in expectation, but without creating new data points. This makes them faster and avoids the overfitting risk of duplicated samples.

Threshold Tuning

Most classifiers output a probability. The default threshold is 0.5: if $P(\text{positive}) \geq 0.5$, predict positive. But 0.5 is arbitrary. When classes are imbalanced, the optimal threshold is usually much lower.

The process: 1. Train a model 2. Get predicted probabilities on the validation set 3. Sweep thresholds from 0.0 to 1.0 4. Compute F1 (or your chosen metric) at each threshold 5. Pick the threshold that maximizes your metric



A model might output $P(\text{fraud}) = 0.15$ for a fraudulent transaction. At threshold 0.5, this is classified as not fraud. At threshold 0.10, it is correctly caught. The probability calibration matters less than the ranking – as long as fraud gets higher probabilities than non-fraud, there exists a threshold that separates them.

Cost-Sensitive Learning

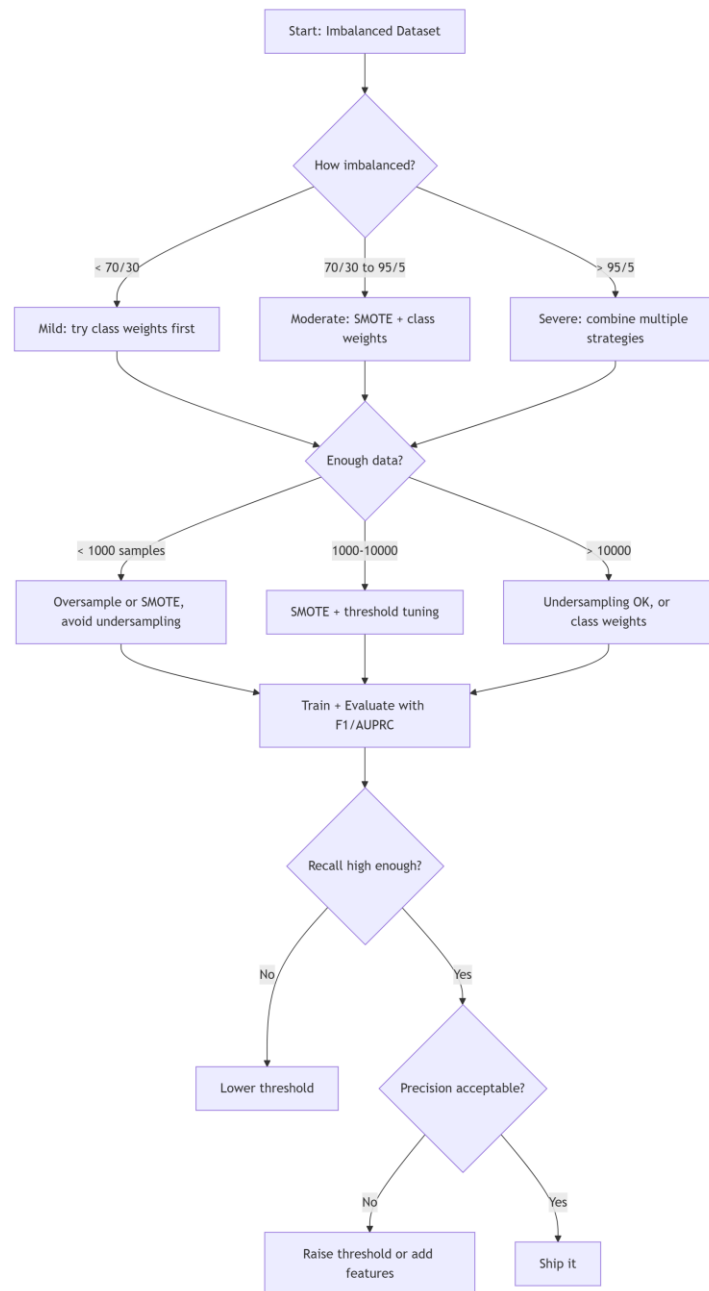
Generalization of class weights. Instead of uniform costs, assign specific misclassification costs:

	Predict Positive	Predict Negative
Actually Positive	0 (correct)	$C_{\text{FN}} = 100$
Actually Negative	$C_{\text{FP}} = 1$	0 (correct)

Missing a fraudulent transaction (FN) costs 100x more than a false alarm (FP). The model optimizes for total cost, not total error count.

This is the most principled approach when you can estimate real-world costs. A missed cancer diagnosis has a very different cost than a false alarm that leads to an extra biopsy. Making these costs explicit forces the right tradeoffs.

Decision Flowchart



Build It

Step 1: Generate an imbalanced dataset

```
import numpy as np

def make_imbalanced_data(n_majority=950, n_minority=50, seed=42):
    rng = np.random.RandomState(seed)
```

```

X_maj = rng.randn(n_majority, 2) * 1.0 + np.array([0.0, 0.0])
X_min = rng.randn(n_minority, 2) * 0.8 + np.array([2.5, 2.5])

X = np.vstack([X_maj, X_min])
y = np.concatenate([np.zeros(n_majority), np.ones(n_minority)])

shuffle_idx = rng.permutation(len(y))
return X[shuffle_idx], y[shuffle_idx]

```

Step 2: SMOTE from scratch

```

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def find_k_neighbors(X, idx, k):
    distances = []
    for i in range(len(X)):
        if i == idx:
            continue
        d = euclidean_distance(X[idx], X[i])
        distances.append((i, d))
    distances.sort(key=lambda x: x[1])
    return [d[0] for d in distances[:k]]

def smote(X_minority, k=5, n_synthetic=100, seed=42):
    rng = np.random.RandomState(seed)
    n_samples = len(X_minority)
    k = min(k, n_samples - 1)
    synthetic = []

    for _ in range(n_synthetic):
        idx = rng.randint(0, n_samples)
        neighbors = find_k_neighbors(X_minority, idx, k)
        neighbor_idx = neighbors[rng.randint(0, len(neighbors))]
        t = rng.random()
        new_point = X_minority[idx] + t * (X_minority[neighbor_idx] - X_minority[idx])
        synthetic.append(new_point)

    return np.array(synthetic)

```

Step 3: Random oversampling and undersampling

```

def random_oversample(X, y, seed=42):
    rng = np.random.RandomState(seed)
    classes, counts = np.unique(y, return_counts=True)
    max_count = counts.max()

    X_resampled = list(X)
    y_resampled = list(y)

```



```

for cls, count in zip(classes, counts):
    if count < max_count:
        cls_indices = np.where(y == cls)[0]
        n_needed = max_count - count
        chosen = rng.choice(cls_indices, size=n_needed, replace=True)
        X_resampled.extend(X[chosen])
        y_resampled.extend(y[chosen])

X_out = np.array(X_resampled)
y_out = np.array(y_resampled)
shuffle = rng.permutation(len(y_out))
return X_out[shuffle], y_out[shuffle]

def random_undersample(X, y, seed=42):
    rng = np.random.RandomState(seed)
    classes, counts = np.unique(y, return_counts=True)
    min_count = counts.min()

    X_resampled = []
    y_resampled = []

    for cls in classes:
        cls_indices = np.where(y == cls)[0]
        chosen = rng.choice(cls_indices, size=min_count, replace=False)
        X_resampled.extend(X[chosen])
        y_resampled.extend(y[chosen])

    X_out = np.array(X_resampled)
    y_out = np.array(y_resampled)
    shuffle = rng.permutation(len(y_out))
    return X_out[shuffle], y_out[shuffle]

```

Step 4: Logistic regression with class weights

```

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def logistic_regression_weighted(X, y, weights, lr=0.01, epochs=200):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = X @ w + b
        pred = sigmoid(z)
        error = pred - y
        weighted_error = error * weights

        gradient_w = (X.T @ weighted_error) / n_samples

```

```

        gradient_b = np.mean(weighted_error)

        w -= lr * gradient_w
        b -= lr * gradient_b

    return w, b

def compute_class_weights(y):
    classes, counts = np.unique(y, return_counts=True)
    n_samples = len(y)
    n_classes = len(classes)
    weight_map = {}
    for cls, count in zip(classes, counts):
        weight_map[cls] = n_samples / (n_classes * count)
    return np.array([weight_map[yi] for yi in y])

```

Step 5: Threshold tuning

```

def find_optimal_threshold(y_true, y_probs, metric="f1"):
    best_threshold = 0.5
    best_score = -1.0

    for threshold in np.arange(0.05, 0.96, 0.01):
        y_pred = (y_probs >= threshold).astype(int)
        tp = np.sum((y_pred == 1) & (y_true == 1))
        fp = np.sum((y_pred == 1) & (y_true == 0))
        fn = np.sum((y_pred == 0) & (y_true == 1))

        if metric == "f1":
            precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
            score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0
        elif metric == "recall":
            score = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        elif metric == "precision":
            score = tp / (tp + fp) if (tp + fp) > 0 else 0.0

        if score > best_score:
            best_score = score
            best_threshold = threshold

    return best_threshold, best_score

```

Step 6: Evaluation functions

```

def confusion_matrix_values(y_true, y_pred):
    tp = np.sum((y_pred == 1) & (y_true == 1))
    tn = np.sum((y_pred == 0) & (y_true == 0))
    fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1))

```

```

    return tp, tn, fp, fn

def compute_metrics(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix_values(y_true, y_pred)
    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

    denom = np.sqrt(float((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn)))
    mcc = (tp * tn - fp * fn) / denom if denom > 0 else 0.0

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "mcc": mcc,
    }

```

Step 7: Compare all approaches

```

X, y = make_imbalanced_data(950, 50, seed=42)
split = int(0.8 * len(y))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Baseline: no treatment
w_base, b_base = logistic_regression_weighted(
    X_train, y_train, np.ones(len(y_train)), lr=0.1, epochs=300
)
probs_base = sigmoid(X_test @ w_base + b_base)
preds_base = (probs_base >= 0.5).astype(int)

# Oversampled
X_over, y_over = random_oversample(X_train, y_train)
w_over, b_over = logistic_regression_weighted(
    X_over, y_over, np.ones(len(y_over)), lr=0.1, epochs=300
)
preds_over = (sigmoid(X_test @ w_over + b_over) >= 0.5).astype(int)

# SMOTE
minority_mask = y_train == 1
X_minority = X_train[minority_mask]
synthetic = smote(X_minority, k=5, n_synthetic=len(y_train) - 2 * int(minority_mask.sum()))
X_smote = np.vstack([X_train, synthetic])
y_smote = np.concatenate([y_train, np.ones(len(synthetic))])
w_sm, b_sm = logistic_regression_weighted(
    X_smote, y_smote, np.ones(len(y_smote)), lr=0.1, epochs=300
)

```

```

)
preds_smote = (sigmoid(X_test @ w_sm + b_sm) >= 0.5).astype(int)

# Class weights
sample_weights = compute_class_weights(y_train)
w_cw, b_cw = logistic_regression_weighted(
    X_train, y_train, sample_weights, lr=0.1, epochs=300
)
probs_cw = sigmoid(X_test @ w_cw + b_cw)
preds_cw = (probs_cw >= 0.5).astype(int)

# Threshold tuning (tune on held-out validation set, not test set)
probs_val = sigmoid(X_val @ w_cw + b_cw)
best_thresh, best_f1 = find_optimal_threshold(y_val, probs_val, metric="f1")
preds_thresh = (probs_cw >= best_thresh).astype(int)

```

The code file runs all of this in a single script and prints results.

Use It

With scikit-learn and imbalanced-learn, these techniques are one-liners:

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y)

model_weighted = LogisticRegression(class_weight="balanced")
model_weighted.fit(X_train, y_train)
print(classification_report(y_test, model_weighted.predict(X_test)))

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
model_smote = LogisticRegression()
model_smote.fit(X_resampled, y_resampled)
print(classification_report(y_test, model_smote.predict(X_test)))

pipeline = Pipeline([
    ("smote", SMOTE()),
    ("model", LogisticRegression(class_weight="balanced")),
])
pipeline.fit(X_train, y_train)
print(classification_report(y_test, pipeline.predict(X_test)))

```

The from-scratch implementations show exactly what each technique does. SMOTE is just k-NN interpolation on the minority class. Class weights multiply the loss. Threshold tuning is a for-loop over cutoffs. No magic.

Exercises

Borderline-SMOTE: modify the SMOTE implementation to only generate synthetic samples for minority points that are near the decision boundary (those whose k-nearest neighbors include majority class samples). Compare results with standard SMOTE on a dataset where classes overlap.

Cost matrix optimization: implement cost-sensitive learning where the cost matrix is a parameter. Create a function that takes a cost matrix and returns optimal predictions that minimize expected cost. Test with different cost ratios (1:10, 1:100, 1:1000) and plot how the precision-recall tradeoff changes.

Threshold calibration: implement Platt scaling (fit a logistic regression on the model's raw outputs to produce calibrated probabilities). Compare the precision-recall curve before and after calibration. Show that calibration does not change the ranking (AUC stays the same) but makes the probabilities more meaningful.

Ensemble with balanced bagging: train multiple models, each on a balanced bootstrap sample (all minority + random subset of majority). Average their predictions. Compare this approach against a single model with SMOTE. Measure both performance and variance across runs.

Imbalance ratio experiment: take a balanced dataset and progressively increase the imbalance ratio (50/50, 70/30, 90/10, 95/5, 99/1). For each ratio, train with and without SMOTE. Plot F1 vs imbalance ratio for both approaches. At what ratio does SMOTE start making a meaningful difference?

Key Terms

Term	What people say	What it actually means
Class imbalance	"One class has way more samples"	The distribution of classes in the dataset is significantly skewed, causing models to favor the majority class
SMOTE	"Synthetic oversampling"	Creates new minority samples by interpolating between existing minority samples and their k-nearest minority neighbors
Class weights	"Making errors on rare classes more expensive"	Multiplying the loss function by class-specific weights so the model penalizes minority misclassification more heavily
Threshold tuning	"Moving the decision boundary"	Changing the probability cutoff for classification from the default 0.5 to a value that optimizes the desired metric
Precision-recall tradeoff	"You cannot have both"	Lowering the threshold catches more positives (higher recall) but also flags more false positives (lower precision), and vice versa

Term	What people say	What it actually means
AUPRC	“Area under the PR curve”	Summarizes the precision-recall curve into a single number; more informative than AUC-ROC when classes are heavily imbalanced
Matthews Correlation Coefficient	“The balanced metric”	A correlation between predicted and actual labels that produces a high score only when the model performs well on both classes
Cost-sensitive learning	“Different mistakes cost different amounts”	Incorporating real-world misclassification costs into the training objective so the model optimizes for total cost, not error count
Random oversampling	“Duplicate the minority”	Repeating minority class samples to balance class counts; simple but risks overfitting to duplicated points

Imbalanced Data Strategy

A decision checklist for handling imbalanced classification. Follow this sequence to pick the right approach for your problem.

Step 1: Measure the imbalance

- 138. Count samples per class
- 139. Compute the imbalance ratio (majority / minority)
- 140. Mild: ratio < 3:1 (e.g., 70/30)
- 141. Moderate: ratio 3:1 to 20:1 (e.g., 95/5)
- 142. Severe: ratio > 20:1 (e.g., 99/1)

Step 2: Pick the right metric

Prefer precision/recall/F1 over accuracy for imbalanced datasets. Choose based on your problem:

Situation	Primary Metric	Secondary Metric
Missing positives is very costly (fraud, disease)	Recall	F2 score
False alarms are costly (spam filter, recommendations)	Precision	F0.5 score
Both matter roughly equally	F1 score	MCC
Need a single ranking metric	AUPRC	AUC-ROC

Situation	Primary Metric	Secondary Metric
Need to compare across datasets	MCC	AUPRC

Step 3: Choose a rebalancing strategy

By imbalance severity

Imbalance	First Try	Second Try	Avoid
Mild (< 3:1)	Class weights	Threshold tuning	Oversampling (unnecessary)
Moderate (3:1 to 20:1)	SMOTE + class weights	Threshold tuning on top	Undersampling (too much data loss)
Severe (> 20:1)	SMOTE + class weights + threshold	Ensemble with balanced bagging	Undersampling alone

By dataset size

Dataset Size	Preferred Strategy	Reason
< 1,000 samples	Oversampling or SMOTE	Cannot afford to lose majority data
1,000 - 10,000	SMOTE + threshold tuning	Enough minority samples for k-NN
> 10,000	Class weights or undersampling	Fast, sufficient minority data

Step 4: Apply the technique

Class weights (always try first)

- In sklearn: `class_weight='balanced'`
- No data modification needed
- Works with any loss-based model
- Equivalent to oversampling in expectation

SMOTE

143. Apply only to training data (never test/validation)
144. Use k=5 neighbors (default)
145. Combine with class weights for best results
146. Watch for noisy synthetic points near the boundary

Threshold tuning

- Train model, get predicted probabilities on validation set
- Sweep thresholds from 0.05 to 0.95
- Pick threshold maximizing your chosen metric
- Always tune on validation data, never test data

Step 5: Validate properly

147. Use stratified cross-validation (preserves class ratios in each fold)
148. Report metrics on the original (non-resampled) test set
149. Never apply SMOTE before splitting – only on training folds
150. Compare against the “always predict majority” baseline

Step 6: Common mistakes to avoid

- Applying SMOTE to the entire dataset before train/test split (data leakage)
- Using accuracy as the evaluation metric
- Not trying class weights first (simplest approach, often sufficient)
- Oversampling and then cross-validating (synthetic points leak across folds)
- Ignoring threshold tuning (free performance, no retraining needed)
- Using random undersampling on small datasets (throws away too much data)

Quick Decision Tree

- Is the imbalance ratio $< 3:1$? -> Try class weights only
- Is the dataset $> 10,000$ samples? -> Class weights + threshold tuning
- Is the dataset $< 1,000$ samples? -> SMOTE + class weights
- Otherwise -> SMOTE + class weights + threshold tuning
- Still not good enough? -> Balanced bagging ensemble

Quiz

Pre-lesson check

151. A fraud detection dataset has 99.9% legitimate transactions and 0.1% fraud. A model predicts ‘legitimate’ for every transaction. What is its accuracy?

a) 50%

b) 0.1%

c) 99.9% **(correct)**

d) 100%

Explanation: Accuracy = 999/1000 = 99.9%. The model catches zero fraud but looks great by accuracy. This is exactly why accuracy is dangerous for imbalanced datasets.

152. Which metric would correctly identify the always-predict-negative model as useless?

a) Accuracy (99.9%)

b) Recall (0%) or F1 score (0%) **(correct)**

c) Specificity (100%)

d) True negative rate (100%)

Explanation: Recall = TP/(TP+FN) = 0/total_positives = 0%. F1 = 200/(0+0) = 0. Both correctly show the model catches nothing in the positive class. Accuracy hides this failure.

Post-lesson check

49. How does SMOTE generate synthetic minority samples?

a) By duplicating existing minority samples exactly

b) By randomly generating points anywhere in the feature space

c) By interpolating between a minority sample and one of its K nearest minority neighbors **(correct)**

d) By flipping the labels of majority class samples

*Explanation: SMOTE picks a minority point, selects one of its K nearest minority neighbors, and creates a new point on the line segment between them: new = x + rand(0,1) (neighbor - x). This produces plausible, non-duplicate samples.**

50. You lower the classification threshold from 0.5 to 0.3 on an imbalanced dataset. What happens to precision and recall?

- a) Both precision and recall increase
- b) Recall increases (more positives caught) but precision decreases (more false positives) **(correct)**
- c) Precision increases but recall decreases
- d) Neither changes – threshold only affects speed
Explanation: Lowering the threshold means more samples are predicted positive. This catches more true positives (recall up) but also adds more false positives (precision down). Threshold tuning trades precision for recall.

51. Why is AUPRC (Area Under Precision-Recall Curve) more informative than AUC-ROC for highly imbalanced datasets?

- a) AUPRC is always higher than AUC-ROC
 - b) A random classifier has AUPRC equal to the positive class rate (e.g., 0.001), making improvements visible, while AUC-ROC starts at 0.5 regardless of imbalance **(correct)**
 - c) AUPRC does not require a threshold
 - d) AUC-ROC cannot be computed for imbalanced data
Explanation: For imbalanced data, AUC-ROC can look deceptively good because the large number of true negatives inflates the true negative rate. AUPRC's baseline equals the positive rate, making real improvements in detecting the minority class much more apparent.
-

Feature Selection

More features is not better. The right features is better.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09, 08 (feature engineering) **Time:** ~75 minutes

Learning Objectives

1. Implement filter methods (variance threshold, mutual information, chi-squared) and wrapper methods (RFE, forward selection) from scratch
2. Explain why mutual information captures nonlinear feature-target relationships that correlation misses
3. Compare L1 regularization (embedded selection) with RFE (wrapper selection) and evaluate their computational tradeoffs
4. Build a feature selection pipeline that combines multiple methods and demonstrate improved generalization on held-out data

The Problem

You have 500 features. Your model trains slowly, overfits constantly, and nobody can explain what it learned. You add more features hoping to improve performance. It gets worse.

This is the curse of dimensionality in action. As the number of features grows, the volume of the feature space explodes. Data points become sparse. Distances between points converge. The model needs exponentially more data to find real patterns. Noise features drown out signal features. Overfitting becomes the default.

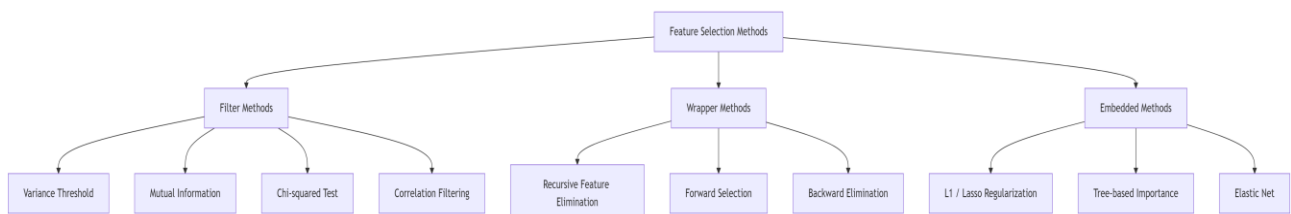
Feature selection is the antidote. Strip away the noise. Remove the redundancy. Keep the features that carry actual information about the target. The result: faster training, better generalization, and models you can actually explain.

The goal is not to use all available information. It is to use the right information.

The Concept

Three Categories of Feature Selection

Every feature selection method falls into one of three categories:



Filter methods score each feature independently using a statistical measure. They do not use a model. Fast, but they miss feature interactions.

Wrapper methods train a model to evaluate feature subsets. They use model performance as the score. Better results, but expensive because they retrain the model many times.

Embedded methods select features as part of model training. L1 regularization drives weights to zero. Decision trees split on the most useful features. Selection happens during fitting, not as a separate step.

Variance Threshold

The simplest filter. If a feature barely varies across samples, it carries almost no information.

Consider a feature that is 0.0 for 999 out of 1000 samples. Its variance is near zero. No model can use it to distinguish between classes. Remove it.

$$\text{variance}(x) = \text{mean}((x - \text{mean}(x))^2)$$

Set a threshold (e.g., 0.01). Drop every feature with variance below it. This removes constant or near-constant features without looking at the target variable at all.

When to use it: as a preprocessing step before other methods. It catches obviously useless features at near-zero cost.

Limitation: a feature can have high variance and still be pure noise. Variance threshold is necessary but not sufficient.

Mutual Information

Mutual information measures how much knowing the value of feature X reduces uncertainty about target Y.

$$I(X; Y) = \sum_x \sum_y p(x, y) * \log(p(x, y) / (p(x) * p(y)))$$

If X and Y are independent, $p(x, y) = p(x) * p(y)$, so the log term is zero and $I(X; Y) = 0$. The more X tells you about Y, the higher the mutual information.

Key advantage over correlation: mutual information captures nonlinear relationships. A feature might have zero correlation with the target but high mutual information because the relationship is quadratic or periodic.

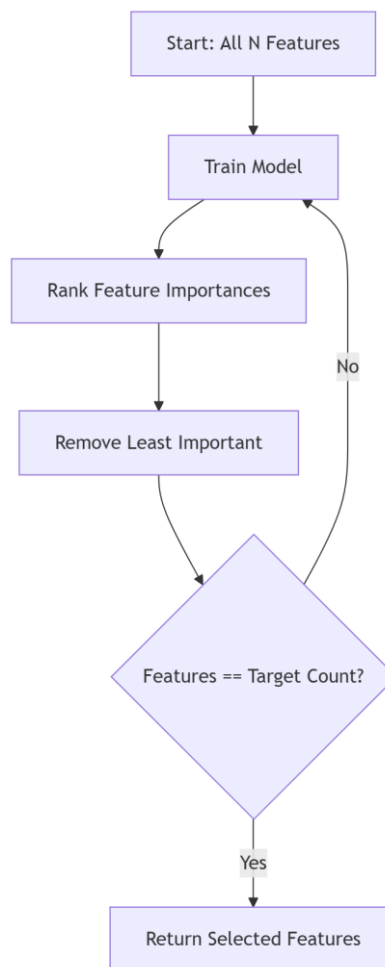
For continuous features, discretize into bins first (histogram-based estimation). The number of bins affects the estimate – too few bins lose information, too many bins add noise. A common choice: $\sqrt{n}$ bins or Sturges' rule $(1 + \log_2(n))$.



Recursive Feature Elimination (RFE)

RFE is a wrapper method. It uses a model's own feature importance to iteratively prune:

1. Train the model with all features
2. Rank features by importance (coefficients for linear models, impurity reduction for trees)
3. Remove the least important feature(s)
4. Repeat until the desired number of features remains



RFE considers feature interactions because the model sees all remaining features together. Removing one feature changes the importance of others. This makes it more thorough than filter methods.

The cost: you train the model $N - \text{target}$ times. With 500 features and a target of 10, that is 490 training runs. For expensive models, this is slow. You can speed it up by removing multiple features per step (e.g., remove the bottom 10% each round).

L1 (Lasso) Regularization

L1 regularization adds the absolute value of weights to the loss function:

```
loss = prediction_error + alpha * sum(|w_i|)
```

The α parameter controls how aggressively features are pruned. Higher α means more weights go to exactly zero.

Why exactly zero? The L1 penalty creates a diamond-shaped constraint region in weight space. The optimal solution tends to land at a corner of this diamond, where one or more weights are zero. L2 regularization (ridge) creates a circular constraint where weights shrink but rarely hit zero.

This is embedded feature selection: the model learns during training which features to ignore. Features with zero weight are effectively removed.

Advantages: single training run, handles correlated features (picks one and zeros the others), built into most linear model implementations.

Limitation: only works for linear models. Cannot capture nonlinear feature importance.

Tree-Based Feature Importance

Decision trees and their ensembles (random forests, gradient boosting) naturally rank features. Every split reduces impurity (Gini or entropy for classification, variance for regression). Features that produce larger impurity reductions are more important.

For a random forest with T trees:

```
importance(feature_j) = (1/T) * sum over all trees of  
    sum over all nodes splitting on feature_j of  
    (n_samples * impurity_decrease)
```

This gives a normalized importance score for each feature. It handles nonlinear relationships and feature interactions automatically.

Caution: tree-based importance is biased toward features with many unique values (high cardinality). A random ID column will appear important because it perfectly splits every sample. Use permutation importance as a sanity check.

Permutation Importance

A model-agnostic method:

1. Train the model and record baseline performance on validation data
2. For each feature: shuffle its values randomly, measure the drop in performance

3. The bigger the drop, the more important the feature

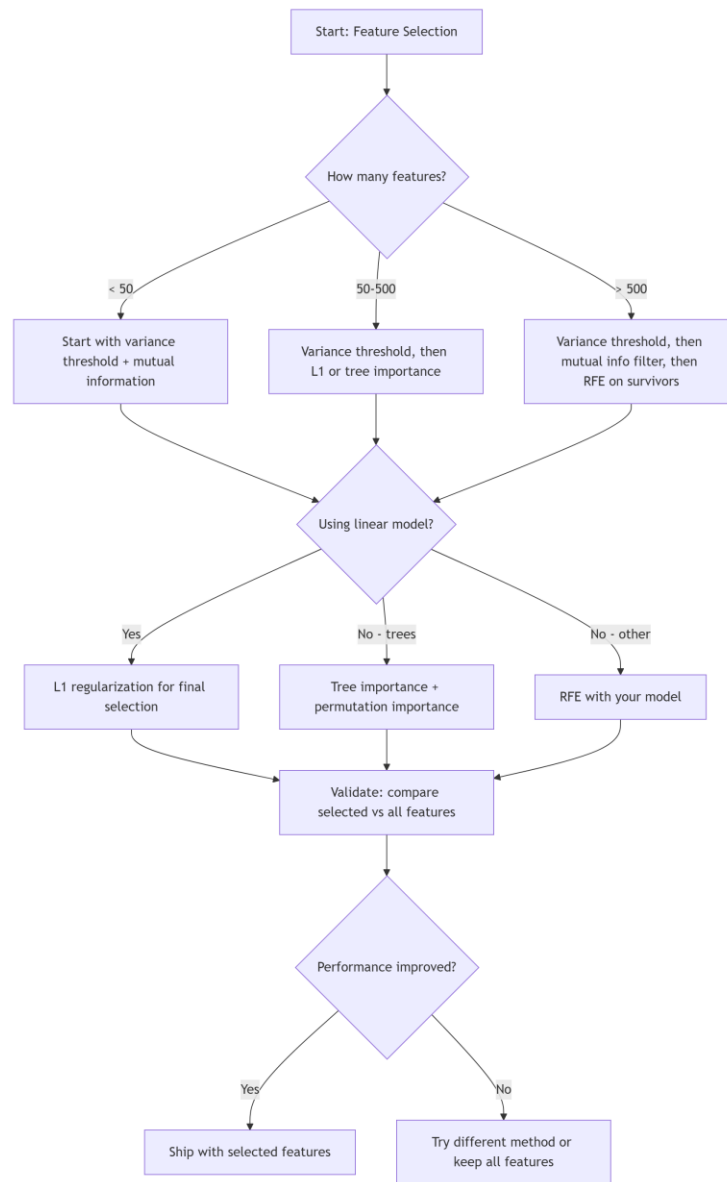
If shuffling a feature does not hurt performance, the model does not depend on it. If performance collapses, that feature is critical.

Permutation importance avoids the cardinality bias of tree-based importance. But it is slow: one full evaluation per feature, repeated multiple times for stability.

Comparison Table

Method	Type	Speed	Nonlinear	Feature Interactions
Variance threshold	Filter	Very fast	No	No
Mutual information	Filter	Fast	Yes	No
Correlation filter	Filter	Fast	No	No
RFE	Wrapper	Slow	Depends on model	Yes
L1 / Lasso	Embedded	Fast	No (linear)	No
Tree importance	Embedded	Medium	Yes	Yes
Permutation importance	Model-agnostic	Slow	Yes	Yes

Decision Flowchart



Build It

Step 1: Generate synthetic data with known feature structure

```
import numpy as np

def make_feature_selection_data(n_samples=500, seed=42):
    rng = np.random.RandomState(seed)

    x1 = rng.randn(n_samples)
    x2 = rng.randn(n_samples)
```



```

x3 = rng.randn(n_samples)
x4 = x1 + 0.1 * rng.randn(n_samples)
x5 = x2 + 0.1 * rng.randn(n_samples)

informative = np.column_stack([x1, x2, x3, x4, x5])

correlated = np.column_stack([
    x1 * 0.9 + 0.1 * rng.randn(n_samples),
    x2 * 0.8 + 0.2 * rng.randn(n_samples),
    x3 * 0.7 + 0.3 * rng.randn(n_samples),
    x1 * 0.5 + x2 * 0.5 + 0.1 * rng.randn(n_samples),
    x2 * 0.6 + x3 * 0.4 + 0.1 * rng.randn(n_samples),
])

noise = rng.randn(n_samples, 10) * 0.5

X = np.hstack([informative, correlated, noise])
y = (2 * x1 - 1.5 * x2 + x3 + 0.5 * rng.randn(n_samples) > 0).astype(int)

feature_names = (
    [f"info_{i}" for i in range(5)]
    + [f"corr_{i}" for i in range(5)]
    + [f"noise_{i}" for i in range(10)]
)

return X, y, feature_names

```

We know the ground truth: features 0-4 are informative (plus 3 and 4 are correlated copies of 0 and 1), features 5-9 are correlated with informative features, features 10-19 are pure noise. A good selection method should rank 0-4 highest and 10-19 lowest.

Step 2: Variance threshold

```

def variance_threshold(X, threshold=0.01):
    variances = np.var(X, axis=0)
    mask = variances > threshold
    return mask, variances

```

Step 3: Mutual information (discrete)

```

def discretize(x, n_bins=10):
    min_val, max_val = x.min(), x.max()
    if max_val == min_val:
        return np.zeros_like(x, dtype=int)
    bin_edges = np.linspace(min_val, max_val, n_bins + 1)
    binned = np.digitize(x, bin_edges[1:-1])
    return binned

def mutual_information(X, y, n_bins=10):

```

```

n_samples, n_features = X.shape
mi_scores = np.zeros(n_features)

y_vals, y_counts = np.unique(y, return_counts=True)
p_y = y_counts / n_samples

for f in range(n_features):
    x_binned = discretize(X[:, f], n_bins)
    x_vals, x_counts = np.unique(x_binned, return_counts=True)
    p_x = dict(zip(x_vals, x_counts / n_samples))

    mi = 0.0
    for xv in x_vals:
        for yi, yv in enumerate(y_vals):
            joint_mask = (x_binned == xv) & (y == yv)
            p_xy = np.sum(joint_mask) / n_samples
            if p_xy > 0:
                mi += p_xy * np.log(p_xy / (p_x[xv] * p_y[yi]))
    mi_scores[f] = mi

return mi_scores

```

Step 4: Recursive Feature Elimination

```

def simple_logistic_importance(X, y, lr=0.1, epochs=100):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = X @ w + b
        pred = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
        error = pred - y
        w -= lr * (X.T @ error) / n_samples
        b -= lr * np.mean(error)

    return w, b

def rfe(X, y, n_features_to_select=5, lr=0.1, epochs=100):
    n_total = X.shape[1]
    remaining = list(range(n_total))
    rankings = np.ones(n_total, dtype=int)
    rank = n_total

    while len(remaining) > n_features_to_select:
        X_subset = X[:, remaining]
        w, _ = simple_logistic_importance(X_subset, y, lr, epochs)
        importances = np.abs(w)

        least_idx = np.argmin(importances)
        original_idx = remaining[least_idx]

```

```

        rankings[original_idx] = rank
        rank -= 1
        remaining.pop(least_idx)

    for idx in remaining:
        rankings[idx] = 1

    selected_mask = rankings == 1
    return selected_mask, rankings

```

Step 5: L1 feature selection

```

def soft_threshold(w, alpha):
    return np.sign(w) * np.maximum(np.abs(w) - alpha, 0)

def l1_feature_selection(X, y, alpha=0.1, lr=0.01, epochs=500):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = X @ w + b
        pred = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
        error = pred - y

        gradient_w = (X.T @ error) / n_samples
        gradient_b = np.mean(error)

        w -= lr * gradient_w
        w = soft_threshold(w, lr * alpha)
        b -= lr * gradient_b

    selected_mask = np.abs(w) > 1e-6
    return selected_mask, w

```

Step 6: Tree-based importance (simple decision tree)

```

def gini_impurity(y):
    if len(y) == 0:
        return 0.0
    classes, counts = np.unique(y, return_counts=True)
    probs = counts / len(y)
    return 1.0 - np.sum(probs ** 2)

def best_split(X, y, feature_idx):
    values = np.unique(X[:, feature_idx])
    if len(values) <= 1:
        return None, -1.0

    best_threshold = None

```

```

best_gain = -1.0
parent_gini = gini_impurity(y)
n = len(y)

for i in range(len(values) - 1):
    threshold = (values[i] + values[i + 1]) / 2.0
    left_mask = X[:, feature_idx] <= threshold
    right_mask = ~left_mask

    n_left = np.sum(left_mask)
    n_right = np.sum(right_mask)

    if n_left == 0 or n_right == 0:
        continue

    gain = parent_gini - (n_left / n) * gini_impurity(y[left_mask]) - (n_right / n)
    * gini_impurity(y[right_mask])

    if gain > best_gain:
        best_gain = gain
        best_threshold = threshold

return best_threshold, best_gain

def tree_importance(X, y, n_trees=50, max_depth=5, seed=42):
    rng = np.random.RandomState(seed)
    n_samples, n_features = X.shape
    importances = np.zeros(n_features)

    for _ in range(n_trees):
        sample_idx = rng.choice(n_samples, size=n_samples, replace=True)
        feature_subset = rng.choice(n_features, size=max(1, int(np.sqrt(n_features))),
replace=False)

        X_boot = X[sample_idx]
        y_boot = y[sample_idx]

        tree_imp = _build_tree_importance(X_boot, y_boot, feature_subset, max_depth)
        importances += tree_imp

    total = importances.sum()
    if total > 0:
        importances /= total

    return importances

def _build_tree_importance(X, y, feature_subset, max_depth, depth=0):
    n_features = X.shape[1]
    importances = np.zeros(n_features)

    if depth >= max_depth or len(np.unique(y)) <= 1 or len(y) < 4:

```

```

        return importances

    best_feature = None
    best_threshold = None
    best_gain = -1.0

    for f in feature_subset:
        threshold, gain = best_split(X, y, f)
        if gain > best_gain:
            best_gain = gain
            best_feature = f
            best_threshold = threshold

    if best_feature is None or best_gain <= 0:
        return importances

    importances[best_feature] += best_gain * len(y)

    left_mask = X[:, best_feature] <= best_threshold
    right_mask = ~left_mask

    importances += _build_tree_importance(X[left_mask], y[left_mask], feature_subset, max_depth, depth + 1)
    importances += _build_tree_importance(X[right_mask], y[right_mask], feature_subset, max_depth, depth + 1)

    return importances

```

Step 7: Run all methods and compare

The code file runs all five methods on the same synthetic dataset and prints a comparison table showing which features each method selects.

Use It

With scikit-learn, feature selection is built into the pipeline:

```

from sklearn.feature_selection import (
    VarianceThreshold,
    mutual_info_classif,
    RFE,
    SelectFromModel,
)
from sklearn.linear_model import Lasso, LogisticRegression
from sklearn.ensemble import RandomForestClassifier

vt = VarianceThreshold(threshold=0.01)
X_filtered = vt.fit_transform(X)

mi_scores = mutual_info_classif(X, y)
top_k = np.argsort(mi_scores)[-10:]

```

```

rfe_selector = RFE(LogisticRegression(), n_features_to_select=10)
rfe_selector.fit(X, y)
X_rfe = rfe_selector.transform(X)

lasso_selector = SelectFromModel(Lasso(alpha=0.01))
lasso_selector.fit(X, y)
X_lasso = lasso_selector.transform(X)

rf = RandomForestClassifier(n_estimators=100)
rf.fit(X, y)
importances = rf.feature_importances_

```

The from-scratch implementations show exactly what happens inside each method. Variance threshold is just computing `var(X, axis=0)` and applying a mask. Mutual information is counting joint and marginal frequencies in a contingency table. RFE is a loop that trains, ranks, and prunes. L1 is gradient descent with a soft-thresholding step. Tree importance accumulates impurity reductions across splits. No magic – just statistics and loops.

The sklearn versions add robustness (e.g., `mutual_info_classif` uses k-NN density estimation instead of binning), speed (C implementations), and pipeline integration.

Exercises

1. **Forward selection:** implement the opposite of RFE. Start with zero features. At each step, add the feature that improves model performance the most. Stop when adding features no longer helps. Compare the selected features against RFE results. Which is faster? Which gives better results?
2. **Stability selection:** run L1 feature selection 50 times, each time on a random 80% subsample of the data, with slightly different α values. Count how often each feature is selected. Features selected in > 80% of runs are “stable.” Compare stable features against single-run L1 selection. Which is more reliable?
3. **Multicollinearity detection:** compute the correlation matrix for all features. Implement a function that, given a correlation threshold (e.g., 0.9), removes one feature from each highly-correlated pair (keeping the one with higher mutual information with the target). Test on the synthetic dataset and verify it removes the redundant correlated features.
4. **Feature selection pipeline:** chain variance threshold, mutual information filter, and RFE into a single pipeline. First remove near-zero-variance features, then keep the top 50% by mutual information, then run RFE on the survivors. Compare this pipeline against running RFE alone on all features. Is the pipeline faster? Is it equally accurate?
5. **Permutation importance from scratch:** implement permutation importance. For each feature, shuffle its values 10 times, measure the average drop in F1 score. Compare the ranking against tree-based importance. Find cases where they disagree and explain why (hint: correlated features).

Key Terms

Term	What people say	What it actually means
Filter method	“Score features independently”	A feature selection approach that ranks features using a statistical measure without training a model, evaluating each feature in isolation
Wrapper method	“Use the model to pick features”	A feature selection approach that evaluates feature subsets by training a model and using its performance as the selection criterion
Embedded method	“The model selects features during training”	Feature selection that happens as part of model fitting, such as L1 regularization driving weights to zero
Mutual information	“How much one variable tells you about another”	A measure of the reduction in uncertainty about Y given knowledge of X, capturing both linear and nonlinear dependencies
Recursive Feature Elimination	“Train, rank, prune, repeat”	An iterative wrapper method that trains a model, removes the least important feature(s), and repeats until a target count is reached
L1 / Lasso regularization	“Penalty that kills features”	Adding the sum of absolute weight values to the loss function, which drives unimportant feature weights to exactly zero
Variance threshold	“Remove constant features”	Dropping features whose variance across samples falls below a specified threshold, filtering out features that carry no information
Feature importance	“Which features matter most”	A score indicating how much each feature contributes to model predictions, computed from split gains (trees) or coefficient magnitudes (linear)
Permutation importance	“Shuffle and measure the damage”	Evaluating feature importance by randomly shuffling each feature’s values and measuring the resulting drop in model performance
Curse of dimensionality	“Too many features, not enough data”	The phenomenon where adding features increases the volume of the feature space exponentially, making data sparse and distances meaningless

Feature Selection Strategy

A quick reference for picking and applying the right feature selection method.

Step 1: Start with cleanup

Before applying any method, remove obviously useless features:

1. **Constant features:** variance = 0. Remove them.
2. **Near-constant features:** variance < 0.01 (or your threshold). Remove them.
3. **Duplicate features:** identical columns. Keep one, drop the rest.
4. **ID columns:** unique per row, carry no generalizable information. Remove them.

This takes seconds and can eliminate 10-30% of features in messy real-world datasets.

Step 2: Choose a method based on your situation

Quick Decision Tree

1. **< 50 features?** Start with mutual information ranking. Keep top K.
2. **50 - 500 features?** Use variance threshold first, then L1 (Lasso) if using a linear model, or tree importance if using trees.
3. **> 500 features?** Chain methods: variance threshold -> mutual information filter (top 50%) -> RFE on survivors.
4. **Need interpretability?** L1 regularization gives you exact zero/nonzero. Tree importance gives ranked scores.
5. **Need to capture nonlinear relationships?** Mutual information or tree-based importance. Avoid L1 (linear only).
6. **Need feature interactions?** RFE or tree-based importance. Filter methods miss interactions.

Method Reference

Method	When to Use	When to Avoid
Variance threshold	Always, as a first step	Never skip this
Mutual information	Quick ranking, nonlinear relationships	When you need feature interaction detection
RFE	Thorough selection, moderate feature count	Very expensive models, > 1000 features
L1 / Lasso	Linear models, fast embedded selection	Nonlinear problems, highly correlated features
Tree importance	Nonlinear relationships, feature interactions	Biased by high-cardinality features

Method	When to Use	When to Avoid
Permutation importance	Model-agnostic validation, final check	Too slow for initial screening

Step 3: Validate your selection

1. Compare model performance with selected features vs all features
2. Use cross-validation, not a single train/test split
3. If performance drops by more than 1-2%, you may have removed useful features
4. If performance improves, you successfully removed noise

Step 4: Handle common pitfalls

Correlated features

1. L1 arbitrarily picks one from a correlated group and zeros the others
2. Compute the correlation matrix first and decide which correlated features to keep
3. Tree importance spreads importance across correlated features

Data leakage

1. Fit feature selection on training data only
2. Apply the same selection to test data
3. In cross-validation, feature selection must happen inside each fold

Overfitting to feature selection

1. RFE with too many iterations can overfit to the training set
2. Validate on held-out data, not the data used for selection
3. Use stability selection (repeat on subsamples) for more robust results

Step 5: Production checklist

- ☐ Variance threshold applied as first filter
- ☐ Feature selection fitted on training data only
- ☐ Selected features documented (names, method used, scores)
- ☐ Performance compared: selected features vs all features
- ☐ Cross-validated, not single-split evaluation
- ☐ Feature selection integrated into the training pipeline (not done manually)
- ☐ Monitoring in place for feature drift (selected features may become stale)

Quiz

Pre-lesson check

1. Why can adding more features actually make a model perform worse?
 1. More features always improve model accuracy
 2. Irrelevant features add noise, increase overfitting risk, and dilute the signal from useful features **(correct)**
 3. Models have a hard limit on the number of features they can accept
 4. More features make the model run out of memory
Explanation: Irrelevant features give the model opportunities to overfit on noise in the training data. They increase dimensionality, making the data sparser and distances less meaningful (curse of dimensionality).
2. What is the key difference between filter and wrapper feature selection methods?
 1. Filter methods use a model to evaluate features; wrapper methods use statistics
 2. Filter methods score features using statistics without a model; wrapper methods train a model to evaluate feature subsets **(correct)**
 3. Filter methods are always more accurate than wrapper methods
 4. Wrapper methods can only select one feature at a time
Explanation: Filter methods (variance threshold, mutual information, correlation) score features with statistical measures. Wrapper methods (RFE, forward selection) train models repeatedly to evaluate different feature subsets.

Post-lesson check

1. Mutual information can detect relationships that Pearson correlation cannot. What kind?
 1. Linear relationships between continuous features
 2. Nonlinear relationships such as quadratic or periodic dependencies **(correct)**

3. Relationships between categorical features only
 4. Relationships that require more than 1000 data points
Explanation: Pearson correlation only measures linear association. A quadratic relationship ($y = x^2$) has zero correlation but high mutual information. MI captures any statistical dependency between variables.
2. L1 (Lasso) regularization performs feature selection as part of training. How?
 1. It removes features with low variance before training starts
 2. It drives the weights of irrelevant features to exactly zero, effectively eliminating them from the model **(correct)**
 3. It ranks features by correlation with the target
 4. It trains separate models for each feature
Explanation: L1 regularization adds $|w|$ penalty to the loss. The geometry of the L1 constraint (diamond shape) causes some weight solutions to land exactly at zero, producing sparse models that automatically select features.
 3. RFE removes the least important feature and retrains. Why is this better than just removing all low-importance features at once?
 1. It is not better – removing all at once is always preferred
 2. Feature importances change as features are removed, so iterative removal accounts for interactions between features **(correct)**
 3. RFE uses a different importance metric than single-step removal
 4. Removing one at a time is only necessary for neural networks
Explanation: Feature importances are relative. When a correlated feature is removed, the importance of its counterpart may increase. Iterative removal lets the model reassess importances at each step, capturing these interactions.
-

AI Engineering From Scratch

A Complete Reference for AI Engineers

13 Lessons

AI ENGINEERING FROM SCRATCH

Phase 3: Deep Learning Core

The Perceptron

The perceptron is the atom of neural networks. Split it open and you find weights, a bias, and a decision.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Linear Algebra Intuition) **Time:** ~60 minutes

Learning Objectives

- Implement a perceptron from scratch in Python, including the weight update rule and step activation function

Explain why a single perceptron can only solve linearly separable problems and demonstrate the XOR failure case
- Construct a multi-layer perceptron by composing OR, NAND, and AND gates to solve XOR
- Train a two-layer network with sigmoid activation and backpropagation to learn XOR automatically

The Problem

You know vectors and dot products. You know that a matrix transforms inputs into outputs. But how does a machine *learn* which transformation to use?

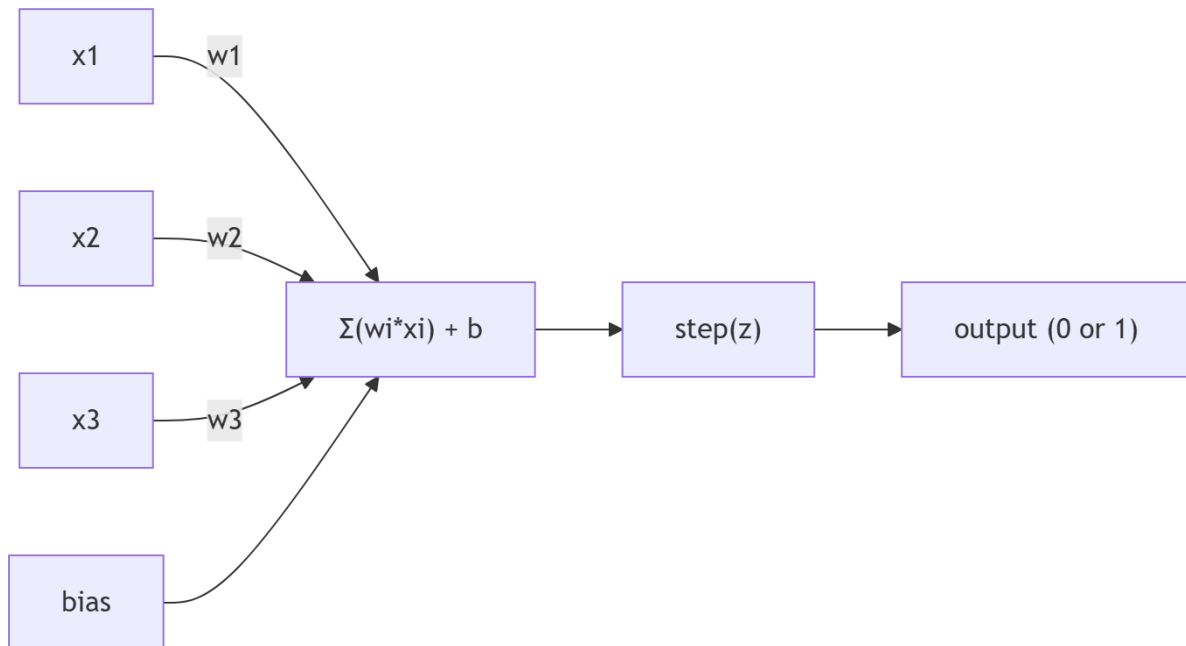
The perceptron answers this. It's the simplest possible learning machine: take some inputs, multiply by weights, add a bias, and make a binary decision. Then adjust. That's it. Every neural network ever built is layers of this idea stacked together.

Understanding the perceptron means understanding what “learning” actually means in code: adjusting numbers until the output matches reality.

The Concept

One Neuron, One Decision

A perceptron takes n inputs, multiplies each by a weight, sums them up, adds a bias, and passes the result through an activation function.



The step function is brutal: if the weighted sum plus bias is ≥ 0 , output 1. Otherwise, output 0.

```

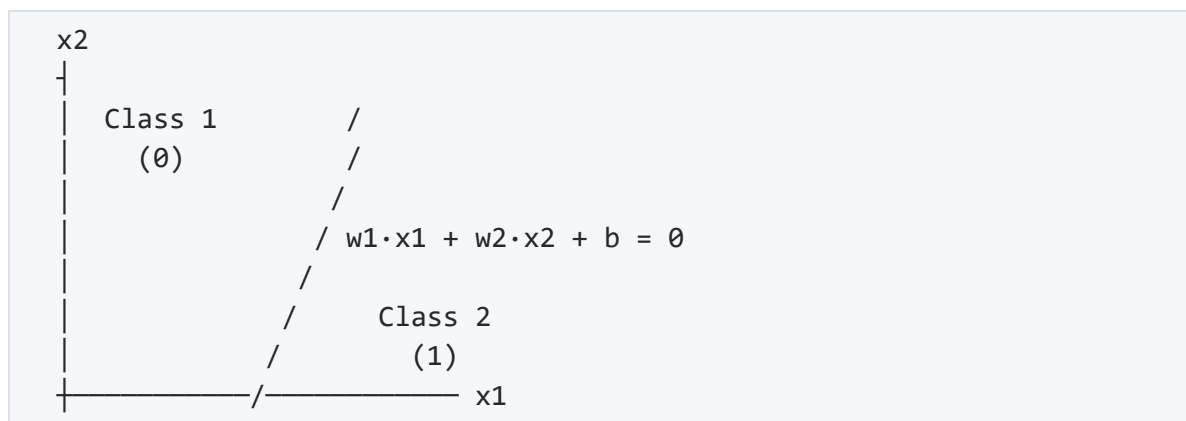
step(z) = 1  if z >= 0
          0  if z < 0

```

This is a linear classifier. The weights and bias define a line (or hyperplane in higher dimensions) that splits the input space into two regions.

The Decision Boundary

For two inputs, the perceptron draws a line through 2D space:



Everything on one side of the line outputs 0. Everything on the other side outputs 1. Training moves this line until it correctly separates the classes.

The Learning Rule

The perceptron learning rule is simple:

```
For each training example (x, y_true):
    y_pred = predict(x)
    error = y_true - y_pred

    For each weight:
        w_i = w_i + learning_rate * error * x_i
    bias = bias + learning_rate * error
```

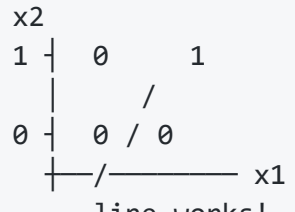
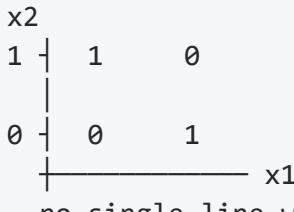
If the prediction is correct, error = 0, nothing changes. If it predicts 0 but should be 1, weights increase. If it predicts 1 but should be 0, weights decrease. The learning rate controls how big each adjustment is.

The XOR Problem

Here's where it breaks. Look at these logic gates:

AND gate:			OR gate:			XOR gate:		
x1	x2	out	x1	x2	out	x1	x2	out
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

AND and OR are linearly separable: you can draw a single line to separate the 0s from the 1s. XOR is not. No single line can separate [0,1] and [1,0] from [0,0] and [1,1].

AND (separable):	XOR (not separable):
	

This is a fundamental limit. A single perceptron can only solve linearly separable problems. Minsky and Papert proved this in 1969 and it nearly killed neural network research for a decade.

The fix: stack perceptrons into layers. A multi-layer perceptron can solve XOR by combining two linear decisions into a nonlinear one.

Build It

Step 1: The Perceptron class

```
class Perceptron:
    def __init__(self, n_inputs, learning_rate=0.1):
        self.weights = [0.0] * n_inputs
        self.bias = 0.0
        self.lr = learning_rate

    def predict(self, inputs):
        total = sum(w * x for w, x in zip(self.weights, inputs))
        total += self.bias
        return 1 if total >= 0 else 0

    def train(self, training_data, epochs=100):
        for epoch in range(epochs):
            errors = 0
            for inputs, target in training_data:
                prediction = self.predict(inputs)
                error = target - prediction
                if error != 0:
                    errors += 1
                    for i in range(len(self.weights)):
                        self.weights[i] += self.lr * error * inputs[i]
                        self.bias += self.lr * error
            if errors == 0:
                print(f"Converged at epoch {epoch + 1}")
                return
        print(f"Did not converge after {epochs} epochs")
```

Step 2: Train on logic gates

```
and_data = [
    ([0, 0], 0),
    ([0, 1], 0),
    ([1, 0], 0),
    ([1, 1], 1),
]

or_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 1),
]

not_data = [
    ([0], 1),
    ([1], 0),
]
```

```

print("=== AND Gate ===")
p_and = Perceptron(2)
p_and.train(and_data)
for inputs, _ in and_data:
    print(f" {inputs} -> {p_and.predict(inputs)}")

print("\n=== OR Gate ===")
p_or = Perceptron(2)
p_or.train(or_data)
for inputs, _ in or_data:
    print(f" {inputs} -> {p_or.predict(inputs)}")

print("\n=== NOT Gate ===")
p_not = Perceptron(1)
p_not.train(not_data)
for inputs, _ in not_data:
    print(f" {inputs} -> {p_not.predict(inputs)}")

```

Step 3: Watch XOR fail

```

xor_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 0),
]

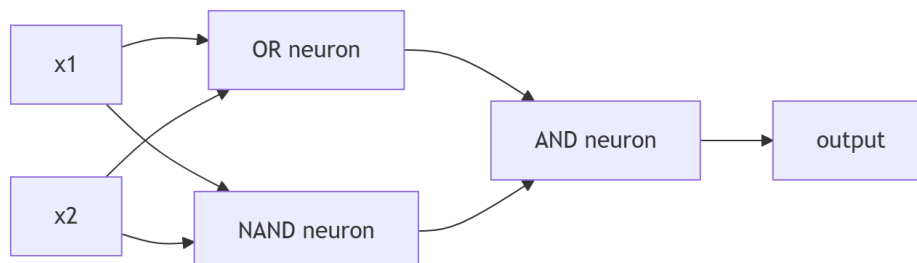
print("\n=== XOR Gate (single perceptron) ===")
p_xor = Perceptron(2)
p_xor.train(xor_data, epochs=1000)
for inputs, expected in xor_data:
    result = p_xor.predict(inputs)
    status = "OK" if result == expected else "WRONG"
    print(f" {inputs} -> {result} (expected {expected}) {status}")

```

It will never converge. This is the hard proof that a single perceptron cannot learn XOR.

Step 4: Solve XOR with two layers

The trick: $XOR = (x1 \text{ OR } x2) \text{ AND NOT } (x1 \text{ AND } x2)$. Combine three perceptrons:



```

def xor_network(x1, x2):
    or_neuron = Perceptron(2)
    or_neuron.weights = [1.0, 1.0]
    or_neuron.bias = -0.5

    nand_neuron = Perceptron(2)
    nand_neuron.weights = [-1.0, -1.0]
    nand_neuron.bias = 1.5

    and_neuron = Perceptron(2)
    and_neuron.weights = [1.0, 1.0]
    and_neuron.bias = -1.5

    hidden1 = or_neuron.predict([x1, x2])
    hidden2 = nand_neuron.predict([x1, x2])
    output = and_neuron.predict([hidden1, hidden2])
    return output

print("\n=== XOR Gate (multi-layer network) ===")
for inputs, expected in xor_data:
    result = xor_network(inputs[0], inputs[1])
    print(f" {inputs} -> {result} (expected {expected})")

```

All four cases correct. Stacking perceptrons into layers creates decision boundaries that no single perceptron can produce.

Step 5: Train a Two-Layer Network

Step 4 hand-wired the weights. That works for XOR, but not for real problems where you don't know the right weights in advance. The fix: replace the step function with sigmoid and learn the weights automatically through backpropagation.

```

class TwoLayerNetwork:
    def __init__(self, learning_rate=0.5):
        import random
        random.seed(0)
        self.w_hidden = [[random.uniform(-1, 1), random.uniform(-1, 1)] for _ in range(
2)]

        self.b_hidden = [random.uniform(-1, 1), random.uniform(-1, 1)]
        self.w_output = [random.uniform(-1, 1), random.uniform(-1, 1)]
        self.b_output = random.uniform(-1, 1)
        self.lr = learning_rate

    def sigmoid(self, x):
        import math
        x = max(-500, min(500, x))
        return 1.0 / (1.0 + math.exp(-x))

    def forward(self, inputs):
        self.inputs = inputs

```

```

        self.hidden_outputs = []
        for i in range(2):
            z = sum(w * x for w, x in zip(self.w_hidden[i], inputs)) + self.b_hidden[i]
            self.hidden_outputs.append(self.sigmoid(z))
        z_out = sum(w * h for w, h in zip(self.w_output, self.hidden_outputs)) + self.b
        _output
        self.output = self.sigmoid(z_out)
        return self.output

    def train(self, training_data, epochs=10000):
        for epoch in range(epochs):
            total_error = 0
            for inputs, target in training_data:
                output = self.forward(inputs)
                error = target - output
                total_error += error ** 2

            d_output = error * output * (1 - output)

            saved_w_output = self.w_output[:]
            hidden_deltas = []
            for i in range(2):
                h = self.hidden_outputs[i]
                hd = d_output * saved_w_output[i] * h * (1 - h)
                hidden_deltas.append(hd)

            for i in range(2):
                self.w_output[i] += self.lr * d_output * self.hidden_outputs[i]
                self.b_output += self.lr * d_output

            for i in range(2):
                for j in range(len(inputs)):
                    self.w_hidden[i][j] += self.lr * hidden_deltas[i] * inputs[j]
                    self.b_hidden[i] += self.lr * hidden_deltas[i]

net = TwoLayerNetwork(learning_rate=2.0)
net.train(xor_data, epochs=10000)
for inputs, expected in xor_data:
    result = net.forward(inputs)
    predicted = 1 if result >= 0.5 else 0
    print(f" {inputs} -> {result:.4f} (rounded: {predicted}, expected {expected})")

```

Two key differences from Step 4. First, sigmoid replaces the step function – it's smooth, so gradients exist. Second, the `train` method propagates error backward from output to hidden layer, adjusting every weight proportionally to its contribution to the error. That's backpropagation in 20 lines.

This is the bridge to Lesson 03. The math behind `d_output` and `hidden_deltas` is the chain rule applied to the network graph. We'll derive it properly there.

Use It

Everything you just built from scratch exists in one import:

```

from sklearn.linear_model import Perceptron as SkPerceptron
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 0, 0, 1])

clf = SkPerceptron(max_iter=100, tol=1e-3)
clf.fit(X, y)
print([clf.predict([x])[0] for x in X])

```

Five lines. Your 30-line Perceptron class does the same thing. The sklearn version adds convergence checks, multiple loss functions, and sparse input support – but the core loop is identical: weighted sum, step function, weight update on error.

The real gap shows up at scale. What changes in production networks:

- 153. The step function becomes sigmoid, ReLU, or other smooth activations
- 154. Weights are learned automatically via backpropagation (Lesson 03)
- 155. Layers get deeper: 3, 10, 100+ layers
- 156. The same principle holds: each layer creates new features from the previous layer’s outputs

A single perceptron can only draw straight lines. Stack them, and you can draw any shape.

Exercises

- 157. Train a perceptron on a NAND gate (the universal gate - any logic circuit can be built from NAND). Verify its weights and bias form a valid decision boundary.
- 158. Modify the Perceptron class to track the decision boundary ($w_1x_1 + w_2x_2 + b = 0$) at each epoch. Print how the line shifts during training on the AND gate.
- 159. Build a 3-input perceptron that outputs 1 only when at least 2 of the 3 inputs are 1 (a majority vote function). Is this linearly separable? Why?

Key Terms

Term	What people say	What it actually means
Perceptron	“A fake neuron”	A linear classifier: dot product of inputs and weights, plus bias, through a step function
Weight	“How important an input is”	A multiplier that scales each input’s contribution to the decision
Bias	“The threshold”	A constant that shifts the decision boundary, letting the perceptron fire even with zero inputs

Term	What people say	What it actually means
Activation function	“The thing that squishes values”	A function applied after the weighted sum - step function for perceptrons, sigmoid/ReLU for modern networks
Linearly separable	“You can draw a line between them”	A dataset where a single hyperplane can perfectly separate the classes
XOR problem	“The thing perceptrons can’t do”	Proof that single-layer networks cannot learn non-linearly-separable functions
Decision boundary	“Where the classifier switches”	The hyperplane $w*x + b = 0$ that divides input space into two classes
Multi-layer perceptron	“A real neural network”	Perceptrons stacked in layers, where each layer’s output feeds the next layer’s input

The Perceptron Pattern

A perceptron computes a weighted sum of inputs plus a bias, then applies a step function to produce a binary output. It is the fundamental unit of neural networks.

$$\text{output} = \text{step}(w_1*x_1 + w_2*x_2 + \dots + w_n*x_n + \text{bias})$$

When a single perceptron is enough

- 160. The problem is linearly separable: a straight line (or hyperplane) can divide the two classes
- 161. Logic gates: AND, OR, NOT, NAND
- 162. Simple threshold decisions: “is the score above X?”
- 163. Binary classifiers on data that clusters into two non-overlapping regions

When you need multiple layers

The problem is not linearly separable: no single line can separate the classes

XOR and parity problems

Any task requiring “this but not that” reasoning (combinations of conditions)

Real-world classification: images, text, audio - almost always non-linear

Decision checklist

Plot or inspect your data. Can you draw a single straight boundary between classes?

Yes: single perceptron works

No: you need at least two layers

Can the problem be decomposed into AND/OR of simpler linear decisions?

This decomposition tells you the minimum network structure

$\text{XOR} = (\text{A OR B}) \text{ AND } (\text{NOT (A AND B)}) = 3 \text{ perceptrons in 2 layers}$

For problems with more than two classes, you need one output node per class

The training rule

```
error = expected - predicted
weight_new = weight_old + learning_rate * error * input
bias_new = bias_old + learning_rate * error
```

If the prediction is correct, nothing changes. If wrong, weights shift to reduce the error. This only works for single-layer perceptrons. Multi-layer networks require backpropagation.

Common mistakes

Trying to learn non-linear patterns with a single perceptron (it will never converge)

Setting the learning rate too high (weights oscillate) or too low (training takes forever)

Forgetting the bias term (without it, the decision boundary must pass through the origin)

Confusing perceptron convergence (guaranteed for linearly separable data) with general neural network convergence (not guaranteed)

Quiz

Pre-lesson check

What mathematical operation does a perceptron perform on its inputs before applying the activation function?

- Matrix inversion
- Weighted sum plus bias **(correct)**
- Fourier transform
- Eigenvalue decomposition

Explanation: A perceptron computes the dot product of inputs and weights, adds a bias term, then passes the result through a step function. This weighted sum plus bias is the core computation.

What does 'linearly separable' mean for a classification problem?

1. The data can be sorted in order
2. A single straight line (or hyperplane) can perfectly separate the classes **(correct)**
3. The features have a linear relationship with the label
4. The data has only two dimensions
Explanation: A dataset is linearly separable when you can draw a single hyperplane that perfectly divides the input space into the correct classes. AND and OR are linearly separable; XOR is not.

Post-lesson check

- Why does a single perceptron fail to learn the XOR function?
 5. The learning rate is too low
 6. XOR has too many inputs
 7. XOR is not linearly separable – no single line can separate the classes **(correct)**
 8. The step function prevents gradient flow
Explanation: XOR places [0,1] and [1,0] on one side and [0,0] and [1,1] on the other. No single straight line can separate these groups, so a single perceptron, which can only draw one linear boundary, cannot solve XOR.
- In the perceptron learning rule, what happens when the prediction matches the target?
 - Weights are doubled
 - Weights are set to zero
 - Nothing changes – error is zero so the update is zero **(correct)**
 - The learning rate is halved
Explanation: The update rule is $w_i = w_i + lr \cdot \text{error} \cdot x_i$. When prediction equals target,

error = 0, so all weight updates are zero. The perceptron only adjusts when it makes a mistake.*

- How is XOR solved using multiple perceptrons?

- By using a larger learning rate on a single perceptron
- By combining OR, NAND, and AND perceptrons in two layers (**correct**)
- By adding more inputs to a single perceptron
- By removing the bias term

Explanation: $XOR = (x1 \text{ OR } x2) \text{ AND NOT}(x1 \text{ AND } x2)$. A hidden layer with an OR neuron and a NAND neuron feeds into an output AND neuron, creating a nonlinear decision boundary from linear components.

Multi-Layer Networks and Forward Pass

One neuron draws a line. Stack them, and you can draw anything.

Type: Build **Languages:** Python **Prerequisites:** Phase 01 (Math Foundations), Lesson 03.01 (The Perceptron) **Time:** ~90 minutes

Learning Objectives

- 164. Build a multi-layer network from scratch with Layer and Network classes that perform a complete forward pass
- 165. Trace matrix dimensions through each layer of a network and identify shape mismatches
- 166. Explain how stacking nonlinear activations enables a network to learn curved decision boundaries
- 167. Solve the XOR problem using a 2-2-1 architecture with hand-tuned sigmoid weights

The Problem

A single neuron is a line drawer. That's it. One straight line through your data. Every real problem in AI – image recognition, language understanding, playing Go – requires curves. Stacking neurons into layers is how you get curves.

In 1969, Minsky and Papert proved this limitation was fatal: a single-layer network cannot learn XOR. Not “struggles to learn” – mathematically cannot. The XOR truth table places [0,1] and [1,0] on one side, [0,0] and [1,1] on the other. No single line separates them.

This killed neural network funding for over a decade. The fix was obvious in hindsight: stop using one layer. Stack neurons into layers. Let the first layer carve the input space into new features, and let the second layer combine those features into decisions no single line could make.

That stack is the multi-layer network. It is the foundation of every deep learning model in production today. The forward pass – data flowing from input through hidden layers to output – is the first thing you need to build before anything else works.

The Concept

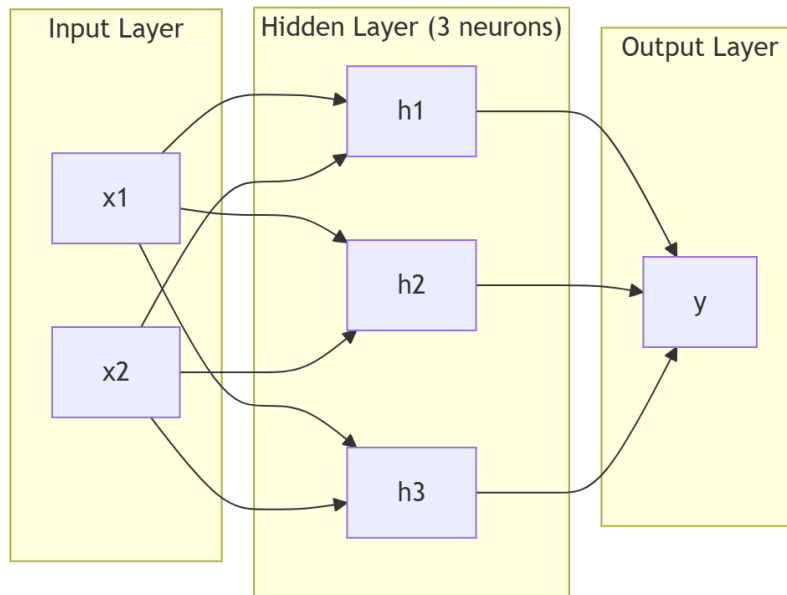
Layers: Input, Hidden, Output

A multi-layer network has three types of layers:

Input layer – not really a layer. It holds your raw data. Two features means two input nodes. No computation happens here.

Hidden layers – where the work happens. Each neuron takes every output from the previous layer, applies weights and a bias, then passes the result through an activation function. “Hidden” because you never see these values directly in the training data.

Output layer – the final answer. For binary classification, one neuron with sigmoid. For multi-class, one neuron per class.



This is a 2-3-1 network. Two inputs, three hidden neurons, one output. Every connection carries a weight. Every neuron (except input) carries a bias.

Each layer produces a vector of numbers called a hidden state. For text, hidden states increase dimensionality – encoding a word as 768 numbers to capture semantic meaning. For images, they reduce dimensionality – compressing millions of pixels into a manageable representation. The hidden state is where the learning lives.

Neurons and Activations

Each neuron does three things:

- Multiply every input by its corresponding weight

- Sum all the products and add a bias

- Pass the sum through an activation function

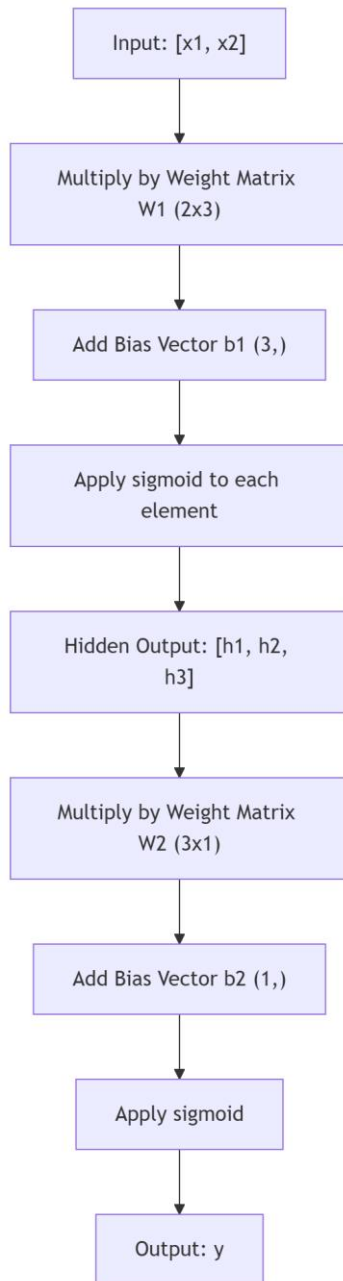
For now, the activation is sigmoid:

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

Sigmoid squashes any number into the range (0, 1). Large positive inputs push toward 1. Large negative inputs push toward 0. Zero maps to 0.5. This smooth curve is what makes learning possible – unlike the perceptron’s hard step, sigmoid has a gradient everywhere.

Forward Pass: How Data Flows

The forward pass pushes input data through the network, layer by layer, until it reaches the output. No learning happens during the forward pass. It is pure computation: multiply, add, activate, repeat.



At each layer, three operations happen in sequence:

$z = W * \text{input} + b$	(linear transformation)
$a = \text{sigmoid}(z)$	(activation)

The output of one layer becomes the input to the next. That is the entire forward pass.

Matrix Dimensions

Tracking dimensions is the single most important debugging skill in deep learning. Here is the 2-3-1 network:

Step	Operation	Dimensions	Result Shape
Input	x	–	(2,)
Hidden linear	$W1 * x + b1$	$W1: (3, 2), b1: (3,)$	(3,)
Hidden activation	$\text{sigmoid}(z1)$	–	(3,)
Output linear	$W2 * h + b2$	$W2: (1, 3), b2: (1,)$	(1,)
Output activation	$\text{sigmoid}(z2)$	–	(1,)

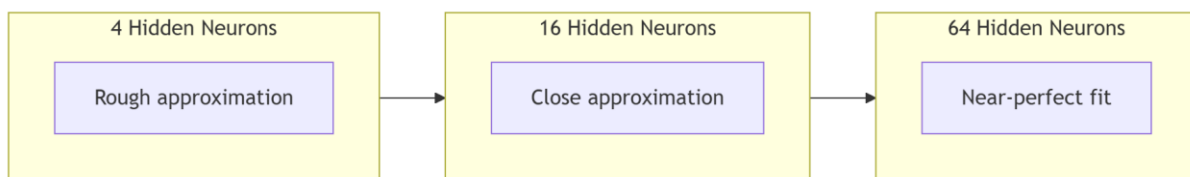
The rule: weight matrix W at layer k has shape (neurons_in_layer_k, neurons_in_layer_k_minus_1). Rows match the current layer. Columns match the previous layer. If the shapes do not line up, you have a bug.

Universal Approximation Theorem

In 1989, George Cybenko proved something remarkable: a neural network with a single hidden layer and enough neurons can approximate any continuous function to any desired accuracy.

This does not mean one hidden layer is always best. It means the architecture is theoretically capable. In practice, deeper networks (more layers, fewer neurons per layer) learn the same functions with far fewer total parameters than shallow-wide networks. That is why deep learning works.

The intuition: each neuron in the hidden layer learns one “bump” or feature. Enough bumps placed in the right locations can approximate any smooth curve. More neurons, more bumps, better approximation.



Composability

Neural networks are composable. You can stack them, chain them, run them in parallel. A Whisper model uses an encoder network to process audio and a separate decoder network to generate text. Modern LLMs are decoder-only. BERT is encoder-only. T5 is encoder-decoder. The architecture choice defines what the model can do.

Build It

Pure Python. No numpy. Every matrix operation written from scratch.

Step 1: Sigmoid Activation

```
import math

def sigmoid(x):
    x = max(-500.0, min(500.0, x))
    return 1.0 / (1.0 + math.exp(-x))
```

The clamp to $[-500, 500]$ prevents overflow. `math.exp(500)` is large but finite. `math.exp(1000)` is ∞ .

Step 2: Layer Class

The most important operation in all of deep learning is matrix multiplication. Every layer, every attention head, every forward pass – it's matmuls all the way down. A linear layer takes an input vector, multiplies it by a weight matrix, and adds a bias vector: $y = Wx + b$. That single equation is 90% of the compute in a neural network.

A layer holds a weight matrix and a bias vector. Its forward method takes an input vector and returns the activated output.

```
class Layer:
    def __init__(self, n_inputs, n_neurons, weights=None, biases=None):
        if weights is not None:
            self.weights = weights
        else:
            import random
            self.weights = [
                [random.uniform(-1, 1) for _ in range(n_inputs)]
                for _ in range(n_neurons)
            ]
        if biases is not None:
            self.biases = biases
        else:
            self.biases = [0.0] * n_neurons

    def forward(self, inputs):
        self.last_input = inputs
        self.last_output = []
        for neuron_idx in range(len(self.weights)):
            z = sum(
                w * x for w, x in zip(self.weights[neuron_idx], inputs)
            )
            z += self.biases[neuron_idx]
            self.last_output.append(sigmoid(z))
        return self.last_output
```

The weight matrix has shape (n_neurons, n_inputs). Each row is one neuron's weights across all inputs. The forward method loops through neurons, computes the weighted sum plus bias, applies sigmoid, and collects the results.

Step 3: Network Class

A network is a list of layers. The forward pass chains them: output of layer k feeds into layer k+1.

```
class Network:
    def __init__(self, layers):
        self.layers = layers

    def forward(self, inputs):
        current = inputs
        for layer in self.layers:
            current = layer.forward(current)
        return current
```

That is the entire forward pass. Four lines of logic. Data goes in, flows through every layer, comes out the other side.

Step 4: XOR with Hand-Tuned Weights

In Lesson 01, we solved XOR by combining OR, NAND, and AND perceptrons. Now do the same thing with our Layer and Network classes. The 2-2-1 architecture: two inputs, two hidden neurons, one output.

```
hidden = Layer(
    n_inputs=2,
    n_neurons=2,
    weights=[[20.0, 20.0], [-20.0, -20.0]],
    biases=[-10.0, 30.0],
)

output = Layer(
    n_inputs=2,
    n_neurons=1,
    weights=[[20.0, 20.0]],
    biases=[-30.0],
)

xor_net = Network([hidden, output])

xor_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 0),
]

for inputs, expected in xor_data:
```

```

result = xor_net.forward(inputs)
predicted = 1 if result[0] >= 0.5 else 0
print(f" {inputs} -> {result[0]:.6f} (rounded: {predicted}, expected: {expected})")

```

The large weights (20, -20) make sigmoid act like a step function. The first hidden neuron approximates OR. The second approximates NAND. The output neuron combines them into AND, which is XOR.

Step 5: Circle Classification

A harder problem: classify 2D points as inside or outside a circle of radius 0.5 centered at the origin. This requires a curved decision boundary – impossible for a single perceptron.

```

import random
import math

random.seed(42)

data = []
for _ in range(200):
    x = random.uniform(-1, 1)
    y = random.uniform(-1, 1)
    label = 1 if (x * x + y * y) < 0.25 else 0
    data.append([x, y], label))

circle_net = Network([
    Layer(n_inputs=2, n_neurons=8),
    Layer(n_inputs=8, n_neurons=1),
])

```

With random weights, the network will not classify well. But the forward pass still runs. This is the point – the forward pass is just computation. Learning the right weights is backpropagation, coming in Lesson 03.

```

correct = 0
for inputs, expected in data:
    result = circle_net.forward(inputs)
    predicted = 1 if result[0] >= 0.5 else 0
    if predicted == expected:
        correct += 1

print(f"Accuracy with random weights: {correct}/{len(data)} ({100*correct/len(data):.1f}%)")

```

Random weights give poor accuracy – often worse than guessing the majority class. After training (Lesson 03), this same architecture with 8 hidden neurons will draw a curved boundary that separates inside from outside.

Use It

PyTorch does everything above in four lines:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 8),
    nn.Sigmoid(),
    nn.Linear(8, 1),
    nn.Sigmoid(),
)

x = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
output = model(x)
print(output)
```

`nn.Linear(2, 8)` is your Layer class: weight matrix of shape (8, 2), bias vector of shape (8,).
`nn.Sigmoid()` is your sigmoid function applied element-wise. `nn.Sequential` is your Network class: chain layers in order.

The difference is speed and scale. PyTorch runs on GPUs, handles batches of millions of samples, and automatically computes gradients for backpropagation. But the forward pass logic is identical to what you just built from scratch.

Exercises

52. Build a 2-4-2-1 network (two hidden layers) and run the forward pass on XOR data with random weights. Print the intermediate hidden layer outputs to see how the representation transforms at each layer.
53. Change the hidden layer size in the circle classifier from 8 to 2, then to 32. Run the forward pass with random weights each time. Does the number of hidden neurons change the output range or distribution? Why?
54. Implement a `count_parameters` method on the Network class that returns the total number of trainable weights and biases. Test it on a 784-256-128-10 network (the classic MNIST architecture). How many parameters does it have?
55. Build a forward pass for a 3-4-4-2 network. Feed it RGB color values (normalized to 0-1) and observe the two outputs. This is the architecture for a simple color classifier with two classes.
56. Replace sigmoid with a “leaky step” function: return $0.01 * z$ if $z < 0$, else 1.0. Run the forward pass on XOR with the same hand-tuned weights from Step 4. Does it still work? Why is the smooth sigmoid preferred over hard cutoffs?

Key Terms

Term	What people say	What it actually means
Forward pass	“Running the model”	Pushing input through every layer – multiply by weights, add bias, activate – to produce an output
Hidden layer	“The middle part”	Any layer between input and output whose values are not directly observed in the data
Multi-layer network	“A deep neural network”	Layers of neurons stacked sequentially, where each layer’s output feeds the next layer’s input
Activation function	“The nonlinearity”	A function applied after the linear transformation that introduces curves into the decision boundary
Sigmoid	“The S-curve”	$\sigma(z) = 1/(1+e^{(-z)})$, squashes any real number to (0,1), smooth and differentiable everywhere
Weight matrix	“The parameters”	A matrix W of shape (current_layer_neurons, previous_layer_neurons) containing learnable connection strengths
Bias vector	“The offset”	A vector added after the matrix multiply that lets neurons activate even when all inputs are zero
Universal approximation	“Neural nets can learn anything”	A single hidden layer with enough neurons can approximate any continuous function – but “enough” can mean billions
Linear transformation	“The matrix multiply step”	$z = W * x + b$, the computation before activation, which maps inputs to a new space
Decision boundary	“Where the classifier switches”	The surface in input space where the network output crosses the classification threshold

You are a neural network architecture advisor. Your job is to recommend a network structure – number of layers, neurons per layer, and activation functions – for a specific problem.

When a user describes their problem, ask clarifying questions if needed, then recommend a concrete architecture. Structure your response as:

Recommended architecture (layer sizes as a list, e.g., [784, 256, 128, 10])

Activation functions for each layer and why

Total parameter count

Why this depth and width

What to try if it does not work

Use this decision framework:

Binary classification (yes/no, spam/not-spam, inside/outside): - Output layer: 1 neuron with sigmoid - Start with one hidden layer. Neurons = 2x to 4x the input dimension. - Architecture: $[n_features, 4*n_features, 1]$ - If accuracy plateaus, add a second hidden layer at half the width of the first.

Multi-class classification (digits 0-9, object categories): - Output layer: one neuron per class with softmax - Start with two hidden layers. First = 2x inputs, second = half the first. - Architecture: $[n_features, 2*n_features, n_features, n_classes]$ - For image inputs (e.g., 784 pixels): $[784, 256, 128, n_classes]$

Regression (predict a continuous number): - Output layer: 1 neuron with no activation (linear output) - Same hidden layer strategy as classification - Architecture: $[n_features, 4n_features, 2n_features, 1]$

Tabular data (structured rows and columns): - Shallow networks work best. 1-3 hidden layers. - Width: 64 to 256 neurons per layer. - Activation: ReLU for hidden layers. - Regularization matters more than depth.

Image data: - Use convolutional layers, not fully connected (covered in later lessons). - If forced to use fully connected: flatten the image and use $[n_pixels, 512, 256, n_classes]$. - This is wasteful. Convolutions share weights and respect spatial structure.

Sequence data (text, time series): - Use recurrent or transformer architectures (covered in later lessons). - If forced to use fully connected: treat the sequence as a flat vector. Results will be poor.

Activation function selection: - Hidden layers: ReLU is the default. Use it unless you have a reason not to. - Output layer for binary classification: sigmoid (squashes to 0-1 probability). - Output layer for multi-class: softmax (squashes to probability distribution). - Output layer for regression: no activation (linear). - Sigmoid in hidden layers: avoid unless the problem specifically needs outputs bounded in (0,1). Causes vanishing gradients in deep networks.

Sizing heuristics: - Total parameters should be 5x to 10x the number of training samples to avoid overfitting without regularization. - More data allows more parameters. - When in doubt, start too small and increase. An overfit model tells you the architecture can learn. An underfit model gives you nothing.

Common mistakes to flag: - Too many layers for small datasets. Two hidden layers handle most tabular problems. - Using sigmoid in every hidden layer. Switch to ReLU. - Output layer mismatch: sigmoid for multi-class (should be softmax) or softmax for binary (should be sigmoid). - No activation between layers. Without activation, stacking layers collapses to a single linear transformation. - Width too narrow in early layers. The first hidden layer should be wider than the input to create a richer representation.

Parameter count formula: - For a fully connected layer from n_in to n_out : $(n_in * n_out) + n_out$ parameters. - Total = sum across all layers. - Example: $[784, 256, 10] = (784*256 + 256) + (256*10 + 10) = 203,530$ parameters.

When the user's problem does not fit any category above, ask: 1. What are the inputs? (dimensions, type: image/tabular/sequence) 2. What is the output? (binary, multi-class, continuous) 3. How much training data do you have? 4. What is your compute budget? (laptop CPU, GPU, cloud)

Then apply the heuristics and recommend a starting architecture they can iterate on.

Quiz

Pre-lesson check

What is the purpose of a hidden layer in a multi-layer network?

- To store the training data
- To transform inputs into new feature representations that enable nonlinear decision boundaries **(correct)**
- To reduce the number of parameters

- To apply the loss function

Explanation: Hidden layers apply weights, biases, and activation functions to create new feature representations. These intermediate features allow the network to learn nonlinear mappings that a single layer cannot.

What does the forward pass do in a neural network?

1. Updates the weights using gradients
2. Computes gradients for backpropagation
3. Pushes input through each layer to produce an output **(correct)**

4. Shuffles the training data

Explanation: The forward pass is pure computation: for each layer, multiply by weights, add bias, apply activation, and pass the result to the next layer. No learning happens during the forward pass.

Post-lesson check

- For a layer with 3 neurons receiving input from 2 neurons, what is the shape of the weight matrix?
 5. (2, 3)
 6. (3, 2) **(correct)**
 7. (3, 3)
 8. (2, 2)

Explanation: The weight matrix has shape (neurons_in_current_layer, neurons_in_previous_layer) = (3, 2). Each row contains one neuron's weights across all inputs.
- What does the Universal Approximation Theorem guarantee?
 - Any network can be trained in polynomial time
 - A single hidden layer with enough neurons can approximate any continuous function **(correct)**
 - Deeper networks always outperform shallow ones
 - Neural networks can solve any computational problem

Explanation: Cybenko (1989) proved that a network with one hidden layer and sufficient neurons can approximate any continuous function to any desired accuracy. In practice, deeper networks achieve the same with fewer total parameters.
- Why does the sigmoid activation function make learning possible, unlike the step function used in perceptrons?
 9. Sigmoid outputs larger values
 10. Sigmoid is faster to compute
 11. Sigmoid is smooth and differentiable everywhere, so gradients exist for backpropagation **(correct)**

12. Sigmoid outputs negative values

*Explanation: The step function has zero gradient almost everywhere and undefined gradient at the threshold. Sigmoid is smooth with a well-defined derivative ($s(1-s)$) at every point, which is essential for gradient-based learning.**

Backpropagation from Scratch

Backpropagation is the algorithm that makes learning possible. Without it, neural networks are just expensive random number generators.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.02 (Multi-Layer Networks) **Time:** ~120 minutes

Learning Objectives

Implement a Value-based autograd engine that builds a computational graph and computes gradients via topological sort

Derive the backward pass for addition, multiplication, and sigmoid using the chain rule

Train a multi-layer network on XOR and circle classification using only your from-scratch backpropagation engine

Identify the vanishing gradient problem in deep sigmoid networks and explain why gradients shrink exponentially

The Problem

Your network has a single hidden layer with 768 inputs and 3072 outputs. That's 2,359,296 weights. It made a wrong prediction. Which weights caused the error? Testing each weight individually means 2.3 million forward passes. Backpropagation computes all 2.3 million gradients in a single backward pass. That's not an optimization. That's the difference between trainable and impossible.

The naive approach: take one weight, nudge it by a tiny amount, run the forward pass again, measure whether the loss went up or down. That gives you the gradient for that weight. Now do it for every weight in the network. Multiply by thousands of training steps and millions of data points. You'd need geological time to train anything useful.

Backpropagation solves this. One forward pass, one backward pass, all gradients computed. The trick is the chain rule from calculus, applied systematically to a computational graph. This is the algorithm that made deep learning practical. Without it, we'd still be stuck on toy problems.

The Concept

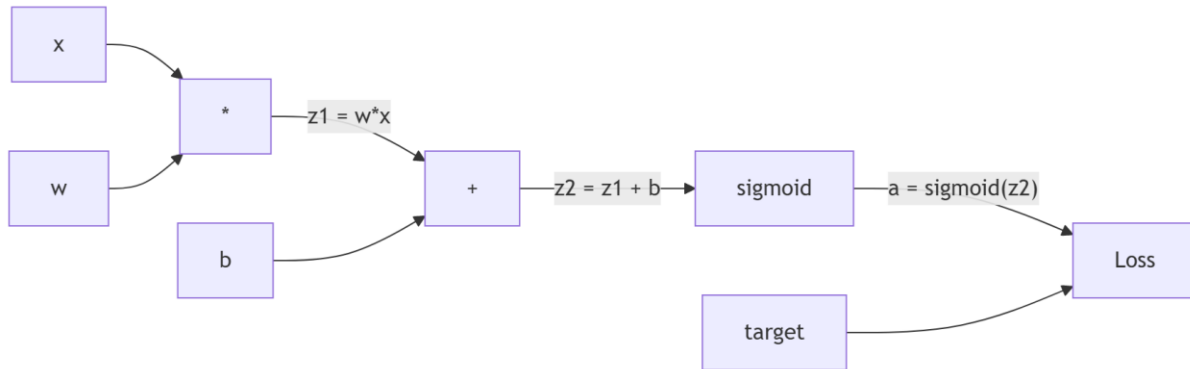
The Chain Rule, Applied to Networks

You saw the chain rule in Phase 01, Lesson 05. Quick recap: if $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$. You multiply derivatives along the chain.

In a neural network, the "chain" is the sequence of operations from input to loss. Each layer applies weights, adds biases, passes through an activation. The loss function compares the final output to the target. Backpropagation traces this chain backward, computing how each operation contributed to the error.

Computational Graphs

Every forward pass builds a graph. Each node is an operation (multiply, add, sigmoid). Each edge carries a value forward and a gradient backward.

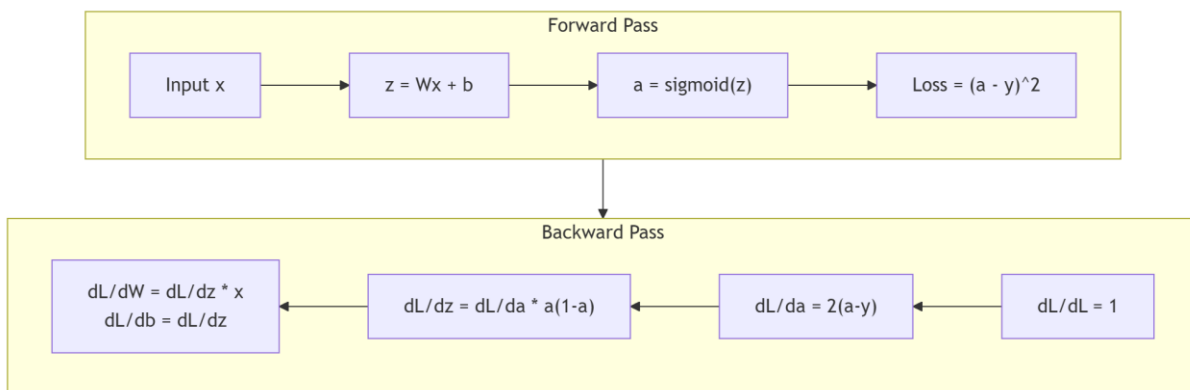


Forward pass: values flow left to right. x and w produce $z1 = w * x$. Add b to get $z2$. Sigmoid gives activation a . Compare a to target y using the loss function.

Backward pass: gradients flow right to left. Start with dL/da (how loss changes with the activation). Multiply by $da/dz2$ (sigmoid derivative). That gives $dL/dz2$. Split into dL/db (which equals $dL/dz2$, since $z2 = z1 + b$) and $dL/dz1$. Then $dL/dw = dL/dz1 * x$ and $dL/dx = dL/dz1 * w$.

Every node in the graph has one job during the backward pass: take the gradient coming from above, multiply by its local derivative, and pass it down.

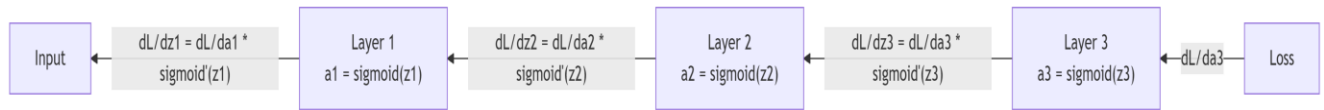
Forward vs Backward



The forward pass stores every intermediate value: z , a , the inputs to each layer. The backward pass needs these stored values to compute gradients. This is the memory-computation tradeoff at the heart of backprop. You trade memory (storing activations) for speed (one pass instead of millions).

Gradient Flow Through a Network

For a 3-layer network, gradients chain through every layer:



At each layer, the gradient gets multiplied by the sigmoid derivative. The sigmoid derivative is $a * (1 - a)$, which maxes out at 0.25 (when $a = 0.5$). Three layers deep, the gradient has been multiplied by at most $0.25^3 = 0.0156$. Ten layers deep: $0.25^{10} = 0.000001$.

Vanishing Gradients

This is the vanishing gradient problem. Sigmoid squashes its output between 0 and 1. Its derivative is always less than 0.25. Stack enough sigmoid layers and gradients shrink to nothing. Early layers barely learn because they receive near-zero gradients.

```

sigmoid(z):      Output range [0, 1]
sigmoid'(z):     Max value 0.25 (at z = 0)

After 5 layers:  gradient * 0.25^5 = 0.001x original
After 10 layers: gradient * 0.25^10 = 0.000001x original
  
```

This is why deep sigmoid networks are nearly impossible to train. The fix – ReLU and its variants – is the subject of Lesson 04. For now, understand that backprop works perfectly. The problem is what it's working through.

Deriving Gradients for a 2-Layer Network

Concrete math for a network with input x , hidden layer with sigmoid, output layer with sigmoid, and MSE loss.

Forward pass:

```

z1 = W1 * x + b1
a1 = sigmoid(z1)
z2 = W2 * a1 + b2
a2 = sigmoid(z2)
L = (a2 - y)^2
  
```

Backward pass (applying chain rule step by step):

```

dL/da2 = 2(a2 - y)
da2/dz2 = a2 * (1 - a2)
dL/dz2 = dL/da2 * da2/dz2 = 2(a2 - y) * a2 * (1 - a2)

dL/dW2 = dL/dz2 * a1
dL/db2 = dL/dz2
  
```

```

dL/da1 = dL/dz2 * w2
da1/dz1 = a1 * (1 - a1)
dL/dz1 = dL/da1 * da1/dz1

dL/dw1 = dL/dz1 * x
dL/db1 = dL/dz1

```

Every gradient is a product of local derivatives traced back from the loss. That's all backpropagation is.

Build It

Step 1: The Value Node

Every number in our computation becomes a Value. It stores its data, its gradient, and how it was created (so it knows how to compute gradients backward).

```

class Value:
    def __init__(self, data, children=(), op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._children = set(children)
        self._op = op

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"

```

No gradient yet (0.0). No backward function yet (no-op). The `_children` track which Values produced this one, so we can topologically sort the graph later.

Step 2: Operations with Backward Functions

Each operation creates a new Value and defines how gradients flow backward through it.

```

def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), '+')

    def _backward():
        self.grad += out.grad
        other.grad += out.grad

    out._backward = _backward
    return out

def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')

```

```
def _backward():
    self.grad += other.data * out.grad
    other.grad += self.data * out.grad

out._backward = _backward
return out
```

For addition: $d(a+b)/da = 1$, $d(a+b)/db = 1$. So both inputs get the output's gradient directly.

For multiplication: $d(ab)/da = b$, $d(ab)/db = a$. Each input gets the other's value times the output gradient.

The += is critical. A Value might be used in multiple operations. Its gradient is the sum of gradients from all paths.

Step 3: Sigmoid and Loss

```
import math

def sigmoid(self):
    x = self.data
    x = max(-500, min(500, x))
    s = 1.0 / (1.0 + math.exp(-x))
    out = Value(s, (self,), 'sigmoid')

    def _backward():
        self.grad += (s * (1 - s)) * out.grad

    out._backward = _backward
    return out
```

Sigmoid derivative: $\text{sigmoid}(x) * (1 - \text{sigmoid}(x))$. We computed $\text{sigmoid}(x) = s$ during the forward pass. Reuse it. No extra work.

```
def mse_loss(predicted, target):
    diff = predicted + Value(-target)
    return diff * diff
```

MSE for a single output: $(\text{predicted} - \text{target})^2$. We express subtraction as addition with a negated Value.

Step 4: Backward Pass

Topological sort ensures we process nodes in the right order – a node's gradient is fully accumulated before we propagate through it.

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
```

```

        if v not in visited:
            visited.add(v)
            for child in v._children:
                build_topo(child)
            topo.append(v)

    build_topo(self)
    self.grad = 1.0
    for v in reversed(topo):
        v._backward()

```

Start at the loss (gradient = 1.0, since $dL/dL = 1$). Walk backward through the sorted graph. Each node's `_backward` pushes gradients to its children.

Step 5: Layer and Network

```

import random

class Neuron:
    def __init__(self, n_inputs):
        scale = (2.0 / n_inputs) ** 0.5
        self.weights = [Value(random.uniform(-scale, scale)) for _ in range(n_inputs)]
        self.bias = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.weights, x)), self.bias)
        return act.sigmoid()

    def parameters(self):
        return self.weights + [self.bias]

class Layer:
    def __init__(self, n_inputs, n_outputs):
        self.neurons = [Neuron(n_inputs) for _ in range(n_outputs)]

    def __call__(self, x):
        out = [n(x) for n in self.neurons]
        return out[0] if len(out) == 1 else out

    def parameters(self):
        params = []
        for n in self.neurons:
            params.extend(n.parameters())
        return params

class Network:
    def __init__(self, sizes):
        self.layers = []
        for i in range(len(sizes) - 1):
            self.layers.append(Layer(sizes[i], sizes[i + 1]))

```

```

def __call__(self, x):
    for layer in self.layers:
        x = layer(x)
        if not isinstance(x, list):
            x = [x]
    return x[0] if len(x) == 1 else x

def parameters(self):
    params = []
    for layer in self.layers:
        params.extend(layer.parameters())
    return params

def zero_grad(self):
    for p in self.parameters():
        p.grad = 0.0

```

A Neuron takes inputs, computes weighted sum + bias, and applies sigmoid. Weight initialization scales by $\sqrt{2/n_inputs}$ to prevent sigmoid saturation in deeper networks. A Layer is a list of Neurons. A Network is a list of Layers. The `parameters()` method collects all learnable Values so we can update them.

Step 6: Train on XOR

```

random.seed(42)
net = Network([2, 4, 1])

xor_data = [
    ([0.0, 0.0], 0.0),
    ([0.0, 1.0], 1.0),
    ([1.0, 0.0], 1.0),
    ([1.0, 1.0], 0.0),
]

learning_rate = 1.0

for epoch in range(1000):
    total_loss = Value(0.0)
    for inputs, target in xor_data:
        x = [Value(i) for i in inputs]
        pred = net(x)
        loss = mse_loss(pred, target)
        total_loss = total_loss + loss

    net.zero_grad()
    total_loss.backward()

    for p in net.parameters():
        p.data -= learning_rate * p.grad

    if epoch % 100 == 0:

```

```

        print(f"Epoch {epoch:4d} | Loss: {total_loss.data:.6f}")

print("\nXOR Results:")
for inputs, target in xor_data:
    x = [Value(i) for i in inputs]
    pred = net(x)
    print(f" {inputs} -> {pred.data:.4f} (expected {target})")

```

Watch the loss decrease. From random predictions to correct XOR outputs, driven entirely by backpropagation computing gradients and nudging weights in the right direction.

Step 7: Circle Classification

In Lesson 02, you hand-tuned weights for circle classification. Now let the network learn them.

```

random.seed(7)

def generate_circle_data(n=100):
    data = []
    for _ in range(n):
        x1 = random.uniform(-1.5, 1.5)
        x2 = random.uniform(-1.5, 1.5)
        label = 1.0 if x1 * x1 + x2 * x2 < 1.0 else 0.0
        data.append([x1, x2], label)
    return data

circle_data = generate_circle_data(80)

circle_net = Network([2, 8, 1])
learning_rate = 0.5

for epoch in range(2000):
    random.shuffle(circle_data)
    total_loss_val = 0.0
    for inputs, target in circle_data:
        x = [Value(i) for i in inputs]
        pred = circle_net(x)
        loss = mse_loss(pred, target)
        circle_net.zero_grad()
        loss.backward()
        for p in circle_net.parameters():
            p.data -= learning_rate * p.grad
        total_loss_val += loss.data

    if epoch % 200 == 0:
        correct = 0
        for inputs, target in circle_data:
            x = [Value(i) for i in inputs]
            pred = circle_net(x)
            predicted_class = 1.0 if pred.data > 0.5 else 0.0
            if predicted_class == target:

```

```

        correct += 1
    accuracy = correct / len(circle_data) * 100
    print(f"Epoch {epoch:4d} | Loss: {total_loss_val:.4f} | Accuracy: {accuracy:.1f}%")

```

We use online SGD here – update weights after each sample instead of accumulating the full batch. This breaks symmetry faster and avoids sigmoid saturation on the full loss landscape. Shuffling the data each epoch prevents the network from memorizing the order.

No hand-tuning. The network discovers the circular decision boundary on its own. That’s the power of backpropagation: you define the architecture, the loss function, and the data. The algorithm figures out the weights.

Use It

PyTorch does everything above in a few lines. The core idea is identical – autograd builds a computational graph during the forward pass and traces it backward to compute gradients.

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid(),
)
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)
criterion = nn.MSELoss()

X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
y = torch.tensor([[0],[1],[1],[0]], dtype=torch.float32)

for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("PyTorch XOR Results:")
with torch.no_grad():
    for i in range(4):
        pred = model(X[i])
        print(f" {X[i].tolist()} -> {pred.item():.4f} (expected {y[i].item()})")

```

`loss.backward()` is your `total_loss.backward()`. `optimizer.step()` is your manual `p.data -= lr * p.grad`. `optimizer.zero_grad()` is your `net.zero_grad()`. Same algorithm, industrial-strength implementation. PyTorch handles GPU acceleration, mixed precision,

gradient checkpointing, and hundreds of layer types. But the backward pass is the same chain rule applied to the same computational graph.

Training runs the forward pass, then the backward pass, then updates weights. Inference runs only the forward pass. No gradients, no updates. This distinction matters because inference is what happens in production. When you call an API like Claude or GPT, you're running inference – your prompt flows forward through the network, and tokens come out the other end. No weights change. Understanding backprop matters because it shaped every weight in that network.

Exercises

Add a `__sub__` method to the Value class ($a - b = a + (-1 * b)$). Then implement a `__neg__` method. Verify that the gradients are correct by comparing with manual calculation for a simple expression like $(a - b)^2$.

Add a `relu` method to Value (output $\max(0, x)$, derivative is 1 if $x > 0$, else 0). Replace sigmoid with `relu` in the hidden layers and train on XOR again. Compare convergence speed. You should see faster training – this previews Lesson 04.

Implement a `__pow__` method on Value for integer powers. Use it to replace `mse_loss` with a proper $(\text{predicted} - \text{target}) ** 2$ expression. Verify gradients match the original implementation.

Add gradient clipping to the training loop: after calling `backward()`, clip all gradients to $[-1, 1]$. Train a deeper network (4+ layers with sigmoid) and compare loss curves with and without clipping. This is your first defense against exploding gradients.

Build a visualization: after training on XOR, print the gradient of every parameter in the network. Identify which layer has the smallest gradients. This demonstrates the vanishing gradient problem you read about in the Concept section.

Key Terms

Term	What people say	What it actually means
Backpropagation	"The network learns"	An algorithm that computes dL/dw for every weight by applying the chain rule backward through the computational graph
Computational graph	"The network structure"	A directed acyclic graph where nodes are operations and edges carry values (forward) and gradients (backward)
Chain rule	"Multiply the derivatives"	If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$ – the mathematical foundation of backpropagation
Gradient	"The direction of steepest ascent"	The partial derivative of the loss with respect to a parameter – tells you how to change that parameter to reduce the loss

Term	What people say	What it actually means
Vanishing gradient	“Deep networks don’t learn”	Gradients shrink exponentially as they propagate through layers with saturating activations like sigmoid
Forward pass	“Running the network”	Computing the output from inputs by sequentially applying each layer’s operations and storing intermediate values
Backward pass	“Computing gradients”	Traversing the computational graph in reverse, accumulating gradients at each node using the chain rule
Learning rate	“How fast it learns”	A scalar that controls the step size when updating weights: $w_{\text{new}} = w_{\text{old}} - \text{lr} * \text{gradient}$
Topological sort	“The right order”	An ordering of graph nodes where each node appears after all nodes it depends on – ensures gradients are fully accumulated before propagation
Autograd	“Automatic differentiation”	A system that builds computational graphs during forward computation and automatically computes gradients – what PyTorch’s engine does

You are a neural network gradient debugger. I will describe a training problem and you will systematically diagnose the root cause and suggest fixes.

Diagnostic Protocol

When I describe a gradient issue, follow this sequence:

1. Classify the Symptom

Determine which category the problem falls into:

Vanishing gradients: Loss plateaus early, early layers have near-zero gradients, deep layers learn but shallow layers don’t

Exploding gradients: Loss shoots to ∞ , weights become NaN, training diverges after a few steps

NaN gradients: Loss becomes NaN, specific layers produce NaN outputs, appears suddenly during training

Dead neurons: Gradients are exactly zero (not just small), specific neurons never activate, loss stops improving

2. Check the Usual Suspects (in order)

For vanishing gradients: - Activation function (sigmoid/tanh in deep networks saturate – switch to ReLU/GELU) - Learning rate too low (gradients exist but updates are too small to matter) - Weight

initialization (too small initial weights compound the shrinking) - Network too deep for the activation choice - Batch normalization missing between layers

For exploding gradients: - Learning rate too high - Weight initialization too large - No gradient clipping (add `torch.nn.utils.clip_grad_norm_`) - Skip connections missing in deep networks - Loss function scale (reduction='sum' vs 'mean')

For NaN gradients: - Division by zero in loss function (add ϵ : $\log(x + 1e-8)$) - Numerical overflow in `exp()` (clamp inputs to sigmoid/softmax) - Learning rate too high causing weight overflow - Zero-length vectors in normalization - `Inf * 0` in masked operations

For dead neurons: - ReLU with negative initialization (neurons start dead and stay dead) - Learning rate too high pushed weights past recovery - Use Leaky ReLU, ELU, or GELU instead of vanilla ReLU - Check weight initialization (He init for ReLU, Xavier for sigmoid/tanh)

3. Provide Diagnostic Code

Give me specific code to run that will reveal the problem:

```
for name, param in model.named_parameters():
    if param.grad is not None:
        grad_mean = param.grad.abs().mean().item()
        grad_max = param.grad.abs().max().item()
        print(f"{name:40s} | mean: {grad_mean:.2e} | max: {grad_max:.2e}")
```

4. Suggest Fixes (ranked by likelihood)

List fixes from most likely to work to least likely. For each fix: - What to change - Why it fixes the problem - Expected impact on training

Input Format

Describe your problem with: - Network architecture (layers, activations, depth) - Loss function - Optimizer and learning rate - What you observe (loss curve, gradient magnitudes, specific error messages) - How many epochs before the problem appears

Output Format

Diagnosis: One sentence naming the root cause

Evidence: What in your description points to this cause

Fix: Code changes to apply, ranked by likelihood

Verification: How to confirm the fix worked

Quiz

Pre-lesson check

What is the chain rule in the context of neural networks?

- A rule for chaining layers together
 - If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$ – multiply derivatives along the path **(correct)**
 - A method for initializing weights
 - A technique for batching data
- Explanation: The chain rule lets you compute the derivative of a composite function by multiplying the local derivatives at each step. Backpropagation applies this systematically through the computational graph.*

Why is backpropagation more efficient than computing each gradient independently?

- It uses less memory
 - It computes all gradients in a single backward pass instead of one forward pass per parameter **(correct)**
 - It only works on small networks
 - It avoids using the chain rule
- Explanation: Computing gradients independently requires one forward pass per parameter (millions of passes for a large network). Backpropagation computes all gradients in one backward pass by reusing intermediate values stored during the forward pass.*

Post-lesson check

168. In the backward pass, why do we use '+=' instead of '=' when accumulating gradients?

- It's a Python convention

- A value might be used in multiple operations, so its gradient is the sum of gradients from all paths **(correct)**
- It prevents overflow
- It makes the code run faster
Explanation: When a Value is used as input to multiple operations (e.g., x used in both xw1 and xw2), its total gradient is the sum of the gradients flowing back from each operation. Using += accumulates these correctly.

169. What causes the vanishing gradient problem in deep sigmoid networks?

1. The learning rate is too small
2. Sigmoid's derivative has a maximum of 0.25, so gradients shrink exponentially through layers **(correct)**
3. The network has too many parameters
4. The loss function is poorly chosen
Explanation: The sigmoid derivative peaks at 0.25 (when $z=0$). Each layer multiplies the gradient by at most 0.25, so after 10 layers the gradient is at most $0.25^{10} = \sim 0.000001$ of the original signal.

170. Why does topological sort matter in the backward pass?

- It makes the code cleaner
- It ensures each node's gradient is fully accumulated before propagating to its children **(correct)**
- It speeds up the forward pass
- It reduces memory usage
Explanation: Topological sort ensures we process nodes in the correct order: a node's gradient must be fully accumulated from all downstream paths before we propagate through it. Without this ordering, gradients would be incomplete.

Activation Functions

Without nonlinearity, your 100-layer network is a fancy matrix multiply. Activations are the gates that let neural networks think in curves.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.03 (Backpropagation) **Time:** ~75 minutes

Learning Objectives

171. Implement sigmoid, tanh, ReLU, Leaky ReLU, GELU, Swish, and softmax with their derivatives from scratch
172. Diagnose the vanishing gradient problem by measuring activation magnitudes through 10+ layers with different activations
173. Detect dead neurons in a ReLU network and explain why GELU avoids this failure mode
174. Select the correct activation function for a given architecture (transformer, CNN, RNN, output layer)

The Problem

Stack two linear transformations: $y = W_2(W_1x + b_1) + b_2$. Expand it: $y = W_2W_1x + W_2b_1 + b_2$. That's just $y = Ax + c$ – a single linear transformation. No matter how many linear layers you stack, the result collapses to one matrix multiply. Your 100-layer network has the same representational power as a single layer.

This is not a theoretical curiosity. It means a deep linear network literally cannot learn XOR, cannot classify a spiral dataset, cannot recognize a face. Without activation functions, depth is an illusion.

Activation functions break the linearity. They warp the output of each layer through a nonlinear function, giving the network the ability to bend decision boundaries, approximate arbitrary functions, and actually learn. But pick the wrong activation and your gradients vanish to zero (sigmoid in deep networks), explode to ∞ (unbounded activations without careful initialization), or your neurons die permanently (ReLU with large negative biases). The choice of activation function directly determines whether your network learns at all.

The Concept

Why Nonlinearity Is Necessary

Matrix multiplication is composable. Multiplying a vector by matrix A then matrix B is identical to multiplying by AB. This means stacking ten linear layers is mathematically equivalent to one linear layer with one big matrix. All those parameters, all that depth – wasted. You need something to break the chain. That's what activation functions do.

Here is the proof. A linear layer computes $f(x) = Wx + b$. Stack two:

```
Layer 1: h = W1 * x + b1
Layer 2: y = W2 * h + b2
```

Substitute:

```
y = W2 * (W1 * x + b1) + b2
y = (W2 * W1) * x + (W2 * b1 + b2)
y = A * x + c
```

One layer. Insert a nonlinear activation $g()$ between layers:

```
h = g(W1 * x + b1)
y = W2 * h + b2
```

Now the substitution breaks. $W2 * g(W1 * x + b1) + b2$ cannot be reduced to a single linear transformation. The network can represent nonlinear functions. Each additional layer with an activation adds representational capacity.

Sigmoid

The original activation function for neural networks.

```
sigmoid(x) = 1 / (1 + e^(-x))
```

Output range: (0, 1). Smooth, differentiable, maps any real number to a probability-like value.

The derivative:

```
sigmoid'(x) = sigmoid(x) * (1 - sigmoid(x))
```

The maximum value of this derivative is 0.25, occurring at $x = 0$. In backpropagation, gradients multiply through layers. Ten layers of sigmoid means the gradient gets multiplied by at most 0.25 ten times:

```
0.25^10 = 0.000000953674
```

Less than one millionth of the original signal. This is the vanishing gradient problem. Gradients in early layers become so small that weights barely update. The network appears to learn – loss decreases in later layers – but the first layers are frozen. Deep sigmoid networks simply do not train.

Additional problem: sigmoid outputs are always positive (0 to 1), which means gradients on weights are always the same sign. This causes zig-zagging during gradient descent.

Tanh

The centered version of sigmoid.

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Output range: (-1, 1). Zero-centered, which eliminates the zig-zag problem.

The derivative:

$$\tanh'(x) = 1 - \tanh(x)^2$$

Maximum derivative is 1.0 at $x = 0$ – four times better than sigmoid. But the vanishing gradient problem still exists. For large positive or negative inputs, the derivative approaches zero. Ten layers still crush the gradient, just less aggressively.

ReLU: The Breakthrough

Rectified Linear Unit. Popularized for deep learning by Nair and Hinton in 2010 (the function itself dates to Fukushima's 1969 work), it changed everything.

$$\text{relu}(x) = \max(0, x)$$

Output range: $[0, \infty)$. The derivative is trivially simple:

$$\text{relu}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

No vanishing gradient for positive inputs. The gradient is exactly 1, passed straight through. This is why deep networks became trainable – ReLU preserves gradient magnitude across layers.

But there is a failure mode: the dead neuron problem. If a neuron's weighted input is always negative (due to a large negative bias or unfortunate weight initialization), its output is always zero, its gradient is always zero, and it never updates. It is permanently dead. In practice, 10-40% of neurons in a ReLU network can die during training.

Leaky ReLU

The simplest fix for dead neurons.

$$\text{leaky_relu}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha * x & \text{if } x \leq 0 \end{cases}$$

Where α is a small constant, typically 0.01. The negative side has a small slope instead of zero, so dead neurons still get a gradient signal and can recover.

GELU: The Modern Default

Gaussian Error Linear Unit. Introduced by Hendrycks and Gimpel in 2016. Default activation in BERT, GPT, and most modern transformers.

$$\text{gelu}(x) = x * \Phi(x)$$

Where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. The approximation used in practice:

$$\text{gelu}(x) \approx 0.5 * x * (1 + \tanh(\sqrt{2/\pi} * (x + 0.044715 * x^3)))$$

GELU is smooth everywhere, allows small negative values (unlike ReLU which hard-clips to zero), and has a probabilistic interpretation: it weights each input by how likely it is to be positive under a Gaussian distribution. This smooth gating outperforms ReLU in transformer architectures because it provides better gradient flow and avoids the dead neuron problem entirely.

Swish / SiLU

Self-gated activation discovered by Ramachandran et al. in 2017 through automated search.

$$\text{swish}(x) = x * \text{sigmoid}(x)$$

Swish is formally $x * \text{sigmoid}(x)$. Google discovered it through automated search over activation function space – a neural network designing parts of neural networks.

Like GELU, it is smooth, non-monotonic, and allows small negative values. The difference is subtle: Swish uses sigmoid for gating while GELU uses the Gaussian CDF. In practice, performance is nearly identical. Swish is used in EfficientNet and some vision models. GELU dominates in language models.

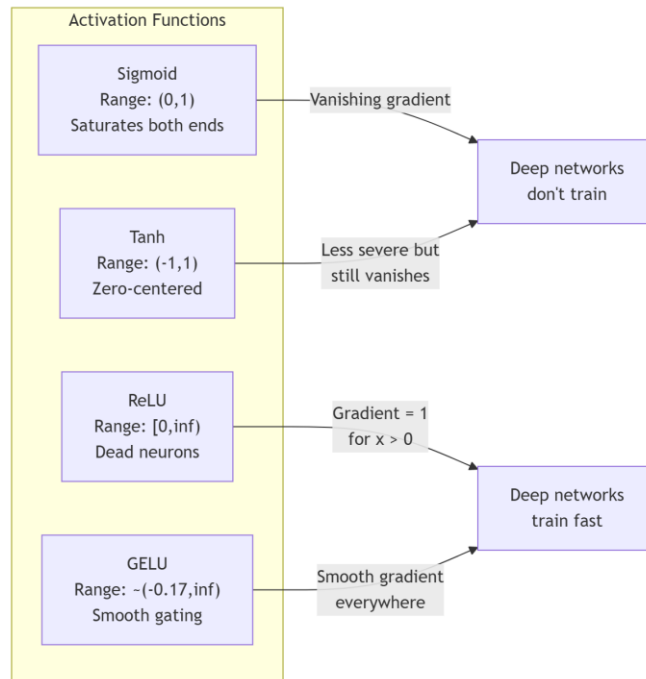
Softmax: The Output Activation

Not used in hidden layers. Softmax converts a vector of raw scores (logits) into a probability distribution.

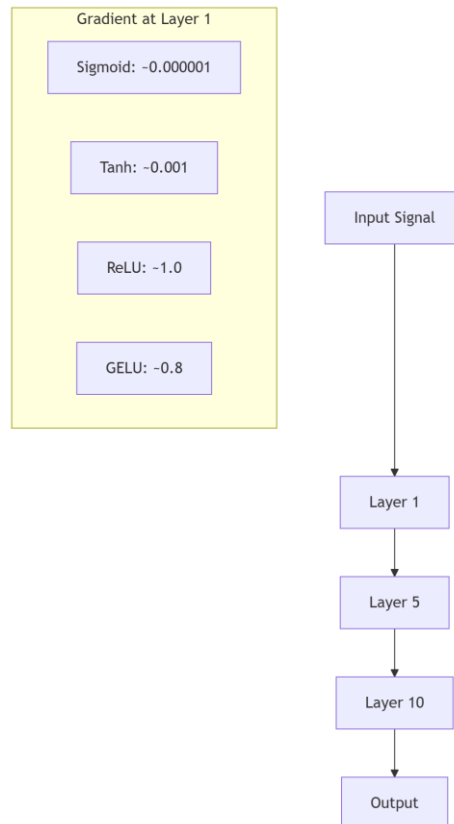
$$\text{softmax}(x_i) = e^{(x_i)} / \sum(e^{(x_j)} \text{ for all } j)$$

Every output is between 0 and 1. All outputs sum to 1. This makes it the standard final activation for multi-class classification. The largest logit gets the highest probability, but unlike argmax, softmax is differentiable and preserves information about relative confidence.

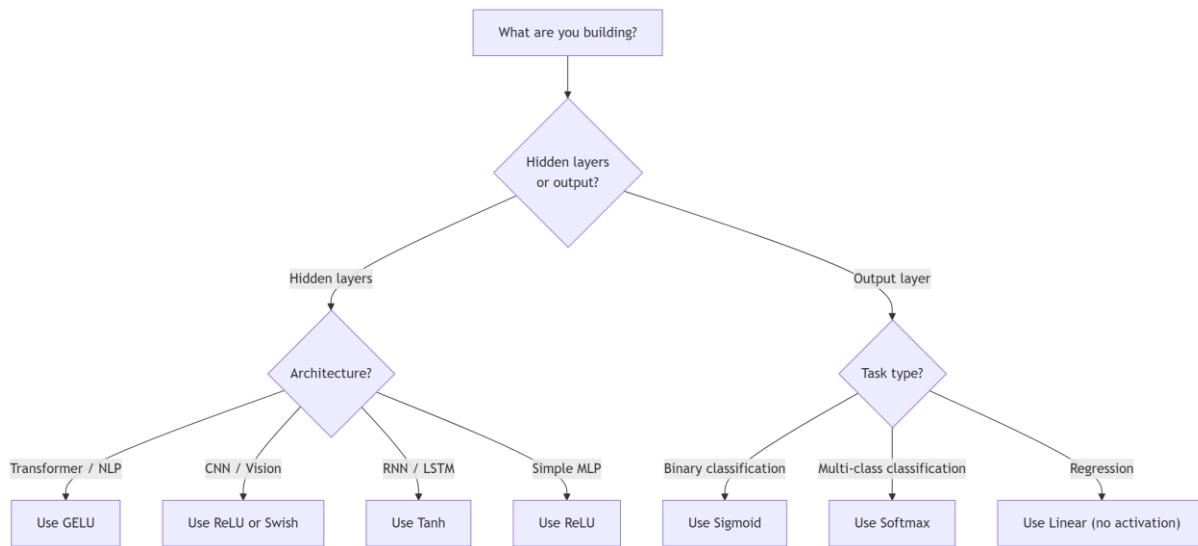
Comparison of Shapes



Gradient Flow Comparison



Which Activation When



Build It

Step 1: Implement All Activation Functions with Derivatives

Each function takes a single float and returns a float. Each derivative function takes the same input and returns the gradient.

```
import math

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

def tanh_act(x):
    return math.tanh(x)

def tanh_derivative(x):
    t = math.tanh(x)
    return 1 - t * t

def relu(x):
    return max(0.0, x)

def relu_derivative(x):
    return 1.0 if x > 0 else 0.0

def leaky_relu(x, alpha=0.01):
```

```

    return x if x > 0 else alpha * x

def leaky_relu_derivative(x, alpha=0.01):
    return 1.0 if x > 0 else alpha

def gelu(x):
    return 0.5 * x * (1 + math.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * x ** 3)))

def gelu_derivative(x):
    phi = 0.5 * (1 + math.erf(x / math.sqrt(2)))
    pdf = math.exp(-0.5 * x * x) / math.sqrt(2 * math.pi)
    return phi + x * pdf

def swish(x):
    return x * sigmoid(x)

def swish_derivative(x):
    s = sigmoid(x)
    return s + x * s * (1 - s)

def softmax(xs):
    max_x = max(xs)
    exps = [math.exp(x - max_x) for x in xs]
    total = sum(exps)
    return [e / total for e in exps]

```

Step 2: Visualize Where Gradients Die

Compute the gradient at 100 evenly-spaced points from -5 to 5. Print a text histogram showing where each activation's gradient is near-zero.

```

def gradient_scan(name, derivative_fn, start=-5, end=5, n=100):
    step = (end - start) / n
    near_zero = 0
    healthy = 0
    for i in range(n):
        x = start + i * step
        g = derivative_fn(x)
        if abs(g) < 0.01:
            near_zero += 1
        else:
            healthy += 1
    pct_dead = near_zero / n * 100
    print(f"{name:15s}: {healthy:3d} healthy, {near_zero:3d} near-zero ({pct_dead:.0f}% dead zone)")

gradient_scan("Sigmoid", sigmoid_derivative)
gradient_scan("Tanh", tanh_derivative)
gradient_scan("ReLU", relu_derivative)
gradient_scan("Leaky ReLU", leaky_relu_derivative)

```

```
gradient_scan("GELU", gelu_derivative)
gradient_scan("Swish", swish_derivative)
```

Step 3: Vanishing Gradient Experiment

Forward-pass a signal through N layers using sigmoid vs ReLU. Measure how the activation magnitude changes.

```
import random

def vanishing_gradient_experiment(activation_fn, name, n_layers=10, n_inputs=5):
    random.seed(42)
    values = [random.gauss(0, 1) for _ in range(n_inputs)]

    print(f"\n{name} through {n_layers} layers:")
    for layer in range(n_layers):
        weights = [random.gauss(0, 1) for _ in range(n_inputs)]
        z = sum(w * v for w, v in zip(weights, values))
        activated = activation_fn(z)
        magnitude = abs(activated)
        bar = "#" * int(magnitude * 20)
        print(f"  Layer {layer+1:2d}: magnitude = {magnitude:.6f} {bar}")
        values = [activated] * n_inputs

    vanishing_gradient_experiment(sigmoid, "Sigmoid")
    vanishing_gradient_experiment(relu, "ReLU")
    vanishing_gradient_experiment(gelu, "GELU")
```

Step 4: Dead Neuron Detector

Create a ReLU network, pass random inputs through it, count how many neurons never fire.

```
def dead_neuron_detector(n_inputs=5, hidden_size=20, n_samples=1000):
    random.seed(0)
    weights = [[random.gauss(0, 1) for _ in range(n_inputs)] for _ in range(hidden_size)]
    biases = [random.gauss(0, 1) for _ in range(hidden_size)]

    fire_counts = [0] * hidden_size

    for _ in range(n_samples):
        inputs = [random.gauss(0, 1) for _ in range(n_inputs)]
        for neuron_idx in range(hidden_size):
            z = sum(w * x for w, x in zip(weights[neuron_idx], inputs)) + biases[neuron_idx]
            if relu(z) > 0:
                fire_counts[neuron_idx] += 1

    dead = sum(1 for c in fire_counts if c == 0)
    rarely_fire = sum(1 for c in fire_counts if 0 < c < n_samples * 0.05)
    healthy = hidden_size - dead - rarely_fire
```

```

print(f"\nDead Neuron Report ({hidden_size} neurons, {n_samples} samples):")
print(f"  Dead (never fired):      {dead}")
print(f"  Barely alive (<5%):        {rarely_fire}")
print(f"  Healthy:                    {healthy}")
print(f"  Dead neuron rate:           {dead/hidden_size*100:.1f}%")

for i, c in enumerate(fire_counts):
    status = "DEAD" if c == 0 else "WEAK" if c < n_samples * 0.05 else "OK"
    bar = "#" * (c * 40 // n_samples)
    print(f"  Neuron {i:2d}: {c:4d}/{n_samples} fires [{status:4s}] {bar}")

dead_neuron_detector()

```

Step 5: Training Comparison – Sigmoid vs ReLU vs GELU

Train the same two-layer network on the circle dataset (points inside a circle = class 1, outside = class 0) with three different activations. Compare convergence speed.

```

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

class ActivationNetwork:
    def __init__(self, activation_fn, activation_deriv, hidden_size=8, lr=0.1):
        random.seed(0)
        self.act = activation_fn
        self.act_d = activation_deriv
        self.lr = lr
        self.hidden_size = hidden_size

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def forward(self, x):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
            self.z1.append(z)

```

```

        self.h.append(self.act(z))

        self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) + self.b
2
        self.out = sigmoid(self.z2)
        return self.out

    def backward(self, target):
        error = self.out - target
        d_out = error * self.out * (1 - self.out)

        for i in range(self.hidden_size):
            d_h = d_out * self.w2[i] * self.act_d(self.z1[i])
            self.w2[i] -= self.lr * d_out * self.h[i]
            for j in range(2):
                self.w1[i][j] -= self.lr * d_h * self.x[j]
            self.b1[i] -= self.lr * d_h
        self.b2 -= self.lr * d_out

    def train(self, data, epochs=200):
        losses = []
        for epoch in range(epochs):
            total_loss = 0
            correct = 0
            for x, y in data:
                pred = self.forward(x)
                self.backward(y)
                total_loss += (pred - y) ** 2
                if (pred >= 0.5) == (y >= 0.5):
                    correct += 1
            avg_loss = total_loss / len(data)
            accuracy = correct / len(data) * 100
            losses.append(avg_loss)
            if epoch % 50 == 0 or epoch == epochs - 1:
                print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f}, accuracy={accuracy:.1f}%")
        return losses

data = make_circle_data()

configs = [
    ("Sigmoid", sigmoid, sigmoid_derivative),
    ("ReLU", relu, relu_derivative),
    ("GELU", gelu, gelu_derivative),
]

results = {}
for name, act_fn, act_d_fn in configs:
    print(f"\n=== Training with {name} ===")
    net = ActivationNetwork(act_fn, act_d_fn, hidden_size=8, lr=0.1)
    losses = net.train(data, epochs=200)

```

```

results[name] = losses

print("\n=== Final Loss Comparison ===")
for name, losses in results.items():
    print(f" {name:10s}: start={losses[0]:.4f} -> end={losses[-1]:.4f} (improvement: {(1 - losses[-1]/losses[0])*100:.1f}%)")

```

Use It

PyTorch provides all of these as both functional and module forms:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

x = torch.randn(4, 10)

relu_out = F.relu(x)
gelu_out = F.gelu(x)
sigmoid_out = torch.sigmoid(x)
swish_out = F.silu(x)

logits = torch.randn(4, 5)
probs = F.softmax(logits, dim=1)

model = nn.Sequential(
    nn.Linear(10, 64),
    nn.GELU(),
    nn.Linear(64, 32),
    nn.GELU(),
    nn.Linear(32, 5),
)

```

Hidden layers in a transformer: GELU. Hidden layers in a CNN: ReLU. Output layer for classification: softmax. Output layer for regression: none (linear). Output layer for probabilities: sigmoid. That's it. Start with these defaults. Change them only when you have evidence.

RNNs and LSTMs use tanh for hidden state and sigmoid for gates, but if you're building from scratch today, you're probably not using RNNs. If neurons are dying in your ReLU network, switch to GELU. Don't reach for Leaky ReLU unless you have a specific reason – GELU solves the dead neuron problem and gives better gradient flow.

Exercises

Implement Parametric ReLU (PReLU) where the negative slope α is a learnable parameter. Train it on the circle dataset and compare to fixed Leaky ReLU.

Run the vanishing gradient experiment with 50 layers instead of 10. Plot the magnitude at each layer for sigmoid, tanh, ReLU, and GELU. At which layer does each activation's signal effectively reach zero?

Implement the ELU (Exponential Linear Unit): $\text{elu}(x) = x$ if $x > 0$, $\alpha * (e^x - 1)$ if $x \leq 0$. Compare its dead neuron rate to ReLU on the same network.

Build a “gradient health monitor” that runs during training: at each epoch, compute the average gradient magnitude at each layer. Print a warning when any layer’s gradient drops below 0.001 or exceeds 100.

Modify the training comparison to use the XOR dataset from Lesson 01 instead of circles. Which activation converges fastest on XOR? Why does this differ from the circle results?

Key Terms

Term	What people say	What it actually means
Activation function	“The nonlinear part”	A function applied to each neuron’s output that breaks linearity, enabling the network to learn nonlinear mappings
Vanishing gradient	“Gradients disappear in deep networks”	Gradients shrink exponentially through layers when the activation’s derivative is less than 1, making early layers untrainable
Exploding gradient	“Gradients blow up”	Gradients grow exponentially through layers when the effective multiplier exceeds 1, causing unstable training
Dead neuron	“A neuron that stopped learning”	A ReLU neuron whose input is permanently negative, producing zero output and zero gradient
Sigmoid	“Squishes values to 0-1”	The logistic function $1/(1+e^{-x})$, historically important but causes vanishing gradients in deep networks
ReLU	“Clips negatives to zero”	$\max(0, x)$ – the activation that made deep learning practical by preserving gradient magnitude
GELU	“The transformer activation”	Gaussian Error Linear Unit, a smooth activation that weights inputs by their probability of being positive
Swish/SiLU	“Self-gated ReLU”	$x * \text{sigmoid}(x)$, discovered through automated search, used in EfficientNet
Softmax	“Turns scores into probabilities”	Normalizes a vector of logits into a probability distribution where all values are in (0,1) and sum to 1
Leaky ReLU	“ReLU that doesn’t die”	$\max(\alpha * x, x)$ where α is small (0.01), preventing dead neurons by allowing small negative gradients

Term	What people say	What it actually means
Saturation	"The flat part of sigmoid"	Regions where an activation's derivative approaches zero, blocking gradient flow
Logit	"The raw score before softmax"	The unnormalized output of the final layer before applying softmax or sigmoid

You are an expert neural network architect. Given a description of a model architecture and task, recommend the optimal activation function for each layer.

Analyze these factors:

57. **Architecture type:** Transformer, CNN, RNN/LSTM, MLP, or hybrid
58. **Task type:** Classification (binary/multi-class), regression, generation, or embedding
59. **Network depth:** Shallow (1-3 layers), medium (4-20 layers), deep (20+ layers)
60. **Known issues:** Vanishing gradients, dead neurons, training instability

Apply these rules:

Hidden layers: - Transformer/NLP: Use GELU (default for BERT, GPT, ViT) - CNN/Vision: Use ReLU. Switch to Swish/SiLU for EfficientNet-style architectures - RNN/LSTM: Use tanh for hidden state, sigmoid for gates - Simple MLP: Use ReLU. Switch to Leaky ReLU if neurons are dying - Deep networks (20+ layers): Avoid sigmoid and tanh entirely. Use ReLU or GELU with proper initialization

Output layer: - Binary classification: Sigmoid (outputs probability in [0,1]) - Multi-class classification: Softmax (outputs probability distribution) - Regression: No activation (linear output) - Multi-label classification: Sigmoid per output (independent probabilities) - Bounded regression: Sigmoid or tanh scaled to target range

Troubleshooting: - Gradients vanishing: Replace sigmoid/tanh with ReLU or GELU - Dead neurons (>10% zero activations): Replace ReLU with Leaky ReLU ($\alpha=0.01$) or GELU - Training instability: Replace ReLU with GELU (smoother gradients) - Slow convergence in transformer: Confirm GELU is used, not ReLU

For each recommendation, state: - The activation function name - Which layers it applies to - Why it fits this specific architecture and task - What failure mode it avoids

Quiz

Pre-lesson check

Why can't you build a useful deep network using only linear layers (no activation functions)?

- a) Linear layers are too slow
- b) Stacking linear layers collapses to a single linear transformation, giving no benefit from depth **(correct)**
- c) Linear layers don't have biases
- d) Linear layers can't handle batched data
*Explanation: $y = W_2(W_1x + b_1) + b_2$ simplifies to $y = Ax + c$. No matter how many linear layers you stack, the result is equivalent to one linear layer. Activation functions break this composability.**

What is the output range of the ReLU activation function?

- a) $(-1, 1)$
- b) $(0, 1)$
- c) $[0, \infty)$ **(correct)**
- d) $(-\infty, \infty)$
Explanation: $\text{ReLU}(x) = \max(0, x)$. It outputs 0 for all negative inputs and passes positive inputs unchanged, giving a range of $[0, \infty)$.

Post-lesson check

- What is the 'dead neuron' problem in ReLU networks?
 1. Neurons that compute too slowly
 2. Neurons whose input is permanently negative, producing zero output and zero gradient forever **(correct)**
 3. Neurons that produce NaN values
 4. Neurons with zero bias
Explanation: If a ReLU neuron's weighted input is always negative (due to bad initialization or large negative bias), it outputs 0 with gradient 0. It can never recover because zero gradient means zero update.

- Which activation function is the default for hidden layers in modern transformers like GPT and BERT?
 5. Sigmoid
 6. ReLU
 7. GELU (**correct**)
 8. Tanh

Explanation: GELU (Gaussian Error Linear Unit) is the default activation in transformers. It provides smooth gradient flow, avoids dead neurons, and has been shown to outperform ReLU in language models.
 - Why is softmax used only in the output layer and never in hidden layers?
 9. It's too slow for hidden layers
 10. It converts a vector into a probability distribution (values sum to 1), which is needed for classification output but not for intermediate representations (**correct**)
 11. It causes exploding gradients
 12. It only works with two classes

Explanation: Softmax normalizes a vector so all values are in (0,1) and sum to 1, creating a probability distribution. Hidden layers need to preserve and transform information, not compress it into probabilities.
-

Loss Functions

Your network makes a prediction. The ground truth says otherwise. How wrong is it? That number is the loss. Pick the wrong loss function and your model optimizes for the wrong thing entirely.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.04 (Activation Functions) **Time:** ~75 minutes

Learning Objectives

- Implement MSE, binary cross-entropy, categorical cross-entropy, and contrastive loss (InfoNCE) from scratch with their gradients
- Explain why MSE fails for classification by demonstrating the “predict 0.5 for everything” failure mode
- Apply label smoothing to cross-entropy and describe how it prevents overconfident predictions
- Choose the correct loss function for regression, binary classification, multi-class classification, and embedding learning tasks

The Problem

A model minimizing MSE on a classification problem will confidently predict 0.5 for everything. It’s minimizing loss. It’s also useless.

The loss function is the only thing your model actually optimizes. Not accuracy. Not F1 score. Not whatever metric you report to your manager. The optimizer takes the gradient of the loss function and adjusts weights to make that number smaller. If the loss function doesn’t capture what you care about, the model will find the mathematically cheapest way to satisfy it, and that way is almost never what you wanted.

Here is a concrete example. You have a binary classification task. Two classes, 50/50 split. You use MSE as your loss. The model predicts 0.5 for every single input. The average MSE is 0.25, which is the minimum possible without actually learning anything. The model has zero discriminative ability but it has technically minimized your loss function. Switch to cross-entropy and the same model is forced to push predictions toward 0 or 1, because $-\log(0.5) = 0.693$ is a terrible loss, while $-\log(0.99) = 0.01$ rewards confident correct predictions. The choice of loss function is the difference between a model that learns and a model that games the metric.

It gets worse. In self-supervised learning, you don’t even have labels. Contrastive loss defines the learning signal entirely: what counts as similar, what counts as different, and how hard the model should push them apart. Get contrastive loss wrong and your embeddings collapse to a single point – every input maps to the same vector. Technically zero loss. Completely worthless.

The Concept

Mean Squared Error (MSE)

The default for regression. Compute the squared difference between prediction and target, average over all samples.

$$\text{MSE} = (1/n) * \text{sum}((y_{\text{pred}} - y_{\text{true}})^2)$$

Why squaring matters: it penalizes large errors quadratically. An error of 2 costs 4x as much as an error of 1. An error of 10 costs 100x. This makes MSE sensitive to outliers – a single wildly wrong prediction dominates the loss.

Real numbers: if your model predicts housing prices and is off by \$10,000 on most houses but off by \$200,000 on one mansion, MSE will aggressively try to fix that one mansion, potentially hurting performance on the other 99 houses.

The gradient of MSE with respect to a prediction is:

$$d\text{MSE}/dy_{\text{pred}} = (2/n) * (y_{\text{pred}} - y_{\text{true}})$$

Linear in the error. Bigger errors get bigger gradients. This is a feature for regression (large errors need large corrections) and a bug for classification (you want to penalize confident wrong answers exponentially, not linearly).

Cross-Entropy Loss

The loss function for classification. Rooted in information theory – it measures the divergence between the predicted probability distribution and the true distribution.

Binary Cross-Entropy (BCE):

$$\text{BCE} = -(y * \log(p) + (1 - y) * \log(1 - p))$$

Where y is the true label (0 or 1) and p is the predicted probability.

Why $-\log(p)$ works: when the true label is 1 and you predict $p = 0.99$, the loss is $-\log(0.99) = 0.01$. When you predict $p = 0.01$, the loss is $-\log(0.01) = 4.6$. That 460x difference is why cross-entropy works. It brutally punishes confident wrong predictions while barely penalizing confident correct ones.

The gradient tells the same story:

$$dBCE/dp = -(y/p) + (1-y)/(1-p)$$

When $y = 1$ and p is near zero, the gradient is $-1/p$ which approaches negative ∞ . The model gets an enormous signal to fix its mistake. When p is near 1, the gradient is tiny. Already correct, nothing to fix.

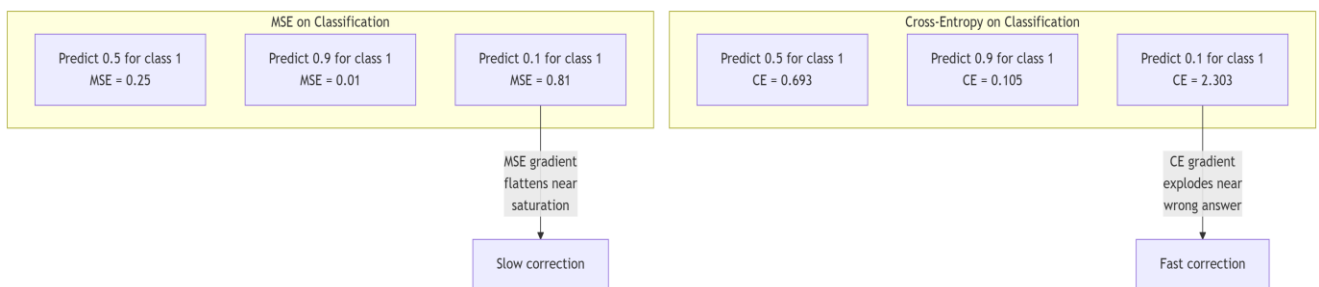
Categorical Cross-Entropy:

For multi-class classification with one-hot encoded targets.

$$\text{CCE} = -\sum(y_i * \log(p_i))$$

Only the true class contributes to the loss (because all other y_i are zero). If there are 10 classes and the correct class gets probability 0.1 (random guessing), the loss is $-\log(0.1) = 2.3$. If the correct class gets probability 0.9, the loss is $-\log(0.9) = 0.105$. The model learns to concentrate probability mass on the right answer.

Why MSE Fails for Classification



MSE gradients flatten when predictions are near 0 or 1 (due to sigmoid saturation). Cross-entropy gradients compensate for this – the $-\log$ cancels the sigmoid's flat regions, giving strong gradients exactly where they are needed most.

Label Smoothing

Standard one-hot labels say “this is 100% class 3 and 0% everything else.” That’s a strong claim. Label smoothing softens it:

$$\text{smooth_label} = (1 - \alpha) * \text{one_hot} + \alpha / \text{num_classes}$$

With $\alpha = 0.1$ and 10 classes: instead of $[0, 0, 1, 0, \dots]$, the target becomes $[0.01, 0.01, 0.91, 0.01, \dots]$. The model targets 0.91 instead of 1.0.

Why this works: a model trying to output exactly 1.0 through a softmax needs to push logits to ∞ . This causes overconfidence, hurts generalization, and makes the model brittle to distribution shift. Label smoothing caps the target at 0.9 (with $\alpha=0.1$), keeping logits in a reasonable range. GPT and most modern models use label smoothing or its equivalent.

Contrastive Loss

No labels. No classes. Just pairs of inputs and the question: are these similar or different?

SimCLR-style contrastive loss (NT-Xent / InfoNCE):

Take one image. Create two augmented views of it (crop, rotate, color jitter). These are the “positive pair” – they should have similar embeddings. Every other image in the batch forms a “negative pair” – they should have different embeddings.

$$L = -\log(\exp(\text{sim}(z_i, z_j) / \tau) / \sum(\exp(\text{sim}(z_i, z_k) / \tau)))$$

Where $\text{sim}()$ is cosine similarity, z_i and z_j are the positive pair, the sum is over all negatives, and τ (temperature) controls how sharp the distribution is. Lower temperature = harder negatives = more aggressive separation.

Real numbers: batch size 256 means 255 negatives per positive pair. Temperature $\tau = 0.07$ (SimCLR default). The loss looks like a softmax over similarities – it wants the positive pair’s similarity to be highest among all 256 options.

Triplet Loss:

Takes three inputs: anchor, positive (same class), negative (different class).

$$L = \max(0, d(\text{anchor}, \text{positive}) - d(\text{anchor}, \text{negative}) + \text{margin})$$

The margin (typically 0.2-1.0) enforces a minimum gap between positive and negative distances. If the negative is already far enough away, the loss is zero – no gradient, no update. This makes training efficient but requires careful triplet mining (choosing hard negatives that are close to the anchor).

Focal Loss

For imbalanced datasets. Standard cross-entropy treats all correctly classified examples equally. Focal loss down-weights easy examples:

$$FL = -\alpha * (1 - p_t)^\gamma * \log(p_t)$$

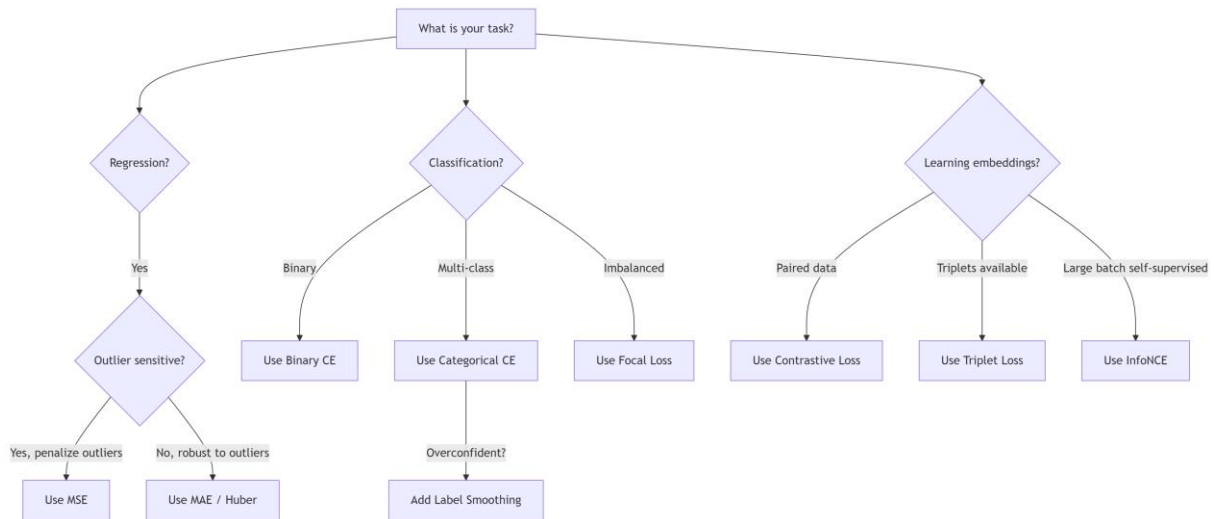
Where p_t is the predicted probability of the true class and γ controls the focusing. With $\gamma = 0$, this is standard cross-entropy. With $\gamma = 2$ (the default):

175. Easy example ($p_t = 0.9$): weight = $(0.1)^2 = 0.01$. Effectively ignored.

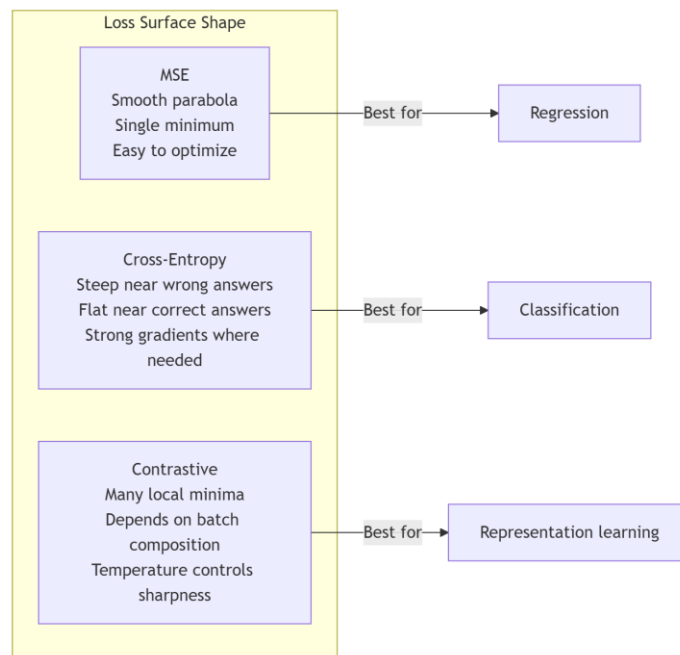
176. Hard example ($p_t = 0.1$): weight = $(0.9)^2 = 0.81$. Full gradient signal.

Focal loss was introduced by Lin et al. for object detection, where 99% of candidate regions are background (easy negatives). Without focal loss, the model drowns in easy background examples and never learns to detect objects. With it, the model focuses its capacity on the hard, ambiguous cases that matter.

Loss Function Decision Tree



Loss Landscape



Build It

Step 1: MSE and Its Gradient

```
def mse(predictions, targets):
    n = len(predictions)
    total = 0.0
    for p, t in zip(predictions, targets):
        total += (p - t) ** 2
```



```

    return total / n

def mse_gradient(predictions, targets):
    n = len(predictions)
    grads = []
    for p, t in zip(predictions, targets):
        grads.append(2.0 * (p - t) / n)
    return grads

```

Step 2: Binary Cross-Entropy

The $\log(0)$ problem is real. If the model predicts exactly 0 for a positive example, $\log(0) = \text{negative } \infty$. Clipping prevents this.

```

import math

def binary_cross_entropy(predictions, targets, eps=1e-15):
    n = len(predictions)
    total = 0.0
    for p, t in zip(predictions, targets):
        p_clipped = max(eps, min(1 - eps, p))
        total += -(t * math.log(p_clipped) + (1 - t) * math.log(1 - p_clipped))
    return total / n

def bce_gradient(predictions, targets, eps=1e-15):
    grads = []
    for p, t in zip(predictions, targets):
        p_clipped = max(eps, min(1 - eps, p))
        grads.append(-(t / p_clipped) + (1 - t) / (1 - p_clipped))
    return grads

```

Step 3: Categorical Cross-Entropy with Softmax

Softmax converts raw logits to probabilities. Then we compute the cross-entropy against one-hot targets.

```

def softmax(logits):
    max_val = max(logits)
    exps = [math.exp(x - max_val) for x in logits]
    total = sum(exps)
    return [e / total for e in exps]

def categorical_cross_entropy(logits, target_index, eps=1e-15):
    probs = softmax(logits)
    p = max(eps, probs[target_index])
    return -math.log(p)

def cce_gradient(logits, target_index):
    probs = softmax(logits)
    grads = list(probs)
    grads[target_index] -= 1.0
    return grads

```

The gradient of softmax + cross-entropy simplifies beautifully: it's just (predicted probability - 1) for the true class, and (predicted probability) for all other classes. This elegant simplification is not a coincidence – it's why softmax and cross-entropy are paired.

Step 4: Label Smoothing

```
def label_smoothed_cce(logits, target_index, num_classes, alpha=0.1, eps=1e-15):
    probs = softmax(logits)
    loss = 0.0
    for i in range(num_classes):
        if i == target_index:
            smooth_target = 1.0 - alpha + alpha / num_classes
        else:
            smooth_target = alpha / num_classes
        p = max(eps, probs[i])
        loss += -smooth_target * math.log(p)
    return loss
```

Step 5: Contrastive Loss (Simplified InfoNCE)

```
def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    if norm_a < 1e-10 or norm_b < 1e-10:
        return 0.0
    return dot / (norm_a * norm_b)

def contrastive_loss(anchor, positive, negatives, temperature=0.07):
    sim_pos = cosine_similarity(anchor, positive) / temperature
    sim_negs = [cosine_similarity(anchor, neg) / temperature for neg in negatives]

    max_sim = max(sim_pos, max(sim_negs)) if sim_negs else sim_pos
    exp_pos = math.exp(sim_pos - max_sim)
    exp_negs = [math.exp(s - max_sim) for s in sim_negs]
    total_exp = exp_pos + sum(exp_negs)

    return -math.log(max(1e-15, exp_pos / total_exp))
```

Step 6: MSE vs Cross-Entropy on Classification

Train the same network from lesson 04 (circle dataset) with both loss functions. Watch cross-entropy converge faster.

```
import random

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))
```

```

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

class LossComparisonNetwork:
    def __init__(self, loss_type="bce", hidden_size=8, lr=0.1):
        random.seed(0)
        self.loss_type = loss_type
        self.lr = lr
        self.hidden_size = hidden_size

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def forward(self, x):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
            self.z1.append(z)
            self.h.append(max(0.0, z))

        self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) + self.b2
        self.out = sigmoid(self.z2)
        return self.out

    def backward(self, target):
        if self.loss_type == "mse":
            d_loss = 2.0 * (self.out - target)
        else:
            eps = 1e-15
            p = max(eps, min(1 - eps, self.out))
            d_loss = -(target / p) + (1 - target) / (1 - p)

        d_sigmoid = self.out * (1 - self.out)
        d_out = d_loss * d_sigmoid

        for i in range(self.hidden_size):
            d_relu = 1.0 if self.z1[i] > 0 else 0.0
            d_h = d_out * self.w2[i] * d_relu
            self.w2[i] -= self.lr * d_out * self.h[i]

```

```

        for j in range(2):
            self.w1[i][j] -= self.lr * d_h * self.x[j]
            self.b1[i] -= self.lr * d_h
        self.b2 -= self.lr * d_out

    def compute_loss(self, pred, target):
        if self.loss_type == "mse":
            return (pred - target) ** 2
        else:
            eps = 1e-15
            p = max(eps, min(1 - eps, pred))
            return -(target * math.log(p) + (1 - target) * math.log(1 - p))

    def train(self, data, epochs=200):
        losses = []
        for epoch in range(epochs):
            total_loss = 0.0
            correct = 0
            for x, y in data:
                pred = self.forward(x)
                self.backward(y)
                total_loss += self.compute_loss(pred, y)
                if (pred >= 0.5) == (y >= 0.5):
                    correct += 1
            avg_loss = total_loss / len(data)
            accuracy = correct / len(data) * 100
            losses.append((avg_loss, accuracy))
            if epoch % 50 == 0 or epoch == epochs - 1:
                print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f}, accuracy={accuracy:.1f}%")
        return losses

```

Use It

PyTorch provides all standard loss functions with numerical stability built in:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

predictions = torch.tensor([0.9, 0.1, 0.7], requires_grad=True)
targets = torch.tensor([1.0, 0.0, 1.0])

mse_loss = F.mse_loss(predictions, targets)
bce_loss = F.binary_cross_entropy(predictions, targets)

logits = torch.randn(4, 10)
labels = torch.tensor([3, 7, 1, 9])
ce_loss = F.cross_entropy(logits, labels)
ce_smooth = F.cross_entropy(logits, labels, label_smoothing=0.1)

```

Use `F.cross_entropy` (not `F.nll_loss` plus manual softmax). It combines log-softmax and negative log-likelihood in one numerically stable operation. Applying softmax separately then taking the log is less stable – you lose precision in the subtraction of large exponentials.

For contrastive learning, most teams use custom implementations or libraries like `lightly` or `pytorch-metric-learning`. The core loop is always the same: compute pairwise similarities, create the softmax over positives and negatives, backpropagate.

Exercises

- Implement Huber loss (smooth L1 loss), which is MSE for small errors and MAE for large errors. Train a regression network predicting $y = \sin(x)$ with MSE vs Huber when 5% of training targets have random noise added (outliers). Compare final test error.
- Add focal loss to the binary classification training loop. Create an imbalanced dataset (90% class 0, 10% class 1). Compare standard BCE vs focal loss ($\gamma=2$) on the minority class recall after 200 epochs.
- Implement triplet loss with semi-hard negative mining. Generate 2D embedding data for 5 classes. For each anchor, find the hardest negative that is still farther than the positive (semi-hard). Compare convergence to random triplet selection.
- Run the MSE vs cross-entropy comparison but track gradient magnitudes at each layer during training. Plot the average gradient norm per epoch. Verify that cross-entropy produces larger gradients in early epochs when the model is most uncertain.
- Implement KL divergence loss and verify that minimizing $KL(\text{true} \parallel \text{predicted})$ gives the same gradients as cross-entropy when the true distribution is one-hot. Then try soft targets (like knowledge distillation) where the “true” distribution comes from a teacher model’s softmax output.

Key Terms

Term	What people say	What it actually means
Loss function	“How wrong the model is”	A differentiable function mapping predictions and targets to a scalar that the optimizer minimizes
MSE	“Average squared error”	Mean of squared differences between predictions and targets; penalizes large errors quadratically
Cross-entropy	“The classification loss”	Measures divergence between predicted probability distribution and true distribution using $-\log(p)$
Binary cross-entropy	“BCE”	Cross-entropy for two classes: $-(y\log(p) + (1-y)\log(1-p))$

Term	What people say	What it actually means
Label smoothing	"Softening the targets"	Replacing hard 0/1 targets with soft values (e.g., 0.1/0.9) to prevent overconfidence and improve generalization
Contrastive loss	"Pull together, push apart"	A loss that learns representations by making similar pairs close and dissimilar pairs far in embedding space
InfoNCE	"The CLIP/SimCLR loss"	Normalized temperature-scaled cross-entropy over similarity scores; treats contrastive learning as classification
Focal loss	"The imbalanced data fix"	Cross-entropy weighted by $(1-p_t)^\gamma$ to down-weight easy examples and focus on hard ones
Triplet loss	"Anchor-positive-negative"	Pushes anchor closer to positive than negative by at least a margin in embedding space
Temperature	"Sharpness knob"	A scalar divisor on logits/similarities that controls how peaked the resulting distribution is; lower = sharper

You are an expert ML debugger. Given a description of a loss curve or training behavior, diagnose the problem and recommend a fix.

Common patterns and their causes:

Loss is NaN or ∞ : - $\log(0)$ in cross-entropy: Add ϵ clipping ($\max(\epsilon, \text{prediction})$) - Exploding gradients: Add gradient clipping ($\max_norm=1.0$) - Learning rate too high: Reduce by 10x - Numerical overflow in softmax: Subtract max logit before exp

Loss decreases then suddenly spikes: - Learning rate too high for current loss landscape region - Fix: Add learning rate warmup (linear ramp over first 1-10% of steps) - Fix: Switch to cosine decay schedule - Fix: Reduce learning rate by 3-5x

Loss plateaus and never improves: - Dead neurons (ReLU): Check activation statistics, switch to GELU - Vanishing gradients: Check gradient norms per layer - Wrong loss function: MSE on classification will plateau at 0.25 for balanced binary - Learning rate too low: Increase by 3-10x

Training loss decreases but validation loss increases: - Overfitting: Add dropout ($p=0.1-0.3$), weight decay (0.01), or data augmentation - Reduce model capacity (fewer layers or smaller hidden size) - Add early stopping with patience=5-20 epochs

Loss is very high and barely decreasing: - Label encoding mismatch: Check that targets match loss function expectations - Softmax applied twice: If using `F.cross_entropy`, do NOT apply softmax manually - Wrong sign: Loss should use negative log likelihood, not positive

All predictions are the same value (e.g., 0.5): - MSE on classification: Switch to cross-entropy - Dead network: Check initialization, ensure activations are non-zero - Bias-only solution: Network ignoring inputs, check input normalization

For each diagnosis: 1. Identify the most likely root cause 2. Provide a specific fix with code or hyperparameter changes 3. Explain how to verify the fix worked 4. Suggest monitoring to prevent recurrence

You are an expert ML engineer. Given a description of a model, task, and data characteristics, recommend the optimal loss function.

Analyze these factors:

- **Task type:** Regression, binary classification, multi-class classification, multi-label, ranking, or representation learning
- **Data distribution:** Balanced vs imbalanced classes, presence of outliers, noise level
- **Model output:** Raw logits, probabilities, embeddings, or continuous values
- **Training stage:** Pre-training, fine-tuning, or distillation

Apply these rules:

Regression: - Default: MSE (mean squared error) - Outliers present: Huber loss ($\delta=1.0$) or MAE (mean absolute error) - Bounded output: MSE with sigmoid/tanh output activation - Probabilistic: Negative log-likelihood with learned variance

Binary classification: - Default: Binary cross-entropy (BCE) - Class imbalance $> 10:1$: Focal loss ($\gamma=2.0$, $\alpha=0.25$) - Label noise: BCE with label smoothing ($\alpha=0.1$) - Calibrated probabilities needed: BCE (naturally calibrated)

Multi-class classification: - Default: Categorical cross-entropy (softmax + NLL) - Overconfident predictions: Add label smoothing ($\alpha=0.1$) - Extreme class imbalance: Focal loss per class - Knowledge distillation: KL divergence with soft targets (temperature=4-20)

Representation learning / Embeddings: - Paired positives and negatives: InfoNCE / NT-Xent (temperature=0.07) - Triplets available: Triplet loss (margin=0.2-1.0) with semi-hard mining - Large batch self-supervised: SimCLR-style contrastive (batch size ≥ 256) - Text-image pairs: CLIP-style contrastive with learned temperature

Common mistakes to flag: - MSE for classification (gradient flattens near 0/1 due to sigmoid saturation) - Cross-entropy without label smoothing on large models (leads to overconfidence) - Contrastive loss with small batch size (too few negatives, collapse risk) - Triplet loss with random mining (wastes compute on easy triplets) - Forgetting ϵ clipping in log computations (NaN from $\log(0)$)

For each recommendation, state: - The loss function name and formula - Why it fits this specific task and data - The key hyperparameters and their recommended values - What failure mode it avoids

Quiz

Pre-lesson check

177. What does the loss function represent in neural network training?

- a) The number of incorrect predictions
- b) A differentiable measure of how wrong the model's predictions are, which the optimizer minimizes (**correct**)
- c) The size of the training dataset
- d) The learning rate schedule

Explanation: The loss function maps predictions and targets to a single scalar that the optimizer minimizes via gradient descent. It must be differentiable so gradients can be computed.

178. Why is cross-entropy preferred over MSE for classification tasks?

- a) Cross-entropy is faster to compute
- b) Cross-entropy punishes confident wrong predictions exponentially via $-\log(p)$, while MSE gives weak gradients near 0 and 1 (**correct**)
- c) MSE only works for regression
- d) Cross-entropy doesn't require labels

Explanation: When a model confidently predicts the wrong class (p near 0 for true class), $-\log(p)$ produces a huge loss and strong gradient. MSE produces weak gradients in the same situation because sigmoid is flat near 0 and 1.

Post-lesson check

61. What happens if you use MSE loss for binary classification?

- a) Training diverges immediately

b) The model can minimize loss by predicting 0.5 for everything, achieving $MSE=0.25$ without learning **(correct)**

c) The model trains normally

d) Gradients explode

Explanation: With MSE on a balanced binary dataset, predicting 0.5 for every input gives $MSE=0.25$ – the minimum achievable without discrimination. The model satisfies the loss without learning any useful patterns.

62. What does label smoothing do and why is it useful?

a) It removes noisy labels from the dataset

b) It replaces hard 0/1 targets with soft values like 0.1/0.9, preventing overconfident predictions and improving generalization **(correct)**

c) It applies a moving average to the loss

d) It smooths the learning rate schedule

Explanation: Label smoothing changes targets from $[0,0,1,0]$ to $[0.025,0.025,0.925,0.025]$ (with $\alpha=0.1$). This prevents the model from pushing logits to ∞ to achieve hard targets, reducing overconfidence.

63. In contrastive loss (InfoNCE), what role does the temperature parameter play?

a) It controls the learning rate

b) It controls how sharp the similarity distribution is – lower temperature means harder separation between positives and negatives **(correct)**

c) It sets the number of negative samples

d) It determines the embedding dimension

Explanation: Temperature divides the similarity scores before softmax. Lower temperature (e.g., 0.07) creates a sharper distribution where the model must clearly separate positives from negatives. Higher temperature is more forgiving.

Optimizers

Gradient descent tells you which direction to move. It says nothing about how far or how fast. SGD is a compass. Adam is GPS with traffic data.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.05 (Loss Functions) **Time:** ~75 minutes

Learning Objectives

- Implement SGD, SGD with momentum, Adam, and AdamW optimizers from scratch in Python
- Explain how Adam's bias correction compensates for zero-initialized moment estimates in early training steps
- Demonstrate why AdamW produces better generalization than Adam with L2 regularization on the same task
- Select the appropriate optimizer and default hyperparameters for transformers, CNNs, GANs, and fine-tuning

The Problem

You computed the gradients. You know that weight #4,721 should decrease by 0.003 to reduce the loss. But 0.003 in what units? Scaled by what? And should you move the same amount on step 1 as on step 1,000?

Vanilla gradient descent applies the same learning rate to every parameter on every step: $w = w - lr * \text{gradient}$. This creates three problems that make training neural networks painful in practice.

First, oscillation. The loss landscape is rarely shaped like a smooth bowl. It's more like a long, narrow valley. The gradient points across the valley (steep direction), not along it (shallow direction). Gradient descent bounces back and forth across the narrow dimension while making tiny progress along the useful one. You've seen this: loss drops fast then plateaus, not because the model converged but because it's oscillating.

Second, one learning rate for all parameters is wrong. Some weights need large updates (they're in the early, underfitting stage). Others need tiny updates (they're near their optimal value). A learning rate that works for the former destroys the latter, and vice versa.

Third, saddle points. In high dimensions, the loss landscape has vast flat regions where the gradient is near zero. Vanilla SGD crawls through these at the speed of the gradient, which is effectively zero. The model looks stuck. It isn't stuck – it's in a flat region with useful descent on the other side. But SGD has no mechanism to push through.

Adam solves all three. It maintains two running averages per parameter – the mean gradient (momentum, handles oscillation) and the mean squared gradient (adaptive rate, handles different scales). Combined with bias correction for the first few steps, it gives you a single optimizer that works on 80% of problems

with default hyperparameters. This lesson builds it from scratch so you understand exactly when and why it fails on the other 20%.

The Concept

Stochastic Gradient Descent (SGD)

The simplest optimizer. Compute the gradient on a mini-batch and step in the opposite direction.

$$w = w - lr * \text{gradient}$$

The “stochastic” means you use a random subset (mini-batch) of data to estimate the gradient, rather than the full dataset. This noise is actually useful – it helps escape sharp local minima. But the noise also causes oscillation.

Learning rate is the only knob. Too high: the loss diverges. Too low: training takes forever. The optimal value depends on the architecture, the data, the batch size, and the current stage of training. For vanilla SGD on modern networks, typical values range from 0.01 to 0.1. But even within a single training run, the ideal learning rate changes.

Momentum

The ball-rolling-downhill analogy is overused but accurate. Instead of stepping by the gradient alone, you maintain a velocity that accumulates past gradients.

$$\begin{aligned} m_t &= \beta * m_{t-1} + \text{gradient} \\ w &= w - lr * m_t \end{aligned}$$

Beta (typically 0.9) controls how much history to keep. With $\beta = 0.9$, the momentum is roughly the average of the last 10 gradients ($1 / (1 - 0.9) = 10$).

Why this fixes oscillation: gradients that point in the same direction accumulate. Gradients that flip direction cancel out. In that narrow valley, the “across” component flips sign each step and gets dampened. The “along” component stays consistent and gets amplified. The result is smooth acceleration in the useful direction.

Real numbers: SGD alone on a badly conditioned loss landscape might take 10,000 steps. SGD with momentum ($\beta=0.9$) typically takes 3,000-5,000 steps on the same problem. The speedup is not marginal.

RMSProp

The first per-parameter adaptive learning rate method that actually worked. Proposed by Hinton in a Coursera lecture (never formally published).

```
s_t = beta * s_{t-1} + (1 - beta) * gradient^2
w = w - lr * gradient / (sqrt(s_t) + epsilon)
```

s_t tracks the running average of squared gradients. Parameters with consistently large gradients get divided by a large number (smaller effective learning rate). Parameters with small gradients get divided by a small number (larger effective learning rate).

This solves the “one learning rate for all parameters” problem. A weight that’s already been getting large updates is probably near its target – slow it down. A weight that’s been getting tiny updates might be undertrained – speed it up.

Epsilon (typically $1e-8$) prevents division by zero when a parameter hasn’t been updated.

Adam: Momentum + RMSProp

Adam combines both ideas. It maintains two exponential moving averages per parameter:

```
m_t = beta1 * m_{t-1} + (1 - beta1) * gradient      (first moment: mean)
v_t = beta2 * v_{t-1} + (1 - beta2) * gradient^2    (second moment: variance)
```

Bias correction is the key detail most explanations skip. At step 1, $m_1 = (1 - \beta_1) * \text{gradient}$. With $\beta_1 = 0.9$, that’s $0.1 * \text{gradient}$ – ten times too small. The moving average hasn’t warmed up yet. Bias correction compensates:

```
m_hat = m_t / (1 - beta1^t)
v_hat = v_t / (1 - beta2^t)
```

At step 1 with $\beta_1 = 0.9$: $m_{\text{hat}} = m_1 / (1 - 0.9) = m_1 / 0.1 = \text{the actual gradient}$. At step 100: $(1 - 0.9^{100})$ is approximately 1.0, so the correction vanishes. Bias correction matters for the first ~10 steps and is irrelevant after ~50.

The update:

```
w = w - lr * m_hat / (sqrt(v_hat) + epsilon)
```

Adam defaults: $lr = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$. These defaults work for 80% of problems. When they don’t, change lr first. Then β_2 . Almost never change β_1 or ϵ .

AdamW: Weight Decay Done Right

L2 regularization adds $\lambda * w^2$ to the loss. In vanilla SGD, this is equivalent to weight decay (subtracting $\lambda * w$ from the weight at each step). In Adam, this equivalence breaks.

The Loshchilov & Hutter insight: when you add L2 to the loss and then Adam processes the gradient, the adaptive learning rate scales the regularization term too. Parameters with large gradient variance get less regularization. Parameters with small variance get more. This is not what you want – you want uniform regularization regardless of the gradient statistics.

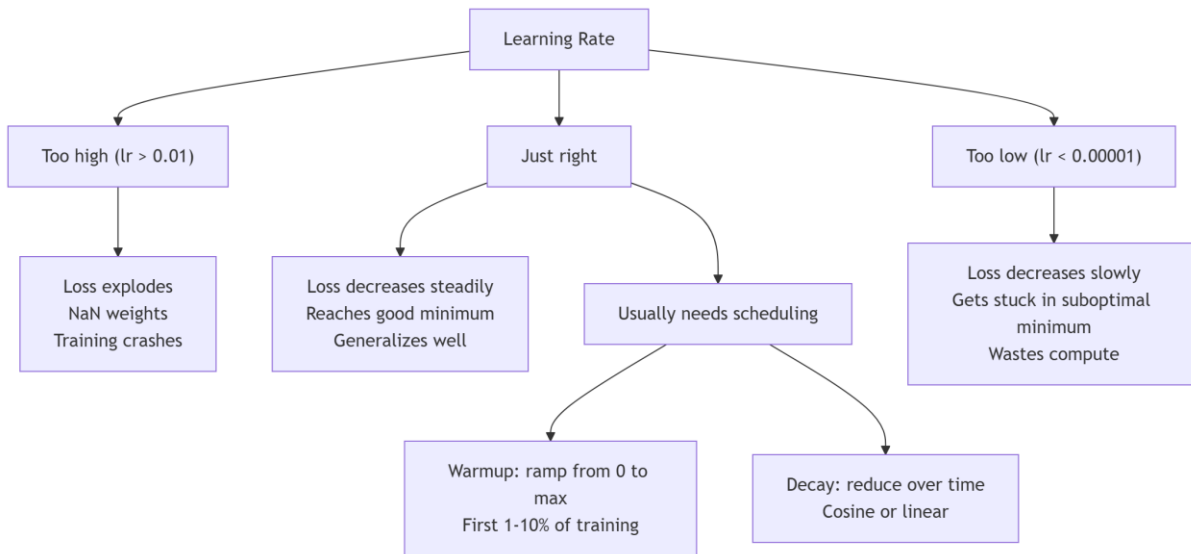
AdamW fixes this by applying weight decay directly to the weights, after the Adam update:

$$w = w - lr * m_hat / (sqrt(v_hat) + epsilon) - lr * lambda * w$$

The weight decay term ($lr * \lambda * w$) is not scaled by Adam's adaptive factor. Every parameter gets the same proportional shrinkage.

This seems like a minor detail. It's not. AdamW converges to better solutions than Adam + L2 regularization on virtually every task. It's the default optimizer in PyTorch for training transformers, diffusion models, and most modern architectures. BERT, GPT, LLaMA, Stable Diffusion – all trained with AdamW.

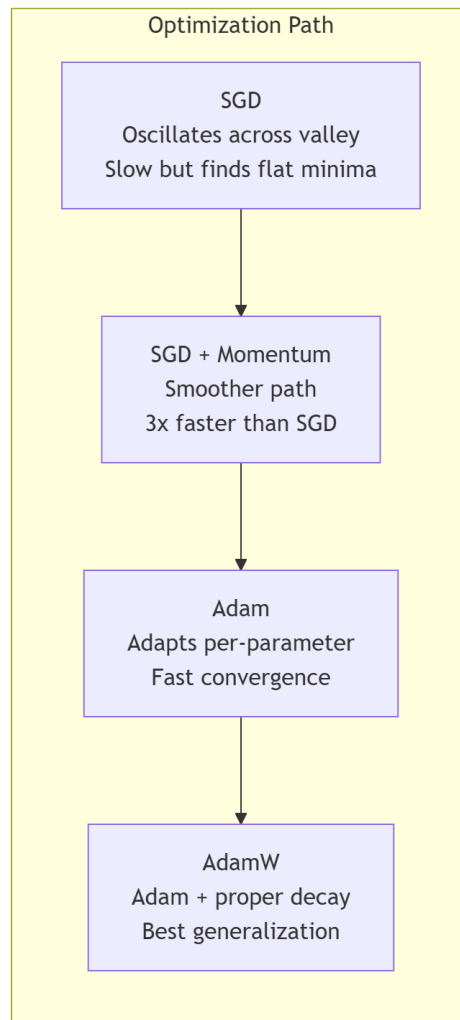
Learning Rate: The Most Important Hyperparameter



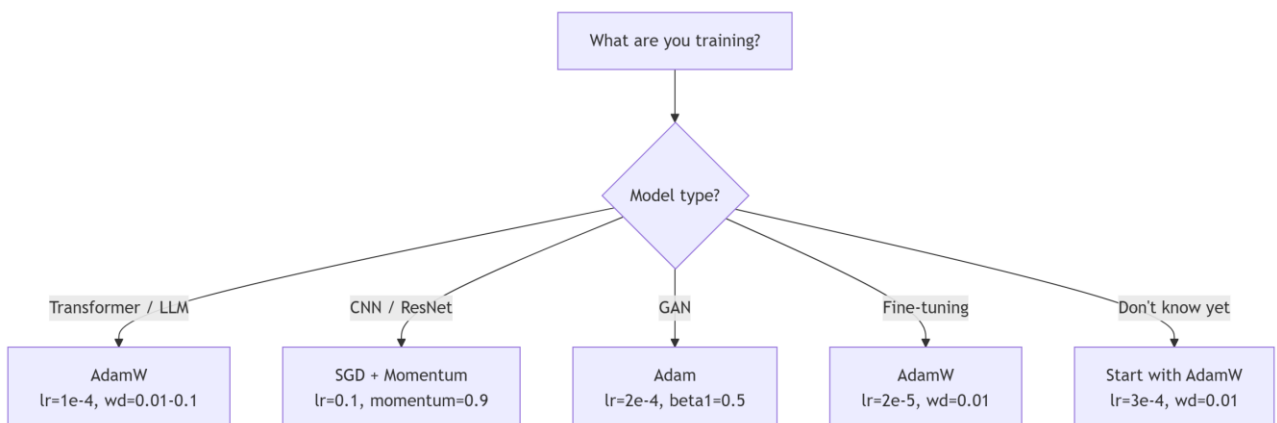
If you tune one hyperparameter, tune the learning rate. A 10x change in learning rate matters more than any architectural decision you'll make. Common defaults:

179. SGD: $lr = 0.01$ to 0.1
180. Adam/AdamW: $lr = 1e-4$ to $3e-4$
181. Fine-tuning pretrained models: $lr = 1e-5$ to $5e-5$
182. Learning rate warmup: linear ramp over first 1-10% of steps

Optimizer Comparison



When Each Optimizer Wins



Build It

Step 1: Vanilla SGD

```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def step(self, params, grads):
        for i in range(len(params)):
            params[i] -= self.lr * grads[i]
```

Step 2: SGD with Momentum

```
class SGDMomentum:
    def __init__(self, lr=0.01, beta=0.9):
        self.lr = lr
        self.beta = beta
        self.velocities = None

    def step(self, params, grads):
        if self.velocities is None:
            self.velocities = [0.0] * len(params)
        for i in range(len(params)):
            self.velocities[i] = self.beta * self.velocities[i] + grads[i]
            params[i] -= self.lr * self.velocities[i]
```

Step 3: Adam

```
import math

class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.m = None
        self.v = None
        self.t = 0

    def step(self, params, grads):
        if self.m is None:
            self.m = [0.0] * len(params)
            self.v = [0.0] * len(params)

        self.t += 1

        for i in range(len(params)):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grads[i]
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * grads[i] ** 2
```

```

m_hat = self.m[i] / (1 - self.beta1 ** self.t)
v_hat = self.v[i] / (1 - self.beta2 ** self.t)

params[i] -= self.lr * m_hat / (math.sqrt(v_hat) + self.epsilon)

```

Step 4: AdamW

```

class AdamW:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8, weight_decay=0.01):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.weight_decay = weight_decay
        self.m = None
        self.v = None
        self.t = 0

    def step(self, params, grads):
        if self.m is None:
            self.m = [0.0] * len(params)
            self.v = [0.0] * len(params)

        self.t += 1

        for i in range(len(params)):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grads[i]
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * grads[i] ** 2

            m_hat = self.m[i] / (1 - self.beta1 ** self.t)
            v_hat = self.v[i] / (1 - self.beta2 ** self.t)

            params[i] -= self.lr * m_hat / (math.sqrt(v_hat) + self.epsilon)
            params[i] -= self.lr * self.weight_decay * params[i]

```

Step 5: Training Comparison

Train the same two-layer network on the circle dataset from lesson 05 with all four optimizers. Compare convergence.

```

import random

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []

```



```

for _ in range(n):
    x = random.uniform(-2, 2)
    y = random.uniform(-2, 2)
    label = 1.0 if x * x + y * y < 1.5 else 0.0
    data.append([x, y], label))
return data

class OptimizerTestNetwork:
    def __init__(self, optimizer, hidden_size=8):
        random.seed(0)
        self.hidden_size = hidden_size
        self.optimizer = optimizer

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in range(hidden_size)]
    ]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def get_params(self):
        params = []
        for row in self.w1:
            params.extend(row)
        params.extend(self.b1)
        params.extend(self.w2)
        params.append(self.b2)
        return params

    def set_params(self, params):
        idx = 0
        for i in range(self.hidden_size):
            for j in range(2):
                self.w1[i][j] = params[idx]
                idx += 1
            for i in range(self.hidden_size):
                self.b1[i] = params[idx]
                idx += 1
            for i in range(self.hidden_size):
                self.w2[i] = params[idx]
                idx += 1
            self.b2 = params[idx]

    def forward(self, x):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
            self.z1.append(z)
            self.h.append(max(0.0, z))

        self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) + self.b

```

2

```

        self.out = sigmoid(self.z2)
        return self.out

    def compute_grads(self, target):
        eps = 1e-15
        p = max(eps, min(1 - eps, self.out))
        d_loss = -(target / p) + (1 - target) / (1 - p)
        d_sigmoid = self.out * (1 - self.out)
        d_out = d_loss * d_sigmoid

        grads = [0.0] * (self.hidden_size * 2 + self.hidden_size + self.hidden_size + 1)

    )

    idx = 0
    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        d_h = d_out * self.w2[i] * d_relu
        grads[idx] = d_h * self.x[0]
        grads[idx + 1] = d_h * self.x[1]
        idx += 2

    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        grads[idx] = d_out * self.w2[i] * d_relu
        idx += 1

    for i in range(self.hidden_size):
        grads[idx] = d_out * self.h[i]
        idx += 1

    grads[idx] = d_out
    return grads

def train(self, data, epochs=300):
    losses = []
    for epoch in range(epochs):
        total_loss = 0.0
        correct = 0
        for x, y in data:
            pred = self.forward(x)
            grads = self.compute_grads(y)
            params = self.get_params()
            self.optimizer.step(params, grads)
            self.set_params(params)

            eps = 1e-15
            p = max(eps, min(1 - eps, pred))
            total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
            if (pred >= 0.5) == (y >= 0.5):
                correct += 1
        avg_loss = total_loss / len(data)
        accuracy = correct / len(data) * 100
        losses.append((avg_loss, accuracy))

```

```

        if epoch % 75 == 0 or epoch == epochs - 1:
            print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f}, accuracy={accuracy:.1f}%")
        return losses

```

Use It

PyTorch optimizers handle parameter groups, gradient clipping, and learning rate scheduling:

```

import torch
import torch.optim as optim

model = torch.nn.Sequential(
    torch.nn.Linear(784, 256),
    torch.nn.ReLU(),
    torch.nn.Linear(256, 10),
)

optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)

for epoch in range(100):
    optimizer.zero_grad()
    output = model(torch.randn(32, 784))
    loss = torch.nn.functional.cross_entropy(output, torch.randint(0, 10, (32,)))
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    scheduler.step()

```

The pattern is always: zero_grad, forward, loss, backward, (clip), step, (schedule). Memorize this order. Getting it wrong (e.g., calling scheduler.step() before optimizer.step()) is a common source of subtle bugs.

For CNNs, many practitioners still prefer SGD + momentum (lr=0.1, momentum=0.9, weight_decay=1e-4) with a step or cosine schedule. SGD finds flatter minima, which often generalize better. For transformers and LLMs, AdamW with warmup + cosine decay is the universal default. Don't fight the consensus without a measured reason.

Exercises

- Implement Nesterov momentum, where you compute the gradient at the "lookahead" position ($w - lr * \beta * v$) instead of the current position. Compare convergence to standard momentum on the circle dataset.
- Implement a learning rate warmup schedule: linear ramp from 0 to max_lr over the first 10% of training steps, then cosine decay to 0. Train with Adam + warmup vs Adam without warmup. Measure how many epochs it takes to reach 90% accuracy on the circle dataset.

- Track the effective learning rate for each parameter during Adam training. The effective rate is $\text{lr} * \hat{m} / (\sqrt{\hat{v}} + \epsilon)$. Plot the distribution of effective rates after 10, 50, and 200 steps. Are all parameters being updated at the same speed?
- Implement gradient clipping (clip by global norm). Set the max gradient norm to 1.0. Train with and without clipping using a high learning rate ($\text{lr}=0.01$ for Adam). Count how many runs diverge (loss goes to NaN) with and without clipping over 10 random seeds.
- Compare Adam vs AdamW on a network with large weights. Initialize all weights to random values in $[-5, 5]$ (much larger than normal). Train for 200 epochs with $\text{weight_decay}=0.1$. Plot the L2 norm of weights over training for both optimizers. AdamW should show faster weight shrinkage.

Key Terms

Term	What people say	What it actually means
Learning rate	“Step size”	The scalar multiplier on the gradient update; the single most impactful hyperparameter in training
SGD	“Basic gradient descent”	Stochastic gradient descent: update weights by subtracting $\text{lr} * \text{gradient}$, computed on a mini-batch
Momentum	“Rolling ball analogy”	Exponential moving average of past gradients; dampens oscillation and accelerates consistent directions
RMSProp	“Adaptive learning rate”	Divides each parameter’s gradient by the running RMS of its recent gradients; equalizes learning rates
Adam	“The default optimizer”	Combines momentum (first moment) and RMSProp (second moment) with bias correction for the initial steps
AdamW	“Adam done right”	Adam with decoupled weight decay; applies regularization directly to weights rather than through the gradient
Bias correction	“Warmup for running averages”	Dividing by $(1 - \beta^t)$ to compensate for the zero-initialization of Adam’s moment estimates
Weight decay	“Shrink the weights”	Subtracting a fraction of the weight value at each step; a regularizer that penalizes large weights
Learning rate schedule	“Changing lr over time”	A function that adjusts the learning rate during training; warmup + cosine decay is the modern default
Gradient clipping	“Capping the gradient norm”	Scaling down the gradient vector when its norm exceeds a threshold; prevents exploding gradient updates

You are an expert deep learning practitioner. Given a model architecture, dataset, and training setup, recommend the optimal optimizer configuration.

Analyze these factors:

- **Architecture:** Transformer, CNN, MLP, GAN, RNN, or hybrid
- **Scale:** Parameters (millions/billions), dataset size, batch size
- **Training stage:** From scratch, fine-tuning, or transfer learning
- **Compute budget:** Single GPU, multi-GPU, or distributed

Apply these rules:

Transformers / LLMs: - Optimizer: AdamW - Learning rate: $1e-4$ to $3e-4$ (pre-training), $1e-5$ to $5e-5$ (fine-tuning) - Weight decay: 0.01 to 0.1 - Beta1: 0.9, Beta2: 0.95 (LLM convention) or 0.999 (default) - Schedule: Linear warmup (1-10% of steps) + cosine decay to 0 or 10% of max lr - Gradient clipping: $\text{max_norm}=1.0$

CNNs / Vision: - Optimizer: SGD + Momentum (traditional) or AdamW (modern) - SGD config: $\text{lr}=0.1$, $\text{momentum}=0.9$, $\text{weight_decay}=1e-4$ - AdamW config: $\text{lr}=3e-4$, $\text{weight_decay}=0.05$ - Schedule: Step decay (divide by 10 at epochs 30, 60, 90) or cosine decay - Batch size: 256 (scale lr linearly with batch size)

GANs: - Optimizer: Adam (not AdamW – weight decay hurts GAN training) - Learning rate: $1e-4$ to $2e-4$ - Beta1: 0.0 or 0.5 (NOT 0.9 – momentum destabilizes GAN training) - Beta2: 0.999 - Equal lr for generator and discriminator (unless training is unstable)

Fine-tuning pretrained models: - Optimizer: AdamW - Learning rate: $2e-5$ to $5e-5$ (10-100x lower than pre-training) - Weight decay: 0.01 - Schedule: Linear warmup (first 6% of steps) + linear decay - Freeze early layers for small datasets

If unsure, start here: - AdamW, $\text{lr}=3e-4$, $\text{weight_decay}=0.01$, $\text{betas}=(0.9, 0.999)$ - Cosine schedule with 5% warmup - Gradient clipping at 1.0 - These defaults work for the majority of tasks

Debugging checklist when training fails: 1. Loss diverging: Reduce lr by 10x 2. Loss plateauing: Increase lr by 3x or add warmup 3. Training unstable (spikes): Add gradient clipping, reduce lr 4. Slow convergence with SGD: Switch to AdamW 5. Poor generalization with Adam: Switch to AdamW (decoupled weight decay)

For each recommendation, state: - The optimizer name and all hyperparameter values - The learning rate schedule (warmup steps, decay type, final lr) - Whether to use gradient clipping and at what threshold - What signs would indicate the configuration needs adjustment

Quiz

Pre-lesson check

- What problem does momentum solve in gradient descent?
 - a) It reduces memory usage
 - b) It dampens oscillation by accumulating past gradients, accelerating movement in consistent directions **(correct)**
 - c) It eliminates the need for a learning rate
 - d) It prevents overfitting

Explanation: In narrow valleys, gradients oscillate across the valley while making slow progress along it. Momentum accumulates past gradients: consistent directions amplify while oscillating directions cancel, giving smoother, faster convergence.
- What is the key difference between Adam and AdamW?
 - a) AdamW uses a higher learning rate
 - b) AdamW applies weight decay directly to weights instead of through the gradient, giving proper regularization **(correct)**
 - c) AdamW doesn't use momentum
 - d) AdamW only works with transformers

Explanation: In Adam + L2, the adaptive learning rate scales the regularization term differently per parameter. AdamW decouples weight decay from the gradient update, applying uniform shrinkage regardless of gradient statistics.

Post-lesson check

Why does Adam use bias correction in early training steps?

- a) To prevent overfitting
- b) The moment estimates are initialized to zero, so early values are biased toward zero; correction compensates for this cold start **(correct)**
- c) To clip large gradients

- d) To speed up convergence

*Explanation: At step 1 with $\beta_1=0.9$, $m_1 = 0.1$ gradient (10x too small). Dividing by $(1 - 0.9^1) = 0.1$ corrects this to the actual gradient. The correction becomes negligible after ~50 steps.**

What are the standard default hyperparameters for Adam?

- a) $\text{lr}=0.1$, $\beta_1=0.5$, $\beta_2=0.5$

- b) $\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$ **(correct)**

- c) $\text{lr}=0.01$, $\beta_1=0.99$, $\beta_2=0.99$

- d) $\text{lr}=0.0001$, $\beta_1=0.8$, $\beta_2=0.9$

Explanation: The original Adam paper (Kingma & Ba, 2014) recommended $\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$. These defaults work well for most problems.

Which optimizer is the modern default for training transformers and LLMs?

- a) Vanilla SGD

- b) SGD with momentum

- c) RMSProp

- d) AdamW **(correct)**

Explanation: AdamW (Adam with decoupled weight decay) is used to train BERT, GPT, LLaMA, and virtually all modern transformers. It combines adaptive learning rates with proper weight decay regularization.

Regularization

Your model gets 99% on training data and 60% on test data. It memorized instead of learning. Regularization is the tax you impose on complexity to force generalization.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.06 (Optimizers) **Time:** ~75 minutes

Learning Objectives

Implement dropout with inverted scaling, L2 weight decay, batch normalization, layer normalization, and RMSNorm from scratch

Measure the train-test accuracy gap and diagnose overfitting using regularization experiments

Explain why transformers use LayerNorm instead of BatchNorm and why modern LLMs prefer RMSNorm

Apply the correct combination of regularization techniques based on the severity of overfitting

The Problem

A neural network with enough parameters can memorize any dataset. This is not a hypothetical – Zhang et al. (2017) proved it by training standard networks on ImageNet with random labels. The networks reached near-zero training loss on completely random label assignments. They memorized a million random input-output pairs with no pattern to learn. Training loss was perfect. Test accuracy was zero.

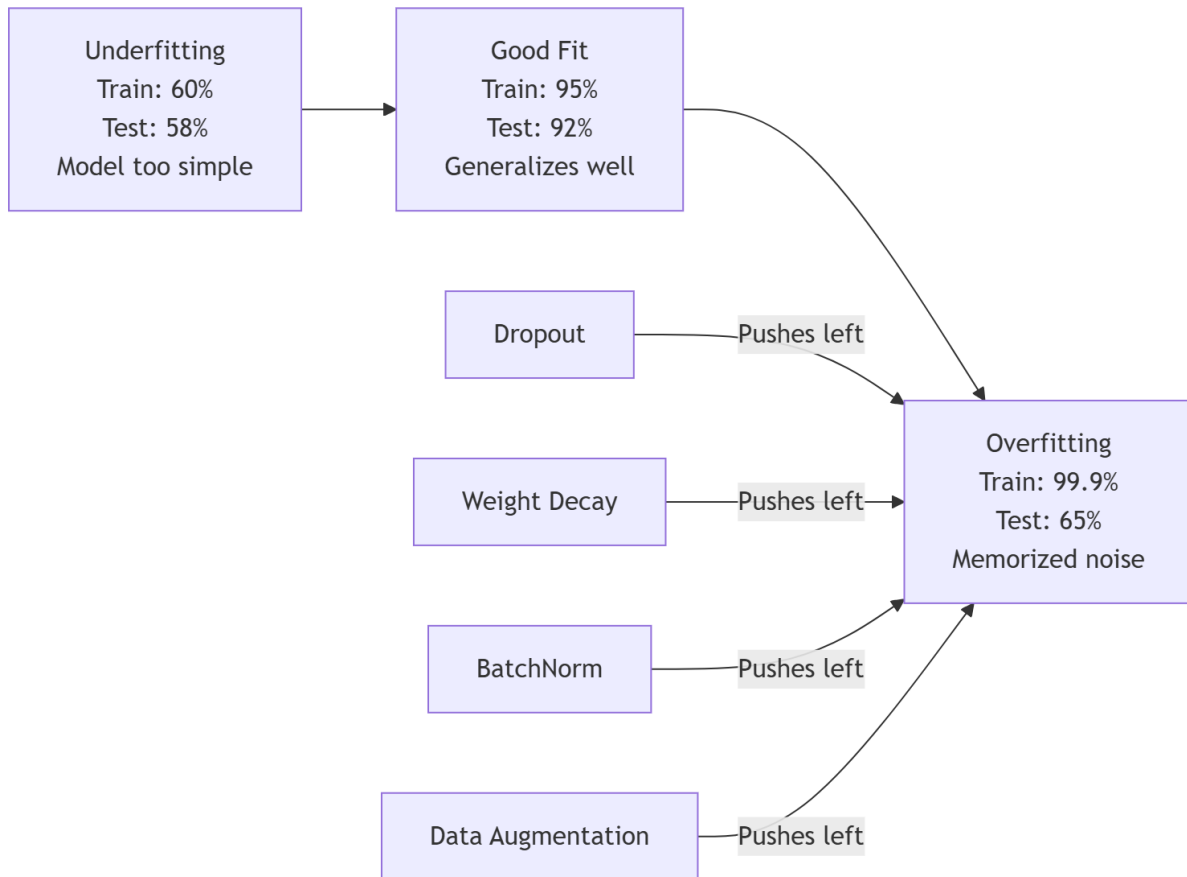
This is the overfitting problem, and it gets worse as models get larger. GPT-3 has 175 billion parameters. The training set has about 500 billion tokens. With that many parameters, the model has enough capacity to memorize significant chunks of the training data verbatim. Without regularization, it would just regurgitate training examples instead of learning generalizable patterns.

The gap between training performance and test performance is the overfitting gap. Every technique in this lesson attacks that gap from a different angle. Dropout forces the network to not rely on any single neuron. Weight decay prevents any single weight from growing too large. Batch normalization smooths the loss landscape so the optimizer finds flatter, more generalizable minima. Layer normalization does the same thing but works where batch normalization fails (small batches, variable-length sequences). RMSNorm does it 10% faster by dropping the mean calculation. Each technique is simple. Together, they're the difference between a model that memorizes and one that generalizes.

The Concept

The Overfitting Spectrum

Every model sits somewhere on a spectrum from underfitting (too simple to capture the pattern) to overfitting (so complex it captures noise). The sweet spot is in between, and regularization pushes models toward it from the overfit side.



Dropout

The simplest regularization technique with the most elegant interpretation. During training, randomly set each neuron's output to zero with probability p .

$$\text{output} = \text{activation}(z) * \text{mask} \quad \text{where } \text{mask}[i] \sim \text{Bernoulli}(1 - p)$$

With $p = 0.5$, half the neurons are zeroed on every forward pass. The network must learn redundant representations because it can't predict which neurons will be available. This prevents co-adaptation – neurons learning to rely on specific other neurons being present.

The ensemble interpretation: a network with N neurons and dropout creates 2^N possible subnetworks (every combination of which neurons are on or off). Training with dropout approximately trains all 2^N subnetworks simultaneously, each on different mini-batches. At test time, you use all neurons (no dropout) and scale outputs by $(1 - p)$ to match the expected value during training. This is equivalent to averaging the predictions of 2^N subnetworks – a massive ensemble from a single model.

In practice, the scaling is applied during training instead of testing (inverted dropout):

```
During training: output = activation(z) * mask / (1 - p)
During testing:  output = activation(z)    (no change needed)
```

This is cleaner because test code doesn't need to know about dropout at all.

Default rates: $p = 0.1$ for transformers, $p = 0.5$ for MLPs, $p = 0.2-0.3$ for CNNs. Higher dropout = stronger regularization = more underfitting risk.

Weight Decay (L2 Regularization)

Add the squared magnitude of all weights to the loss:

```
total_loss = task_loss + (lambda / 2) * sum(w_i^2)
```

The gradient of the regularization term is $\lambda * w$. This means at every step, each weight is shrunk toward zero by a fraction proportional to its magnitude. Large weights get penalized more. The model is pushed toward solutions where no single weight dominates.

Why this helps generalization: overfit models tend to have large weights that amplify noise in the training data. Weight decay keeps weights small, which limits the model's effective capacity and forces it to rely on robust, generalizable features rather than memorized quirks.

The λ hyperparameter controls the strength. Typical values:

- 0.01 for AdamW on transformers
- $1e-4$ for SGD on CNNs
- 0.1 for heavily overfit models

As discussed in lesson 06: weight decay and L2 regularization are equivalent in SGD but not in Adam. Always use AdamW (decoupled weight decay) when training with Adam.

Batch Normalization

Normalize the output of each layer across the mini-batch before passing it to the next layer.

For a mini-batch of activations at some layer:

```
mu = (1/B) * sum(x_i)           (batch mean)
sigma^2 = (1/B) * sum((x_i - mu)^2) (batch variance)
x_hat = (x_i - mu) / sqrt(sigma^2 + eps) (normalize)
y = gamma * x_hat + beta        (scale and shift)
```

γ and β are learnable parameters that let the network undo the normalization if that's optimal. Without them, you'd be forcing every layer's output to be zero-mean unit-variance, which might not be what the network wants.

Training vs inference split: During training, μ and σ come from the current mini-batch. During inference, you use running averages accumulated during training (exponential moving average with momentum = 0.1, meaning 90% old + 10% new).

Why BatchNorm works is still debated. The original paper claimed it reduces “internal covariate shift” (the distribution of layer inputs changing as earlier layers update). Santurkar et al. (2018) showed this explanation is wrong. The actual reason: BatchNorm makes the loss landscape smoother. The gradients are more predictive, the Lipschitz constants are smaller, and the optimizer can take larger steps safely. This is why BatchNorm lets you use higher learning rates and converge faster.

BatchNorm has a fundamental limitation: it depends on batch statistics. With batch size 1, the mean and variance are meaningless. With small batches (< 32), the statistics are noisy and hurt performance. This matters for tasks like object detection (where memory limits batch size) and language modeling (where sequence lengths vary).

Layer Normalization

Normalize across features instead of across the batch. For a single sample:

```
mu = (1/D) * sum(x_j)           (feature mean)
sigma^2 = (1/D) * sum((x_j - mu)^2) (feature variance)
x_hat = (x_j - mu) / sqrt(sigma^2 + eps)
y = gamma * x_hat + beta
```

D is the feature dimension. Each sample is normalized independently – no dependence on batch size. This is why transformers use LayerNorm instead of BatchNorm. Sequences have variable lengths, batch sizes are often small (or 1 during generation), and the computation is identical between training and inference.

LayerNorm in transformers is applied after each self-attention block and each feed-forward block (Post-LN), or before them (Pre-LN, which is more stable for training).

RMSNorm

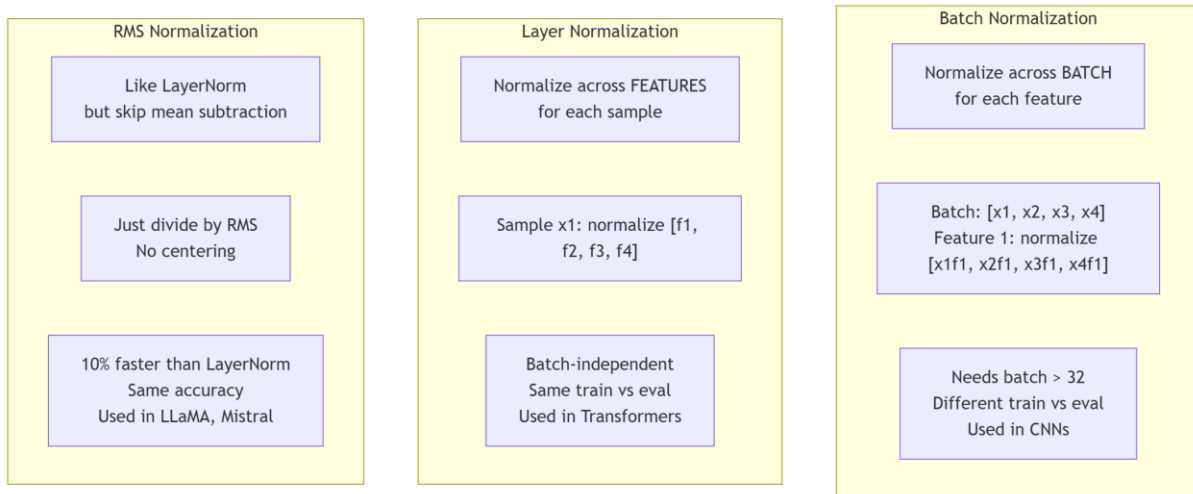
LayerNorm without the mean subtraction. Proposed by Zhang & Sennrich (2019).

```
rms = sqrt((1/D) * sum(x_j^2))
y = gamma * x / rms
```

That’s it. No mean computation, no β parameter. The observation: the re-centering (mean subtraction) in LayerNorm contributes very little to the model’s performance, but costs computation. Removing it gives the same accuracy with about 10% less overhead.

LLaMA, LLaMA 2, LLaMA 3, Mistral, and most modern LLMs use RMSNorm instead of LayerNorm. At the scale of billions of parameters and trillions of tokens, that 10% savings is significant.

Normalization Comparison



Data Augmentation as Regularization

Not a model modification but a data modification. Transform training inputs while preserving labels:

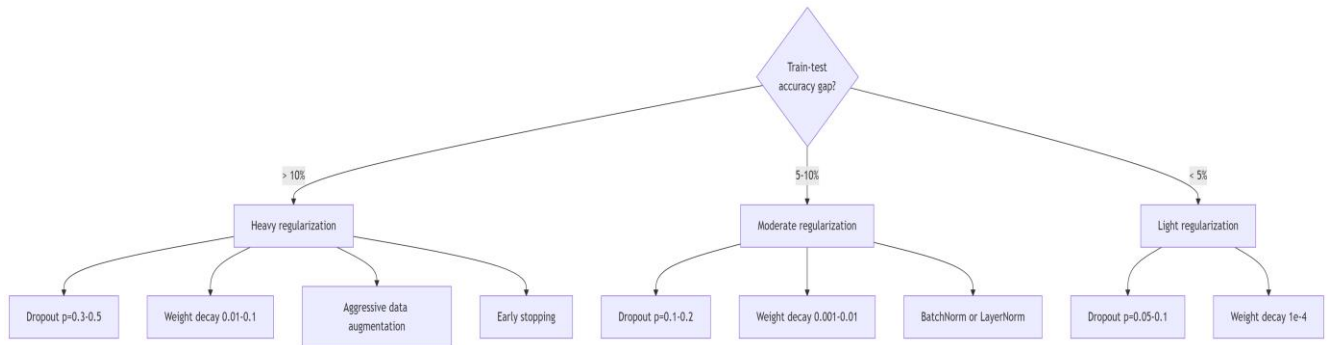
- 183. Images: random crop, flip, rotation, color jitter, cutout
- 184. Text: synonym replacement, back-translation, random deletion
- 185. Audio: time stretch, pitch shift, noise addition

The effect is identical to regularization: it increases the effective size of the training set, making it harder for the model to memorize specific examples. A model that only sees each image once in its original form can memorize it. A model that sees 50 augmented versions of each image is forced to learn the invariant structure.

Early Stopping

The simplest regularizer: stop training when validation loss starts increasing. The model hasn't overfit yet at that point. In practice, you track validation loss every epoch, save the best model, and continue training for a "patience" window (typically 5-20 epochs). If validation loss doesn't improve within the patience window, you stop and load the best saved model.

When to Apply What



Build It

Step 1: Dropout (Train and Eval Mode)

```
import random
import math

class Dropout:
    def __init__(self, p=0.5):
        self.p = p
        self.training = True
        self.mask = None

    def forward(self, x):
        if not self.training:
            return list(x)
        self.mask = []
        output = []
        for val in x:
            if random.random() < self.p:
                self.mask.append(0)
                output.append(0.0)
            else:
                self.mask.append(1)
                output.append(val / (1 - self.p))
        return output

    def backward(self, grad_output):
        grads = []
        for g, m in zip(grad_output, self.mask):
            if m == 0:
                grads.append(0.0)
            else:
                grads.append(g / (1 - self.p))
        return grads
```

Step 2: L2 Weight Decay

```
def l2_regularization(weights, lambda_reg):
    penalty = 0.0
    for w in weights:
        penalty += w * w
    return lambda_reg * 0.5 * penalty

def l2_gradient(weights, lambda_reg):
    return [lambda_reg * w for w in weights]
```

Step 3: Batch Normalization

```
class BatchNorm:
    def __init__(self, num_features, momentum=0.1, eps=1e-5):
        self.gamma = [1.0] * num_features
        self.beta = [0.0] * num_features
        self.eps = eps
        self.momentum = momentum
        self.running_mean = [0.0] * num_features
        self.running_var = [1.0] * num_features
        self.training = True
        self.num_features = num_features

    def forward(self, batch):
        batch_size = len(batch)
        if self.training:
            mean = [0.0] * self.num_features
            for sample in batch:
                for j in range(self.num_features):
                    mean[j] += sample[j]
            mean = [m / batch_size for m in mean]

            var = [0.0] * self.num_features
            for sample in batch:
                for j in range(self.num_features):
                    var[j] += (sample[j] - mean[j]) ** 2
            var = [v / batch_size for v in var]

            for j in range(self.num_features):
                self.running_mean[j] = (1 - self.momentum) * self.running_mean[j] + self.momentum * mean[j]
                self.running_var[j] = (1 - self.momentum) * self.running_var[j] + self.momentum * var[j]
        else:
            mean = list(self.running_mean)
            var = list(self.running_var)

        self.x_hat = []
        output = []
        for sample in batch:
            normalized = []
```

```

        out_sample = []
        for j in range(self.num_features):
            x_h = (sample[j] - mean[j]) / math.sqrt(var[j] + self.eps)
            normalized.append(x_h)
            out_sample.append(self.gamma[j] * x_h + self.beta[j])
        self.x_hat.append(normalized)
        output.append(out_sample)
    return output

```

Step 4: Layer Normalization

```

class LayerNorm:
    def __init__(self, num_features, eps=1e-5):
        self.gamma = [1.0] * num_features
        self.beta = [0.0] * num_features
        self.eps = eps
        self.num_features = num_features

    def forward(self, x):
        mean = sum(x) / len(x)
        var = sum((xi - mean) ** 2 for xi in x) / len(x)

        self.x_hat = []
        output = []
        for j in range(self.num_features):
            x_h = (x[j] - mean) / math.sqrt(var + self.eps)
            self.x_hat.append(x_h)
            output.append(self.gamma[j] * x_h + self.beta[j])
        return output

```

Step 5: RMSNorm

```

class RMSNorm:
    def __init__(self, num_features, eps=1e-6):
        self.gamma = [1.0] * num_features
        self.eps = eps
        self.num_features = num_features

    def forward(self, x):
        rms = math.sqrt(sum(xi * xi for xi in x) / len(x) + self.eps)
        output = []
        for j in range(self.num_features):
            output.append(self.gamma[j] * x[j] / rms)
        return output

```

Step 6: Training With and Without Regularization

```

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

```

```

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

class RegularizedNetwork:
    def __init__(self, hidden_size=16, lr=0.05, dropout_p=0.0, weight_decay=0.0):
        random.seed(0)
        self.hidden_size = hidden_size
        self.lr = lr
        self.dropout_p = dropout_p
        self.weight_decay = weight_decay
        self.dropout = Dropout(p=dropout_p) if dropout_p > 0 else None

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in range(hidden_size)]

        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def forward(self, x, training=True):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
            self.z1.append(z)
            self.h.append(max(0.0, z))

            if self.dropout and training:
                self.dropout.training = True
                self.h = self.dropout.forward(self.h)
            elif self.dropout:
                self.dropout.training = False
                self.h = self.dropout.forward(self.h)

        self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) + self.b2

        self.out = sigmoid(self.z2)
        return self.out

    def backward(self, target):
        eps = 1e-15
        p = max(eps, min(1 - eps, self.out))
        d_loss = -(target / p) + (1 - target) / (1 - p)
        d_sigmoid = self.out * (1 - self.out)

```



```

d_out = d_loss * d_sigmoid

for i in range(self.hidden_size):
    d_relu = 1.0 if self.z1[i] > 0 else 0.0
    d_h = d_out * self.w2[i] * d_relu
    self.w2[i] -= self.lr * (d_out * self.h[i] + self.weight_decay * self.w2[i])
)

for j in range(2):
    self.w1[i][j] -= self.lr * (d_h * self.x[j] + self.weight_decay * self.w1[i][j])
    self.b1[i] -= self.lr * d_h
self.b2 -= self.lr * d_out

def evaluate(self, data):
    correct = 0
    total_loss = 0.0
    for x, y in data:
        pred = self.forward(x, training=False)
        eps = 1e-15
        p = max(eps, min(1 - eps, pred))
        total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
        if (pred >= 0.5) == (y >= 0.5):
            correct += 1
    return total_loss / len(data), correct / len(data) * 100

def train_model(self, train_data, test_data, epochs=300):
    history = []
    for epoch in range(epochs):
        total_loss = 0.0
        correct = 0
        for x, y in train_data:
            pred = self.forward(x, training=True)
            self.backward(y)
            eps = 1e-15
            p = max(eps, min(1 - eps, pred))
            total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
            if (pred >= 0.5) == (y >= 0.5):
                correct += 1
        train_loss = total_loss / len(train_data)
        train_acc = correct / len(train_data) * 100
        test_loss, test_acc = self.evaluate(test_data)
        history.append((train_loss, train_acc, test_loss, test_acc))
        if epoch % 75 == 0 or epoch == epochs - 1:
            gap = train_acc - test_acc
            print(f"    Epoch {epoch:3d}: train_acc={train_acc:.1f}%, test_acc={test_acc:.1f}%, gap={gap:.1f}%")
    return history

```

Use It

PyTorch provides all normalization and regularization as modules:

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 10),
)

model.train()
out_train = model(torch.randn(32, 784))

model.eval()
out_test = model(torch.randn(1, 784))

```

The `model.train()` / `model.eval()` toggle is critical. It switches dropout on/off and tells BatchNorm to use batch statistics vs running statistics. Forgetting `model.eval()` before inference is one of the most common bugs in deep learning. Your test accuracy will fluctuate randomly because dropout is still active and BatchNorm is using mini-batch statistics.

For transformers, the pattern is different:

```

class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, nhead=8, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_model * 4),
            nn.GELU(),
            nn.Linear(d_model * 4, d_model),
            nn.Dropout(dropout),
        )
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        attended, _ = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attended))
        x = self.norm2(x + self.ff(x))
        return x

```

LayerNorm, not BatchNorm. Dropout $p=0.1$, not $p=0.5$. These are the transformer defaults.

Exercises

- Implement spatial dropout for 2D data: instead of dropping individual neurons, drop entire feature channels. Simulate this by treating groups of consecutive features as channels and dropping whole groups. Compare the train-test gap to standard dropout on the circle dataset with `hidden_size=32`.
- Implement label smoothing from lesson 05 combined with dropout from this lesson. Train with four configurations: neither, dropout only, label smoothing only, both. Measure the final train-test accuracy gap for each. Which combination gives the smallest gap?
- Add a BatchNorm layer between the hidden layer and the activation in your circle-dataset network. Train with and without BatchNorm at learning rates 0.01, 0.05, and 0.1. BatchNorm should allow stable training at higher learning rates where the vanilla network diverges.
- Implement early stopping: track test loss each epoch, save the best weights, and stop if test loss hasn't improved for 20 epochs. Run the regularized network for 1000 epochs. Report which epoch had the best test accuracy and how many epochs of computation you saved.
- Compare LayerNorm vs RMSNorm on a 4-layer network (not just 2). Initialize both with the same weights. Train for 200 epochs and compare final accuracy, training speed (time per epoch), and gradient magnitudes at the first layer. Verify that RMSNorm is faster with the same accuracy.

Key Terms

Term	What people say	What it actually means
Overfitting	"Model memorized the data"	When a model's training performance significantly exceeds its test performance, indicating it learned noise rather than signal
Regularization	"Preventing overfitting"	Any technique that constrains model complexity to improve generalization: dropout, weight decay, normalization, augmentation
Dropout	"Random neuron deletion"	Zeroing random neurons during training with probability p , forcing redundant representations; equivalent to training an ensemble
Weight decay	"L2 penalty"	Shrinking all weights toward zero by subtracting $\lambda * w$ at each step; penalizes complexity through weight magnitude
Batch normalization	"Normalize per batch"	Normalizing layer outputs across the batch dimension using batch statistics during training and running averages during inference
Layer normalization	"Normalize per sample"	Normalizing across features within each sample; batch-independent, used in transformers where batch size varies

Term	What people say	What it actually means
RMSNorm	“LayerNorm without the mean”	Root mean square normalization; drops the mean subtraction from LayerNorm for 10% speedup with equal accuracy
Early stopping	“Stop before overfit”	Halting training when validation loss stops improving; the simplest regularizer, often used alongside others
Data augmentation	“More data from less”	Transforming training inputs (flip, crop, noise) to increase effective dataset size and force invariance learning
Generalization gap	“Train-test split”	The difference between training and test performance; regularization aims to minimize this gap

You are an expert ML engineer specializing in model generalization. Given training metrics and model details, diagnose overfitting and recommend a regularization strategy.

Analyze these inputs:

- **Training accuracy vs test/validation accuracy** (the gap)
- **Model size:** Number of parameters relative to dataset size
- **Architecture:** Transformer, CNN, MLP, or other
- **Current regularization:** What’s already applied
- **Training duration:** How many epochs, has validation loss started increasing

Apply these diagnostic rules:

Gap < 3%: No significant overfitting - Continue training, model may still be underfitting - Consider increasing model capacity if test accuracy is low

Gap 3-10%: Mild overfitting - Add dropout ($p=0.1$ for transformers, $p=0.2-0.3$ for MLPs/CNNs) - Add weight decay (0.01 for AdamW, $1e-4$ for SGD) - Add normalization if not present (LayerNorm for transformers, BatchNorm for CNNs)

Gap 10-20%: Moderate overfitting - All of the above, plus: - Data augmentation (random crop, flip, color jitter for images) - Label smoothing ($\alpha=0.1$) - Early stopping (patience=10-20 epochs) - Reduce model capacity (fewer layers or smaller hidden dim)

Gap > 20%: Severe overfitting - All of the above, plus: - Increase dropout to $p=0.3-0.5$ - Increase weight decay to 0.1 - Aggressive data augmentation (mixup, cutmix, randaugment) - Consider getting more training data - Consider simpler model architecture

Architecture-specific defaults:

Transformers: - LayerNorm (or RMSNorm) after attention and FFN blocks - Dropout $p=0.1$ on attention weights and residual connections - Weight decay 0.01-0.1 via AdamW - Label smoothing 0.1

CNNs: - BatchNorm after convolutions - Dropout $p=0.2-0.5$ before final linear layers (not between conv layers) - Weight decay $1e-4$ - Data augmentation (critical for CNNs)

MLPs: - Dropout $p=0.3-0.5$ between hidden layers - BatchNorm or LayerNorm between layers - Weight decay 0.01 - Careful: MLPs overfit easily, regularization is essential

Common mistakes: - Applying BatchNorm with batch size < 16 (use LayerNorm instead) - Forgetting `model.eval()` during inference (dropout stays active, BatchNorm uses batch stats) - Using the same dropout rate everywhere (attention needs less than FFN) - Weight decay on bias and normalization parameters (exclude them)

For each recommendation: - State the technique and its hyperparameters - Explain why it addresses the specific overfitting pattern - Specify the expected impact on the train-test gap - Warn about any side effects (e.g., dropout slows convergence)

Quiz

Pre-lesson check

186. What is overfitting in neural networks?

- a) The model is too small to learn the data
- b) The model memorizes training data instead of learning generalizable patterns, showing a large gap between train and test accuracy **(correct)**
- c) The model trains too slowly
- d) The loss function is wrong

Explanation: Overfitting occurs when a model achieves high training accuracy but poor test accuracy. It has memorized the training data's noise rather than learning the underlying patterns.

187. How does dropout regularize a neural network?

- a) It removes the worst-performing neurons permanently

- b) It randomly zeroes neurons during training, forcing the network to learn redundant representations **(correct)**
- c) It reduces the learning rate
- d) It removes outliers from the training data
Explanation: During each forward pass, dropout randomly sets neuron outputs to zero with probability p . This prevents co-adaptation (neurons relying on specific others) and is equivalent to training an ensemble of 2^N subnetworks.

Post-lesson check

Why do transformers use LayerNorm instead of BatchNorm?

- a) LayerNorm is faster to compute
- b) LayerNorm normalizes across features per sample (batch-independent), which works with variable sequence lengths and small batch sizes **(correct)**
- c) BatchNorm causes gradient explosion
- d) LayerNorm was invented more recently
Explanation: BatchNorm depends on batch statistics, which are noisy with small batches and meaningless with batch size 1 (common during generation). LayerNorm normalizes across features within each sample, independent of batch size.

What is the key difference between RMSNorm and LayerNorm?

- a) RMSNorm uses batch statistics
- b) RMSNorm skips the mean subtraction, only dividing by the root mean square, giving ~10% speedup with equal accuracy **(correct)**
- c) RMSNorm adds learnable parameters
- d) RMSNorm only works on CNNs
Explanation: RMSNorm removes the mean subtraction step from LayerNorm, which contributes little to accuracy but adds computation. LLaMA, Mistral, and most modern LLMs use RMSNorm for this efficiency gain.

Why is it critical to call `model.eval()` before running inference in PyTorch?

- a) It speeds up computation
- b) It disables dropout and switches BatchNorm to use running statistics instead of batch statistics, giving deterministic outputs **(correct)**

c) It frees GPU memory

d) It enables gradient computation

Explanation: Without `model.eval()`, dropout randomly zeroes neurons during inference (causing random output variation) and BatchNorm uses current-batch statistics instead of the stable running averages accumulated during training.

Weight Initialization and Training Stability

Initialize wrong and training never starts. Initialize right and 50 layers train as smoothly as 3.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.04 (Activation Functions), Lesson 03.07 (Regularization) **Time:** ~90 minutes

Learning Objectives

Implement zero, random, Xavier/Glorot, and Kaiming/He initialization strategies and measure their effect on activation magnitudes through 50 layers

Derive why Xavier init uses $\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$ and Kaiming uses $\text{Var}(w) = 2/\text{fan_in}$

Demonstrate the symmetry problem with zero initialization and explain why random scale alone is insufficient

Match the correct initialization strategy to the activation function: Xavier for sigmoid/tanh, Kaiming for ReLU/GELU

The Problem

Initialize all weights to zero. Nothing learns. Every neuron computes the same function, receives the same gradient, and updates identically. After 10,000 epochs, your 512-neuron hidden layer is still 512 copies of the same neuron. You paid for 512 parameters and got 1.

Initialize them too large. Activations explode through the network. By layer 10, values hit $1e15$. By layer 20, they overflow to ∞ . Gradients follow the same trajectory in reverse.

Initialize them randomly from a standard normal distribution. Works for 3 layers. At 50 layers, the signal collapses to zero or detonates to ∞ depending on whether the random scale was slightly too small or slightly too large. The boundary between “works” and “broken” is razor-thin.

Weight initialization is the most underrated decision in deep learning. Architecture gets papers. Optimizers get blog posts. Initialization gets a footnote. But get it wrong and nothing else matters – your network is dead before training begins.

The Concept

The Symmetry Problem

Every neuron in a layer has the same structure: multiply inputs by weights, add bias, apply activation. If all weights start at the same value (zero is the extreme case), every neuron computes the same output. During backpropagation, every neuron receives the same gradient. During the update step, every neuron changes by the same amount.

You're stuck. The network has hundreds of parameters, but they all move in lockstep. This is called symmetry, and random initialization is the brute-force way to break it. Each neuron starts at a different point in weight space, so each learns a different feature.

But "random" is not enough. The *scale* of the randomness determines whether the network trains.

Variance Propagation Through Layers

Consider a single layer with `fan_in` inputs:

$$z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$

If each weight w_i is drawn from a distribution with variance $\text{Var}(w)$ and each input x has variance $\text{Var}(x)$, the output variance is:

$$\text{Var}(z) = \text{fan_in} * \text{Var}(w) * \text{Var}(x)$$

If $\text{Var}(w) = 1$ and `fan_in` = 512, the output variance is 512x the input variance. After 10 layers: $512^{10} = 1.2e27$. Your signal has exploded.

If $\text{Var}(w) = 0.001$, the output variance shrinks by $0.001 * 512 = 0.512$ per layer. After 10 layers: $0.512^{10} = 0.00013$. Your signal has vanished.

The goal: choose $\text{Var}(w)$ so that $\text{Var}(z) = \text{Var}(x)$. Signal magnitude stays constant across layers.

Xavier/Glorot Initialization

Glorot and Bengio (2010) derived the solution for sigmoid and tanh activations. To keep variance constant in both the forward and backward pass:

$$\text{Var}(w) = 2 / (\text{fan_in} + \text{fan_out})$$

In practice, weights are drawn from:

$$w \sim \text{Uniform}(-\text{limit}, \text{limit}) \quad \text{where } \text{limit} = \sqrt{6 / (\text{fan_in} + \text{fan_out})}$$

or:

$$w \sim \text{Normal}(0, \sqrt{2 / (\text{fan_in} + \text{fan_out})})$$

This works because sigmoid and tanh are roughly linear near zero, where properly initialized activations live. The variance stays stable through dozens of layers.

Kaiming/He Initialization

ReLU kills half the outputs (everything negative becomes zero). The effective fan_in is halved because on average half the inputs are zeroed. Xavier init doesn't account for this – it underestimates the variance needed.

He et al. (2015) adjusted the formula:

$$\text{Var}(w) = 2 / \text{fan_in}$$

Weights are drawn from:

$$w \sim \text{Normal}(0, \sqrt{2 / \text{fan_in}})$$

The factor of 2 compensates for ReLU zeroing half the activations. Without it, the signal shrinks by $\sim 0.5x$ per layer. With 50 layers: $0.5^{50} = 8.8e-16$. Kaiming init prevents this.

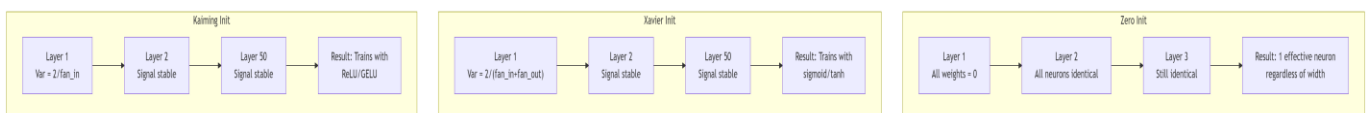
Transformer Initialization

GPT-2 introduced a different pattern. Residual connections add the output of each sub-layer to its input:

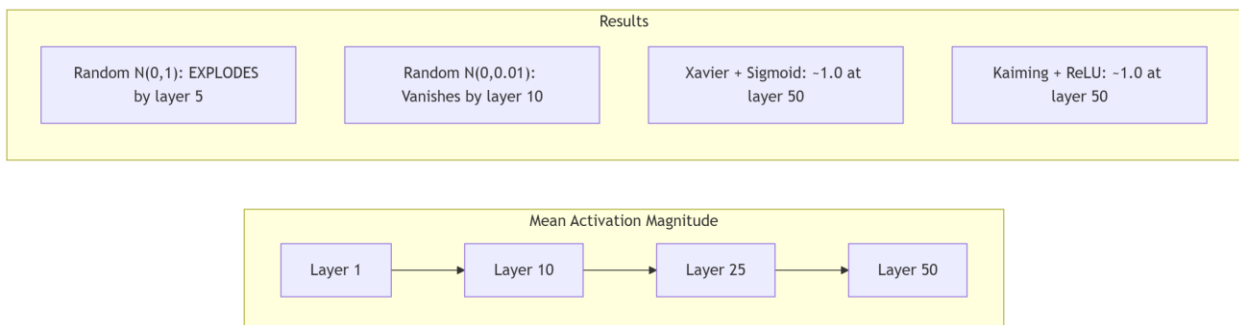
$$x = x + \text{sublayer}(x)$$

Each addition increases variance. With N residual layers, variance grows proportionally to N. GPT-2 scales the weights of residual layers by $1/\sqrt{2N}$, where N is the number of layers. This keeps the accumulated signal magnitude stable.

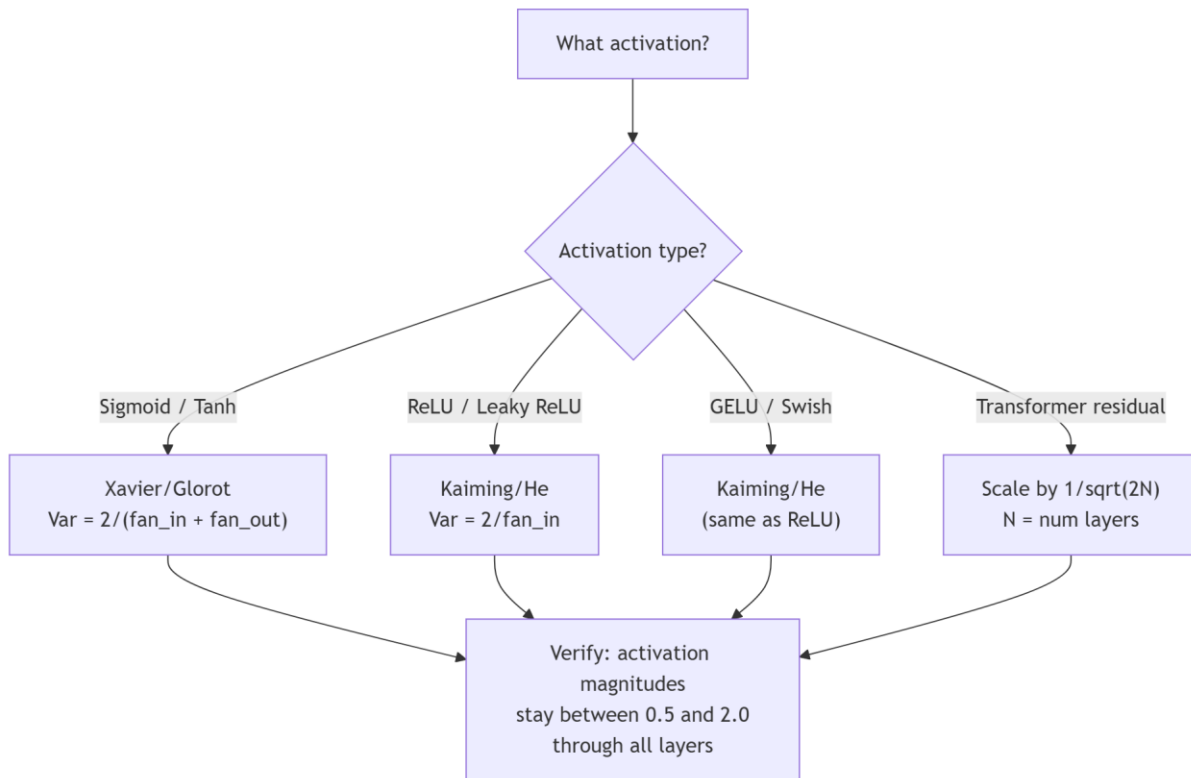
Llama 3 (405B parameters, 126 layers) uses a similar scheme. Without this scaling, the residual stream would grow unbounded through 126 layers of attention and feedforward blocks.



Activation Magnitude Through 50 Layers



Choosing the Right Init



Build It

Step 1: Initialization Strategies

Four ways to initialize a weight matrix. Each returns a list of lists (a 2D matrix) with `fan_in` columns and `fan_out` rows.

```
import math
import random

def zero_init(fan_in, fan_out):
    return [[0.0 for _ in range(fan_in)] for _ in range(fan_out)]

def random_init(fan_in, fan_out, scale=1.0):
    return [[random.gauss(0, scale) for _ in range(fan_in)] for _ in range(fan_out)]

def xavier_init(fan_in, fan_out):
    std = math.sqrt(2.0 / (fan_in + fan_out))
    return [[random.gauss(0, std) for _ in range(fan_in)] for _ in range(fan_out)]
```

```
def kaiming_init(fan_in, fan_out):
    std = math.sqrt(2.0 / fan_in)
    return [[random.gauss(0, std) for _ in range(fan_in)] for _ in range(fan_out)]
```

Step 2: Activation Functions

We need sigmoid, tanh, and ReLU to test each init strategy with its intended activation.

```
def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def tanh_act(x):
    return math.tanh(x)

def relu(x):
    return max(0.0, x)
```

Step 3: Forward Pass Through 50 Layers

Pass random data through a deep network and measure mean activation magnitude at each layer.

```
def forward_deep(init_fn, activation_fn, n_layers=50, width=64, n_samples=100):
    random.seed(42)
    layer_magnitudes = []

    inputs = [[random.gauss(0, 1) for _ in range(width)] for _ in range(n_samples)]

    for layer_idx in range(n_layers):
        weights = init_fn(width, width)
        biases = [0.0] * width

        new_inputs = []
        for sample in inputs:
            output = []
            for neuron_idx in range(width):
                z = sum(weights[neuron_idx][j] * sample[j] for j in range(width)) + biases[neuron_idx]
                output.append(activation_fn(z))
            new_inputs.append(output)
        inputs = new_inputs

        magnitudes = []
        for sample in inputs:
            magnitudes.append(sum(abs(v) for v in sample) / width)
        mean_mag = sum(magnitudes) / len(magnitudes)
        layer_magnitudes.append(mean_mag)

    return layer_magnitudes
```

Step 4: The Experiment

Run all combinations: zero init, random $N(0,1)$, random $N(0,0.01)$, Xavier with sigmoid, Xavier with tanh, Kaiming with ReLU. Print the magnitude at key layers.

```
def run_experiment():
    configs = [
        ("Zero init + Sigmoid", lambda fi, fo: zero_init(fi, fo), sigmoid),
        ("Random  $N(0,1)$  + ReLU", lambda fi, fo: random_init(fi, fo, 1.0), relu),
        ("Random  $N(0,0.01)$  + ReLU", lambda fi, fo: random_init(fi, fo, 0.01), relu),
        ("Xavier + Sigmoid", xavier_init, sigmoid),
        ("Xavier + Tanh", xavier_init, tanh_act),
        ("Kaiming + ReLU", kaiming_init, relu),
    ]

    print(f"{'Strategy':<30} {'L1':>10} {'L5':>10} {'L10':>10} {'L25':>10} {'L50':>10}")
    print("-" * 80)

    for name, init_fn, act_fn in configs:
        mags = forward_deep(init_fn, act_fn)
        row = f"{'name':<30}"
        for idx in [0, 4, 9, 24, 49]:
            val = mags[idx]
            if val > 1e6:
                row += f" {'EXPLODED':>10}"
            elif val < 1e-6:
                row += f" {'VANISHED':>10}"
            else:
                row += f" {val:>10.4f}"
        print(row)
```

Step 5: Symmetry Demonstration

Show that zero init produces identical neurons.

```
def symmetry_demo():
    random.seed(42)
    weights = zero_init(2, 4)
    biases = [0.0] * 4

    inputs = [0.5, -0.3]
    outputs = []
    for neuron_idx in range(4):
        z = sum(weights[neuron_idx][j] * inputs[j] for j in range(2)) + biases[neuron_idx]
        outputs.append(sigmoid(z))

    print("\nSymmetry Demo (4 neurons, zero init):")
    for i, out in enumerate(outputs):
```

```

    print(f"  Neuron {i}: output = {out:.6f}")
    all_same = all(abs(outputs[i] - outputs[0]) < 1e-10 for i in range(len(outputs)))
    print(f"  All identical: {all_same}")
    print(f"  Effective parameters: 1 (not {len(weights) * len(weights[0])})")

```

Step 6: Layer-by-Layer Magnitude Report

Print a visual bar chart of activation magnitudes through 50 layers.

```

def magnitude_report(name, magnitudes):
    print(f"\n{name}:")
    for i, mag in enumerate(magnitudes):
        if i % 5 == 0 or i == len(magnitudes) - 1:
            if mag > 1e6:
                bar = "X" * 50 + " EXPLODED"
            elif mag < 1e-6:
                bar = "." + " VANISHED"
            else:
                bar_len = min(50, max(1, int(mag * 10)))
                bar = "#" * bar_len
            print(f"  Layer {i+1:3d}: {bar} ({mag:.6f}")

```

Use It

PyTorch provides these as built-in functions:

```

import torch
import torch.nn as nn

layer = nn.Linear(512, 256)

nn.init.xavier_uniform_(layer.weight)
nn.init.xavier_normal_(layer.weight)

nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')

nn.init.zeros_(layer.bias)

```

When you call `nn.Linear(512, 256)`, PyTorch defaults to Kaiming uniform initialization. That’s why most simple networks “just work” – PyTorch already made the right choice. But when you build custom architectures or go deeper than 20 layers, you need to understand what’s happening and potentially override the default.

For transformers, HuggingFace models typically handle initialization in their `_init_weights` method. GPT-2’s implementation scales residual projections by $1/\sqrt{N}$. If you’re building a transformer from scratch, you need to add this yourself.

Exercises

- Add LeCun initialization ($\text{Var} = 1/\text{fan_in}$, designed for SELU activation). Run the 50-layer experiment with LeCun init + tanh and compare to Xavier + tanh.
- Implement the GPT-2 residual scaling: multiply the output of each layer by $1/\sqrt{2N}$ before adding to the residual stream. Run 50 layers with and without scaling, measure how fast the residual magnitude grows.
- Create an “init health check” function that takes a network’s layer dimensions and activation type, then recommends the correct initialization and warns if the current init will cause problems.
- Run the experiment with $\text{fan_in} = 16$ vs $\text{fan_in} = 1024$. Xavier and Kaiming adapt to fan_in , but random init doesn’t. Show how the gap between “works” and “breaks” widens with larger layers.
- Implement orthogonal initialization (generate a random matrix, compute its SVD, use the orthogonal matrix U). Compare to Kaiming for ReLU networks at 50 layers.

Key Terms

Term	What people say	What it actually means
Weight initialization	“Set starting weights randomly”	The strategy for choosing initial weight values that determines whether a network can train at all
Symmetry breaking	“Make neurons different”	Using random initialization to ensure neurons learn distinct features instead of computing identical functions
Fan-in	“Number of inputs to a neuron”	The number of incoming connections, which determines how input variance accumulates in the weighted sum
Fan-out	“Number of outputs from a neuron”	The number of outgoing connections, relevant for maintaining gradient variance during backpropagation
Xavier/Glorot init	“The sigmoid initialization”	$\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$, designed to preserve variance through sigmoid and tanh activations
Kaiming/He init	“The ReLU initialization”	$\text{Var}(w) = 2/\text{fan_in}$, accounts for ReLU zeroing half the activations
Variance propagation	“How signals grow or shrink through layers”	The mathematical analysis of how activation variance changes layer by layer based on weight scale
Residual scaling	“GPT-2’s init trick”	Scaling residual connection weights by $1/\sqrt{2N}$ to prevent variance growth through N transformer layers

Term	What people say	What it actually means
Dead network	"Nothing trains"	A network where poor initialization causes all gradients to be zero or all activations to saturate
Exploding activations	"Values go to ∞ "	When weight variance is too high, causing activation magnitudes to grow exponentially through layers

You are a neural network initialization expert. Given a network architecture and observed training behavior, diagnose initialization problems and recommend the correct strategy.

Diagnostic Protocol

1. Gather Architecture Details

Before recommending initialization, determine: - Layer types and sizes (Linear, Conv2d, Embedding, etc.) - Activation functions used in hidden layers - Whether residual connections exist - Total depth (number of weight layers) - Framework being used (PyTorch, TensorFlow, JAX)

2. Match Init to Architecture

Apply these rules:

Sigmoid or Tanh activations: - Use Xavier/Glorot: $\text{Var}(w) = 2 / (\text{fan_in} + \text{fan_out})$ - PyTorch: `nn.init.xavier_normal_(layer.weight)` or `nn.init.xavier_uniform_(layer.weight)` - Bias: initialize to zero

ReLU, Leaky ReLU, or GELU activations: - Use Kaiming/He: $\text{Var}(w) = 2 / \text{fan_in}$ - PyTorch: `nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')` - Bias: initialize to zero

Transformer with residual connections: - Use Kaiming for attention and feedforward weights - Scale residual projection weights by $1/\sqrt{2*N}$ where N = number of layers - Embedding layers: `Normal(0, 0.02)` is the GPT convention

Convolutional layers: - Same rules as linear: Kaiming for ReLU, Xavier for sigmoid/tanh - $\text{fan_in} = \text{channels_in} * \text{kernel_height} * \text{kernel_width}$

Batch/Layer normalization: - Weight (γ): initialize to 1.0 - Bias (β): initialize to 0.0

3. Diagnose Common Problems

Symptoms of bad initialization:

Symptom	Likely Cause	Fix
Loss stuck at random baseline from epoch 0	Zero init or symmetric init	Use Xavier/Kaiming random init
Loss immediately NaN or Inf	Scale too large, activations overflow	Reduce init scale, use Kaiming
Loss decreases then plateaus early	Vanishing activations in deep layers	Switch from Xavier to Kaiming for ReLU
Some neurons always output zero	Dead neurons from ReLU + bad init	Use Kaiming, or switch to GELU
Gradient magnitudes vary 1000x across layers	Inconsistent init strategy	Apply same init scheme to all layers

4. Verification Steps

After applying initialization, verify with:

```
for name, param in model.named_parameters():
    if 'weight' in name:
        print(f"{name:40s} | mean: {param.data.mean():.4e} | std: {param.data.std():.4e}")
```

Then after one forward pass:

```
hooks = []
for name, module in model.named_modules():
    if isinstance(module, nn.Linear):
        hooks.append(module.register_forward_hook(
            lambda m, i, o, n=name: print(f"{n:30s} | act mean: {o.abs().mean():.4f} | act std: {o.std():.4f}")
        ))
```

Healthy signs: - Activation means between 0.1 and 2.0 across all layers - No layer with all-zero activations - Standard deviation roughly consistent across layers

Quiz

Pre-lesson check

- What happens if you initialize all weights in a neural network to zero?

1. The network trains normally but slowly
 2. All neurons compute identical outputs and receive identical gradients, so the network has only 1 effective neuron per layer **(correct)**
 3. The network diverges
 4. Zero init is the recommended default
Explanation: With zero weights, every neuron in a layer computes the same function, receives the same gradient, and updates identically. This 'symmetry' means hundreds of parameters behave as one.
- Why does the scale of random weight initialization matter?
 - Larger weights train faster
 - If variance is too high, activations explode; if too low, activations vanish – both prevent training **(correct)**
 - Scale only matters for the output layer
 - It doesn't matter as long as weights are nonzero
*Explanation: Each layer multiplies variance by fan_in . $\text{Var}(w)$. If this product is > 1 , signal explodes exponentially through layers. If < 1 , it vanishes. Proper initialization keeps this product at exactly 1.**

Post-lesson check

188. What is the formula for Kaiming/He initialization variance?

- a) $\text{Var}(w) = 1/\text{fan_in}$
- b) $\text{Var}(w) = 2/\text{fan_in}$ **(correct)**
- c) $\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$
- d) $\text{Var}(w) = 1/(\text{fan_in} + \text{fan_out})$

Explanation: Kaiming init uses $\text{Var}(w) = 2/\text{fan_in}$. The factor of 2 compensates for ReLU zeroing half the activations (negative values become 0), which effectively halves the fan_in .

189. When should you use Xavier/Glorot initialization instead of Kaiming/He?

- a) Always – Xavier is universally better
- b) When using sigmoid or tanh activations, which don't zero half the outputs like ReLU
(correct)
- c) When training on small datasets

- d) When using Adam optimizer

Explanation: Xavier init uses $Var(w) = 2/(fan_in + fan_out)$, designed for activations that are roughly linear near zero (sigmoid, tanh). Kaiming's extra factor of 2 compensates for ReLU's half-zeroing, which Xavier doesn't need.

190. Why does GPT-2 scale residual layer weights by $1/\sqrt{2N}$?

- a) To speed up training
- b) Each residual addition increases variance, so scaling prevents the accumulated signal from growing unbounded through N layers **(correct)**
- c) To reduce the number of parameters
- d) To improve tokenization

Explanation: Residual connections add sublayer output to the input: $x = x + sublayer(x)$. Each addition increases variance. With N residual layers, variance grows proportionally to N. Scaling by $1/\sqrt{2N}$ keeps the signal stable.

Learning Rate Schedules and Warmup

The learning rate is the single most important hyperparameter. Not the architecture. Not the dataset size. Not the activation function. The learning rate. If you tune nothing else, tune this.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.06 (Optimizers), Lesson 03.08 (Weight Initialization) **Time:** ~90 minutes

Learning Objectives

64. Implement constant, step decay, cosine annealing, warmup + cosine, and 1cycle learning rate schedules from scratch
65. Demonstrate the three failure modes of learning rate selection: divergence (too high), stalling (too low), and oscillation (no decay)
66. Explain why warmup is necessary for Adam-based optimizers and how it stabilizes early training
67. Compare convergence speed across all five schedules on the same task and select the appropriate one for a given training budget

The Problem

Set the learning rate to 0.1. Training diverges – loss jumps to ∞ in 3 steps. Set it to 0.0001. Training crawls – after 100 epochs, the model has barely moved from random. Set it to 0.01. Training works for 50 epochs, then the loss oscillates around a minimum it can never reach because the steps are too large.

The optimal learning rate is not a constant. It changes during training. Early on, you want large steps to cover ground quickly. Late in training, you want tiny steps to settle into a sharp minimum. The difference between a 90% accurate model and a 95% accurate model is often just the schedule.

Every major model published in the last three years uses a learning rate schedule. Llama 3 used peak $lr=3e-4$ with 2000 warmup steps and cosine decay to $3e-5$. GPT-3 used $lr=6e-4$ with warmup over 375 million tokens. These are not arbitrary choices. They are the result of extensive hyperparameter sweeps that cost millions of dollars.

You need to understand schedules because the defaults will not work for your problem. When you fine-tune a pretrained model, the right schedule is different than training from scratch. When you increase batch size, the warmup period needs to change. When training breaks at step 10,000, you need to know whether it's a schedule problem or something else.

The Concept

Constant Learning Rate

The simplest approach. Pick a number, use it for every step.

$$lr(t) = lr_0$$

Rarely optimal. It's either too high for the end of training (oscillation around the minimum) or too low for the beginning (wasted compute on tiny steps). Works fine for small models and debugging. A terrible choice for anything that trains for more than an hour.

Step Decay

The old-school approach from the ResNet era. Cut the learning rate by a factor (usually 10x) at fixed epochs.

$$lr(t) = lr_0 * \gamma^{\text{floor}(\text{epoch} / \text{step_size})}$$

Where $\gamma = 0.1$ and `step_size = 30` means: lr drops by 10x every 30 epochs. ResNet-50 used this – `lr=0.1`, drop by 10x at epochs 30, 60, and 90.

The problem: the optimal decay points depend on the dataset and architecture. Move to a different problem and you need to re-tune when to drop. The transitions are abrupt – loss can spike when the rate suddenly changes.

Cosine Annealing

Smooth decay from the maximum learning rate to a minimum, following a cosine curve:

$$lr(t) = lr_min + 0.5 * (lr_max - lr_min) * (1 + \cos(\pi * t / T))$$

Where t is the current step and T is the total number of steps.

At $t=0$, the cosine term is 1, so $lr = lr_max$. At $t=T$, the cosine term is -1, so $lr = lr_min$. The decay is gentle at first, accelerates in the middle, and becomes gentle again near the end.

This is the default for most modern training runs. No hyperparameters to tune beyond `lr_max` and `lr_min`. The cosine shape matches the empirical observation that most learning happens in the middle of training – you want reasonable step sizes during that critical period.

Warmup: Why You Start Small

Adam and other adaptive optimizers maintain running estimates of gradient mean and variance. At step 0, these estimates are initialized to zero. The first few gradient updates are based on garbage statistics. If your learning rate is large during this period, the model takes huge, poorly-directed steps.

Warmup fixes this. Start with a tiny learning rate (often `lr_max / warmup_steps` or even zero) and linearly ramp up to `lr_max` over the first N steps. By the time you reach the full learning rate, Adam's statistics have stabilized.

```
lr(t) = lr_max * (t / warmup_steps)    for t < warmup_steps
```

Typical warmup: 1-5% of total training steps. Llama 3 trained for ~1.8 trillion tokens and warmed up for 2000 steps. GPT-3 warmed up over 375 million tokens.

Linear Warmup + Cosine Decay

The modern default. Ramp up linearly, then decay with cosine:

```
if t < warmup_steps:
    lr(t) = lr_max * (t / warmup_steps)
else:
    progress = (t - warmup_steps) / (total_steps - warmup_steps)
    lr(t) = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos(pi * progress))
```

This is what Llama, GPT, PaLM, and most modern transformers use. The warmup prevents early instability. The cosine decay settles the model into a good minimum.

1cycle Policy

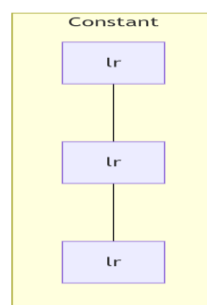
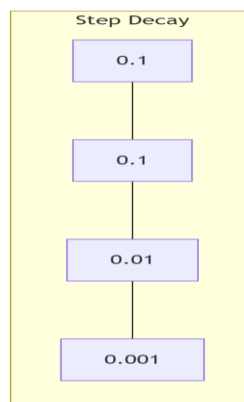
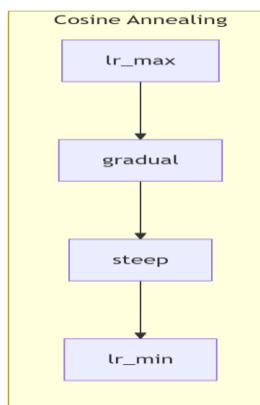
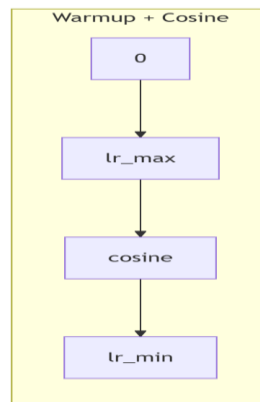
Leslie Smith's discovery (2018): ramp the learning rate up from a low value to a high value in the first half of training, then ramp it back down in the second half. Counterintuitive – why would you *increase* the learning rate midway through?

The theory: a high learning rate acts as regularization by adding noise to the optimization trajectory. The model explores more of the loss landscape during the ramp-up phase, finding better basins. The ramp-down phase then refines within the best basin found.

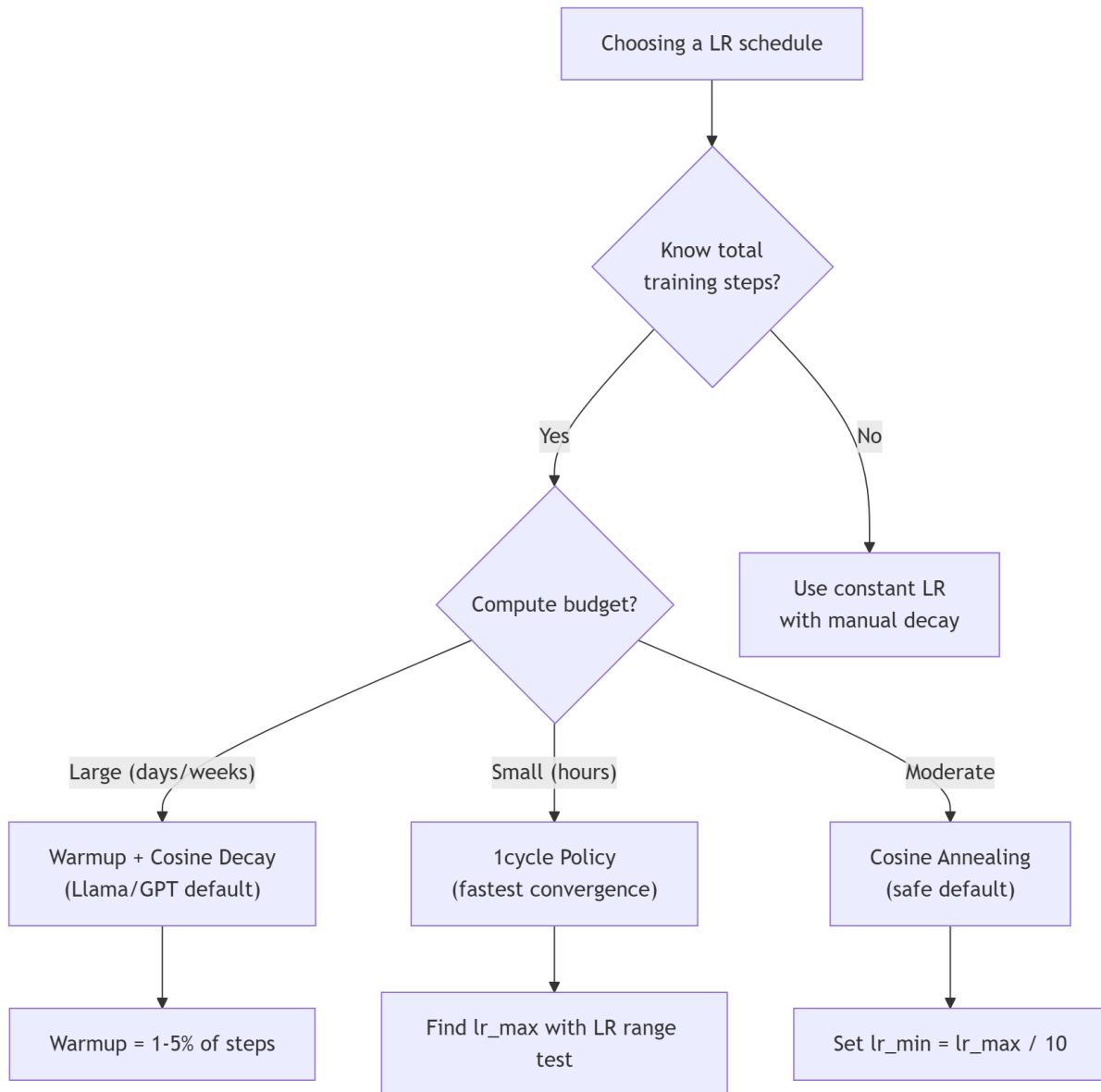
```
Phase 1 (0 to T/2):    lr ramps from lr_max/25 to lr_max
Phase 2 (T/2 to T):    lr ramps from lr_max to lr_max/10000
```

1cycle often trains faster than cosine annealing for a fixed compute budget. The tradeoff: you must know the total number of steps in advance.

Schedule Shapes



Decision Flowchart



Real Numbers from Published Models



Build It

Step 1: Schedule Functions

Each function takes the current step and returns the learning rate at that step.

```
import math

def constant_schedule(step, lr=0.01, **kwargs):
    return lr

def step_decay_schedule(step, lr=0.1, step_size=100, gamma=0.1, **kwargs):
    return lr * (gamma ** (step // step_size))

def cosine_schedule(step, lr=0.01, total_steps=1000, lr_min=1e-5, **kwargs):
    if step >= total_steps:
        return lr_min
    return lr_min + 0.5 * (lr - lr_min) * (1 + math.cos(math.pi * step / total_steps))
```

```
def warmup_cosine_schedule(step, lr=0.01, total_steps=1000, warmup_steps=100, lr_min=1e-5, **kwargs):
    if total_steps <= warmup_steps:
        return lr * (step / max(warmup_steps, 1))
    if step < warmup_steps:
        return lr * step / warmup_steps
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return lr_min + 0.5 * (lr - lr_min) * (1 + math.cos(math.pi * progress))

def one_cycle_schedule(step, lr=0.01, total_steps=1000, **kwargs):
    mid = max(total_steps // 2, 1)
    if step < mid:
        return (lr / 25) + (lr - lr / 25) * step / mid
    else:
        progress = (step - mid) / max(total_steps - mid, 1)
        return lr * (1 - progress) + (lr / 10000) * progress
```

Step 2: Visualize All Schedules

Print a text-based plot showing how each schedule evolves over training.

```
def visualize_schedule(name, schedule_fn, total_steps=500, **kwargs):
    steps = list(range(0, total_steps, total_steps // 20))
    if total_steps - 1 not in steps:
        steps.append(total_steps - 1)

    lrs = [schedule_fn(s, total_steps=total_steps, **kwargs) for s in steps]
    max_lr = max(lrs) if max(lrs) > 0 else 1.0

    print(f"\n{name}:")
    for s, lr_val in zip(steps, lrs):
        bar_len = int(lr_val / max_lr * 40)
        bar = "#" * bar_len
        print(f" Step {s:4d}: lr={lr_val:.6f} {bar}")
```

Step 3: Training Network

A simple two-layer network on the circle dataset, same as previous lessons, but now we vary the schedule.

```
import random

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))
```

```

def relu(x):
    return max(0.0, x)

def relu_deriv(x):
    return 1.0 if x > 0 else 0.0

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

def train_with_schedule(schedule_fn, schedule_name, data, epochs=300, base_lr=0.05, **k
wargs):
    random.seed(0)
    hidden_size = 8
    total_steps = epochs * len(data)

    std = math.sqrt(2.0 / 2)
    w1 = [[random.gauss(0, std) for _ in range(2)] for _ in range(hidden_size)]
    b1 = [0.0] * hidden_size
    w2 = [random.gauss(0, std) for _ in range(hidden_size)]
    b2 = 0.0

    step = 0
    epoch_losses = []

    for epoch in range(epochs):
        total_loss = 0
        correct = 0

        for x, target in data:
            lr = schedule_fn(step, lr=base_lr, total_steps=total_steps, **kwargs)

            z1 = []
            h = []
            for i in range(hidden_size):
                z = w1[i][0] * x[0] + w1[i][1] * x[1] + b1[i]
                z1.append(z)
                h.append(relu(z))

            z2 = sum(w2[i] * h[i] for i in range(hidden_size)) + b2
            out = sigmoid(z2)

            error = out - target
            d_out = error * out * (1 - out)

```

```

        for i in range(hidden_size):
            d_h = d_out * w2[i] * relu_deriv(z1[i])
            w2[i] -= lr * d_out * h[i]
            for j in range(2):
                w1[i][j] -= lr * d_h * x[j]
            b1[i] -= lr * d_h
        b2 -= lr * d_out

        total_loss += (out - target) ** 2
        if (out >= 0.5) == (target >= 0.5):
            correct += 1
        step += 1

    avg_loss = total_loss / len(data)
    accuracy = correct / len(data) * 100
    epoch_losses.append(avg_loss)

    return epoch_losses

```

Step 4: Compare All Schedules

Train the same network with each schedule and compare final loss and convergence behavior.

```

def compare_schedules(data):
    configs = [
        ("Constant", constant_schedule, {}),
        ("Step Decay", step_decay_schedule, {"step_size": 15000, "gamma": 0.1}),
        ("Cosine", cosine_schedule, {"lr_min": 1e-5}),
        ("Warmup+Cosine", warmup_cosine_schedule, {"warmup_steps": 3000, "lr_min": 1e-5
    })),
        ("1cycle", one_cycle_schedule, {}),
    ]

    print(f"\n{'Schedule':<20} {'Start Loss':>12} {'Mid Loss':>12} {'End Loss':>12} {'Best Loss':>12}")
    print("-" * 70)

    for name, schedule_fn, extra_kwargs in configs:
        losses = train_with_schedule(schedule_fn, name, data, epochs=300, base_lr=0.05,
        **extra_kwargs)
        mid_idx = len(losses) // 2
        best = min(losses)
        print(f"{name:<20} {losses[0]:>12.6f} {losses[mid_idx]:>12.6f} {losses[-1]:>12.6f} {best:>12.6f}")

```

Step 5: LR Too High vs Too Low

Demonstrate the three failure modes: too high (divergence), too low (crawling), and just right.

```

def lr_sensitivity(data):
    learning_rates = [1.0, 0.1, 0.01, 0.001, 0.0001]

    print("\nLR Sensitivity (constant schedule, 100 epochs):")
    print(f"  {'LR':>10} {'Start Loss':>12} {'End Loss':>12} {'Status':>15}")
    print("  " + "-" * 52)

    for lr in learning_rates:
        losses = train_with_schedule(constant_schedule, f"lr={lr}", data, epochs=100,
base_lr=lr)
        start = losses[0]
        end = losses[-1]

        if end > start or math.isnan(end) or end > 1.0:
            status = "DIVERGED"
        elif end > start * 0.9:
            status = "BARELY MOVED"
        elif end < 0.15:
            status = "CONVERGED"
        else:
            status = "LEARNING"

        end_str = f"{end:.6f}" if not math.isnan(end) else "NaN"
        print(f"  {lr:>10.4f} {start:>12.6f} {end_str:>12} {status:>15}")

```

Use It

PyTorch provides schedulers in `torch.optim.lr_scheduler`:

```

import torch
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, StepLR

model = nn.Sequential(nn.Linear(10, 64), nn.ReLU(), nn.Linear(64, 1))
optimizer = optim.Adam(model.parameters(), lr=3e-4)

scheduler = CosineAnnealingLR(optimizer, T_max=1000, eta_min=1e-5)

for step in range(1000):
    loss = train_step(model, optimizer)
    scheduler.step()

```

For warmup + cosine, use a `λ` scheduler or the `get_cosine_schedule_with_warmup` from HuggingFace:

```

from transformers import get_cosine_schedule_with_warmup

scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=2000,

```

```
num_training_steps=100000,
)
```

The HuggingFace function is what most Llama and GPT fine-tuning scripts use. When in doubt, use warmup + cosine with warmup = 3-5% of total steps. It works for almost everything.

Exercises

Implement exponential decay: $lr(t) = lr_0 * \gamma^t$ where $\gamma = 0.999$. Compare to cosine annealing on the circle dataset.

Implement the learning rate range test (Leslie Smith): train for a few hundred steps while exponentially increasing the LR from $1e-7$ to 1. Plot loss vs LR. The optimal max LR is just before the loss starts increasing.

Train with warmup + cosine but vary the warmup length: 0%, 1%, 5%, 10%, 20% of total steps. Find the sweet spot where training is most stable.

Implement cosine annealing with warm restarts (SGDR): reset the learning rate to lr_max every T steps and decay again. Compare to standard cosine on a longer training run.

Build a “schedule surgeon” that monitors training loss and automatically switches from warmup to cosine when the loss stabilizes, and reduces lr if the loss plateaus for too long.

Key Terms

Term	What people say	What it actually means
Learning rate	“How fast the model learns”	The scalar that multiplies the gradient to determine the parameter update size
Schedule	“Change the LR over time”	A function that maps training step to learning rate, designed to optimize convergence
Warmup	“Start with a small LR”	Linearly ramping the LR from near-zero to the target value over the first N steps to stabilize optimizer statistics
Cosine annealing	“Smooth LR decay”	Decreasing the LR following a cosine curve from lr_max to lr_min over training
Step decay	“Drop LR at milestones”	Multiplying the LR by a factor (usually 0.1) at fixed epoch intervals
1cycle policy	“Up then down”	Leslie Smith’s method of ramping LR up then down in a single cycle for faster convergence

Term	What people say	What it actually means
LR range test	“Find the best learning rate”	Training briefly while increasing LR to find the value where loss starts diverging
Cosine with warm restarts	“Reset and repeat”	Periodically resetting the LR to lr_max and decaying again (SGDR)
Eta min	“The floor for the LR”	The minimum learning rate that the schedule decays to
Peak learning rate	“The maximum LR”	The highest LR reached during training, typically after warmup

You are a learning rate schedule expert. Given a training setup, recommend the optimal schedule, peak learning rate, warmup duration, and decay target.

Input

I will describe: - Model architecture (type, parameter count, number of layers) - Dataset size (number of samples or tokens) - Batch size - Optimizer (SGD, Adam, AdamW, etc.) - Total training duration (epochs or steps) - Whether training from scratch or fine-tuning

Decision Rules

Schedule Selection

Scenario	Recommended Schedule	Reason
Transformer from scratch	Warmup + Cosine	Standard for GPT, Llama, BERT
CNN from scratch	Step Decay or Cosine	ResNet convention, both work well
Fine-tuning pretrained model	Warmup + Linear Decay	Gentler than cosine, less risk of forgetting
Quick experiment (<1 hour)	1cycle	Fastest convergence for fixed budget
Unknown duration	Cosine with Warm Restarts	Adapts to any length

Peak Learning Rate

Optimizer	From Scratch	Fine-tuning
SGD	0.01 - 0.1	0.001 - 0.01
Adam/AdamW	1e-4 - 1e-3	1e-5 - 5e-5

Scale with batch size: when doubling batch size, multiply LR by $\sqrt{2}$ (linear scaling rule).

Warmup Duration

From scratch: 1-5% of total steps

Fine-tuning: 5-10% of total steps (more conservative)

Large batch (>1024): increase warmup proportionally

Minimum LR

- Cosine: $lr_min = lr_max / 10$ to $lr_max / 100$
- Linear decay: $lr_min = 0$ is fine
- 1cycle: automatically handles min LR

Output Format

For each recommendation, provide:

191. **Schedule:** Name and formula
192. **Peak LR:** Specific value with rationale
193. **Warmup:** Number of steps and percentage
194. **Decay target:** Final LR value
195. **PyTorch code:** Ready to use

```
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR
from transformers import get_cosine_schedule_with_warmup

optimizer = torch.optim.AdamW(model.parameters(), lr=PEAK_LR, weight_decay=0.01)
scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=WARMUP,
    num_training_steps=TOTAL,
)
```

Troubleshooting

If training is unstable: - **Loss spikes early:** Increase warmup steps or reduce peak LR - **Loss plateaus mid-training:** Peak LR too low, or schedule decaying too fast - **Loss oscillates at end:** Min LR too high, reduce lr_min - **Fine-tuning catastrophic forgetting:** Reduce peak LR by 10x, increase warmup

Quiz

Pre-lesson check

196. Why is a constant learning rate usually suboptimal for training neural networks?

- It uses too much memory
- It's either too high for late training (causes oscillation) or too low for early training (wastes compute) **(correct)**
- Constant learning rates cause overfitting

- They only work with SGD

Explanation: The optimal step size changes during training. Early on, large steps cover ground quickly. Late in training, small steps are needed to settle into a minimum. A constant rate can't serve both needs.

197. What is the purpose of learning rate warmup?

1. To prevent overfitting
2. To let adaptive optimizer statistics (momentum, variance) stabilize before taking large steps **(correct)**
3. To increase the batch size gradually

4. To initialize weights properly

Explanation: Adam's moment estimates are initialized to zero. Early gradient updates are based on unreliable statistics. Warmup starts with a tiny LR and ramps up, giving Adam time to accumulate meaningful estimates.

Post-lesson check

- What learning rate schedule do Llama 3, GPT-3, and most modern LLMs use?
 - a) Constant learning rate
 - b) Step decay every 30 epochs
 - c) Linear warmup followed by cosine decay **(correct)**

d) Exponential decay

Explanation: Linear warmup + cosine decay is the standard for transformer training. Llama 3 used 2000 warmup steps with cosine decay from $3e-4$ to $3e-5$. This schedule requires no milestone tuning.

- What makes the 1cycle policy different from other schedules?

a) It uses a constant learning rate

b) It ramps the learning rate UP in the first half of training, then back down – the high LR phase acts as regularization **(correct)**

c) It only works with SGD

d) It requires no hyperparameter tuning

Explanation: Leslie Smith's 1cycle ramps LR from low to high (first half) then high to very low (second half). The high-LR phase helps the model explore more of the loss landscape before settling into the best basin.

- If training loss suddenly spikes and diverges, what is the most likely learning rate issue?

a) Learning rate is too low

b) Learning rate is too high, causing the optimizer to overshoot the minimum **(correct)**

c) The warmup period is too long

d) The schedule decays too slowly

Explanation: A learning rate that's too high causes the optimizer to take steps larger than the loss basin, overshooting the minimum and causing the loss to increase. This manifests as sudden divergence.

Build Your Own Mini Framework

You have built neurons, layers, networks, backprop, activations, loss functions, optimizers, regularization, initialization, and LR schedules. All as separate pieces. Now wire them together into a framework. Not PyTorch. Not TensorFlow. Yours.

Type: Build **Languages:** Python **Prerequisites:** All of Phase 03 (Lessons 01-09) **Time:** ~120 minutes

Learning Objectives

68. Build a complete deep learning framework (~500 lines) with Module, Linear, ReLU, Sigmoid, Dropout, BatchNorm, Sequential, loss functions, optimizers, and DataLoader
69. Explain the Module abstraction (forward, backward, parameters) and why train/eval mode toggling is necessary
70. Wire all components into a working training loop that trains a 4-layer network on circle classification
71. Map each component of your framework to its PyTorch equivalent (nn.Module, nn.Sequential, optim.Adam, DataLoader)

The Problem

You have ten lessons of building blocks scattered across separate files. A `Value` class here, a training loop there, weight initialization in another file, learning rate schedules in yet another. To train a network, you copy-paste from five different lessons and wire them together by hand.

That is what frameworks solve. PyTorch gives you `nn.Module`, `nn.Sequential`, `optim.Adam`, `DataLoader`, and a training loop pattern that ties them together. TensorFlow gives you `keras.Layer`, `keras.Sequential`, `keras.optimizers.Adam`. These are not magic. They are organizational patterns that make it possible to define, train, and evaluate networks without reinventing the plumbing every time.

You are going to build the same thing in ~500 lines of Python. No numpy. No external dependencies. A framework that can define any feedforward network, train it with SGD or Adam, batch the data, apply dropout and batch normalization, use any activation, and schedule the learning rate.

When you finish, you will understand exactly what happens when you write `model = nn.Sequential(...)` in PyTorch. You will understand why `model.train()` and `model.eval()` exist. You will understand why `optimizer.zero_grad()` is a separate call. You will understand all of it, because you built all of it.

The Concept

The Module Abstraction

Every layer in PyTorch inherits from `nn.Module`. A Module has three responsibilities:

forward() – compute the output given inputs

parameters() – return all trainable weights

backward() – compute gradients (handled by autograd in PyTorch, explicit in ours)

A Linear layer is a Module. A ReLU activation is a Module. A dropout layer is a Module. A batch normalization layer is a Module. They all have the same interface.

Sequential Container

`nn.Sequential` chains Modules. Forward pass: feed data through Module 1, then Module 2, then Module 3. Backward pass: reverse the chain. The container itself is a Module – it has `forward()`, `parameters()`, and `backward()`. This is the composite pattern: a sequence of Modules is itself a Module.

Training vs Evaluation Mode

Dropout randomly zeroes neurons during training but passes everything through during evaluation. Batch normalization uses batch statistics during training but running averages during evaluation. The `train()` and `eval()` methods toggle this behavior. Every Module has a `training` flag.

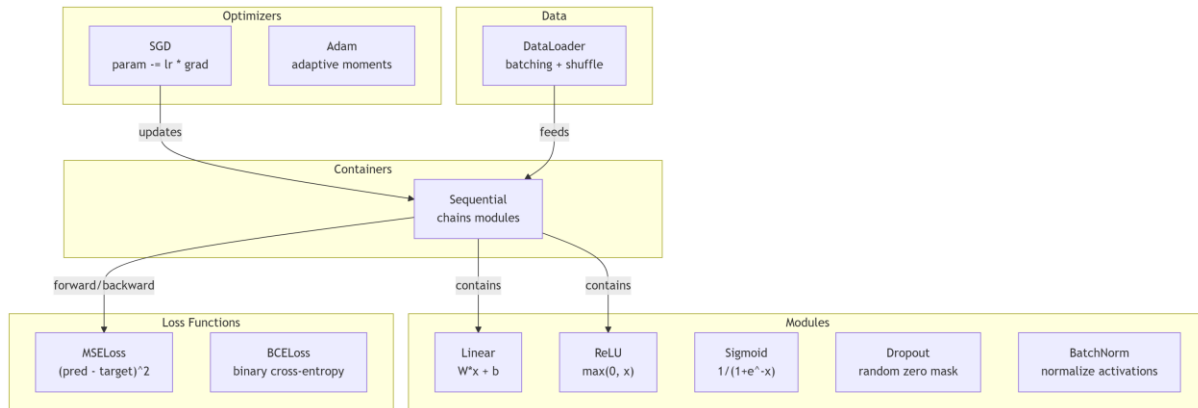
Optimizer

The optimizer updates parameters using their gradients. SGD: $\text{param} -= \text{lr} * \text{grad}$. Adam: maintains momentum and variance estimates, then updates. The optimizer does not know about the network architecture – it only sees a flat list of parameters and their gradients.

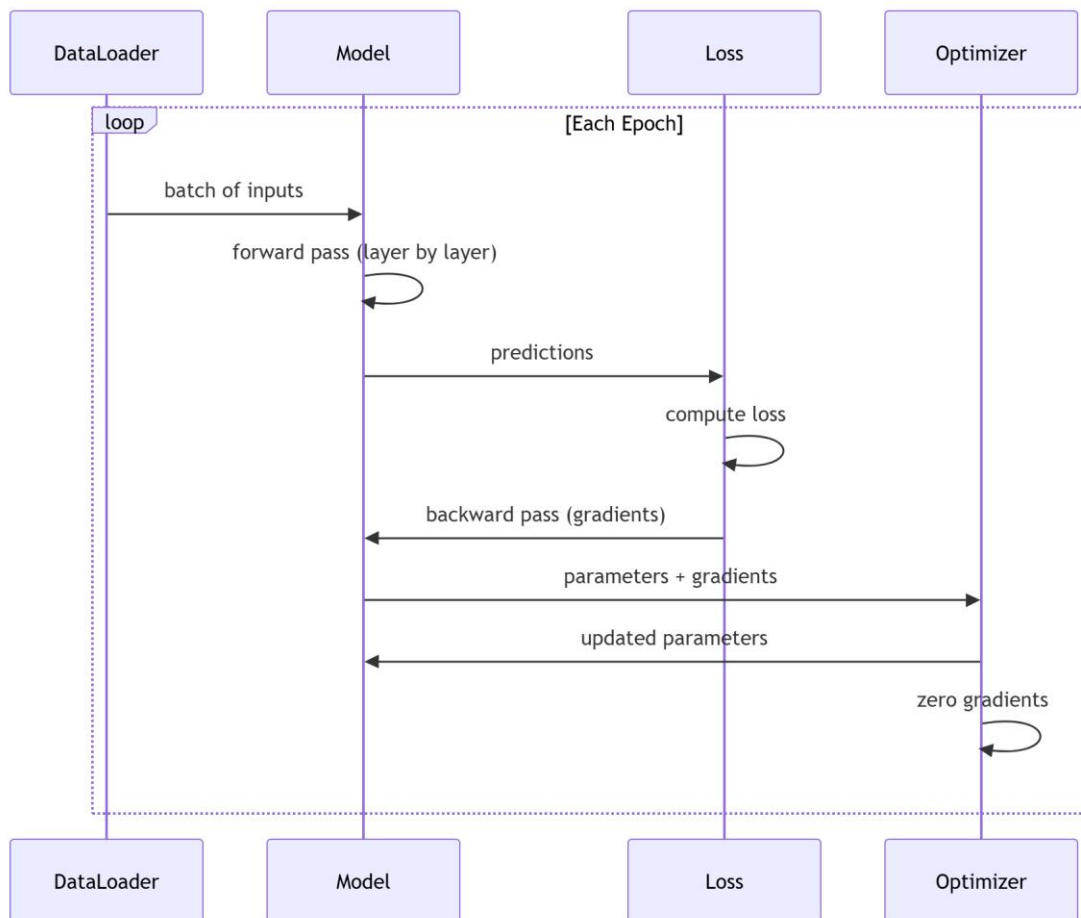
DataLoader

Batching matters for two reasons. First, you cannot fit the entire dataset in memory for large problems. Second, mini-batch gradient descent provides noise that helps escape local minima. The `DataLoader` splits data into batches and optionally shuffles between epochs.

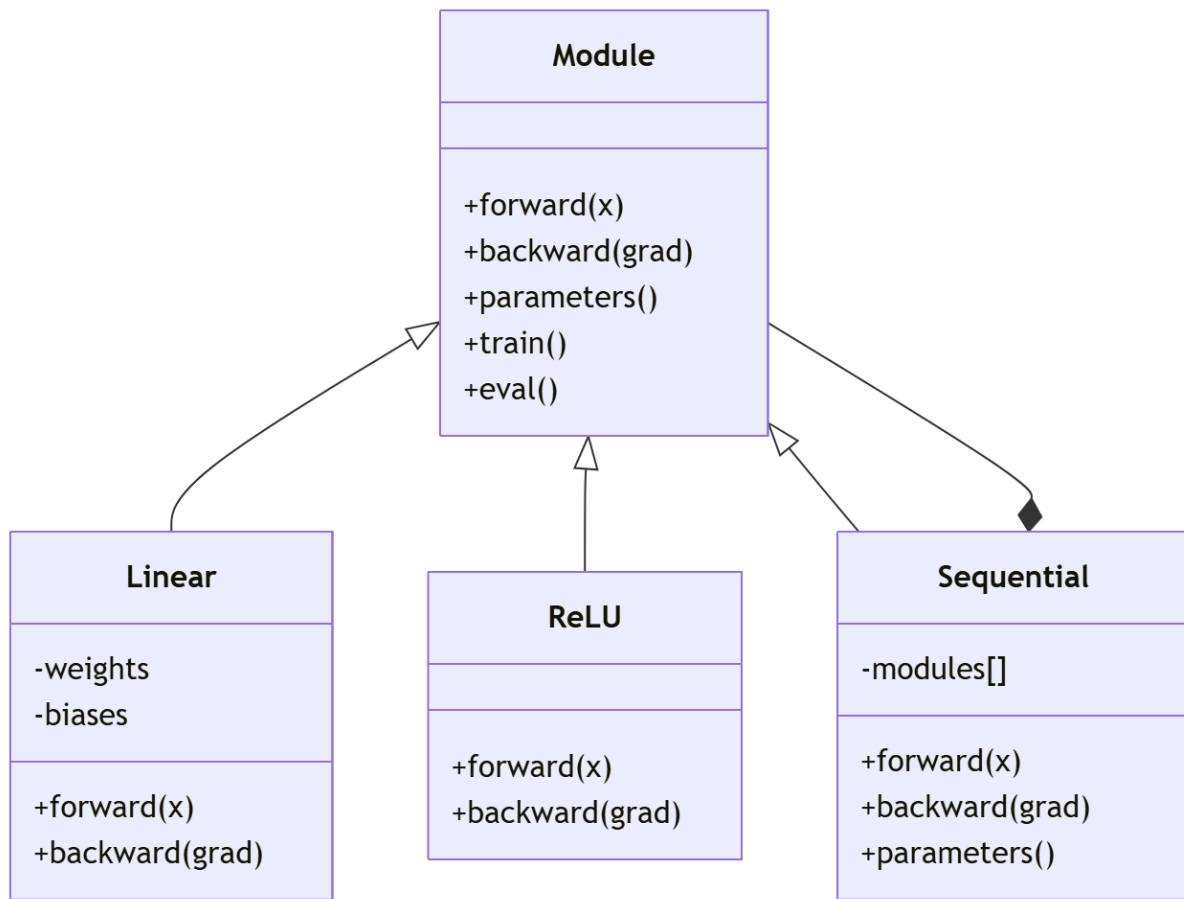
Framework Architecture



Training Loop



Module Hierarchy



Build It

Step 1: Module Base Class

The abstract interface that every layer implements.

```
class Module:
    def __init__(self):
        self.training = True

    def forward(self, x):
        raise NotImplementedError

    def backward(self, grad):
        raise NotImplementedError

    def parameters(self):
        return []

    def train(self):
```

```

        self.training = True

    def eval(self):
        self.training = False

```

Step 2: Linear Layer

The fundamental building block. Stores weights and biases, computes $Wx + b$ forward, and weight/input gradients backward.

```

import math
import random

class Linear(Module):
    def __init__(self, fan_in, fan_out):
        super().__init__()
        std = math.sqrt(2.0 / fan_in)
        self.weights = [[random.gauss(0, std) for _ in range(fan_in)] for _ in range(fan_out)]
        self.biases = [0.0] * fan_out
        self.weight_grads = [[0.0] * fan_in for _ in range(fan_out)]
        self.bias_grads = [0.0] * fan_out
        self.fan_in = fan_in
        self.fan_out = fan_out
        self.input = None

    def forward(self, x):
        self.input = x
        output = []
        for i in range(self.fan_out):
            val = self.biases[i]
            for j in range(self.fan_in):
                val += self.weights[i][j] * x[j]
            output.append(val)
        return output

    def backward(self, grad):
        input_grad = [0.0] * self.fan_in
        for i in range(self.fan_out):
            self.bias_grads[i] += grad[i]
            for j in range(self.fan_in):
                self.weight_grads[i][j] += grad[i] * self.input[j]
                input_grad[j] += grad[i] * self.weights[i][j]
        return input_grad

    def parameters(self):
        params = []
        for i in range(self.fan_out):
            for j in range(self.fan_in):
                params.append((self.weights, i, j, self.weight_grads))

```

```
        params.append((self.biases, i, None, self.bias_grads))
    return params
```

Step 3: Activation Modules

ReLU, Sigmoid, and Tanh as Modules. Each caches what it needs for the backward pass.

```
class ReLU(Module):
    def __init__(self):
        super().__init__()
        self.mask = None

    def forward(self, x):
        self.mask = [1.0 if v > 0 else 0.0 for v in x]
        return [max(0.0, v) for v in x]

    def backward(self, grad):
        return [g * m for g, m in zip(grad, self.mask)]

class Sigmoid(Module):
    def __init__(self):
        super().__init__()
        self.output = None

    def forward(self, x):
        self.output = []
        for v in x:
            v = max(-500, min(500, v))
            self.output.append(1.0 / (1.0 + math.exp(-v)))
        return self.output

    def backward(self, grad):
        return [g * o * (1 - o) for g, o in zip(grad, self.output)]

class Tanh(Module):
    def __init__(self):
        super().__init__()
        self.output = None

    def forward(self, x):
        self.output = [math.tanh(v) for v in x]
        return self.output

    def backward(self, grad):
        return [g * (1 - o * o) for g, o in zip(grad, self.output)]
```


Step 4: Dropout Module

Randomly zeroes elements during training. Scales remaining elements by $1/(1-p)$ so expected values stay the same. Does nothing during eval.

```
class Dropout(Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p
        self.mask = None

    def forward(self, x):
        if not self.training:
            return x
        self.mask = [0.0 if random.random() < self.p else 1.0 / (1 - self.p) for _ in x]

        return [v * m for v, m in zip(x, self.mask)]

    def backward(self, grad):
        if self.mask is None:
            return grad
        return [g * m for g, m in zip(grad, self.mask)]
```

Step 5: BatchNorm Module

Normalizes activations to zero mean and unit variance per feature across the batch. Maintains running statistics for eval mode.

```
class BatchNorm(Module):
    def __init__(self, size, momentum=0.1, eps=1e-5):
        super().__init__()
        self.size = size
        self.gamma = [1.0] * size
        self.beta = [0.0] * size
        self.gamma_grads = [0.0] * size
        self.beta_grads = [0.0] * size
        self.running_mean = [0.0] * size
        self.running_var = [1.0] * size
        self.momentum = momentum
        self.eps = eps
        self.x_norm = None
        self.std_inv = None
        self.batch_input = None

    def forward_batch(self, batch):
        batch_size = len(batch)
        output_batch = []

        if self.training:
            mean = [0.0] * self.size
            for sample in batch:
```

```

        for j in range(self.size):
            mean[j] += sample[j]
        mean = [m / batch_size for m in mean]

        var = [0.0] * self.size
        for sample in batch:
            for j in range(self.size):
                var[j] += (sample[j] - mean[j]) ** 2
        var = [v / batch_size for v in var]

        self.std_inv = [1.0 / math.sqrt(v + self.eps) for v in var]

        self.x_norm = []
        self.batch_input = batch
        for sample in batch:
            normed = [(sample[j] - mean[j]) * self.std_inv[j] for j in range(self.s
size)]

            self.x_norm.append(normed)
            output = [self.gamma[j] * normed[j] + self.beta[j] for j in range(self.
size)]
            output_batch.append(output)

        for j in range(self.size):
            self.running_mean[j] = (1 - self.momentum) * self.running_mean[j] + sel
f.momentum * mean[j]
            self.running_var[j] = (1 - self.momentum) * self.running_var[j] + self.
momentum * var[j]
        else:
            std_inv = [1.0 / math.sqrt(v + self.eps) for v in self.running_var]
            for sample in batch:
                normed = [(sample[j] - self.running_mean[j]) * std_inv[j] for j in rang
e(self.size)]
                output = [self.gamma[j] * normed[j] + self.beta[j] for j in range(self.
size)]
                output_batch.append(output)

        return output_batch

def forward(self, x):
    result = self.forward_batch([x])
    return result[0]

def backward(self, grad):
    if self.x_norm is None:
        return grad
    for j in range(self.size):
        self.gamma_grads[j] += self.x_norm[0][j] * grad[j]
        self.beta_grads[j] += grad[j]
    return [grad[j] * self.gamma[j] * self.std_inv[j] for j in range(self.size)]

def parameters(self):
    params = []
    for j in range(self.size):

```

```

        params.append((self.gamma, j, None, self.gamma_grads))
        params.append((self.beta, j, None, self.beta_grads))
    return params

```

Step 6: Sequential Container

Chains modules. Forward goes left-to-right, backward goes right-to-left.

```

class Sequential(Module):
    def __init__(self, *modules):
        super().__init__()
        self.modules = list(modules)

    def forward(self, x):
        for module in self.modules:
            x = module.forward(x)
        return x

    def backward(self, grad):
        for module in reversed(self.modules):
            grad = module.backward(grad)
        return grad

    def parameters(self):
        params = []
        for module in self.modules:
            params.extend(module.parameters())
        return params

    def train(self):
        self.training = True
        for module in self.modules:
            module.train()

    def eval(self):
        self.training = False
        for module in self.modules:
            module.eval()

```

Step 7: Loss Functions

MSE and Binary Cross-Entropy. Each returns the loss value and provides a backward() that returns the gradient.

```

class MSELoss:
    def __call__(self, predicted, target):
        self.predicted = predicted
        self.target = target
        n = len(predicted)
        self.loss = sum((p - t) ** 2 for p, t in zip(predicted, target)) / n
        return self.loss

```

```

def backward(self):
    n = len(self.predicted)
    return [2 * (p - t) / n for p, t in zip(self.predicted, self.target)]

class BCELoss:
    def __call__(self, predicted, target):
        self.predicted = predicted
        self.target = target
        eps = 1e-7
        n = len(predicted)
        self.loss = 0
        for p, t in zip(predicted, target):
            p = max(eps, min(1 - eps, p))
            self.loss += -(t * math.log(p) + (1 - t) * math.log(1 - p))
        self.loss /= n
        return self.loss

    def backward(self):
        eps = 1e-7
        n = len(self.predicted)
        grads = []
        for p, t in zip(self.predicted, self.target):
            p = max(eps, min(1 - eps, p))
            grads.append((-t / p + (1 - t) / (1 - p)) / n)
        return grads

```

Step 8: SGD and Adam Optimizers

Both take a parameter list and update weights using gradients.

```

class SGD:
    def __init__(self, parameters, lr=0.01):
        self.params = parameters
        self.lr = lr

    def step(self):
        for container, i, j, grad_container in self.params:
            if j is not None:
                container[i][j] -= self.lr * grad_container[i][j]
            else:
                container[i] -= self.lr * grad_container[i]

    def zero_grad(self):
        for container, i, j, grad_container in self.params:
            if j is not None:
                grad_container[i][j] = 0.0
            else:
                grad_container[i] = 0.0

```

```

class Adam:
    def __init__(self, parameters, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
        self.params = parameters
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps
        self.t = 0
        self.m = [0.0] * len(parameters)
        self.v = [0.0] * len(parameters)

    def step(self):
        self.t += 1
        for idx, (container, i, j, grad_container) in enumerate(self.params):
            if j is not None:
                g = grad_container[i][j]
            else:
                g = grad_container[i]

            self.m[idx] = self.beta1 * self.m[idx] + (1 - self.beta1) * g
            self.v[idx] = self.beta2 * self.v[idx] + (1 - self.beta2) * g * g

            m_hat = self.m[idx] / (1 - self.beta1 ** self.t)
            v_hat = self.v[idx] / (1 - self.beta2 ** self.t)

            update = self.lr * m_hat / (math.sqrt(v_hat) + self.eps)

            if j is not None:
                container[i][j] -= update
            else:
                container[i] -= update

    def zero_grad(self):
        for container, i, j, grad_container in self.params:
            if j is not None:
                grad_container[i][j] = 0.0
            else:
                grad_container[i] = 0.0

```

Step 9: DataLoader

Splits data into batches, optionally shuffles each epoch.

```

class DataLoader:
    def __init__(self, data, batch_size=32, shuffle=True):
        self.data = data
        self.batch_size = batch_size
        self.shuffle = shuffle

    def __iter__(self):
        indices = list(range(len(self.data)))
        if self.shuffle:

```

```

        random.shuffle(indices)
    for start in range(0, len(indices), self.batch_size):
        batch_indices = indices[start:start + self.batch_size]
        batch = [self.data[i] for i in batch_indices]
        inputs = [item[0] for item in batch]
        targets = [item[1] for item in batch]
        yield inputs, targets

def __len__(self):
    return (len(self.data) + self.batch_size - 1) // self.batch_size

```

Step 10: Train a 4-Layer Network on Circle Classification

Wire everything together. Define a model, pick a loss, pick an optimizer, run the training loop.

```

def make_circle_data(n=500, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], [label])
    return data

def train():
    random.seed(42)

    model = Sequential(
        Linear(2, 16),
        ReLU(),
        Linear(16, 16),
        ReLU(),
        Linear(16, 8),
        ReLU(),
        Linear(8, 1),
        Sigmoid(),
    )

    criterion = BCELoss()
    optimizer = Adam(model.parameters(), lr=0.01)

    data = make_circle_data(500)
    split = int(len(data) * 0.8)
    train_data = data[:split]
    test_data = data[split:]

    loader = DataLoader(train_data, batch_size=16, shuffle=True)

    model.train()

```

```

for epoch in range(100):
    total_loss = 0
    total_correct = 0
    total_samples = 0

    for batch_inputs, batch_targets in loader:
        batch_loss = 0
        for x, t in zip(batch_inputs, batch_targets):
            pred = model.forward(x)
            loss = criterion(pred, t)
            batch_loss += loss

            optimizer.zero_grad()
            grad = criterion.backward()
            model.backward(grad)
            optimizer.step()

            predicted_class = 1.0 if pred[0] >= 0.5 else 0.0
            if predicted_class == t[0]:
                total_correct += 1
            total_samples += 1

        total_loss += batch_loss

    avg_loss = total_loss / total_samples
    accuracy = total_correct / total_samples * 100

    if epoch % 10 == 0 or epoch == 99:
        print(f"Epoch {epoch:3d} | Loss: {avg_loss:.6f} | Train Accuracy: {accuracy:.1f}%")

    model.eval()
    correct = 0
    for x, t in test_data:
        pred = model.forward(x)
        predicted_class = 1.0 if pred[0] >= 0.5 else 0.0
        if predicted_class == t[0]:
            correct += 1
    test_accuracy = correct / len(test_data) * 100
    print(f"\nTest Accuracy: {test_accuracy:.1f}% ({correct}/{len(test_data)})")

    return model, test_accuracy

```

Use It

Here is the PyTorch equivalent of what you just built:

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

```

```

model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid(),
)

criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    model.train()
    for inputs, targets in dataloader:
        optimizer.zero_grad()
        predictions = model(inputs)
        loss = criterion(predictions, targets)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        test_predictions = model(test_inputs)

```

The structure is identical. Sequential, Linear, ReLU, Sigmoid, BCELoss, Adam, zero_grad, backward, step, train, eval. Every concept maps one-to-one. The difference is that PyTorch handles autograd automatically (no need to implement backward() in each module), runs on GPU, and has been optimized for years. But the bones are the same.

Now when you see PyTorch code, you know exactly what is happening at every line. That understanding is the whole point.

Exercises

Add a SoftmaxCrossEntropyLoss class for multi-class classification. Softmax the predictions, compute cross-entropy loss, and handle the combined backward pass. Test it on a 3-class spiral dataset.

Implement learning rate scheduling in the optimizer: add a set_lr() method and wire in the cosine schedule from Lesson 09. Train the circle classifier with warmup + cosine and compare to constant LR.

Add a save() and load() method to Sequential that serializes all weights to a JSON file and loads them back. Verify that a loaded model produces the same predictions as the original.

Implement weight decay (L2 regularization) in the Adam optimizer. Add a weight_decay parameter that shrinks weights toward zero each step. Compare training with decay=0 vs decay=0.01.

Replace the per-sample training loop with proper mini-batch gradient accumulation: accumulate gradients across all samples in a batch, then divide by batch size and take one optimizer step. Measure whether this changes convergence speed.

Key Terms

Term	What people say	What it actually means
Module	“A layer”	The base abstraction in a framework – anything with forward(), backward(), and parameters()
Sequential	“Stack layers in order”	A container that chains modules, applying them in sequence for forward and reverse for backward
Forward pass	“Run the network”	Computing the output by passing input through each module in order
Backward pass	“Compute gradients”	Propagating the loss gradient through each module in reverse to compute parameter gradients
Parameters	“The trainable weights”	All values in the network that the optimizer can update – weights and biases
Optimizer	“The thing that updates weights”	An algorithm that uses gradients to update parameters, implementing SGD, Adam, or other rules
DataLoader	“The thing that feeds data”	An iterator that splits a dataset into batches, optionally shuffling between epochs
Training mode	“model.train()”	A flag that enables stochastic behavior like dropout and batch normalization with batch stats
Evaluation mode	“model.eval()”	A flag that disables dropout and uses running statistics for batch normalization
Zero grad	“Clear the gradients”	Resetting all parameter gradients to zero before computing the next batch’s gradients

You are a neural network framework architect. Given a task description, design a complete network architecture using the standard framework abstractions: Module, Sequential, Linear, activations, loss functions, optimizers, and DataLoaders.

Input

I will describe: - The task (classification, regression, generation, etc.) - Input shape and type - Output shape and type - Dataset size - Constraints (latency, memory, training time)

Design Protocol

1. Choose the Architecture

Task	Architecture	Typical Depth
Binary classification	MLP with sigmoid output	2-4 layers
Multi-class classification	MLP with softmax output	2-4 layers
Regression	MLP with linear output	2-4 layers
Image classification	CNN + MLP head	5-50+ layers
Sequence modeling	Transformer	6-96 layers
Tabular data	MLP with batch norm	3-5 layers

2. Size Each Layer

Rules of thumb: - First hidden layer: 2-4x the input dimension - Subsequent layers: same width or gradually narrowing - Output layer: matches the number of classes or target dimensions - Wider networks generalize better with enough data. Deeper networks learn more abstract features.

3. Select Components

For each layer, specify: - **Linear(fan_in, fan_out)**: the affine transformation - **Activation**: ReLU for most cases, GELU for transformers - **Normalization**: BatchNorm after linear (before activation) for MLPs - **Regularization**: Dropout(0.1-0.5) after activation

4. Pick Loss and Optimizer

Task	Loss Function	Optimizer
Binary classification	BCELoss or BCEWithLogitsLoss	Adam (lr=1e-3)
Multi-class	CrossEntropyLoss	Adam (lr=1e-3)
Regression	MSELoss or L1Loss	Adam (lr=1e-3)
Fine-tuning	Same as task	AdamW (lr=1e-5)

5. Configure Training

- **Batch size**: 32-256 for MLPs, 8-64 for large models
- **Epochs**: start with 100, add early stopping

- **LR schedule:** warmup + cosine for >50 epochs, constant for quick experiments
- **Weight init:** Kaiming for ReLU, Xavier for sigmoid/tanh

Output Format

Provide:

- **Architecture diagram** in PyTorch Sequential notation
- **Parameter count** estimate
- **Training configuration** (optimizer, LR, schedule, batch size)
- **Expected training time** estimate
- **Potential issues** and how to avoid them

Example output:

```
model = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 64),
    nn.BatchNorm1d(64),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(64, num_classes),
)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=100)
loader = DataLoader(dataset, batch_size=64, shuffle=True)
```

Always justify each design choice. State what you would change if the model underperforms.

Quiz

Pre-lesson check

198. What three responsibilities does the Module abstraction have in a deep learning framework?

1. Load data, preprocess data, augment data

2. `forward()` for computation, `backward()` for gradients, `parameters()` for trainable weights **(correct)**
3. Compile, optimize, deploy
4. Tokenize, embed, decode
Explanation: Every Module implements `forward()` to compute output, `backward()` to propagate gradients, and `parameters()` to expose trainable weights. This uniform interface lets any module be composed with any other.

199. Why does Sequential process modules in reverse order during the backward pass?

- It's arbitrary – either direction works
- Gradients flow from the loss backward through the network, so the last layer computes gradients first **(correct)**
- Reverse order uses less memory
- It prevents gradient explosion
Explanation: The backward pass starts at the loss and propagates gradients toward the input. The last layer receives the loss gradient first, computes its local gradients, and passes them to the previous layer.

Post-lesson check

- Why is `optimizer.zero_grad()` a separate call instead of being done automatically?
 - It's a PyTorch design mistake
 - It allows gradient accumulation across multiple batches before taking a single optimizer step **(correct)**
 - It saves memory
 - It makes debugging easier
Explanation: Separating `zero_grad` from `step` enables gradient accumulation: you can run `backward()` multiple times (across mini-batches) and sum the gradients before calling `step()`. This simulates larger batch sizes.

- What is the correct order of operations in a training loop?
 - backward, forward, zero_grad, step
 - forward, loss, zero_grad, backward, step
 - zero_grad, forward, loss, backward, step **(correct)**
 - forward, zero_grad, backward, loss, step

Explanation: The standard pattern: zero_grad (clear old gradients), forward (compute predictions), loss (compute scalar loss), backward (compute gradients), step (update parameters). Getting this order wrong causes subtle bugs.
 - What is the role of the DataLoader in the framework?
 1. It trains the model
 2. It splits data into batches and optionally shuffles between epochs for mini-batch gradient descent **(correct)**
 3. It computes the loss function
 4. It initializes the weights

Explanation: The DataLoader handles two practical concerns: batching (you can't fit all data in memory) and shuffling (random order prevents the model from memorizing data sequence).
-

Introduction to PyTorch

You built the engine from pistons and crankshafts. Now learn the one everyone actually drives.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.10 (Build Your Own Mini Framework) **Time:** ~75 minutes

Learning Objectives

Build and train neural networks using PyTorch's `nn.Module`, `nn.Sequential`, and `autograd`

Use PyTorch tensors, GPU acceleration, and the standard training loop (`zero_grad`, `forward`, `loss`, `backward`, `step`)

Convert your from-scratch mini framework components to their PyTorch equivalents

Profile and compare training speed between your pure-Python framework and PyTorch on the same task

The Problem

You have a working mini framework. Linear layers, ReLU, dropout, batch norm, Adam, a `DataLoader`, a training loop. It trains a 4-layer network on a circle classification problem in pure Python.

It is also 500x slower than PyTorch on the same problem.

Your mini framework processes one sample at a time with nested Python loops. PyTorch dispatches the same operations to optimized C++/CUDA kernels that run on GPU. On a single NVIDIA A100, PyTorch trains a ResNet-50 (25.6M parameters) on ImageNet (1.28M images) in about 6 hours. Your framework would take roughly 3,000 hours on the same task – if it didn't run out of memory first.

Speed is not the only gap. Your framework has no GPU support. No automatic differentiation – you hand-wrote `backward()` for every module. No serialization. No distributed training. No mixed precision. No way to debug gradient flow without print statements.

PyTorch fills every one of these gaps. And it does so while keeping the exact same mental model you already built: `Module`, `forward()`, `parameters()`, `backward()`, `optimizer.step()`. The concepts transfer one-to-one. The syntax is nearly identical. The difference is that PyTorch wraps a decade of systems engineering behind the same interface you designed from scratch.

The Concept

Why PyTorch Won

In 2015, TensorFlow required you to define a static computation graph before running anything. You built the graph, compiled it, then fed data through it. Debugging meant staring at graph visualizations. Changing the architecture meant rebuilding the graph from scratch.

PyTorch launched in 2017 with a different philosophy: eager execution. You write Python. It runs immediately. `y = model(x)` actually computes `y` right now, not “add a node to a graph that will compute `y` later.” This meant standard Python debugging tools worked. `print()` worked. `pdb` worked. `if/else` in your forward pass worked.

By 2020, the market had spoken. PyTorch’s share in ML research papers went from 7% (2017) to over 75% (2022). Meta, Google DeepMind, OpenAI, Anthropic, and Hugging Face all use PyTorch as their primary framework. TensorFlow 2.x adopted eager execution in response – tacit admission that PyTorch’s design was correct.

The lesson: developer experience compounds. A framework that is 10% slower but 50% faster to debug wins every time.

Tensors

A tensor is a multi-dimensional array with three critical properties: shape, dtype, and device.

```
import torch

x = torch.zeros(3, 4)           # shape: (3, 4), dtype: float32, device: cpu
x = torch.randn(2, 3, 224, 224) # batch of 2 RGB images, 224x224
x = torch.tensor([1, 2, 3])     # from a Python list
```

Shape is the dimensionality. A scalar is shape `()`, a vector is `(n,)`, a matrix is `(m, n)`, a batch of images is `(batch, channels, height, width)`.

Dtype controls precision and memory.

dtype	Bits	Range	Use case
float32	32	~7 decimal digits	Default training
float16	16	~3.3 decimal digits	Mixed precision
bfloat16	16	Same range as float32, less precision	LLM training
int8	8	-128 to 127	Quantized inference

Device determines where computation happens.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(3, 4, device=device)
x = x.to("cuda")
x = x.cpu()
```

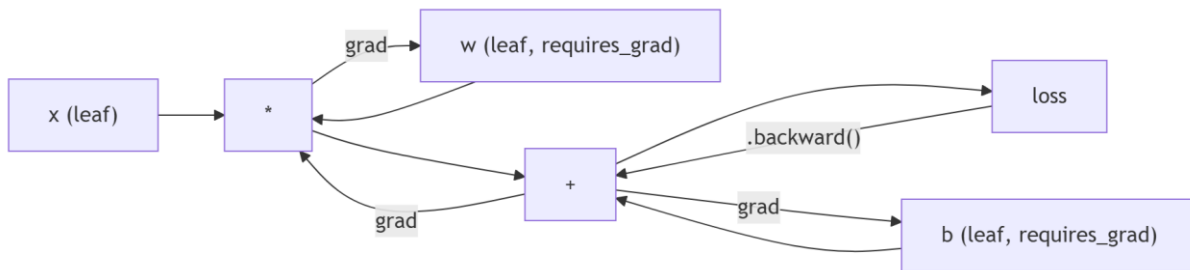
Every operation requires all tensors on the same device. This is the #1 PyTorch error beginners hit: `RuntimeError: Expected all tensors to be on the same device`. Fix it by moving everything to the same device before computation.

Reshaping is constant-time – it changes the metadata, not the data.

```
x = torch.randn(2, 3, 4)
x.view(2, 12)      # reshape to (2, 12) -- must be contiguous
x.reshape(6, 4)    # reshape to (6, 4) -- works always
x.permute(2, 0, 1) # reorder dimensions
x.unsqueeze(0)     # add dimension: (1, 2, 3, 4)
x.squeeze()        # remove size-1 dimensions
```

Autograd

Your mini framework required you to implement `backward()` for every module. PyTorch does not. It records every operation on tensors into a directed acyclic graph (the computational graph) and then traverses that graph in reverse to compute gradients automatically.



The key difference from your framework: PyTorch uses tape-based autodiff. Every operation appends to a “tape” during the forward pass. Calling `.backward()` replays the tape in reverse.

```
x = torch.randn(3, requires_grad=True)
y = x ** 2 + 3 * x
z = y.sum()
z.backward()
print(x.grad) # dz/dx = 2x + 3
```

Three rules of autograd:

- Only leaf tensors with `requires_grad=True` accumulate gradients

- Gradients accumulate by default – call `optimizer.zero_grad()` before each backward pass

- `torch.no_grad()` disables gradient tracking (use during evaluation)

nn.Module

`nn.Module` is the base class for every neural network component in PyTorch. You already built this abstraction in Lesson 10. PyTorch's version adds automatic parameter registration, recursive module discovery, device management, and state dict serialization.

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

When you assign an `nn.Module` or `nn.Parameter` as an attribute in `__init__`, PyTorch automatically registers it. `model.parameters()` recursively collects every registered parameter. This is why you never have to manually gather weights like you did in the mini framework.

Key building blocks:

Module	What it does	Parameters
<code>nn.Linear(in, out)</code>	$Wx + b$	$in * out + out$
<code>nn.Conv2d(in_ch, out_ch, k)</code>	2D convolution	$in_ch * out_ch * k * k + out_ch$
<code>nn.BatchNorm1d(features)</code>	Normalize activations	$2 * features$
<code>nn.Dropout(p)</code>	Random zeroing	0
<code>nn.ReLU()</code>	$\max(0, x)$	0
<code>nn.GELU()</code>	Gaussian error linear	0
<code>nn.Embedding(vocab, dim)</code>	Lookup table	$vocab * dim$
<code>nn.LayerNorm(dim)</code>	Per-sample normalization	$2 * dim$

Loss Functions and Optimizers

PyTorch ships production-ready versions of everything you built.

Loss functions (from `torch.nn`):

Loss	Task	Input
<code>nn.MSELoss()</code>	Regression	Any shape
<code>nn.CrossEntropyLoss()</code>	Multi-class classification	Logits (not softmax)
<code>nn.BCEWithLogitsLoss()</code>	Binary classification	Logits (not sigmoid)
<code>nn.L1Loss()</code>	Regression (robust)	Any shape
<code>nn.CTCLoss()</code>	Sequence alignment	Log probabilities

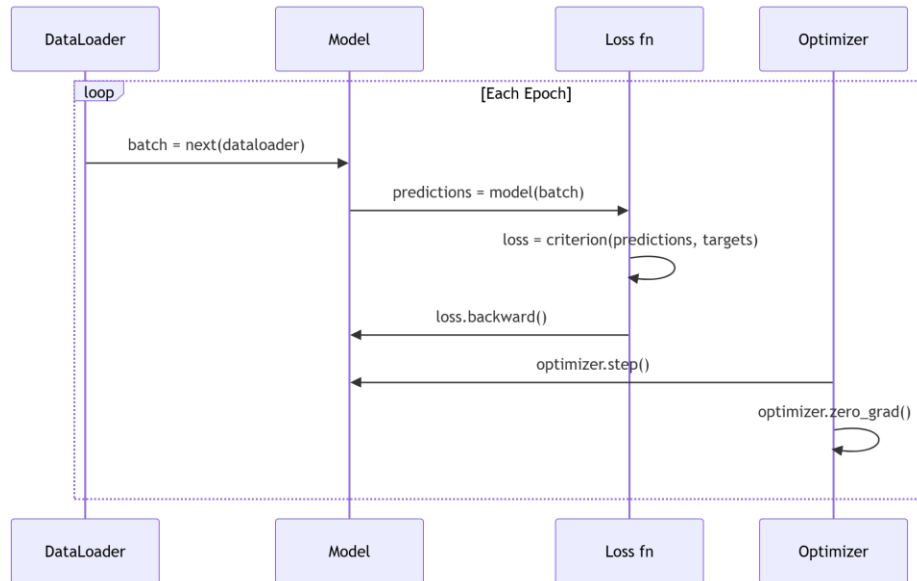
Note: `CrossEntropyLoss` combines `LogSoftmax` + `NLLLoss` internally. Pass raw logits, not softmax outputs. This is a common mistake that produces wrong gradients silently.

Optimizers (from `torch.optim`):

Optimizer	When to use	Typical LR
<code>SGD(params, lr, momentum)</code>	CNNs, well-tuned pipelines	0.01–0.1
<code>Adam(params, lr)</code>	Default starting point	1e-3
<code>AdamW(params, lr, weight_decay)</code>	Transformers, fine-tuning	1e-4–1e-3
<code>LBFGS(params)</code>	Small-scale, second-order	1.0

The Training Loop

Every PyTorch training loop follows the same 5-step pattern. You already know this from Lesson 10.



The canonical pattern:

```
for epoch in range(num_epochs):
    model.train()
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
```

Five lines inside the batch loop. Five lines that trained GPT-4, Stable Diffusion, and LLaMA. The architecture changes. The data changes. These five lines do not.

Dataset and DataLoader

PyTorch's Dataset is an abstract class with two methods: `__len__` and `__getitem__`. DataLoader wraps it with batching, shuffling, and multi-process data loading.

```
from torch.utils.data import Dataset, DataLoader

class MNISTDataset(Dataset):
    def __init__(self, images, labels):
        self.images = images
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.images[idx], self.labels[idx]
```

```
loader = DataLoader(dataset, batch_size=64, shuffle=True, num_workers=4)
```

`num_workers=4` spawns 4 processes to load data in parallel while the GPU trains on the current batch. On disk-bound workloads (large images, audio), this alone can double training speed.

GPU Training

Moving a model to GPU:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

This recursively moves every parameter and buffer to the GPU. Then move each batch during training:

```
inputs, targets = inputs.to(device), targets.to(device)
```

Mixed precision halves memory usage and doubles throughput on modern GPUs (A100, H100, RTX 4090) by running forward/backward in float16 while keeping the master weights in float32:

```
from torch.amp import autocast, GradScaler

scaler = GradScaler()
for inputs, targets in loader:
    with autocast(device_type="cuda"):
        outputs = model(inputs)
        loss = criterion(outputs, targets)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad()
```

Comparison: Mini Framework vs PyTorch vs JAX

Feature	Mini Framework (L10)	PyTorch	JAX
Autodiff	Manual backward()	Tape-based autograd	Functional transforms
Execution	Eager (Python loops)	Eager (C++ kernels)	Traced + JIT compiled
GPU support	No	Yes (CUDA, ROCm, MPS)	Yes (CUDA, TPU)
Speed (MNIST MLP)	~300s/epoch	~0.5s/epoch	~0.3s/epoch
Module system	Custom Module class	nn.Module	Stateless functions (Flax/Equinox)

Feature	Mini Framework (L10)	PyTorch	JAX
Debugging	print()	print(), pdb, breakpoint()	Harder (JIT tracing breaks print)
Ecosystem	None	Hugging Face, Lightning, timm	Flax, Optax, Orbx
Learning curve	You built it	Moderate	Steep (functional paradigm)
Production use	Toy problems	Meta, OpenAI, Anthropic, HF	Google DeepMind, Midjourney

Build It

A 3-layer MLP trained on MNIST using only PyTorch primitives. No high-level wrappers. No `torchvision.datasets`. We download and parse the raw data ourselves.

Step 1: Load MNIST From Raw Files

MNIST ships as 4 gzipped files: training images (60,000 x 28 x 28), training labels, test images (10,000 x 28 x 28), test labels. We download them and parse the binary format.

```
import torch
import torch.nn as nn
import struct
import gzip
import urllib.request
import os

def download_mnist(path="./mnist_data"):
    base_url = "https://storage.googleapis.com/cvdf-datasets/mnist/"
    files = [
        "train-images-idx3-ubyte.gz",
        "train-labels-idx1-ubyte.gz",
        "t10k-images-idx3-ubyte.gz",
        "t10k-labels-idx1-ubyte.gz",
    ]
    os.makedirs(path, exist_ok=True)
    for f in files:
        filepath = os.path.join(path, f)
        if not os.path.exists(filepath):
            urllib.request.urlretrieve(base_url + f, filepath)

def load_images(filepath):
    with gzip.open(filepath, "rb") as f:
        magic, num, rows, cols = struct.unpack(">IIII", f.read(16))
        data = f.read()
        images = torch.frombuffer(bytearray(data), dtype=torch.uint8)
        images = images.reshape(num, rows * cols).float() / 255.0
```

```

    return images

def load_labels(filepath):
    with gzip.open(filepath, "rb") as f:
        magic, num = struct.unpack(">II", f.read(8))
        data = f.read()
        labels = torch.frombuffer(bytearray(data), dtype=torch.uint8).long()
    return labels

```

Step 2: Define the Model

A 3-layer MLP: 784 -> 256 -> 128 -> 10. ReLU activations. Dropout for regularization. No batch norm to keep it simple.

```

class MNISTModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 10),
        )

    def forward(self, x):
        return self.net(x)

```

The output layer produces 10 raw logits (one per digit). No softmax – CrossEntropyLoss handles that internally.

Parameter count: $784 \times 256 + 256 + 256 \times 128 + 128 + 128 \times 10 + 10 = 235,146$. Tiny by modern standards. GPT-2 small has 124M. This trains in seconds.

Step 3: Training Loop

The canonical forward-loss-backward-step pattern.

```

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)

```

```

        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total

def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total

```

Note `torch.no_grad()` during evaluation. This disables autograd, reducing memory usage and speeding up inference. Without it, PyTorch builds a computational graph you never use.

Step 4: Wire Everything Together

```

def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    download_mnist()
    train_images = load_images("./mnist_data/train-images-idx3-ubyte.gz")
    train_labels = load_labels("./mnist_data/train-labels-idx1-ubyte.gz")
    test_images = load_images("./mnist_data/t10k-images-idx3-ubyte.gz")
    test_labels = load_labels("./mnist_data/t10k-labels-idx1-ubyte.gz")

    train_dataset = torch.utils.data.TensorDataset(train_images, train_labels)
    test_dataset = torch.utils.data.TensorDataset(test_images, test_labels)
    train_loader = torch.utils.data.DataLoader(
        train_dataset, batch_size=64, shuffle=True
    )
    test_loader = torch.utils.data.DataLoader(
        test_dataset, batch_size=256, shuffle=False
    )

    model = MNISTModel().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

```

num_params = sum(p.numel() for p in model.parameters())
print(f"Device: {device}")
print(f"Parameters: {num_params:,}")
print(f"Train samples: {len(train_dataset):,}")
print(f"Test samples: {len(test_dataset):,}")
print()

for epoch in range(10):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    test_loss, test_acc = evaluate(
        model, test_loader, criterion, device
    )
    print(
        f"Epoch {epoch+1:2d} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}"
    )

torch.save(model.state_dict(), "mnist_mlp.pt")
print(f"\nModel saved to mnist_mlp.pt")
print(f"Final test accuracy: {test_acc:.4f}")

```

Expected output after 10 epochs: ~97.8% test accuracy. Training time on CPU: ~30 seconds. On GPU: ~5 seconds. On your mini framework with the same architecture: ~45 minutes.

Use It

Quick Comparison: Mini Framework vs PyTorch

Mini Framework (Lesson 10)	PyTorch
model = Sequential(Linear(784, 256), ReLU(), ...)	model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), ...)
pred = model.forward(x)	pred = model(x)
optimizer.zero_grad()	optimizer.zero_grad()
grad = criterion.backward() then model.backward(grad)	loss.backward()
optimizer.step()	optimizer.step()
No GPU	model.to("cuda")

Mini Framework (Lesson 10)	PyTorch
Manual backward for every module	Autograd handles everything

The interface is nearly identical. The difference is everything under the hood.

Saving and Loading Models

```
torch.save(model.state_dict(), "model.pt")

model = MNISTModel()
model.load_state_dict(torch.load("model.pt", weights_only=True))
model.eval()
```

Always save `state_dict()` (the parameter dictionary), not the model object. Saving the model object uses pickle, which breaks when you refactor code. State dicts are portable.

Learning Rate Scheduling

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=10
)
for epoch in range(10):
    train_one_epoch(model, train_loader, criterion, optimizer, device)
    scheduler.step()
```

PyTorch ships 15+ schedulers: StepLR, ExponentialLR, CosineAnnealingLR, OneCycleLR, ReduceLROnPlateau. All plug into the same optimizer interface.

Exercises

Add batch normalization. Insert `nn.BatchNorm1d` after each linear layer (before the activation). Compare test accuracy and training speed vs the dropout-only version. Batch norm should reach 98%+ in fewer epochs.

Implement a learning rate finder. Train for one epoch with exponentially increasing learning rate (from $1e-7$ to 1.0). Plot loss vs LR. The optimal LR is just before the loss starts climbing. Use this to pick a better LR for the MNIST model.

Port to GPU with mixed precision. Add `torch.amp.autocast` and `GradScaler` to the training loop. Measure throughput (samples/second) with and without mixed precision on GPU. On an A100, expect ~2x speedup.

Build a custom Dataset. Download Fashion-MNIST (same format as MNIST but with clothing items). Implement a `FashionMNISTDataset(Dataset)` class with `__getitem__` and `__len__`. Train the same MLP and compare accuracy. Fashion-MNIST is harder – expect ~88% vs ~98%.

Replace Adam with SGD + momentum. Train with `SGD(params, lr=0.01, momentum=0.9)`. Compare convergence curves. Then add a `CosineAnnealingLR` scheduler and see if SGD catches up to Adam by epoch 10.

Key Terms

Term	What people say	What it actually means
Tensor	“A multi-dimensional array”	A typed, device-aware array with automatic differentiation support baked into every operation
Autograd	“Automatic backprop”	A tape-based system that records operations during forward pass, then replays them in reverse to compute exact gradients
<code>nn.Module</code>	“A layer”	The base class for any differentiable computation block – registers parameters, supports nesting, handles train/eval modes
<code>state_dict</code>	“The model weights”	An <code>OrderedDict</code> mapping parameter names to tensors – the portable, serializable representation of a trained model
<code>.backward()</code>	“Compute gradients”	Traverse the computational graph in reverse, computing and accumulating gradients for every leaf tensor with <code>requires_grad=True</code>
<code>.to(device)</code>	“Move to GPU”	Recursively transfer all parameters and buffers to the specified device (CPU, CUDA, MPS)
<code>DataLoader</code>	“The data pipeline”	An iterator that batches, shuffles, and optionally parallelizes data loading from a <code>Dataset</code>
Mixed precision	“Use float16”	Train with float16 forward/backward for speed while keeping float32 master weights for numerical stability
Eager execution	“Run it now”	Operations execute immediately when called, not deferred to a later compilation step – the core design choice that differentiates PyTorch from TF 1.x
<code>zero_grad</code>	“Reset gradients”	Set all parameter gradients to zero before the next backward pass, since PyTorch accumulates gradients by default

You are a PyTorch training debugger. Given a description of training behavior (loss values, accuracy, error messages, or unexpected outputs), diagnose the root cause and provide a fix.

Input

I will describe: - What I expected to happen - What actually happened (loss curve, accuracy, error message, or output) - Relevant code snippets - Hardware (CPU/GPU, memory)

Diagnosis Protocol

1. Classify the Symptom

Symptom	Category	Likely Causes
Loss is NaN	Numerical instability	LR too high, missing gradient clipping, $\log(0)$, division by zero
Loss stays flat	Not learning	LR too low, dead ReLU, wrong loss function, data not shuffled
Loss explodes	Divergence	LR too high, no gradient clipping, weight init wrong
Loss decreases then plateaus	Convergence issue	Need LR schedule, model too small, data bottleneck
Train acc high, test acc low	Overfitting	Need dropout, weight decay, more data, early stopping
Train acc low, test acc low	Underfitting	Model too small, LR wrong, bug in data pipeline
RuntimeError: device mismatch	Device management	Tensors on different devices (CPU vs CUDA)
RuntimeError: size mismatch	Shape error	Wrong dimensions in linear layer, missing reshape/flatten
CUDA out of memory	Memory	Batch size too large, gradient accumulation needed, mixed precision needed
Training is very slow	Performance	No GPU, num_workers=0, no pin_memory, no mixed precision

2. Check These First (90% of Issues)

Is the data correct? Print a batch. Check shapes, ranges, and labels. Visualize an image if applicable.

Is the loss function correct? CrossEntropyLoss expects raw logits. BCEWithLogitsLoss expects raw logits. If you apply softmax/sigmoid before these, the gradients are wrong.

Are you calling zero_grad()? Missing zero_grad means gradients accumulate across batches. Loss will look normal at first then diverge.

Are you calling `model.train()` and `model.eval()`? Dropout and BatchNorm behave differently in each mode. Forgetting `model.eval()` during validation inflates your reported metrics.

Are all tensors on the same device? Print `tensor.device` for inputs, labels, and model parameters.

3. Advanced Checks

- **Gradient flow:** `for name, p in model.named_parameters(): print(name, p.grad.abs().mean())` – if any gradient is 0 or NaN, that layer is dead
- **Weight magnitudes:** `for name, p in model.named_parameters(): print(name, p.abs().mean())` – if weights are huge (>100) or tiny ($<1e-6$), initialization or learning rate is wrong
- **Learning rate:** Try 10x smaller and 10x larger. If neither helps, the bug is elsewhere
- **Batch size 1 overfitting:** Train on a single batch. If the model cannot overfit one batch to 100% accuracy, there is a bug in the model or data pipeline

Output Format

Provide:

200. **Diagnosis:** One-sentence root cause
201. **Evidence:** What in the symptoms points to this cause
202. **Fix:** Exact code change with before/after
203. **Verification:** How to confirm the fix worked
204. **Prevention:** How to avoid this in the future

Always start with the simplest possible cause. Most PyTorch bugs are one of: wrong device, wrong loss function, missing `zero_grad`, or wrong tensor shape.

Canonical Training Loop

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Model().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

for epoch in range(num_epochs):
    model.train()
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
```

```

model.eval()
with torch.no_grad():
    for inputs, targets in val_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        outputs = model(inputs)

```

Mixed Precision Training

```

from torch.amp import autocast, GradScaler

scaler = GradScaler()
for inputs, targets in train_loader:
    inputs, targets = inputs.to(device), targets.to(device)
    optimizer.zero_grad()
    with autocast(device_type="cuda"):
        outputs = model(inputs)
        loss = criterion(outputs, targets)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

```

Use when: training on GPU with float16-capable hardware (V100, A100, H100, RTX 3090+). Expect ~1.5-2x speedup and ~50% memory reduction.

Gradient Accumulation

```

accumulation_steps = 4
optimizer.zero_grad()
for i, (inputs, targets) in enumerate(train_loader):
    inputs, targets = inputs.to(device), targets.to(device)
    outputs = model(inputs)
    loss = criterion(outputs, targets) / accumulation_steps
    loss.backward()
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

```

Use when: effective batch size needs to be larger than GPU memory allows. Dividing loss by accumulation_steps keeps gradient scale consistent.

Save and Load

```

torch.save({
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "loss": loss.item(),
}, "checkpoint.pt")

```

```
checkpoint = torch.load("checkpoint.pt", weights_only=True)
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

Always save optimizer state for resuming training. For inference-only, save just `model.state_dict()`.

Custom Dataset

```
class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, data_dir, transform=None):
        self.samples = self._load_samples(data_dir)
        self.transform = transform

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        x, y = self.samples[idx]
        if self.transform:
            x = self.transform(x)
        return x, y

    def _load_samples(self, data_dir):
        ...
```

DataLoader Configuration

```
train_loader = torch.utils.data.DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,
    pin_memory=True,
    drop_last=True,
    persistent_workers=True,
)
```

Parameter	What it does	When to use
<code>num_workers=4</code>	Parallel data loading	Always on multi-core machines
<code>pin_memory=True</code>	Page-locked CPU memory	When training on GPU
<code>drop_last=True</code>	Drop incomplete final batch	When using BatchNorm
<code>persistent_workers=True</code>	Keep workers alive across epochs	When <code>num_workers > 0</code>

Learning Rate Schedules

```
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=1e-3,
    total_steps=num_epochs * len(train_loader),
    pct_start=0.1,
)

for epoch in range(num_epochs):
    for inputs, targets in train_loader:
        ...
        optimizer.step()
        scheduler.step()
```

OneCycleLR: best default for most tasks. Warms up to max_lr, then cosine decays. Call `scheduler.step()` after every batch, not every epoch.

Weight Initialization

```
def init_weights(module):
    if isinstance(module, nn.Linear):
        nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
        if module.bias is not None:
            nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Conv2d):
        nn.init.kaiming_normal_(module.weight, mode="fan_out", nonlinearity="relu")

model.apply(init_weights)
```

Inference Mode

```
model.eval()

with torch.inference_mode():
    outputs = model(inputs)
```

`torch.inference_mode()` is faster than `torch.no_grad()` because it disables autograd entirely rather than just suppressing gradient computation.

Common Mistakes Checklist

- Applying softmax before CrossEntropyLoss (it includes log_softmax internally)
- Forgetting to call `model.eval()` during validation
- Forgetting to move tensors to the same device as the model
- Not calling `optimizer.zero_grad()` (gradients accumulate by default)
- Using `torch.no_grad()` during training (disables gradient computation)

- Setting num_workers too high (spawns too many processes, thrashes memory)
 - Not using pin_memory=True when training on GPU
 - Saving the entire model object instead of state_dict (breaks on refactor)
-

Quiz

Pre-lesson check

- What is autograd in PyTorch?
 - A data loading library
 - An automatic differentiation engine that records operations and computes gradients without manual backward implementations **(correct)**
 - A model compression tool
 - A hyperparameter tuning framework

Explanation: PyTorch's autograd builds a computational graph during forward computation and automatically computes gradients when you call .backward(). This eliminates the need to manually implement backward() in each module.
- What does torch.no_grad() do and when should you use it?
 - It prevents overfitting
 - It disables gradient tracking for inference, saving memory and computation when you don't need to train **(correct)**
 - It freezes all model weights
 - It enables GPU acceleration

Explanation: During inference, you don't need gradients. torch.no_grad() disables gradient tracking, which saves the memory that would be used to store the computational graph and speeds up computation.

Post-lesson check

- What does nn.Linear(784, 256) create in PyTorch?

- A ReLU activation layer
 - A fully connected layer with a (256, 784) weight matrix and a (256,) bias vector **(correct)**
 - A dropout layer with $p=784/256$
 - A batch normalization layer
Explanation: `nn.Linear(in_features, out_features)` creates a layer that computes $y = x @ W^T + b$, with W of shape (256, 784) and b of shape (256,). This is the PyTorch equivalent of the `Layer` class you built from scratch.
 - Which PyTorch method computes gradients for all parameters in the computational graph?
 1. `optimizer.step()`
 2. `model.forward()`
 3. `loss.backward()` **(correct)**
 4. `optimizer.zero_grad()`
Explanation: `loss.backward()` traverses the computational graph in reverse, computing dL/dp for every parameter p that requires gradients. `optimizer.step()` then uses these gradients to update the parameters.
 - Why is PyTorch's training loop significantly faster than the pure-Python mini framework?
 - a) PyTorch uses a different algorithm
 - b) PyTorch runs operations as optimized C++/CUDA kernels on GPU, while pure Python loops are interpreted one operation at a time **(correct)**
 - c) PyTorch uses smaller data types
 - d) PyTorch skips the backward pass
Explanation: PyTorch delegates matrix operations to highly optimized C++ and CUDA kernels that run on GPU with massive parallelism. Pure Python executes loops sequentially with interpreter overhead on each operation.
-

Introduction to JAX

PyTorch mutates tensors. TensorFlow builds graphs. JAX compiles pure functions. That last one changes how you think about deep learning.

Type: Build **Languages:** Python **Prerequisites:** Phase 03 Lessons 01-10, basic NumPy **Time:** ~90 minutes

Learning Objectives

Write pure-function neural network code using JAX's functional API (`jax.numpy`, `jax.grad`, `jax.jit`, `jax.vmap`)

Explain the key design difference between PyTorch's eager mutation and JAX's functional compilation model

Apply jit compilation and vmap vectorization to accelerate training loops compared to naive Python

Train a simple network in JAX and contrast the explicit state management with PyTorch's object-oriented approach

The Problem

You know how to build neural networks in PyTorch. You define an `nn.Module`, call `.backward()`, step the optimizer. It works. Millions of people use it.

But PyTorch has a constraint baked into its DNA: it traces operations eagerly, one at a time, in Python. Every `tensor + tensor` is a separate kernel launch. Every training step re-interprets the same Python code. This works fine until you need to train a 540-billion-parameter model across 2,048 TPUs. Then the overhead kills you.

Google DeepMind trains Gemini on JAX. Anthropic trained Claude on JAX. These are not small operations – they are the largest neural network training runs on Earth. They chose JAX because it treats your training loop as a compilable program, not a sequence of Python calls.

JAX is NumPy with three superpowers: automatic differentiation, JIT compilation to XLA, and automatic vectorization. You write a function that processes one example. JAX gives you a function that processes a batch, computes gradients, compiles to machine code, and runs across multiple devices. All without changing the original function.

The Concept

The JAX Philosophy

JAX is a functional framework. No classes, no mutable state, no `.backward()` method. Instead:

PyTorch	JAX
nn.Module class with state	Pure function: $f(\text{params}, x) \rightarrow y$
loss.backward()	<code>jax.grad(loss_fn)(params, x, y)</code>
Eager execution	JIT compilation via XLA
for x in batch: manual loop	<code>jax.vmap(f)</code> auto-vectorization
DataParallel / FSDP	<code>jax.pmap(f)</code> auto-parallelism
Mutable <code>model.parameters()</code>	Immutable pytree of arrays

This is not a style preference. It is a compiler constraint. JIT compilation requires pure functions – same inputs always produce same outputs, no side effects. That restriction is what makes 100x speedups possible.

jax.numpy: The Familiar Surface

JAX reimplements the NumPy API on accelerators:

```
import jax.numpy as jnp

a = jnp.array([1.0, 2.0, 3.0])
b = jnp.array([4.0, 5.0, 6.0])
c = jnp.dot(a, b)
```

Same function names. Same broadcasting rules. Same slicing semantics. But the arrays live on GPU/TPU, and every operation is traceable by the compiler.

One critical difference: JAX arrays are immutable. No `a[0] = 5`. Instead: `a = a.at[0].set(5)`. This feels awkward for a week, then it clicks – immutability is what makes transformations like `grad`, `jit`, and `vmap` composable.

jax.grad: Functional Autodiff

PyTorch attaches gradients to tensors (`.grad`). JAX attaches gradients to functions.

```
import jax

def f(x):
    return x ** 2

df = jax.grad(f)
df(3.0)
```

`jax.grad` takes a function and returns a new function that computes the gradient. No `.backward()` call. No computation graph stored on tensors. The gradient is just another function you can call, compose, or JIT-compile.

This composes arbitrarily:

```
d2f = jax.grad(jax.grad(f))
d2f(3.0)
```

Second derivatives. Third derivatives. Jacobians. Hessians. All by composing `grad`. PyTorch can do this too (`torch.autograd.functional.hessian`), but it is bolted on. In JAX, it is the foundation.

The constraint: `grad` only works on pure functions. No print statements inside (they run during tracing, not execution). No mutation of external state. No random number generation without explicit key management.

jit: Compile to XLA

```
@jax.jit
def train_step(params, x, y):
    loss = loss_fn(params, x, y)
    return loss

fast_step = jax.jit(train_step)
```

On the first call, JAX traces the function – it records which operations happen, without executing them. Then it hands that trace to XLA (Accelerated Linear Algebra), Google’s compiler for TPUs and GPUs. XLA fuses operations, eliminates redundant memory copies, and generates optimized machine code.

Subsequent calls skip Python entirely. The compiled code runs on the accelerator at C++ speed.

When JIT helps: - Training steps (same computation repeated thousands of times) - Inference (same model, different inputs) - Any function called more than once with similar-shaped inputs

When JIT hurts: - Functions with Python control flow that depends on values (`if x > 0` where `x` is a traced array) - One-shot computations (compilation overhead exceeds runtime) - Debugging (tracing hides the actual execution)

The control flow restriction is real. `jax.lax.cond` replaces `if/else`. `jax.lax.scan` replaces `for` loops. These are not optional – they are the price of compilation.

vmap: Automatic Vectorization

You write a function that processes one example:

```
def predict(params, x):
    return jnp.dot(params['w'], x) + params['b']
```

vmap lifts it to process a batch:

```
batch_predict = jax.vmap(predict, in_axes=(None, 0))
```

`in_axes=(None, 0)` means: do not batch over params (shared), batch over axis 0 of x. No manual for loop. No reshaping. No batch dimension threading. JAX figures out the batch dimension and vectorizes the entire computation.

This is not syntactic sugar. `vmap` generates fused vectorized code that runs 10-100x faster than a Python loop. And it composes with `jit` and `grad`:

```
per_example_grads = jax.vmap(jax.grad(loss_fn), in_axes=(None, 0, 0))
```

Per-example gradients. One line. This is nearly impossible in PyTorch without hacks.

pmap: Data Parallelism Across Devices

```
parallel_step = jax.pmap(train_step, axis_name='devices')
```

`pmap` replicates the function across all available devices (GPUs/TPUs) and splits the batch. Inside the function, `jax.lax.pmean` and `jax.lax.psum` synchronize gradients across devices.

Google trains Gemini across thousands of TPU v5e chips using `pmap` (and its successor `shard_map`). The programming model: write the single-device version, wrap with `pmap`, done.

Pytrees: The Universal Data Structure

JAX operates on “pytrees” – nested combinations of lists, tuples, dicts, and arrays. Your model parameters are a pytree:

```
params = {
    'layer1': {'w': jnp.zeros((784, 256)), 'b': jnp.zeros(256)},
    'layer2': {'w': jnp.zeros((256, 128)), 'b': jnp.zeros(128)},
    'layer3': {'w': jnp.zeros((128, 10)), 'b': jnp.zeros(10)},
}
```

Every JAX transformation – `grad`, `jit`, `vmap` – knows how to traverse pytrees. `jax.tree.map(f, tree)` applies `f` to every leaf. This is how optimizers update all parameters at once:

```
params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
```

No `.parameters()` method. No parameter registration. The tree structure is the model.

Functional vs Object-Oriented

PyTorch stores state inside objects:

```
class Model(nn.Module):
    def __init__(self):
        self.linear = nn.Linear(784, 10)

    def forward(self, x):
        return self.linear(x)
```

JAX uses pure functions with explicit state:

```
def predict(params, x):
    return jnp.dot(x, params['w']) + params['b']
```

The params are passed in. Nothing is stored. Nothing is mutated. This makes every function testable, composable, and compilable. It also means you manage the params yourself – or use a library like Flax or Equinox.

The JAX Ecosystem

JAX gives you primitives. Libraries give you ergonomics:

Library	Role	Style
Flax (Google)	Neural network layers	nn.Module with explicit state
Equinox (Patrick Kidger)	Neural network layers	Pytree-based, Pythonic
Optax (DeepMind)	Optimizers + LR schedules	Composable gradient transforms
Orbax (Google)	Checkpointing	Save/restore pytrees
CLU (Google)	Metrics + logging	Training loop utilities

Optax is the standard optimizer library. It separates the gradient transformation (Adam, SGD, clipping) from the parameter update, making it trivial to compose:

```
optimizer = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adam(learning_rate=1e-3),
)
```

When to Use JAX vs PyTorch

Factor	JAX	PyTorch
TPU support	First-class (Google built both)	Community-maintained (torch_xla)

Factor	JAX	PyTorch
GPU support	Good (CUDA via XLA)	Best-in-class (native CUDA)
Debugging	Hard (tracing + compilation)	Easy (eager, line-by-line)
Ecosystem	Research-focused (Flax, Equinox)	Massive (HuggingFace, torchvision, etc.)
Hiring	Niche (Google/DeepMind/Anthropic)	Mainstream (everywhere)
Large-scale training	Superior (XLA, pmap, mesh)	Good (FSDP, DeepSpeed)
Prototyping speed	Slower (functional overhead)	Faster (mutate and go)
Production inference	TensorFlow Serving, Vertex AI	TorchServe, Triton, ONNX
Who uses it	DeepMind (Gemini), Anthropic (Claude)	Meta (Llama), OpenAI (GPT), Stability AI

The honest answer: use PyTorch unless you have a specific reason to use JAX. Those reasons are – TPU access, need for per-example gradients, multi-device training at massive scale, or working at Google/DeepMind/Anthropic.

Random Numbers in JAX

JAX does not have a global random state. Every random operation requires an explicit PRNG key:

```
key = jax.random.PRNGKey(42)
key1, key2 = jax.random.split(key)
w = jax.random.normal(key1, shape=(784, 256))
```

This is annoying at first. But it guarantees reproducibility across devices and compilations – a property that PyTorch’s `torch.manual_seed` cannot guarantee in multi-GPU settings.

Build It

Step 1: Setup and Data

We will train a 3-layer MLP on MNIST using JAX and Optax. 784 inputs, two hidden layers of 256 and 128 neurons, 10 output classes.

```
import jax
import jax.numpy as jnp
from jax import random
import optax

def get_mnist_data():
    from sklearn.datasets import fetch_openml
```

```
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X = mnist.data.astype('float32') / 255.0
y = mnist.target.astype('int')
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]
return X_train, y_train, X_test, y_test
```

Step 2: Initialize Parameters

No class. Just a function that returns a pytree:

```
def init_params(key):
    k1, k2, k3 = random.split(key, 3)
    scale1 = jnp.sqrt(2.0 / 784)
    scale2 = jnp.sqrt(2.0 / 256)
    scale3 = jnp.sqrt(2.0 / 128)
    params = {
        'layer1': {
            'w': scale1 * random.normal(k1, (784, 256)),
            'b': jnp.zeros(256),
        },
        'layer2': {
            'w': scale2 * random.normal(k2, (256, 128)),
            'b': jnp.zeros(128),
        },
        'layer3': {
            'w': scale3 * random.normal(k3, (128, 10)),
            'b': jnp.zeros(10),
        },
    }
    return params
```

He-initialization, done manually. Three PRNG keys split from one seed. Every weight is an immutable array in a nested dict.

Step 3: Forward Pass

```
def forward(params, x):
    x = jnp.dot(x, params['layer1']['w']) + params['layer1']['b']
    x = jax.nn.relu(x)
    x = jnp.dot(x, params['layer2']['w']) + params['layer2']['b']
    x = jax.nn.relu(x)
    x = jnp.dot(x, params['layer3']['w']) + params['layer3']['b']
    return x

def loss_fn(params, x, y):
    logits = forward(params, x)
    one_hot = jax.nn.one_hot(y, 10)
    return -jnp.mean(jnp.sum(jax.nn.log_softmax(logits) * one_hot, axis=-1))
```


Pure functions. Params in, prediction out. No `self`, no stored state. `loss_fn` computes cross-entropy from scratch – softmax, log, negative mean.

Step 4: JIT-Compiled Training Step

```
@jax.jit
def train_step(params, opt_state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state, loss

@jax.jit
def accuracy(params, x, y):
    logits = forward(params, x)
    preds = jnp.argmax(logits, axis=-1)
    return jnp.mean(preds == y)
```

`jax.value_and_grad` returns both the loss value and the gradients in one pass. The `@jax.jit` decorator compiles both functions to XLA. After the first call, each training step runs without touching Python.

Step 5: Training Loop

```
optimizer = optax.adam(learning_rate=1e-3)

X_train, y_train, X_test, y_test = get_mnist_data()
X_train, X_test = jnp.array(X_train), jnp.array(X_test)
y_train, y_test = jnp.array(y_train), jnp.array(y_test)

key = random.PRNGKey(0)
params = init_params(key)
opt_state = optimizer.init(params)

batch_size = 128
n_epochs = 10

for epoch in range(n_epochs):
    key, subkey = random.split(key)
    perm = random.permutation(subkey, len(X_train))
    X_shuffled = X_train[perm]
    y_shuffled = y_train[perm]

    epoch_loss = 0.0
    n_batches = len(X_train) // batch_size
    for i in range(n_batches):
        start = i * batch_size
        xb = X_shuffled[start:start + batch_size]
        yb = y_shuffled[start:start + batch_size]
        params, opt_state, loss = train_step(params, opt_state, xb, yb)
        epoch_loss += loss
```

```

train_acc = accuracy(params, X_train[:5000], y_train[:5000])
test_acc = accuracy(params, X_test, y_test)
print(f"Epoch {epoch + 1:2d} | Loss: {epoch_loss / n_batches:.4f} | "
      f"Train Acc: {train_acc:.4f} | Test Acc: {test_acc:.4f}")

```

10 epochs. ~97% test accuracy. The first epoch is slow (JIT compilation). Epochs 2-10 are fast.

Notice what is missing: no `.zero_grad()`, no `.backward()`, no `.step()`. The entire update is one composed function call. Gradients are computed, transformed by Adam, and applied to parameters – all inside `train_step`.

Use It

Flax: The Google Standard

Flax is the most common JAX neural network library. It adds `nn.Module` back, but with explicit state management:

```

import flax.linen as nn

class MLP(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.Dense(256)(x)
        x = nn.relu(x)
        x = nn.Dense(128)(x)
        x = nn.relu(x)
        x = nn.Dense(10)(x)
        return x

model = MLP()
params = model.init(jax.random.PRNGKey(0), jnp.ones((1, 784)))
logits = model.apply(params, x_batch)

```

Same structure as PyTorch, but `params` is separate from the model. `model.init()` creates `params`. `model.apply(params, x)` runs the forward pass. The model object has no state.

Equinox: The Pythonic Alternative

Equinox (by Patrick Kidger) represents models as pytrees:

```

import equinox as eqx

model = eqx.nn.MLP(
    in_size=784, out_size=10, width_size=256, depth=2,
    activation=jax.nn.relu, key=jax.random.PRNGKey(0)
)
logits = model(x)

```

The model itself is a pytree. No `.apply()` needed. Parameters are just the model's leaves. This is closer to how JAX thinks.

Optax: Composable Optimizers

Optax decouples the gradient transformation from the update:

```
schedule = optax.warmup_cosine_decay_schedule(
    init_value=0.0, peak_value=1e-3,
    warmup_steps=1000, decay_steps=50000
)

optimizer = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adamw(learning_rate=schedule, weight_decay=0.01),
)
```

Gradient clipping, learning rate warmup, weight decay – all composed as a chain of transforms. Each transform sees the gradients, modifies them, and passes them to the next. No monolithic optimizer class.

Ship It

Installation:

```
pip install jax jaxlib optax flax
```

For GPU support:

```
pip install jax[cuda12]
```

For TPU (Google Cloud):

```
pip install jax[tpu] -f https://storage.googleapis.com/jax-releases/libtpu_releases.html
```

Performance gotchas:

72. First JIT call is slow (compilation). Warm up before benchmarking.
73. Avoid Python loops over JAX arrays inside JIT. Use `jax.lax.scan` or `jax.lax.fori_loop`.
74. `jax.debug.print()` works inside JIT. Regular `print()` does not.
75. Profile with `jax.profiler` or TensorBoard. XLA compilation can hide bottlenecks.
76. JAX pre-allocates 75% of GPU memory by default. Set `XLA_PYTHON_CLIENT_PREALLOCATE=false` to disable.

Checkpointing:

```
import orbax.checkpoint as ocp
checkpointer = ocp.PyTreeCheckpointer()
checkpointer.save('/tmp/model', params)
restored = checkpointer.restore('/tmp/model')
```

This lesson produces: - outputs/prompt-jax-optimizer.md – a prompt for choosing the right JAX optimizer configuration - outputs/skill-jax-patterns.md – a skill covering functional patterns in JAX

Exercises

Add dropout to the MLP. In JAX, dropout requires a PRNG key – thread a key through the forward pass and split it for each dropout layer. Compare test accuracy with and without.

Use `jax.vmap` to compute per-example gradients for a batch of 32 MNIST images. Compute the gradient norm for each example. Which examples have the largest gradients, and why?

Replace the manual forward function with a generic `mlp_forward(params, x)` that works for any number of layers. Use `jax.tree.leaves` to determine the depth automatically.

Benchmark the training step with and without `@jax.jit`. Time 100 steps of each. How large is the speedup on your hardware? What is the compilation overhead on the first call?

Implement gradient clipping by composing `optax.chain(optax.clip_by_global_norm(1.0), optax.adam(1e-3))`. Train with and without clipping. Plot the gradient norm over training to see the effect.

Key Terms

Term	What people say	What it actually means
XLA	“The thing that makes JAX fast”	Accelerated Linear Algebra – a compiler that fuses operations and generates optimized GPU/TPU kernels from a computation graph
JIT	“Just-in-time compilation”	JAX traces the function on first call, compiles to XLA, then runs the compiled version on subsequent calls
Pure function	“No side effects”	A function where the output depends only on inputs – no global state, no mutation, no randomness without explicit keys
vmap	“Auto-batching”	Transforms a function that processes one example into one that processes a batch, without rewriting
pmap	“Auto-parallelism”	Replicates a function across multiple devices and splits the input batch

Term	What people say	What it actually means
Pytree	“Nested dict of arrays”	Any nested structure of lists, tuples, dicts, and arrays that JAX can traverse and transform
Tracing	“Recording the computation”	JAX executes the function with abstract values to build a computation graph, without computing real results
Functional autodiff	“grad of a function”	Computing derivatives by transforming functions, not by attaching gradient storage to tensors
Optax	“JAX’s optimizer library”	A composable library of gradient transformations – Adam, SGD, clipping, scheduling – that chain together
Flax	“JAX’s nn.Module”	Google’s neural network library for JAX, adding layer abstractions while keeping state explicit

You are a JAX training configuration expert. Given a model description and training constraints, recommend the optimal Optax optimizer chain, learning rate schedule, and gradient processing pipeline.

Input

I will describe: - Model architecture (MLP, Transformer, CNN, etc.) - Parameter count - Dataset size and batch size - Hardware (GPU count, TPU pod slice, single device) - Training budget (time or step count) - Known issues (gradient explosion, slow convergence, overfitting)

Decision Protocol

1. Choose Base Optimizer

Scenario	Optimizer	Why
Default / prototyping	<code>optax.adam(1e-3)</code>	Reliable, fast convergence
Large Transformer (>1B params)	<code>optax.adamw(lr, weight_decay=0.1)</code>	Weight decay prevents overfitting at scale
Fine-tuning pretrained model	<code>optax.adamw(1e-5, weight_decay=0.01)</code>	Low LR preserves pretrained features
Memory-constrained	<code>optax.sgd(lr, momentum=0.9)</code>	2x less optimizer state than Adam
Second-order approximation	<code>optax.lamb(lr)</code>	Large-batch training (batch >8K)

Scenario	Optimizer	Why
Sparse gradients	optax.adafactor(lr)	Factored second moments, less memory

2. Choose Learning Rate Schedule

Training length	Schedule	Optax code
< 10K steps	Constant	optax.constant_schedule(lr)
10K - 100K steps	Warmup + cosine decay	optax.warmup_cosine_decay_schedule(init_value=0, peak_value=lr, warmup_steps=N, decay_steps=total)
> 100K steps	Warmup + linear decay	optax.join_schedules([optax.linear_schedule(0, lr, warmup), optax.linear_schedule(lr, 0, total - warmup)], [warmup])
Fine-tuning	Warmup + constant	optax.join_schedules([optax.linear_schedule(0, lr, 100), optax.constant_schedule(lr)], [100])

Warmup steps rule of thumb: 1-5% of total training steps. For Transformers, 2000 steps minimum.

3. Add Gradient Processing

Build the chain from these components:

```
optimizer = optax.chain(
    optax.clip_by_global_norm(max_norm),    # gradient clipping
    optax.add_decayed_weights(decay),      # L2 regularization (if not using adamw)
    base_optimizer,                        # adam, sgd, etc.
)
```

Issue	Fix	Typical value
Gradient explosion	optax.clip_by_global_norm(max_norm)	1.0 for Transformers, 5.0 for CNNs
Gradient noise	optax.clip(max_delta)	1.0
Overfitting	optax.add_decayed_weights(weight_decay)	0.01 - 0.1
Unstable early training	Warmup schedule	1-5% of total steps

4. Multi-Device Considerations

For pmap-based training: - Gradients are already averaged across devices via `jax.lax.pmean` - Scale learning rate linearly with device count (linear scaling rule) - Scale warmup steps proportionally - Effective batch size = per-device batch * num_devices

5. Checkpointing the Optimizer State

```
import orbax.checkpoint as ocp
checkpointer = ocp.PyTreeCheckpointner()
checkpointer.save(path, {'params': params, 'opt_state': opt_state})
```

Always checkpoint both params and opt_state. Adam stores momentum and variance – losing them resets training progress.

Output Format

Provide:

Complete Optax chain as runnable Python code

Learning rate schedule with warmup/decay steps calculated

Expected behavior (convergence speed, memory usage, known risks)

Monitoring advice (which metrics to watch, what values indicate problems)

Example output:

```
total_steps = 50000
warmup_steps = 2000

schedule = optax.warmup_cosine_decay_schedule(
    init_value=0.0,
    peak_value=3e-4,
    warmup_steps=warmup_steps,
    decay_steps=total_steps,
    end_value=1e-6,
)

optimizer = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adamw(learning_rate=schedule, weight_decay=0.1),
)

opt_state = optimizer.init(params)
```

Always explain why each component is in the chain. State what to change first if training diverges.

JAX Functional Patterns

JAX transforms pure functions. Every pattern below follows one rule: write a function that takes inputs and returns outputs, with no side effects. Then transform it.

The Four Transforms

grad – Differentiate a function

```
grads = jax.grad(loss_fn)(params, x, y)
loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
```

Use when: you need gradients for optimization. Constraint: the function must return a scalar. For non-scalar outputs, use `jax.jacobian`.

jit – Compile a function

```
fast_fn = jax.jit(f)
```

Use when: the function will be called more than once with same-shaped inputs. Constraint: no Python control flow that depends on traced values. Use `jax.lax.cond` for conditionals, `jax.lax.scan` for loops.

vmap – Vectorize a function

```
batch_fn = jax.vmap(f, in_axes=(None, 0))
```

Use when: you wrote a function for one example and need it to work on batches. `in_axes` specifies which argument axis to batch over. `None` means do not batch (broadcast).

pmap – Parallelize across devices

```
parallel_fn = jax.pmap(f, axis_name='devices')
```

Use when: you have multiple GPUs/TPUs and want data parallelism. Inside the function, `jax.lax.pmean(x, 'devices')` averages across devices.

Composition Rules

Transforms compose. The order matters:

```
per_example_grads = jax.jit(jax.vmap(jax.grad(loss_fn), in_axes=(None, 0, 0)))
```

Reading right to left: take gradient of `loss_fn`, vectorize over examples, compile the result.

Valid compositions: - `jit(grad(f))` – compiled gradient computation - `jit(vmap(f))` – compiled batched computation - `vmap(grad(f))` – per-example gradients - `pmap(jit(f))` – parallel compiled computation - `grad(jit(f))` – gradient of compiled function (same as `jit(grad(f))`)

Parameter Management Pattern

JAX parameters are pytrees (nested dicts of arrays):

```
params = {
    'layer1': {'w': jnp.zeros((784, 256)), 'b': jnp.zeros(256)},
    'layer2': {'w': jnp.zeros((256, 10)), 'b': jnp.zeros(10)},
}
```

Update all parameters at once:

```
params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
```

Count parameters:

```
n_params = sum(p.size for p in jax.tree.leaves(params))
```

PRNG Key Management

JAX requires explicit random keys:

```
key = jax.random.PRNGKey(0)
key, subkey = jax.random.split(key)
noise = jax.random.normal(subkey, shape)
```

For multiple random operations, split once:

```
keys = jax.random.split(key, n)
```

Never reuse a key. Always split before using.

Common Mistakes

- **Mutating arrays inside jit:** JAX arrays are immutable. Use `x.at[i].set(v)` instead of `x[i] = v`.
- **Using Python print inside jit:** `print` runs during tracing, not execution. Use `jax.debug.print("{x}", x)`.
- **Python if/for inside jit on traced values:** Use `jax.lax.cond`, `jax.lax.switch`, `jax.lax.scan`, `jax.lax.fori_loop`.
- **Forgetting `.block_until_ready()`:** JAX uses async dispatch. For benchmarking, call `.block_until_ready()` to wait for actual completion.

- **Reusing PRNG keys:** Two operations with the same key produce the same “random” values. Always split.
- **Global state in jitted functions:** Global variables are captured at trace time. Changes after tracing are invisible. Pass everything as arguments.

Decision Checklist

- 205. Is the function called more than once? Add `@jax.jit`.
- 206. Does it need gradients? Wrap with `jax.grad` or `jax.value_and_grad`.
- 207. Does it process one example but you have a batch? Wrap with `jax.vmap`.
- 208. Do you have multiple devices? Wrap with `jax.pmap`.
- 209. Does it use randomness? Thread PRNG keys through explicitly.
- 210. Does it have Python control flow on array values? Replace with `jax.lax` primitives.

When to Use JAX

Use JAX when: - You need per-example gradients (differential privacy, Fisher information) - You are training on TPUs (JAX is the native framework) - You need higher-order derivatives (Hessians, Jacobians) - You want to compile the entire training step to a single kernel - Your team is at Google DeepMind or Anthropic

Use PyTorch when: - You want the largest ecosystem (HuggingFace, torchvision, Lightning) - You prioritize debugging ease over raw speed - You are deploying to NVIDIA GPUs with TorchServe/Triton - You are hiring (more PyTorch developers exist) - You want to iterate fast on new architectures

Quiz

Pre-lesson check

- 211. What is the fundamental design difference between PyTorch and JAX?
 - PyTorch is faster
 - PyTorch mutates tensors eagerly while JAX compiles pure functions without side effects **(correct)**
 - JAX doesn’t support GPUs
 - PyTorch doesn’t support automatic differentiation

Explanation: PyTorch uses eager execution with mutable state (e.g., `tensor.grad` is

modified in place). JAX embraces functional programming: functions are pure, state is explicit, and jit compilation optimizes entire computation graphs.

212. What does `jax.jit` do?

1. It adds dropout regularization
2. It compiles a Python function into optimized XLA code that runs much faster than interpreted Python **(correct)**
3. It initializes model weights
4. It computes gradients
Explanation: `jax.jit` traces a function and compiles it to XLA (Accelerated Linear Algebra) machine code. The first call is slow (compilation), but subsequent calls run the optimized compiled version.

Post-lesson check

- What does `jax.vmap` do?
 5. Vectorizes a function to run over a batch dimension without writing explicit loops **(correct)**
 6. Validates model architecture
 7. Manages GPU memory
 8. Computes second-order gradients
Explanation: `jax.vmap` automatically vectorizes a function written for a single example to process an entire batch. You write code for one sample and `vmap` handles batching, often more efficiently than manual batch loops.
- How does JAX handle model state (weights) differently than PyTorch?
 - a) JAX stores weights on CPU only
 - b) JAX requires explicit state passing – weights are function arguments, not mutable object attributes **(correct)**

- c) JAX doesn't support trainable weights
 - d) JAX and PyTorch handle state identically
Explanation: In PyTorch, weights live inside nn.Module objects and are mutated in place. In JAX, weights are passed explicitly as function arguments and returned as new values. No mutation, no hidden state.
- When would you choose JAX over PyTorch?
 - a) For quick prototyping of small models
 - b) When training at massive scale on TPU pods, where compilation and functional transforms provide significant speedups (**correct**)
 - c) When you need the largest ecosystem of pretrained models
 - d) For deployment on mobile devices
Explanation: JAX excels at scale: jit compilation eliminates Python overhead, pmap handles multi-device parallelism naturally, and XLA optimization across the full computation graph gives significant speedups on TPU clusters.
-

Debugging Neural Networks

Your network compiled. It ran. It produced a number. The number is wrong and nothing crashed. Welcome to the hardest kind of debugging – the kind where there is no error message.

Type: Build **Languages:** Python, PyTorch **Prerequisites:** Phase 03 Lessons 01-10 (especially backpropagation, loss functions, optimizers) **Time:** ~90 minutes

Learning Objectives

77. Diagnose common neural network failures (NaN loss, flat loss curve, overfitting, oscillation) using systematic debugging strategies
78. Apply the “overfit one batch” technique to verify that your model architecture and training loop are correct
79. Inspect gradient magnitudes, activation distributions, and weight norms to identify vanishing/exploding gradient problems
80. Build a debugging checklist that covers data pipeline, model architecture, loss function, optimizer, and learning rate issues

The Problem

Traditional software crashes when it is broken. A null pointer throws an exception. A type mismatch fails at compile time. An off-by-one error produces a clearly wrong output.

Neural networks do not give you that luxury.

A broken neural network runs to completion, prints a loss value, and outputs predictions. The loss might decrease. The predictions might look plausible. But the model is silently wrong – learning shortcuts, memorizing noise, or converging to a useless local minimum. Google researchers estimated that 60-70% of ML debugging time is spent on “silent” bugs that produce no errors but degrade model quality.

The difference between a working model and a broken one is often a single misplaced line: a missing `zero_grad()`, a transposed dimension, a learning rate off by 10x. the canonical “Recipe for Training Neural Networks” (2019) opens with this: “The most common neural net mistakes are bugs that don’t crash.”

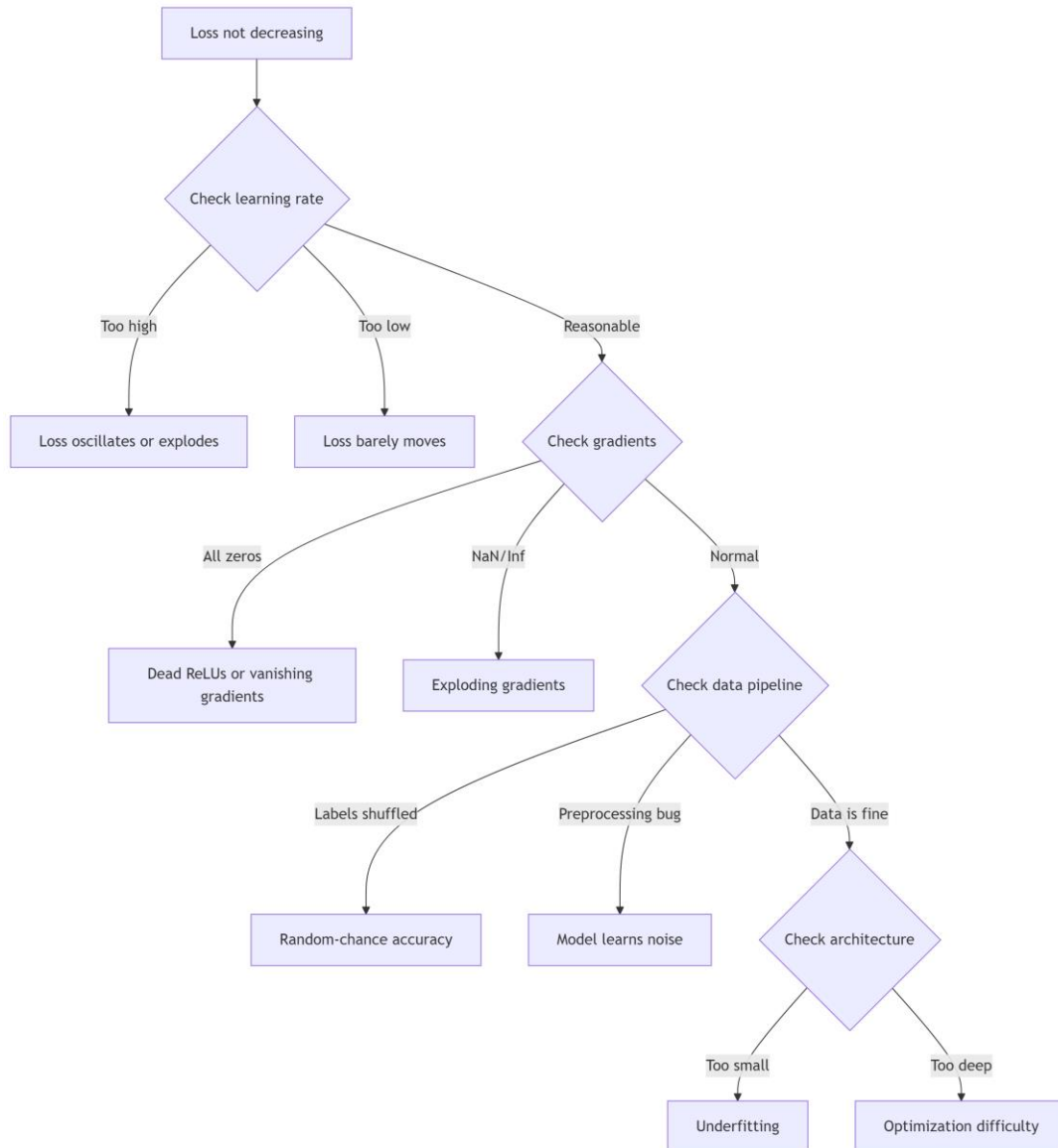
This lesson teaches you to find those bugs.

The Concept

The Debugging Mindset

Forget print-and-pray debugging. Neural network debugging requires a systematic approach because the feedback loop is slow (minutes to hours per training run) and the symptoms are ambiguous (bad loss could mean 20 different things).

The golden rule: **start simple, add complexity one piece at a time, and verify each piece independently.**



Symptom 1: Loss Not Decreasing

This is the most common complaint. The training loop runs, epochs tick by, and the loss stays flat or oscillates wildly.

Wrong learning rate. Too high: loss oscillates or jumps to NaN. Too low: loss decreases so slowly it looks flat. For Adam, start at $1e-3$. For SGD, start at $1e-1$ or $1e-2$. Always try 3 learning rates spanning 10x each (e.g., $1e-2$, $1e-3$, $1e-4$) before concluding something else is wrong.

Dead ReLUs. If a ReLU neuron receives a large negative input, it outputs 0 and its gradient is 0. It never activates again. If enough neurons die, the network cannot learn. Check: print the fraction of activations that are exactly 0 after each ReLU layer. If >50% are dead, switch to LeakyReLU or reduce the learning rate.

Vanishing gradients. In deep networks with sigmoid or tanh activations, gradients shrink exponentially as they propagate backward. By the time they reach the first layer, they are ~ 0 . The first layers stop learning. Fix: use ReLU/GELU, add residual connections, or use batch normalization.

Exploding gradients. The opposite problem – gradients grow exponentially. Common in RNNs and very deep networks. Loss jumps to NaN. Fix: gradient clipping (`torch.nn.utils.clip_grad_norm_`), lower learning rate, or add normalization.

Symptom 2: Loss Decreasing But Model is Bad

The loss goes down. Training accuracy hits 99%. But test accuracy is 55%. Or the model produces nonsensical outputs on real data.

Overfitting. The model memorizes training data instead of learning patterns. Gap between training and validation loss grows over time. Fix: more data, dropout, weight decay, early stopping, data augmentation.

Data leakage. Test data leaked into training. Accuracy is suspiciously high. Common causes: shuffling before splitting, preprocessing with statistics from the full dataset, duplicate samples across splits. Fix: split first, preprocess second, check for duplicates.

Label errors. 5-10% of labels in most real datasets are wrong (Northcutt et al., 2021 – “Pervasive Label Errors in Test Sets”). The model learns the noise. Fix: use confident learning to find and fix mislabeled examples, or use loss truncation to ignore high-loss samples.

Symptom 3: NaN or Inf in Loss

The loss value becomes nan or ∞ . Training is dead.

Learning rate too high. Gradient updates overshoot so far that weights explode. Fix: reduce by 10x.

$\log(0)$ or $\log(\text{negative})$. Cross-entropy loss computes $\log(p)$. If your model outputs exactly 0 or a negative probability, the log explodes. Fix: clamp predictions to $[\text{eps}, 1-\text{eps}]$ where $\text{eps}=1e-7$.

Division by zero. Batch normalization divides by standard deviation. A batch with constant values has $\text{std}=0$. Fix: add ϵ to the denominator (PyTorch does this by default, but custom implementations might not).

Numerical overflow. Large activations fed into `exp()` produce `Inf`. Softmax is especially prone. Fix: subtract the max before exponentiating (the log-sum-exp trick).

Technique 1: Gradient Checking

Compare your analytical gradients (from backprop) to numerical gradients (from finite differences). If they disagree, your backward pass has a bug.

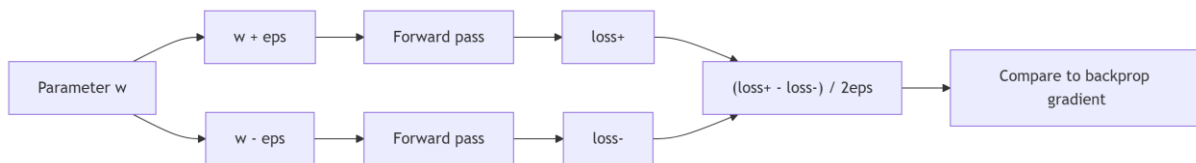
Numerical gradient for parameter `w`:

```
grad_numerical = (loss(w + eps) - loss(w - eps)) / (2 * eps)
```

Agreement metric (relative difference):

```
rel_diff = |grad_analytical - grad_numerical| / max(|grad_analytical|, |grad_numerical|, 1e-8)
```

If `rel_diff < 1e-5`: correct. If `rel_diff > 1e-3`: almost certainly a bug.



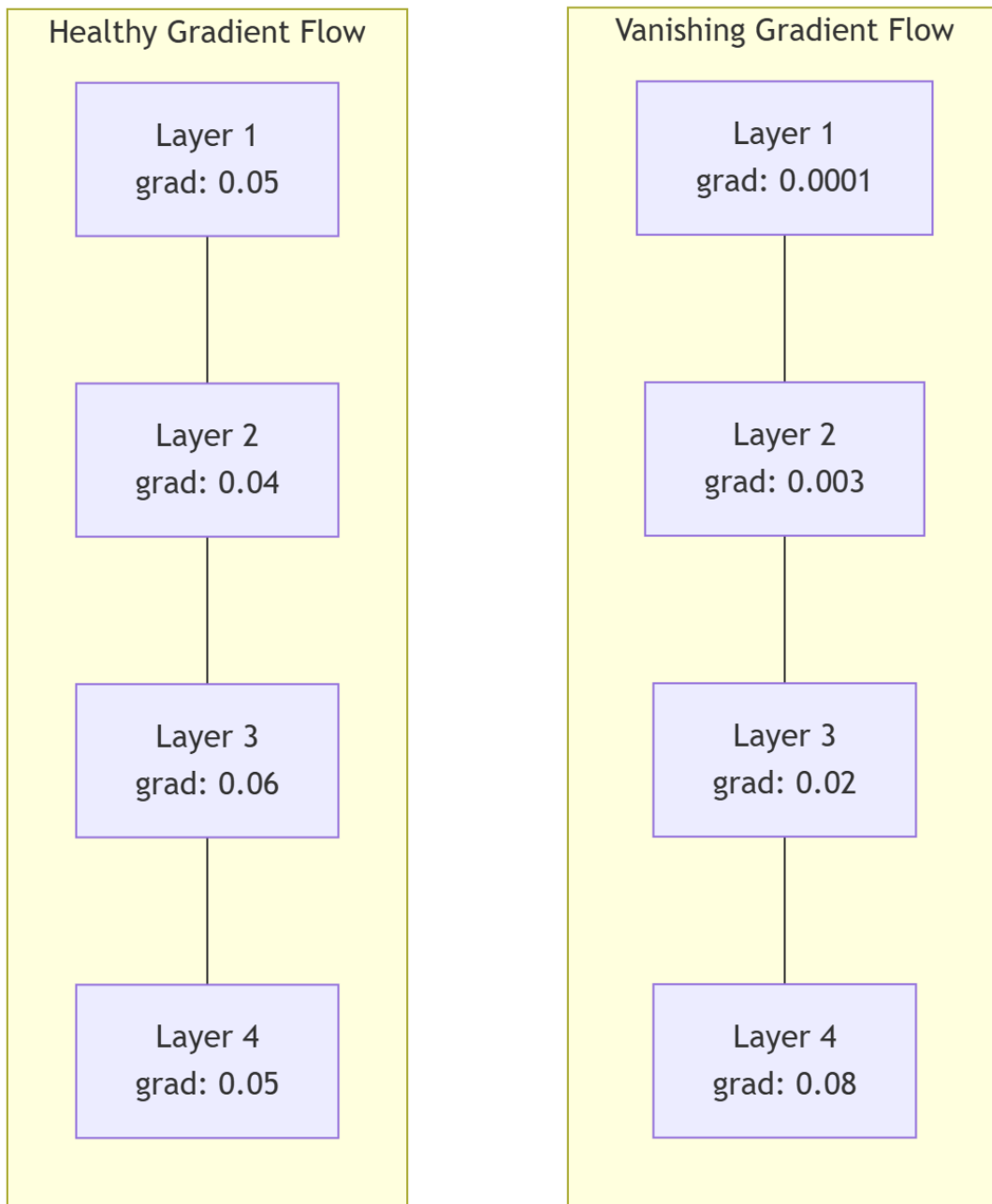
Technique 2: Activation Statistics

Monitor the mean and standard deviation of activations after each layer during training. Healthy networks maintain activations with mean near 0 and std near 1 (after normalization) or at least bounded.

Health indicator	Mean	Std	Diagnosis
Healthy	~0	~1	Network is learning normally
Saturated	>>0 or <<0	~0	Activations stuck at extreme values
Dead	0	0	Neurons are dead (all zeros)
Exploding	>>10	>>10	Activations growing without bound

Technique 3: Gradient Flow Visualization

Plot the average gradient magnitude for each layer. In a healthy network, gradient magnitudes should be roughly similar across layers. If early layers have gradients 1000x smaller than later layers, you have vanishing gradients.



Technique 4: The Overfit-One-Batch Test

The single most important debugging technique in deep learning.

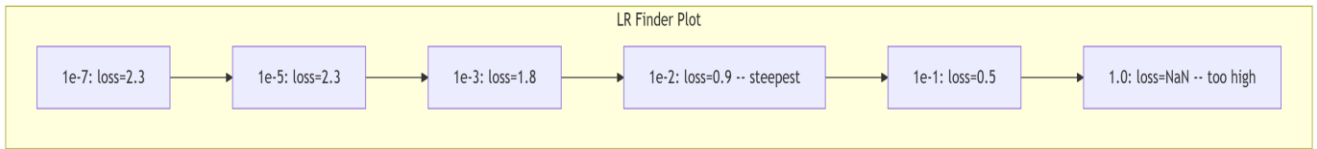
Take one small batch (8-32 samples). Train on it for 100+ iterations. The loss should go to nearly zero and training accuracy should hit 100%. If it does not, your model or training loop has a fundamental bug – do not proceed to full training.

This test catches: - Broken loss functions - Broken backward passes - Architecture too small to represent the data - Optimizer not connected to model parameters - Data and labels misaligned

This takes 30 seconds to run and saves hours of debugging full training runs.

Technique 5: Learning Rate Finder

Leslie Smith (2017) proposed sweeping the learning rate from very small ($1e-7$) to very large (10) over one epoch while recording the loss. Plot loss vs learning rate. The optimal learning rate is roughly 10x smaller than the rate where loss starts decreasing fastest.



Best LR in this example: $\sim 1e-3$ (one order of magnitude before the steepest point).

Common PyTorch Bugs

These are the bugs that waste the most collective hours in the PyTorch community:

Bug	Symptom	Fix
Forgetting <code>optimizer.zero_grad()</code>	Gradients accumulate across batches, loss oscillates	Add <code>optimizer.zero_grad()</code> before <code>loss.backward()</code>
Forgetting <code>model.eval()</code> at test time	Dropout and batch norm behave differently, test accuracy varies between runs	Add <code>model.eval()</code> and <code>torch.no_grad()</code>
Wrong tensor shapes	Silent broadcasting produces wrong results, no error	Print shapes after every operation during debugging
CPU/GPU mismatch	<code>RuntimeError: expected CUDA tensor</code>	Use <code>.to(device)</code> on model AND data
Not detaching tensors	Computation graph grows forever, OOM	Use <code>.detach()</code> or with <code>torch.no_grad()</code>
In-place operations breaking autograd	<code>RuntimeError: modified by in-place operation</code>	Replace <code>x += 1</code> with <code>x = x + 1</code>
Data not normalized	Loss stuck at random-chance level	Normalize inputs to mean=0, std=1

Bug	Symptom	Fix
Labels as wrong dtype	Cross-entropy expects Long, got Float	Cast labels: labels.long()

The Master Debugging Table

Symptom	Likely cause	First thing to try
Loss stuck at - $\log(1/\text{num_classes})$	Model predicting uniform distribution	Check data pipeline, verify labels match inputs
Loss NaN after a few steps	Learning rate too high	Reduce LR by 10x
Loss NaN immediately	$\log(0)$ or division by zero	Add ϵ to log/division operations
Loss oscillating wildly	LR too high or batch size too small	Reduce LR, increase batch size
Loss decreasing then plateaus	LR too high for fine-tuning phase	Add LR schedule (cosine or step decay)
Training acc high, test acc low	Overfitting	Add dropout, weight decay, more data
Training acc = test acc = chance	Model not learning anything	Run overfit-one-batch test
Training acc = test acc but both low	Underfitting	Bigger model, more layers, more features
Gradients all zero	Dead ReLUs or detached computation graph	Switch to LeakyReLU, check <code>.requires_grad</code>
Out of memory during training	Batch too large or graph not freed	Reduce batch size, use <code>torch.no_grad()</code> for eval

Build It

A diagnostic toolkit that monitors activations, gradients, and loss curves. You will deliberately break a network and use the toolkit to diagnose each problem.

Step 1: The NetworkDebugger Class

Hooks into a PyTorch model to record activation and gradient statistics per layer.

```
import torch
import torch.nn as nn
import math
```

```

class NetworkDebugger:
    def __init__(self, model):
        self.model = model
        self.activation_stats = {}
        self.gradient_stats = {}
        self.loss_history = []
        self.lr_losses = []
        self.hooks = []
        self._register_hooks()

    def _register_hooks(self):
        for name, module in self.model.named_modules():
            if isinstance(module, (nn.Linear, nn.Conv2d, nn.ReLU, nn.LeakyReLU)):
                hook = module.register_forward_hook(self._make_activation_hook(name))
                self.hooks.append(hook)
                hook = module.register_full_backward_hook(self._make_gradient_hook(name))
                self.hooks.append(hook)

    def _make_activation_hook(self, name):
        def hook(module, input, output):
            with torch.no_grad():
                out = output.detach().float()
                self.activation_stats[name] = {
                    "mean": out.mean().item(),
                    "std": out.std().item(),
                    "fraction_zero": (out == 0).float().mean().item(),
                    "min": out.min().item(),
                    "max": out.max().item(),
                }
        return hook

    def _make_gradient_hook(self, name):
        def hook(module, grad_input, grad_output):
            if grad_output[0] is not None:
                with torch.no_grad():
                    grad = grad_output[0].detach().float()
                    self.gradient_stats[name] = {
                        "mean": grad.mean().item(),
                        "std": grad.std().item(),
                        "abs_mean": grad.abs().mean().item(),
                        "max": grad.abs().max().item(),
                    }
        return hook

    def record_loss(self, loss_value):
        self.loss_history.append(loss_value)

    def check_loss_health(self):
        if len(self.loss_history) < 2:
            return "NOT_ENOUGH_DATA"
        recent = self.loss_history[-10:]

```

```

    if any(math.isnan(v) or math.isinf(v) for v in recent):
        return "NAN_OR_INF"
    if len(self.loss_history) >= 20:
        first_half = sum(self.loss_history[:10]) / 10
        second_half = sum(self.loss_history[-10:]) / 10
        if second_half >= first_half * 0.99:
            return "NOT DECREASING"
    if len(recent) >= 5:
        diffs = [recent[i+1] - recent[i] for i in range(len(recent)-1)]
        if max(diffs) - min(diffs) > 2 * abs(sum(diffs) / len(diffs)):
            return "OSCILLATING"
    return "HEALTHY"

def check_activations(self):
    issues = []
    for name, stats in self.activation_stats.items():
        if stats["fraction_zero"] > 0.5:
            issues.append(f"DEAD_NEURONS: {name} has {stats['fraction_zero']:.0%}
zero activations")
        if abs(stats["mean"]) > 10:
            issues.append(f"EXPLODING_ACTIVATIONS: {name} mean={stats['mean']:.2f
}")
        if stats["std"] < 1e-6:
            issues.append(f"COLLAPSED_ACTIVATIONS: {name} std={stats['std']:.2e}"
)
    return issues if issues else ["HEALTHY"]

def check_gradients(self):
    issues = []
    grad_magnitudes = []
    for name, stats in self.gradient_stats.items():
        grad_magnitudes.append((name, stats["abs_mean"]))
        if stats["abs_mean"] < 1e-7:
            issues.append(f"VANISHING_GRADIENT: {name} abs_mean={stats['abs_mean']
:.2e}")
        if stats["abs_mean"] > 100:
            issues.append(f"EXPLODING_GRADIENT: {name} abs_mean={stats['abs_mean']
:.2e}")
    if len(grad_magnitudes) >= 2:
        first_mag = grad_magnitudes[0][1]
        last_mag = grad_magnitudes[-1][1]
        if last_mag > 0 and first_mag / last_mag > 100:
            issues.append(f"GRADIENT_RATIO: first/last = {first_mag/last_mag:.0f}
x (vanishing)")
    return issues if issues else ["HEALTHY"]

def print_report(self):
    print("\n=== NETWORK DEBUGGER REPORT ===")
    print(f"\nLoss health: {self.check_loss_health()}")
    if self.loss_history:

```

```

        print(f" Last 5 losses: {[f'{v:.4f}' for v in self.loss_history[-5:]]}")
    print("\nActivation diagnostics:")
    for item in self.check_activations():
        print(f" {item}")
    print("\nGradient diagnostics:")
    for item in self.check_gradients():
        print(f" {item}")
    print("\nPer-layer activation stats:")
    for name, stats in self.activation_stats.items():
        print(f" {name}: mean={stats['mean']:.4f} std={stats['std']:.4f} zero=
{stats['fraction_zero']:.1%}")
    print("\nPer-layer gradient stats:")
    for name, stats in self.gradient_stats.items():
        print(f" {name}: abs_mean={stats['abs_mean']:.2e} max={stats['max']:.2e
}")

    def remove_hooks(self):
        for hook in self.hooks:
            hook.remove()
        self.hooks.clear()

```

Step 2: The Overfit-One-Batch Test

```

def overfit_one_batch(model, x_batch, y_batch, criterion, lr=0.01, steps=200):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    model.train()
    print("\n=== OVERFIT ONE BATCH TEST ===")
    print(f"Batch size: {x_batch.shape[0]}, Steps: {steps}")

    for step in range(steps):
        optimizer.zero_grad()
        output = model(x_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        optimizer.step()

        if step % 50 == 0 or step == steps - 1:
            with torch.no_grad():
                preds = (output > 0).float() if output.shape[-1] == 1 else output.argmax(
x(dim=1)
                targets = y_batch if y_batch.dim() == 1 else y_batch.squeeze()
                acc = (preds.squeeze() == targets).float().mean().item()
                print(f" Step {step:3d} | Loss: {loss.item():.6f} | Accuracy: {acc:.1%
}")

    final_loss = loss.item()
    if final_loss > 0.1:
        print(f"\n FAIL: Loss did not converge ({final_loss:.4f}). Model or training
loop is broken.")

```

```

        return False
    print(f"\n PASS: Loss converged to {final_loss:.6f}")
    return True

```

Step 3: Learning Rate Finder

```

def find_learning_rate(model, x_data, y_data, criterion, start_lr=1e-7, end_lr=10, steps=100):
    import copy
    original_state = copy.deepcopy(model.state_dict())
    optimizer = torch.optim.SGD(model.parameters(), lr=start_lr)
    lr_mult = (end_lr / start_lr) ** (1 / steps)

    model.train()
    results = []
    best_loss = float("inf")
    current_lr = start_lr

    print("\n=== LEARNING RATE FINDER ===")

    for step in range(steps):
        optimizer.zero_grad()
        output = model(x_data)
        loss = criterion(output, y_data)

        if math.isnan(loss.item()) or loss.item() > best_loss * 10:
            break

        best_loss = min(best_loss, loss.item())
        results.append((current_lr, loss.item()))

        loss.backward()
        optimizer.step()

        current_lr *= lr_mult
        for param_group in optimizer.param_groups:
            param_group["lr"] = current_lr

    model.load_state_dict(original_state)

    if len(results) < 10:
        print(" Could not complete LR sweep -- loss diverged too quickly")
        return results

    min_loss_idx = min(range(len(results)), key=lambda i: results[i][1])
    suggested_lr = results[max(0, min_loss_idx - 10)][0]

    print(f" Swept {len(results)} steps from {start_lr:.0e} to {results[-1][0]:.0e}")
    print(f" Minimum loss {results[min_loss_idx][1]:.4f} at lr={results[min_loss_idx][0]:.2e}")

```

```

print(f" Suggested learning rate: {suggested_lr:.2e}")

return results

```

Step 4: Gradient Checker

```

def _flat_to_multi_index(flat_idx, shape):
    multi_idx = []
    remaining = flat_idx
    for dim in reversed(shape):
        multi_idx.insert(0, remaining % dim)
        remaining //= dim
    return tuple(multi_idx)

def gradient_check(model, x, y, criterion, eps=1e-4):
    model.train()
    x_double = x.double()
    y_double = y.double()
    model_double = model.double()

    print("\n=== GRADIENT CHECK ===")
    overall_max_diff = 0
    checked = 0

    for name, param in model_double.named_parameters():
        if not param.requires_grad:
            continue

        layer_max_diff = 0

        model_double.zero_grad()
        output = model_double(x_double)
        loss = criterion(output, y_double)
        loss.backward()
        analytical_grad = param.grad.clone()

        num_checks = min(5, param.numel())
        for i in range(num_checks):
            idx = _flat_to_multi_index(i, param.shape)
            original = param.data[idx].item()

            param.data[idx] = original + eps
            with torch.no_grad():
                loss_plus = criterion(model_double(x_double), y_double).item()

            param.data[idx] = original - eps
            with torch.no_grad():
                loss_minus = criterion(model_double(x_double), y_double).item()

            param.data[idx] = original

```



```

        numerical = (loss_plus - loss_minus) / (2 * eps)
        analytical = analytical_grad[idx].item()

        denom = max(abs(numerical), abs(analytical), 1e-8)
        rel_diff = abs(numerical - analytical) / denom

        layer_max_diff = max(layer_max_diff, rel_diff)
        checked += 1

    overall_max_diff = max(overall_max_diff, layer_max_diff)
    status = "OK" if layer_max_diff < 1e-5 else "MISMATCH"
    print(f" {name}: max_rel_diff={layer_max_diff:.2e} [{status}]")

model.float()

print(f"\n Checked {checked} parameters")
if overall_max_diff < 1e-5:
    print(" PASS: Gradients match (rel_diff < 1e-5)")
elif overall_max_diff < 1e-3:
    print(" WARN: Small differences (1e-5 < rel_diff < 1e-3)")
else:
    print(" FAIL: Gradient mismatch detected (rel_diff > 1e-3)")
return overall_max_diff

```

Step 5: Deliberately Broken Networks

Now apply the toolkit to broken networks and diagnose each one.

```

def demo_broken_networks():
    torch.manual_seed(42)
    x = torch.randn(64, 10)
    y = (x[:, 0] > 0).long()

    print("\n" + "=" * 60)
    print("BUG 1: Learning rate too high (lr=10)")
    print("=" * 60)
    model1 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    debugger1 = NetworkDebugger(model1)
    optimizer1 = torch.optim.SGD(model1.parameters(), lr=10.0)
    criterion = nn.CrossEntropyLoss()
    for step in range(20):
        optimizer1.zero_grad()
        out = model1(x)
        loss = criterion(out, y)
        debugger1.record_loss(loss.item())
        loss.backward()
        optimizer1.step()
    debugger1.print_report()
    debugger1.remove_hooks()

    print("\n" + "=" * 60)

```

```

print("BUG 2: Dead ReLUs from bad initialization")
print("=" * 60)
model2 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 32), nn.ReLU(),
nn.Linear(32, 2))
with torch.no_grad():
    for m in model2.modules():
        if isinstance(m, nn.Linear):
            m.weight.fill_(-1.0)
            m.bias.fill_(-5.0)
debugger2 = NetworkDebugger(model2)
optimizer2 = torch.optim.Adam(model2.parameters(), lr=1e-3)
for step in range(50):
    optimizer2.zero_grad()
    out = model2(x)
    loss = criterion(out, y)
    debugger2.record_loss(loss.item())
    loss.backward()
    optimizer2.step()
debugger2.print_report()
debugger2.remove_hooks()

print("\n" + "=" * 60)
print("BUG 3: Missing zero_grad (gradients accumulate)")
print("=" * 60)
model3 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
debugger3 = NetworkDebugger(model3)
optimizer3 = torch.optim.SGD(model3.parameters(), lr=0.01)
for step in range(50):
    out = model3(x)
    loss = criterion(out, y)
    debugger3.record_loss(loss.item())
    loss.backward()
    optimizer3.step()
debugger3.print_report()
debugger3.remove_hooks()

print("\n" + "=" * 60)
print("HEALTHY NETWORK: Correct setup for comparison")
print("=" * 60)
model_good = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
debugger_good = NetworkDebugger(model_good)
optimizer_good = torch.optim.Adam(model_good.parameters(), lr=1e-3)
for step in range(50):
    optimizer_good.zero_grad()
    out = model_good(x)
    loss = criterion(out, y)
    debugger_good.record_loss(loss.item())
    loss.backward()
    optimizer_good.step()
debugger_good.print_report()
debugger_good.remove_hooks()

```

```

print("\n" + "=" * 60)
print("OVERFIT-ONE-BATCH TEST (healthy model)")
print("=" * 60)
model_test = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
overfit_one_batch(model_test, x[:8], y[:8], criterion)

print("\n" + "=" * 60)
print("LEARNING RATE FINDER")
print("=" * 60)
model_lr = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
find_learning_rate(model_lr, x, y, criterion)

print("\n" + "=" * 60)
print("GRADIENT CHECK")
print("=" * 60)
model_grad = nn.Sequential(nn.Linear(10, 8), nn.ReLU(), nn.Linear(8, 2))
gradient_check(model_grad, x[:4], y[:4], criterion)

```

Use It

PyTorch Built-in Tools

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(768, 256),
    nn.ReLU(),
    nn.Linear(256, 10),
)

with torch.autograd.detect_anomaly():
    output = model(input_tensor)
    loss = criterion(output, target)
    loss.backward()

for name, param in model.named_parameters():
    if param.grad is not None:
        print(f"{name}: grad_mean={param.grad.abs().mean():.2e}")

```

Weights & Biases Integration

```

import wandb

wandb.init(project="debug-training")

for epoch in range(100):
    loss = train_one_epoch()
    wandb.log({
        "loss": loss,

```

```

        "lr": optimizer.param_groups[0]["lr"],
        "grad_norm": torch.nn.utils.clip_grad_norm_(model.parameters(), float("inf")),
    })

    for name, param in model.named_parameters():
        if param.grad is not None:
            wandb.log({f"grad/{name}": wandb.Histogram(param.grad.cpu().numpy())})

```

TensorBoard

```

from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter("runs/debug_experiment")

for epoch in range(100):
    loss = train_one_epoch()
    writer.add_scalar("Loss/train", loss, epoch)

    for name, param in model.named_parameters():
        writer.add_histogram(f"weights/{name}", param, epoch)
        if param.grad is not None:
            writer.add_histogram(f"gradients/{name}", param.grad, epoch)

```

The Debug Checklist (Before Full Training)

- Run overfit-one-batch test. If it fails, stop.
- Print model summary – verify parameter count is reasonable.
- Run a single forward pass with random data – check output shape.
- Train for 5 epochs – verify loss decreases.
- Check activation statistics – no dead layers, no explosions.
- Check gradient flow – no vanishing, no exploding.
- Verify data pipeline – print 5 random samples with labels.

Exercises

Add an exploding gradient detector. Modify the NetworkDebugger to detect when gradients exceed a threshold and automatically suggest a gradient clipping value. Test it on a 20-layer network with no normalization.

Build a dead neuron resurrector. Write a function that identifies dead ReLU neurons (always outputting 0) and reinitializes their incoming weights with Kaiming initialization. Show that this recovers a network where >70% of neurons are dead.

Implement the learning rate finder with plotting. Extend `find_learning_rate` to save results as a CSV and write a separate script that reads the CSV and displays the LR vs loss curve using matplotlib. Identify the optimal LR for ResNet-18 on CIFAR-10.

Create a data pipeline validator. Write a function that checks for: duplicate samples across train/test splits, label distribution imbalance (>10:1 ratio), input normalization (mean near 0, std near 1), and NaN/Inf values in the data. Run it on a deliberately corrupted dataset.

Debug a real failure. Take the mini-framework from Lesson 10, introduce a subtle bug (e.g., transpose the weight matrix in backward), and use gradient checking to locate exactly which parameter has incorrect gradients. Document the debugging process.

Key Terms

Term	What people say	What it actually means
Silent bug	“It runs but gives bad results”	A bug that produces no error but degrades model quality – the dominant failure mode in ML
Dead ReLU	“The neurons died”	A ReLU neuron whose input is always negative, so it outputs 0 and receives 0 gradient permanently
Vanishing gradients	“Early layers stop learning”	Gradients shrink exponentially through layers, making weights in early layers effectively frozen
Exploding gradients	“Loss went to NaN”	Gradients grow exponentially through layers, causing weight updates so large they overflow
Gradient checking	“Verify backprop is correct”	Comparing analytical gradients from backprop to numerical gradients from finite differences
Overfit-one-batch	“The most important debug test”	Training on a single small batch to verify the model CAN learn – if it cannot, something is fundamentally broken
LR finder	“Sweep to find the right learning rate”	Exponentially increasing the learning rate over one epoch and picking the rate just before loss diverges
Data leakage	“Test data leaked into training”	When information from the test set contaminates training, producing artificially high accuracy
Activation statistics	“Monitor layer health”	Tracking mean, std, and zero-fraction of each layer’s output to detect dead, saturated, or exploding neurons
Gradient clipping	“Cap the gradient magnitude”	Scaling gradients down when their norm exceeds a threshold, preventing exploding gradient updates

You are a neural network debugging expert. Given a description of training behavior, diagnose the root cause and prescribe a fix.

Input

I will describe: - The loss curve behavior (flat, oscillating, NaN, decreasing then plateau) - Model architecture (layers, activations, normalization) - Training configuration (optimizer, learning rate, batch size, epochs) - Any activation or gradient statistics available - The dataset (size, type, preprocessing)

Diagnostic Protocol

Step 1: Classify the Symptom

Symptom	Category
Loss not decreasing at all	OPTIMIZATION FAILURE
Loss NaN or Inf	NUMERICAL INSTABILITY
Loss decreasing but model bad	GENERALIZATION FAILURE
Loss oscillating wildly	HYPERPARAMETER PROBLEM
Training works, inference wrong	EVAL MODE BUG

Step 2: Run the Decision Tree

OPTIMIZATION FAILURE: 1. Is the learning rate reasonable? (Adam: $1e-4$ to $1e-2$, SGD: $1e-3$ to $1e-1$) 2. Are gradients flowing? Check gradient magnitude per layer. 3. Are neurons alive? Check fraction of zero activations after ReLU. 4. Does the model pass the overfit-one-batch test? 5. Are parameters actually being updated? Compare weights before/after a step.

NUMERICAL INSTABILITY: 1. Is learning rate too high? Reduce by 10x. 2. Is there a $\log(0)$ or division by zero? Add ϵ . 3. Are activations overflowing in $\exp()$? Use log-sum-exp trick. 4. Is batch norm getting a constant batch? Add ϵ to denominator.

GENERALIZATION FAILURE: 1. Is there a train/test gap? If $>10\%$ accuracy gap, overfitting. 2. Is there data leakage? Check for duplicates across splits. 3. Are labels correct? Manually inspect 20 random samples. 4. Is the test distribution different from training? Check feature distributions.

HYPERPARAMETER PROBLEM: 1. Run the learning rate finder to get the right order of magnitude. 2. Try batch sizes: 32, 64, 128, 256. 3. Try gradient clipping at 1.0.

EVAL MODE BUG: 1. Is `model.eval()` called before inference? 2. Is `torch.no_grad()` used for inference? 3. Are dropout and batch norm behaving correctly?

Step 3: Prescribe the Fix

For each diagnosis, provide: 1. The specific code change needed 2. Expected behavior after the fix 3. How to verify the fix worked

Output Format

```
SYMPTOM: [description]
DIAGNOSIS: [root cause]
EVIDENCE: [what confirms this diagnosis]
FIX: [specific code change]
VERIFICATION: [how to confirm the fix worked]
ALTERNATIVE: [if the fix does not work, try this next]
```

Common Patterns

Architecture	Common bug	Fix
Deep MLP (>5 layers)	Vanishing gradients	Add residual connections or batch norm
CNN	Shape mismatch after pooling	Print shapes after every layer
RNN/LSTM	Exploding gradients	Clip gradients to norm 1.0
Transformer	Attention scores overflow	Scale by $1/\sqrt{d_k}$
Fine-tuning pretrained	Catastrophic forgetting	Use 10-100x smaller LR than pretraining
GAN	Mode collapse	Check discriminator accuracy, adjust training ratio

Always start with the simplest possible diagnosis. The bug is almost always simpler than you think.

Neural Network Debug Checklist

Systematic debugging protocol for when training goes wrong. Work through these in order – most bugs are caught in the first 3 steps.

Before training (prevent bugs)

- Print model architecture and parameter count. Does the size make sense for your data?
- Run a single forward pass with random input. Does the output shape match your target shape?
- Check that labels are the correct dtype (CrossEntropyLoss needs Long, BCELoss needs Float)
- Verify data normalization: inputs should have mean near 0 and std near 1
- Print 5 random (input, label) pairs. Do the labels match what you expect?

- Confirm train/test split has no duplicate samples

Overfit-one-batch test (60 seconds, catches 80% of bugs)

213. Take 8-32 samples from your training set
214. Train for 200 steps with a reasonable learning rate
215. Loss should approach 0. Training accuracy should hit 100%
216. If it fails: the bug is in your model, loss function, or training loop – not your data or hyperparameters
217. If it passes: proceed to full training

Loss not decreasing

- Check learning rate. Try 3 values: `current/10`, `current`, `current*10`
- Print gradient norms per layer. All zeros means dead network or detached graph
- Check `requires_grad=True` on parameters. Check that `loss.backward()` is called
- Check that `optimizer.zero_grad()` is called before `loss.backward()`
- Check that `optimizer.step()` is called after `loss.backward()`
- Verify model parameters are passed to the optimizer: `optimizer = Adam(model.parameters())`

Loss is NaN or Inf

218. Reduce learning rate by 10x
219. Add ϵ to all `log()` calls: `torch.log(x + 1e-7)`
220. Add ϵ to all division: `x / (y + 1e-8)`
221. Clamp predictions: `torch.clamp(pred, 1e-7, 1 - 1e-7)` before BCE loss
222. Use `torch.autograd.detect_anomaly()` to find the exact operation
223. Check for NaN in input data: `assert not torch.isnan(x).any()`

Loss oscillating

- Reduce learning rate by 3-10x
- Increase batch size (reduces gradient noise)
- Add gradient clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`
- Switch from SGD to Adam (adaptive LR per parameter)
- Add learning rate warmup for the first 5-10% of training

Overfitting (train acc high, test acc low)

- 224. Add dropout (start with $p=0.1$, increase to 0.5)
- 225. Add weight decay to optimizer: `Adam(params, weight_decay=1e-4)`
- 226. Reduce model size (fewer layers or narrower layers)
- 227. Add data augmentation
- 228. Use early stopping: stop when validation loss increases for 5+ epochs
- 229. Check for data leakage between train and test sets

Underfitting (both train and test acc low)

- Increase model capacity (more layers, wider layers)
- Train for more epochs
- Increase learning rate (carefully)
- Remove regularization temporarily to verify the model can learn
- Check that your model is expressive enough for the task

Dead ReLU neurons

- Check fraction of zero activations per layer. $>50\%$ is a problem
- Switch to LeakyReLU(0.01) or GELU
- Use Kaiming initialization for weights
- Reduce learning rate (large updates can push neurons into the dead zone)
- Add batch normalization before activation functions

Quick reference: learning rate starting points

Optimizer	Task	Starting LR
Adam	Training from scratch	1e-3
Adam	Fine-tuning pretrained	1e-5
SGD + momentum	Training from scratch	1e-1
SGD + momentum	Fine-tuning pretrained	1e-3
AdamW	Transformer training	3e-4

Quick reference: batch size effects

Batch size	Gradient noise	Memory	Generalization
8-16	High (noisy)	Low	Often better
32-64	Moderate	Moderate	Good default
128-256	Low (smooth)	High	May need warmup
512+	Very low	Very high	Needs LR scaling

When nothing works

- 230. Simplify the model to 1 hidden layer. Does it learn?
- 231. Simplify the data to 100 samples. Does it overfit?
- 232. Replace your loss with MSE. Does it converge?
- 233. Replace your optimizer with SGD(lr=0.01). Does it make progress?
- 234. Replace your data with synthetic data (e.g., $y = x[0] > 0$). Does it learn?
- 235. If none of these work: the bug is in code you are not looking at (data loading, preprocessing, tensor shapes)

Quiz

Pre-lesson check

Why is neural network debugging harder than traditional software debugging?

- a) Neural networks use different programming languages
 - b) Networks can produce wrong outputs without any error messages or crashes – the code runs but the results are silently incorrect (**correct**)
 - c) Neural networks can't be tested
 - d) There are no debugging tools for neural networks
- Explanation: Traditional bugs crash or throw exceptions. Neural network bugs produce a number that's just wrong – a loss that doesn't decrease, accuracy that plateaus, or outputs that are subtly incorrect. No error message tells you what went wrong.*

What is the 'overfit one batch' debugging technique?

- a) Training on the full dataset until it overfits
- b) Training on a single small batch until the loss reaches near-zero, verifying the model can memorize at least a few examples **(correct)**
- c) Using a very large batch size

- d) Overfitting the validation set

Explanation: If your model can't memorize a tiny batch (e.g., 4-8 samples) to near-zero loss, something is fundamentally broken: wrong architecture, broken loss function, or incorrect training loop. This is the first diagnostic to run.

Post-lesson check

Your loss is NaN after a few training steps. What is the most likely cause?

- a) The dataset is too small
- b) Exploding gradients or numerical overflow, often from a learning rate that's too high or missing gradient clipping **(correct)**
- c) The model has too few parameters

- d) The activation function is wrong

Explanation: NaN loss typically comes from numerical overflow: gradients explode, weights grow unbounded, and operations like $\log(0)$ or $\exp(1000)$ produce ∞ /NaN. Lower the learning rate or add gradient clipping.

Your training loss decreases but validation loss stays flat from the start. What does this indicate?

- a) The model is overfitting
- b) The model is learning training data patterns but they don't generalize – likely a data pipeline issue (train/val data mismatch) or severe overfitting **(correct)**
- c) The learning rate is too low

- d) The model needs more layers

Explanation: If validation loss never improves, the model is either memorizing training noise (overfitting), or there's a data pipeline bug where train and validation distributions are fundamentally different.

What should you check first when your loss curve is completely flat (loss doesn't decrease at all)?

- Try a different architecture
- Verify gradients are nonzero, the learning rate is high enough, and the loss function actually depends on the model's predictions (**correct**)
- Add more training data
- Switch to a different optimizer

Explanation: A flat loss means the model isn't updating. Common causes: zero gradients (dead neurons, detached tensors), learning rate too small, or a loss function that doesn't flow gradients to the model (e.g., using `.item()` before `.backward()`).

End of Volume I

This concludes **AI Engineering From Scratch: Volume I — Foundations**. You have now covered the mathematical foundations, classical machine learning fundamentals, and the core principles of deep learning. With 53 lessons across three phases (Phases 1, 2, and 3), you have:

- Built a solid intuition for linear algebra, calculus, probability, and optimization.
- Implemented classical ML algorithms from scratch, including linear and logistic regression, decision trees, SVMs, and KNN.
- Learned crucial skills in model evaluation, feature engineering, and hyperparameter tuning.
- Mastered the inner workings of neural networks by building a full deep learning framework from scratch.
- Gained practical proficiency in PyTorch and JAX.

This foundation is the bedrock upon which all advanced AI models are built. With this knowledge, you are prepared to move on to applied domains such as computer vision, natural language processing, audio, and generative AI in the subsequent volumes.

What's Next?

The journey continues in **Volume II**, which will cover **Computer Vision (Phase 4)**, taking you from pixels to understanding images, video, 3D, and vision-language models.

Keep building. Keep learning. The field of AI is vast, and you've just built the most important tool you need to explore it: a deep, practical understanding of its core principles.